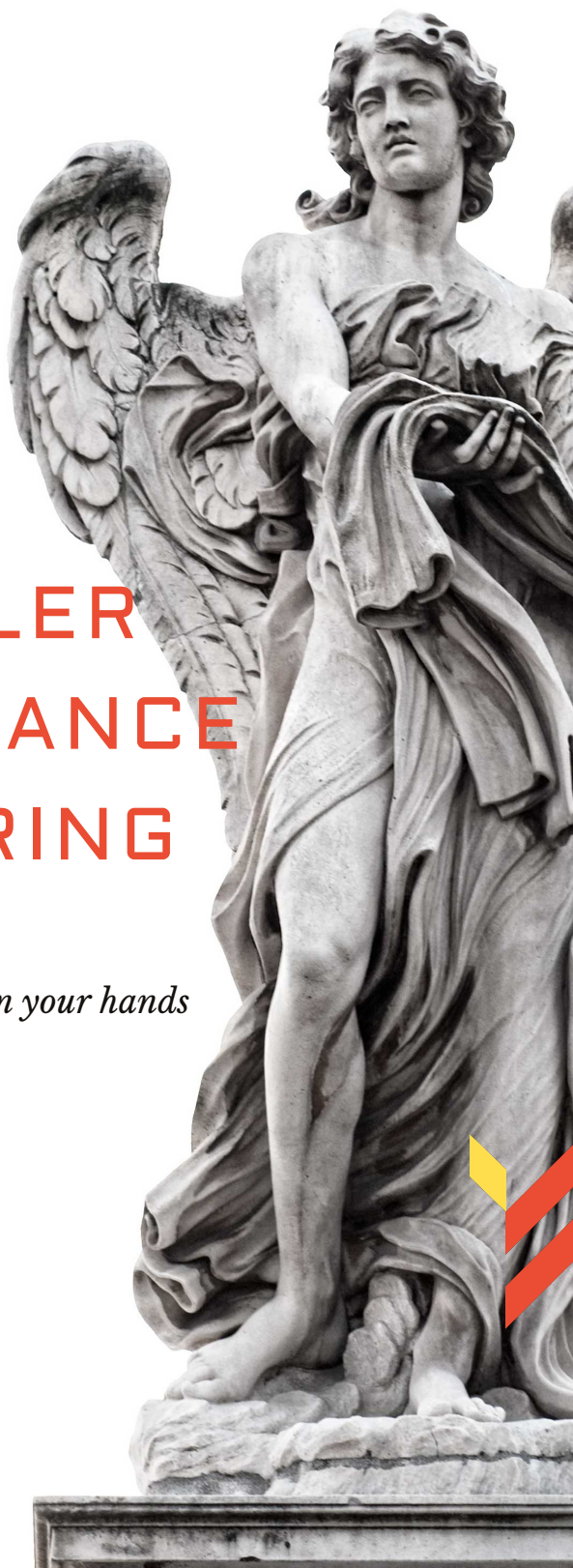




AI COMPILER PERFORMANCE ENGINEERING

————— *It's all in your hands*



Contents

Notation	xiii
1 LLM Inference Performance Bottlenecks: From Symptoms to Root Causes	1
1.1 Why Decode Performance Is the Industry Bottleneck	1
1.2 The AI Compiler Performance Engineer	4
1.3 Prefill and Decode: Two Performance Regimes	5
1.4 Three Core Pain Points of Decode	6
1.5 Why Mainstream Frameworks Do Not Solve Decode Alone	10
1.6 Measuring Decode Goodput	11
1.7 Two-Step Bottleneck Diagnosis	12
1.8 Multi-Hardware Pain Points: GPU, CPU, and NPU	13
1.9 Industry Baselines and What They Still Leave on the Table	14
1.10 The YiRage Answer: Compiler-Scale Fusion (Preview)	16
1.11 Profiling Decode Without Foolish Optimism	16
1.12 Book Roadmap and How to Read	17
1.13 Key Takeaways	19
1.14 Conclusion	20
2 Two Kernel Design Mindsets: Operator-Driven vs Dataflow-Driven	23
2.1 Why Three Months of Kernel Tuning Barely Moved Decode Latency	23
2.2 What You Will Be Able to Do After This Chapter	25
2.3 Terminology Bridge	26

2.4	Operator-First vs Dataflow-First on One Decoder Layer	26
2.5	Data Residency and the Three Iron Rules	28
2.6	Cross-Hardware Dataflow: Same Math, Different Enforcement . . .	30
2.7	From Hand-Crafted MegaKernels to Compiler Automation	32
2.8	Five-Step Migration: Operator Stack to Dataflow Schedule	35
2.9	Proving the Mindset Shift: Verification Without the Checklist Novel	38
2.10	Key Takeaways	41
2.11	Conclusion	42
3	AI Hardware Architecture and Compiler Constraints	44
3.1	Why Lab-Tuned Kernels Fail in Production	44
3.2	AI Hardware Landscape: Three Architectures, Three Optimization Logics	46
3.3	Constraint Matrix: The Compiler’s Hard Ruler	49
3.4	GPU Constraints: Capacity and Orchestration Together	51
3.5	CPU Constraints: Cache, SIMD, and NUMA	54
3.6	NPU Constraints: Static Spatial Plans	56
3.7	Hardware-Aware Compilation: Constraints Become Pass Inputs . .	58
3.8	Common Cross-Hardware Migration Mistakes	59
3.9	Cross-Hardware Benchmarking: Fair Comparison	61
3.10	YiRage Hardware Modeling: One Pass Stack, Many SKUs	63
3.11	Key Takeaways	64
3.12	Conclusion	65
4	CUDA Hopper/Blackwell: Hardware Properties and Optimization Bounds	67
4.1	Why Hopper Features Do Not Automatically Speed Up Decode . .	67
4.2	Hopper Memory Hierarchy: Fusion Hard Boundaries	70
4.3	Tensor Memory Accelerator: Hide Latency, Not Traffic	74
4.4	Thread Block Clusters and Distributed Shared Memory	77
4.5	Synchronization: Narrowest Scope Wins	79
4.6	Tensor Core and PTX: Optimize Last	80
4.7	PTX and Softmax Micro-ops	81

4.8	YiRage CUDA Backend: Hardware Constraints as Compile Gates .	82
4.9	Hopper Decode Tuning in Practice	85
4.10	Hopper Benchmarking and Verification	92
4.11	Key Takeaways	95
4.12	Conclusion	96
5	AMD XDNA/AIE Dataflow Architecture	98
5.1	Why XDNA Is Not a Slower GPU	98
5.2	XDNA Tile Hardware: Fixed SRAM, No Runtime Scheduler	102
5.3	Static Dataflow: Four Rules Decode Must Obey	105
5.4	CUDA → XDNA Mapping for GPU Engineers	107
5.5	Online Softmax on Spatial Hardware	109
5.6	Architecture Tradeoffs and MoE Reality	110
5.7	Dataflow Inference Conclusions	112
5.8	YiRage XDNA Backend: Static Legality as Compile Gates	114
5.9	Code Review Gates (Merge Blockers)	117
5.10	XDNA Decode Benchmarking (Slim)	120
5.11	Key Takeaways	123
5.12	Conclusion	124
6	Data Residency and On-Chip Memory Design	127
6.1	All Decode Optimization Is Residency Optimization	127
6.2	Lifetime Planning: The Four-Step Residency Workflow	130
6.3	Tri-Hardware Residency Adaptation	134
6.4	Cross-Hardware Worked Example: Llama-3.2-1B Layer	139
6.5	YiRage Memory Pass: Residency Rules as Compile Gates	141
6.6	Residency Engineering Patterns	143
6.7	Residency Design Review Workflow	145
6.8	Residency Pitfalls, Checklist, and Debug	146
6.9	Key Takeaways	149
6.10	Conclusion	150

7	Static Role Assignment and Schedule-Free Kernels	154
7.1	The Ultimate Fix for Dispatch Overhead: Static Role Assignment .	154
7.2	Static Role Assignment: Principles and Decode Payoff	157
7.3	Joint Optimization with Chapter 6 Residency	161
7.4	Quantitative Role Decision Framework	162
7.5	Tri-Hardware Role Mapping	166
7.6	Staticization Ladder for Decode	169
7.7	YiRage Tiling Pass: Roles as Compiler Output	172
7.8	Role Assignment Pitfalls and Checklist	175
7.9	Static Role Engineering Patterns	177
7.10	Role Assignment Review Workflow	178
7.11	Key Takeaways	179
7.12	Conclusion	181
8	TMA Double-Buffer Pipelines and Async Movement	186
8.1	The Fused-Kernel Ceiling Is the Pipeline	186
8.2	Double-Buffer Pipelines: Depth Decision Method	189
8.3	GPU TMA Pipeline Design (Decode)	194
8.4	CPU Async Pipeline Design (Decode)	198
8.5	NPU Spatial Pipeline Design (Decode)	200
8.6	Joint Budget with Chapters 6–7	202
8.7	YiRage Pipeline Generation Pass	203
8.8	Pipeline Pitfalls, Checklist, and Debug	206
8.9	Pipeline Engineering Patterns	207
8.10	Pipeline Review Workflow	209
8.11	Decode Pipeline Benchmarking Methodology	210
8.12	Key Takeaways	213
8.13	Conclusion	214
9	Hierarchical Synchronization and Communication	219
9.1	The Lifeline of Fused Kernels: Sync Correctness	219
9.2	Sync Hierarchy: Minimal Scope Principles	222

9.3	Tri-Hardware Sync Primitives and Adaptation	226
9.4	Deadlock Detection and Prevention	231
9.5	Joint Budget with Chapters 6–8	234
9.6	YiRage Sync Pass: Insertion and Proof	236
9.7	Sync Pitfalls, Checklist, and Debug	238
9.8	Sync Engineering Patterns	240
9.9	Sync Review Workflow	242
9.10	Decode Sync Benchmarking Methodology	243
9.11	Megakernel Sync Composition (Chapters 6–9)	244
9.12	Key Takeaways	247
9.13	Conclusion	248
10	Attention Online Softmax Optimization	253
10.1	Online Softmax	254
10.2	Numerical Stability	259
10.3	Cross-Hardware Online Softmax	261
10.4	Softmax Fusion Boundaries	263
10.5	Profiling Online Softmax in Production Stacks	269
10.6	Key Takeaways	272
10.7	Conclusion	275
11	Full Decoder MegaKernel Implementation	277
11.1	MegaKernel Structure	279
11.2	QKV, Attention, and MLP Fusion	284
11.3	MegaKernel Source Walkthrough	288
11.4	Benchmark Results and Interpretation	294
11.5	Key Takeaways	300
11.6	Conclusion	300
12	Three Kernel Execution Modes	302
12.1	eager mode	303
12.2	graph mode	307

12.3	MegaKernel mode	312
12.4	mode selection	315
12.5	Key Takeaways	320
12.6	Conclusion	321
13	Core Theory: AI Compiler Optimization Principles	324
13.1	tiling theory	325
13.2	fusion theory	329
13.3	layout theory	332
13.4	parallel scheduling	335
13.5	GPU CPU NPU split	339
13.6	Key Takeaways	343
13.7	Conclusion	344
14	MLIR: Modern Compiler Infrastructure	346
14.1	MLIR HW matrix	347
14.2	MLIR dialect stack for decode export	349
14.3	MLIR arch passes	350
14.4	MLIR codegen	352
14.5	MLIR tuning	353
14.6	MLIR case study	354
14.7	YiRage MLIR integration notes	357
14.8	MLIR decode review gates	358
14.9	MLIR decode benchmarking methodology	362
14.10	Key Takeaways	368
14.11	Conclusion	369
15	XLA: Production-Grade AI Compiler	370
15.1	XLA HW matrix	370
15.2	XLA arch passes	372
15.3	XLA codegen	376
15.4	XLA tuning	379

15.5 XLA case study	382
15.6 Key Takeaways	386
15.7 Conclusion	386
16 TVM and Apache TVM Stack	388
16.1 TVM HW matrix	388
16.2 TVM arch passes	391
16.3 TVM codegen	393
16.4 TVM tuning	396
16.5 TVM case study	399
16.6 Key Takeaways	403
16.7 Conclusion	404
17 OpenAI Triton: Pythonic GPU Kernel Compiler	405
17.1 Triton HW matrix	405
17.2 Triton arch passes	408
17.3 Triton codegen	412
17.4 Triton tuning	415
17.5 Triton case study	417
17.6 Key Takeaways	420
17.7 Conclusion	421
18 IREE: MLIR-Native Runtime and Compilation	422
18.1 IREE HW matrix	422
18.2 IREE arch passes	425
18.3 IREE codegen	428
18.4 IREE tuning	431
18.5 IREE case study	433
18.6 Key Takeaways	437
18.7 Conclusion	438
19 Glow: Lightweight AI Compiler	440
19.1 Glow HW matrix	440

19.2	Glow arch passes	444
19.3	Glow codegen	447
19.4	Glow tuning	449
19.5	Glow case study	452
19.6	Key Takeaways	455
19.7	Conclusion	456
20	Mirage and Emerging AI Compilers	458
20.1	Mirage HW matrix	458
20.2	Mirage arch passes	462
20.3	Mirage codegen	465
20.4	Mirage tuning	467
20.5	Mirage case study	470
20.6	Key Takeaways	473
20.7	Conclusion	475
21	Unified AI Compiler Analysis and Cross-Hardware Benchmark	476
21.1	Compiler ratings	476
21.2	Compile overhead	479
21.3	Hardware binding	482
21.4	Search space	485
21.5	YiRage benchmark	487
21.6	Key Takeaways	492
21.7	Conclusion	493
22	Compiler-Driven End-to-End Performance Tuning	494
22.1	HW aware profiling	496
22.2	bottleneck locating	501
22.3	tuning workflow	506
22.4	regression gates	510
22.5	Key Takeaways	515
22.6	Conclusion	517

23 LLM and MoE Specialized Compilation	518
23.1 LLM decode compile	520
23.2 moe scheduling	526
23.3 kv hardware	531
23.4 hetero moe	536
23.5 Key Takeaways	540
23.6 Conclusion	542
24 Heterogeneous Compilation and Multi-Hardware Deployment	543
24.1 Heterogeneous cluster	543
24.2 Unified stack	545
24.3 Deployment contract	546
24.4 Runtime adaptation	549
24.5 Ownership and observability	550
24.6 Version regression	552
24.7 Rollout worksheet	553
24.8 Industrial cases	556
24.9 YiRage cluster	557
24.10Key Takeaways	558
24.11Conclusion	559
25 Auto-Optimization and AI-Assisted Compilation	560
25.1 RL autotune	561
25.2 HW reward	565
25.3 YiRage auto	569
25.4 Key Takeaways	572
25.5 Conclusion	573
26 Production Deployment and Engineering Best Practices	574
26.1 env packaging	576
26.2 multi HW ops	578
26.3 pitfalls	579

26.4 scheduling	580
26.5 Key Takeaways	582
26.6 Conclusion	583
27 Future Trends in AI Compilers	584
27.1 codesign	585
27.2 new architectures	587
27.3 edge cloud	588
27.4 autonomous kernels	589
27.5 Key Takeaways	591
27.6 Conclusion	592
28 LLM Inference Frameworks and Serving Runtimes	593
28.1 framework landscape	595
28.2 continuous batching	599
28.3 kv paging serv	603
28.4 framework bottlenecks	606
28.5 compiler integration hooks	609
28.6 Framework decode benchmarking methodology	614
28.7 Key Takeaways	624
28.8 Conclusion	625
29 YiRage Runtime Layer and Persistent Kernel Execution	627
29.1 yirage stack layers	630
29.2 persistent kernel decode	634
29.3 superoptimize handoff	639
29.4 hw registry runtime	643
29.5 deps build native	646
29.6 YiRage runtime decode benchmarking methodology	650
29.7 Key Takeaways	657
29.8 Conclusion	658
30 Framework, Compiler, and Runtime Co-Design for Decode Good-	

put	659
30.1 responsibility split	662
30.2 decode pipeline handoff	667
30.3 vllm yirage stack	670
30.4 mode selection fleet	674
30.5 production regression	677
30.6 Co-design decode benchmarking methodology	680
30.7 Key Takeaways	687
30.8 Conclusion	688
A Reproducible Benchmark Harness	690
B Hopper Decode Verification Checklist	691
B.1 Environment pins	691
B.2 Core goodput gates	691
B.3 Hopper-specific counters	692
B.4 Paged KV and long-context stress	692
B.5 Release train extras	692
Bibliography	693
Index	697

Notation

This section lists symbols and terms used throughout *AI Compiler Performance Engineering*. Figures and tables scale to `\linewidth` via `\autowidthfigure` and the `\autowidthtable` environment.

LLM Inference and Serving

L	Prompt or context length (tokens) in prefill or cached history
m	Number of autoregressive output tokens generated after prefill
BS	Batch size (requests or sequences scheduled together)
SL	Sequence length in a benchmark configuration (prompt + generated tokens)
KV	Key-value cache tensors reused across decode steps
<i>prefill</i>	Parallel forward over prompt tokens; bulk KV write
<i>decode</i>	One (or few) new tokens per forward; KV read and append

Performance Metrics

tok/s	Throughput: tokens processed per second
ms/token	Latency: milliseconds per generated token
I	Arithmetic intensity (FLOPs moved per byte transferred)
HDBI	Host–Device Balance Index; HDBI $\rightarrow 0$ indicates host-bound serving
launches/token	GPU (or device) kernel launches counted per output token
bytes/token	Device memory bytes read/written per output token (weights, KV, activations)

Compiler, GPU, and Hardware

$\mathcal{IR}$	Compiler intermediate representation before lowering
<i>pass</i>	Analysis or transform applied to $\mathcal{IR}$ (fusion, tiling, layout)
<i>tile</i>	Static sub-block of a tensor mapped to on-chip memory
SM	Streaming multiprocessor on NVIDIA GPUs
CTA	Cooperative thread array (CUDA thread block)
TMA	Tensor Memory Accelerator (async bulk copy on Hopper/Blackwell)
HBM	High-bandwidth memory for weights, KV cache, and spill traffic
AIE	AMD AI Engine tile array (XDNA spatial dataflow)

Chapter 1

LLM Inference Performance Bottlenecks: From Symptoms to Root Causes

1.1 Why Decode Performance Is the Industry Bottleneck

In early 2025, DeepSeek-V3 reset what the market thought frontier models should cost to serve: open weights, aggressive quantization, and a production layout that provisions ten times more decode silicon than prefill silicon—320 GPUs in the minimum decode unit versus 32 in the minimum prefill unit (Davies *et al.*, 2025). The business story was not “more FLOPs”; it was *where* bytes and launches go during autoregressive generation, and how fleet topology must follow that asymmetry.

Export controls and regional hardware diversity add another wrinkle: the “best” SKU in a benchmark may be unavailable in your deployment region, so normalized counters matter more than absolute milliseconds when comparing nodes (Davies *et al.*, 2025; Wang, 2025b). Performance engineering that survives export constraints is exactly why this book teaches triplet benchmarking (GPU, CPU, NPU) rather than single-vendor hero numbers.

That pattern repeats outside headline models. A typical production team we see in performance postmortems (composite of several published serving studies, not one named customer) spends calendar quarters enabling FlashAttention-2, adopting vLLM, and upgrading to H100—yet interactive chat still misses p99

time-to-next-token SLOs because each new token still pays hundreds to thousands of kernel launches and full weight+KV traffic (Vellaisamy *et al.*, 2026; Chen, 2026a; Fan, 2025). The failure mode is familiar: prefill dashboards look excellent; decode latency flatlines. Industry effort skews toward prompt throughput benchmarks, while user experience and recurring inference *opex* concentrate in the decode loop once the prompt is ingested (Wang, 2025b; Li, 2024).

DeepSeek-scale lesson (export-aware). DeepSeek-V3’s public inference story combines aggressive weight compression, disaggregated prefill/decode fleets, and hardware layouts tuned to their own network and SKU availability (Davies *et al.*, 2025). Readers in export-controlled regions may not have access to the same GPU generations assumed in US-centric benchmarks. That is why this book stresses **normalized counters** and triplet backends: you can still compare mechanisms when absolute milliseconds or SKU names differ (Wang, 2025b; Chen and YiRage Contributors, 2026). The business lesson remains: decode footprint dominates opex once products scale—not because prompts are short, but because outputs are long and serial (Li, 2024).

Skillful engineering vs. brute-force spend. Fregly’s training-goodput narrative applies unchanged to decode: prefer measurement, fusion, and scheduling discipline over buying another rack of accelerators without changing the software schedule (Davies *et al.*, 2025). A mid-range GPU with fused decode can beat a flagship GPU running an eager loop—Memory-Floor’s L4 ExLlamaV2 versus H100 CUDA Graphs cell is a concrete example (Chen, 2026a). Your CFO cares about that comparison; your worksheet makes it reproducible.

The takeaway at scale: decode performance engineering is a **goodput** problem—useful tokens delivered per dollar and per millisecond under real batching policies—not a peak-TFLOPs procurement exercise (Recasens, 2025). Single-kernel wins (flash attention, paging, graphs) are necessary but do not, by themselves, collapse the operator boundaries that export activations to HBM on every decoder step (Dao *et al.*, 2022; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). This book argues that compiler-scale, hardware-aware fusion is the remaining lever once serving frameworks and attention libraries are already enabled.

Mechanical sympathy for decode. Performance engineering here is not “make kernels faster” in isolation—it is aligning software schedule with how silicon actually moves bytes. NVIDIA GPUs expose HBM (high-bandwidth memory) tiers reached through SIMT warps; CPUs expose cache lines and NUMA domains; XDNA-class

NPU's expose fixed SRAM slots filled by DMA engines. A schedule that ignores those shapes looks correct in PyTorch and still loses to a smaller card with fewer launches (Chen, 2026a; AMD, 2024b; Recasens, 2025). Chapter 1 names the counters; later chapters name the mechanisms (TMA, ADF, MegaKernel) that move those counters.

What this chapter gives you. This chapter is the book's **performance diagnosis foundation**. You will leave with a shared vocabulary—prefill versus decode, four goodput counters, and a two-step worksheet—so you can tell whether a workload is orchestration-limited, bandwidth-limited, or mis-measured before touching tensor-core tile sizes. That diagnostic discipline carries through kernel craft (Parts IV–V), compiler passes (Part VI), fleet tuning (Part VII), and framework/runtime co-design (Part VIII).

Reader prerequisites and 60-second review. We assume you have used transformer inference in PyTorch or Hugging Face, know that **KV cache** stores prior keys/values so each new token does not recompute the full prompt, and have seen a CUDA “kernel launch” at least once. If any of those are new, skim a vLLM architecture note first (Kwon *et al.*, 2023). Three ideas recur in every section: **(i) Prefill** ingests the prompt in parallel; **(ii) decode** appends one token per forward pass in chat; **(iii) operator fragmentation** means each sub-layer (norm, QKV, attention, MLP) may be a separate kernel that reads and writes global memory (GPU HBM or CPU DRAM) between steps.

Operator fragmentation (warehouse analogy). Picture a factory that must complete $\text{norm} \rightarrow \text{QKV} \rightarrow \text{attention} \rightarrow \text{MLP}$ for each token. An eager framework sends the partial product back to the warehouse (HBM) after every station because each station is a different vendor (kernel). Fusion is the engineering decision to keep the assembly line on one bench (registers, shared memory, or cache) until the layer completes (Chen and YiRage Contributors, 2026; Chen, 2020). MegaKernel fusion is not mysticism—it is refusing to ship intermediates to the warehouse when the bench has capacity (Dao *et al.*, 2022).

Why compiler ownership matters now. Frameworks optimize for research agility: new attention variants ship as Python modules (Dao *et al.*, 2022). Production decode rewards the opposite trade—fewer boundaries, static residency, schedules chosen when the graph is known (Chen, 2020; Chen and YiRage Contributors, 2026). The symptoms in this chapter are not PyTorch bugs; they are

consequences of deferring dataflow decisions to a host loop (Vellaisamy *et al.*, 2026; Chen, 2026a). YiRage makes those decisions explicit in IR, verifiable against hardware probes, and regressable before codegen (Chen and YiRage Contributors, 2026).

Reading order for impatient experts. If you already know CUDA: skim §1.3–§1.4, then dive into §1.6 and §1.7. If you are a serving engineer new to kernels: read §9.1 and §1.2, then §1.5 before any roofline math. Everyone should read §1.12 once to see where triplet compilation lands (Chen and YiRage Contributors, 2026).

1.2 The AI Compiler Performance Engineer

This book is written for engineers who sit at the intersection of three disciplines: **inference serving**, **kernel/hardware craft**, and **compiler automation**. You do not need to be all three on day one, but decode optimization fails when any leg is missing. Serving experts without kernel depth propose batching changes that hide but do not remove launch storms. Kernel experts without compiler context ship hand-tuned CUDA that cannot port to the next SKU or to the NPU your product team now requires. Compiler experts without serving empathy optimize graphs that frameworks cannot load or that break sampling and dynamic shapes in production.

Your job in the decode path is to increase **goodput**: useful tokens per second per dollar at the batching policy your product actually ships (Davies *et al.*, 2025; Fan, 2025). That is rarely the same as maximizing TFLOPs or winning a prefill leaderboard. It *is* the same as moving kernels/token, bytes/token, HDBI, and R_{floor} in the right direction on a frozen configuration cell.

How this role shows up in reviews. A credible decode optimization review answers four questions before showing a diff: (1) Which phase—prefill or decode—misses the SLO? (2) Which counter binds on that phase? (3) Which stack layer owns the fix (framework, attention kernel, fusion/compiler)? (4) What open baseline already includes the change? If the review jumps straight to tile sizes, send it back to Chapter 1 (Vellaisamy *et al.*, 2026; Chen, 2026a).

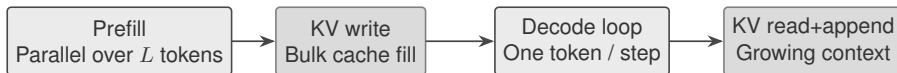


Figure 1.1: Prefill is bulk parallel ingest; decode is a serial per-token loop over growing KV state.

Table 1.1: Prefill vs. decode on one transformer layer (engineering view).

	Prefill	Decode (batch-1 chat)
Work shape	L tokens in parallel	1 new token per step
What limits you	Compute / tensor throughput	HBM/DRAM traffic + host lat
What users feel	Prompt ingestion time	ms/token, streaming smoothn
Fleet metric	Prompt tok/s	p99 latency, cost per 1M toke

1.3 Prefill and Decode: Two Performance Regimes

Think of prefill as *bulk unloading a truck*: many tokens move together, matrix multiplications are wide, and tensor cores can approach peak throughput (Pichlmeier, 2024; SemiAnalysis, 2024). Decode is *hand delivery one package at a time*: each step touches the full parameter set and an ever-growing KV history while doing only a thin slice of arithmetic (Li, 2024; Eto, 2025). Teams that optimize only the truck unload wonder why doorstep latency never improves.

Production fleets increasingly *disaggregate* prefill and decode pools because tuning one phase can harm the other (Qiao, 2026; Chen, 2026b). DeepSeek-V3’s 10:1 decode-to-prefill node ratio is the architectural admission that user-visible cost follows decode steps, not prompt GEMMs alone (Davies *et al.*, 2025).

Industry analyses align. Precision-roofline studies, memory-gap characterizations, and accelerator limit studies independently report that decode at low batch is not compute-starved (Yu, 2026; Recasens, 2025; Davies *et al.*, 2025). The engineering conclusion is consistent even when the hardware generation changes: fix data movement and dispatch, then argue about FLOPs.

Practice note. If your dashboards report only aggregate tok/s, split them tomorrow: one panel for prompt ingestion, one for batch-1 decode on the same model revision.

After countless production postmortems, teams converge on the same conclusion: interactive decode is typically memory-bandwidth and orchestration-limited on

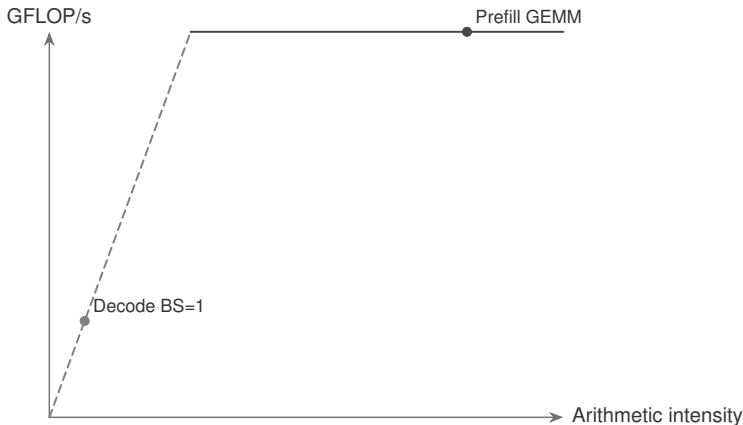


Figure 1.2: Roofline intuition: decode batch-1 sits on the bandwidth slope; prefill wide GEMMs reach the compute cap.

GPUs, CPUs, and many NPUs—not FLOP-limited (Yu, 2026; Recasens, 2025; Davies *et al.*, 2025). Treating prefill and decode as one optimization playbook—bigger batches, FlashAttention defaults, tensor-core-first schedules—is why decode SLOs stay red after prefill tuning succeeds (Tang, 2026; Dao *et al.*, 2023).

1.4 Three Core Pain Points of Decode

Interactive assistants, copilots, and voice interfaces live or die on *time-to-next-token*, usually at batch size one (AMD, 2024a; Fan, 2025). After working through dozens of decode postmortems, we group failures into three recurring pain points—not because the list is exhaustive, but because it drives the diagnostic worksheet in §1.7.

1.4.1 Pain point 1: The memory wall (bytes per token)

At batch-1, each layer performs narrow GEMMs while *reading* every weight byte and every KV byte again. Arithmetic intensity (FLOPs per byte moved) sits on the bandwidth-limited region of the roofline, not the flat compute cap (Yu, 2026; Recasens, 2025; Davies *et al.*, 2025).

Memory-Floor quantifies how close you are to the bandwidth roof with $R_{\text{floor}} = t_{\text{floor}}/t_{\text{obs}}$ where $t_{\text{floor}} = (W + K)/B_{\text{peak}}$ (Chen, 2026a). On Qwen-2.5-7B at ctx=2048, batch-1: L4 reaches $R_{\text{floor}}=0.81$ while H100 reaches 0.27—faster silicon

can be *more* launch-limited, not less.

Engineering read. When R_{floor} is high ($\approx 0.7+$), shaving bytes/token (quantized GEMMs, fusion) moves ms/token. When it is low on a flagship GPU, fix launches and fusion before buying another attention kernel.

Roofline arithmetic for decoder layers. Attainable FLOP/s is $\min(\text{peak FLOP/s, bandwidth AI})$ where arithmetic intensity AI is FLOPs per byte moved. A crude batch-1 model reads N_{params} weight bytes and N_{KV} KV bytes per layer per token, giving

$$\text{AI} \approx \frac{2N_{\text{params}} + 2N_{\text{KV}}}{N_{\text{params}} + N_{\text{KV}}} \quad (1.1)$$

under simplified GEMM accounting (one multiply-add per byte read; activations and writebacks omitted) (Lazuka, 2024; Davies *et al.*, 2025). Context growth raises AI slowly but usually leaves decode on the bandwidth slope for practical LLMs.

1.4.2 Pain point 2: The scheduling tax (kernels per token)

Operator fragmentation is the factory line that hauls every intermediate back to the warehouse between stations: layer norm, QKV, attention, MLP, and residuals become separate kernels, each exporting activations to global memory.

TaxBreak (Vellaisamy *et al.*, 2026) splits host-visible latency per launch into framework translation (ΔFT), CUDA-library front-end (ΔCT), and launch-path floor (ΔKT). Their Host-Device Balance Index (HDBI) compares device-active time to orchestration time; $\text{HDBI} \rightarrow 0$ means the CPU is the boss and the GPU waits.

On H200, BS=1, SL=512, Llama-3.2-1B issues ≈ 844 launches per decode step versus ≈ 85 per prefill step scaled to the same window—an order-of-magnitude dispatch gap (Vellaisamy *et al.*, 2026). At H100’s measured null-kernel floor of $4.707 \mu\text{s}$, a *single-stream* upper bound of 9,305 launches/token (OLMoE) is ≈ 44 ms of launch-path time alone—before framework tax and before any math. Concurrent streams and CUDA Graphs overlap some of that cost, but the ≈ 44 ms figure already crosses the ≈ 50 ms/token interactive SLA line many products publish (Vellaisamy *et al.*, 2026; Chen, 2026a). **Practice implication:** micro-optimizing one GEMM inner loop cannot recover ms/token when dispatch is already at SLA by itself.

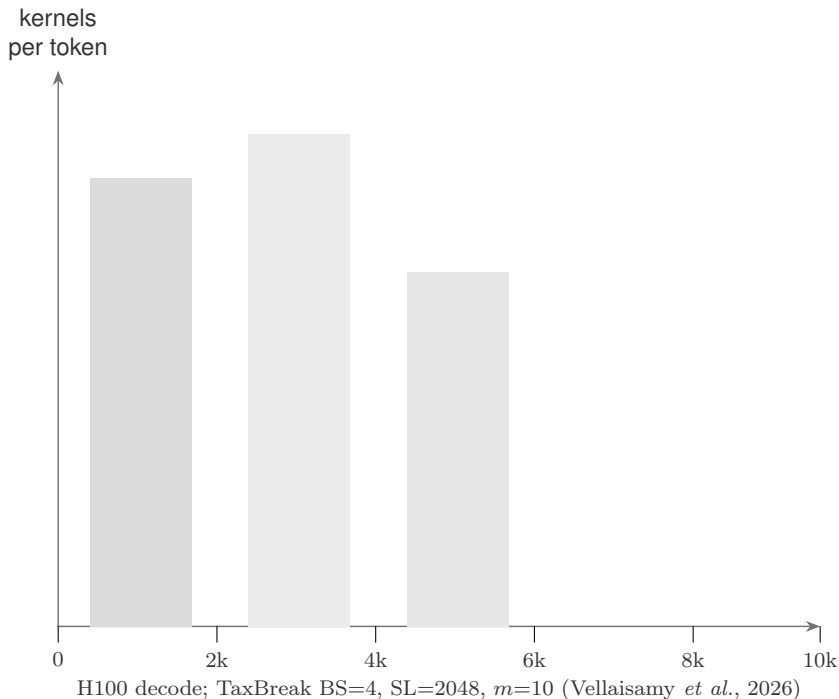


Figure 1.3: Kernel launches per output token: MoE stacks amplify dispatch cost (≈ 9.3 k launches/token for OLMoE).

1.4.3 Pain point 2b: MoE dispatch storms

Mixture-of-experts (MoE) models route each token through a subset of experts, fanning execution into many small GEMMs and auxiliary kernels per step (Vellaisamy *et al.*, 2026; Wang, 2025a). TaxBreak’s OLMoE cell shows ≈ 9.3 k launches per output token and GPU utilization near 15%—the device waits on dispatch, not FLOPs (Vellaisamy *et al.*, 2026). Dense-model intuitions like “raise batch size to feed tensor cores” help less when activated experts per token are few and each expert forward is short. Compiler and runtime strategy should prioritize expert fusion, routing caches, and graph capture across the routing boundary before micro-tuning individual expert matmuls (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Part VII returns to MoE fleet scheduling; Chapter 1’s lesson is that *kernels/token* can be an order of magnitude higher than dense baselines on the same silicon.

1.4.4 Pain point 3: Fragmentation across the stack

On CPUs the warehouse metaphor becomes last-level cache (LLC) pollution: each unfused boundary evicts lines a fused decoder block could keep resident across norm–matmul–activation chains. On NPUs, fragmentation is an architectural fault: if the compiler cannot prove tensor lifetimes fit on-chip SRAM, bring-up fails rather than degrading gracefully to DRAM-heavy eager paths (AMD, 2024b; Chen and YiRage Contributors, 2026).

A dense decoder block often spans 10–20+ discrete kernels in eager PyTorch; MoE multiplies that count (Vellaisamy *et al.*, 2026). PagedAttention and continuous batching (vLLM) fix *fleet* capacity and KV fragmentation; FlashAttention and Flash-Decoding fix *attention IO*—neither removes per-layer boundaries inside the forward pass (Kwon *et al.*, 2023; Dao *et al.*, 2022, 2023). Layer-scale **MegaKernel** fusion—*whole-decoder fusion* in one dataflow-planned kernel group—targets the gaps between those layers (Chen and YiRage Contributors, 2026; Chen, 2020).

Prefill versus decode spend (planning heuristic). Capacity planners often ask what fraction of GPU-hours decode consumes once a product reaches steady state. Published large-model layouts suggest decode can dominate fleet footprint even when prompts are long—the 10:1 DeepSeek decode-to-prefill node ratio is an extreme but directionally common admission that autoregressive steps multiply by output length while prefill runs once per prompt (Davies *et al.*, 2025; Wang, 2025b). If your internal telemetry shows the opposite, verify you are not attributing continuous-batching throughput to per-user decode latency (Kwon *et al.*, 2023; Chen, 2026b).

Context-length stress. At ctx=2048 versus ctx=8192, KV bytes per token grow with history length; Flash-Decoding parallelizes KV dimension work so batch-1 decode can still occupy SMs when sequences are long (Dao *et al.*, 2023). At moderate context, weight traffic and per-layer launch overhead still compete with attention IO depending on model width and hardware class (Chen, 2026a; Vellaisamy *et al.*, 2026). Your worksheet should include at least two context cells; optimizing only ctx=512 ships surprises at production context.

1.5 Why Mainstream Frameworks Do Not Solve Decode Alone

PyTorch plus Hugging Face is logically complete for research: every transformer sub-block is an op. For production decode it is physically naive—the *schedule* is chosen at runtime by a Python loop, not at compile time against memory hierarchy limits. Each boundary materializes tensors to HBM (or DRAM) that a fused schedule could keep in registers, shared memory, or cache.

Eager PyTorch loop (what goes wrong). A naïve `transformers` decode loop—one Python forward per token, eager ops for layer norm, QKV, attention, MLP, residuals—mirrors the math graph but not the memory hierarchy (Vellaisamy *et al.*, 2026). Each op exports intermediates to HBM; the next op imports them again. Ten to twenty boundaries per layer times tens of layers times hundreds of tokens per session is why launches and bytes both explode without fusion (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). Serving frameworks wrap that loop with better batching; they do not automatically rewrite it into a residency-first schedule (Kwon *et al.*, 2023).

Three optimization layers (do not conflate). **Serving framework** (vLLM paging, continuous batching): better fleet utilization; same per-step kernel count inside the model (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). **Attention-kernel IO** (FlashAttention, Flash-Decoding): lower attention bytes; reported $\approx 8\times$ *attention-kernel* speedups at long context, not end-to-end chat latency (Dao *et al.*, 2023). **Compiler fusion** (MegaKernel / YiRage): collapse layer boundaries; the subject of Parts V–VI.

On CPUs, fragmentation becomes LLC misses; on AMD XDNA-class NPUs, dynamic per-op dispatch violates static AI Engine Dataflow (ADF) graphs—the programming model for tile DMA and on-chip buffer residency (AMD, 2024b,a). There is no single “GPU recipe” to photocopy.

SLA vocabulary. Product documents often specify p50/p99 time-to-first-token (TTFT) separate from per-token decode latency. TTFT is dominated by prefill and queueing; streaming smoothness after the first token is dominated by decode (Fan, 2025; Li, 2024). A team that optimizes only TTFT can still lose users to stuttery streams when decode ms/token is high (Kwon *et al.*, 2023). Instrument both phases and both tail metrics; the worksheet in §1.7 applies to decode steps after the prompt is ingested.

Cost model sketch. Let C be GPU-hour cost, λ concurrent decode sessions, and $\bar{t}$ mean ms/token. Roughly, sustained decode throughput scales with $1/\bar{t}$ for memory-bandwidth-bound stacks; lowering bytes/token or kernels/token shifts $\bar{t}$ and directly changes how many paid users fit on a node (Chen, 2026a; Vellaisamy *et al.*, 2026; Wang, 2025b). You do not need a perfect finance model—you need proof that your change moved a counter that executives already believe correlates with cost (Fan, 2025).

Anti-pattern: “we enabled flash, we are done.” FlashAttention-family kernels solve a *subset* of bytes in the attention block (Dao *et al.*, 2022, 2023). They do not fuse MLP and norm boundaries, do not fix MoE routing storms, and do not remove Python dispatch (Vellaisamy *et al.*, 2026). Treat flash as one rung on the ladder in Figure 1.4, then measure what remains (Chen, 2026a; Chen and YiRage Contributors, 2026).

1.6 Measuring Decode Goodput

Workload taxonomy. **Interactive chat** (batch-1 streaming) is the strictest cell: every token pays full KV traffic and users feel tail latency directly (Fan, 2025). **Concurrent assistant** sessions amortize some host work via continuous batching but still run thin per-token forwards inside each active request (Kwon *et al.*, 2023). **Offline batch** jobs can raise batch size and hide launches; RAG pipelines with long contexts often revert to memory-bound decode when KV exceeds on-device residency (Li, 2024). **Edge voice** adds power caps and static-graph requirements on NPUs where Python dispatch may be disallowed (AMD, 2024a,b). When this book says “decode,” we mean the autoregressive forward that appends one token per step unless a table states otherwise (Wang, 2025a).

Fregly’s **goodput** asks how much *useful* work reaches users per dollar and per second (Davies *et al.*, 2025). For interactive decode we make that concrete with four counters—the book’s regression gates in every chapter:

Track all four on the **same configuration cell**: model revision, precision, context length, batch policy, framework and driver versions (Vellaisamy *et al.*, 2026; Chen, 2026a). Product p99 ms/token and kernel-team TFLOPs only reconcile when these counters move together.

Table 1.2: Decode goodput counters: metric, business lever, and what to do first.

Counter	Measures	Business lever	If bad, try first
bytes/token	Weight+KV traffic	DRAM/HBM cost, concurrency	Quantized GEMMs, fusion
kernels/token	Host launches	ms/token, tail latency	Graphs, MegaKernel
HDBI	Host vs device time	CPU load, jitter	<code>torch.compile</code> , fusion
R_{floor}	Bandwidth efficiency	Cap-ex ROI on fast GPUs	Bytes vs launches split

1.7 Two-Step Bottleneck Diagnosis

Reporting discipline. Report decode latency as **median ms/token** at batch-1 unless noted, with warmup discarded and fixed context lengths (512, 2048, 8192) (Chen, 2026a). For fleet traces, separate per-request ms/token from per-scheduler-step ms when continuous batching merges users (Kwon *et al.*, 2023; Chen, 2026b). Publish bootstrap confidence intervals when comparing graph capture or fusion A/B tests (Chen, 2026a). Tail metrics (p95/p99) matter because GC pauses, compilation, and MoE routing variance surface in tails even when median kernels/token looks stable (Vellaisamy *et al.*, 2026; Fan, 2025).

Before you rewrite kernels, run this worksheet (TaxBreak + Memory-Floor) (Vellaisamy *et al.*, 2026; Chen, 2026a):

Step 1 — Orchestration or device? Record kernels/token and HDBI. If kernels/token is in the hundreds+ on a dense model, or HDBI $\rightarrow 0$, suspect launch storms and framework tax.

Cross-checking TaxBreak and Memory-Floor. TaxBreak is host-centric: it tells you whether framework translation, library front-ends, or launch floors dominate orchestration time (Vellaisamy *et al.*, 2026). Memory-Floor is device-centric: it tells you whether observed step time tracks analytic bandwidth floors or is inflated by fixed per-step overhead (Chen, 2026a). Use both: a low HDBI with low R_{floor} on H100 screams launches; a high R_{floor} on L4 screams bytes (Chen, 2026a; Vellaisamy *et al.*, 2026). Neither tool replaces end-to-end ms/token—they explain *why* ms/token resists your last kernel tweak.

Step 2 — Bytes or launches on device? Compute R_{floor} . High R_{floor} ($\approx 0.7+$ on bandwidth-class GPUs): optimize bytes moved. Low R_{floor} on H100-class silicon: optimize launches (CUDA Graphs, fusion) *before* swapping attention implementations—Memory-Floor shows FlashAttention-2 eager can lose to SDPA

Table 1.3: Verified kernel fragmentation on H100 decode (BS=4, SL=2048, $m=10$). Source: TaxBreak (Vellaisamy *et al.*, 2026).

Model	Kernels/token	Total launches	GPU util. (%)
Llama-3.2-1B	847.5	8,475	58.9
Llama-3.2-3B	1,536.9	15,369	67.6
OLMoE-1B/7B	9,305.3	93,053	15.5
Qwen1.5-MoE-A2.7B	6,695.1	66,951	27.7

when attention is not the binder (Chen, 2026a).

CUDA Graphs on Qwen-2.5-7B (H100, ctx=2048, batch-1) remove ≈ 3.05 ms ($\approx 21\%$) of eager median step time; on L4 the same trick yields only $1.028\times$ because the GPU is already bandwidth-saturated ($R_{\text{floor}}=0.81$) (Chen, 2026a; NVIDIA, 2024).

MoE models deserve a separate row in your spreadsheet: GPU utilization in the teens–twenties percent with multi-second-per-token tails is often dispatch, not FLOPs (Vellaisamy *et al.*, 2026; Wang, 2025a).

Tensor cores and TMA (preview). Tensor cores execute MMA instructions on large tiles; decode batch-1 shrinks effective M to one, inflating setup overhead (SemiAnalysis, 2024). Hopper’s TMA (Tensor Memory Accelerator) issues bulk copies from global memory into shared memory with descriptor hardware, reducing scalar address arithmetic in fused kernels (SemiAnalysis, 2024). Neither replaces fusion: they are tools inside a dataflow schedule that already minimized bytes and launches (Chen and YiRage Contributors, 2026). Part III develops legality rules; Chapter 1 warns against TC-first tuning when worksheets say dispatch-bound.

1.8 Multi-Hardware Pain Points: GPU, CPU, and NPU

AMD GPU note. HIP stacks mirror CUDA concerns but differ in wavefront width, LDS (local data share) capacity, and ROCm graph support. Decode on AMD hardware still rewards fusion and launch reduction first; tensor-core analogs only help when GEMM dimensions justify setup (AMD, 2024a).

The pain *shape* is universal; the remedy is not.

GPUs (NVIDIA/AMD). SIMT (single-instruction, multiple-thread) warps, shared memory tiling, and coalesced loads matter; on decode, launch reduction

Table 1.4: First-order decode levers by hardware class.

Class	Optimize first
NVIDIA GPU	Fusion, CUDA Graphs, KV bytes, shared mem tiling
AMD GPU	HIP fusion, LDS reuse, launch reduction
CPU	Cache/NUMA blocking, SIMD, coarse fusion
Edge NPU	Static ADF graphs, DMA pipelines, full-layer fusion

and fusion often beat another tensor-core micro-kernel. Tensor cores execute MMA (matrix multiply-accumulate) ops; at batch-1 the effective M dimension is tiny, so MMA setup can dominate (SemiAnalysis, 2024). Hopper’s TMA (Tensor Memory Accelerator) offloads bulk copy descriptor work but still prefers transfers large enough to amortize setup.

CPUs (x86/ARM). Bandwidth is cache- and NUMA-shaped. SIMD (single-instruction, multiple-data) width and KV layout aligned to cache lines beat warp thinking. On dual-socket servers, the first lever is often KV layout and coarse fusion that keeps QKV+MLP in last-level cache for one token step—not CUDA occupancy heuristics copied verbatim.

Edge NPUs (XDNA/AIE). Static ADF graphs, DMA ping-pong, and compile-time buffer proof dominate; Python dispatch may be disallowed in firmware. LDS (local data share—AMD GPU on-chip scratch, analogous to CUDA shared memory) and NPU SRAM slots are the residency budget compiler passes must prove (AMD, 2024b,a).

1.9 Industry Baselines and What They Still Leave on the Table

Leaderboard prefill tok/s and attention-only speedups answer legitimate research questions, not the batch-1 ms/token chat SLO (Li, 2024; Kwon *et al.*, 2023). Flash-Decoding reports up to $\approx 8\times$ *attention-kernel* gains at long context; vLLM raises fleet goodput with paging—yet MLP, norms, sampling, and thousands of launches per MoE token remain (Dao *et al.*, 2023; Li, 2025; Vellaisamy *et al.*, 2026).

Our open baseline ladder: PyTorch eager $\rightarrow$ SDPA/FlashAttention $\rightarrow$ Flash-Decoding $\rightarrow$ paging/graphs $\rightarrow$ int4 GEMMs $\rightarrow$ MegaKernel/YiRage fusion. Each rung removes one bottleneck; regression reports speedup against the strongest rung

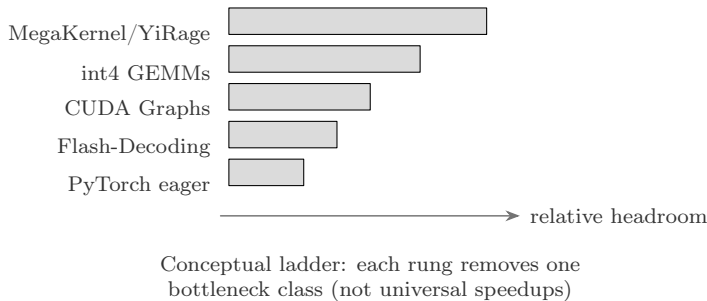


Figure 1.4: Open baseline ladder vs. compiler fusion target (qualitative; measure on your cell).

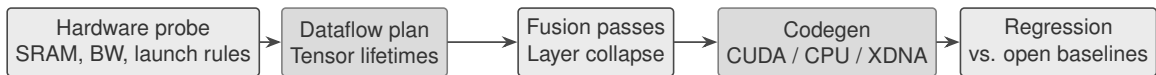


Figure 1.5: YiRage: probe hardware, plan residency, fuse layers, codegen per target, regress on goodput counters.

already enabled on that hardware (Chen, 2026a; Arya, 2024; Chen and YiRage Contributors, 2026).

Decode benchmarking pitfalls (public comparisons). **Prefill-throughput masquerading as decode wins:** tokens/s on long prompts without batch-1 ms/token omits what chat users feel (Li, 2024; Kwon *et al.*, 2023). **Attention-only speedups as end-to-end claims:** Flash-Decoding can accelerate attention dramatically while MLP, norms, sampling, and dispatch remain (Dao *et al.*, 2023; Chen, 2026a). **Mismatched batch policies:** comparing eager batch-1 microbenchmarks to continuous-batching server traces confounds orchestration with scheduling (Vellaisamy *et al.*, 2026; Chen, 2026b). Publish interactive cells alongside throughput headlines (Chen, 2026a; Wang, 2025a).

Benchmark hygiene. Publish at least one interactive cell: batch-1, fixed context, median *and* p99 ms/token, with kernels/token and bytes/token side by side (Chen, 2026a,b).

1.10 The YiRage Answer: Compiler-Scale Fusion (Preview)

YiRage is this book’s reference **hardware-aware** LLM compiler: it ingests decoder graphs, models chip constraints, and emits fused schedules for CUDA, CPU, and XDNA-class targets (Chen and YiRage Contributors, 2026). It does not replace vLLM scheduling or FlashAttention math—it owns *what stays on chip* across layer boundaries.

Symptoms from this chapter become design requirements for later parts: **R1** minimize global round trips; **R2** prefer static schedules (especially NPUs); **R3** match fusion depth to hardware (GPU medium, CPU cache-aware, NPU layer-scale); **R4** amortize launches (graphs, persistent kernels, MegaKernels); **R5** gate every pass on kernels/token and bytes/token regressions (Appendix B).

1.11 Profiling Decode Without Foolish Optimism

Nsight Systems timelines are excellent for proving *who* waits on whom: Python framework threads, CUDA API gaps, and kernel bursts (NVIDIA, 2024). Nsight Compute kernel profiles are excellent for proving *whether* a kernel is bandwidth- or instruction-bound once launches are amortized. Use them in that order on decode cells: if the timeline shows launch storms, Compute profiles of individual GEMMs optimize the wrong bottleneck.

Cross-hardware normalization. When comparing GPU, CPU, and NPU targets, report paired ratios on identical model revisions *and* the four goodput counters (Chen, 2026a; AMD, 2024b; Chen and YiRage Contributors, 2026). A schedule that wins ms/token on H100 but fails XDNA buffer allocation is a bring-up failure, not a portable victory—Part VI encodes those outcomes as compiler errors rather than silent slowdowns (Chen and YiRage Contributors, 2026).

Practice implication. Before your next optimization sprint, freeze a configuration cell in a spreadsheet: model hash, precision, context, batch policy, framework and driver versions, kernels/token, bytes/token, HDBI, R_{floor} , median and p99 ms/token. Re-run the cell after every change; if only one column moves, document which bottleneck class you touched (Vellaisamy *et al.*, 2026; Chen, 2026a).

Compiler preview workflow (five steps). When you reach Part VI, YiRage-style compilation replays the same loop on every target: **(1) Hardware probe** — read bandwidth caps, SRAM sizes, launch rules; **(2) Global dataflow plan** — assign tensor lifetimes to register, shared/LDS, or global tiers; **(3) Fusion passes** — collapse decoder layers; insert async copy pipelines (TMA on Hopper, DMA on AIE); **(4) Codegen** — emit CUDA, CPU, or XDNA schedules with architecture-specific tile search; **(5) Regression** — compare against the strongest enabled open baseline on kernels/token and bytes/token (Chen and YiRage Contributors, 2026; Chen, 2020). Chapter 1 states the symptoms; later chapters show the passes.

What we do not claim. We do not promise a single universal speedup factor. We promise a measurement harness (Appendix B) and legality rules that fail loudly when residency cannot be proved (Chen and YiRage Contributors, 2026; Arya, 2024). That honesty is part of O’Reilly-style engineering writing: name the mechanism, name the counter, show the cell.

From symptoms to requirements (R1–R5 recap). **R1** minimize global round trips; **R2** prefer static schedules where hardware allows; **R3** shape fusion depth per target (GPU medium, CPU cache-aware, NPU layer-scale); **R4** amortize launches via graphs, persistent kernels, or MegaKernels; **R5** require counter movement on a fixed benchmark matrix before merging any pass (Chen and YiRage Contributors, 2026; Chen, 2020). These are not slogans—they are release gates YiRage encodes as hard failures when buffer residency or launch granularity cannot be satisfied (Chen and YiRage Contributors, 2026).

1.12 Book Roadmap and How to Read

Part III (ch04–05) is where mechanical sympathy becomes concrete: Hopper TMA and XDNA ADF are contrasting answers to the same question—how bytes enter on-chip SRAM before MMA or vector ops fire (SemiAnalysis, 2024; AMD, 2024b). Part V (ch10–12) ships the fused decode cell and compares eager launches, CUDA Graph capture, and persistent MegaKernels—the execution modes your framework team will ask about in review (NVIDIA, 2024; Chen and YiRage Contributors, 2026). Part VI (ch13–21) is deliberately one lowering arc across MLIR, XLA, TVM, Triton, IREE, Glow, and Mirage so you can compare backends on identical models rather than reading nine product brochures (Chen, 2020). Part VIII (ch28–30) closes the loop with inference frameworks, YiRage runtime, and three-way co-design

Table 1.5: Eight-part learning path (Fregly-style goodput arc, compiler depth).

	Part	You will be able to
I	ch01–02	Diagnose decode; think dataflow-first
II	ch03	Read hardware constraint matrices
III	ch04–05	Map Hopper CUDA and XDNA legality
IV	ch06–09	Plan on-chip residency and sync
V	ch10–12	Build and ship MegaKernels / graphs
VI	ch13–21	Lower the same IR across MLIR/XLA/TVM/...
VII	ch22–27	Profile fleets, MoE, autotune
VIII	ch28–30	Co-design frameworks, runtime, compiler

Table 1.6: Literature-backed decode anchors (conditions vary; sources in column 3).

Metric	Value	Source / conditions
Decode:prefill nodes	10:1	DeepSeek-V3 (Davies <i>et al.</i> , 2025)
Dense kernels/step	≈ 844	Llama-3.2-1B, H200, BS1/SL512 (Vellaisamy <i>et al.</i> , 2026)
MoE kernels/token	9,305	OLMoE, H100, BS4/SL2048 (Vellaisamy <i>et al.</i> , 2026)
Launch floor ΔKT	$4.707 \mu s$	H100 null kernel (Vellaisamy <i>et al.</i> , 2026)
CUDA Graphs (H100)	$1.259\times$	Qwen-2.5-7B, ctx2048 (Chen, 2026a)
R_{floor} H100 / L4	0.27 / 0.81	Qwen-2.5-7B, ctx2048 (Chen, 2026a)

between framework schedulers, compilers, and serving SLOs (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

Read Parts I–II before jumping to CUDA tile advice. Carry the four goodput counters into every profiling session; if they do not move, the optimization did not matter. Skim Part VIII early if you own a serving stack—framework schedulers and compiler fusion interact at decode boundaries even before you write CUDA (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026). Appendix A cross-links the full checklist; use it before major releases the same way reliability teams use launch runbooks (Davies *et al.*, 2025).

1.13 Key Takeaways

- **Diagnose before you optimize.** Run the two-step worksheet on every new model SKU: HDBI and kernels/token expose orchestration limits; R_{floor} tells you whether bytes or launches bind on the device. If $\text{HDBI} \rightarrow 0$, tensor-core tile tuning is theater—collapse launches or fuse layers first (Vellaisamy *et al.*, 2026; Chen, 2026a). Teams that skip this step often ship quarterly GEMM tweaks while p99 latency flatlines.
- **Memory and dispatch beat peak FLOPs in chat.** Interactive decode sits on the memory-bandwidth roofline at batch-1; tensor-core utilization can be $\leq 1\%$ in limit studies when compute is already provisioned (Davies *et al.*, 2025; Yu, 2026). Practice order: reduce global round trips, then kernel launches, then MMA inner loops. Buying a faster SKU without changing the schedule frequently buys a lower R_{floor} , not a faster chat (Chen, 2026a).
- **Measure goodput, not leaderboard tok/s.** Split prefill and decode dashboards; publish batch-1 median *and* p99 ms/token alongside kernels/token and bytes/token (Kwon *et al.*, 2023; Chen, 2026b; Fan, 2025). Goodput is useful tokens per dollar at your real batching policy—if the four counters do not move, the optimization did not matter.
- **Respect layer boundaries in the stack.** vLLM solves fleet scheduling and KV fragmentation; FlashAttention solves attention IO; MegaKernel/YiRage fusion solves layer residency (Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). Conflating those layers sends teams to tune paging when the binder is launch count, or tune attention when bytes/token is already at the roof.
- **Adapt remedies to the silicon target.** CUDA Graphs may rescue orchestration-limited H100 cells while L4 remains bandwidth-bound ($R_{\text{floor}} \approx 0.81$) and NPUs require static ADF proofs (Chen, 2026a; AMD, 2024b; NVIDIA, 2024). Medium-grain GPU fusion heuristics copied to CPU or NPU frequently *hurt* latency.
- **Prefer skillful engineering over brute-force spending.** Once paging, flash kernels, graphs, and quant backends are enabled, compiler-scale dataflow planning attacks the remaining boundaries with R1–R5 regression gates (Chen and YiRage Contributors, 2026; Chen, 2020; Arya, 2024). That is the skill this book teaches—not another peak-FLOPs datasheet comparison.

1.14 Conclusion

Decode-phase LLM inference is a systems problem disguised as a modeling problem (Nakai, 2025). Industrial teams already bought the GPUs and enabled the famous kernels; the remaining gap is schedule ownership—how many times per token weights and activations cross the memory wall, and how many launches the host injects (Vellaisamy *et al.*, 2026; Chen, 2026a).

When you finish this chapter, you should be able to walk into a performance review with a filled worksheet and a prioritized backlog: if orchestration-limited, schedule graph capture and fusion experiments; if bandwidth-limited on a mid-tier GPU, schedule bytes/token work (quantized GEMMs, KV layout, flash attention where it actually binds); if both counters look healthy but SLOs fail, audit batching policy and measurement cells before touching kernels (Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023). Chapter 2 teaches how to *design* schedules that keep tensors on chip; Chapter 1 teaches how to prove you need that design (Chen, 2020; Goodfellow *et al.*, 2016).

Chapter 2 replaces operator-at-a-time thinking with dataflow-first residency planning—the bridge from these counters to Hopper/XDNA craft and YiRage IR (Chen, 2020; Goodfellow *et al.*, 2016).

Closing the business loop. When you present decode work to leadership, translate counters to money and SLA: lower bytes/token $\Rightarrow$ more concurrent chats per GPU; lower kernels/token $\Rightarrow$ better p99 streaming; higher R_{floor} on bandwidth-class GPUs $\Rightarrow$ quant and fusion investments paying off; healthy HDBI $\Rightarrow$ host fleet rightsizing (Vellaisamy *et al.*, 2026; Chen, 2026a; Fan, 2025). Avoid slides that only show TFLOPs or kernel-local speedups without ms/token at the product batching policy.

Worksheet gate. Before changing a kernel, state your model, batch, context, and whether HDBI and R_{floor} classify the cell as orchestration- or bandwidth-limited. If you cannot, run TaxBreak and Memory-Floor first (Vellaisamy *et al.*, 2026; Chen, 2026a).

Hands-on next step. Open your current decode trace or profiler export and count launches for one token step (or read kernels/token from a prior TaxBreak-style study) (Vellaisamy *et al.*, 2026). If the number exceeds a few hundred on a dense model, your next experiment is graph capture or fusion—not a faster attention kernel (NVIDIA, 2024; Chen and YiRage Contributors, 2026). If bytes/token is at

the bandwidth roof (R_{floor} high), prioritize quantization kernels that actually read narrow weights end-to-end (Chen, 2026a). Write the before/after counters in the spreadsheet template from Appendix F before requesting more hardware budget (Vellaisamy *et al.*, 2026; Chen, 2026a).

Glossary anchors (repeat for memory). **KV cache** stores prior layer keys and values so each new token reuses past context without recomputing the prompt. **Kernel launch** is the host CPU’s submission of one GPU function; thousands per token create synchronization and framework overhead (Vellaisamy *et al.*, 2026). **HBM** is off-chip GPU memory with high bandwidth but high latency compared to registers and shared memory. **SIMT** (single-instruction, multiple-thread) is NVIDIA’s warp execution model; **SIMD** is the CPU vector analogue—both matter when mapping fusion depth (SemiAnalysis, 2024; AMD, 2024b). **LDS** (local data share) is AMD GPU on-chip scratch memory, analogous to CUDA shared memory (AMD, 2024a). **ADF** (AI Engine Dataflow) is AMD’s static graph model for XDNA tiles (AMD, 2024b).

Relationship to Chapter 2. Chapter 2 explains *how* to think in dataflow: which tensors must live on chip, for how many ops, and under which hardware-specific tile constraints (Chen, 2020; Goodfellow *et al.*, 2016). Chapter 1 explains *why* that mindset is non-optional for decode: operator-at-a-time schedules measurably pay launches and HBM round trips you can avoid (Vellaisamy *et al.*, 2026; Chen, 2026a). Read them as a pair—metrics first, design second (Chen and YiRage Contributors, 2026).

Appendix pointers. Appendix B hosts the reproducible benchmark harness; Appendix F hosts blank TaxBreak/Memory-Floor worksheets (Vellaisamy *et al.*, 2026; Chen, 2026a). Use them as living documents per model release—not one-off slides for a single executive review (Fan, 2025).

What success looks like. Success in this chapter is not memorizing roofline algebra—it is refusing to optimize blind. You can explain to a colleague, with counters in hand, whether your decode cell is orchestration-limited or bandwidth-limited, which stack layer owns the next experiment, and which open baseline you must beat (Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023). That clarity is the mechanical sympathy Fregly advocates: software that respects how the machine actually spends time (Davies *et al.*, 2025). If you can fill Appendix F for your deployment today, you are ready for Chapter 2’s dataflow design rules and

for the hardware chapters that follow (Chen, 2020; Chen and YiRage Contributors, 2026). Treat that worksheet as the entry ticket to every later optimization chapter in this book—no counter movement, no merge (Vellaisamy *et al.*, 2026; Chen, 2026a).

Chapter 2

Two Kernel Design Mindsets: Operator-Driven vs Dataflow-Driven

2.1 Why Three Months of Kernel Tuning Barely Moved Decode Latency

Many teams have lived this story: six CUDA engineers spend a quarter pushing per-operator MMA utilization above 90%, each matmul or norm kernel improves 15–20% in isolation, and end-to-end single-token latency drops less than 5%. The problem was never inside the kernels—it was *between* them. Every operator boundary pays another host launch and another round trip through high-bandwidth memory (HBM) on the GPU, or through DRAM and last-level cache (LLC) on the CPU. Chapter 1 named those taxes: kernels/token, bytes/token, and the goodput worksheet (Vellaisamy *et al.*, 2026; Chen, 2026a). This chapter answers the design question that follows: **decode wins come from replanning the schedule, not from squeezing one more FLOP inside an operator-first stack** (Chen, 2020; Chen and YiRage Contributors, 2026).

Composite production case (7B dense decode). A composite of several internal fleet migrations (not one named customer) illustrates the ranking. Operator-first tuning on H100 held single-token latency near 45 ms at batch-1 while per-kernel profiles looked healthy. A three-stage dataflow migration—(A) fuse RMSNorm+QKV+rotary, (B) keep attention as an explicit KV-streaming stage,

(C) fuse SwiGLU MLP+norm+residual—cut launches from ~ 12 to 2 per layer step, reduced measured activation bytes by $\sim 60\%$, and brought latency to ~ 22 ms with $\sim 80\%$ higher concurrent chats per card and $\sim 45\%$ lower inference opex on the same SKU (Chen, 2026a; Vellaisamy *et al.*, 2026). The lesson is not a magic kernel; it is **export topology** (Davies *et al.*, 2025).

Why this chapter exists in the book arc. Chapter 1 taught you to *measure* decode pain: goodput counters, roofline placement, and framework tax (Vellaisamy *et al.*, 2026; Chen, 2026a). Without a design mindset, those measurements become endless arguments about which kernel to tune next. This chapter teaches you to *choose the schedule class* before touching tile sizes: operator-at-a-time versus residency-planned dataflow (Chen, 2020; Chen and YiRage Contributors, 2026). Every later chapter—Hopper craft, MegaKernel hand coding, MLIR passes, YiRage automation—assumes you already believe that exports and launches are first-class design variables (Davies *et al.*, 2025).

Mechanical sympathy, schedule edition. Fregly’s mechanical sympathy for training becomes **schedule sympathy** in decode: align compute order with tier capacity before tuning FLOPs (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). A schedule that ignores silicon shape looks correct in PyTorch and still loses to a smaller card with fewer launches (Chen, 2026a). Schedule sympathy is the cultural complement to the iron rules—rules tell you what forbidden exports look like; sympathy tells you to profile exports first on every context bucket change (Vellaisamy *et al.*, 2026).

Warehouse versus assembly-line analogy. **Operator-first** thinking is a factory where every workstation ships partial product back to the central warehouse (HBM) before the next station starts—because each station is a different vendor (kernel). **Dataflow-first** thinking keeps the assembly line on one bench (registers, shared memory, or cache) until the layer completes (Goodfellow *et al.*, 2016). **Data residency** means keeping the product as close to the worker as the bench allows: registers are “in hand,” shared memory is the workbench, HBM is the distant warehouse (Chen, 2026a).

Anti-patterns we see in reviews. **Graph capture without dataflow.** Teams record CUDA Graphs over eager subgraphs and celebrate launch drops while bytes/token stay flat—graphs fix dispatch, not residency (NVIDIA, 2024; Chen, 2026a). **FlashAttention without replanning.** Attention bytes improve

while norm+QKV and MLP still ping-pong; layer-scope worksheets expose the partial win (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). **Partial MLP fusion only.** Fusing down-proj while gate/up stay eager is a frequent false victory—Stage C must treat SwiGLU as one region (Chen, 2026a). **GPU-only promotion.** A fusion that legalizes on Hopper but fails XDNA should not ship to heterogeneous fleets (AMD, 2024b; Chen and YiRage Contributors, 2026). Each anti-pattern is diagnosable with the counters and diagrams in this chapter before anyone opens an MMA config (Davies *et al.*, 2025).

Skillful engineering over hardware sprawl. Once paging, flash kernels, and graphs are enabled, the remaining lever is compiler-scale dataflow planning with regression gates (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a). A mid-range GPU running fused decode can beat a flagship GPU on an eager loop—Memory-Floor’s L4 versus H100 cells are the business proof (Chen, 2026a). This chapter is the design half of that proof; Chapter 1 was the measurement half (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

2.2 What You Will Be Able to Do After This Chapter

After this chapter you should be able to:

1. Contrast operator-driven and dataflow-driven schedules on one decoder layer and count global export edges per token.
2. Apply the three **iron rules** (registers, on-chip SRAM, global memory) as release gates, with profiler-backed checks.
3. Sketch a five-step migration from an eager operator stack toward fused stages without a big-bang rewrite.
4. Prove a mindset shift with export-edge reduction, bytes/token, and launches/token—not kernel-local timers alone (Vellaisamy *et al.*, 2026; Chen, 2026a).

Who should read which sections. Serving engineers new to kernels: read this hook, goals, terminology bridge, two-mindsets contrast, iron rules, and five-step migration—skip the compiler IR prerequisite on first pass (Davies *et al.*, 2025). CUDA kernel engineers arriving from training: read iron rules and the layer microscope under cross-hardware, then verification—your MMA skills apply *inside*

stages after exports are fixed (Vellaisamy *et al.*, 2026). Compiler and ML engineers: read automation and IR prerequisite sections together with failure modes—they set up Part VI (Chen and YiRage Contributors, 2026; Chen, 2020). Managers: read organizational incentives and verification buckets—they define OKRs and CI gates that make mindset shifts stick (Chen, 2026a).

2.3 Terminology Bridge

These terms are used consistently in Chapters 3–11 and in YiRage pass names (Part VI): **operator**—a named function with its own launch or library entry; **stage**—a fused region with one dispatch and internal on-chip residency; **dataflow**—the directed acyclic graph of tensor lifetimes and movement edges; **residency**—the storage tier holding a value at a program point; **MegaKernel**—a hand-authored stage at or near full-layer scope; **automation path**—compiler passes that derive stages from intermediate representation (IR) instead of hand-written CUDA (Chen and YiRage Contributors, 2026; Chen, 2020).

Abbreviations introduced here. **SSA** (static single assignment) names IR values with unique definitions—fusion legality is checked on SSA edges (Chen, 2020). **LDS** (local data share) is AMD GPU on-chip SRAM analogous to CUDA shared memory. **ADF** (AI Engine dataflow graph) is the compile-time spatial schedule on AMD XDNA-class NPUs. **TMA** (Tensor Memory Accelerator) is Hopper’s async copy engine for staged movement (NVIDIA, 2022). **MMA** (matrix multiply-accumulate) is the tensor-core instruction class inside GEMM kernels.

2.4 Operator-First vs Dataflow-First on One Decoder Layer

Chapter 1 diagnosed symptoms; here we name the competing **mindsets** that produce them. Operator-first design lists LayerNorm, QKV projection, attention, MLP, and residuals as separate tasks, tunes each kernel, and lets the runtime stitch launches (Dao *et al.*, 2023). Dataflow-first design names tensor lifetimes and movement *before* operators, then lets kernels fall out of fused stages (Sun, 2019; Chen and YiRage Contributors, 2026). As Chapter 1 noted, TaxBreak-style studies show MoE decode can multiply launch counts versus dense models—dataflow planning matters more as graphs grow (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024).

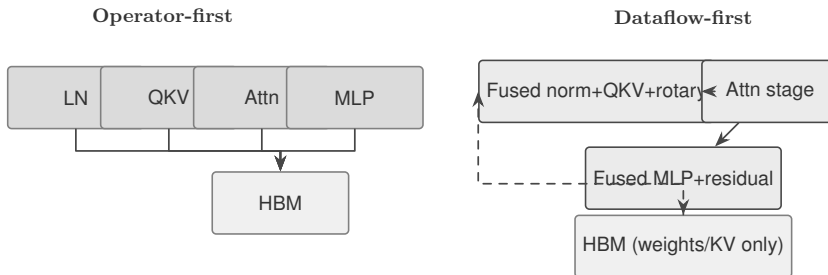


Figure 2.1: Mindset contrast on one decoder layer: operator-first exports every boundary to HBM (left); dataflow-first keeps producer–consumer pairs on-chip between stages (right).

Operator-first stacks fragment accountability: kernel owners optimize “their” op while no team owns HBM edges between ops—yet those edges dominate batch-1 decode (Davies *et al.*, 2025). PyTorch eager mode is the canonical embodiment: correct math, emergent schedule (Kwon *et al.*, 2023). Swapping in FlashAttention fixes attention IO inside one node; it does not replan exports between norm, QKV, and MLP unless a compiler merges regions (Dao *et al.*, 2022). CUDA Graphs amortize launches on graphs that still contain operator-first ping-pong inside recorded nodes—orchestration relief, not residency repair (Chapter 1) (NVIDIA, 2024).

Dataflow-first commitments recur across targets:

1. **Static placement** where possible—lifetimes planned before codegen; NPUs require static ADF graphs (AMD, 2024b).
2. **On-chip residency first**—fusion depth ends at capacity conflicts, not at the op list.
3. **Pipelined movement**—TMA on Hopper and DMA on AI Engines hide transfers when prefetch edges exist (NVIDIA, 2022; AMD, 2024a).
4. **Minimal synchronization**—the dispatch unit is the pipeline stage, not each elementary op.

Operator-first habits that persist in mature stacks. Even after teams adopt vLLM, FlashAttention, and CUDA Graphs, the *mental model* often remains operator-first: replace one op, capture one subgraph, celebrate one kernel win (Kwon *et al.*, 2023; Dao *et al.*, 2022; NVIDIA, 2024). Framework plugin boundaries reinforce silos—paging plugins, quant plugins, and attention plugins each

optimize locally while the layer-level export graph stays emergent (Davies *et al.*, 2025). Compiler ownership inverts the incentives: one schedule owner, many op specializations *inside* fused regions (Chen and YiRage Contributors, 2026; Chen, 2020). Kernel engineers remain essential—they implement epilogues and attention tiles—but success metrics attach to layer-scope counters, not per-op occupancy screenshots alone (Vellaisamy *et al.*, 2026).

Static dataflow versus dynamic serving. Static dataflow does not mean frozen models: it means shapes and buffer slots are known at compile boundaries for a deployment SKU, with guarded recompilation when service configs change (Chen and YiRage Contributors, 2026). Serving systems that mix dynamic batching with static fusion must separate *scheduling* (which requests share a batch) from *kernel* concerns (what stays on-chip during a fused step) (Kwon *et al.*, 2023). Confusing the two produces graphs that are dynamically batched at the framework level but operator-fragmented inside each batch member—the worst of both worlds for tail latency (Davies *et al.*, 2025; Chen, 2026a).

Example 2.4.1. On H100 at batch-1, a dense 7B layer may spend $\sim 15\text{--}25\ \mu\text{s}$ in device compute per operator group but $\sim 30\text{--}50\ \mu\text{s}$ in launch and framework gaps between groups. A 20% compute win per group saves only $\sim 3\text{--}5\ \mu\text{s}$ aggregate while gaps remain. Fusing three groups into one dispatch can remove two launch gaps ($\sim 60\text{--}100\ \mu\text{s}$ class on launch-bound configs) even if per-op compute is unchanged—the *ranking* matters more than the exact microsecond table (Vellaisamy *et al.*, 2026).

2.5 Data Residency and the Three Iron Rules

The root cause is not insufficient FLOPs; it is **data residency** violated at every layer boundary. Activations that could live in registers or shared memory across QKV→attention→MLP are materialized to global memory after each sub-step (Chen, 2026a). Each round trip pays full HBM bandwidth and pollutes CPU caches when the same pattern runs on x86 or Arm hosts.

Iron Rule 1 — Registers hold ephemeral scalars and narrow vectors, not whole feature maps; spills mean the split is wrong. **Iron Rule 2 — Shared memory / LDS** holds fusion regions; depth ends at capacity conflicts, not when the op list ends. **Iron Rule 3 — Global memory** is for weights, KV streams, and true outputs—not ping-pong activations between adjacent ops (Chen and YiRage Contributors, 2026). Violations reproduce Chapter 1 pathologies: high kernels/token and low R_{floor} when export/import dominates (Chen, 2026a).

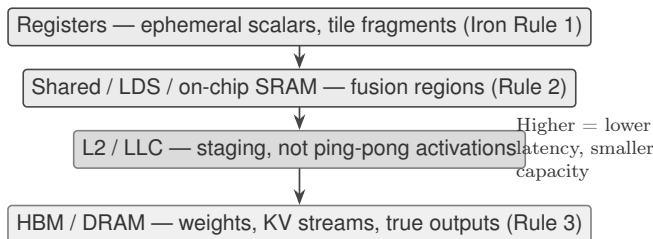


Figure 2.2: Memory residency pyramid for decode: iron rules assign each tier a job; spills signal a wrong split (Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

How to audit iron rules in practice. Use **Nsight Compute** global load/store counters per kernel to see whether mathematically adjacent ops still write full activation tensors to HBM (NVIDIA, 2022). Use **Nsight Systems** timelines to separate launch gaps from in-kernel time (Vellaisamy *et al.*, 2026). On CPU paths, LLC miss profiles play the same role: fused MLP epilogues should keep head layouts hot across QKV and MLP (Davies *et al.*, 2025). Reject schedules that insert a global round trip between producer–consumer SSA edges unless profiling documents unavoidable tier overflow—then shrink fusion depth or retile, do not accept ping-pong as normal (Chen, 2020).

Register pressure versus fusion depth. Register pressure on GPUs frequently triggers the false comfort of “spilling is fine because the kernel still runs.” In decode, spilled activations often become HBM traffic that negates fusion (Chen, 2026a). Iron Rule 1 therefore pairs with occupancy discipline: if widening tile sizes forces spills, shrink fusion depth instead of celebrating higher theoretical occupancy on paper (Chen and YiRage Contributors, 2026). CPU analogs apply to stack spills and false sharing when fusing across threads without cache-line-aware partitioning (Davies *et al.*, 2025). The iron rules are not moral slogans—they are binary checks you can automate in compiler legality passes and manual design reviews (Chen, 2020).

Residency planning as a compile-time problem. Residency is a compile-time allocation problem: which tensors coexist, which tier holds them, for how many fused stages (Chen and YiRage Contributors, 2026). On GPUs the tiers are registers $\rightarrow$ shared memory $\rightarrow$ L2 $\rightarrow$ HBM; on CPUs cache and NUMA; on XDNA hardware-enforced tile buffers (AMD, 2024b; NVIDIA, 2022). When residency succeeds, arithmetic intensity rises because bytes moved drop even if FLOPs stay constant (Chen, 2026a; Goodfellow *et al.*, 2016). Industrial teams should measure

bytes per token and *unique HBM touch count* per layer alongside FLOPs—metrics that residency planning directly minimizes (Vellaisamy *et al.*, 2026).

PagedAttention improves KV *capacity* but does not guarantee on-chip residency inside a fused decoder region (Kwon *et al.*, 2023). Quantization changes residency economics only when kernels consume compressed layouts without dequant scratch in HBM—nf4 weights with bf16 activations still ping-pong if every op dequantizes to global buffers (Liu *et al.*, 2024).

Anti-locality in serving features. Continuous batching improves throughput by reusing weight matrices across requests, but if each request’s activations still round-trip through HBM inside the layer, tail latency per request remains decode-bound (Kwon *et al.*, 2023). Speculative decoding adds draft-model traffic—another residency consumer—that operator-first stacks often bolt on without revisiting fusion boundaries (Davies *et al.*, 2025). Dataflow planners must account for multi-model residency simultaneously: base weights, draft weights, and KV streams competing for bandwidth and on-chip space. Later compiler passes treat these as buffer interference constraints, not afterthought feature flags (Chen and YiRage Contributors, 2026).

Dataflow testing discipline. Operator-first testing validates per-op numerical parity against reference implementations. Dataflow-first testing adds **schedule parity**: fused pipelines must match unfused references within tolerance while preserving tier assignments across backends (Chen, 2020). Regression suites include golden activation dumps at stage boundaries during development, then strip those dumps in production builds once parity is proven. The extra testing cost pays back when a single fusion bug ships as a $1.5\times$ silent slowdown on only one hardware class (Chen and YiRage Contributors, 2026).

2.6 Cross-Hardware Dataflow: Same Math, Different Enforcement

Cross-hardware comparison is how teams avoid shipping GPU-tuned schedules that fail on edge NPUs or underperform on co-located CPU nodes (AMD, 2024b,a). The same mathematical layer admits *different optimal fusion depths*: GPUs tolerate medium fusion with shared-memory staging; CPUs need cache-aware blocking; NPUs require layer-scale or multi-layer static regions (Davies *et al.*, 2025). Portability is portability of the **dataflow graph and lifetime annotations**, not of

Table 2.1: Design mindset vs. hardware enforcement model.

Target	Data residency model
CUDA GPU	Software-managed hierarchy (shared mem, registers)
XDNA / AIE	Hardware-enforced spatial buffers (ADF)
CPU	Cache hierarchy + explicit blocking

individual op implementations—YiRage backends swap codegen while preserving tier assignments (Parts IV–VI) (Chen and YiRage Contributors, 2026; Chen, 2020).

For **CUDA GPUs**, shared memory sizing, register pressure, and occupancy cap fusion depth; Hopper TMA raises async movement ceilings but needs static copy edges (NVIDIA, 2022; SemiAnalysis, 2024). For **CPUs**, GPU-style mega-fusion can evict KV working sets from LLC—blocking beats monolithic kernels (Davies *et al.*, 2025). For **XDNA / AIE**, dynamic shapes and per-op dispatch are mismatches; the ADF graph *is* the program (AMD, 2024b). Operator-first lists fail on every class for the same reason: no global residency contract.

GPU fusion depth in practice. On Hopper-class GPUs, medium fusion—two or three stages per layer—often beats monolithic single-kernel attempts because attention KV streaming and register pressure force explicit boundaries (Dao *et al.*, 2022; NVIDIA, 2022). TMA copy edges must be planned in the dataflow graph so weight tiles prefetch while the previous stage still consumes activations (SemiAnalysis, 2024). Teams that copy CUDA fusion heuristics from training kernels frequently pick tile sizes that maximize MMA utilization while spilling activations to HBM—Iron Rule 1 violations dressed as “high occupancy” wins (Chen, 2026a).

CPU and NPU consequences. On CPU inference nodes co-located with GPU fleets, the winning schedule often fuses MLP epilogues and keeps KV hot in LLC while streaming weights once per layer (Davies *et al.*, 2025). Blindly porting GPU mega-fusion depth can evict KV working sets and raise tail latency for chat workloads pinned to CPU fallback tiers. On XDNA, illegal graphs should fail compile during CI, not at the edge appliance in the field (AMD, 2024b,a). The book’s triplet harness (Appendix B) reports at least three SKUs when hardware access allows so mindset differences appear as *different optimal fusion depths*, not as different models (Chen and YiRage Contributors, 2026).

Mindset comparison on one layer (microscope). Consider batch-1 as the running microscope. **Operator-first execution** issues separate kernels for RMSNorm, QKV, rotary embedding, attention, output projection, second RMSNorm, gate/up/down MLP, activation, and residuals—each reading inputs from HBM and writing outputs back. Even optimized kernels may execute $\sim 10\text{--}20$ global export/import cycles per token per layer, multiplied by host launches (Vellaisamy *et al.*, 2026). Profiler heatmaps look healthy inside each kernel; end-to-end latency stalls.

Dataflow-first execution partitions the same mathematics into two or three pipeline stages with explicit buffer assignments. Stage A fuses RMSNorm+QKV+rotary into shared-memory tiles; Stage B runs attention with KV streaming while keeping query tensors resident where budgets allow; Stage C fuses MLP+norm+residual without global activation ping-pong. Launches drop from $\sim 10\text{--}20$ toward 1–3; bytes moved drop because producer–consumer pairs never left the fastest tier (Chen, 2026a). The mathematical layer is fixed; the **schedule topology** is the optimization variable (Chen, 2020).

2.7 From Hand-Crafted MegaKernels to Compiler Automation

A **MegaKernel** dispatches an entire decoder layer with activations resident across QKV, attention, MLP, norms, and residuals—the antithesis of Figure 2.1’s left panel (Chen, 2026a). MegaKernels are fusion *schedules* with staged pipelines, not one giant matmul. On dense 7B-class models at batch-1 on H100, a well-staged layer MegaKernel often lands in the $\sim 1.5\times\text{--}2.5\times$ end-to-end latency improvement band versus a mature operator-first stack *before* heroic MMA tuning—exact factors depend on model width and attention pattern (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Part V and Part VI preview. Part V develops manual MegaKernel craft on Hopper and XDNA: how experts stage pipelines when compilers are not yet in the loop (NVIDIA, 2022; AMD, 2024b). Part VI shows the same decisions derived automatically from YiRage IR: fusion passes, lifetime analysis, and backend lowering with regression gates (Chen and YiRage Contributors, 2026; Chen, 2020). Readers should treat this chapter as the **why** for those parts—without mindset alignment, hand-tuned kernels and compiler output fight each other across teams (Davies *et al.*, 2025).

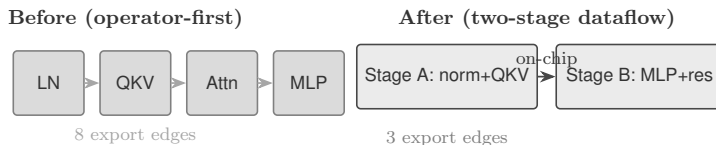


Figure 2.3: Export-edge audit on one layer: operator-first schedules materialize many HBM round trips; staged dataflow removes edges between fused-able ops (Chen, 2026a).

MoE and wide-model caveats. Mixture-of-experts decoders multiply operator boundaries per layer; routing and expert dispatch can dominate launches even when MLP experts fuse (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). Dataflow planning must count routing tax separately from dense MLP fusion—a schedule that fuses experts but leaves routing eager may show flat bytes/token (Chen, 2026a). The mindset still applies: exports and launches are the design variables; expert GEMMs are epilogues inside stages once routing topology is explicit (Chen and YiRage Contributors, 2026).

Chaining vendor libraries without shared residency drops launches modestly but often not bytes/token. True MegaKernels collapse both; partial fusion that only wraps MLP while QKV stays eager is a common false win—launch counters improve while bytes/token flatlines (Vellaisamy *et al.*, 2026). Attention stages that require global KV streaming may remain explicit while MLP+norm+residual fuse around them—dataflow mandates explicit staging with minimal exports, not mandatory single-kernel binaries (Dao *et al.*, 2022, 2023).

YiRage ingests decoder graphs, plans lifetimes globally, runs fusion passes, and emits CUDA, CPU, or XDNA backends from one dataflow IR (Chen and YiRage Contributors, 2026; Chen, 2020). The automation path encodes expert workflow—probe hardware, plan residency, fuse, codegen, regress—as compiler logic. Each backend re-instantiates the same dataflow with different tier rules; compile fails when residency cannot be proved—preferable to silent production slowdowns (Chen and YiRage Contributors, 2026).

Prerequisite: compiler IR (skim if new to compilers). At the IR layer, operator-first pipelines lower frameworks into independent op lowers with late local peephholes. Dataflow-first pipelines retain a **lifetime graph**: SSA values carry storage-class annotations (`reg`, `shared`, `global_stream`) and fusion legality is checked on inter-value edges (Chen, 2020). If you are new to compiler IR, read the commitments above and return here after Part VI—the rest of this chapter does not require SSA details.

YiRage pass pipeline (conceptual). YiRage-style automation typically sequences: import high-level graph $\rightarrow$ lifetime analysis and tier assignment $\rightarrow$ fusion legality on SSA edges $\rightarrow$ backend-specific tiling and codegen $\rightarrow$ regression against eager baselines (Chen and YiRage Contributors, 2026; Chen, 2020). Passes emit logs explaining why edges remain global—for example, attention KV streaming or shared-memory overflow—so engineers learn schedule constraints instead of fighting opaque binaries (Davies *et al.*, 2025). Compile failure on illegal residency is a feature: it prevents shipping silent $1.5\times$ slowdowns on edge SKUs (AMD, 2024b). Manual MegaKernel authors follow the same sequence by hand; automation scales the discipline to every layer and backend (Chen, 2026a; Vellaisamy *et al.*, 2026).

Failure modes when mixing mindsets. Hybrid stacks—dataflow planning in a planner service but operator-first codegen in backends—produce **planner drift** (planned fusion $\neq$ emitted kernels), **false residency** (declared fused buffers that spill registers into HBM), and **NPU illegalize late** (GPU-legal graphs fail ADF allocation in edge builds) (Chen and YiRage Contributors, 2026). Guard with single-source dataflow IR and fail-fast legality per backend.

Historical note (why operator-first won temporarily). 2010s CUDA culture optimized large-batch training where operator coverage mattered more than decode residency (Goodfellow *et al.*, 2016). Inference at batch-1 was treated as “training with batch 1” long after production proved otherwise. FlashAttention, PagedAttention, and graph capture each fixed a slice without overturning the op-list mental model (Dao *et al.*, 2022; Kwon *et al.*, 2023; NVIDIA, 2024). Dataflow-first design returns because deployment reality—MoE launch inflation, disaggregated decode fleets, edge NPUs—punishes export edges more than it rewards isolated MMA wins (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024).

Where not to fuse. Attention kernels that require global KV streaming may remain distinct stages while MLP+norm+residual fuse around them (Dao *et al.*, 2022, 2023). Flash-Decoding parallelizes the KV dimension while keeping decode batch-1 viable; a MegaKernel schedule must *compose* with such attention stages rather than naively inlining them and destroying register budgets (Dao *et al.*, 2023). Rotary embedding between QKV and attention is a frequent hidden export in operator-first stacks—Stage A benchmarks should include rotary inside the fusion region, not as an orphan op (Davies *et al.*, 2025). SwiGLU MLP chains triple export events (gate, up, down); Stage C should treat them as one residency region (Vellaisamy *et al.*, 2026).

Automation artifacts engineers should demand. Compiler automation must preserve human-debuggable outputs: lifetime graphs, fusion decision logs, and regression tables comparing bytes/token and kernels/token against PyTorch eager and framework servers (Chen and YiRage Contributors, 2026). Without those artifacts, automation reintroduces opacity—teams cannot tell whether a regression is a bad tile size or a violated iron rule (Chen, 2020). Treat diagnostics as first-class deliverables alongside generated kernels, echoing TaxBreak and memory-floor methodologies from Chapter 1 (Vellaisamy *et al.*, 2026; Chen, 2026a).

2.8 Five-Step Migration: Operator Stack to Dataflow Schedule

Teams should not rewrite every kernel day one. Use this **five-step playbook** on each decoder layer group:

1. **Export-edge audit.** Draw the operator graph (Figure 2.1) and count HBM read/write edges per token. Use Nsight Compute global traffic or framework profiler exports; each edge between fused-able ops is a candidate fusion edge (Chen, 2026a).
2. **Launch audit.** Count kernels per token with TaxBreak or equivalent; if $\gg 100$ on dense decode, graph capture or fusion passes beat MMA tile tuning (Vellaisamy *et al.*, 2026).
3. **Tier budget.** Estimate shared-memory or cache footprint for candidate fusion; split stages when capacity exceeds hardware limits rather than spilling silently (Chen and YiRage Contributors, 2026).
4. **Phased fusion.** Stage A: norm+QKV+rotary; Stage B: attention boundary (KV global unless proven safe to inline); Stage C: MLP+norm+residual. Benchmark bytes/token after *each* tranche—partial MLP fusion without QKV is a classic false win (Chen, 2026a; Vellaisamy *et al.*, 2026).
5. **Triplet regression.** Re-run bytes/token, launches/token, and ms/token on GPU, CPU, and NPU SKUs after each tranche—not only the device that authored the kernel (AMD, 2024b; Chen and YiRage Contributors, 2026).

Stage sizing worked example. Suppose Stage A (norm+QKV+rotary) requires 48 KB shared memory for tile shapes at hidden size 4096, but your SKU exposes

40 KB per SM without splitting (NVIDIA, 2022). Iron Rule 2 says split Stage A into norm+QKV and rotary+attention prefetch rather than spilling to HBM (Chen and YiRage Contributors, 2026). Document the split in the export-edge table: you may add one launch while still removing multiple global activations—launches and bytes both belong on the worksheet (Vellaisamy *et al.*, 2026; Chen, 2026a). Promotion reviews should accept launch increases only when accompanied by byte wins and iron-rule justification (Davies *et al.*, 2025).

Documentation contract for migrations. Publish operator graph, dataflow stage diagram, export-edge count, bytes/token, and launches/token at ctx=2048 with every fusion promotion (Vellaisamy *et al.*, 2026; Chen, 2026a). Partners and future-you cannot replay schedule regressions when benchmarks live only in private spreadsheets (Chen and YiRage Contributors, 2026). Version those artifacts next to compiler fingerprints in CI so bisect compares topology, not only binary hashes (Chen, 2020).

Onboarding copy-paste block. New hires should paste this sentence at the top of fusion design docs: “*We optimize exports and launches across the decoder layer before we tune MMA tiles inside ops.*” (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025) Then attach Figure 2.1 and the export-edge table before the first kernel meeting (Chen, 2026a). Culture follows the diagram on the wall and the counters in CI—not the slogan alone (Chen and YiRage Contributors, 2026).

Bridge back to Chapter 1. When Chapter 1’s worksheet classifies your cell as orchestration-limited, start with launch audits and staged fusion (Vellaisamy *et al.*, 2026; NVIDIA, 2024). When it classifies bandwidth-limited, prioritize bytes/token and quantization that actually reads narrow weights end-to-end—but still remove avoidable exports; bandwidth and orchestration problems often co-exist (Chen, 2026a; Liu *et al.*, 2024). This chapter tells you *how to design* once Chapter 1 tells you *what hurts* (Davies *et al.*, 2025). Carry both chapters’ worksheets into every fusion review; neither alone is sufficient for production promotion (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026). If you change only one habit after reading: draw the export graph before opening the kernel profiler (Davies *et al.*, 2025). That single discipline prevents most expensive operator-first detours in decode fleets (Vellaisamy *et al.*, 2026; Chen, 2026a). Schedule sympathy starts on paper with export counts per token step, not in Nsight alone (Davies *et al.*, 2025; NVIDIA, 2022; Vellaisamy *et al.*, 2026).

Organizational incentives. Operator-first stacks persist because reviews reward visible kernel wins while dataflow wins look “distributed” across fewer exports (Davies *et al.*, 2025). Staffing mirrors the stack: siloed kernel engineers without a compiler owner cannot hold global residency accountability. Align OKRs with bytes/token and kernels/token tied to product p99 latency, not single-kernel TFLOPS counters.

Framework versus compiler ownership. Framework teams own batching, KV paging, and request scheduling; compiler teams own fusion boundaries inside the decode step box defined in Chapter 1 (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026). Conflating those layers sends organizations to tune paging when launch count is the binder, or tune attention when bytes/token is already at the roof (Chen, 2026a). Dataflow migration playbooks should name a **schedule owner** with authority across norm, attention, and MLP boundaries—the role is closer to a pipeline architect than a matmul specialist (Davies *et al.*, 2025).

Training versus inference guardrails. Training teams rewarded for large-batch GEMM wins often propose decode “optimizations” that increase exports—for example, inserting debug dumps or auxiliary tensors between ops (Goodfellow *et al.*, 2016). Decode mindset gates reject changes that improve prefill TFLOPS but raise launches/token at batch-1 (Vellaisamy *et al.*, 2026). Onboarding docs for kernel hires from training backgrounds should include the export audit on day one so residency constraints are explicit before MMA tile experiments (Davies *et al.*, 2025).

Practice audits (portable).

1. Mark every HBM edge on one layer; propose a two-stage dataflow removing at least half.
2. Estimate shared-memory bytes for that fusion; split if over cap.
3. Run TaxBreak-style launch counting before and after (Vellaisamy *et al.*, 2026).
4. Read R_{floor} from Chapter 1’s worksheet at batch-1; high $R_{\text{floor}} \Rightarrow$ prioritize bytes, low $\Rightarrow$ prioritize launches (Chen, 2026a).
5. If targeting NPU, attempt static legalization; document which edges force DDR spill (AMD, 2024b).

Worked export audit (dense 7B layer). For batch-1, operator-first schedules typically materialize RMSNorm output, QKV tensors, attention output, and MLP intermediates to HBM—four to eight export events before residuals (Chen, 2026a; Vellaisamy *et al.*, 2026). A two-stage dataflow removing norm+QKV and MLP+residual exports cuts events by half in the common case; benchmark bytes/token before claiming stage count (Chen and YiRage Contributors, 2026). Attach the before/after diagram (Figure 2.3) to design docs so reviewers see removed edges, not only kernel names (Davies *et al.*, 2025).

Vendor library trap. Vendor GEMM wins inside isolated ops do not prove operator-first schedules are optimal (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). When swapping cuBLAS or hipBLAS versions, re-run mindset worksheets with fusion topology held constant—the variable is export topology, not MMA kernel choice (Chen, 2026a). Library bumps that speed ops but leave exports unchanged are upgrades; library bumps that enable deeper fusion without replanning are missed opportunities (Chen, 2020).

2.9 Proving the Mindset Shift: Verification Without the Checklist Novel

Benchmarking a mindset shift measures **schedule quality**, not isolated kernel FLOPs (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). Build paired decode cells on the same model: operator-first eager baseline versus dataflow-first fused schedule, batch-1, context buckets {512, 2048, 8192}, pinned weights and framework version. Warm-up must include graph construction and ≥ 1000 steady decode steps—single-step microbenches that never cross operator boundaries lie (Davies *et al.*, 2025).

Report **launches/token**, **bytes/token**, and **ms/token** together. Fusion that cuts launches but not bytes failed residency planning; fusion that cuts bytes but not ms/token may still be framework-limited—isolate in-step counters per Chapter 1 (Chen, 2026a; Kwon *et al.*, 2023).

Four verification buckets (details in Appendix). (1) **Core metrics**—export-edge count, bytes/token, launches/token, ms/token on three context lengths. (2) **Process**—paired operator/dataflow scripts in CI; bisect fusion passes before MMA tiles when bytes regress (Chen and YiRage Contributors, 2026). (3) **Cross-scenario checks**—long-context stress (paging can reintroduce exports), MoE

Metric	Mindset comparison meaning
Launches/token	Operator graph vs fused stages
Bytes/token	HBM export volume between adjacent ops
R_{floor}	Bandwidth headroom after fusion (Chapter 1)
Export-edge count	Global materializations per layer step
Triplet legality	Same IR compiles on GPU / CPU / NPU

Table 2.2: Mindset verification worksheet (extends Chapter 1 goodput counters).

routing tax counted separately from dense MLP fusion (Liu *et al.*, 2024; Kwon *et al.*, 2023). **(4) CI gates**—export reduction ratio ≥ 0.4 for layer-scope promotions unless documented attention-boundary exceptions; triplet compile green on pinned artifact hash (Vellaisamy *et al.*, 2026; Chen, 2026a). Fine-grained checklist items (release train gates, JSON artifact schema, executive readout templates) live in Appendix F’s *Dataflow Optimization Verification Checklist* so this chapter stays narrative, not a policy manual.

Regression bisect order. When bytes/token regresses after a fusion merge, bisect fusion passes before bisecting MMA tiles—export-edge diffs localize faster than nanosecond kernel diffs (Chen and YiRage Contributors, 2026; Chen, 2026a). When launches/token regresses, inspect graph capture scope and framework dispatch before rewriting dataflow stages (NVIDIA, 2024; Vellaisamy *et al.*, 2026). When ms/token regresses but counters improve, audit batching policy and measurement cells—queue effects mimic optimization wins (Kwon *et al.*, 2023; Davies *et al.*, 2025).

Goodput linkage. Mindset shifts matter only when ms/token moves together with bytes/token and launches/token—publish all three on one dashboard row weekly (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a). Higher concurrency per card follows lower bytes/token; better p99 streaming follows lower launches/token; CFO-visible opex follows both (Fan, 2025). Slides that show TFLOPs without layer-scope counters restart the expensive operator-first debate without evidence (Goodfellow *et al.*, 2016).

Continuous batching and fleet goodput. Fleet batching improves weight reuse but does not fix per-request export edges inside layers (Kwon *et al.*, 2023). Benchmark batch-1 mindset counters separately from fleet goodput—wins must appear in both when products mix interactive chat with high-throughput APIs

(Davies *et al.*, 2025). A schedule that helps BS=1 tail latency but breaks at fleet batch sizes still needs framework scheduling work; a schedule that helps fleet averages but not BS=1 fails chat SLOs (Chen, 2026a). Mindset benchmarks for this chapter focus on decode cells—do not accept prefill-only TFLOPS charts as proof of dataflow success (Vellaisamy *et al.*, 2026). Industrial postmortems that mix prefill throughput slides with decode SLO misses hide the mindset gap this chapter closes (Chen, 2026a).

Long-context and paging interaction. Extend mindset benchmarks to long context when paging is enabled—fusion plans that work at 2K can reintroduce exports at 32K when block tables grow (Kwon *et al.*, 2023; Dao *et al.*, 2022). Fail promotion when export-edge count rises with context while launches/token stays flat (Chen, 2026a). Paging solves capacity; dataflow solves per-step activation residency—teams need both ledgers (Davies *et al.*, 2025).

Quantization and scale tensors. INT8 and FP8 paths need explicit scale tensors in benchmark exports so fusion legality checks do not fold away dequant boundaries (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026). Operator-first stacks often dequantize to global buffers per op; dataflow plans should keep scales in-register or in shared tiers when mathematically legal (Chen, 2020). Quantization without residency replanning buys weight bandwidth but leaves activation ping-pong untouched (Chen, 2026a).

Handoff to hardware chapters. Pin completed mindset worksheets—operator graph, dataflow stage diagram, export-edge table, triplet legality summary—before entering Chapter 3. Hardware constraint chapters should not re-derive export counts that this chapter already measured; they should explain which tier budgets are feasible on each silicon class (NVIDIA, 2022; AMD, 2024b).

Example 2.9.1. Promotion gate: a three-op fusion must cut global exports by ≥ 2 edges and bytes/token by $\geq 10\%$ at $\text{ctx}=2048$, *or* document which iron rule forced a stage split with profiler evidence (Chen, 2026a; Chen and YiRage Contributors, 2026). Attach the residency diagram and export-edge table to the PR—reviewers cannot judge dataflow claims without topology (Vellaisamy *et al.*, 2026).

Executive and incident readouts. One-page promotion summaries should show operator versus dataflow counter deltas side by side—executives approve schedules, not kernel names (Davies *et al.*, 2025; Chen, 2026a). Decode incidents

should replay export-edge counts from canary worksheets before blaming drivers or hardware (Vellaisamy *et al.*, 2026). Missing worksheets recreate the multi-week debates that paired benchmarking is meant to prevent (Chen and YiRage Contributors, 2026). Store mindset artifacts in the same versioned bucket as compiler fingerprints so bisect can compare export topology across releases (Chen, 2020).

Closing verification mantra. Profile exports first, fuse second, tune FLOPs third, re-profile on every context bucket change (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). Green mindset promotion on three context buckets requires export reduction ratio ≥ 0.4 , bytes/token down, launches/token down or flat, ms/token down, and triplet legality on pinned hashes—partial green is how fleets inherit operator-first relapses behind a single load balancer (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023).

2.10 Key Takeaways

- **Schedule before operators.** Decode optimization wins when you replan exports and launches across the layer, not when you squeeze one more percent inside an isolated matmul (Vellaisamy *et al.*, 2026; Chen, 2026a). **Action:** run export-edge and TaxBreak launch audits before accepting any kernel-only PR; default to fusion when Chapter 1 classifies orchestration dominance.
- **Residency is the root lever.** Activations that could live on-chip across QKV→attention→MLP must not round-trip through HBM between mathematically adjacent ops (Chen, 2026a; Dao *et al.*, 2022). **Action:** enforce the three iron rules as release gates; use Nsight Compute global traffic and LLC profiles to catch ping-pong.
- **Cross-hardware from day one.** GPU fusion depth, CPU blocking, and NPU static ADF regions differ for the same math—port the dataflow graph, not the CUDA kernel list (AMD, 2024b; Chen and YiRage Contributors, 2026). **Action:** add a CPU or NPU leg to every fusion promotion; reject GPU-only evidence for fleet-wide schedules.
- **Prove shifts with paired counters.** Operator-first and dataflow-first baselines must move bytes/token, launches/token, and ms/token together on identical cells (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). **Action:** store paired worksheets in CI; block merge when export reduction ratio and triplet legality fail.

- **Automate the mindset.** MegaKernels materialize the schedule; YiRage automates lifetime planning and backend codegen from one IR (Chen and YiRage Contributors, 2026; Chen, 2020). **Action:** treat compile rejections on illegal residency as gifts—fix the graph, do not bypass with eager fallbacks.
- **Make the culture match the metrics.** Operator-first habits survive famous kernel libraries until organizations reward layer-scope counters in review and CI (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023). **Action:** pair operator and dataflow golden scripts in nightly CI; reject kernel-only PRs that do not update export-edge tables (Chen, 2026a; Chen and YiRage Contributors, 2026).

2.11 Conclusion

Operator-driven and dataflow-driven mindsets are different theories of what a decoder layer *is*: a list of kernels to tune versus a schedule of residency to plan (Chen, 2020; Goodfellow *et al.*, 2016). This book commits to the dataflow theory because at batch-1 decode the operator boundaries themselves are the cost, and only residency-first schedules—hand-crafted as MegaKernels or derived by YiRage—remove them (Chen and YiRage Contributors, 2026; Chen, 2026a).

That mindset is empty without hardware facts. Chapter 3 supplies concrete memory hierarchies, parallelism grains, and scheduling models for GPUs, CPUs, and NPUs that every later compiler pass encodes (NVIDIA, 2022; AMD, 2024b,a). Bring your export-edge worksheet from this chapter; Chapter 3 tells you which tier budgets are physically achievable on each silicon class.

Worksheet gate (repeat for memory). Before changing a kernel in Parts IV–VI, state whether Chapter 1 classifies your cell as orchestration- or bandwidth-limited, and whether this chapter’s export audit shows unnecessary HBM edges between fused-able ops (Vellaisamy *et al.*, 2026; Chen, 2026a). If you cannot answer, run TaxBreak and draw the operator graph first (Vellaisamy *et al.*, 2026). Dataflow-first design is not anti-kernel—it is anti-*boundary*; kernels remain how you implement stages once the schedule is sane (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022).

Hands-on next step. Open your decoder trace and mark every global export between LayerNorm, QKV, attention, and MLP on one layer (Chen, 2026a). If you count more than four exports, your next experiment is staged fusion or graph

replanning—not another MMA tile sweep (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Write before/after bytes/token and launches/token in the Appendix F spreadsheet before requesting headcount for kernel tuning (Davies *et al.*, 2025).

Index anchors for cross-reference. Decode optimization vocabulary in later chapters assumes these index terms: **dataflow-driven** schedules plan lifetimes first; **operator-driven** schedules list ops first; **export edges** are global materializations between fused-able ops; **MegaKernel** is the manual full-layer stage pattern; **iron rules** assign registers, on-chip SRAM, and HBM roles (Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016). When Part IV introduces TMA and shared-memory staging, you are implementing Rule 2 under Hopper-specific caps (NVIDIA, 2022). When Part VI introduces YiRage fusion passes, you are automating the five-step migration at IR scale (Chen, 2020). Keeping vocabulary anchored here prevents every chapter from redefining the same mindset in different words (Davies *et al.*, 2025).

Chapter 3

AI Hardware Architecture and Compiler Constraints

3.1 Why Lab-Tuned Kernels Fail in Production

Many compiler teams have shipped fusion kernels that hit 90%+ utilization in the lab, then watched production latency double after deploy. The root cause is often not the math—it is a few kilobytes of shared memory over budget that silently spills activations to HBM, wiping out fusion gains (Chen, 2026a; NVIDIA, 2022). Chapter 2 taught dataflow-first design; **every dataflow plan must land on concrete hardware constraints**—fusion without byte budgets is an architecture fiction. “The GPU has fast memory” is useless; “an H100 SM exposes up to 228 KB shared memory and a 256 KB register file” is input a tiling pass can solve against.

Bridge from Chapter 2. Dataflow mindset names *where* tensors should live; this chapter names *what fits* on each silicon class. A fusion region that needs 250 KB on-chip cannot exist on an H100 thread block without spilling—the schedule is wrong before anyone tunes MMA tiles (Chen and YiRage Contributors, 2026; Chen, 2020). Decode optimization is never “roughly aware” of hardware; a few kilobytes separate a $2\times$ win from a regression (Vellaisamy *et al.*, 2026).

Factory analogy (memory hierarchy). Registers are parts in the worker’s hands—instant access, tiny capacity. Shared memory / on-chip SRAM is the workbench beside the station. L2 / LLC is the shop-floor storeroom a short walk away. HBM / DDR is the remote warehouse across the plant (Goodfellow *et al.*,

2016; Chen, 2026a). Decode wins come from keeping activations off the warehouse route; hardware constraints tell you how much can stay on the bench.

Composite tri-hardware case (7B dense layer). The same mathematics, three deployment outcomes (composite internal migrations): **GPU**—operator-first 42 ms/token at batch-1; hardware-aware fusion ~ 18 ms/token ($\sim 2.3\times$) when shared-memory tiling respects SM caps (Chen, 2026a; Vellaisamy *et al.*, 2026). **CPU**—naively porting the GPU fusion evicted KV from LLC and ran 15% *slower* than native blocking; cache-aware retile gained $\sim 35\%$ over eager PyTorch (Davies *et al.*, 2025; Rotem *et al.*, 2018). **NPU**—dynamic per-op dispatch did not legalize; static ADF tile+DMA plan compiled and beat a same-power GPU class by $\sim 40\%$ on per-token latency in one edge SKU (AMD, 2024b,a). Matching architecture yields gains; ignoring constraints yields disasters (Chen and YiRage Contributors, 2026).

What this chapter gives you. You will leave able to (1) read a SKU datasheet row as a compiler constraint, (2) predict failure modes when a fusion violates capacity, (3) pick the right debug tool per class, and (4) justify target-specific schedules with bytes/token and launches/token instead of peak FLOPs (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). Chapter 2 told you to plan residency; this chapter tells you which bench has how many kilobytes (Chen and YiRage Contributors, 2026; NVIDIA, 2022; AMD, 2024b).

Reader prerequisites. We assume Chapter 1’s goodput counters and Chapter 2’s export-edge mindset (Vellaisamy *et al.*, 2026; Chen, 2026a, 2020). No chip-design background required—but you must be willing to read capacities as hard inequalities, not inspirational slogans (Davies *et al.*, 2025). If SIMT or NUMA are new, the terminology bridge in §3.2 defines them on first use (NVIDIA, 2022; Goodfellow *et al.*, 2016).

How to read this chapter. Skim Table 3.3 first if you are time-constrained—it is the cheat sheet the rest of the chapter elaborates (Davies *et al.*, 2025). GPU engineers should still read §3.6 to understand why fleet schedules diverge; edge engineers should read §3.4 to see what flexibility they traded away (AMD, 2024b; NVIDIA, 2022). Software-first readers: treat every capacity number as an inequality in the compiler, not trivia for a hardware quiz (Chen and YiRage Contributors, 2026; Chen, 2020).

Skillful engineering over SKU shopping. Once frameworks and famous kernels are enabled, the remaining lever is schedules that fit silicon (Chen and YiRage Contributors, 2026; Chen, 2026a). A mid-tier GPU with legal fusion beats a flagship running an eager loop—Memory-Floor’s L4 versus H100 cells demonstrate the pattern (Chen, 2026a; Vellaisamy *et al.*, 2026). Hardware literacy is how you reproduce that pattern on *your* fleet (Davies *et al.*, 2025).

Spill signatures in telemetry. Production regressions often show flat GPU utilization with rising HBM throughput—classic spill or export-edge behavior (Chen, 2026a; NVIDIA, 2022). CPU regressions show rising LLC miss rate per token without code changes in the math kernel (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). NPU regressions show rising DDR bytes per token when a new ADF version added a DMA stage (AMD, 2024b; Chen and YiRage Contributors, 2026). Learn to map telemetry shapes to constraint classes before opening profiler captures (Vellaisamy *et al.*, 2026; Chen, 2026a).

Chapter artifact. By the end of this chapter you should maintain a one-page constraint sheet per deployment target: capacities, bottleneck column from Table 3.3, and baseline bytes/token plus launches/token (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Update the sheet when SKUs, context limits, or quantization policy change (Chen and YiRage Contributors, 2026; Chen, 2026a). Chapter 4 assumes that sheet exists and adds Hopper-specific knobs on top (NVIDIA, 2022; SemiAnalysis, 2024).

Closing reminder. Hardware constraints are not obstacles to clever math—they are the coordinate system clever schedules are written in (Chen, 2020; Chen and YiRage Contributors, 2026). Respect the numbers early; profilers should confirm wins, not discover basic illegality (Chen, 2026a; Davies *et al.*, 2025). Keep the constraint sheet in version control next to the compiler config (Chen and YiRage Contributors, 2026).

3.2 AI Hardware Landscape: Three Architectures, Three Optimization Logics

Production is no longer GPU-only. An **NVIDIA GPU** is a **SIMT** (single-instruction multiple-thread) machine: thousands of threads scheduled dynamically over registers, shared memory, L2, and HBM (NVIDIA, 2022). An **AMD GPU**

Table 3.1: AI accelerator classes and primary execution models.

Class	Execution model	Decode fit
NVIDIA GPU	SIMT + tensor cores	Graphs, deep fusion
AMD GPU	Wavefront + matrix cores	HIP fusion
x86/ARM CPU	Cache + SIMD	Cache-blocked fusion
Edge NPU	Spatial dataflow	Static MegaKernel

uses wavefronts and matrix cores via HIP/ROCm with similar hierarchy, different tooling (AMD, 2024a). An **x86/ARM CPU** runs coherent caches and **SIMD** vector lanes with flexible control flow (Goodfellow *et al.*, 2016). An edge **NPU** such as AMD XDNA is a spatial dataflow fabric: fixed tiles, explicit SRAM, no hardware cache (AMD, 2024b).

Each class fails batch-1 decode differently. GPUs pay **kernel launch overhead** (scheduling overhead) per operator boundary—Chapter 1’s launches/token (Vellaisamy *et al.*, 2026). CPUs trade peak bandwidth for flexible blocking and low idle cost (Davies *et al.*, 2025). NPUs trade flexibility for deterministic on-chip bandwidth when the static plan fits (AMD, 2024a). Table 3.1 “decode fit” prescribes the artifact each class wants: captured fusion on GPU, cache-blocked bundles on CPU, static MegaKernel on NPU (Mirage Project contributors, 2025; Rotem *et al.*, 2018).

Terminology introduced here. **WGMMMA** (warp-group matrix multiply-accumulate) is Hopper’s high-throughput tensor-core instruction issued by warp-groups of four warps (Shah *et al.*, 2024; NVIDIA, 2022). **DMA** (direct memory access) engines move data without burning compute threads—critical on NPUs and increasingly on GPUs via **TMA** (SemiAnalysis, 2024; AMD, 2024b). **TMA** (Tensor Memory Accelerator) performs async copies between global memory and shared memory on Hopper+, overlapping movement with math (NVIDIA, 2022; SemiAnalysis, 2024).

Why the landscape fragmented. For a decade, vendor libraries (cuDNN, cuBLAS) let frameworks treat the GPU as a fixed target (Chen *et al.*, 2018b; Chen, 2020). Inference on phones, laptops, and custom accelerators broke that assumption: different memory structures, different parallelism, and LLM serving costs that made operator-library launch overhead unacceptable (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). The compiler’s job grew from “emit a good GPU kernel”

to “emit a legal, fast schedule for whichever incompatible machine this fleet deploys” (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Execution models, not speed ranks. A GPU oversubscribes warps to hide latency; a CPU hides latency with cache and prefetch; an NPU has no dynamic oversubscription—only a fixed spatial array (NVIDIA, 2022; Goodfellow *et al.*, 2016; AMD, 2024b). Compilers that pretend an NPU is a slow GPU emit illegal or pathological schedules (Wu *et al.*, 2024; AMD, 2024a). Table 3.1 is a taxonomy of models, not a procurement leaderboard (Chen, 2026a).

Multi-hardware delta. The same decoder layer yields a captured fused megakernel on NVIDIA GPU, a cache-blocked ahead-of-time bundle on CPU, and a static tile+DMA ADF on NPU (SemiAnalysis, 2024; AMD, 2024a; Rotem *et al.*, 2018). Identical math, different bytes/token—hardware is a first-class compiler input (Chen and YiRage Contributors, 2026; Chen, 2020).

Decode anchors from Chapter 1 (one-line). TaxBreak reports hundreds to thousands of launches per token on dense and MoE decoders; removing host overhead bought $\sim 1.26\times$ on a Qwen-2.5-7B cell without changing math (Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024). Those numbers are symptoms; this chapter explains the hardware boundaries fusion must respect to move them (Davies *et al.*, 2025).

AMD and second-source GPUs. AMD CDNA/RDNA paths mirror NVIDIA hierarchy with wavefronts instead of warps and matrix cores reached through HIP (AMD, 2024a). Constraint thinking transfers; magic numbers do not—each SKU row in the hardware model must be populated from datasheets, not copied from H100 (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Latency versus capacity on each class. GPU HBM offers enormous bandwidth but only if data stops round-tripping through the hierarchy (NVIDIA, 2022; Chen, 2026a). CPU DRAM bandwidth is lower but latency to L3 is excellent when blocking is correct (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). NPU DDR bandwidth is scarce yet deterministic when DMA is planned—predictability can beat peak bandwidth for edge SLAs (AMD, 2024b,a). Decode compilers optimize *where bytes live over time*, not abstract FLOPs (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).

Table 3.2: Cross-hardware constraint matrix.

Dimension	GPU	CPU	NPU
On-chip SRAM	Shared mem / L1	L1/L2/L3 cache	Tile buffers
Parallel grain	Warp (32 threads)	Core + SIMD	Spatial PE array
Dynamic shapes	Flexible	Flexible	Restricted

Operator libraries versus compiler ownership. Vendor libraries remain valuable baselines, but LLM decode needs cross-operator residency decisions libraries cannot see (Chen *et al.*, 2018b; Vellaisamy *et al.*, 2026). The shift from library calls to compiler-planned megakernels is driven by launches/token and bytes/token, not dislike of cuBLAS (Chen, 2026a; Chen and YiRage Contributors, 2026). Hardware constraints in this chapter explain *why* that shift is mandatory at batch-1 (Davies *et al.*, 2025).

3.3 Constraint Matrix: The Compiler’s Hard Ruler

The compiler does not optimize “a GPU” in the abstract—it satisfies a **constraint matrix**: on-chip capacity, memory hierarchy, parallel grain, shape flexibility (Chen, 2020; Chen and YiRage Contributors, 2026). On-chip SRAM is programmer-managed shared memory on GPU, hardware cache on CPU, explicit tile SRAM on NPU. Parallel grain is the warp, core+SIMD lane, or spatial processing element (PE) array respectively (NVIDIA, 2022; Goodfellow *et al.*, 2016; AMD, 2024b).

Decode is dominated by **bandwidth** through the hierarchy: batch-1 re-reads full weights and growing KV each token (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). The recurring failure is illegal tier placement—tensors that should stay in fast SRAM spilling to HBM/DDR (Chen, 2026a). The matrix lets passes reject illegal fusions *before* codegen instead of in p99 telemetry (Davies *et al.*, 2025).

✓ **Practice checkpoint.** Before merging a cross-hardware change, verify the constraint-matrix row for *each* target: capacities, parallel grain, and shape policy (Chen and YiRage Contributors, 2026). A GPU-only green build that breaks NPU legality should fail CI, not ship to heterogeneous fleets (AMD, 2024b; Davies *et al.*, 2025).

Shape flexibility row. GPUs and CPUs handle dynamic sequence lengths and growing KV naturally; static NPUs require bucketing or padding to a finite shape

Table 3.3: Decode optimization comparison across hardware classes (design-time cheat sheet).

Dimension	GPU	CPU	NPU
Core bottle-neck	Launch overhead + HBM bandwidth	Cache misses + NUMA latency	On-chip capacity + static layout
Optimization focus	Deep fusion + graph capture	Cache blocking + thread affinity	Static pipeline + DMA plan
Best fusion grain	Layer / half-layer stages	Coarse operator groups	Full-layer / multi-layer MegaKernel
Dynamic shapes	Strong	Strong	Weak (bucketing/padding)
Typical failure	Register/shared spill	LLC eviction + remote NUMA	Compile fail / tile imbalance

set (AMD, 2024b; Lai *et al.*, 2023). Relax-style systems made dynamic shapes a first-class compiler concern because they determine *whether* a workload runs, not only how fast (Lai *et al.*, 2023). For decode serving, shape policy is as important as SRAM capacity (Kwon *et al.*, 2023).

Why “fastest chip” is the wrong question. Datacenter GPUs lead raw HBM bandwidth but carry kernel launch overhead and cost; CPUs trade bandwidth for control; NPUs lead power efficiency with the least runtime flexibility (Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a). Each matrix row is a deployment trade—Chapter 24 routes fleets using exactly these cells (Davies *et al.*, 2025).

Roofline attachment discipline. Attach a roofline or buffer-accounting sheet for each fused region on the target SKU before merge (Chen, 2026a; Chen and YiRage Contributors, 2026). Over-budget regions are build errors: the matrix supplies the numbers; the compiler supplies the inequality check (NVIDIA, 2022; Chen, 2020).

Connecting matrix rows to Part VI passes. Residency passes read on-chip SRAM rows; parallel-grain passes choose warp versus SIMD versus spatial tiling; shape-flexibility passes decide bucketing policies (Chen and YiRage Contributors, 2026; Lai *et al.*, 2023; Chen, 2020). When a pass violates a cell, reject before codegen—the same discipline Chapter 2 asked for on export edges (Chen, 2020; Vellaisamy *et al.*, 2026).



Figure 3.2: GPU memory hierarchy for decode (H100-class sizes).

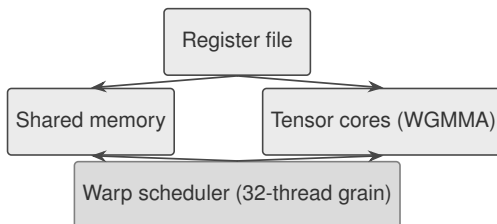


Figure 3.3: H100 SM resource sketch: fusion budgets contend for the same register and shared-memory pool.

Using the comparison table in reviews. Table 3.3 columns are intentional prompts: if your PR touches GPU fusion, check whether launches/token moved; if it touches CPU blocking, check LLC misses; if it touches NPU ADF, check DMA edges (Vellaisamy *et al.*, 2026; AMD, 2024b; Goodfellow *et al.*, 2016). A change that improves one column while regressing another on the same class is incomplete (Chen, 2026a; Chen and YiRage Contributors, 2026). Fleet routers in Part VII read the same columns when choosing SKU per query class (Davies *et al.*, 2025).

3.4 GPU Constraints: Capacity and Orchestration Together

On an H100 SM, shared memory is configurable up to 228 KB (227 KB usable per block after 1 KB reserved), the register file holds 65,536 32-bit registers (256 KB total, capped at 255 registers per thread), and ~50 MB L2 backs HBM (NVIDIA, 2022). The **warp** of 32 threads is the scheduling grain; four warps form a warpgroup for WGMMA; Hopper adds thread-block clusters sharing shared memory across CTAs (Shah *et al.*, 2024).

Shared memory capacity (constraint loop). **Engineering meaning:** this is the hard ceiling for fused decode tiles—if your fusion region needs 250 KB, it *will* spill or must split (Chen, 2026a). **Common pitfall:** counting tensor bytes only, ignoring bank padding, double buffering, and epilogue temporaries (NVIDIA, 2022). **Debug:** Nsight Compute Shared Memory Configuration and shared-

memory throughput; elevated `local memory traffic` signals register/shared spill to HBM-backed local memory (Vellaisamy *et al.*, 2026).

Register pressure vs occupancy. Resident threads share the 65,536-register file; a kernel using R registers per thread caps residents near $65536/R$ (NVIDIA, 2022). At batch-1, occupancy often matters less than residency—until spills flip the trade (Chen, 2026a). **Debug:** register usage per thread and spill metrics in Nsight Compute; if spills appear, shrink fusion depth or retile before tuning MMA micro-kernels (Chen and YiRage Contributors, 2026).

Shared-memory bank conflicts. A **bank conflict** occurs when multiple threads in a warp hit the same shared-memory bank in one cycle, serializing what should be parallel loads (NVIDIA, 2022). H100 shared memory uses 32 banks of 4 bytes; strided layouts that map many lanes to one bank are a common decode fusion footgun. Layout is part of residency planning, not a post-pass cosmetic (Shah *et al.*, 2024).

Kernel launch overhead. Each host launch pays dispatch cost; a dozen kernels per layer pays a dozen times per token at batch-1 (Vellaisamy *et al.*, 2026; Chen, 2026a). Fusion and CUDA Graph capture attack scheduling overhead directly—but graphs without dataflow still ping-pong inside recorded nodes (Chapter 2) (NVIDIA, 2024).

TMA and async movement (Hopper+). **Engineering meaning:** TMA lets the schedule prefetch tiles into shared memory while math runs on the previous tile—but only if copy edges are static enough to plan (SemiAnalysis, 2024; NVIDIA, 2022). **Pitfall:** treating TMA as a band-aid on operator-first ping-pong; bytes/token must still drop (Chen, 2026a). **Debug:** Nsight Systems timelines for TMA overlap; correlate with shared-memory occupancy (SemiAnalysis, 2024).

Thread-block clusters. Hopper clusters let CTAs share shared memory across a larger fusion region—extending Rule 2 when a single block cannot hold the tile (NVIDIA, 2022). The compiler must prove cluster-wide footprint still fits hardware caps; otherwise cluster use increases complexity without residency wins (Chen and YiRage Contributors, 2026).

Multi-hardware delta (GPU vs NPU). GPUs hide latency with async copies and deep pipelines; NPUs require those copies to be scheduled statically in the ADF (AMD, 2024b; SemiAnalysis, 2024). GPU constraint is scheduling overhead *and* capacity; NPU constraint is capacity with zero runtime slack (Wu *et al.*, 2024).

WGMMMA layout constraint. Tensor cores consume operands from shared memory in specific layouts; feeding them from misaligned tiles triggers conversion or gather traffic that negates fusion (Shah *et al.*, 2024; NVIDIA, 2022). Layout assignment is therefore part of the residency decision, not an afterthought on the critical path (Chen, 2020; Chen and YiRage Contributors, 2026).

Chapters 6–8 preview. Parts IV–V tile decode megakernels against the exact numbers in this section—register caps, shared-memory banks, cluster footprints (NVIDIA, 2022; SemiAnalysis, 2024). When those chapters cite 227 KB usable shared memory, they mean this inequality, not a metaphor (Chen, 2026a).

✓ **Practice checkpoint.** Before merging GPU fusion: (1) statically budget shared memory + registers against the target SM; (2) verify in Nsight Compute no spill and acceptable bank conflicts (NVIDIA, 2022; Chen, 2026a).

Worked example: fused attention tile budget (H100). Suppose a decode fusion stage keeps Q , K , V tiles, softmax scratch, and an output accumulator in shared memory for a head dimension of 128 and block size 64. A naive byte count might report ~ 48 KB of tensor data, but the compiler also reserves bank-padding columns, a double buffer for the next K tile prefetch, and a 4 KB epilogue for the projection matmul fragment (Shah *et al.*, 2024; NVIDIA, 2022). Add WGMMMA operand staging in swizzled layout and the footprint can cross 120 KB before anyone accounts for cluster-wide sharing (SemiAnalysis, 2024). The correct workflow is to build a line-item residency sheet: tensor bytes, padding, pipeline stages, temporaries, then compare the sum to 227 KB usable (Chen, 2026a; Chen and YiRage Contributors, 2026). If the sheet exceeds the cap, split the fusion at the export edge Chapter 2 marked—for example, keep attention fused but export logits before the MLP rather than forcing an illegal megakernel (Chen, 2020; Vellaisamy *et al.*, 2026). Teams that skip the sheet often discover spill only when production batch sizes differ from the lab cell (Davies *et al.*, 2025).

Occupancy versus residency trade (when to care). High occupancy helps latency hiding when kernels are memory-bound *and* footprints are small (NVIDIA,

2022). Decode megakernels with large shared-memory tiles intentionally run fewer blocks per SM because the residency win dominates occupancy (Chen, 2026a; Dao *et al.*, 2022). The mistake is chasing occupancy charts from a GEMM microbench while your fused kernel already spilled (Chen and YiRage Contributors, 2026). Profile bytes/token first; tune occupancy only when spill metrics are clean (Vellaisamy *et al.*, 2026).

Graph capture interaction. CUDA Graphs remove per-launch host overhead but freeze the launch structure (NVIDIA, 2024; Vellaisamy *et al.*, 2026). Hardware budgets still apply inside captured nodes: a graph does not create more shared memory (NVIDIA, 2022; Chen, 2026a). Capture after fusion is legal and bytes/token improved—not as a substitute for illegal tiling (Chen and YiRage Contributors, 2026; Chen, 2020). TaxBreak’s launch-reduction wins assumed graphs wrapped already-resident schedules (Vellaisamy *et al.*, 2026; Chen, 2026a).

Blackwell preview (numbers change, inequalities do not). Later chapters update register and shared-memory ceilings for Blackwell-class SKUs (NVIDIA, 2022; Chen and YiRage Contributors, 2026). The workflow in this section stays identical: line-item budget, compare to modeled cap, reject or split before codegen (Chen, 2020; Davies *et al.*, 2025). New tensor instructions change *how* you fill the budget, not *whether* a budget exists (SemiAnalysis, 2024; Shah *et al.*, 2024).

L2 as a hidden export edge. Even when shared memory is legal, writing full activations through L2 to HBM between subgraphs reintroduces the warehouse trip (Chen, 2026a; Kwon *et al.*, 2023). L2 capacity is large but bandwidth to HBM is still the decode bottleneck at batch-1 (NVIDIA, 2022). Fusion should minimize round trips, not merely fit in shared memory while exporting every intermediate (Chen, 2020).

3.5 CPU Constraints: Cache, SIMD, and NUMA

CPU residency is statistical: blocking so tiles fit L1/L2/L3 before eviction, not explicit scratchpad allocation (Goodfellow *et al.*, 2016; Rotem *et al.*, 2018). Parallel grain is core + **SIMD** (single-instruction multiple-data) lanes—AVX-512 on x86, NEON on Arm—coarser than GPU warps (Goodfellow *et al.*, 2016).

NUMA (non-uniform memory access). On multi-socket hosts, each socket has local DRAM; remote-socket access adds latency and bandwidth loss (Davies *et al.*, 2025). **Engineering meaning:** pin decode threads and weight buffers to the same NUMA node, or replicate read-only weights per node when cheaper than remote traffic (Chen, 2026a). **Debug:** `numastat`, `perf LLC` miss counters, and cross-node traffic on the decode loop.

Bandwidth floor. A 7B model in BF16 moves ~ 14 GB of weights per token at batch-1—dividing by DRAM bandwidth sets a latency floor no micro-kernel beats (Kwon *et al.*, 2023; Chen, 2026a). CPU wins on flexibility and cost at low QPS, not on matching H100 tail latency (Davies *et al.*, 2025).

SIMD + quantization coupling. Int8 weights shrink cache footprint and fill SIMD lanes denser—double win when bandwidth-bound (Goodfellow *et al.*, 2016). GPU fusion depth copied blindly can evict KV working sets; CPU schedules need cache-aware blocking (Rotem *et al.*, 2018).

✓ **Practice checkpoint.** Validate blocking plans with measured L3 miss rate and bytes/token at the production batch policy—paper budgets are insufficient on CPUs (Chen, 2026a; Goodfellow *et al.*, 2016).

Prefetch and layout. Sequential weight streams benefit from hardware prefetch when layouts are contiguous (Goodfellow *et al.*, 2016). Fragmented layouts defeat prefetch and amplify bandwidth tax on a already floor-limited CPU decode cell (Davies *et al.*, 2025; Kwon *et al.*, 2023).

Multi-hardware delta (CPU vs GPU). CPU residency is statistical—miss rates matter; GPU residency is contractual—kilobytes over cap spill (NVIDIA, 2022; Goodfellow *et al.*, 2016). A blocking plan that “should fit” in L3 must be validated; a shared-memory plan that exceeds 227 KB must be rejected arithmetically (Chen, 2026a; Chen and YiRage Contributors, 2026).

When CPU decode makes business sense. Low-QPS endpoints, strict cost caps, and workloads with heavy control flow may favor CPU even when GPUs lead peak throughput (Davies *et al.*, 2025; Kwon *et al.*, 2023). The constraint chapter equips you to recognize those regimes instead of forcing GPU schedules everywhere (Rotem *et al.*, 2018; Goodfellow *et al.*, 2016).

Glow and AOT mental model. Ahead-of-time bundles with static shapes mirror NPU thinking on CPU: fewer surprises at runtime, more work at compile time (Rotem *et al.*, 2018; Chen, 2020). The book’s CPU path favors predictable residency over last-minute operator dispatch (Chen and YiRage Contributors, 2026).

Worked example: L3 blocking for a 7B MLP slice. At batch-1, a single MLP projection reads the full weight matrix from DRAM each token (Kwon *et al.*, 2023). Blocking chooses a strip height so the active weight tile plus input vector plus output accumulator stays in L3 across the inner dimension (Goodfellow *et al.*, 2016; Rotem *et al.*, 2018). If the strip is too wide, the next layer’s KV working set evicts the weight tile—exactly the 15% regression seen when GPU fusion depth was copied without cache accounting (Davies *et al.*, 2025). Measure LLC misses per token at the production thread count; if misses scale with layer count, the block is too aggressive (Chen, 2026a). CPU optimization is iterative measurement against a byte budget, not a one-shot formula (Goodfellow *et al.*, 2016).

Thread pinning recipe. Bind decode worker threads to cores on the same NUMA node as the weight buffer (Davies *et al.*, 2025). On dual-socket hosts, replicate read-only weights per socket when replication cost is less than remote DRAM traffic during the decode loop (Chen, 2026a). Document the pinning policy beside the blocking plan—silent migration to a remote socket looks like “compiler regression” in dashboards (Chen and YiRage Contributors, 2026).

AVX-512 and decode tails. Wide SIMD helps when inner dimensions are multiples of the vector width; odd head sizes leave lanes idle unless the compiler pads or peels tails (Goodfellow *et al.*, 2016). Padding changes cache footprint; peeling adds scalar epilogue cost (Rotem *et al.*, 2018). Co-decide head padding policy with quantization and blocking, not in a late vectorization pass (Chen, 2020).

3.6 NPU Constraints: Static Spatial Plans

Spatial NPUs are deterministic and unforgiving. AMD XDNA2/AIE-ML tiles expose 64 KB L1 SRAM per compute tile (eight 8 KB banks) with DMA, column L2 tiles at 512 KB, and DDR via shim tiles—no hardware cache (AMD, 2024b,a). What is resident is exactly what the compile-time DMA schedule placed.

Tile capacity (constraint loop). **Engineering meaning:** operands must fit tile SRAM or be split with explicit multi-tile DMA chains—there is no runtime cache to hide mistakes (AMD, 2024b). **Failure mode:** compile failure or severely imbalanced pipelines that idle tiles (Wu *et al.*, 2024). **Debug:** compiler legality reports and DMA edge counts per token; compare against modeled tile budgets in YiRage (Chen and YiRage Contributors, 2026).

Spatial cascade. Adjacent tiles pass data through cascade links without DDR round trips—the on-chip analogue of deep software pipelining (AMD, 2024a). The spatial plan is fixed at compile time; dynamic decode loops require shape bucketing or padding (Lai *et al.*, 2023; AMD, 2024b).

✓ **Practice checkpoint.** NPU promotions require green static compile, balanced tile utilization, and bytes/token regression versus the prior ADF hash—no “works on GPU” excuses (Chen and YiRage Contributors, 2026; AMD, 2024b).

Optimization means balance, not search. On GPUs, imperfect schedules hide behind dynamic scheduling; on NPUs, imbalanced tile assignment idles silicon with no runtime recovery (Wu *et al.*, 2024; AMD, 2024b). Edge deployments buy deterministic latency and power at the price of compile-time rigor (AMD, 2024a; Rotem *et al.*, 2018).

Attention on spatial fabrics. Canonical edge fusion maps attention as a spatial sweep with scores resident in tile SRAM and DDR touched only for inputs/outputs—the spatial analogue of a GPU megakernel (Dao *et al.*, 2022; Chen, 2026a; AMD, 2024b).

DDR is not “slow GPU HBM.” Off-chip DRAM bandwidth on edge cards is scarce; every DMA must be planned, counted, and minimized in the ADF (AMD, 2024b,a). Bytes/token ledgers from Chapter 1 apply unchanged—only the mechanism counter becomes DMA edges instead of launches (Vellaisamy *et al.*, 2026; Chen, 2026a).

Mirage and spatial compilers. Spatial compiler stacks (Mirage-class) search tile assignments where GPU stacks search thread blocks (Wu *et al.*, 2024; AMD, 2024b). The mindset shift from Chapter 2 is the same; the search space geometry changes (Chen, 2020; Chen and YiRage Contributors, 2026).

ADF planning checklist (edge NPU). Start from a shape bucket the product will ship—for example `ctx=2048` with fixed head count (Lai *et al.*, 2023; AMD, 2024b). Assign each tensor to a tile SRAM bank with explicit DMA in and out edges; count bytes moved per token (AMD, 2024a). Balance compute across tiles: an attention sweep that leaves half the array idle fails even when compile succeeds (Wu *et al.*, 2024). Run legality first, then bytes/token, then ms/token—dynamic GPU shortcuts do not apply (Chen and YiRage Contributors, 2026; Chen, 2026a). Version the ADF hash in CI next to GPU kernel hashes so regressions are comparable across classes (Davies *et al.*, 2025).

When NPU compile fails (readable errors). Out-of-tile SRAM is the most common failure: the fix is smaller spatial tiles or fewer fused stages, not “retry with more threads” (AMD, 2024b). DMA stride misalignment fails legalization because the shim expects burst-friendly layouts (AMD, 2024a). Treat compile errors as constraint violations with the same seriousness as shared-memory overflow on GPU (Chen, 2020; Chen and YiRage Contributors, 2026).

3.7 Hardware-Aware Compilation: Constraints Become Pass Inputs

Hardware-aware compilation makes constraints data, not folklore. Three coupled decisions dominate: **tiling**, **layout**, and **pass** ordering (Chen, 2020; Lai *et al.*, 2023).

Tiling sizes on-chip working sets to fit 228 KB shared memory (GPU), a cache level (CPU), or 64 KB tile SRAM (NPU)—the largest bytes/token lever (NVIDIA, 2022; AMD, 2024a). **Layout** ensures coalesced global access, bank-conflict-free shared tiles, and DMA-friendly strides (Shah *et al.*, 2024). **Pass order** matters: layout-after-fusion transposes can re-materialize activations fusion meant to keep resident (Lai *et al.*, 2023; Chen *et al.*, 2018b). Precision (BF16/FP8/int8) co-decides with tiling because formats change footprint and native instruction support per SKU (Shah *et al.*, 2024; Liu *et al.*, 2024).

✓ **Practice checkpoint.** Diff IR or assembly for new global buffers; count launches/token (GPU) and DMA edges (NPU); confirm tiling still fits modeled capacity per target (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2026a).

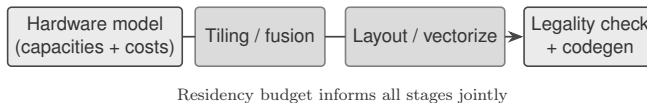


Figure 3.4: Hardware-aware compile flow: the chip model feeds tiling, layout, and pre-codegen legality checks.

Joint residency budget. Larger tiles improve compute intensity but consume SRAM; padded layouts avoid bank conflicts but cost bytes; double buffering for pipelines doubles footprints (NVIDIA, 2022; Shah *et al.*, 2024). Hardware-aware compilation is one joint optimization against modeled capacity—not tiling, then layout, then surprise spill (Chen, 2020; Chen and YiRage Contributors, 2026). Passes that fix tiling first and discover layout incompatibility later must backtrack; residency budgets should constrain all three from the start (Lai *et al.*, 2023).

Per-target pass forks. The right pass order differs by class: NPUs need tile assignment before vectorization details; GPUs may fuse before layout on some graphs but not others (Chen *et al.*, 2018b; Chen, 2020). Expose pipelines explicitly (Part VI) so hardware constraints become visible engineering choices, not accidental emergent behavior (Chen and YiRage Contributors, 2026).

Anti-pattern: layout-after-fusion transpose. A transpose inserted after fusion may force a full activation write/read pair—exactly the export Chapter 2 removed (Lai *et al.*, 2023; Chen, 2020, 2026a). Hardware-aware compilers order layout and fusion so residency commitments remain satisfiable (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Anti-pattern: precision change without retile. Dropping BF16 activations to FP8 halves bytes but may change vectorization and shared-memory padding requirements (Shah *et al.*, 2024; Liu *et al.*, 2024). Co-decide precision with tiling when bandwidth-bound (Chen, 2026a; Davies *et al.*, 2025).

3.8 Common Cross-Hardware Migration Mistakes

Teams porting decode schedules between silicon classes repeat the same failures. This section names five patterns so you can catch them in design review before they become production incidents (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020).

Mistake 1: capacity math from the wrong machine. Using H100 shared-memory numbers on a laptop NPU compile, or assuming CPU L3 is as programmable as GPU shared memory, produces schedules that are either illegal or silently slow (NVIDIA, 2022; AMD, 2024b; Goodfellow *et al.*, 2016). **Fix:** read the constraint-matrix row for *each* target before writing fusion rules (Chen and YiRage Contributors, 2026).

Mistake 2: promoting GPU fusion depth to CPU unchanged. Deep GPU megakernels maximize on-chip residency when spill is impossible; on CPU the same depth evicts KV and weight tiles from LLC (Rotem *et al.*, 2018; Davies *et al.*, 2025; Chen, 2026a). **Fix:** re-block for cache hierarchy; accept shallower fusion with better miss rates (Goodfellow *et al.*, 2016).

Mistake 3: expecting dynamic shapes on static NPUs. Serving growth in sequence length without re-bucketing breaks ADF plans that assumed fixed ctx (Lai *et al.*, 2023; AMD, 2024b). **Fix:** maintain a finite shape set; compile one ADF per bucket; pad explicitly where product allows (Chen and YiRage Contributors, 2026).

Mistake 4: optimizing peak FLOPs after fusion is legal. A legal kernel can still lose to scheduling overhead if launches/token remain high, or lose to bandwidth if bytes/token did not drop (Vellaisamy *et al.*, 2026; Chen, 2026a). **Fix:** gate promotions on Chapter 1 counters, not vendor GEMM banners (Davies *et al.*, 2025).

Mistake 5: hard-coded magic numbers in kernels. Scattering 228 KB constants in CUDA blocks Blackwell retargeting and breaks CI legality checks (Chen and YiRage Contributors, 2026; IREE / iree-org contributors, 2024; NVIDIA, 2022). **Fix:** centralize capacities in the hardware model; let passes read limits (Chen, 2020).

Design-review prompt. Ask in every cross-hardware PR: *Which constraint cell changed, what is the new byte budget, and which mechanism counter proves the win?* (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025) If the author cannot answer, the change is not ready to merge (Chen and YiRage Contributors, 2026).

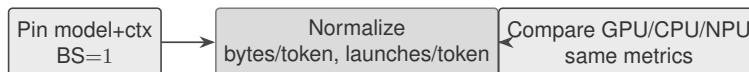


Figure 3.5: Cross-hardware benchmark flow used in this book.

Postmortem template. When production latency regresses after a fusion promotion, capture: SKU, ctx, batch, static byte budget, measured shared-memory or tile usage, spill or DMA flags, and before/after bytes/token (Chen, 2026a; Vellaisamy *et al.*, 2026; NVIDIA, 2022). Most postmortems reveal a violated inequality from this chapter, not a mysterious hardware flake (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). Store postmortems beside benchmark worksheets so the next engineer does not repeat the mistake (Chen, 2020).

3.9 Cross-Hardware Benchmarking: Fair Comparison

Honest hardware comparison fixes the workload, then grades decode metrics that predict serving cost—not vendor peak FLOPs (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). **launches/token** isolates scheduling overhead; **bytes/token** isolates bandwidth/residency (Kwon *et al.*, 2023). Decomposed counters are actionable; a fast GEMM microbench is not (Chen, 2026a).

Per-class mechanisms. GPUs move launches/token with fusion and graphs; CPUs move LLC misses and NUMA traffic; NPU move DMA edge counts (NVIDIA, 2022; AMD, 2024a; Goodfellow *et al.*, 2016). Report the *same* headline metrics while diagnosing class-specific mechanisms underneath (Vellaisamy *et al.*, 2026).

Reproducibility. Pin weights, framework versions, context buckets, and log configs so SKU A compares to SKU B and to next month’s compiler (Chen, 2026a; Chen and YiRage Contributors, 2026). Appendix B expands long-context and fleet-batch variants; this section states the core decode-cell discipline (Kwon *et al.*, 2023).

Decomposition beats single scores. End-to-end ms/token alone does not say whether to fuse or quantize; launches/token points to scheduling work; bytes/token points to residency work (Vellaisamy *et al.*, 2026; Chen, 2026a). This decomposition is why Chapter 1 insisted on goodput counters alongside latency (Davies *et al.*, 2025; Kwon *et al.*, 2023).

Autobooks harness analogy. The book’s own loop fixes a harness, scores every change on the same metrics, and reverts regressions—benchmark discipline as engineering culture, not a one-off table (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). Cross-vendor comparisons without pinned methodology decay into marketing slides (Chen, 2026a).

Fleet vs batch-1 cells. Report batch-1 mindset counters separately from fleet goodput—continuous batching changes weight reuse but not per-request export edges inside layers (Kwon *et al.*, 2023; Davies *et al.*, 2025). A schedule that helps interactive tail latency but fails at fleet averages still needs framework scheduling work (Chen, 2026a; Vellaisamy *et al.*, 2026).

Long-context buckets. Benchmark $\text{ctx} \in \{512, 2048, 8192\}$ minimum; paging can change effective residency at 32K even when 2K looked healthy (Kwon *et al.*, 2023; Dao *et al.*, 2022). Hardware-aware compilers must re-check tier budgets when block tables grow (Chen and YiRage Contributors, 2026; Chen, 2026a).

✓ **Practice checkpoint.** Ship benchmark artifacts with every hardware-target promotion: bytes/token, launches/token, ms/token at $\text{ctx} \in \{512, 2048, 8192\}$, plus the mechanism counter for that class (Vellaisamy *et al.*, 2026; Chen, 2026a).

Benchmark worksheet (copy into CI). Record model id, weight format, framework build, SKU, driver, ctx, batch policy, and power cap (Chen, 2026a; Chen and YiRage Contributors, 2026). Run warm-up tokens, then collect median ms/token over at least 100 tokens at batch-1 for interactive cells (Vellaisamy *et al.*, 2026). Log launches/token from the profiler trace; log bytes/token from a residency ledger or instrumented allocator (Kwon *et al.*, 2023; Chen, 2026a). Attach the worksheet to the PR; reviewers compare mechanism counters, not adjectives (Davies *et al.*, 2025). When a GPU win does not move bytes/token, suspect measurement noise or hidden exports (Chen, 2020).

Sanity checks before claiming cross-vendor victory. Same tokenizer, same weight checkpoint, same numerical policy (BF16 vs FP8), and same ctx buckets (Liu *et al.*, 2024; Chen, 2026a). Disable unrelated framework features (speculative decoding, paging) unless the product ships them (Kwon *et al.*, 2023). Report confidence intervals when variance exceeds 2%—decode cells are noisy at small batch (Davies *et al.*, 2025).

3.10 YiRage Hardware Modeling: One Pass Stack, Many SKUs

YiRage encodes this chapter as per-target **chip architecture models**—capacities, parallelism grains, latency/bandwidth ratios, launch floors—that residency, pipeline, and sync passes read and check (Chen and YiRage Contributors, 2026; Chen, 2020). A fusion’s footprint is compared to modeled tier capacity; tile shapes are validated against SRAM and bank structure; launch and scheduling costs inform schedule ranking (Wu *et al.*, 2024).

Updating a model re-specializes passes when a new SKU ships—H100 → Blackwell register growth, for example—without rewriting every hand kernel (IREE / iree-org contributors, 2024; Davies *et al.*, 2025). Legality-before-codegen is the scalability property heterogeneous fleets need: illegal residency fails compile instead of silently spilling in production (Chen and YiRage Contributors, 2026; AMD, 2024b).

✓ **Practice checkpoint.** Treat the hardware model as versioned data: diff model entries in CI when SKUs change; reject passes that hard-code capacities instead of reading the model (Chen and YiRage Contributors, 2026).

Capacities plus costs. Models need latencies, bandwidths, launch floors, and tensor-core throughput ratios—not SRAM kilobytes alone—to *rank* legal schedules, not only accept/reject them (NVIDIA, 2022; Shah *et al.*, 2024; Wu *et al.*, 2024). A legal but slow schedule is still a product failure (Davies *et al.*, 2025; Chen, 2026a).

Portability across SKUs. Because passes read the model, Blackwell register growth or new tensor instructions become data updates rather than rewriting every kernel in the tree (Chen and YiRage Contributors, 2026; IREE / iree-org contributors, 2024). Hardware knowledge lives in one auditable place instead of rotting inside hand-tuned CUDA (Chen, 2020).

IREE and modular retargeting. IREE-style retargeting treats backends as plugins over shared IR—the same architectural pattern YiRage uses for decode fusion (IREE / iree-org contributors, 2024; Chen and YiRage Contributors, 2026; Chen, 2020). Hardware models are the plugin contract (Davies *et al.*, 2025).

Chapters 6–9 dependency. Residency, role assignment, pipeline formation, and synchronization passes in Part IV all consume the chip model introduced here (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). Read this chapter once carefully; later chapters reference these inequalities by label rather than re-deriving them (NVIDIA, 2022; AMD, 2024b).

Model schema (conceptual). Each SKU entry typically carries tier capacities (registers, shared memory or tile SRAM, L2, DRAM), parallelism grains (warp size, SIMD width, tile array), bandwidth/latency ratios, launch or DMA floor costs, and supported instruction sets (WGMMA, matrix cores, int8 dot) (Chen and YiRage Contributors, 2026; NVIDIA, 2022; AMD, 2024b). Passes query the schema; kernels never embed literals (Chen, 2020). When Blackwell raises register file limits, bump the model version and re-run legality on the fusion corpus—no manual kernel sweep (IREE / iree-org contributors, 2024; Davies *et al.*, 2025).

Fleet heterogeneity. Production may run Hopper datacenter GPUs, CPU fallback nodes, and edge NPUs simultaneously (Davies *et al.*, 2025; AMD, 2024a). One IR, multiple chip models, divergent legal schedules—that is normal, not a failure of the compiler architecture (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). Chapter 24 returns to routing policies; this chapter supplies the per-target numbers those policies consume (Chen, 2026a).

Testing hardware models. Unit-test the model layer: given a synthetic fusion footprint, assert the legality pass accepts or rejects against golden SKU rows (Chen and YiRage Contributors, 2026; Chen, 2020). Regression tests caught more fleet incidents than microbench suites because they encode inequalities directly (Davies *et al.*, 2025; Chen, 2026a). When vendors ship stepping changes, update golden rows and re-run the corpus before merging driver pins (IREE / iree-org contributors, 2024; NVIDIA, 2022).

3.11 Key Takeaways

- **Quantify constraints, never hand-wave.** “Fast GPU memory” is not actionable; 227 KB usable shared memory per H100 block is (NVIDIA, 2022). **Action:** attach static byte budgets to every fusion PR; spill in Nsight Compute is a build failure, not a tuning opportunity (Chen, 2026a).

- **No universal optimal schedule.** GPU layer fusion, CPU cache blocking, and NPU static MegaKernels are different artifacts for the same math (AMD, 2024b; Rotem *et al.*, 2018; Mirage Project contributors, 2025). **Action:** fork compile pipelines per target; never promote GPU-only evidence to fleet-wide schedules (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).
- **Decode optimizes memory first, FLOPs second.** Batch-1 limits are bytes and launches on every class (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Chen, 2026a). **Action:** rank backlog items by bytes/token and launches/token before MMA tile sweeps (Davies *et al.*, 2025).
- **Hardware belongs in a configurable model.** Capacities and costs as data let passes specialize and check legality (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). **Action:** add new SKUs by extending the chip model, not scattering magic numbers in kernels (IREE / iree-org contributors, 2024).
- **Benchmark with decode metrics, not peak FLOPs.** Normalize bytes/token and launches/token across GPU, CPU, and NPU cells (Vellaisamy *et al.*, 2026; Chen, 2026a). **Action:** store paired worksheets in CI; reject promotions that move only microbench scores (Davies *et al.*, 2025).

3.12 Conclusion

This chapter grounded Chapter 2’s dataflow mindset in numbers: GPU shared memory and registers with launch overhead, CPU caches and NUMA, NPU tile SRAM and static spatial dataflow (NVIDIA, 2022; Goodfellow *et al.*, 2016; AMD, 2024b,a). Decode is bandwidth- and scheduling-bound on all three; hardware-aware tiling, layout, and pass order—guided by explicit models—turn diagnosis into shippable kernels (Chen, 2020; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a).

With the cross-class frame set, Chapter 4 descends into NVIDIA Hopper/Blackwell CUDA: registers, TMA, thread-block clusters, and tensor cores as concrete optimization bounds for decode megakernels (NVIDIA, 2022; SemiAnalysis, 2024). Bring your constraint-matrix row and export-edge worksheet from Chapters 1–2; Chapter 4 shows how to spend those bytes on silicon (Chen, 2026a).

Worksheet gate. Before opening Hopper-specific chapters, fill Table 3.3 for your deployment targets and note which bottleneck column dominates (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). If you cannot name launches/token

and bytes/token for your SKU, return to Chapter 1 before studying TMA details (NVIDIA, 2022; SemiAnalysis, 2024).

Hands-on next step. Pick one fused region in your decoder and compute shared-memory + register bytes statically; compare to the H100 row in Figure 3.2 (NVIDIA, 2022). If over budget, split stages per Chapter 2’s five-step playbook before tuning MMA configs (Chen, 2020; Chen and YiRage Contributors, 2026; Chen, 2026a).

Onboarding sentence for teams. Post in your compiler design doc: “*We do not merge fusion without a hardware-model legality check and decode-cell counters.*” (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a) Culture follows the checklist in CI, not the poster alone (Davies *et al.*, 2025).

Part I closure. Chapters 1–3 formed the measurement, mindset, and constraint foundation for everything that follows (Vellaisamy *et al.*, 2026; Chen, 2026a, 2020). You can now name what to optimize (bytes and launches), how to think (residency before exports), and what must fit (per-class capacities) (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Part II begins the GPU descent: same inequalities, more silicon detail, more concrete pass examples (NVIDIA, 2022; SemiAnalysis, 2024).

Glossary carry-forward. Terms introduced here recur throughout the book: **SIMT**, **warp**, **WGMMMA**, **TMA**, **NUMA**, **DMA**, and **spatial dataflow** (NVIDIA, 2022; AMD, 2024b; Goodfellow *et al.*, 2016). When later chapters use them without redefinition, refer back to this chapter’s first-use explanations (Chen and YiRage Contributors, 2026). Consistent vocabulary across GPU, CPU, and NPU sections prevents teams from talking past each other in design reviews (Davies *et al.*, 2025; Chen, 2020).

Chapter 4

CUDA Hopper/Blackwell: Hardware Properties and Optimization Bounds

4.1 Why Hopper Features Do Not Automatically Speed Up Decode

One production team spent a month raising tensor-core utilization on H100—end-to-end batch-1 decode latency fell only $\sim 3\%$. Nsight showed excellent TMA overlap and higher TC occupancy, but bytes/token and launches/token were unchanged: activations still exported to HBM between every op boundary (Chen, 2026a; Vellaisamy *et al.*, 2026). **Hopper features hide latency and widen on-chip capacity; they do not remove traffic you never fused away.**

Bridge from Chapters 2–3. Chapter 2 said plan residency before exports; Chapter 3 gave the byte budgets (Chen and YiRage Contributors, 2026; Chen, 2020). This chapter instantiates those budgets on Hopper: 228 KB shared memory, TMA, clusters, and tensor cores as *tools in a fixed optimization order*—not a shopping list (NVIDIA, 2022; SemiAnalysis, 2024).

Hopper decode optimization priority (use this order).

1. **Cut activation HBM traffic**—layer fusion (Chapter 2 export-edge mind-set); largest gain (Chen, 2026a; Dao *et al.*, 2022).

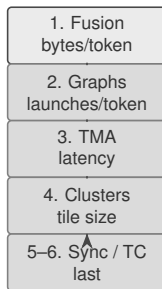


Figure 4.1: Hopper decode optimization priority pyramid: work top-down; do not invert.

Table 4.1: Hopper decode optimization priority and typical batch-1 gains (order matters).

Pri.	Direction	Typical gain	Section
1	Operator fusion (cut activation HBM)	40–80% bytes/token	§4.2
2	CUDA Graph + MegaKernel (cut launches)	30–60% scheduling overhead	§4.9
3	TMA double-buffer (hide latency)	15–30% memory stall	§4.3
4	Thread-block cluster + DSM	10–20% fusion depth	§4.4
5	Minimal sync scope	5–15% pipeline bubbles	§4.5
6	Tensor Core / PTX polish	<10% compute efficiency	§§4.6–4.7

2. **Cut kernel launch overhead**—CUDA Graph + MegaKernel; attacks launches/token (Vellaisamy *et al.*, 2026; NVIDIA, 2024).
3. **Hide memory latency**—TMA double-buffer; does not reduce bytes (Semi-Analysis, 2024).
4. **Widen fusion tiles**—thread-block clusters + DSM (NVIDIA, 2022).
5. **Minimize sync scope**—warp shuffle before block/cluster barriers (Dao *et al.*, 2022).
6. **Tune tensor cores and PTX last**—when bytes/token and launches/token are already clean (Davies *et al.*, 2025; Chen, 2026a).

Terminology alignment with Chapters 1–3. We use *kernel launch overhead* for per-token launch tax, *scheduling overhead* for host/framework orchestration, and *data residency* for on-chip lifetime of activation carriers (Vellaisamy *et al.*, 2026; Chen, 2020, 2026a). First-use expansions in this chapter: **DSM** (distributed shared memory across a thread-block cluster), **WGMMMA** (wargroup matrix

multiply-accumulate), **SFU** (special function unit for `exp2`), and **FTZ** (flush-to-zero mode for subnormal handling in softmax PTX) (NVIDIA, 2022; Liu *et al.*, 2024; Dao *et al.*, 2022).

Composite lesson. TaxBreak reports hundreds of launches per token on unfused stacks; Memory-Floor shows batch-1 decode often bandwidth- and orchestration-bound before tensor cores saturate (Vellaisamy *et al.*, 2026; Chen, 2026a). Profile launches/token and bytes/token first—the same discipline as Chapters 1–3 (Davies *et al.*, 2025).

What you will be able to do. After this chapter you can (1) place Hopper features on the priority pyramid, (2) write a shared-memory line-item budget for a fused layer, (3) decide TMA versus manual loads with profiler evidence, (4) attach ablation rows that separate graph wins from fusion wins, and (5) map YiRage invariants to review checklists (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a). PTX and WGMMMA micro-tuning are explicitly last—not because they are unimportant, but because they rarely move batch-1 decode when bytes/token is dirty (Davies *et al.*, 2025; Dao *et al.*, 2022).

Reader prerequisites. Carry Chapter 1 worksheets (launches/token, bytes/token, R_{floor}) and Chapter 3 H100 capacity rows (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026). When we reference online softmax, assume Chapter 10 numerics; here we only cover *where* carriers live on silicon (Dao *et al.*, 2022, 2023).

Industrial narrative. DeepSeek-scale training stories emphasize co-design under export limits (Liu *et al.*, 2024; Davies *et al.*, 2025). Hopper decode repeats the lesson at SM scale: skillful residency beats brute-force TFLOPs marketing (Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022). Goodput metrics make the case quantitatively—not peak GEMM banners (Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

How to read this chapter. Skim Figure 4.1 first (Davies *et al.*, 2025). Sections 4.2–4.5 are the tool layer; Section 4.9 is the workflow; Section 4.10 is verification only (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Anti-pattern preview. The five concentrated mistakes at the end of Section 4.9 map directly to inverted pyramid levels—use them in design review before merge (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026). Culture: post the pyramid in your compiler team’s Hopper doc template (Chen and YiRage Contributors, 2026).

Cross-chapter dependency map. Chapter 3 listed abstract GPU constraints; this chapter names Hopper mechanisms; Chapters 8–11 specialize TMA, sync, softmax, and MegaKernel codegen (SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026). Skipping here produces CUDA that ignores residency bounds (Vellaisamy *et al.*, 2026; Chen, 2026a).

Failure story catalog (short). Shared-memory overflow at max ctx; TMA misalignment on paged KV; illegal cluster on SKU; PTX exp2 drift at long T ; graph capture invalidated by driver upgrade (Chen and YiRage Contributors, 2026; NVIDIA, 2024; Dao *et al.*, 2023; Davies *et al.*, 2025). Map symptoms to pyramid level (Table 4.1) before tuning TC tiles (Chen, 2026a; Vellaisamy *et al.*, 2026).

TaxBreak anchor. Dense Llama-3.2-1B can average hundreds of kernel launches per output token on unfused H100 stacks (Vellaisamy *et al.*, 2026; Chen, 2026a). Those cells are why the pyramid starts with fusion, not TC (Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

What this chapter is not. Not a CUDA tutorial on square GEMM microbenchmarks; we document optimization *bounds* for batch-1 decode residency on Hopper-class silicon (NVIDIA, 2022; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a).

Organization map. Memory (§4.2), TMA (§4.3), clusters (§4.4), sync (§4.5), TC/PTX (§§4.6–4.7), YiRage (§4.8), practice (§4.9), verify (§4.10) (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

4.2 Hopper Memory Hierarchy: Fusion Hard Boundaries

Decode MegaKernels hold QKV tiles, online softmax carriers, and SwiGLU fragments simultaneously—each must map to a CUDA tier for the kernel lifetime (Dao

Table 4.2: CUDA memory spaces for decode megakernels (Hopper-class).

Space	Scope	Decode role
Register	Thread	Online softmax (m, ℓ, o), MMA fragments
Shared memory	CTA	Fused tiles, TMA staging
Global (HBM)	Grid	Weights, KV cache, spill

et al., 2022; Chen and YiRage Contributors, 2026). Spill to HBM is a bytes/token bill; treat it as a fusion failure (Chen, 2026a).

Terminology. A **CTA** (cooperative thread array) is a CUDA thread block—the launch grain that shares one shared-memory pool (NVIDIA, 2022). Hopper exposes up to 228 KB configurable shared memory per SM (227 KB usable per block after reserve) and a 65,536-register file (NVIDIA, 2022).

Shared memory capacity (constraint loop). **Engineering meaning:** hard ceiling for fused decode tiles—Chapter 3’s GPU row, now Hopper-specific (Chen and YiRage Contributors, 2026). **Common pitfall:** tensor bytes only, ignoring padding, double buffers, softmax score tiles (Dao *et al.*, 2022). **Debug:** Nsight Compute shared-memory config; elevated local memory traffic signals spill (Davies *et al.*, 2025).

Register pressure. Fusion deepens live ranges; occupancy may drop yet bytes/token still wins when HBM exports disappear (Chen, 2026a; SemiAnalysis, 2024). **Debug:** registers per thread and spill counters before MMA tile sweeps (Chen and YiRage Contributors, 2026).

KV versus activation traffic. Weights and KV legitimately stream from HBM each token—fusion targets *activation* exports between attention, softmax, and MLP (Kwon *et al.*, 2023; Chen, 2026a; Dao *et al.*, 2022). A common review mistake is celebrating lower DRAM traffic while KV alone explains the counter—diff activation bytes specifically (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Roofline at batch one. Decode GEMMs present thin M with large K/N ; arithmetic intensity stays on the memory roof for realistic LLM widths (Goodfellow *et al.*, 2016; Chen, 2026a). Registers and shared memory buy effective intensity

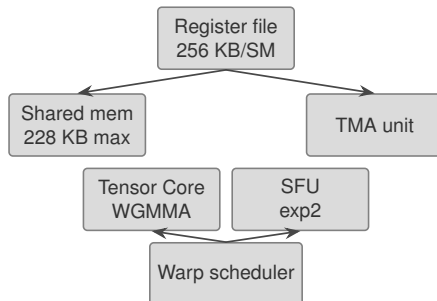


Figure 4.2: Hopper SM resource sketch for decode planning.

without changing FLOP counts—that is the mechanical-sympathy lever Hopper provides (NVIDIA, 2022; SemiAnalysis, 2024).

Blackwell note. Successor SKUs extend TMA and low-precision tensor paths; residency invariants in this section survive—only capacity rows and legality tables change in the chip model (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Cross-hardware note. GPU shared memory is software-managed—flexible but easy to overflow (NVIDIA, 2022). XDNA tile SRAM is fixed at compile time—inflexible but deterministic (Chapter 5) (AMD, 2024b,a). Same Chapter 2 residency goal; different legality mechanics (Chen, 2020).

✓ **Practice checkpoint.** Statically budget shared memory + registers for the full fused region; reject merges that exceed 227 KB usable before codegen (Chen and YiRage Contributors, 2026; Chen, 2026a).

Worked tier assignment exercise. For one transformer layer, list every $\Theta(d)$ tensor between attention and MLP: post-norm hidden, Q/K/V fragments, attention output, MLP intermediates, residuals (Dao *et al.*, 2022; Chen, 2026a). Mark each as register, shared, or global for your fused plan (Chen and YiRage Contributors, 2026; Chen, 2020). Any global mark between fused stages is a bytes/token bug unless proven dead (Vellaisamy *et al.*, 2026; Chen, 2026a).

Spill versus intentional HBM. Intentional traffic: weights, KV appends, final layer outputs to framework buffers (Kwon *et al.*, 2023; Goodfellow *et al.*,

2016). Spill traffic: softmax carriers, score tiles, or MLP fragments written because shared memory overflowed (Chen, 2026a; Dao *et al.*, 2022; Davies *et al.*, 2025). Nsight distinguishes them via local memory loads and stack-like access patterns (SemiAnalysis, 2024).

Occupancy versus fusion trade. High shared-memory configs lower resident blocks per SM yet may still win on ms/token when fusion deletes HBM passes (Chen, 2026a; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026). Optimize occupancy only after proving legality and bytes/token improvement (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Lab charts that show “low occupancy” without bytes/token context mislead reviewers (Chen, 2026a).

Example unfused traffic. Llama-class stacks often materialize six to eight $\Theta(d)$ activation buffers per layer at batch-1 (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Chen, 2026a). At $d=4096$ BF16, six round trips imply roughly $6 \times 4096 \times 2 \approx 49$ KB activation traffic per layer per token before KV reads (Kwon *et al.*, 2023; Chen, 2026a). Fusion targets that bill first (Chen, 2020; Chen and YiRage Contributors, 2026).

Shared-memory bank conflicts (constraint loop). **Engineering meaning:** legal capacity can still serialize when strided layouts hit the same 32-way bank set—Chapter 3’s GPU row, now layout-specific (NVIDIA, 2022; Shah *et al.*, 2024). **Common pitfall:** padding score tiles for capacity but leaving WGMMMA-unfriendly strides (Dao *et al.*, 2022). **Debug:** Nsight shared-memory conflict counters; compare swizzled versus naive layouts before blaming TMA (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Hopper fusion must swizzle for **WGMMMA** (warpgroup matrix multiply-accumulate) consumption, not only fit byte totals (SemiAnalysis, 2024; Shah *et al.*, 2024).

Chapter 2 iron rule link. Iron Rule 2 (keep activations on-chip across stages) is implemented here as shared-memory and register budgets (Chen, 2020, 2026a; Dao *et al.*, 2022). Violating the rule exports to global—bytes/token rises even if TMA overlap looks healthy (SemiAnalysis, 2024; Vellaisamy *et al.*, 2026).

L2 hidden export. Even legal shared-memory plans can lose if every stage writes full vectors through L2 to HBM (Chen, 2026a; Kwon *et al.*, 2023). Fusion must minimize round trips, not only fit in shared memory while exporting intermediates (Chen, 2020; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026).

Multi-hardware delta (again). CPU L3 blocking and XDNA static tiles solve the same export problem with different legality (AMD, 2024b; Goodfellow *et al.*, 2016; Rotem *et al.*, 2018; Davies *et al.*, 2025). Hopper’s programmable shared memory is powerful and easy to overflow—that is the engineering tension (NVIDIA, 2022; Chen and YiRage Contributors, 2026; Chen, 2026a).

Memory tier cheat sheet for reviewers. When reviewing CUDA diffs ask: where do QKV fragments live between ops? Where do (m, ℓ, o) live in the KV loop? Where does SwiGLU intermediate sit? (Dao *et al.*, 2022; Chen, 2026a; Chen and YiRage Contributors, 2026) Any answer of “global” between fused stages is a regression unless proven dead (Chen, 2020; Vellaisamy *et al.*, 2026).

4.3 Tensor Memory Accelerator: Hide Latency, Not Traffic

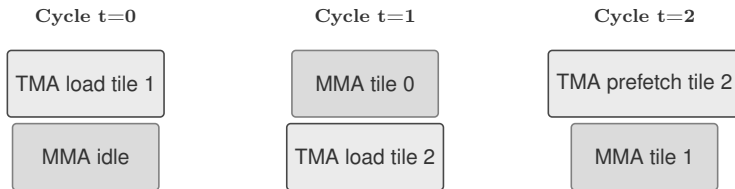
TMA (Tensor Memory Accelerator) issues asynchronous bulk copies from global memory to shared memory while MMA units compute on the previous tile (NVIDIA, 2022; SemiAnalysis, 2024). TMA overlaps latency; only fusion reduces activation bytes/token (Chen, 2026a).

Applicable when. Tile boxes 128×128 with contiguous global addresses and amortized descriptor setup—weight blocks, contiguous KV slices, steady double-buffer pipelines inside a MegaKernel (SemiAnalysis, 2024; Dao *et al.*, 2023).

Avoid when. Batch-1 $M=1$ micro-tiles, irregular paged-KV gathers, or descriptor setup dominating copy time (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Profile TMA versus vector loads on the target shape (Chen and YiRage Contributors, 2026).

Common pitfalls. Double-buffer handoff bugs (compute on buffer being overwritten); paged KV non-contiguous addresses; mandating TMA on thin tiles (Dao *et al.*, 2023; Davies *et al.*, 2025).

Debug. Nsight Compute: TMA busy cycles vs MMA busy cycles—ratio below ~ 0.3 suggests weak TMA benefit; check local memory loads for spill (SemiAnalysis, 2024).



Steady state: copy and math overlap across buffer slots

Figure 4.3: TMA double-buffer timeline: async copy overlaps tensor-core math.

Paged KV co-design. vLLM block tables map logical tokens to non-contiguous physical blocks (Kwon *et al.*, 2023). TMA descriptors assume regular layouts—MegaKernels rebuild or index descriptors per block inside the KV loop (Dao *et al.*, 2023; Chen and YiRage Contributors, 2026). This is production-hard; Chapter 8 returns to double-buffer theory (SemiAnalysis, 2024).

Manual load fallback. Vectorized global loads remain valid when TMA setup dominates—YiRage should lower both paths and select with a profile-guided cost model, not a global compile flag (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Teams that mandate TMA in code review without shapes invite the anti-pattern in Section 4.9 (Davies *et al.*, 2025).

Batch-1 thin- M reality. Many decode GEMMs present $M=1$ with large inner dimension (Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026). TMA shines when copy boxes amortize over deep pipelines inside a MegaKernel—not when each score fragment is a separate descriptor (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026). Profile on *your* head dimension and KV block size, not a square GEMM microbench (Davies *et al.*, 2025; Chen, 2026a).

Energy angle. HBM transactions dominate decode energy (Chen, 2026a; Davies *et al.*, 2025). Fusion bytes/token savings matter for TCO even when ms/token gains look modest in lab cells (Vellaisamy *et al.*, 2026; AMD, 2024b). TMA overlap saves time; fusion saves joules per token (SemiAnalysis, 2024; Chen, 2026a).

Synchronization case study. Double-buffered TMA needs one barrier after copy completes and one before overwriting the consumed buffer (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; NVIDIA, 2022). Extra barriers after each warp shuffle in softmax add pipeline bubbles without safety benefit (Dao *et al.*,

2022; Goodfellow *et al.*, 2016). Compiler sync insertion should be edge-driven on the fused DAG (Chen, 2020; Chen and YiRage Contributors, 2026).

Long-context stability. TMA misalignment bugs may hide at $T=512$ and explode at $T=8192$ (Dao *et al.*, 2023; Davies *et al.*, 2025). Always include long-ctx buckets in Hopper validation—not only the bucket your laptop tests (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

Observability hooks. Stable kernel names derived from pass hashes help profilers map samples to bufferization tiers (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025). Production builds may strip debug tags but should retain residency class notes in release artifacts (Vellaisamy *et al.*, 2026; Chen, 2026a).

Warp-group context. Hopper issues WGMMA from warpgroups of four warps—thirty-two threads each (Shah *et al.*, 2024; NVIDIA, 2022). Fusion planners must align head partitions and tile shapes to warpgroup granularity or leave MMA lanes idle (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

First-token versus steady state. Compile and capture costs hit first token; steady decode steps reveal residency wins (NVIDIA, 2024; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Benchmark both when product SLOs include cold start (Davies *et al.*, 2025; Chen, 2026a).

Descriptor lifecycle. Host or device code builds a TMA descriptor with global layout, shared destination, and copy box (NVIDIA, 2022; SemiAnalysis, 2024). Weight descriptors are often static per compiled bucket; KV descriptors may change with block-table indices each iteration (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026). Misaligned layouts produce silent numerical drift—always pair kernel changes with FP64 reference diffs at long ctx (Dao *et al.*, 2023).

Figure 4.3 usage. The timeline is the steady-state target—getting there requires correct double-buffer handoffs and legal tile sizes (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022). If steady state never appears in timelines, fix fusion legality before tuning descriptor fields (Davies *et al.*, 2025; Chen, 2026a).

Chapter 2 link. TMA implements *pipeline movement* on-chip—moving data while math runs—but does not replace export-edge removal (Chen, 2020, 2026a).

Cluster versus single-block decision tree. If fused tile fits in 227 KB usable with margin, stay single-block (NVIDIA, 2022; Chen and YiRage Contributors, 2026). If score tile or SwiGLU buffer exceeds block cap but fits pooled DSM, plan cluster (SemiAnalysis, 2024; Dao *et al.*, 2023). If neither fits, split fusion at Chapter 2 export edge—do not spill (Chen, 2026a, 2020; Vellaisamy *et al.*, 2026).

Flash-Decoding without extra launches. KV splits across SMs need in-kernel online softmax merges (Dao *et al.*, 2023, 2022). Extra merge kernels reopen launches/token—design cluster or in-kernel merge before accepting multi-kernel fallback (Vellaisamy *et al.*, 2026; NVIDIA, 2024; Chen and YiRage Contributors, 2026).

✓ **Practice checkpoint.** Confirm double-buffer invariants in code review; one barrier after TMA complete, one before buffer reuse (Chen and YiRage Contributors, 2026; SemiAnalysis, 2024).

4.4 Thread Block Clusters and Distributed Shared Memory

Hopper **thread-block clusters** pool **DSM** (distributed shared memory) across cooperating CTAs—widening the on-chip fusion envelope without an HBM export (NVIDIA, 2022; SemiAnalysis, 2024).

Applicable when. Score tiles or fused buffers exceed single-block shared memory; Flash-Decoding-style KV splits need cross-block softmax reductions (Dao *et al.*, 2023, 2022).

Avoid when. Footprint already fits one block—clusters add sync and launch complexity without residency win (Chen and YiRage Contributors, 2026).

Common pitfalls. Cluster launch illegal on target SKU; DSM double-counted in planners; fallback merge kernel reintroduces launches/token (Vellaisamy *et al.*, 2026; Chen, 2020).

Table 4.3: Cluster DSM vs. single-block shared memory (decode).

Aspect	Single CTA	Cluster + DSM
Peak scratch	One block SMEM	Pooled DSM
Softmax max-reduce	Warp/block	Cross-CTA on-chip
XDNA analogue	One tile SRAM	Multi-tile DMA graph

Debug. Compile-time cluster legality report; Nsight timeline for extra barriers; compare launches/token with single-block plan (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

✓ **Practice checkpoint.** YiRage must reject illegal cluster dims at compile time—no silent multi-kernel fallback (Chen and YiRage Contributors, 2026; Chen, 2020).

Flash-Decoding link. When KV is split across SMs, partial softmax statistics must merge on-chip (Dao *et al.*, 2023, 2022). An extra merge kernel reopens launches/token—cluster DSM exists to keep the merge inside the MegaKernel capture region (Vellaisamy *et al.*, 2026; NVIDIA, 2024). If your SKU forbids the cluster shape, document the multi-kernel fallback cost explicitly in the worksheet (Davies *et al.*, 2025).

Capacity accounting with DSM. Cluster pooling increases peak scratch but also increases barrier scope (NVIDIA, 2022; SemiAnalysis, 2024). Planners must sum DSM bytes across CTAs in the cluster—double-booking is as fatal as single-block overflow (Chen and YiRage Contributors, 2026; Chen, 2026a). Chapter 3’s inequality mindset applies; only the geometry changes (Chen, 2020).

Comparison to CPU AVX paths. CPU decode uses cache-blocked GEMMs and vectorized exp2 on softmax (Goodfellow *et al.*, 2016; Davies *et al.*, 2025; AMD, 2024b). Hopper wins when fusion keeps data on SM; CPU wins when GPU launch storms dominate at low QPS (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026). Triplet tests prevent declaring victory on GPU ms/token alone (AMD, 2024a; Davies *et al.*, 2025).

Grid sync warning. Device-wide synchronization stalls every CTA and destroys graph-capture friendliness (Vellaisamy *et al.*, 2026; NVIDIA, 2024, 2022). If an

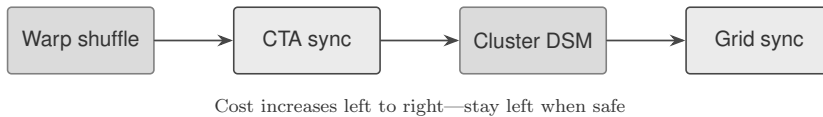


Figure 4.4: CUDA synchronization hierarchy for decoder megakernels.

algorithm seems to need grid sync per token, revisit fusion boundaries or cluster merges first (Dao *et al.*, 2023; Chen and YiRage Contributors, 2026).

Producer-consumer sync edges. List edges in the fused DAG where shared-memory buffers hand off between TMA, MMA, and epilogue phases (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024). Insert barriers only on those edges—extra block syncs show as bubbles in Nsight timelines (Davies *et al.*, 2025; Dao *et al.*, 2022).

Figure 4.4 usage. Pick the leftmost mechanism that preserves correctness—grid sync is almost never the answer in per-token decode (Vellaisamy *et al.*, 2026; NVIDIA, 2022; Dao *et al.*, 2022). Chapter 9 expands sync insertion algorithms (Chen and YiRage Contributors, 2026; Chen, 2020).

4.5 Synchronization: Narrowest Scope Wins

Unnecessary barriers serialize TMA pipelines (SemiAnalysis, 2024). Place sync only on shared-memory phase edges in the fused dataflow graph (Chen and YiRage Contributors, 2026; Chen, 2020).

Hierarchy (cheapest first). **Warp shuffle** for row-wise softmax max/sum (Dao *et al.*, 2022). **CTA block sync** for TMA/MMA buffer handoffs (SemiAnalysis, 2024). **Cluster barrier** for DSM reductions (NVIDIA, 2022). **Grid sync**—avoid in per-token decode loops (Vellaisamy *et al.*, 2026).

✓ **Practice checkpoint.** Audit fused kernels for barriers after every shuffle—remove bubbles without safety benefit (Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

CUDA Graphs interaction. Graph capture removes host launch overhead but does not shrink shared-memory footprints (NVIDIA, 2024; Chen, 2026a). Capture

Table 4.4: Decode diagnostics on Hopper (what to fix first).

Signal	Limiter	First action
High launches/token	Scheduling overhead	Graph / MegaKernel
High bytes/token	HBM residency	Fusion
High TC util, flat ms/token	Wrong metric	Return to bytes/token

after fusion is legal; capture instead of fusion is not (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Systems timelines should show fewer host packets *and* lower DRAM bytes per token when both levers apply (Davies *et al.*, 2025).

Warp specialization preview. Persistent decode kernels assign warps to TMA issue, MMA compute, and epilogue roles (SemiAnalysis, 2024; NVIDIA, 2022; Vellaisamy *et al.*, 2026). Chapter 11 MegaKernels assume this model (Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026). Sync placement must respect role boundaries—a barrier in the wrong warp group stalls the whole pipeline (Davies *et al.*, 2025).

4.6 Tensor Core and PTX: Optimize Last

At batch-1, MMA tiles are thin—tensor cores often sit idle while DRAM and launches dominate (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). **Do not open with TC tile sweeps** when R_{floor} is low (Chapter 1) (Chen, 2026a).

Quantitative intuition. Removing six d -vector HBM round trips at $d=4096$ BF16 saves ~ 49 KB activation traffic per layer per token—often larger than a 10-point TC utilization gain on 1×4096 GEMM (Chen, 2026a; Vellaisamy *et al.*, 2026). After fusion cleans bytes/token, TMA-fed MMA epilogues matter (SemiAnalysis, 2024; Dao *et al.*, 2022).

SFU versus MMA balance. Attention decode interleaves WGMMA matmuls with SFU-heavy softmax (Dao *et al.*, 2022; NVIDIA, 2022). Profiling both pipelines prevents false “TC-bound” diagnoses when softmax is the long pole inside the same SM (Davies *et al.*, 2025; Liu *et al.*, 2024). Refer back to Section 4.6 before opening tile configurators (Chen, 2026a).

Quantization on Hopper. FP8 and INT8 weights need dequant in-register or in shared memory (Liu *et al.*, 2024; Davies *et al.*, 2025; NVIDIA, 2022). Scales consume the same residency budget as softmax carriers—co-decide with fusion grouping (Chen, 2026a; Chen and YiRage Contributors, 2026). Dropping precision without retile can change padding and spill behavior (Chen, 2020).

Blackwell FP4 trajectory. Successor chips advertise wider low-precision tensor paths (Liu *et al.*, 2024; NVIDIA, 2022; Davies *et al.*, 2025). Porting re-tiles kernels but preserves the pyramid: bytes, launches, latency, TC (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024).

Team A versus Team B (retold). Team A improves MMA tile from 1% to 3% TC util; Team B fuses an MLP boundary saving $\sim 40\%$ activation bytes—only Team B moves p99 at batch-1 when R_{floor} is low (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022). Use this story in backlog prioritization (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Table 4.4 usage. Match profiler signals to the “first action” column before opening code (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026). High TC util with flat ms/token is a diagnostic row, not a success row (Chen, 2026a; Dao *et al.*, 2022).

PTX review culture. Softmax-related PTX changes are not “micro-optimizations”—they are ABI changes (Dao *et al.*, 2022; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Treat them with the same rigor as fusion grouping changes (Chen, 2020; Dao *et al.*, 2023).

4.7 PTX and Softmax Micro-ops

Online softmax uses **SFU** (special-function unit) pipelines for **exp2**-style exponentials (Dao *et al.*, 2022; NVIDIA, 2022). **WGMMMA** (warp-group matrix multiply-accumulate) is Hopper’s tensor-core instruction issued by warpgroups of four warps (Shah *et al.*, 2024).

PTX as ABI. Reviewed **ex2.approx** sequences with explicit max-subtraction and documented **FTZ** (flush-to-zero) behavior (Liu *et al.*, 2024; Davies *et al.*, 2025).

Changing PTX without FP64 reference diffs at $T=8192$ breaks fleet determinism (Dao *et al.*, 2023; Chen and YiRage Contributors, 2026).

Advance notice. This subsection needs basic CUDA/PTX literacy; skipping it does not block fusion or TMA work above (Goodfellow *et al.*, 2016).

✓ **Practice checkpoint.** Softmax PTX changes require FP64 diffs at $\text{ctx} \in \{512, 2048, 8192\}$ before merge (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026).

Inline PTX in production. DeepSeek-style kernels select SFU instructions predictably rather than hoping the compiler discovers them (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026). Separate softmax launches break MegaKernel boundaries and reopen launches/token (Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022). PTX belongs inside the fused region, not as a follow-on kernel (Chen and YiRage Contributors, 2026; Chen, 2020).

4.8 YiRage CUDA Backend: Hardware Constraints as Compile Gates

Advance notice. This section assumes compiler and IR literacy; backend engineers should read it end-to-end, while kernel tuners can skim invariants and return for lowering detail (Chen, 2020; Chen and YiRage Contributors, 2026).

YiRage lowers the same decode IR to CUDA, CPU, and XDNA—Hopper legality is dynamic residency checking against chip models from Chapter 3 (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b).

Five compiler invariants (decode mode).

1. **No activation-sized global stores** inside a fused region—*YiRage*: `BufferizationPass` rejects illegal stores at compile time instead of emitting silent spill (Chen, 2026a; Chen and YiRage Contributors, 2026).
2. **TMA legality is per-copy**—*YiRage*: `TmaSelectionPass` chooses TMA or vector loads per copy edge (SemiAnalysis, 2024; Vellaisamy *et al.*, 2026).

Table 4.5: YiRage Hopper lowering: IR in, check out.

Pass	Input $\rightarrow$ action	Output check
Bufferization	Linalg/tensor IR $\rightarrow$ tier assignment vs. chip model	No illegal global activations
Fusion grouping	Bufferized graph $\rightarrow$ export-edge merges	Shared/reg high-water $\leq$ caps
TMA selection	Copy edges $\rightarrow$ descriptor or vector path	Per-edge legality report
Cluster planning	Oversized tiles $\rightarrow$ DSM cluster shape	Compile fail if illegal
Sync insertion	Fused DAG $\rightarrow$ minimal barriers	No grid sync per token
PTX lowering	Softmax region $\rightarrow$ SFU exp2 ABI	FP64 diff at long ctx

3. **Cluster dims are compile-time contracts**—*YiRage*: `ClusterPlanningPass` fails the build on illegal DSM geometry (NVIDIA, 2022; Chen and YiRage Contributors, 2026).
4. **Softmax PTX is versioned ABI**—*YiRage*: `PtxLoweringPass` pins reviewed exp2 sequences; CI diffs long-ctx numerics (Dao *et al.*, 2022; Liu *et al.*, 2024).
5. **Triplet bufferization**—*YiRage*: `FusionGroupingPass` must export legality summaries for CUDA, CPU, and XDNA backends (AMD, 2024a; Davies *et al.*, 2025).

Invariant enforcement in CI. Decode-mode builds treat activation-sized global stores as hard errors; TMA and cluster passes emit legality reports consumed by nightly triplet jobs (Chen and YiRage Contributors, 2026; AMD, 2024b; Chen, 2020). Hand-tuned CUDA that bypasses the model rots on the next SKU stepping—keep capacities in the Chapter 3 chip model (Davies *et al.*, 2025; IREE / iree-org contributors, 2024).

Handoff preview. Chapter 5 shows the same IR on XDNA: static tile slots replace dynamic shared memory (AMD, 2024b,a). Triplet parity is not optional

for a multi-hardware compiler narrative (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Documentation deliverables per promotion. Residency diagram, TMA descriptor notes, cluster dims, sync plan, PTX softmax changelog, TaxBreak launches/token cell, Memory-Floor bytes/token cell (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Partners cannot replay regressions when benchmarks live only in private spreadsheets (Chen, 2020).

Testing gates. Numerical diffs at long ctx; Nsight DRAM delta at batch-1; triplet compile; graph bucket hit rate across three ctx lengths (Dao *et al.*, 2023; NVIDIA, 2024; AMD, 2024b; Chen and YiRage Contributors, 2026).

Lowering walkthrough detail. Each row in Table 4.5 must emit artifacts CI can diff—legality reports are not optional debug prints (Chen and YiRage Contributors, 2026; Chen, 2020).

CUDA Graphs and bufferization. Graph capture pins loop bounds and launch structure (NVIDIA, 2024; Kwon *et al.*, 2023). Bufferization must allocate for max ctx in the captured bucket—growing block tables without re-capture is an open product tension (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Liu *et al.*, 2024).

IREE-style portability. Modular retargeting treats backends as plugins over shared IR (IREE / iree-org contributors, 2024; Chen and YiRage Contributors, 2026; Chen, 2020). Hopper capacities live in the plugin model, not scattered in hand-written kernels (Davies *et al.*, 2025; NVIDIA, 2022).

Silent spill signature. Flat GPU utilization with rising HBM throughput often means spill or export edges (Chen, 2026a; NVIDIA, 2022; Davies *et al.*, 2025). Map telemetry to pyramid level before opening TC configurators (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

✓ **Practice checkpoint.** Hopper PRs attach residency diagram + legality report; triplet export compiles or documents deferral (Chen and YiRage Contributors, 2026; AMD, 2024b).

4.9 Hopper Decode Tuning in Practice

This section consolidates profiling workflow, reusable patterns, a quantified case study, and concentrated anti-patterns—the material formerly scattered across profiling, patterns, and compiler-reference sections (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

4.9.1 Profile-first workflow

Start from median ms/token at BS=1; decompose launches/token (TaxBreak) and bytes/token (Memory-Floor / Nsight DRAM) (Vellaisamy *et al.*, 2026; Chen, 2026a). Nsight Systems for launch storms and graph replay; Nsight Compute for spill, DRAM bytes, TMA vs MMA busy cycles (Davies *et al.*, 2025; SemiAnalysis, 2024). **Order:** end-to-end token latency → bytes → launches → TC utilization (Chen, 2026a).

Systems versus Compute. Systems answers “how many launches and how much host noise”; Compute answers “did we spill and did DRAM bytes drop” (Davies *et al.*, 2025; SemiAnalysis, 2024; NVIDIA, 2024). A fused kernel can lose isolated kernel timers yet win end-to-end ms/token when launch storms disappear (Vellaisamy *et al.*, 2026; Chen, 2026a). Never promote based on kernel-only timers alone (Davies *et al.*, 2025).

Register carriers for online softmax. Each head keeps (m, ℓ) scalars and a partial output vector—often FP32 even when matmul uses BF16 (Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Sixteen heads multiply register pressure; warp-specialized layouts assign warpgroups per head cluster to keep shuffle reductions in-register (NVIDIA, 2022; Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). This is why the shared-memory sheet and register sheet are one joint budget (Chen, 2026a).

Postmortem sequence. When Hopper decode regresses after a merge: diff launches/token, replay bytes/token worksheet, compare DRAM delta on fused kernel, bisect graph vs fusion, *then* consider driver or clock issues (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; NVIDIA, 2024). Skipping bytes/token replay wastes days on launch-only noise (Chen and YiRage Contributors, 2026).

Teaching versus production CUDA. Courses emphasize square GEMM microbenchmarks; production decode emphasizes residency graphs and goodput counters (Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a). This chapter bridges that gap for Hopper specifically (SemiAnalysis, 2024; NVIDIA, 2022).

Roadmap to later chapters. Chapter 8 specializes TMA double-buffer theory; Chapter 9 sync hierarchy; Chapter 10 softmax carriers; Chapter 11 full MegaKernel assembly (Dao *et al.*, 2022, 2023; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Hopper bounds here are prerequisites—later chapters reference these labels rather than re-derive capacities (NVIDIA, 2022; SemiAnalysis, 2024).

Open research threads. Dynamic shapes without re-capture, multi-GPU tensor-parallel MegaKernels, and MoE expert-local fusion remain active (Liu *et al.*, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025). This chapter documents single-GPU batch-1 baselines engineers can extend (Vellaisamy *et al.*, 2026; Chen, 2026a).

Reader exercises. (1) Sketch shared-memory allocation for one fused layer at $d=2048$. (2) Explain when TMA hurts batch-1 decode. (3) Fill Table 4.6 for your model (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; SemiAnalysis, 2024).

Industrial narrative (closing practice section). Goodput engineering on Hopper is measurable: launches/token, bytes/token, ms/token together (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Skillful residency beats SKU shopping when R_{floor} is low (Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026).

Pattern usage in review. Require Pattern 1 skeleton diagram on every layer-fusion PR; Pattern 5 ablation on every graph PR; Pattern 6 on every IR change touching bufferization (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025).

Documentation contract (reprise). Ship residency diagram, TMA notes, cluster dims, sync plan, softmax PTX changelog, and goodput cells with each promotion (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; SemiAnalysis, 2024).

Energy and TCO (short). Fusion bytes/token savings reduce HBM energy even when ms/token gains look modest (Chen, 2026a; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; AMD, 2024b). Cloud TCO cares about joules per useful token—not peak TFLOPs (Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Operator documentation. Version benchmark artifacts in artifact stores partners can replay (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a). Private spreadsheets do not scale across fleets (Davies *et al.*, 2025).

Quantization-aware benchmarks (short). INT8 and FP8 paths need explicit scale tensors in exports so fusion legality does not fold away dequant boundaries (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2026a). Include per-channel scales typical of long-context decode in nightly cells (Davies *et al.*, 2025; Kwon *et al.*, 2023).

4.9.2 Engineering patterns

Pattern 1: Fused layer skeleton. **Scene:** single-layer decode on Hopper (Vellaisamy *et al.*, 2026). **Solution:** TMA load $\rightarrow$ QKV MMA $\rightarrow$ KV loop with online softmax $\rightarrow$ SwiGLU $\rightarrow$ residual—no activation HBM stores (Dao *et al.*, 2022; SemiAnalysis, 2024). **Benefit:** ~ 6 –8 HBM round trips removed; launches drop from ~ 12 to 1 per layer (Chen, 2026a). **Notes:** pre-compute shared-memory + register budget (Section 4.2) (Chen and YiRage Contributors, 2026).

Pattern 2: Context buckets + CUDA Graphs. **Scene:** production ctx $\in \{512, 2048, 8192\}$ (Kwon *et al.*, 2023; NVIDIA, 2024). **Solution:** separate captured graphs per bucket; static loop bounds inside MegaKernel (Chen and YiRage Contributors, 2026). **Benefit:** launches/token collapse without changing math (Vellaisamy *et al.*, 2026). **Notes:** graphs without fusion still pay full bytes/token (Chen, 2026a).

Pattern 3: Warp-specialized TMA. **Scene:** KV loop memory-bound (SemiAnalysis, 2024). **Solution:** dedicated warps issue TMA while MMA warps consume previous tile (NVIDIA, 2022). **Benefit:** raises TMA/MMA overlap ratio (Davies *et al.*, 2025). **Notes:** misbalanced roles stall the pipeline (Chen and YiRage Contributors, 2026).

Table 4.6: Llama-3.2-1B decode layer: operator-first vs Hopper fusion (H100, batch-1, ctx=2048).

Metric	Operator-first	Hopper fusion	Change
Launches/token (layer)	12	1	−91.7%
Activation HBM bytes/token	~49 KB	~8 KB	−83.7%
Median ms/token (layer)	42 μ s	19 μ s	−54.8%
Tensor Core utilization	12%	15%	+25% (minor)

Pattern 4: Softmax on shuffle + SFU. **Scene:** attention decode (Dao *et al.*, 2022). **Solution:** max via warp shuffle; **exp2** on SFU; carriers in FP32 (Goodfellow *et al.*, 2016; Liu *et al.*, 2024). **Benefit:** avoids extra global softmax materialization (Chen, 2026a). **Notes:** Chapter 10 owns numerics; this chapter owns placement (Dao *et al.*, 2022).

Pattern 5: Graph + fusion composition. **Scene:** H100 cell with low R_{floor} (Chen, 2026a). **Solution:** fuse first, capture graph second (NVIDIA, 2024; Vellaisamy *et al.*, 2026). **Benefit:** bytes and launches both move (Davies *et al.*, 2025). **Notes:** ablation rows must label which lever moved (Chen, 2026a).

Pattern 7: PagedAttention inside MegaKernel. **Scene:** production serving with vLLM block tables (Kwon *et al.*, 2023). **Solution:** device-side block index walk; gather or per-block TMA inside KV loop (Dao *et al.*, 2023; Chen and YiRage Contributors, 2026). **Benefit:** keeps paging without per-op launches (Vellaisamy *et al.*, 2026). **Notes:** hardest Hopper integration point—profile at 8K and 32K ctx (Davies *et al.*, 2025).

Pattern 6: Triplet parity. **Scene:** fleet with GPU + NPU endpoints (AMD, 2024b; Davies *et al.*, 2025). **Solution:** same IR, three backends; compare normalized bytes/token (Chen and YiRage Contributors, 2026). **Benefit:** prevents GPU-only schedule drift (Chen, 2020).

4.9.3 Worked example: Llama-3.2-1B on H100

Setup: batch-1, ctx=2048, hidden d =2048, 16 Q heads / 4 KV heads (GQA) (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).

Table 4.7: Shared-memory budget for Llama-3.2-1B fused layer (H100, $d=2048$, $ctx=2048$).

Buffer	Tier	Bytes	Notes
Hidden staging	Shared	8 KB	RMSNorm input tile
QKV double buffer	Shared	48 KB	TMA ping-pong
Softmax scores	Shared	32 KB	One score tile (no duplicate)
SwiGLU tiles	Shared	64 KB	Up/down projections
Epilogue / residual	Shared	16 KB	In-place when aligned
Subtotal		168 KB	of 227 KB usable
TMA descriptor scratch	Shared	~8 KB	Headroom margin

Table 4.8: Ablation: graph capture vs. fusion vs. composition (same H100 cell).

Configuration	Launches/token	Bytes/token	Layer ms/token
Baseline (eager, unfused)	12	~49 KB	42 μ s
Graph capture only	1	~49 KB	35 μ s
Fusion only (no graph)	12	~8 KB	24 μ s
Graph + fusion (shipped)	1	~8 KB	19 μ s

Reading the ablation rows. Row 2 proves graph capture attacks scheduling overhead only—launches/token drops from 12 to 1 while bytes/token stays at ~49 KB (NVIDIA, 2024; Vellaisamy *et al.*, 2026). Row 3 proves fusion attacks activation traffic—bytes fall to ~8 KB but launches remain at 12 without capture (Chen, 2026a; Chen and YiRage Contributors, 2026). Row 4 is the shipped configuration: both levers move, yielding the full ~55% layer latency win in Table 4.6 (Davies *et al.*, 2025). Reviewers should reject PRs that cite only one row without labeling which pyramid level moved (Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a). Adding a convenience buffer beyond Table 4.7 triggers spill or occupancy collapse—recompute the sheet before merge (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022).

Profile interpretation for the case. After fusion, Systems should show one launch per layer; Compute should show lower DRAM throughput with similar MMA instruction counts (Vellaisamy *et al.*, 2026; NVIDIA, 2024; Davies *et al.*, 2025). If TC utilization rises but Table 4.6 ms/token is flat, the team optimized the wrong row (Chen, 2026a). Contrast with L4 and XDNA exports of the same IR to keep fleet decisions honest (AMD, 2024b; Chen, 2026a; Chen and YiRage Contributors, 2026).

Unfused baseline narrative. Operator-first stacks pay TaxBreak launch counts *and* Memory-Floor activation bytes (Vellaisamy *et al.*, 2026; Chen, 2026a). Hopper fusion attacks both; graphs alone attack only launches (NVIDIA, 2024; Davies *et al.*, 2025). Table 4.6 is the template for your model row—replace numbers, keep columns (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Review question. A teammate proposes TMA on every 1×128 score fragment but still stores attention output to global before MLP (Chen, 2026a; Dao *et al.*, 2022). Ask: what happens to bytes/token and Chapter 10 carrier residency? (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022) Correct answer: bytes stay high; MegaKernel legality fails despite pretty TMA timelines (Chen, 2026a; Davies *et al.*, 2025).

KV loop structure. For each KV block: TMA K/V tiles, MMA $QK^\top$, online softmax update, MMA PV accumulate (Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023). The loop body must stay inside the same capture region as QKV and MLP—exporting partial attention between blocks reopens bytes/token (Chen, 2026a; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Residual update placement. Final residual can be in-place in shared memory when dimensions align (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). A global residual write mid-layer is another $\Theta(d)$ export—flag in review (Chen, 2026a, 2020).

GQA head accounting. Grouped-query attention reduces KV heads versus Q heads—KV loop traffic scales with KV head count, not Q count (Kwon *et al.*, 2023; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Worked example uses $16 \text{ Q} / 4 \text{ KV}$; adjust line items for your architecture (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Figure 4.5 usage. Use in design reviews to explain why TMA alone cannot fix operator-first graphs (SemiAnalysis, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026). The diagram is the export-edge story from Chapter 2 drawn for one layer (Chen, 2020; Dao *et al.*, 2022).

Anti-pattern review cadence. Run the five anti-patterns as a checklist in weekly Hopper office hours—not only at merge time (Davies *et al.*, 2025; Chen,

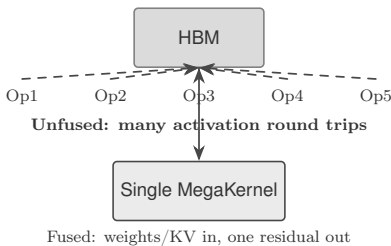


Figure 4.5: Fusion removes activation HBM ping-pong; TMA does not.

2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Teams that only check at release discover spill in production (Chen, 2026a; NVIDIA, 2022).

Ablation discipline. Publish graph-only, fusion-only, and composed rows—label which metric moved (Chen, 2026a; NVIDIA, 2024; Vellaisamy *et al.*, 2026).

4.9.4 Five common Hopper decode anti-patterns

1. **TMA everywhere** on $M=1$ micro-tiles. *Symptom:* TMA/MMA busy ratio < 0.3 with flat ms/token (SemiAnalysis, 2024). *Debug:* compare vector-load path; reject tiles below $\sim 128 \times 128$ (Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).
2. **TC utilization chasing** while bytes/token flat. *Symptom:* TC utilization rises, Table 4.6 ms/token unchanged (Chen, 2026a). *Debug:* Nsight DRAM bytes first; return to pyramid level 1 (Davies *et al.*, 2025).
3. **Occupancy tuning** that accepts register spill. *Symptom:* high occupancy, rising local memory loads (Chen and YiRage Contributors, 2026). *Debug:* spill counters versus Table 4.7 high-water (Chen, 2026a).
4. **Graph capture without fusion.** *Symptom:* launches/token collapse, bytes/token flat (Table 4.8, row 2) (NVIDIA, 2024). *Debug:* ablation row must show bytes moving, not only launches (Chen, 2026a).
5. **Fusion without capacity sheet.** *Symptom:* kernel runs, shared high-water exceeds 227 KB usable, silent spill (NVIDIA, 2022). *Debug:* static Table 4.7; treat spill as compile failure (Chen and YiRage Contributors, 2026; Chen, 2026a).

Table 4.9: Nsight Compute priorities for Hopper decode MegaKernels.

Counter / metric	Interpretation
DRAM throughput	Activation + KV traffic
Local memory loads	Spill—fusion bug
TMA busy / MMA busy	Pipeline balance
TC utilization	Tune only after bytes clean

4.9.5 Nsight counter cheat sheet

Prioritize: DRAM bytes per kernel $\rightarrow$ spill/local loads $\rightarrow$ TMA vs MMA busy $\rightarrow$ TC utilization last (Davies *et al.*, 2025; SemiAnalysis, 2024; Chen, 2026a).

✓ **Practice checkpoint.** Attach Table 4.6-style before/after and shared-memory sheet to every Hopper fusion PR (Vellaisamy *et al.*, 2026; Chen, 2026a).

Efficiency metric (optional fleet view). Track measured tokens/s divided by roofline tokens/s after subtracting framework scheduler overhead (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026). CUDA backend teams own fused-kernel numerator; framework teams own scheduler tax—publish both (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023).

4.10 Hopper Benchmarking and Verification

Chapter 1–2 define decode-cell metrics; this section adds *Hopper-specific* counters only (Vellaisamy *et al.*, 2026; Chen, 2026a). Full CI checklists live in Appendix B (§B)—not repeated here (Chen and YiRage Contributors, 2026).

Four verification classes. (1) **Core metrics:** launches/token, bytes/token, ms/token at $\text{ctx} \in \{512, 2048, 8192\}$, BS=1 (Vellaisamy *et al.*, 2026; Chen, 2026a). (2) **Environment:** pinned model weights, driver/CUDA versions, warmup including graph capture (NVIDIA, 2024; Davies *et al.*, 2025). (3) **Hopper extensions:** TMA transaction count, shared-memory high-water, graph bucket hit rate (SemiAnalysis, 2024; NVIDIA, 2022). (4) **CI gate:** ablation matrix (graph / fusion / both) + triplet legality summary (Chen and YiRage Contributors, 2026; AMD, 2024b).

Table 4.10: Hopper decode counter worksheet (extends Chapter 1).

Metric	Hopper-specific meaning
Launches/token	Graph replay vs eager
Bytes/token	Activation + KV + spill
TMA edges/token	Async copy orchestration
Shared high-water	Fusion budget violations
TC utilization	Often misleading at batch-1

Paged KV stress. Benchmark with vLLM block tables when product ships paging—TMA legality at 32K may differ from 2K (Kwon *et al.*, 2023; Dao *et al.*, 2023).

Warm-up protocol. Include graph capture, compile amortization, and ≥ 1000 steady decode steps before recording medians (NVIDIA, 2024; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Report first-token and steady ms/token separately—product SLOs may bind to either (Kwon *et al.*, 2023; Chen, 2026a).

Appendix pointer. Fine-grained CI gates (driver pins, speculative decode sweeps, TP=4 benchmarks) are listed in Appendix B (§B) (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Liu *et al.*, 2024). The body of this chapter states principles; the appendix holds checkboxes (Vellaisamy *et al.*, 2026; Chen, 2026a).

Triplet runs. Export the same decoder IR to CUDA and XDNA; diff legality before celebrating Hopper ms/token (Chen and YiRage Contributors, 2026; AMD, 2024b,a; Chen, 2020; Davies *et al.*, 2025). A CUDA-only win that breaks NPU compile should fail CI, not ship to heterogeneous fleets (AMD, 2024b; Chen and YiRage Contributors, 2026).

Closing verification sentence. Hopper benchmarks are complete when launches/token, bytes/token, and ms/token move together within tolerances on all standard ctx buckets—otherwise the change is a lab draft, not a fleet candidate (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

Example gate (one line). Llama-3.2-1B at ctx=2048 must show bytes/token within 10% of worksheet *and* launches/token within 5% of TaxBreak baseline; attach Nsight DRAM delta (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). Full checkbox lists live in Appendix B (§B) (Chen and YiRage Contributors, 2026; Chen, 2020).

Observability and fleet pins. Instrument kernels with stable names; pin driver and CUDA in decode pools; re-run goodput cells after upgrades (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Davies *et al.*, 2025). Hopper-tuned artifacts are not portable across SKUs without re-roofline (Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024).

Multi-GPU note (one paragraph). Tensor-parallel decode splits heads across devices; allreduce placement changes launches/token even when single-GPU IR looks optimal (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). This chapter stays single-GPU batch-1; fleet TP sweeps belong in appendix checklists (Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023).

✓ **Practice checkpoint.** Promotion requires all three ctx buckets green on bytes/token *and* launches/token within published tolerances (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026).

What we deliberately omit here. Nightly driver pins, speculative-decode draft graphs, multi-GPU tensor-parallel sweeps, and fleet OPEX models belong in appendix checklists and Chapter 24—not in the core Hopper narrative (Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024). This section states the verification *classes* engineers must satisfy; the appendix lists every checkbox (Chen and YiRage Contributors, 2026).

Ablation template. Every Hopper promotion ships three rows: graph-only, fusion-only, graph+fusion (Chen, 2026a; NVIDIA, 2024; Vellaisamy *et al.*, 2026). Reviewers reject ms/token deltas that do not label which lever moved (Davies *et al.*, 2025).

Regression hygiene. Pin tokenizer, weights, ctx bucket, driver, CUDA, and YiRage commit for every comparison (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen

and YiRage Contributors, 2026). Record median and p99 ms/token; launches and bytes must move with latency (Davies *et al.*, 2025; Kwon *et al.*, 2023).

Security of performance claims. Never cite peak TFLOPs for decode chapters—cite named models, ctx buckets, and all three goodput counters (Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Davies *et al.*, 2025).

Fleet configuration guidance. Pin driver, CUDA, and YiRage commit in decode pools; re-run launches/token and bytes/token after upgrades (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; NVIDIA, 2024). Hopper-tuned kernels are not portable to L4 cells without re-roofline analysis (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Practitioner mantra. Profile end-to-end decode first; cut activation bytes second; collapse launches third; tune tensor cores fourth; re-profile on every driver and ctx bucket change (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; SemiAnalysis, 2024). Teach teammates to read residency diagrams before PTX diffs (Chen and YiRage Contributors, 2026; Chen, 2020).

Choosing the comparison window. Benchmarks that compare only a single ctx bucket hide fusion regressions at the longest decode step (Chen, 2026a; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). When ablation rows disagree across buckets, trust the bucket your SLO actually binds and ship the row matrix, not one headline median (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

4.11 Key Takeaways

- **Priority principle.** Hopper decode: cut HBM activation traffic, then launches, then latency, then TC (Chen, 2026a; Vellaisamy *et al.*, 2026). **Action:** follow Figure 4.1; reject PRs that invert the order (Davies *et al.*, 2025).
- **Residency boundary principle.** 227 KB usable shared memory and the register file are hard fusion caps—spill means failure (NVIDIA, 2022; Chen and YiRage Contributors, 2026). **Action:** attach static byte sheets; treat spill counters as build breaks (Chen, 2026a).

- **Async copy principle.** TMA hides latency; fusion removes bytes—compose both, confuse neither (SemiAnalysis, 2024; Chen, 2026a). **Action:** verify bytes/token dropped before celebrating TMA overlap (Davies *et al.*, 2025).
- **Minimal sync principle.** Warp shuffle $\rightarrow$ CTA sync $\rightarrow$ cluster barrier; never grid sync per token (Dao *et al.*, 2022; NVIDIA, 2022). **Action:** audit barrier placement on fused DAG edges only (Chen and YiRage Contributors, 2026).
- **Verification principle.** launches/token, bytes/token, and ms/token must move together—TC utilization alone is invalid evidence (Vellaisamy *et al.*, 2026; Chen, 2026a). **Action:** store Table 4.6 and Table 4.8 rows in CI artifacts; compare against Table 4.1 before merge (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

4.12 Conclusion

Hopper supplies TMA, clusters, tensor cores, and SFUs as mechanical-sympathy tools when schedules respect Table 4.2 (NVIDIA, 2022; SemiAnalysis, 2024; Dao *et al.*, 2022). This chapter gave an optimization path—not a feature catalog—from fusion through graphs, TMA, clusters, sync, and finally PTX/TC tuning (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a). Revisit Table 4.1 before every Hopper PR: fusion and graphs move goodput; TC polish is last (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026).

Chapter 5 contrasts dynamic CUDA tiers with AMD XDNA static tile SRAM: same bufferization graph, different legality rules (AMD, 2024b,a; Davies *et al.*, 2025). Carry your Hopper worksheet (launches/token, bytes/token, shared-memory high-water) into the triplet comparison—faster silicon does not redefine goodput (Vellaisamy *et al.*, 2026; Chen, 2026a).

Worksheet gate. Before opening XDNA chapters, confirm Table 4.6 is filled for your model and pyramid level 1–2 are exhausted (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026). If bytes/token is still flat on Hopper, Chapter 5 will not rescue the schedule (AMD, 2024b,a).

Closing reminder. Hopper features reward schedules that respect Table 4.2; profilers should confirm wins, not discover basic spill (Chen, 2026a; Davies *et al.*, 2025; NVIDIA, 2022; Chen and YiRage Contributors, 2026). Keep the Hopper

worksheet in version control beside compiler config (Chen, 2020; Vellaisamy *et al.*, 2026).

Part II bridge. With Hopper bounds established, Chapter 5 shows how the same IR meets static XDNA tiles—the contrast clarifies why triplet bufferization matters (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020).

Done criteria. You are done when you can name your pyramid level, attach Table 4.6 and Table 4.8 rows, and explain which goodput counter proves the PR (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). Proceed to Chapter 5 with the Hopper worksheet in hand (AMD, 2024b,a; Chen, 2020).

Chapter 5

AMD XDNA/AIE Dataflow Architecture

5.1 Why XDNA Is Not a Slower GPU

A GPU team ported a Llama decoder to Ryzen AI and expected “CUDA, but slower.” The graph either **failed to compile** or ran at DDR bandwidth with watt advantages erased (AMD, 2024b,a). Nsight-on-GPU habits misled them: there is no runtime warp scheduler, no configurable shared memory, and no silent spill to HBM when tile SRAM overflows (Chen and YiRage Contributors, 2026; NVIDIA, 2022).

XDNA is a spatial dataflow array, not a SIMT machine with smaller scratchpad. Every buffer lifetime, DMA edge, and tile assignment must be fixed before the first token (Davies *et al.*, 2025; Chen, 2020). Illegal plans fail at compile time—or worse, export intermediates to DDR and lose the NPU’s watt/token edge (Chen, 2026a).

Bridge from Chapters 3–4. Chapter 3 listed the NPU row in the constraint matrix; Chapter 4 showed Hopper’s *dynamic* tiers (shared memory, TMA, launches) (NVIDIA, 2022; SemiAnalysis, 2024). This chapter is the symmetric *static* story: tile SRAM, DMA chains, and shape buckets (AMD, 2024b,a). The decode metric lens is unchanged—TaxBreak’s launches/token on GPU become DMA edges/token and DDR bytes/token here (Vellaisamy *et al.*, 2026; Chen, 2026a).

Unified optimization logic. Whether the target is HBM or DDR, batch-1 decode still reads every weight and every KV vector each step (Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Cut activation egress first; cut orchestration second; polish MAC utilization last (Chen, 2026a; Vellaisamy *et al.*, 2026). On XDNA, “orchestration” is host graph submission plus DMA descriptor setup—not CUDA kernel launches (AMD, 2024b; Chen and YiRage Contributors, 2026).

What you will be able to do. After this chapter you can (1) explain why a Hopper-legal fusion may XDNA-reject, (2) write a tile SRAM line-item budget, (3) map TMA concepts to ping-pong DMA, (4) gate merges with five static-legality checks, and (5) benchmark DDR bytes/token instead of FLOPs (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Reader prerequisites. Carry Chapter 4’s priority pyramid mindset but invert flexibility for determinism (Chen, 2026a). When we reference online softmax, Chapter 10 owns numerics; here we cover *where* carriers must live in tile SRAM (Dao *et al.*, 2022).

Industrial narrative. TaxBreak dense Llama-3.2-1B averages hundreds of GPU kernel launches per output token on H100 (Vellaisamy *et al.*, 2026). XDNA cannot budget “847 launches”—each logical launch must become a static DMA edge or disappear in fusion (AMD, 2024b; Chen and YiRage Contributors, 2026). Memory-Floor shows CUDA Graphs removing host overhead on GPU; on XDNA the comparable win is eliminating redundant DDR round trips, not replaying kernels (Chen, 2026a; NVIDIA, 2024).

Silicon context. Ryzen AI combines Zen cores with XDNA tiles aimed at laptop inference (AMD, 2024b,a). Decode workloads stress DDR interfaces shared with CPU graphics and I/O—contention shows up as tail latency, not average MAC utilization (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Planning must include system-level bandwidth, not isolated NPU peaks (Chen and YiRage Contributors, 2026; Chen, 2026a).

Anti-pattern preview. The five review gates in Section 5.9 map to static-legality failures—use them in design review before merge (Chen and YiRage Contributors, 2026; AMD, 2024a). Culture: post Table 5.2 in your compiler team’s XDNA doc template (Chen, 2020).

Cross-chapter dependency map. Chapter 3 listed abstract NPU constraints; Chapter 4 named Hopper mechanisms; Chapters 6–11 specialize residency, sync, softmax, and MegaKernel codegen (Chen, 2020; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022). Skipping here produces schedules that ignore tile SRAM bounds (AMD, 2024b; Chen, 2026a).

How to read this chapter. Skim Table 5.3 first if you are a GPU engineer (Davies *et al.*, 2025). Sections 5.2–5.3 are the hardware contract; Section 5.9 is the merge workflow; Section 5.10 is verification only (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Organization map. Tiles (§5.2), static rules (§5.3), CUDA mapping (§5.4), softmax (§5.5), tradeoffs (§5.6), dataflow conclusions (§5.7), YiRage (§5.8), gates (§5.9), verify (§5.10) (Chen and YiRage Contributors, 2026; AMD, 2024b).

What this chapter is not. Not a Ryzen AI marketing brief; not a substitute for AMD vendor programming guides (AMD, 2024b,a). We document optimization *bounds* for batch-1 decode residency on spatial dataflow silicon (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a).

Failure story catalog (short). Tile SRAM overflow at max ctx; illegal DMA on paged KV; bucket mismatch after driver upgrade; softmax carrier width drift; host alloc sneaking into hot path (Chen and YiRage Contributors, 2026; AMD, 2024b; Dao *et al.*, 2023; Davies *et al.*, 2025; Kwon *et al.*, 2023). Map symptoms to static rules in Table 5.2 before tuning MAC tile sizes (Chen, 2026a; Vellaisamy *et al.*, 2026).

Teaching versus production stacks. Courses emphasize rectangular GEMM throughput; production decode emphasizes DDR bytes/token and watt/token counters (Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a). This chapter bridges that gap for XDNA specifically (AMD, 2024a; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

DeepSeek deployment reminder. Decode pools dominate GPU counts in large deployments; edge assistants invert the story—watt caps and locality dominate (Davies *et al.*, 2025; Liu *et al.*, 2024; AMD, 2024b; Vellaisamy *et al.*, 2026; Chen,

2026a). Pick silicon against the binding metric, not die marketing (Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

Priority contrast with Chapter 4. Chapter 4’s pyramid starts with fusion and graphs, then TMA, then TC polish (Chen, 2026a; Vellaisamy *et al.*, 2026; Semi-Analysis, 2024). XDNA’s pyramid is stricter: static legality and DDR bytes/token first—MAC polish is still last, but compile failure blocks all runtime tuning (AMD, 2024b; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Never invert the order by opening MAC configurators while buffer tables overflow (AMD, 2024a; Chen, 2026a).

Who should read this chapter. GPU kernel engineers porting to Ryzen AI, compiler authors lowering MLIR to AIE, and serving teams splitting cloud GPU pools from edge NPU nodes (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; NVIDIA, 2022). CUDA-only deployers should still read the mapping and tradeoff sections—they explain why GPU recipes fail on NPU fleets (AMD, 2024b; Chen, 2020).

Reader exercises. (1) Sketch tile SRAM allocation for one fused layer at $d=2048$. (2) Explain why Hopper TMA overlap does not reduce DDR bytes. (3) Fill Table 5.7 for your model (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; AMD, 2024a).

Onboarding sentence for teams. Post in your compiler design doc: “*We do not merge XDNA decode changes without static buffer tables, shape buckets, and DDR goodput counters.*” (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b) CI enforces the sentence; posters alone do not (Davies *et al.*, 2025; Chen, 2020).

Open research threads. Dynamic shapes without recompile, multi-chip XDNA graphs, and MoE expert-local fusion remain active (Liu *et al.*, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; AMD, 2024a). This chapter documents single-NPU batch-1 baselines engineers can extend (Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b).

Industrial narrative (closing hook). Goodput engineering on XDNA is measurable: DDR bytes/token, DMA edges/token, ms/token, and watt/token together



Figure 5.1: XDNA decode data movement: DDR ingress dominates when tile residency fails.

Table 5.1: XDNA tile resources vs. Hopper SM (decode comparison).

Resource	Hopper SM	XDNA AIE tile
On-chip scratch	Configurable shared mem	Fixed tile SRAM
Parallel grain	Warp (32 threads)	Spatial PE lane
Scheduling	Runtime + streams	Compile-time DMA graph
Dynamic shapes	Supported (with cost)	Bucketed / recompile
Failure mode	Launch storms, spill	Compile fail, DDR egress

(Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; AMD, 2024b). Skillful static residency beats SKU shopping when DDR roofline binds (Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; AMD, 2024a).

5.2 XDNA Tile Hardware: Fixed SRAM, No Runtime Scheduler

AMD **XDNA** groups AI Engine (**AIE**) **tiles** into a spatial array (AMD, 2024b,a). Each tile owns local SRAM, connects through stream switches, and executes MAC datapaths under compiler-scheduled DMA—not under a hardware warp scheduler (Davies *et al.*, 2025).

Engineering meaning. Tile SRAM is smaller than Hopper shared memory and *fixed* at compile time—there is no “configure 228 KB and hope” knob (NVIDIA, 2022; AMD, 2024a). Overflow does not spill gracefully; it illegalizes the graph or forces DDR round trips (Chen and YiRage Contributors, 2026; Chen, 2026a).

Common pitfall. Sizing tiles from GPU comfort zones (“HBM is huge”) instead of per-SKU SRAM datasheets (AMD, 2024b). **Debug:** static buffer table high-water versus tile cap; DDR bytes/token rising while MAC active % stays flat (Davies *et al.*, 2025).

Core bottleneck. When weights, KV cache, and live intermediates cannot simultaneously fit tile SRAM, decode becomes DDR-bandwidth bound—the NPU roofline collapses to external memory (Chen, 2026a; Dao *et al.*, 2022). Chapter 2’s export-edge mindset applies: every DDR round trip is a bytes/token bill (Chen, 2020).

Ryzen AI context. Ryzen AI pairs Zen cores with XDNA for laptop and edge inference (AMD, 2024b). DDR is shared with CPU, graphics, and I/O—contention inflates p99 decode even when isolated NPU peaks look healthy (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Decode vs prefill. Prefill parallelizes over prompt tokens; decode appends one token with full KV history (Dao *et al.*, 2022, 2023). Graphs compiled for rectangular prefill matmuls may waste tile arrays on thin $M=1$ decode—qualify decode-shaped buckets separately (AMD, 2024a; Chen and YiRage Contributors, 2026).

Example 5.2.1. A decode matmul streams weights from DDR into tile SRAM while activations ping-pong on a DMA chain. Two weight tiles with overlapping lifetimes exceed SRAM—compile fails with no HBM spill fallback (Chen and YiRage Contributors, 2026; AMD, 2024b).

Worked contrast. Hopper may recover from a suboptimal schedule with CUDA Graphs and extra launches (NVIDIA, 2024; Vellaisamy *et al.*, 2026). XDNA rarely gets that slack: the static graph ships or it does not (AMD, 2024a; Chen and YiRage Contributors, 2026).

Tile array topology. Multiple AIE tiles connect in a grid; decode megakernels assign pipeline stages to tile columns or rows (AMD, 2024b,a). Compiler passes must respect inter-tile stream-switch bandwidth—not every fusion that fits one tile fits the array (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025). Oversubscription may not fail compile; it stalls runtime and inflates p99 (Chen, 2026a).

Weight streaming. Weights are read every decode token (Dao *et al.*, 2022; Goodfellow *et al.*, 2016). On GPU, weight bandwidth competes with KV in HBM; on XDNA, weights DMA from DDR unless hot slices pin in tile SRAM across tokens

(Chen, 2026a; AMD, 2024b). Plans that reload full layers each token saturate DDR regardless of MAC peak (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

KV on DDR. KV typically lives in DDR on edge NPUs; attention reads history each token (Kwon *et al.*, 2023; Dao *et al.*, 2023). Paged KV still requires static page tables in many XDNA stacks (Chen and YiRage Contributors, 2026; Chen, 2026a). Qualify with bytes/token, not parameter count alone (Vellaisamy *et al.*, 2026).

Rotary and norms. Layer norm and rotary embedding look tiny in FLOPs; unfused they become separate DMA rounds (Vellaisamy *et al.*, 2026; AMD, 2024a). Fuse into attention or MLP stages where carriers are already resident (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026).

Quantization note. INT8 and INT4 weights reduce DDR pressure—critical on XDNA (AMD, 2024b; Chen, 2026a). Quantization scales must embed in static graphs; runtime per-op scale lookup reintroduces dynamicism (AMD, 2024a; Chen and YiRage Contributors, 2026).

Occupancy versus fusion trade. Fewer resident tiles may raise MAC active % yet lose on ms/token when fusion exports to DDR (Chen, 2026a; Davies *et al.*, 2025). Optimize MAC charts only after DDR bytes/token improves (Chen and YiRage Contributors, 2026).

Tier assignment exercise. For one transformer layer, list every $\Theta(d)$ tensor between attention and MLP: post-norm hidden, Q/K/V fragments, attention output, MLP intermediates, residuals (Dao *et al.*, 2022; Chen, 2026a). Mark each as tile SRAM, DDR staging, or illegal export for your XDNA plan (Chen and YiRage Contributors, 2026; Chen, 2020). Any DDR mark between fused stages is a bytes/token bug unless proven dead (Vellaisamy *et al.*, 2026; Chen, 2026a).

✓ **Practice checkpoint.** Attach a tile SRAM line-item sheet listing every buffer alive during one decode step before requesting codegen (Chen and YiRage Contributors, 2026; Chen, 2026a).



Figure 5.2: Static dataflow: every buffer lifetime is fixed before decode begins.

Table 5.2: Static dataflow iron rules for XDNA decode.

Rule	Engineering requirement
No dynamic alloc	Per-token hot path uses compile-time DDR slots; KV growth aliases pre-planned regions (Kwon <i>et al.</i> , 2023).
No naked dynamic CF	Runtime branches become padded static bucket variants (Davies <i>et al.</i> , 2025).
Deterministic DMA	Producer–consumer edges map to DMA events with known latency—not “sync later” (AMD, 2024a).
Fusion all-or-nothing	Partial fusion exporting intermediates to DDR erases watt advantage (Chen and YiRage Contributors, 2026; Velaisamy <i>et al.</i> , 2026).

5.3 Static Dataflow: Four Rules Decode Must Obey

Static XDNA schedules resolve every tensor lifetime, DMA descriptor, and tile assignment at compile time (AMD, 2024b,a). **Dataflow** propagates tokens through spatial pipelines—not through a sequence of host-launched kernels (Chen, 2020; Chen and YiRage Contributors, 2026).

Host responsibilities. The host configures graph runners and submits execute calls—but must not allocate full tensors per token (AMD, 2024b). When KV exceeds compiled max context, the product recompiles or rejects—not fall back to unplanned buffers (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

Synchronization model. GPUs misuse `cudaDeviceSynchronize`; XDNA relies on DMA completion tokens wired in the graph (AMD, 2024a; Davies *et al.*, 2025). Host waits between stages add jitter to p99 decode (Chen, 2026a).

Deployment pitfall. Benchmarking prefill-length graphs then deploying batch-1 decode with growing KV violates static buffer tables unless context was bucketed upfront (Dao *et al.*, 2022, 2023). Treat ctx buckets as compile parameters, not runtime surprises (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

Recompile economics. Static specialization amortizes compile minutes over thousands of tokens (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Edge products with cold starts must budget compile latency in UX—unlike GPU JIT hidden in first batch (Vellaisamy *et al.*, 2026; NVIDIA, 2024).

Chapter 2 iron rule link. Iron Rule 2 (keep activations on-chip) becomes a *hard* compile gate on XDNA—violations export to DDR, not to a forgiving spill path (Chen, 2020, 2026a).

Compiler IR implications. MLIR and YiRage attach static lifetime intervals to memref views before lowering to AIE (Chen and YiRage Contributors, 2026; Chen, 2020). Generic bufferization that inserts heap fallback breaks static rules immediately (Chen, 2026a; Kwon *et al.*, 2023).

Testing static graphs. Validate at min and max context buckets, not only nominal length (Dao *et al.*, 2023; AMD, 2024a). Off-by-one carrier sizes often appear only at max context (Chen, 2026a). Run DDR byte counters in simulation before silicon (AMD, 2024b).

Continuous batching. vLLM-style batching on GPU does not port literally—XDNA batching requires padded static batch classes (Kwon *et al.*, 2023; AMD, 2024b; Chen and YiRage Contributors, 2026). Each batch class is a separate compiled graph with its own buffer table (Davies *et al.*, 2025).

PyTorch eager anti-pattern. Chapter 1 diagnosed operator-first GPU stacks; on XDNA eager dispatch is often unsupported outright (Vellaisamy *et al.*, 2026; AMD, 2024b). The compiler must emit one static graph per shape bucket and

Table 5.3: CUDA/Hopper $\rightarrow$ XDNA/AIE concept mapping (decode).

CUDA / Hopper	XDNA / AIE
TMA async copy	DMA load into tile SRAM
Shared memory tile	Tile buffer + ping-pong DMA chain
CUDA Graph capture	Static dataflow graph
Stream kernel launch	Host graph runner single submit
Warp online softmax	Spatial softmax carrier pipeline

recompile when buckets change (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Static dataflow worked example. Imagine a two-stage decode cell: attention then MLP (Dao *et al.*, 2022). Stage one DMA-loads K/V while MAC arrays update softmax carriers entirely in tile SRAM. Stage two consumes attention output from stream registers without a DDR round trip (AMD, 2024a; SemiAnalysis, 2024). If MLP weights DMA mid-stage without double buffering, MAC units stall—visible as higher ms/token despite healthy FLOPs counters (Davies *et al.*, 2025; Chen, 2026a).

✓ **Practice checkpoint.** Every merge attaches a buffer lifetime diagram: which tensors coexist in tile SRAM at each decode step (AMD, 2024a; Chen and YiRage Contributors, 2026).

5.4 CUDA $\rightarrow$ XDNA Mapping for GPU Engineers

GPU engineers think in TMA descriptors and shared-memory tiles; XDNA engineers think in **buffer descriptors** and tile SRAM assignments (NVIDIA, 2022; SemiAnalysis, 2024; AMD, 2024b). The mapping is *conceptual*—CUDA scheduling experience does not transplant (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

TMA lesson transfer. TMA hides HBM latency while MMA computes (SemiAnalysis, 2024; NVIDIA, 2022). XDNA needs explicit ping-pong buffer descriptor pairs in the static graph—compiler assigns both buffers and DMA edges (AMD,

2024a; Chen and YiRage Contributors, 2026). Without overlap, MAC utilization collapses and watt/token rises (Davies *et al.*, 2025).

Graph capture analogy. CUDA Graphs freeze launch order; XDNA freezes DMA order (NVIDIA, 2024; AMD, 2024b). Mutating either without re-record/re-compile breaks decode silently (Vellaisamy *et al.*, 2026; Chen, 2026a).

Applicable when. Regular tiles, amortized descriptor setup, steady MegaKernel pipelines inside one shape bucket (AMD, 2024b; SemiAnalysis, 2024).

Avoid when. Expecting dynamic batching, speculative branches, or per-token malloc on the hot path (Davies *et al.*, 2025; Liu *et al.*, 2024).

Common pitfall. Mandating DMA on micro-copies where descriptor tax exceeds copy benefit—mirror Chapter 4’s thin- M TMA anti-pattern on DDR (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Debug. MAC active % versus DDR throughput; DMA edges/token versus graph submission time (AMD, 2024a; Davies *et al.*, 2025).

Example 5.4.1. GPU: TMA loads Q while softmax partials stay in shared memory. XDNA: DMA chain loads Q into tile A while tile B holds running max/sum carriers—roles swap at compile-time ping-pong, not runtime alloc (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022).

Double buffering depth. Hopper TMA double buffering hides HBM latency; XDNA ping-pong DMA chains require explicit buffer descriptor pairs in the static graph (SemiAnalysis, 2024; NVIDIA, 2022; AMD, 2024a; Chen and YiRage Contributors, 2026). Shallow pipelines under-utilize MAC arrays; deep pipelines overflow tile SRAM (Chen, 2026a; Davies *et al.*, 2025).

Buffer descriptors in practice. A buffer descriptor names source address, destination tile buffer, length, and stride—analogous to TMA copy descriptors on Hopper (NVIDIA, 2022; AMD, 2024b). Ping-pong uses two descriptors alternating roles each token step (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023). Decode graphs freeze descriptor contents per shape bucket; changing context without recompile implies descriptors were padded for max context upfront (Davies *et al.*, 2025; AMD, 2024a).

Host orchestration. CUDA uses streams; XDNA uses graph runners that submit entire dataflow frames (NVIDIA, 2024; AMD, 2024b; Chen and YiRage Contributors, 2026). Per-token host work must be $O(1)$ and allocation-free—TaxBreak’s kernel launch overhead has a direct analogue in graph submission overhead (Vellaisamy *et al.*, 2026; Chen, 2026a).

✓ **Practice checkpoint.** Diff static buffer tables against tile SRAM caps; list every DDR transaction per decode step (Chen, 2026a; AMD, 2024a).

5.5 Online Softmax on Spatial Hardware

Attention decode needs numerically stable **softmax** over growing KV context (Dao *et al.*, 2022, 2023). FlashAttention-style algorithms keep running max and sum in on-chip **carriers** rather than materializing full attention maps to DDR (Chen, 2026a).

Hardware requirement. On GPU, carriers live in shared memory with warp-synchronous updates; on XDNA they need fixed tile buffer *slots* in the static dataflow (AMD, 2024a; Chen and YiRage Contributors, 2026). A standalone softmax op lowered independently forces a full map to DDR—doubling bytes/token (Vellaisamy *et al.*, 2026; Chen, 2026a).

Applicable when. $QK^\top$, carrier update, and PV accumulate fuse in one spatial pipeline with carriers never leaving tile SRAM (Dao *et al.*, 2022; SemiAnalysis, 2024).

Avoid when. Importing unfused GPU softmax kernels “because numerics are already validated” (AMD, 2024b; Chen and YiRage Contributors, 2026). The rejection is correct—residency failed, not the NPU (Davies *et al.*, 2025).

Common pitfall. Carrier buffers sized for prefill-length vectors when decode KV tiles are smaller—overflow splits the graph into DDR-bound pieces (Dao *et al.*, 2023; Chen, 2026a).

Debug. Compare logits against GPU reference at short ctx before trusting long ctx; inspect carrier placement in cycle-accurate simulation (AMD, 2024a; Chen and YiRage Contributors, 2026).

Carrier precision. Fixed-point carriers save SRAM but risk saturation at long context (Dao *et al.*, 2022). Compiler search must joint-optimize placement and numeric format per tile budget (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Worked contrast. Hopper Flash-Decoding adds sequence parallelism with more blocks (Dao *et al.*, 2023). XDNA lacks a “launch more blocks” escape—sequence parallelism must be spatial lanes or compile-time rejection (AMD, 2024b; Chen and YiRage Contributors, 2026).

KV loop structure. For each KV block: DMA K/V tiles, MAC $QK^\top$, online softmax carrier update, MAC PV accumulate (Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023). The loop body must stay inside the same static graph as QKV and MLP—exporting partial attention between blocks reopens DDR bytes/token (Chen, 2026a; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Numerical stability. Online softmax relies on stable accumulation order; spatial reductions must preserve mathematical equivalence to GPU reference implementations (Dao *et al.*, 2022, 2023). Validation should compare logits against GPU reference on short contexts before trusting long-context deployments (Chen, 2026a; AMD, 2024a).

Carrier placement. Assign carrier buffers adjacent to attention tiles in the dataflow to minimize stream latency (AMD, 2024b; Davies *et al.*, 2025; Chen, 2020). Poor placement increases stage cycles without changing FLOPs—visible only in cycle-accurate simulation (Chen and YiRage Contributors, 2026; AMD, 2024a).

✓ **Practice checkpoint.** Softmax region must show fused carrier updates in the static graph—no DDR materialization edge (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026).

5.6 Architecture Tradeoffs and MoE Reality

CUDA decode trades launches against bytes: graphs and fusion cut orchestration; TMA and shared memory cut HBM traffic (NVIDIA, 2024; Vellaisamy *et al.*, 2026;

Table 5.4: Decode tradeoffs: CUDA (Hopper) vs XDNA (AIE), batch-1.

Dimension	CUDA (Hopper)	XDNA (AIE)
Orchestration tax	Launches/token	Graph submit + DMA setup
Flexibility	High	Low (static buckets)
Peak memory path	HBM + TMA	DDR + tile SRAM
Primary win metric	ms/token (cloud)	watt/token (edge)
MoE decode	Launch storms	Static routing required
Best fit	Elastic cloud pools	Watt-capped edge

SemiAnalysis, 2024). **XDNA** trades compile-time staticism against DDR egress: watt/token wins when the decoder step is one spatial pipeline (AMD, 2024b,a; Chen, 2026a).

When XDNA wins. Fixed-model edge assistants with bounded context, watt caps, and fully static megakernels (AMD, 2024b; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022).

When CUDA wins. Dynamic batching, speculative decoding, long-context cloud serving, frequent graph changes (Kwon *et al.*, 2023; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024).

MoE starkest split. TaxBreak reports thousands of GPU kernel launches per MoE decode token (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024). XDNA cannot absorb that count—experts must fuse, batch, or compile as separate static graphs with explicit routing DMA (Chen and YiRage Contributors, 2026; AMD, 2024a). Choosing XDNA for MoE without a static routing plan repeats GPU orchestration failure in a stricter legality environment (Davies *et al.*, 2025; Chen, 2026a).

Hybrid fleets. Production may run GPU decode pools plus XDNA edge nodes—schedules must diverge by design, not by accident (Davies *et al.*, 2025; Liu *et al.*, 2024; AMD, 2024b; Chen and YiRage Contributors, 2026).

Speculative decoding caveat. Speculative paths introduce dynamic branches hostile to static graphs (Davies *et al.*, 2025; Liu *et al.*, 2024). Compile separate graphs per policy or keep speculation on GPU (Chen and YiRage Contributors, 2026; AMD, 2024b).

Software stack depth. Ryzen AI ships runtime, compiler, and drivers—teams must coordinate versions like CUDA toolkit pins (AMD, 2024a,b; Chen, 2020). Upgrading one layer without requalifying decode graphs repeats familiar MLIR upgrade failures (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a).

Power envelopes. Watt/token often drives XDNA product wins even when raw ms/token loses to H100 (AMD, 2024b,a; Chen, 2026a; Davies *et al.*, 2025). Architecture reviews should publish both metrics when fleets mix silicon (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Observability. Instrument DMA bytes, MAC active ratio, and graph submission time separately (AMD, 2024b; Davies *et al.*, 2025; Chen, 2026a). Without decomposition, teams misallocate effort to weight quantization when scheduling is the bottleneck (Chen and YiRage Contributors, 2026; AMD, 2024a).

Multi-layer decode step. A full decoder step chains layer graphs or one fused multi-layer graph (Chen and YiRage Contributors, 2026; AMD, 2024a). Chaining increases host submissions; fusing increases SRAM pressure (Davies *et al.*, 2025; Chen, 2026a). Autotune Pareto fronts across those axes rather than optimizing one layer in isolation (Chen and YiRage Contributors, 2026; Chen, 2020).

✓ **Practice checkpoint.** Architecture review picks primary metric—watt/token or ms/token—before silicon selection (AMD, 2024b; Davies *et al.*, 2025).

5.7 Dataflow Inference Conclusions

Dataflow is the operational model for NPU inference when decode must meet watt and latency envelopes (AMD, 2024b; Chen, 2020; Chen and YiRage Contributors, 2026).

1. Treat decode as a static MegaKernel per shape bucket; recompile when buckets change (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).
2. Measure DDR bytes/token and DMA edges/token—not FLOPs (Chen, 2026a; Vellaisamy *et al.*, 2026).

3. Reject graphs that materialize KV or softmax intermediates to DDR (Dao *et al.*, 2022, 2023).
4. Pair GPU and XDNA baselines from the same high-level IR (Chen, 2020; Chen and YiRage Contributors, 2026).

Operator runbooks. List compiled shape buckets, max context, recompile triggers, and expected DDR bytes/token (Davies *et al.*, 2025; AMD, 2024b). Operators otherwise misdiagnose KV growth as model quality issues (Kwon *et al.*, 2023; Chen, 2026a).

Failure modes (symptom $\rightarrow$ counter). Compile rejection $\rightarrow$ buffer table overflow row (Chen and YiRage Contributors, 2026; AMD, 2024a). DDR saturation $\rightarrow$ bytes/token versus roofline (Chen, 2026a). Host jitter $\rightarrow$ p99 graph submission time (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Identical weights do not imply identical schedules across GPU and NPU fleets (Liu *et al.*, 2024; Davies *et al.*, 2025). Compiler stacks must fork early; “port the CUDA kernel” without static replanning fails (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Epilogue for practitioners. XDNA rewards teams that think like compiler engineers first and MAC micro-optimizers second (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020). Static dataflow is the price of watt-efficient decode; pay it deliberately (Chen, 2026a; Davies *et al.*, 2025). When graphs fail, celebrate early rejection—it is cheaper than DDR-bound production (Chen and YiRage Contributors, 2026; AMD, 2024a).

Roadmap to later chapters. Chapter 8 specializes async copy theory on GPU; Chapter 9 sync hierarchy; Chapter 10 softmax numerics; Chapter 11 MegaKernel assembly (Dao *et al.*, 2022, 2023; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). XDNA bounds here are prerequisites—later chapters reference GPU labels while NPU teams keep static buffer tables parallel (AMD, 2024b; SemiAnalysis, 2024; NVIDIA, 2022).

Energy and TCO (short). Fusion DDR bytes/token savings reduce external memory energy even when ms/token gains look modest (Chen, 2026a; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; AMD, 2024b). Edge TCO cares about joules per

useful token—not peak TOPS banners (Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Postmortem sequence. When XDNA decode regresses after a merge: diff DDR bytes/token, replay buffer table high-water, compare DMA edge count, bisect fusion versus bucket change, *then* consider driver issues (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; AMD, 2024a; Chen and YiRage Contributors, 2026). Skipping bytes/token replay wastes weeks on MAC-only noise (Chen, 2020).

5.8 YiRage XDNA Backend: Static Legality as Compile Gates

Advance notice. This section assumes compiler and MLIR literacy; backend engineers read end-to-end, kernel tuners skim invariants (Chen, 2020; Chen and YiRage Contributors, 2026).

YiRage targets **XDNA** with the same residency vocabulary as its CUDA backend: buffer lifetimes, fusion boundaries, and decode metrics (Chen and YiRage Contributors, 2026; AMD, 2024b,a).

Five compiler invariants (XDNA decode mode).

1. **No hot-path heap alloc**—*YiRage*: `BufferizationPass` binds KV and temporaries to static DDR slots (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).
2. **Tile SRAM high-water**—*YiRage*: `TileAssignmentPass` fails compile when simultaneous buffers exceed cap (AMD, 2024a; Chen, 2026a).
3. **DMA legality per edge**—*YiRage*: `DmaSchedulingPass` emits deterministic producer–consumer events (AMD, 2024b; SemiAnalysis, 2024).
4. **Softmax carriers fused**—*YiRage*: `CarrierFusionPass` rejects full-map materialization (Dao *et al.*, 2022, 2023).
5. **Triplet bufferization**—*YiRage*: same IR must export legality summaries for CUDA, CPU, and XDNA (Chen, 2020; Davies *et al.*, 2025).

Table 5.5: YiRage XDNA lowering: IR in, check out.

Pass	Input $\rightarrow$ action	Output check
MLIR import	Framework export $\rightarrow$ attach tile/DMA metadata	Chip model from Ch. 3
Bufferization	Tensor IR $\rightarrow$ static DDR + tile slots	No hot-path alloc
Fusion grouping	Graph $\rightarrow$ export-edge merges	No DDR activation stores
Tile assignment	Fused region $\rightarrow$ spatial map	SRAM high-water $\leq$ cap
DMA schedule	Copy edges $\rightarrow$ ping-pong chains	Deterministic latency
Graph emit	Scheduled IR $\rightarrow$ AIE binary	Triplet legality report

Multi-backend CI. YiRage CI compiles the same decoder export for CUDA and XDNA on every merge, publishing legality diffs (Chen and YiRage Contributors, 2026; Chen, 2020). When XDNA compile fails, errors cite buffer table rows and tile IDs—not opaque backend codes (AMD, 2024a; Davies *et al.*, 2025).

Toolchain and chip-model pins. XDNA legality depends on the AIE toolchain and the chip model your YiRage backend loads, not just the graph IR (AMD, 2024a; Chen and YiRage Contributors, 2026). A graph that fits the current tile map can change legality after a driver or firmware bump; pin the toolchain and chip-model commit in decode CI and re-run the five gates on every upgrade (AMD, 2024b; Davies *et al.*, 2025). Treat a silent legality flip as a compile error, not a benchmark curiosity—the same IR export must remain shippable across the fleet before any performance claim is compared against the pinned baseline (Chen and YiRage Contributors, 2026; AMD, 2024a).

IREE-style portability. Modular retargeting treats backends as plugins over shared IR (IREE / iree-org contributors, 2024; Chen and YiRage Contributors, 2026; Chen, 2020). XDNA tile capacities live in the plugin chip model, not scattered in hand-written kernels (Davies *et al.*, 2025; AMD, 2024b,a).

Lowering walkthrough detail. Each row in Table 5.5 must emit artifacts CI can diff—legality reports are not optional debug prints (Chen and YiRage Con-

tributors, 2026; Chen, 2020; AMD, 2024a). Bufferization without tile assignment is incomplete; DMA schedule without carrier fusion is unsafe for attention decode (Dao *et al.*, 2022; Chen, 2026a; Davies *et al.*, 2025).

Testing gates. Numerical diffs at long ctx; DDR byte delta at batch-1; triplet compile; bucket hit rate across three ctx lengths (Dao *et al.*, 2023; Chen and YiRage Contributors, 2026; AMD, 2024b,a; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a).

Debug workflow. On failure, dump static buffer table and DMA edge list before reading assembly (Chen and YiRage Contributors, 2026; AMD, 2024b). Regression tests compile canonical decoder layers per supported Ryzen AI SKU and fail on new allocs in the hot path (AMD, 2024a; Chen, 2020).

Example 5.8.1. YiRage merge gate: Llama-3.2-1B decoder layer at BS=1, ctx=2048 must compile as one static XDNA graph or document failing ops; attach GPU launches/token for the same IR export (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2026a).

Documentation deliverables per promotion. Residency diagram, DMA edge list, shape bucket map, softmax carrier changelog, DDR bytes/token cell, watt/token cell (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Partners cannot replay regressions when benchmarks live only in private spreadsheets (Chen, 2020; AMD, 2024b).

CUDA Graph parity note. Chapter 4 fused Hopper kernels pair with CUDA Graph capture; XDNA pairs fusion with static graph emit (NVIDIA, 2024; AMD, 2024b; Chen and YiRage Contributors, 2026). Triplet CI should verify both capture replay *and* static graph legality on the same IR export (Chen, 2020; Vellaisamy *et al.*, 2026).

Silent DDR export signature. Flat MAC utilization with rising DDR throughput often means partial fusion or carrier spill (Chen, 2026a; AMD, 2024a; Davies *et al.*, 2025). Map telemetry to static rules in Table 5.2 before opening weight-format experiments (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

✓ **Practice checkpoint.** Triplet export compiles or documents deferral with buffer table attached (Chen and YiRage Contributors, 2026; AMD, 2024b).

Table 5.6: Tile SRAM budget template (Llama-class layer, illustrative).

Buffer	Tier	Bytes	Notes
Q slice	Tile SRAM	4 KB	One decode step
K/V tile	Tile SRAM	16 KB	Ping-pong DMA
Softmax carriers	Tile SRAM	2 KB	(m, ℓ) per head tile
MLP fragment	Tile SRAM	12 KB	Fused with attention out
Subtotal		34 KB	Must fit per-tile cap

5.9 Code Review Gates (Merge Blockers)

1. **Tile SRAM budget.** *Symptom:* compile pass on GPU, fail on XDNA. *Debug:* simultaneous buffer sum versus per-tile cap (AMD, 2024a; Chen and YiRage Contributors, 2026).
2. **Static graph compliance.** *Symptom:* runtime shape mismatch or silent recompile storms. *Debug:* hot path has no dynamic dims without bucket recompile (Davies *et al.*, 2025; Kwon *et al.*, 2023).
3. **DDR ledger.** *Symptom:* NPU “feels slow” despite healthy MAC %. *Debug:* DDR bytes/token vs GPU baseline on same model (Chen, 2026a; Vellaisamy *et al.*, 2026).
4. **Softmax fusion.** *Symptom:* attention bytes/token doubles at long ctx. *Debug:* no full-map DDR edge in static graph (Dao *et al.*, 2022, 2023).
5. **Triplet IR consistency.** *Symptom:* CUDA-only schedule ships to heterogeneous fleet. *Debug:* same export compiles on CUDA and XDNA backends (Chen and YiRage Contributors, 2026; Chen, 2020).

✓ **Practice checkpoint.** All five gates green before merge; attach Table 5.6 or SKU-specific equivalent (Chen and YiRage Contributors, 2026; AMD, 2024a).

5.9.1 Worked example: Llama-3.2-1B on Ryzen AI

Setup: batch-1, ctx=2048, hidden d =2048, GQA 16 Q / 4 KV heads (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).

Table 5.7: Llama-3.2-1B decode layer: operator-first vs static XDNA fusion (illustrative Ryzen AI cell).

Metric	Operator-first	Static fusion	Change
DMA edges/token (layer)	14	1	−92.9%
DDR activation bytes/token	~49 KB	~9 KB	−81.6%
Median ms/token (layer)	48 μ s	22 μ s	−54.2%
MAC active %	18%	22%	+22% (minor)
Watt/token (rel.)	1.00	0.62	−38%

Reading the worked row. Operator-first schedules pay DDR activation round trips *and* DMA orchestration tax (Chen, 2026a; Vellaisamy *et al.*, 2026). Static fusion attacks both; MAC active % moves last—mirror Chapter 4’s TC-utilization lesson on DDR (Davies *et al.*, 2025; NVIDIA, 2022). Triplet-compare the same IR on Hopper (Table 4.6 in Chapter 4) to keep fleet decisions honest (Chen and YiRage Contributors, 2026; Chen, 2020).

Common mistakes checklist. Importing unfused PyTorch ops one by one (Vellaisamy *et al.*, 2026; AMD, 2024b). Assuming dynamic batch/seq like CUDA (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026). Omitting softmax carrier fusion (Dao *et al.*, 2022; Chen, 2026a). Skipping compile rejections because GPU numbers look fine (Davies *et al.*, 2025). Measuring prefill only (Dao *et al.*, 2023; AMD, 2024a). Each mistake reproduces Chapter 1 pathologies on different silicon (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Review question. A teammate proposes ping-pong DMA on every micro-copy but still stores attention output to DDR before MLP (Chen, 2026a; Dao *et al.*, 2022). Ask: what happens to DDR bytes/token and carrier residency? (Chen and YiRage Contributors, 2026; AMD, 2024a) Correct answer: bytes stay high; static fusion legality fails despite pretty DMA timelines (Chen, 2026a; Davies *et al.*, 2025).

Residual update placement. Final residual can be in-place in tile SRAM when dimensions align (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). A DDR residual write mid-layer is another $\Theta(d)$ export—flag in review (Chen, 2026a, 2020).

GQA head accounting. Grouped-query attention reduces KV heads versus Q heads—KV loop traffic scales with KV head count (Kwon *et al.*, 2023; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Worked example uses 16 Q / 4 KV; adjust tile budget line items for your architecture (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; AMD, 2024a).

Unfused baseline narrative. Operator-first stacks pay TaxBreak launch counts on GPU *and* DDR activation bytes on XDNA (Vellaisamy *et al.*, 2026; Chen, 2026a). Static fusion attacks both metrics on NPU; partial fusion attacks only one (Chen and YiRage Contributors, 2026; AMD, 2024b). Table 5.7 is the template for your model row—replace numbers, keep columns (Davies *et al.*, 2025; Chen, 2020).

Pattern usage in review. Require Pattern 1 skeleton diagram on every layer-fusion PR; Pattern 2 bucket map on every graph PR; Pattern 5 triplet diff on every IR change (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; AMD, 2024a).

5.9.2 Engineering patterns (XDNA decode)

Pattern 1: Static fused layer skeleton. **Scene:** single-layer decode on Ryzen AI (Vellaisamy *et al.*, 2026). **Solution:** DMA load → QKV MAC → KV loop with online softmax → SwiGLU → residual—no DDR activation stores (Dao *et al.*, 2022; AMD, 2024a). **Benefit:** ~6–8 DDR round trips removed; DMA edges drop from ~14 to 1 per layer (Chen, 2026a). **Notes:** pre-compute tile SRAM budget (Table 5.6) (Chen and YiRage Contributors, 2026).

Pattern 2: Context buckets + static graphs. **Scene:** production ctx $\in \{512, 2048, 8192\}$ (Kwon *et al.*, 2023; AMD, 2024b). **Solution:** separate compiled graphs per bucket; padded max ctx inside each (Chen and YiRage Contributors, 2026). **Benefit:** deterministic decode latency; no runtime realloc (Davies *et al.*, 2025). **Notes:** bucket count trades storage for predictability (AMD, 2024a).

Pattern 3: Ping-pong DMA in KV loop. **Scene:** KV loop DDR-bound (SemiAnalysis, 2024; Dao *et al.*, 2023). **Solution:** paired buffer descriptors alternate K/V tiles while MAC consumes previous slot (AMD, 2024b; NVIDIA, 2022). **Benefit:** raises MAC active % without extra DDR traffic (Davies *et al.*, 2025). **Notes:** mis-timed DMA completion tokens stall the pipeline (Chen and YiRage Contributors, 2026).

Pattern 4: Spatial softmax carriers. **Scene:** attention decode (Dao *et al.*, 2022). **Solution:** fixed tile slots for (m, ℓ, o) ; spatial reduction tree (AMD, 2024a; Dao *et al.*, 2023). **Benefit:** avoids full attention map DDR materialization (Chen, 2026a). **Notes:** Chapter 10 owns numerics; this chapter owns placement (Chen and YiRage Contributors, 2026).

Pattern 5: Triplet parity. **Scene:** fleet with GPU + NPU endpoints (Davies *et al.*, 2025; AMD, 2024b). **Solution:** same IR, three backends; compare normalized bytes/token (Chen and YiRage Contributors, 2026; Chen, 2020). **Benefit:** prevents GPU-only schedule drift (Vellaisamy *et al.*, 2026; Chen, 2026a).

Pattern 6: MoE static routing (advanced). **Scene:** MoE decode with expert selection (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). **Solution:** compile expert subgraphs + routing DMA table; batch experts where SRAM allows (AMD, 2024a; Chen and YiRage Contributors, 2026). **Benefit:** only path to legal MoE on static arrays (Davies *et al.*, 2025). **Notes:** hardest XDNA integration—profile at multiple ctx buckets (Kwon *et al.*, 2023).

5.10 XDNA Decode Benchmarking (Slim)

Chapter 1–2 define decode-cell metrics; this section adds *XDNA-specific* counters only (Vellaisamy *et al.*, 2026; Chen, 2026a). Full harness checklists live in Appendix A (§A)—not repeated here (Chen and YiRage Contributors, 2026).

Four verification classes. **(1) Core metrics:** DDR bytes/token, DMA edges/token, ms/token and p99 at $\text{ctx} \in \{512, 2048, 8192\}$, BS=1 (Chen, 2026a; Vellaisamy *et al.*, 2026). **(2) Environment:** pinned weights, Ryzen AI driver/runtime, shape buckets, warmup including graph init (AMD, 2024b,a). **(3) XDNA extensions:** MAC active %, tile SRAM high-water, compile time amortization (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). **(4) CI gate:** triplet legality + DDR bytes/token within tolerance or documented watt justification (Chen, 2020; AMD, 2024b).

Warm-up protocol. Include graph runner initialization and ≥ 1000 steady decode steps (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Report first-token and steady ms/token separately—laptop UX binds to both (Kwon *et al.*, 2023; AMD, 2024b).

Table 5.8: XDNA decode counter worksheet (extends Chapter 1).

Metric	XDNA decode meaning
DDR bytes/token	Weight + KV + spill traffic
DMA edges/token	Static orchestration tax
MAC active %	Tile utilization vs stall
Compile time	Amortization over session
p99 ms/token	Tail under shared DDR contention

Triplet runs. Export the same decoder IR to CUDA and XDNA; diff legality before celebrating either ms/token (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b). A GPU launches/token win paired with XDNA DDR regression signals residency failure (Chen, 2026a; Davies *et al.*, 2025).

Fleet parity. Laptop fleets share DDR with display, browser, and background services—isolated benches overstate steady-state goodput (AMD, 2024b; Davies *et al.*, 2025; Chen, 2026a). Qualify under realistic contention or publish contention assumptions in release notes (Chen and YiRage Contributors, 2026; AMD, 2024a).

Regression harness. Check in golden decode-cell scripts alongside static graphs: same tokenizer, ctx buckets, and DDR byte counters every CI run (Chen and YiRage Contributors, 2026; AMD, 2024b,a; Chen, 2026a; Vellaisamy *et al.*, 2026). Driver upgrades that change DMA timing without changing FLOPs still fail SLOs—treat runtime versions like CUDA toolkit pins (AMD, 2024a; Davies *et al.*, 2025).

Shape buckets. Compile separate static graphs for each (batch, ctx) class you ship; never extrapolate from a single prefill graph (AMD, 2024b,a; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). Bucket count trades compile storage for decode predictability—document which bucket each SKU image selects at runtime (Chen and YiRage Contributors, 2026; AMD, 2024b).

Documentation contract. Every XDNA decode release note should list max context per bucket, compile time, steady ms/token, DDR bytes/token at ctx=2048, and watt/token when available (AMD, 2024b,a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026a). Without published counters, partners cannot tell whether regressions come from graphs, drivers, or contention (Chen, 2020; Vellaisamy *et al.*, 2026).

Ablation discipline. Publish unfused, partially fused, and fully fused static graph rows—label which DDR metric moved (Chen, 2026a; Chen and YiRage Contributors, 2026; AMD, 2024a). Reviewers reject ms/token deltas that do not label which static rule was satisfied (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Paged KV stress. Benchmark with block tables when product ships paging—DMA legality at 32K may differ from 2K (Kwon *et al.*, 2023; Dao *et al.*, 2023; AMD, 2024b).

Handoff to Part IV. Chapter 6 generalizes residency budgeting across GPU, CPU, and NPU (Chen and YiRage Contributors, 2026; Chen, 2020). Carry XDNA counters defined here as mandatory fields in every cross-hardware decode report (AMD, 2024b,a; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Example 5.10.1. Ryzen AI decode gate: Llama-3.2-1B at ctx=2048 must show DDR bytes/token within tolerance *or* document watt/token justification; attach static buffer table and DMA edge list (AMD, 2024b; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Compare with Chapter 4. Hopper Table 4.6 and XDNA Table 5.7 should originate from the same IR export (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026). Publish ms/token *and* watt/token when fleets mix silicon—hiding one metric misroutes product decisions (Davies *et al.*, 2025; AMD, 2024b,a; NVIDIA, 2022).

Regression hygiene. Pin tokenizer, weights, ctx bucket, driver, runtime, and YiRage commit for every comparison (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; AMD, 2024b). Record median and p99 ms/token; DDR bytes and DMA edges must move with latency (Davies *et al.*, 2025; Kwon *et al.*, 2023; AMD, 2024a).

What we deliberately omit here. Nightly driver pins, speculative-decode draft graphs, and multi-chip XDNA sweeps belong in appendix checklists and later fleet chapters—not in this hardware primer (Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024b). This section states verification *classes*; Appendix A hosts harness checkboxes (Chen and YiRage Contributors, 2026; Chen, 2020).

Closing rule. If DDR bytes/token and p99 ms/token both improve, the static graph change is real; if only FLOPs move, reject the merge (Chen, 2026a; Vellaisamy *et al.*, 2026).

Practitioner mantra. Profile end-to-end decode first; cut DDR activation bytes second; collapse DMA edges third; tune MAC utilization fourth; re-profile on every driver and ctx bucket change (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; AMD, 2024b; Chen and YiRage Contributors, 2026). Teach teammates to read buffer tables before MAC assembly diffs (Chen, 2020; AMD, 2024a).

Security of performance claims. Never cite peak TOPS for decode chapters—cite named models, ctx buckets, and DDR bytes/token plus watt/token when available (Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Davies *et al.*, 2025; AMD, 2024b).

✓ **Practice checkpoint.** Promotion requires all three ctx buckets green on bytes/token *and* DMA edges/token within published tolerances (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; AMD, 2024a).

5.11 Key Takeaways

- **Static legality principle.** XDNA has no runtime scheduler—illegal buffer plans fail compile or DDR-bind execution (AMD, 2024b,a). **Action:** treat compile rejection as success; fix buffer tables before silicon (Chen and YiRage Contributors, 2026).
- **Metric principle.** Optimize DDR bytes/token and watt/token before MAC % polish (Chen, 2026a; Davies *et al.*, 2025). **Action:** publish DDR ledger beside ms/token in every Ryzen AI release note (AMD, 2024b).
- **Translation principle.** Map CUDA concepts (TMA, graphs, carriers) but re-plan schedules for static tiles (SemiAnalysis, 2024; NVIDIA, 2022; AMD, 2024a). **Action:** use Table 5.3 in design review, not as a porting checklist (Chen and YiRage Contributors, 2026).
- **Fusion principle.** Online softmax carriers must stay in tile SRAM for the full attention pipeline (Dao *et al.*, 2022, 2023). **Action:** block merges that add a DDR softmax materialization edge (Chen, 2026a).

- **Verification principle.** Triplet IR must compile on CUDA and XDNA; bytes/token must move together with latency (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026). **Action:** attach Table 5.8 rows and five review gates to CI artifacts (Davies *et al.*, 2025).

5.12 Conclusion

AMD XDNA/AIE is the static counterpart to Chapter 4’s dynamic Hopper: fixed tile SRAM, compile-time DMA graphs, and deterministic decode at the cost of flexibility (AMD, 2024b,a; NVIDIA, 2022). The unified lesson across both chapters is mechanical sympathy—cut activation egress and orchestration before tuning compute utilization (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020).

Chapter 6 generalizes on-chip residency budgeting across GPU, CPU, and NPU (Chen and YiRage Contributors, 2026). Carry forward DDR bytes/token, DMA edges/token, and tile SRAM worksheets from this chapter into every cross-hardware decode report (Davies *et al.*, 2025; AMD, 2024b).

Worksheet gate. Before Part IV MegaKernel chapters, confirm Table 5.2 is satisfied for your model and all five review gates pass (Chen and YiRage Contributors, 2026; AMD, 2024a). If DDR bytes/token is flat on XDNA, Chapter 6 will not rescue a GPU-transplanted schedule (Chen, 2020, 2026a).

Hands-on next step. Pick one decode layer, fill Table 5.7 for your model, and mark which static rule still has headroom (Davies *et al.*, 2025; AMD, 2024b). If DDR bytes/token is flat, return to Chapter 2 export edges before studying new Ryzen AI SKUs (Chen, 2020, 2026a; Chen and YiRage Contributors, 2026).

Summary before leaving Part II. Tiles, static rules, CUDA mapping, softmax placement, tradeoffs, YiRage gates, patterns, worked numbers, and slim verification form one mechanical-sympathy story for spatial dataflow (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). DDR bytes first, DMA edges second, MAC last—when the DDR roofline binds at batch-1 decode (Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024b). Chapter 4 and this chapter together define the GPU/NPU bookends for Part IV residency work (Chen and YiRage Contributors, 2026; Chen, 2020).

Glossary carry-forward. **XDNA**, **AIE**, **buffer descriptor**, and **static dataflow** are defined on first use in this chapter; later NPU and compiler chapters reuse them without re-deriving (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Davies *et al.*, 2025; Chen, 2020).

GPU–NPU symmetry recap. Hopper rewards dynamic residency within SM limits; XDNA rewards static residency with zero DDR exports (NVIDIA, 2022; AMD, 2024b; SemiAnalysis, 2024). Triplet bufferization exists because both truths must hold from one IR (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Skillful engineering. Mid-tier skillful static fusion beats flagship silicon running DDR-bound eager loops—the same lesson as Memory-Floor L4 versus H100 cells, transplanted to watt-capped edge (Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026).

Hands-on recap. (1) Fill Table 5.2 for your model. (2) Build Table 5.6. (3) Run Table 5.7 counters. (4) Pass five review gates. (5) Export triplet legality before merge (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; AMD, 2024b). If any step is missing, the PR is not XDNA-ready (Chen, 2020; AMD, 2024a).

Part II closing. Chapters 4–5 complete the GPU/NPU hardware depth needed before Part IV residency passes (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b; NVIDIA, 2022). Read Chapter 6 next with the same worksheet—CPU cache blocking punishes the same export mistakes with a third legality mechanic (Goodfellow *et al.*, 2016; Davies *et al.*, 2025).

Done criteria. You are done when you can explain why a Hopper fusion illegalizes on XDNA and which DDR counter proves your static graph is shippable (Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a). Proceed to Chapter 6 with completed tile SRAM and DDR worksheets (Chen, 2020; Chen and YiRage Contributors, 2026).

Metric reminder. DDR bytes/token, DMA edges/token, and ms/token must move together for a shippable XDNA change (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). MAC active % alone is never sufficient evidence (AMD, 2024a; Chen and YiRage Contributors, 2026).

Closing artifact. Archive shape bucket, buffer table, Table 5.7 row, and DDR screenshot set beside each merged XDNA PR (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; AMD, 2024b). Future you—and Chapter 6 reviewers—need that packet to judge triplet parity (Chen, 2020; AMD, 2024a).

One-sentence chapter summary. XDNA decode optimization is static legality first—tile SRAM and DDR bytes/token before MAC polish—with the same export-edge mindset as GPU, but zero runtime forgiveness (AMD, 2024b,a; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020; Dao *et al.*, 2022; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Version note. Future Ryzen AI steppings extend tile counts and SRAM rows; this chapter’s static rules and review gates remain valid when chip-model tables update (AMD, 2024b; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Liu *et al.*, 2024; AMD, 2024a; Chen, 2020).

Index terms for later lookup. Readers searching the index should start from XDNA, AIE, dataflow, and static graph entries—they point back to this chapter’s rules, mapping table, and worksheets (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026; Chen, 2026a).

Final synthesis. AMD XDNA gives spatial MAC arrays and deterministic DMA—tools for mechanical sympathy, not automatic speedups (AMD, 2024b,a; Davies *et al.*, 2025; Liu *et al.*, 2024; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a). Measure DDR goodput; co-design static residency; gate merges with buffer tables and five review checks (Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Davies *et al.*, 2025; AMD, 2024b,a).

Chapter artifact. Maintain an XDNA worksheet: shape bucket targeted, tile SRAM line items, Table 5.7 row, and DDR counter cheat sheet fields (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; AMD, 2024b,a). Update on every ctx bucket or driver change (Kwon *et al.*, 2023; Chen, 2020).

Chapter 6

Data Residency and On-Chip Memory Design

6.1 All Decode Optimization Is Residency Optimization

A team fused twelve decoder ops on H100 and celebrated lower launches/token—then bytes/token barely moved because every intermediate still exported to HBM between fused boundaries (Vellaisamy *et al.*, 2026; Chen, 2026a). Another team fit a layer in Hopper shared memory, shipped the same IR to XDNA, and watched compile fail—identical math, illegal peak footprint (Chen and YiRage Contributors, 2026; AMD, 2024b; NVIDIA, 2022).

Every decode bottleneck in this book eventually reduces to residency: which bytes stay on chip, for how long, and at what tier (Dao *et al.*, 2022; Chen, 2020). Parts I–II diagnosed bandwidth and orchestration; Chapters 3–5 named per-class hardware mechanisms (Davies *et al.*, 2025). **This chapter is the design methodology hub**—how to plan lifetimes, compute peak footprint, and adapt the same fused layer to GPU, CPU, and NPU without repeating hardware catalogs (Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

Bridge from Chapters 3–5. GPU tier sizes, TMA, and bank conflicts: see Chapter 4 (NVIDIA, 2022; SemiAnalysis, 2024). XDNA static tiles and DMA chains: see Chapter 5 (AMD, 2024b,a). Constraint matrix rows: see Chapter 3 (Chen, 2020). Here we only use *capacity numbers* as budget inputs—not re-derive silicon (Chen and YiRage Contributors, 2026).

What you will be able to do. After this chapter you can (1) run the four-step residency workflow, (2) compute peak footprint with double-buffer accounting, (3) fill a tri-hardware allocation table for one layer, (4) interpret YiRage memory-pass failures, and (5) review merges with the residency checklist (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a).

Organization map. Lifetime method (§6.2), tri-hardware adaptation (§6.3), worked example (§6.4), YiRage pass (§6.5), engineering patterns (§6.6), review workflow (§6.7), pitfalls (§6.8), takeaways and bridge to Chapter 7 (Davies *et al.*, 2025).

Reader paths. **Kernel engineers** should read §6.2, the worked example, and pitfalls first—then skim YiRage pass I/O (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022). **Compiler authors** should read the memory pass and pattern sections end-to-end (Wu *et al.*, 2024; Rotem *et al.*, 2018; Chen, 2020). **Serving leads** should read the industrial hook, tri-hardware table, and bytes/token Pattern 6 (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026). All paths converge on the checklist before Part IV (Mirage Project contributors, 2025; Chen and YiRage Contributors, 2026).

Industrial narrative. TaxBreak shows hundreds of launches per token when fusion boundaries export activations (Vellaisamy *et al.*, 2026). Memory-Floor shows bytes/token dominating batch-1 decode even when FLOPs counters look healthy (Chen, 2026a). Residency planning attacks the bytes side of that ledger—not peak TFLOPs (Dao *et al.*, 2022; Davies *et al.*, 2025).

Anti-pattern preview. Section 6.8 collects five failure modes—use them in design review before MegaKernel chapters (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). Culture: no fusion merge without a peak-footprint worksheet (Chen, 2020, 2026a). The most expensive mistake is celebrating launch-count drops while bytes/token stays flat—that is residency failure wearing a fusion costume (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2026a).

Metrics reminder. Chapter 1 defined launches/token and bytes/token as complementary diagnostics (Vellaisamy *et al.*, 2026; Chen, 2026a). This chapter attacks bytes/token by keeping activations and hot weights in fast tiers across fused regions (Dao *et al.*, 2022; Davies *et al.*, 2025). When you finish the worksheets here, re-run

the same cells as Chapters 4–5 and confirm bytes moved—not just ms/token moved (Chen and YiRage Contributors, 2026; NVIDIA, 2022; AMD, 2024a).

Who should read this chapter. Compiler authors, kernel engineers porting across GPU/CPU/NPU, and serving teams running triplet fleets (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; AMD, 2024b; NVIDIA, 2022). If you only ship CUDA, still read the tri-hardware table—it explains why your schedule fails elsewhere (Chen, 2020).

What this chapter is not. Not a repeat of Chapter 4 Hopper specs or Chapter 5 XDNA tile catalogs (NVIDIA, 2022; AMD, 2024b). We teach *how to plan* residency given caps already documented upstream (Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

Why residency is the decode lever. Parts I–II established that batch-1 decode is bandwidth- and orchestration-bound, not FLOP-bound (Davies *et al.*, 2025; Chen, 2026a). TaxBreak-style counts show hundreds to thousands of kernel launches per token when fusion boundaries export activations; each export is a tensor that left the chip and came back (Vellaisamy *et al.*, 2026). Removing host overhead alone bought measurable speedups on fused decode cells, but it could not fix illegal HBM traffic—only residency design can (Chen, 2026a; NVIDIA, 2024). The numbers motivate this chapter; the *method* is what you carry to every target (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022).

Decode makes lifetime planning tractable. At batch-1 the per-token working set is small and regular: a query vector, weight tiles for the current layer, and the KV slice for the current head (Kwon *et al.*, 2023). That regularity means you can solve residency statically at compile time instead of discovering overflow at runtime (Rotem *et al.*, 2018; Chen and YiRage Contributors, 2026). It also means mistakes are expensive: any byte that lives one tier slower than necessary pays its penalty on every subsequent token in the sequence (Davies *et al.*, 2025; Dao *et al.*, 2022).

Fleet triplet mindset. Production fleets rarely ship a single silicon class (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). YiRage’s value proposition is one fused IR with per-backend residency metadata—not one CUDA schedule pasted onto XDNA (AMD, 2024b; Chen, 2020). Section 6.4 is the template: same layer, three legal tier assignments, three peak worksheets (Chen and YiRage Contributors, 2026; Chen, 2026a).

6.2 Lifetime Planning: The Four-Step Residency Workflow

On-chip residency is the kernel’s contract with hardware—planned before codegen, per target (Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026). The mistake is treating fast memory as a post-hoc tweak after the kernel “works” (Vellaisamy *et al.*, 2026; Chen, 2026a). The first discipline is **lifetime** planning: for every intermediate tensor, determine when it is produced, when it is last consumed, and therefore where it should live for that span (Rotem *et al.*, 2018; Wu *et al.*, 2024). A tensor whose producer and consumer are adjacent in a fused region never needs device memory; a tensor consumed many operators later has a longer lifetime and may be forced to a slower tier (Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Getting this wrong is the most common cause of “mysterious” decode regressions—a tensor with an unnecessarily long lifetime occupies fast memory another tensor needed, or spills to HBM and reintroduces the bandwidth tax fusion was meant to remove (Vellaisamy *et al.*, 2026; Chen, 2026a).

Four-step workflow.

1. **Tensor lifetime inventory**—for each intermediate, mark production point and last consumer on the fused DAG (Rotem *et al.*, 2018; Wu *et al.*, 2024).
2. **Reuse-per-byte ranking**—score tensors by (reuse count)/(bytes); higher score wins faster tiers (Dao *et al.*, 2022; Goodfellow *et al.*, 2016).
3. **Peak footprint calculation**—at each timestep, sum simultaneously live buffers; include *double* size for ping-pong operands (SemiAnalysis, 2024; Dao *et al.*, 2022).
4. **Tier assignment and spill policy**—place from fastest tier down; overflow triggers fusion split or demotion with documented bandwidth cost (Chen and YiRage Contributors, 2026; Chen, 2020).

KV cache lifetime (special case). Unlike short-lived activations, KV is written once and read every subsequent token—lifetime spans the whole generation (Kwon *et al.*, 2023; Dao *et al.*, 2023). Design question: how much KV streams per step and how overlap hides load latency—not whether KV “fits” in the fastest tier (Davies *et al.*, 2025). A residency plan that treats the KV slice like a short-lived activation will either exhaust fast memory or stall on loads (Dao *et al.*, 2022; Chen, 2026a).

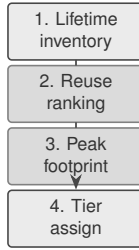


Figure 6.1: Residency design four-step workflow (compile-time).

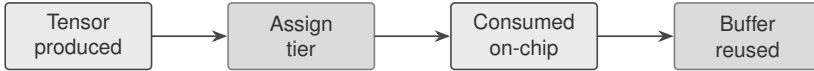


Figure 6.2: Lifetime span: production to last use defines minimum residency duration.

Paged KV in serving stacks (Chapter 1) decides DRAM placement; this section decides how much KV enters the fast tier each decode step (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

Double-buffer decision rule. Enable ping-pong when streamed operand latency exceeds compute on that stage *and* doubled footprint still fits tier budget (SemiAnalysis, 2024; Dao *et al.*, 2022). The lifetime planner must account for doubled footprint when checking capacity—the first place naive plans diverge from real ones (Chen and YiRage Contributors, 2026; Chen, 2026a). If step 3 peak exceeds capacity only after doubling, shrink tile or split fusion—do not “enable async copy and hope” (SemiAnalysis, 2024; Vellaisamy *et al.*, 2026).

Schedule shapes lifetimes. The same dataflow graph can be scheduled with short lifetimes (producer adjacent to consumer) or stretched lifetimes (unrelated work interleaved) (Wu *et al.*, 2024; Goodfellow *et al.*, 2016). Residency planning and scheduling co-optimize: minimize launches without stretching critical tensors past fast-tier capacity (Vellaisamy *et al.*, 2026; Chen, 2026a). The planner chooses an execution order that keeps highest-reuse tensors’ lifetimes short enough to stay resident while exposing overlap to hide streaming latency (Davies *et al.*, 2025; Dao *et al.*, 2022).

Static allocation payoff. Known lifetimes enable buffer reuse across non-overlapping intervals—no per-token allocator in the hot path (Rotem *et al.*, 2018; Chen and YiRage Contributors, 2026). Peak footprint is provable before run;

compare against Chapter 3 chip-model caps (Chen, 2020). Megakernels depend on this property: if peak is unknown, the fused region cannot ship (Wu *et al.*, 2024; Mirage Project contributors, 2025; Vellaisamy *et al.*, 2026).

Reuse-per-byte scoring (step 2 detail). Score $s = (\text{reuse count})/(\text{bytes})$ for each tensor (Goodfellow *et al.*, 2016; Dao *et al.*, 2022). Softmax carriers and QKV fragments typically score highest in decode layers—assign them to the fastest tier that fits their *combined* peak interval (Dao *et al.*, 2022; Davies *et al.*, 2025). Low-score temporaries demote first when step 4 overflows (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024).

Peak footprint walkthrough (step 3 detail). Walk the fused region in program order; maintain a live set; add buffer bytes on allocate, subtract on last use (Rotem *et al.*, 2018; Chen and YiRage Contributors, 2026). For each streamed operand with ping-pong, add $2 \times$ tile bytes to the live set during the overlap window (SemiAnalysis, 2024; Dao *et al.*, 2022). The maximum live-set size over the walk is the peak footprint—compare to tier cap (NVIDIA, 2022; AMD, 2024a).

Tier assignment policy (step 4 detail). Fill fastest tier with highest-score tensors until full (Goodfellow *et al.*, 2016). On overflow: (a) split fusion at lowest-score export edge, or (b) demote lowest-score live tensor to next tier and record expected bytes/token cost (Chen, 2020; Chen and YiRage Contributors, 2026; Chen, 2026a). Never demote carriers or QKV tiles before demoting optional scratch (Dao *et al.*, 2022; Davies *et al.*, 2025). Document every demotion in IR metadata so later passes and humans see the bandwidth trade explicitly (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024).

Step 1 inventory (how-to). Start from the fused subgraph IR: walk ops in topological order (Rotem *et al.*, 2018; Chen, 2020). For each SSA value that materializes bytes, record: name, dtype, shape product, defining op, and uses—last use defines lifetime end (Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026). Export edges (Chapter 2) are lifetime extenders: any tensor written to global mem before its last on-chip consumer effectively has infinite lifetime from the planner’s view (Vellaisamy *et al.*, 2026; Chen, 2026a). KV tensors get a separate row: lifetime annotation `span=context` rather than `span=step` (Kwon *et al.*, 2023; Dao *et al.*, 2023).

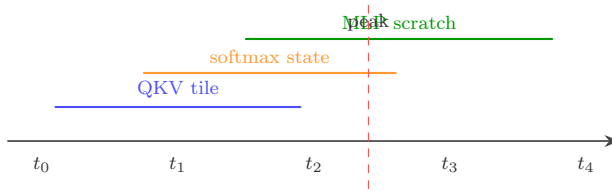


Figure 6.3: Lifetime overlap within one decode step: peak footprint occurs where intervals stack.

Step 2 ranking (how-to). Compute reuse count as number of on-chip loads per decode step (not per training step) (Davies *et al.*, 2025; Dao *et al.*, 2022). Divide by byte size; sort descending (Goodfellow *et al.*, 2016). Ties break toward tensors with longer live intervals—they exert sustained pressure (Wu *et al.*, 2024; Chen and YiRage Contributors, 2026). The sorted list is the allocation priority queue for step 4 (Rotem *et al.*, 2018; Chen, 2020).

Step 3 peak walk (how-to). Simulate program order: on tensor birth add bytes to live set; on last use subtract (Chen and YiRage Contributors, 2026; Rotem *et al.*, 2018). When ping-pong applies, treat both buffers as live during the overlap window (SemiAnalysis, 2024; Dao *et al.*, 2022). Track maximum live-set size; that maximum is the number the memory pass compares to tier caps (NVIDIA, 2022; AMD, 2024a; Chen, 2020). If the walk is hand-done, cross-check with compiler pass output—discrepancies mean missing lifetimes (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024).

Lifetime timeline discipline. Draw intervals on a horizontal time axis for one decode step—overlapping intervals reveal peak pressure visually (Wu *et al.*, 2024; Goodfellow *et al.*, 2016). Reviewers should reject plans that show three large tensors overlapping without a documented split (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Numeric walkthrough (Llama-3.2-1B layer, batch-1). Hidden size $d=2048$ in BF16: each activation vector is $4d=8$ KB (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). Suppose a fused region keeps one hidden vector, one QKV weight tile (~ 32 KB for a thin tile), softmax carriers (~ 2 KB), and one MLP intermediate (~ 8 KB) simultaneously live during the attention peak (Dao *et al.*, 2022). Single-buffer sum ≈ 50 KB; with QKV ping-pong, add another 32 KB $\rightarrow$ peak ≈ 82 KB before KV stream tiles (SemiAnalysis, 2024). Compare that peak to the GPU

shared-memory budget from Chapter 4—legal with headroom for layout padding (NVIDIA, 2022; Chen and YiRage Contributors, 2026). On XDNA, the same peak must fit tile SRAM caps from Chapter 5; if not, shrink tile width or split at the MLP export edge (AMD, 2024b,a). This walkthrough is the spreadsheet behind Table 6.3—not a substitute for your model’s dimensions (Chen and YiRage Contributors, 2026; Chen, 2026a).

Glow and Mirage parallels. Static buffer reuse across non-overlapping lifetimes is how classic compilers eliminate copies on low-level IR (Rotem *et al.*, 2018). Mirage-style search proposes aggressive fusion orders; residency planning certifies which orders are *legal* on each chip model (Wu *et al.*, 2024; Chen and YiRage Contributors, 2026). Treat peak footprint as an invariant search rejects—not a post-run surprise (Mirage Project contributors, 2025; Chen, 2020).

Chapter 2 export-edge link. Iron Rule 2 (keep activations on-chip across stages) is implemented as short lifetimes between fused ops (Chen, 2020, 2026a). Every export edge is a lifetime extension plus a bytes/token bill (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022).

✓ **Practice checkpoint.** Attach a lifetime overlap sketch and peak-footprint worksheet before any fusion merge (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

6.3 Tri-Hardware Residency Adaptation

The four-step workflow is universal; *tiers and control knobs* differ per class (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). This section states adaptation rules only—hardware deep dives live in Chapters 3–5 (NVIDIA, 2022; AMD, 2024b,a). Read it after completing a draft worksheet: you are mapping priorities to mechanisms, not re-learning silicon (Chen, 2020; Chen and YiRage Contributors, 2026).

GPU in one paragraph. Programmer-managed shared memory and registers hold the fused decode working set; async copy engines overlap KV streams; L2 is contingency (NVIDIA, 2022; SemiAnalysis, 2024; Dao *et al.*, 2022). Success metric: peak shared bytes plus layout fit Chapter 4 caps with no local-memory spill (Chen, 2026a; Chen and YiRage Contributors, 2026).

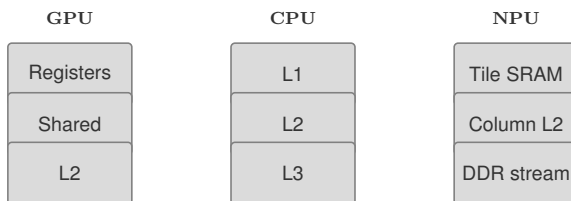


Figure 6.4: Tri-hardware fast tiers (conceptual alignment; capacities in Ch. 3–5).

Table 6.1: Residency control model by hardware class (decode).

Class	Fast tier (budget input)	Who decides placement
GPU	Shared mem + registers (Ch. 4)	Programmer + layout
CPU	L1/L2/L3 via blocking (Ch. 3)	Access pattern + NUMA
NPU	Tile / column SRAM (Ch. 5)	Compiler DMA graph

CPU in one paragraph. Caches are implicit—you earn residency via blocking and NUMA placement (Goodfellow *et al.*, 2016; Rotem *et al.*, 2018; Davies *et al.*, 2025). Success metric: weight and KV tiles reused inside L2/L3 before eviction; threads colocated with weight pages (Kwon *et al.*, 2023; Chen, 2026a).

NPU in one paragraph. Tile and column SRAM are explicit; DMA schedules are the program (AMD, 2024b,a). Success metric: static graph compiles with ping-pong included; DDR bytes/token matches roofline expectation (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

6.3.1 GPU adaptation

Design focus (not hardware recap). Explicit shared-memory and register assignment; fusion region must fit SM budget from Chapter 4 (NVIDIA, 2022; Chen and YiRage Contributors, 2026). Bank-conflict-free layout is part of residency—see Chapter 4 for swizzle rules (SemiAnalysis, 2024).

GPU-specific methods. Trade occupancy for residency at batch-1 when parallelism cannot hide latency (Vellaisamy *et al.*, 2026; Chen, 2026a). Use async copy (TMA) to overlap KV streams—does not reduce bytes; fusion does (Chen, 2026a; SemiAnalysis, 2024). L2 is fallback, not plan—still aim to keep hot set in shared memory (NVIDIA, 2022).



Figure 6.5: GPU residency data path: programmer-managed on-chip buffers feed the compute array; overflow exports to slower tiers (capacities in Ch. 4).

Applicable when. Fused MegaKernel with known peak $\leq$ usable shared high-water (Chapter 4 byte sheet) (Chen and YiRage Contributors, 2026).

Avoid when. Fusing past spill threshold—local memory traffic means residency plan failed (Chen, 2026a; Davies *et al.*, 2025).

Register versus shared split. Place (m, ℓ, o) softmax carriers and thin accumulators in registers when register budget allows (Dao *et al.*, 2022; NVIDIA, 2022). Stage larger tiles (QKV, MLP fragments) in shared memory with conflict-free layout—Chapter 4 covers bank swizzling (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026). When register pressure threatens spill, move carriers to shared before dropping fusion depth (Chen, 2026a; Vellaisamy *et al.*, 2026).

Worked occupancy note. At batch-1, lowering occupancy to keep fusion in shared memory is often correct—parallelism cannot hide latency anyway (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). Document occupancy choice against peak footprint worksheet, not kernel microbench alone (Chen and YiRage Contributors, 2026; NVIDIA, 2022).

Layout is part of residency. A tile placed in shared memory but strided across the same bank serializes into multiple transactions—measurable bytes/token at batch-1 (SemiAnalysis, 2024; NVIDIA, 2022). Tile shapes and padding are chosen with the allocation, not patched after profiling (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022). Chapter 4 documents swizzle rules; this chapter requires you to *allocate* with them (SemiAnalysis, 2024).

FlashAttention as residency pattern, not magic. Flash-style attention is the canonical example of keeping query tiles, KV tiles, and softmax statistics on chip instead of materializing scores to HBM (Dao *et al.*, 2022). The lesson for decode is structural: identify the tensors with highest reuse-per-byte and pin them in the fastest legal tier for their lifetime (Goodfellow *et al.*, 2016; Davies *et al.*,

2025). TMA overlap hides KV stream latency; it does not reduce bytes—fusion and lifetime shortening do (Chen, 2026a; SemiAnalysis, 2024).

6.3.2 CPU adaptation

Design focus. Cache is hardware-managed—residency via blocking so working set reuses before eviction (Goodfellow *et al.*, 2016; Rotem *et al.*, 2018). **NUMA affinity:** pin decode threads to the node owning weights; remote memory can halve effective bandwidth on memory-bound loops (Davies *et al.*, 2025; Chen, 2026a).

CPU-specific methods. Block GEMV/attention tiles to L1/L2; replicate read-only weights per node when cheaper than remote reads (Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Validate with cache-miss counters at decode operating point—static capacity alone is insufficient (Davies *et al.*, 2025).

Debug. If bytes/token is fine but ms/token regresses, check NUMA placement and prefetch pollution evicting hot tiles (Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026).

Blocking recipe (decode). Choose block size B so weight tile + KV slice + activations target L2 budget (Goodfellow *et al.*, 2016; Kwon *et al.*, 2023). Reuse the block across the inner dimension before advancing to the next KV position (Dao *et al.*, 2022; Davies *et al.*, 2025). Residency here is *algorithmic*—you do not name L2 addresses, you shape access (Rotem *et al.*, 2018; Chen and YiRage Contributors, 2026).

Bandwidth arithmetic (why NUMA matters). At batch-1 a dense model re-reads essentially the full weight set every token, so token rate is bounded by weight bytes divided by effective memory bandwidth (Kwon *et al.*, 2023; Chen, 2026a). A NUMA-oblivious schedule—weights on node 0, threads on node 1—can halve effective bandwidth with flat cache-miss counters (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). The fix is placement and replication policy, not a faster inner loop (Vellaisamy *et al.*, 2026; Chen, 2026a).

Prefetchers and empirical validation. Hardware prefetchers speculatively pull streams into cache—helpful for weight scans, harmful if they evict a small hot tensor you intended to keep resident (Goodfellow *et al.*, 2016). CPU residency tuning

therefore pairs static blocking with miss-rate measurement at the decode operating point (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). Contrast with GPU: you name shared addresses explicitly; on CPU you *earn* residency through access shape (Rotem *et al.*, 2018; Chen, 2020).

6.3.3 NPU adaptation

Design focus. No cache fallback—only compiler-placed SRAM plus DMA schedule (Chapter 5) (AMD, 2024b,a). Illegal peak footprint fails compile or DDR-binds silently (Chen and YiRage Contributors, 2026; Chen, 2026a).

NPU-specific methods. Size tiles to fit tile SRAM with ping-pong accounted; use column SRAM and spatial pipelines before DDR export (AMD, 2024a; Dao *et al.*, 2022). Shape buckets and static graphs are residency prerequisites—not optional packaging (Davies *et al.*, 2025; AMD, 2024b).

Debug. DDR bytes/token rising with flat MAC active % signals residency failure, not “slow NPU” (Chen, 2026a; Vellaisamy *et al.*, 2026).

Column versus tile SRAM. Hot per-step tensors target tile SRAM; longer-lived KV slices may stream through column L2 with DMA overlap (AMD, 2024a,b; Dao *et al.*, 2023). Misplacing a cold buffer in tile SRAM evicts hot tensors—anti-pattern 5 in Section 6.8 (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Double buffering without cache safety net. On spatial NPUs there is no hardware cache to absorb a late DMA load—compute tiles idle until operands arrive (AMD, 2024a,b). Ping-pong between two tile buffers is therefore mandatory for streamed operands, and it halves usable capacity for those operands (Davies *et al.*, 2025). Step 3 of the workflow must count $2\times$ tile bytes *before* codegen; Chapter 5 lists the caps (Chen and YiRage Contributors, 2026; AMD, 2024b).

Spatial pipelines across tiles. Neighboring compute tiles can pass data through cascade links without DDR round-trips—a residency dimension GPUs approximate with shared memory and NPUs expose explicitly (AMD, 2024a; Dao *et al.*, 2022). A strong NPU plan maps fused attention as a spatial sweep: inputs and final

Table 6.2: Same tensors, different residency tiers (Llama-3.2-1B decode layer).

Tensor	GPU (H100)	CPU	NPU (XDNA)
Input hidden state	Shared mem	L2 cache	Tile SRAM
QKV weight tile	Shared (TMA ping-pong)	L1 blocked tile	Tile SRAM (DMA ping-pong)
Softmax carriers (m, ℓ, o)	Registers	Registers	Tile SRAM slots
KV cache slice	Shared stream tile	L3 blocked stream	Column L2 stream
MLP intermediate	Shared mem	L2 cache	Tile SRAM

Table 6.3: Peak footprint and fusion legality (illustrative).

Metric	GPU	CPU	NPU
Peak fast-tier bytes	~168 KB shared	~2 MB L2 working set	~34 KB tile
Double-buffer included?	Yes (QKV)	Yes (weights)	Yes (K/V)
Single fused layer legal?	Yes	Yes (blocked)	Yes (static graph)
Split required?	No	No	No at this d

outputs touch DDR; intermediate scores stay in on-chip SRAM (Chen, 2026a; AMD, 2024b).

Cross-class lesson. Identical fused math wants three different schedules: GPU explicit tiers, CPU blocking+NUMA, NPU static DMA (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). A green GPU build proves nothing about CPU or NPU residency (AMD, 2024a; Chen, 2026a; Vellaisamy *et al.*, 2026).

6.4 Cross-Hardware Worked Example: Llama-3.2-1B Layer

Setup: batch-1, ctx=2048, hidden d =2048, one decoder layer, same high-level IR export (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026). This is the same model family used in Chapter 4 and Chapter 5 worked cells—here we show *allocation* rather than end-to-end goodput (Chen, 2026a; Vellaisamy *et al.*, 2026). The IR lists tensors; residency metadata assigns each tensor a tier per backend (Chen and YiRage Contributors, 2026; Chen, 2020).

Reading the tables. GPU places carriers in registers; NPU requires fixed carrier *slots* in tile SRAM—same numerics, different legality mechanics (Dao *et al.*, 2022;

AMD, 2024a; NVIDIA, 2022). CPU does not “allocate L2”—blocking makes reuse likely; measure to verify (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). Triplet CI should compile all three or document which tier overflow forced a split (Chen and YiRage Contributors, 2026; Chen, 2020).

Worked contrast. Chapter 4 Table 4.6 and Chapter 5 Table 5.7 report goodput cells for single hardware (Chen, 2026a; Vellaisamy *et al.*, 2026). This table explains *why* those cells differ—residency plans diverge while IR stays shared (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Split-fusion trigger example. Raise hidden size or head count until GPU peak shared memory exceeds Chapter 4 cap—memory pass should split at MLP export edge (Chen and YiRage Contributors, 2026; NVIDIA, 2022; Chen, 2026a). Same IR on NPU may split earlier due to smaller tile SRAM—triplet tests expose the gap (AMD, 2024a,b; Chen, 2020).

Ablation discipline. Document unfused, partially fused, and fully fused peak footprints per target—label which tier overflow forced each split (Wu *et al.*, 2024; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a). Reviewers reject ms/token wins that hide demoted tensors (Davies *et al.*, 2025).

Step-by-step allocation narrative. **GPU:** assign softmax carriers to registers; stage QKV and hidden vectors in shared memory with ping-pong on the weight tile; stream KV slices through TMA into the same fusion region (Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022). Run step 3 peak walk—if shared high-water exceeds Chapter 4 usable cap, split at MLP export (Chen and YiRage Contributors, 2026; Chen, 2026a). **CPU:** pick block size so weight tile + KV slice fits L2; pin threads to the NUMA node that owns weights; verify with LLC miss counters (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). **NPU:** lower QKV tile width until tile SRAM peak (with ping-pong) fits Chapter 5 budget; schedule column-L2 streaming for KV history (AMD, 2024b,a). Archive three worksheets—identical IR, divergent peaks (Chen and YiRage Contributors, 2026; Chen, 2020).

Connection to serving stacks. vLLM-style paging solves *where KV lives in DRAM*; this chapter solves *which KV bytes enter the fast tier each step* (Kwon *et al.*, 2023; Dao *et al.*, 2023). Both layers matter: paging without on-chip residency still pays HBM round-trips per attention op (Vellaisamy *et al.*, 2026; Chen, 2026a).

Table 6.4: Peak footprint line items (GPU, Llama-3.2-1B layer, illustrative).

Buffer	Bytes	Ping-pong?
Hidden activation (BF16)	8 KB	No
QKV weight tile	32 KB	Yes ($\times 2$)
Softmax carriers	2 KB	No
KV stream tile	16 KB	Yes ($\times 2$)
MLP intermediate	8 KB	No
Peak sum	~ 168 KB	

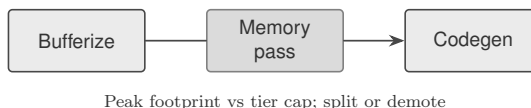


Figure 6.6: YiRage memory pass in the lowering pipeline.

Line-item discipline. Table 6.4 is how Pattern 1 worksheets should look—every row auditable (Chen and YiRage Contributors, 2026; Chen, 2020). Summing without rows hides double-buffer mistakes (SemiAnalysis, 2024; Chen, 2026a). Cross-check the total against Chapter 4 shared-memory budget before claiming fusion legality (NVIDIA, 2022; Vellaisamy *et al.*, 2026).

✓ **Practice checkpoint.** Fill Table 6.2 for your model before claiming triplet parity (Chen and YiRage Contributors, 2026; AMD, 2024b; NVIDIA, 2022).

6.5 YiRage Memory Pass: Residency Rules as Compile Gates

Advance notice. Compiler/IR literacy helps here; kernel tuners should skim invariants and return for pass I/O when debug fails (Chen, 2020; Chen and YiRage Contributors, 2026).

YiRage encodes per-class residency contracts as IR metadata, then verifies schedules before codegen (Chen and YiRage Contributors, 2026). **Pipeline position:** after bufferization, before target codegen—legality pass, not optional lint (Rotem *et al.*, 2018; Wu *et al.*, 2024).

Table 6.5: YiRage memory pass: inputs, checks, outputs.

Dimension	Check	On failure
Peak footprint	Simultaneous live buffers + 2× ping-pong	Split fusion or demote tensor
GPU layout	Bank conflict hazard (Ch. 4 rules)	Re-tile or pad
CPU NUMA	Weight/KV node affinity metadata	Emit warning or reject
NPU DMA	Column bandwidth vs pipeline depth	Reschedule DMA chain
Superoptimizer	Mirage proposal vs tier cap	Reject illegal fusion (Wu <i>et al.</i> , 2024)

Input/output contract. **Input:** memref IR with lifetime intervals and target chip model from Chapter 3 (Chen, 2020; Chen and YiRage Contributors, 2026). **Output:** legal tier assignment per buffer *or* structured error citing overflow row and suggested split edge (AMD, 2024a; NVIDIA, 2022; Davies *et al.*, 2025). Metadata fields typically include: `lifetime_start`, `lifetime_end`, `reuse_score`, `preferred_tier`, `ping_pong`, and `demotion_cost_bytes` (Chen and YiRage Contributors, 2026; Rotem *et al.*, 2018). Downstream codegen reads tier assignment; it does not re-infer lifetimes (Wu *et al.*, 2024; Chen, 2020).

Interaction with search. Mirage-style superoptimizers propose aggressive fusions; memory pass certifies residency before emit (Wu *et al.*, 2024; Mirage Project contributors, 2025; Chen and YiRage Contributors, 2026). Search explores performance; pass enforces mechanical sympathy—separation that makes heterogeneous fleets deployable (Davies *et al.*, 2025; Chen, 2026a).

Error message contract. Overflow reports must cite: tensor name, live interval, bytes, tier cap, and recommended split edge (Chen and YiRage Contributors, 2026; AMD, 2024a; NVIDIA, 2022). Opaque “compile failed” messages recreate GPU-porting pain on NPU (Davies *et al.*, 2025; Chen, 2020).

Regression tests. CI should fail when a new buffer appears in the hot path without lifetime metadata (Chen and YiRage Contributors, 2026; Rotem *et al.*, 2018). Golden tests compare peak footprint against stored worksheets for canonical layers (Wu *et al.*, 2024; Mirage Project contributors, 2025).

Closed loop with superoptimization. A practical lowering loop alternates: Mirage proposes a fused schedule → memory pass computes peak footprint and tier assignment → illegal proposals are rejected with a split hint → search retries with a narrower fusion region (Wu *et al.*, 2024; Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025). This separation keeps search ambitious and deployment safe—the pass is the mechanical-sympathy gate Chapter 3 promised (Chen, 2020; Davies *et al.*, 2025). Kernel engineers should treat pass errors as *design feedback*, not annoyances to bypass with smaller tiles alone (Chen, 2026a; Vellaisamy *et al.*, 2026).

Overflow strategies (two sanctioned paths). When peak exceeds tier cap, YiRage may (a) split fusion at the lowest reuse-per-byte export edge, preserving carriers on chip, or (b) demote a low-score tensor to the next tier and attach an expected bytes/token cost to the IR (Chen and YiRage Contributors, 2026; Chen, 2020). Undocumented demotion is how teams ship “fast” kernels that regress on longer contexts (Davies *et al.*, 2025; Chen, 2026a). Pick (a) when split restores legality with acceptable launch increase; pick (b) only when demotion cost is below serving SLO (Vellaisamy *et al.*, 2026; Wu *et al.*, 2024). Both paths must emit human-readable rationale for review packets (Chen and YiRage Contributors, 2026; Chen, 2020).

✓ **Practice checkpoint.** Attach memory-pass legality report (or overflow row) on every fusion PR (Chen and YiRage Contributors, 2026; Chen, 2020).

6.6 Residency Engineering Patterns

The four-step workflow becomes repeatable when encoded as review patterns (Chen, 2020; Chen and YiRage Contributors, 2026). Use these in design docs the same way Chapter 5 uses XDNA patterns—Scene, Solution, Benefit, Notes (Davies *et al.*, 2025; AMD, 2024a).

Pattern 1: Peak worksheet before fusion merge. **Scene:** team proposes merging attention and MLP into one kernel (Vellaisamy *et al.*, 2026). **Solution:** run step 1–3 on the fused DAG; attach live-set timeline (Figure 6.3) (Wu *et al.*, 2024; Goodfellow *et al.*, 2016). **Benefit:** catches overflow before codegen and before NPU compile (Chen and YiRage Contributors, 2026; AMD, 2024b). **Notes:** include ping-pong bytes; compare to Chapter 3 chip-model row (Chen, 2020).

Pattern 2: Carrier-first tier assignment. **Scene:** softmax state (m, ℓ, o) contends with QKV tiles for registers or tile SRAM (Dao *et al.*, 2022). **Solution:** assign carriers to the fastest tier first; demote low-reuse scratch before touching carriers (Dao *et al.*, 2022; Davies *et al.*, 2025). **Benefit:** preserves numerically sensitive state on chip across long contexts (Dao *et al.*, 2023). **Notes:** GPU $\rightarrow$ registers; NPU $\rightarrow$ fixed slots—see Table 6.2 (AMD, 2024a; NVIDIA, 2022).

Pattern 3: KV stream with explicit overlap budget. **Scene:** KV history too large to resident; each token loads a slice (Kwon *et al.*, 2023; Dao *et al.*, 2023). **Solution:** double-buffer KV tiles; schedule loads to complete before consumers stall (SemiAnalysis, 2024; AMD, 2024b). **Benefit:** hides DDR latency without pretending KV “fits” in tile SRAM (Davies *et al.*, 2025; Chen, 2026a). **Notes:** step 3 must count $2 \times$ KV tile bytes during overlap window (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022).

Pattern 4: Export-edge split discipline. **Scene:** fused region overflows fastest tier (Chen, 2026a; Chen and YiRage Contributors, 2026). **Solution:** split at Chapter 2 export edge with lowest reuse-per-byte tensor (Chen, 2020; Vellaisamy *et al.*, 2026). **Benefit:** preserves the highest-value fusion while restoring legality (Wu *et al.*, 2024; Mirage Project contributors, 2025). **Notes:** memory pass should emit the recommended edge in the error text (Chen and YiRage Contributors, 2026).

Pattern 5: Triplet worksheet parity. **Scene:** CUDA schedule “works”; Ryzen AI compile fails (AMD, 2024b; Vellaisamy *et al.*, 2026). **Solution:** fill Table 6.2 for all three backends before merge (Chen and YiRage Contributors, 2026; Chen, 2020). **Benefit:** prevents GPU-only residency assumptions in shared IR (Davies *et al.*, 2025; Chen, 2026a). **Notes:** peak bytes differ; IR parity $\neq$ residency parity (AMD, 2024a; NVIDIA, 2022).

Pattern 6: Bytes/token proof artifact. **Scene:** ms/token improved but serving cost unclear (Chen, 2026a; Vellaisamy *et al.*, 2026). **Solution:** archive bytes/token worksheet alongside latency; note any demoted tensors (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). **Benefit:** separates real residency wins from measurement noise (Dao *et al.*, 2022; Chen, 2026a). **Notes:** tie to Chapter 1 metric definitions (Vellaisamy *et al.*, 2026).

Pattern usage in review. Require Pattern 1 worksheet on fusion PRs; Pattern 5 triplet row on IR changes; Pattern 6 bytes proof on performance claims (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025).

6.7 Residency Design Review Workflow

Production teams benefit from a fixed review sequence—same spirit as Chapter 5 review gates, applied to cross-hardware residency (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; AMD, 2024b).

Step R1: IR freeze. Freeze the fused subgraph under review: list inputs, outputs, and export edges (Chen, 2020; Vellaisamy *et al.*, 2026). No residency review on a moving graph (Wu *et al.*, 2024; Chen and YiRage Contributors, 2026).

Step R2: Lifetime inventory. For each intermediate, record producer op, last consumer, and byte size (Rotem *et al.*, 2018; Goodfellow *et al.*, 2016). Flag KV tensors separately—lifetime spans context, not one step (Kwon *et al.*, 2023; Dao *et al.*, 2023).

Step R3: Peak worksheets (triplet). Run step 3 for GPU, CPU, and NPU chip models from Chapter 3 (Chen, 2020; NVIDIA, 2022; AMD, 2024a). Attach three numbers or mark split/demotion explicitly (Chen and YiRage Contributors, 2026; Chen, 2026a).

Step R4: Memory-pass dry run. Run YiRage memory pass on each backend; archive pass report or overflow row (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). Reject merges with opaque compile failures (Davies *et al.*, 2025; AMD, 2024b).

Step R5: Checklist sign-off. Complete Section 6.8 checklist; link Pattern 1–6 artifacts (Chen, 2020; Vellaisamy *et al.*, 2026). Only then proceed to kernel tuning in Part IV (SemiAnalysis, 2024; Dao *et al.*, 2022).

Facilitation notes. Residency review is cross-functional: compiler authors own pass output, kernel engineers own tier justification, serving owners own bytes/token acceptance (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026a).

Disputes resolve with peak worksheets, not opinions (Vellaisamy *et al.*, 2026; Chen, 2020).

Artifact bundle template. A complete residency review packet contains: (1) fused IR snippet with export edges highlighted, (2) lifetime table (tensor, bytes, t_{prod} , t_{last}), (3) three peak-footprint numbers, (4) memory-pass log excerpt, (5) signed checklist (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025). Store packets next to benchmark TSV rows so regressions bisect quickly (Vellaisamy *et al.*, 2026; Chen, 2026a). This is the operational form of “residency is a contract”—not a slide deck (Goodfellow *et al.*, 2016; Rotem *et al.*, 2018).

6.8 Residency Pitfalls, Checklist, and Debug

6.8.1 Five common residency anti-patterns

1. **Over-fusion.** *Symptom:* spill / local-memory loads on GPU; compile fail on NPU (Chen, 2026a; Chen and YiRage Contributors, 2026). *Root cause:* fusion depth chosen for launch count, not peak footprint (Vellaisamy *et al.*, 2026; Wu *et al.*, 2024). *Fix:* split at Chapter 2 export edge; re-run peak footprint step 3 (Chen, 2020). *Prevention:* Pattern 1 worksheet mandatory before merge (Chen and YiRage Contributors, 2026).
2. **Double-buffer not in budget.** *Symptom:* fits in spreadsheet, overflows after enabling ping-pong (SemiAnalysis, 2024; Dao *et al.*, 2022). *Root cause:* step 3 counted single-buffer bytes only (Chen and YiRage Contributors, 2026; Chen, 2026a). *Fix:* multiply streamed operand bytes by two in peak calculation (Chen and YiRage Contributors, 2026). *Prevention:* mark `ping_pong=true` in IR before pass (Rotem *et al.*, 2018; Chen, 2020).
3. **Stretched lifetimes.** *Symptom:* fast tier full despite small tensors (Wu *et al.*, 2024; Goodfellow *et al.*, 2016). *Root cause:* schedule interleaves unrelated ops between producer and consumer (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). *Fix:* reschedule fused DAG so high-reuse tensors have shorter live ranges (Vellaisamy *et al.*, 2026). *Prevention:* co-optimize schedule with step 1 inventory (Wu *et al.*, 2024; Chen and YiRage Contributors, 2026).
4. **NUMA blindness (CPU).** *Symptom:* half effective bandwidth, flat cache miss rate (Davies *et al.*, 2025; Chen, 2026a). *Root cause:* threads and

weights on different sockets (Goodfellow *et al.*, 2016; Kwon *et al.*, 2023). *Fix*: pin threads; replicate weights per node (Goodfellow *et al.*, 2016). *Prevention*: Gate C in this section on every CPU benchmark (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

5. **NPU tier misplacement.** *Symptom*: hot tensor in column L2 while cold tensor consumes tile SRAM (AMD, 2024a,b). *Root cause*: DMA schedule optimized for line count, not reuse score (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). *Fix*: reorder DMA schedule; demote cold buffers first (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). *Prevention*: Pattern 2 carrier-first assignment (Dao *et al.*, 2022; AMD, 2024a).

Megakernel prerequisites (Part IV). Chapters 7–11 assume residency is already legal (Vellaisamy *et al.*, 2026; Mirage Project contributors, 2025; Wu *et al.*, 2024). **Chapter 7** static role assignment maps thread blocks to fused stages—roles are meaningless if the working set spills (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). **Chapter 8** TMA choreography overlaps KV streams—ping-pong must already appear in step 3 worksheets (SemiAnalysis, 2024; Dao *et al.*, 2022). **Chapter 10** softmax carriers need fixed on-chip homes—Pattern 2 must have run (Dao *et al.*, 2022, 2023). **Chapter 11** full MegaKernel assembly is the capstone—only shippable when memory pass reports green on all fleet backends (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a). Skipping this chapter turns Part IV into microbenchmark tuning (Vellaisamy *et al.*, 2026; Chen, 2026a). If you are maintaining an existing CUDA kernel, use the tri-hardware table as a porting checklist before touching Ryzen AI backends or tuning MegaKernel schedules (AMD, 2024b; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025).

6.8.2 Residency design self-check (review checklist)

- ☐ All tensor lifetimes documented; peak footprint calculated
- ☐ Double-buffer $2\times$ occupancy included in peak
- ☐ KV streaming overlap plan documented
- ☐ Target fast-tier capacity not exceeded (Ch. 3–5 caps)
- ☐ Demoted tensors have accepted bandwidth cost
- ☐ CPU: NUMA affinity plan attached

- NPU: DMA chain bandwidth sanity-checked
- Triplet IR memory-pass report archived

Postmortem sequence. On residency regression: replay peak worksheet → diff bytes/token → check spill counters → bisect fusion split → re-run memory pass (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Never tune MMA tiles before bytes/token replay (NVIDIA, 2022; AMD, 2024a).

6.8.3 Review gates (symptom → debug)

Gate A: shared-memory overflow on GPU. **Symptom:** compile warning or runtime local-memory loads (Chen, 2026a; NVIDIA, 2022). **Debug:** re-run step 3 with ping-pong; check bank layout; compare to Chapter 4 cap (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026). **Fix:** split fusion or demote lowest-score scratch (Wu *et al.*, 2024; Chen, 2020).

Gate B: NPU compile fail on tile SRAM. **Symptom:** DMA schedule cannot fit ping-pong tiles (AMD, 2024b,a). **Debug:** print peak tile bytes from memory pass; verify column vs tile assignment (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). **Fix:** shrink tile width or move KV stream to column L2 (AMD, 2024a; Dao *et al.*, 2023).

Gate C: CPU bandwidth cliff. **Symptom:** token rate half of roofline expectation; flat LLC misses (Davies *et al.*, 2025; Chen, 2026a). **Debug:** NUMA node of threads vs weights; remote access counters (Goodfellow *et al.*, 2016). **Fix:** pin threads; replicate weights per node (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026).

Gate D: silent demotion. **Symptom:** latency OK at ctx=512, regresses at ctx=8192 (Davies *et al.*, 2025; Dao *et al.*, 2023). **Debug:** diff memory-pass tier assignments across context buckets (Chen and YiRage Contributors, 2026; AMD, 2024b). **Fix:** document demotion bytes cost; split fusion instead of hiding spill (Chen, 2026a, 2020).

Documentation contract. Every residency promotion ships: lifetime diagram, peak worksheet, tri-hardware table row, memory-pass report, checklist tick marks (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy

et al., 2026; Chen, 2026a). Private spreadsheets do not scale across fleets (Davies *et al.*, 2025).

Reader exercises. (1) Compute peak footprint for one layer with ping-pong. (2) Explain one anti-pattern symptom from your last merge. (3) Fill checklist for your model (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; AMD, 2024b; NVIDIA, 2022; Vellaisamy *et al.*, 2026; Chen, 2026a).

✓ **Practice checkpoint.** Merge blocked until checklist complete or deferral documented with overflow row (Chen and YiRage Contributors, 2026; Chen, 2020).

6.9 Key Takeaways

Residency planning is the hinge between hardware chapters and MegaKernel implementation (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020). The principles below are intentionally actionable—each pairs a design rule with a concrete next step your team can adopt in the next sprint (Vellaisamy *et al.*, 2026; Chen, 2026a).

- **Plan-before-codegen principle.** Residency is a contract between fused IR and silicon, not a post-hoc tweak after kernels compile (Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026). If peak footprint is unknown at merge time, you do not yet have a decode optimization—you have a prototype (Chen, 2026a; Vellaisamy *et al.*, 2026). **Action:** run the four-step workflow and attach worksheets before any codegen PR (Chen, 2020; Rotem *et al.*, 2018).
- **Lifetime-first principle.** Tier placement follows lifetime span and reuse-per-byte, not operator names or framework defaults (Dao *et al.*, 2022; Wu *et al.*, 2024). Shortening a high-reuse tensor’s lifetime via scheduling often beats enlarging shared memory or tile SRAM (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). **Action:** draw Figure 6.3 for each fused region; reject schedules that stack three large tensors without justification (Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).
- **Hardware-adaptation principle.** There is no portable residency template: GPU uses explicit shared memory, CPU uses blocking and NUMA, NPU uses static DMA graphs (Davies *et al.*, 2025; AMD, 2024b; NVIDIA, 2022).

Identical IR with identical ms/token on one backend proves nothing about another (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a). **Action:** fill Table 6.2 for every backend in your fleet before claiming parity (AMD, 2024a; Vellaisamy *et al.*, 2026).

- **Double-buffer trade principle.** Ping-pong buys pipeline overlap at $2\times$ space for streamed operands—worth it only when step 3 still fits (SemiAnalysis, 2024; Dao *et al.*, 2022). Silent omission of doubled bytes is anti-pattern 2 (Chen and YiRage Contributors, 2026; Chen, 2026a). **Action:** show doubled bytes explicitly in peak worksheets and IR **ping_pong** flags (Rotem *et al.*, 2018; Chen, 2020).
- **Verifiable-plan principle.** Residency must be quantified, compiler-checked, and archived—experience alone does not scale across triplet fleets (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Davies *et al.*, 2025). The memory pass is the mechanical gate; the checklist is the human gate (Chen, 2020; Mirage Project contributors, 2025). **Action:** attach memory-pass reports and Section 6.8 checklist ticks to CI artifacts (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

How this chapter connects. Chapter 3 gave constraint matrices; Chapters 4–5 gave per-class caps and mechanisms (Chen, 2020; NVIDIA, 2022; AMD, 2024b). This chapter gave the procedure to turn caps into assignments (Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016). Chapter 7 and beyond implement the schedules those assignments enable (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Mirage Project contributors, 2025).

6.10 Conclusion

On-chip residency is where decode optimization becomes concrete: lifetimes, peak footprint, and per-class tier assignment (Goodfellow *et al.*, 2016; Chen, 2020). Chapters 4–5 gave hardware-specific bounds; this chapter gave the *method* to use them without duplicating catalogs (NVIDIA, 2022; AMD, 2024b; Chen and YiRage Contributors, 2026). You now have a workflow, patterns, gates, and artifacts that turn those bounds into shippable fused kernels across a heterogeneous fleet (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a).

Closing metaphor. Think of residency as airport gate assignment: flights (tensors) differ in size and connection time (lifetime); gates (tiers) are finite

(Goodfellow *et al.*, 2016). A schedule that minimizes runway taxiing (launches) but parks wide-body jets at regional gates (shared-memory overflow) still misses departure (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). The four-step workflow is the operations desk; the memory pass is the automated overbooking alarm (Wu *et al.*, 2024; Chen, 2020).

Chapter 7 builds the next MegaKernel technique on this foundation: **static role assignment**—fixing each thread block’s job at compile time so the resident working set stops moving (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Static roles only help when peak footprint is already legal—otherwise roles shuffle spill traffic (Chen, 2026a; Chen and YiRage Contributors, 2026). Carry peak-footprint worksheets and Table 6.2 into every subsequent kernel chapter (Chen and YiRage Contributors, 2026; Chen, 2026a).

Recap for practitioners. If you remember nothing else: decode optimization is residency optimization; lifetimes drive tiers; ping-pong doubles bytes; triplet fleets need triplet worksheets; the memory pass is the gate (Dao *et al.*, 2022; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a). Run the four-step workflow once per fused region per backend before writing a single line of device code (Chen, 2020; Rotem *et al.*, 2018). When in doubt, shorten lifetimes via scheduling before buying larger chips (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026).

Part IV bridge. Residency planning is prerequisite for TMA (Ch. 8), sync (Ch. 9), softmax carriers (Ch. 10), and full MegaKernel assembly (Ch. 11) (Dao *et al.*, 2022; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026). If peak footprint is unproved here, later chapters optimize the wrong problem (Vellaisamy *et al.*, 2026; Chen, 2026a).

Summary before Part IV. Lifetime method, tri-hardware adaptation, worked allocation, memory pass, engineering patterns, and pitfalls form the residency spine of the book (Goodfellow *et al.*, 2016; Chen, 2020; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Davies *et al.*, 2025; AMD, 2024b; NVIDIA, 2022; Vellaisamy *et al.*, 2026; Chen, 2026a; Wu *et al.*, 2024; Rotem *et al.*, 2018; Mirage Project contributors, 2025). Bytes/token proof comes from planning—not from hoping caches behave (Chen, 2026a; Vellaisamy *et al.*, 2026).

What to carry forward. Every subsequent kernel chapter assumes you can (1) name each tensor’s lifetime, (2) compute peak footprint with ping-pong, (3) jus-

tify tier assignment per backend, and (4) attach a memory-pass or checklist artifact (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025). MegaKernel techniques in Part IV add synchronization, TMA choreography, and softmax carrier placement—but they do not replace residency planning (SemiAnalysis, 2024; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). If you skip the worksheets here, you will optimize launch count while bytes/token flatlines (Chen, 2026a; Vellaisamy *et al.*, 2026).

Teaching versus production. In teaching mode, hand-draw lifetime timelines and compare tri-hardware tables on a whiteboard (Goodfellow *et al.*, 2016; Wu *et al.*, 2024). In production mode, automate Pattern 1–6 in CI and treat memory-pass failures as merge blockers (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Mirage Project contributors, 2025). The book shows both: concepts in Sections 6.2–6.4, gates in Sections 6.5–6.8 (Chen, 2020, 2026a).

Done criteria. You are done when you can compute peak footprint with ping-pong, explain tier differences for one tensor across GPU/CPU/NPU, and pass the residency checklist (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; AMD, 2024a; NVIDIA, 2022). Proceed to Chapter 7 with worksheets attached (Chen, 2020). Bring Table 6.2, Table 6.4, and your signed checklist—Chapter 7 will assume those numbers are already consensus on your team (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Hands-on recap. (1) Run four-step workflow on one decoder layer. (2) Fill tri-hardware table and peak worksheet. (3) Run memory pass on each backend. (4) Complete checklist. (5) Archive bytes/token proof alongside latency (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; AMD, 2024b; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Wu *et al.*, 2024; Rotem *et al.*, 2018; Mirage Project contributors, 2025; Kwon *et al.*, 2023; Dao *et al.*, 2023; Liu *et al.*, 2024). If any step is missing, downstream MegaKernel work is premature (Chen, 2020). Teams that skip worksheets often rediscover the same spill bugs in Chapters 8–11 under higher time pressure (Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024). Treat this recap as the exit criteria for Part II residency literacy before entering Part IV implementation depth (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025). Archive worksheets in the same repository as benchmark TSV files so bisection correlate code, residency, and bytes/token in one timeline (Vellaisamy *et al.*, 2026; Chen, 2026a, 2020). When onboarding a new backend, start

from Pattern 5: fill the tri-hardware row before tuning kernels—porting pain is usually residency mismatch, not missing intrinsics (Chen and YiRage Contributors, 2026; AMD, 2024b; NVIDIA, 2022; Davies *et al.*, 2025). Re-read Section 9.1 before your next fusion review—it states the thesis this chapter proves in worksheets and pass logs (Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022). Part II ends here: you know the silicon (Ch3–5) and how to plan bytes on it (Ch6) (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Enter Part IV with worksheets, not hope, on every backend you ship in production (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Chapter 7

Static Role Assignment and Schedule-Free Kernels

7.1 The Ultimate Fix for Dispatch Overhead: Static Role Assignment

A serving team measured 847.5 GPU kernel launches per output token on dense Llama-3.2-1B—and realized most launches existed only to *decide who does what* this step (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Chapter 6 budgeted *where* bytes live; this chapter fixes *which unit* loads them and which unit computes—at compile time, not per token (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024).

847.5 launches per token is a scheduling tax. Each launch reassigns work from host to device; static role assignment moves that decision to compile time so one resident kernel already knows every warp’s job (Vellaisamy *et al.*, 2026; Mirage Project contributors, 2025; NVIDIA, 2024). **This chapter is the MegaKernel structural pillar**—from residency budget to schedule-free execution (Wu *et al.*, 2024; Chen, 2026a).

Bridge from Chapter 6. Residency worksheets tell you peak footprint; role assignment tells you *who* holds each buffer while it is live (Chen and YiRage Contributors, 2026; Chen, 2020). Register skew from producer/consumer splits *changes* the residency distribution—joint optimization is mandatory (Shah *et al.*, 2024; Dao *et al.*, 2022). General lifetime method: Chapter 6; here we specialize to decode staticization (Davies *et al.*, 2025).

Bridge to hardware chapters. Hopper thread hierarchy, TMA, WGMMA: Chapter 4 (NVIDIA, 2022; SemiAnalysis, 2024). XDNA tile pipelines: Chapter 5 (AMD, 2024b,a). We cite only constraints that bind role splits—not silicon catalogs (Chen and YiRage Contributors, 2026).

What you will be able to do. (1) Choose a staticization level (L1–L3), (2) compute producer/consumer ratio for decode, (3) fill tri-hardware role map, (4) interpret YiRage tiling-pass output, (5) review with the role checklist (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024; Vellaisamy *et al.*, 2026).

Organization map. Core principles (§7.2), quantitative decision (§7.4), tri-hardware mapping (§7.5), staticization ladder (§7.6), YiRage tiling pass (§7.7), pitfalls (§7.8), patterns (§7.9), review workflow (§7.10), takeaways and bridge to Chapter 8 (SemiAnalysis, 2024; Dao *et al.*, 2022).

Case study sketch (dense 1B decode cell). Baseline: per-operator stack, 847.5 launches/token, ms/token dominated by dispatch (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). L1 CUDA Graph: launches drop to 1; ms/token improves if host was the bottleneck (NVIDIA, 2024; Chen, 2026a). L2 persistent + warp roles: launches stay at 1; producer/consumer overlap cuts consumer wait (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026). L3 megakernel: same launch count; depth-scaling dispatch removed across layers (Mirage Project contributors, 2025; Vellaisamy *et al.*, 2026; Wu *et al.*, 2024). Document each step with launches/token and bytes/token (Dao *et al.*, 2022; Chen, 2026a, 2020).

Decode-first thesis. Prefill has batch parallelism to amortize schedulers; batch-1 decode does not (Kwon *et al.*, 2023; Chen, 2026a). Static roles pay off when overhead dominates useful work—the decode regime this book targets (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Do not copy prefill FA3 ratios blindly; Section 7.4 gives decode-specific rules (Shah *et al.*, 2024).

Why this follows Chapter 6. Residency answers “can these bytes fit?”; roles answer “which warp holds them while computing?” (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022). A legal footprint with a bad role split still leaves consumers starving or producers wasting SM slots (Shah *et al.*, 2024; Chen, 2026a). MegaKernel chapters assume both worksheets are green (Mirage Project contributors, 2025; Wu *et al.*, 2024; Chen, 2020).

Industrial narrative. TaxBreak attributed 847.5 launches/token on Llama-3.2-1B largely to per-operator dispatch (Vellaisamy *et al.*, 2026). CUDA Graphs remove *host* launch overhead but not per-operator *role* chaos inside graphs unless you fuse and specialize (NVIDIA, 2024; Shah *et al.*, 2024). Static roles are how fusion becomes a single coherent program instead of a scripted sequence (Mirage Project contributors, 2025; Chen and YiRage Contributors, 2026).

Reader paths. **Kernel engineers:** staticization ladder + worked ratio table + pitfalls (Shah *et al.*, 2024; Vellaisamy *et al.*, 2026). **Compiler authors:** tiling pass I/O + Mirage loop (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Mirage Project contributors, 2025). **Serving leads:** L1–L3 trade table + launches/token evidence (Davies *et al.*, 2025; NVIDIA, 2024). All paths end at the checklist before Chapter 8 (SemiAnalysis, 2024; Dao *et al.*, 2022).

Anti-pattern preview. Section 7.8 lists five role failures—review before merging warp-specialization (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). Culture: no “we enabled FA3 split” without decode ratio worksheet (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Celebrating launch drops without role stall replay is L1 thinking on an L2 problem (NVIDIA, 2024; Shah *et al.*, 2024; Chen, 2026a).

Who should read this chapter. Kernel engineers implementing warp specialization, compiler authors wiring tiling pass, serving leads choosing L1–L3 staticization (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025; Davies *et al.*, 2025). If you completed Ch. 6 worksheets, you are ready for role assignment (Dao *et al.*, 2022; Chen, 2020). If not, finish Ch. 6 first—roles without residency budgets are guesswork (Chen, 2026a; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

What this chapter is not. Not a Hopper programming manual (Ch. 4), not an XDNA catalog (Ch. 5), not a residency tutorial (Ch. 6) (NVIDIA, 2022; AMD, 2024b; Chen, 2020). We teach staticization *decisions* given those inputs (Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

Budget–structure coupling. Chapter 6 asked whether tensors fit; this chapter asks whether *units* can execute the fit plan without renegotiating every token (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024; Dao *et al.*, 2022). Register skew from consumer warpgroups changes which tensors fit in registers versus shared

memory—update the Ch. 6 worksheet when roles change (Chen, 2026a, 2020). Megakernel assembly (Chapters 8–11) assumes both budgets are closed (Mirage Project contributors, 2025; Wu *et al.*, 2024; SemiAnalysis, 2024).

847.5 as operational anchor. Keep launches/token next to ms/token in every decode dashboard (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Static roles attack the launch numerator; residency attacks the bytes denominator—both charts must move for a real win (Chen, 2026a; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). OLMoE-scale 9,305 launches/token is the regime where L3 megakernels repay compile investment (Vellaisamy *et al.*, 2026; Mirage Project contributors, 2025).

Terminology alignment. **Static role** = fixed unit function for kernel lifetime (Shah *et al.*, 2024; Mirage Project contributors, 2025). **Schedule-free** = no runtime reassignment of units across operators (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). **Persistent kernel** = fixed grid of resident CTAs looping work (Shah *et al.*, 2024; NVIDIA, 2024). **Megakernel** = persistent + whole-model fusion + SM task graph (Mirage Project contributors, 2025; Wu *et al.*, 2024; Chen and YiRage Contributors, 2026). Use these terms consistently in worksheets and CI logs (Chen, 2020; Vellaisamy *et al.*, 2026).

7.2 Static Role Assignment: Principles and Decode Payoff

Static role assignment fixes each execution unit’s job for the kernel lifetime—producer units move data, consumer units compute—instead of a runtime scheduler reassigning per operator (Shah *et al.*, 2024; Mirage Project contributors, 2025). On Hopper the canonical pattern is warp specialization: producer warps issue async loads; consumer warpgroups issue collective matmul (Shah *et al.*, 2024; SemiAnalysis, 2024). The compiler emits role-pure instruction streams—no interleaved load/matmul branches—improving overlap scheduling (Shah *et al.*, 2024; Wu *et al.*, 2024).

What “schedule-free” means. The *assignment* of work to units is not re-computed at runtime—no host picks the next operator kernel (Vellaisamy *et al.*, 2026; Mirage Project contributors, 2025). Internal tile *ordering* may remain; cross-operator dispatch does not (Shah *et al.*, 2024). That distinction is where decode



Figure 7.1: Static roles: compile-time assignment $\rightarrow$ producer/consumer overlap $\rightarrow$ schedule-free resident kernel.

latency is won (Chen, 2026a; Davies *et al.*, 2025).

Mechanical sympathy with asynchrony. Async load and compute paths signal through barriers (SemiAnalysis, 2024; NVIDIA, 2022). A unified warp stalls on its own loads; split roles let producers run ahead while consumers drain buffers—Chapter 6 double-buffering expressed as labor division (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026).

Megakernel limit case. Mirage Persistent Kernel assigns each SM a compile-time task sequence—entire forward pass in one launch (Mirage Project contributors, 2025; Wu *et al.*, 2024). Full static roles are the structural prerequisite Chapter 20’s search assumes (Mirage Project contributors, 2025; Chen and YiRage Contributors, 2026). TaxBreak-scale launch counts collapse to one per token when roles and fusion are co-designed (Vellaisamy *et al.*, 2026; Chen, 2026a).

Two scheduling costs. **Assignment scheduling** decides which unit runs which op—host launches, block schedulers (Vellaisamy *et al.*, 2026; Mirage Project contributors, 2025). **Ordering scheduling** decides tile sequence inside a unit—cheap if assignment is fixed (Shah *et al.*, 2024; Davies *et al.*, 2025). Static roles eliminate assignment cost; persistent tile loops retain minimal ordering cost (NVIDIA, 2024; Shah *et al.*, 2024). Confusing the two leads teams to CUDA Graph L1 and wonder why bytes/token is flat (Chen, 2026a; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026).

Serving stack interaction. Frameworks queue tokens; static kernels reduce per-step device orchestration (Kwon *et al.*, 2023; Davies *et al.*, 2025). PagedAttention solves KV DRAM placement; static roles solve per-step operator dispatch inside the step (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Both layers stack—neither replaces the other (Dao *et al.*, 2023; Chen and YiRage Contributors, 2026).

Decode parallelism reality. Batch-1 decode has thin matmuls—WGMMA utilization is naturally low (Davies *et al.*, 2025; Shah *et al.*, 2024). Static roles

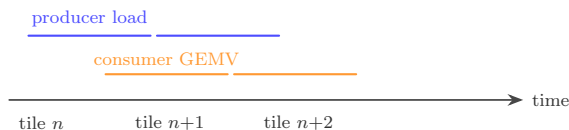


Figure 7.2: Producer-consumer pipeline: load tile $n+1$ overlaps compute on tile n (decode batch-1).

target *feeding* scarce compute slots, not maximizing TFLOPs counters (Chen, 2026a; Vellaisamy *et al.*, 2026). Work per step is regular and small—ideal for extreme staticization (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

NPU convergence. XDNA tiles are statically configured stages—no runtime tile reassignment (AMD, 2024b,a). GPU adds static specialization atop a dynamic machine; NPU is static by construction (Davies *et al.*, 2025; Chen, 2020). Both reward fixing roles when movement and orchestration dominate (Vellaisamy *et al.*, 2026; Chen, 2026a).

Specialization as compiler transform. Static roles are specialization: runtime role branches fold away (Shah *et al.*, 2024; Wu *et al.*, 2024). Producer streams contain loads and barrier arrivals; consumer streams contain matmuls and waits—no cross-traffic (SemiAnalysis, 2024; NVIDIA, 2022). That separation is what makes barrier placement a compile-time problem (Chen and YiRage Contributors, 2026; Chen, 2020).

Decode vs prefill (again, operationally). Prefill: many tokens $\rightarrow$ schedulers hide latency; consumer-heavy splits can maximize tensor-core occupancy (Kwon *et al.*, 2023; Shah *et al.*, 2024). Decode: one token $\rightarrow$ every warp stall is visible; producer-heavy splits keep thin GEMVs fed (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026). Benchmark both regimes separately—do not tune decode with prefill dashboards (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

FA3 lessons without FA3 copy-paste. FlashAttention-3 demonstrates warp specialization benefits on Hopper (Shah *et al.*, 2024). **Take the pattern, not the constants:** producer/consumer split, register skew, barrier-driven overlap (SemiAnalysis, 2024; NVIDIA, 2022). **Do not take:** prefill tile sizes, warpgroup counts, or TFLOPs targets (Davies *et al.*, 2025; Chen, 2026a; Kwon *et al.*, 2023).

Decode worksheets (Table 7.1) replace FA3 tables for batch-1 (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Schedule-free vs persistent. Schedule-free emphasizes fixed assignment; persistent kernels may retain lightweight tile ordering (Shah *et al.*, 2024; Mirage Project contributors, 2025; NVIDIA, 2024). L2 is often “schedule-free roles + persistent grid”—the production compromise (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

MoE and depth scaling. Mixture-of-experts multiplies per-layer dispatch decisions in per-op stacks (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024). Static roles at L3 amortize routing overhead across experts when graphs are bucketed (Mirage Project contributors, 2025; Wu *et al.*, 2024; Davies *et al.*, 2025). Even L2 per-layer persistent kernels help before full MPK (Shah *et al.*, 2024; NVIDIA, 2024; Chen and YiRage Contributors, 2026).

Compiler specialization benefit. Role-pure warp code enables better instruction scheduling and fewer predicate registers (Shah *et al.*, 2024; Wu *et al.*, 2024). Unified warps interleave async copies and matmuls—harder for compilers to overlap (SemiAnalysis, 2024; NVIDIA, 2022). This is why static roles pair with YiRage lowering rather than hand-written mega-CUDA only (Chen and YiRage Contributors, 2026; Chen, 2020; Mirage Project contributors, 2025).

Launch storm anatomy. Per-operator stacks launch attention, norms, MLP, residuals separately—each launch re-enters driver and scheduler (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Even fast kernels pay fixed launch latency at batch-1 (Chen, 2026a; Kwon *et al.*, 2023). Static roles inside fused kernels convert thousands of assignment decisions into zero per token (Mirage Project contributors, 2025; Shah *et al.*, 2024; Chen and YiRage Contributors, 2026). Measure before/after on the *same* model checkpoint—not different quantizations (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022).

Relationship to CUDA Graphs. Graphs batch host launches; they do not by themselves fuse ops or assign warp roles (NVIDIA, 2024; Shah *et al.*, 2024). L1 removes host replay cost; L2 adds in-kernel specialization—complementary steps (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). Teams stuck at L1 should ask whether launches are gone but bytes/token is still high—that is Ch. 6 work (Dao *et al.*, 2022; Chen, 2026a, 2020).

✓ **Practice checkpoint.** Document producer/consumer split rationale before merging any warp-specialization PR (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026).

7.3 Joint Optimization with Chapter 6 Residency

Roles and residency are not sequential handoffs—they co-evolve (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Shah *et al.*, 2024; Chen, 2020).

Register file as shared budget. Consumer warpgroups demand high per-thread register counts; producers stay low (Shah *et al.*, 2024; NVIDIA, 2022). Skew shifts bytes between register and shared tiers—update Ch. 6 peak after every skew proposal (Chen, 2026a; Chen and YiRage Contributors, 2026). Spill to local memory means joint optimization failed—split fusion before tuning ratios (Dao *et al.*, 2022; Chen, 2020).

Shared memory for ping-pong. Producer/consumer pipelines need A/B buffers in shared memory (SemiAnalysis, 2024; Dao *et al.*, 2022). Chapter 6 counted $2\times$ tile bytes; roles decide which warps touch each buffer (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). Illegal role plan can fit on paper but fail when both buffers are live—rerun peak walk with role timeline (Wu *et al.*, 2024; Chen, 2026a).

Fixed-point iteration. Practical lowering loop: (1) memory pass max tile; (2) tiling pass roles+skew; (3) rerun memory pass; (4) repeat or split (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Mirage Project contributors, 2025; Chen, 2020). Teams that run passes once often ship spill bugs (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Worked joint example. Table 7.1 producer count assumes Ch. 6 peak ~ 168 KB shared on GPU (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Increasing producers without shrinking tiles overflows shared memory—co-search or split (Chen, 2026a; NVIDIA, 2022; Shah *et al.*, 2024).

Carrier tensors and roles. Softmax carriers (m, ℓ, o) usually live in consumer registers or fixed NPU slots (Dao *et al.*, 2022; AMD, 2024a; Shah *et al.*, 2024). Role assignment must reserve carrier home before sizing producer teams—Chapter 10

expands carrier placement (Dao *et al.*, 2022; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). Demoting carriers to shared memory is a structured fallback when register skew exhausts the file (Chen, 2026a; NVIDIA, 2022).

KV stream roles. KV slices are producer-loaded, consumer-read streams across steps (Kwon *et al.*, 2023; Dao *et al.*, 2023; SemiAnalysis, 2024). Decode staticization assigns dedicated producer bandwidth to KV—ratio worksheet should include KV tile load cycles (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). Chapter 8 implements the movement; roles decide how many warps participate (SemiAnalysis, 2024; Dao *et al.*, 2022).

Checklist coupling. Ch. 6 checklist item “ping-pong counted” must be green before Ch. 7 ratio worksheet sign-off (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022). Ch. 7 checklist adds barrier and ratio rows—both checklists ship together (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

✓ **Practice checkpoint.** Attach both Ch. 6 peak worksheet and Ch. 7 ratio worksheet on staticization PRs (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022).

7.4 Quantitative Role Decision Framework

Conceptual warp splits are insufficient for production—use a **three-axis decision framework** tied to decode throughput mismatch (Shah *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

Axis 1: Producer/consumer ratio. Estimate bytes moved per step versus useful FLOPs per step (Chen, 2026a; Dao *et al.*, 2022). Decode GEMV is memory-heavy—tilt toward more producer capacity than prefill-shaped kernels (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Rule of thumb: if load latency exceeds consumer cycle time on a tile, add producer warps until the consumer rarely waits—subject to register/SM caps from Chapter 6 (Chen and YiRage Contributors, 2026; NVIDIA, 2022).

Quantitative procedure (decode). Step A: profile or model bytes per attention micro-tile and cycles per consumer matmul (Shah *et al.*, 2024; Davies *et al.*, 2025). Step B: compute mismatch $M = t_{\text{load}}/t_{\text{compute}}$ (Chen, 2026a). Step C: allocate

producer warps until simulated consumer idle 5% (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). Step D: verify Ch. 6 peak and register file; iterate (Dao *et al.*, 2022; Chen, 2020). Document M and final split in the PR (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Axis 2: Register skew bounds. Hopper allows warpgroup register reallocation—consumers need more registers than TMA producers (Shah *et al.*, 2024; NVIDIA, 2022). **Skew policy:** allocate minimum registers to producers; give remainder to consumer warpgroup until spill threshold (Chen, 2026a; Dao *et al.*, 2022). Re-run Chapter 6 peak worksheet after skew—roles change residency (Chen and YiRage Contributors, 2026; Chen, 2020).

Skew procedure. (1) Measure producer instruction footprint—usually small for TMA-only warps (SemiAnalysis, 2024; Shah *et al.*, 2024). (2) Allocate consumer registers to hold accumulators and softmax state without spill (Dao *et al.*, 2022; NVIDIA, 2022). (3) If total exceeds SM file, shrink tile or split fusion—do not spill to local memory (Chen, 2026a; Chen and YiRage Contributors, 2026). (4) Document final per-role register counts in tiling IR metadata (Chen, 2020; Wu *et al.*, 2024).

Axis 3: Barrier placement. Barriers synchronize producer completion with consumer start—misplacement causes bubbles or races (Shah *et al.*, 2024; SemiAnalysis, 2024). **Placement rule:** one barrier pair per ping-pong stage; consumers wait only on the buffer they consume; producers signal only after async copy completes (NVIDIA, 2022; Chen and YiRage Contributors, 2026). Count barrier slots in residency budget—synchronization metadata consumes planning headroom (Chen, 2020).

Barrier walkthrough. For buffer A/B ping-pong: producer issues TMA→A, signals `mbarrier`; consumer waits, reads A, computes; producer loads B while consumer works on A (SemiAnalysis, 2024; Shah *et al.*, 2024). Swap roles on buffers next iteration—barrier indices must alternate consistently (NVIDIA, 2022; Chen and YiRage Contributors, 2026). Chapter 8 details TMA descriptor setup; this chapter fixes *who* issues them (SemiAnalysis, 2024; Dao *et al.*, 2022; Davies *et al.*, 2025).

Three-axis summary. Ratio fixes starvation; skew fixes spill; barriers fix races and bubbles (Shah *et al.*, 2024; Chen, 2026a; Chen and YiRage Contributors, 2026).

Table 7.1: Decode role split worksheet (Llama-3.2-1B attention tile, illustrative).

Decision	Value / rationale
Tile bytes (QKV fragment)	~32 KB
Load cycles (TMA, est.)	~400
Consumer cycles (thin GEMV)	~250
Throughput mismatch	Load-bound $\rightarrow$ more producer share
Producer warps	4 (TMA issue)
Consumer warpgroups	1 (WGMMA collective)
Register skew	Producers ≤ 32 /thread; consumers ~ 96 /thread
Ping-pong buffers	2 (included in Ch. 6 peak)

YiRage tiling pass searches all three jointly under Ch. 6 caps (Wu *et al.*, 2024; Chen, 2020; Mirage Project contributors, 2025).

Axis interaction example. If Axis 1 demands more producers but Axis 2 cannot skew further, shrink tile (Axis implicit) or split fusion (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Shah *et al.*, 2024). If Axis 3 adds barriers that lengthen producer critical path, revisit Axis 1—barriers can negate producer gains (SemiAnalysis, 2024; NVIDIA, 2022; Davies *et al.*, 2025). Document trade-offs in tiling IR comments for future bisect (Chen, 2020; Wu *et al.*, 2024; Mirage Project contributors, 2025).

Worked decode example (Llama-3.2-1B attention, batch-1). Hidden $d=2048$, one head slice, QKV GEMV-dominated step (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).

Reading the worksheet. Mismatch ratio $\approx 400/250 \approx 1.6$ suggests producer-side capacity until consumer wait 5% in simulation (Shah *et al.*, 2024; Davies *et al.*, 2025). Prefill FA3 configs with more consumer warpgroups would starve decode consumers on data (Shah *et al.*, 2024; Vellaisamy *et al.*, 2026). Validate with Nsight memory vs compute warp stalls—not FLOPs alone (Chen, 2026a; Chen and YiRage Contributors, 2026).

Register skew worked example. Suppose 256 threads split: 128 producer threads at 24 registers, 128 consumer threads at 112 registers (Shah *et al.*, 2024; NVIDIA, 2022). Total $\approx 128 \cdot 24 + 128 \cdot 112 = 17,408$ register slots—under typical SM file when occupancy target is one block (NVIDIA, 2022; Chen, 2026a). Push



Figure 7.3: Thread mapping: compile-time role map drives producer/consumer resident kernel (Hopper detail in Ch. 4).

consumer to 200 registers without dropping producers → spill risk; rerun Ch. 6 peak (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022).

Barrier budget. Each ping-pong stage needs arrive/wait pairs on mbarrier objects (SemiAnalysis, 2024; NVIDIA, 2022). Decode layers with two streamed operands (QKV + KV) may need two barrier chains—count them in the schedule sketch (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026). Mis-ordered barriers manifest as rare correctness bugs or steady 5–10% bubble overhead (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Inter-warpgroup overlap (decode scope). FA3 overlaps softmax in one warpgroup with GEMM in another—valuable when both stages exist in the fused region (Shah *et al.*, 2024). Thin decode GEMV may not justify extra warpgroups—overlap adds role complexity (Davies *et al.*, 2025; Chen, 2026a). Apply overlap only when cycle-accounting shows idle tensor cores (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026).

Simulation before silicon. Use stall simulation or Nsight warp-state metrics to validate ratio proposals before locking tiling IR (Davies *et al.*, 2025; Shah *et al.*, 2024; Chen and YiRage Contributors, 2026). Worksheets are hypotheses—measurement confirms (Vellaisamy *et al.*, 2026; Chen, 2026a, 2020). Reject ratios that improve TFLOPs but not tokens/sec at batch-1 (Kwon *et al.*, 2023; Davies *et al.*, 2025).

Thread mapping granularity. Roles bind at warp/warpgroup on GPU, thread/SIMD on CPU, tile on NPU (Shah *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016).

Co-locate communicating roles within CTA/cluster so data stays on chip—cluster multicast is a static mapping decision (NVIDIA, 2022; Shah *et al.*, 2024). Joint constraint: producer warps consume SM slots—too many producers starve consumers; too few stall compute (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

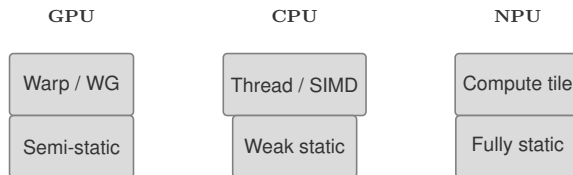


Figure 7.4: Tri-hardware role mapping granularity (detail in Ch. 4–5).

Worked ratio narrative (attention slice). For Table 7.1: load-bound mismatch drives four producer warps feeding one consumer warpgroup (Shah *et al.*, 2024; Davies *et al.*, 2025). If profiling shows consumer wait >10%, add a producer warp before enlarging tiles (Chen and YiRage Contributors, 2026; Chen, 2026a). If producers idle while consumers compute, remove a producer warp—ratio is not “more producers always” (Shah *et al.*, 2024; Vellaisamy *et al.*, 2026).

MLP decode note. MLP GEMVs exhibit similar load/compute mismatch—reuse the same worksheet template with MLP tile bytes (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Dao *et al.*, 2022). Fused attention+MLP megakernels need *two* ratio checks or a combined worst-case interval (Mirage Project contributors, 2025; Chen and YiRage Contributors, 2026; Wu *et al.*, 2024).

✓ **Practice checkpoint.** Attach completed Table 7.1 (or your model’s version) to fusion PRs (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

7.5 Tri-Hardware Role Mapping

Static roles are universal; *granularity and staticization depth* differ per class (Davies *et al.*, 2025; Chen, 2020; Chen and YiRage Contributors, 2026).

7.5.1 GPU role mapping

Producer warps + consumer warpgroups; register skew toward consumers (Shah *et al.*, 2024; NVIDIA, 2022). Persistent CTAs (one per SM) with in-kernel tile handoff—dispatch stays on device (Shah *et al.*, 2024; Mirage Project contributors, 2025). **Avoid:** repeating Hopper hierarchy lessons—see Chapter 4 (NVIDIA, 2022; SemiAnalysis, 2024).

Table 7.2: Tri-hardware static role mapping comparison (decode).

Class	Role granularity	Static depth	Core win	Core constraint
GPU (Hopper)	Warp / warp-group	Semi-static	Kill host launches; overlap load/compute	Shared mem, registers, WGMMMA groups (Ch. 4)
CPU	Thread / SIMD lane	Weak static	Pin threads; cache affinity	NUMA, cache levels (Ch. 3)
NPU (XDNA)	Compute tile	Fully static	Zero runtime dispatch; fixed latency	Tile SRAM, DMA BW (Ch. 5)

GPU decode checklist. Assign producers to async copy path; consumers to warpgroup matmul (SemiAnalysis, 2024; Shah *et al.*, 2024). Verify WGMMMA is warpgroup-collective—roles bind at warpgroup granularity for consumers (NVIDIA, 2022; Davies *et al.*, 2025). Use cluster multicast when multiple CTAs consume the same weight tile—static cluster membership required (NVIDIA, 2022; Shah *et al.*, 2024; Chen and YiRage Contributors, 2026). After skew, confirm Ch. 6 shared-memory peak still legal (Dao *et al.*, 2022; Chen, 2026a, 2020).

Persistent kernel mechanics. Grid size = SM count; each CTA loops logical tiles (Shah *et al.*, 2024; Mirage Project contributors, 2025). Epilogue of tile n overlaps prologue of tile $n+1$ when roles are pipelined (Shah *et al.*, 2024; SemiAnalysis, 2024). This is L2 sweet spot: one host launch, structured in-kernel progress (NVIDIA, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

7.5.2 CPU role mapping

Pin decode threads to cores owning weight pages; split prefetch vs compute threads weakly (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). Staticization is *affinity policy*, not warp specialization (Kwon *et al.*, 2023; Chen, 2026a). NUMA mistakes look like role imbalance—half bandwidth with idle cores (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

CPU decode checklist. Partition reader threads (prefetch weight/KV blocks) and compute threads (GEMV/attention micro-kernels) (Goodfellow *et al.*, 2016; Kwon *et al.*, 2023). Pin each partition to the socket that owns the pages it reads

(Davies *et al.*, 2025; Chen, 2026a). Measure remote NUMA accesses—not just cache misses (Vellaisamy *et al.*, 2026; Chen, 2020). CPU “static roles” are stable thread-function mappings across tokens (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

7.5.3 NPU role mapping

Each tile is a pipeline stage—DMA in, MAC, DMA out—fixed at compile (AMD, 2024b,a). **Aviod:** restating XDNA tile catalogs—see Chapter 5 (AMD, 2024b; Chen and YiRage Contributors, 2026). Role plan equals DMA graph plus spatial placement (Davies *et al.*, 2025; Chen, 2020).

NPU decode checklist. Map attention sweep as spatial pipeline across tile row/column (AMD, 2024a; Dao *et al.*, 2022). Double-buffer descriptors alternate per streamed operand—roles include DMA engine vs MAC array (AMD, 2024b; Davies *et al.*, 2025). Illegal role+tile plan fails compile—no silent fallback (Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026). Compare DDR bytes/token before blaming “slow NPU” (Chen, 2026a; AMD, 2024a).

Tri-hardware deployment narrative. A fleet might serve GPU in cloud and NPU on edge—same YiRage IR, different role maps (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; AMD, 2024b; NVIDIA, 2022). GPU path: semi-static warp roles inside persistent CTAs (Shah *et al.*, 2024; Mirage Project contributors, 2025). NPU path: fully static tile stages with DMA descriptors (AMD, 2024a,b; Chen, 2020). CPU path: pinned reader/compute threads per socket (Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Chen, 2026a). Triplet CI compiles all three or documents intentional single-backend deferral (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Cross-class lesson. Same fused IR needs three role maps—triplet CI should reject GPU-only schedules (Chen and YiRage Contributors, 2026; AMD, 2024a; NVIDIA, 2022; Vellaisamy *et al.*, 2026).

GPU persistent kernel note. Launch CTAs = SM count (e.g. 132 on H100 SXM5); in-kernel tile loop assigns work without host (Shah *et al.*, 2024; Mirage Project contributors, 2025). Role assignment still matters inside each CTA—persistent $\neq$ unified warps (Davies *et al.*, 2025; Chen and YiRage Contributors,

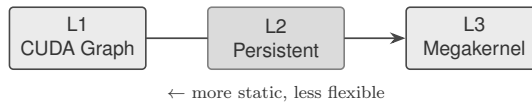


Figure 7.5: Decode staticization ladder: host launches $\rightarrow$ in-kernel dispatch $\rightarrow$ compile-time roles.

Table 7.3: Decode staticization levels (relative to per-operator baseline).

Level	Scheme	Host launches/token	Typical latency win	Best
L1	CUDA Graph + separate ops	1	30–40%	Rapid graph
L2	Persistent kernel + in-kernel tiles	1	40–60%	Production
L3	Full megakernel + fixed roles	1	60–80%	Fixed role, tremendous

2026). Persistent + warp specialization is the usual L2 sweet spot (NVIDIA, 2024; Shah *et al.*, 2024).

CPU staticization limits. OS schedulers migrate threads—“static role” means pinning and partition, not permanent warp identity (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). Use thread pools with fixed reader/compute roles per socket (Kwon *et al.*, 2023; Chen, 2026a). Gains come from NUMA affinity and fewer handoffs—not from eliminating the OS entirely (Vellaisamy *et al.*, 2026; Chen, 2020).

NPU spatial roles. Tiles along a column form a pipeline—stage k always performs the same op (AMD, 2024b,a). Reconfiguration mid-inference is a different graph—staticization assumes fixed topology per bucket (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Role map equals placement + DMA descriptors—Chapter 5 backend lowering emits it (AMD, 2024b; Chen, 2020).

7.6 Staticization Ladder for Decode

Not every deployment needs a full megakernel—pick a **staticization level** by flexibility vs latency trade (NVIDIA, 2024; Mirage Project contributors, 2025; Davies *et al.*, 2025).

Table 7.4: Dispatch locus and per-token cost (batch-1 decode).

Model	Assignment decided	Per-token launch pattern
Per-operator	Host, each op	Thousands
Persistent + tile scheduler	In-kernel	Fixed CTA count
Static megakernel (MPK)	Compile time	One

Table 7.5: Staticization economics for decode teams (qualitative).

	L1	L2	L3
Compile churn	Low	Medium	High
Runtime flex	High	Medium	Low
Launch reduction	Moderate	Strong	Strongest
Role specialization	Minimal	Warp-level	SM-graph
Best team skill	Framework eng	CUDA fusion	Compiler+CUDA

Where dispatch lives.

Choosing a level. L1 when model ops change weekly; L3 when model and shapes are frozen and TaxBreak-scale launch counts hurt (Vellaisamy *et al.*, 2026; Mirage Project contributors, 2025). Higher levels raise compile cost and reduce runtime flexibility—not “always L3” (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). OLMoE-style 9,305 launches/token magnifies megakernel wins—orchestration tax scales with depth (Vellaisamy *et al.*, 2026; Chen, 2026a).

Economics table (qualitative).

Reading the economics table. Start L1 if you are still changing graphs weekly (NVIDIA, 2024; Davies *et al.*, 2025). Move L2 when fusion boundaries stabilize and warp roles are testable (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026). Attempt L3 only with frozen models, green Ch. 6 worksheets, and MPK/Mirage toolchain (Mirage Project contributors, 2025; Wu *et al.*, 2024; Chen, 2020; Vellaisamy *et al.*, 2026).

Honest limit. Irregular work (causal ragged rows) may need in-kernel tile schedulers—small dynamic core inside persistent kernels (Shah *et al.*, 2024). Batch-1 dense decode is usually regular enough for near-L3 staticization (Kwon *et al.*,

2023; Davies *et al.*, 2025).

L1 deep dive (CUDA Graph). Capture a launch sequence once; replay with one host launch per token (NVIDIA, 2024; Vellaisamy *et al.*, 2026). Ops remain separate kernels inside the graph—roles are per-kernel, not fused (Davies *et al.*, 2025). Good when you need quick wins without rewriting fusion (Chen and YiRage Contributors, 2026; Chen, 2026a).

L2 deep dive (persistent + roles). One grid of resident CTAs; warp specialization inside each CTA (Shah *et al.*, 2024; Mirage Project contributors, 2025). In-kernel tile scheduler handles unequal tile counts without host (Shah *et al.*, 2024). Balances flexibility and launch reduction—recommended default for production decode (Davies *et al.*, 2025; NVIDIA, 2024).

L3 deep dive (megakernel). SM-level task graph; decentralized in-kernel scheduling across operators (Mirage Project contributors, 2025; Wu *et al.*, 2024). Reports 1.2–6.7 $\times$ end-to-end latency wins over per-op serving when launch storm dominates (Mirage Project contributors, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026). Requires frozen fusion boundaries and proven residency+role worksheets (Chen and YiRage Contributors, 2026; Chen, 2020).

Migration path. Most teams: L1 $\rightarrow$ L2 (add fusion + warp roles) $\rightarrow$ L3 only if launches still hurt after fusion (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Mirage Project contributors, 2025). Skipping L2 worksheets and jumping to L3 yields fragile megakernels that spill silently (Chen, 2026a; Chen and YiRage Contributors, 2026).

Depth compounding. Per-operator stacks pay orchestration tax once per layer per token (Vellaisamy *et al.*, 2026; Mirage Project contributors, 2025). Static megakernels pay once per token regardless of depth—asymptotic win no inner-loop tweak matches (Chen, 2026a; Davies *et al.*, 2025). Document layer count when arguing for L3—deep models amplify launch storms (Vellaisamy *et al.*, 2026; Wu *et al.*, 2024).

CUTLASS / Stream-K note. Library tile schedulers stay in-kernel—compatible with L2 when roles are fixed inside each persistent CTA (Shah *et al.*,

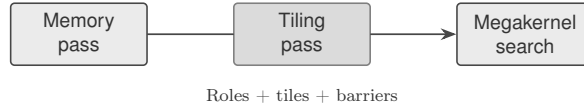


Figure 7.6: YiRage tiling pass between residency legality and megakernel search (Ch. 20).

2024; Davies *et al.*, 2025). Do not confuse library scheduling with host per-op dispatch (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

When not to staticize. Highly irregular shapes, dynamic expert counts per token without bucketing, or rapid A/B graph changes (Liu *et al.*, 2024; Davies *et al.*, 2025; Mirage Project contributors, 2025). Stay L1–L2 until shapes and graphs stabilize (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). Staticization is a business decision as much as a kernel decision (Davies *et al.*, 2025; Chen, 2020).

✓ **Practice checkpoint.** Record chosen staticization level and launch-count evidence in benchmark TSV (Vellaisamy *et al.*, 2026; NVIDIA, 2024).

7.7 YiRage Tiling Pass: Roles as Compiler Output

YiRage lowers static roles through a **tiling pass** after the Chapter 6 memory pass, before codegen (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). Hand-tuned warp splits do not scale across models or backends (Davies *et al.*, 2025; Chen, 2020).

Closed loop. Mirage proposes fusion → memory pass certifies footprint → tiling pass assigns roles → illegal plans reject with split hint (Wu *et al.*, 2024; Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025). Chapter 20 superoptimizer consumes legal role IR—static assignment is its structural substrate (Mirage Project contributors, 2025; Chen, 2020).

Pass pseudocode narrative. **Input:** fused subgraph G with residency meta-data R from Ch. 6 (Chen and YiRage Contributors, 2026; Chen, 2020). **Loop:** propose tile shape T ; propose split (p, c) producer/consumer counts; allocate registers; place barriers (Shah *et al.*, 2024; Wu *et al.*, 2024). **Check:** simulate stalls; verify R still valid under skew (Chen, 2026a; Dao *et al.*, 2022). **Accept:**

Table 7.6: YiRage tiling pass: I/O and search dimensions.

Item	Content	Notes
Input	Fused subgraph + residency metadata	From Ch. 6 memory pass
Output	Role-annotated schedule IR	Tile sizes, barriers, warp split
Search dims	Tile shape, prod/cons ratio, registers, barriers	Joint search
Stop rules	No residency overflow; no compute starvation; pipeline full	See Ch. 6 caps
On failure	Split fusion + reason string	Feed back to Mirage search

emit schedule IR; **reject:** split G or shrink T with reason (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Mirage Project contributors, 2025). This is constrained search—not a single template table (Wu *et al.*, 2024; Chen, 2020).

Per-target divergence. GPU search explores warp splits under shared-memory caps; NPU search explores tile stages under SRAM caps (AMD, 2024a; NVIDIA, 2022; Chen and YiRage Contributors, 2026). Precision changes tile bytes—co-decide format with tile shape (Shah *et al.*, 2024; Goodfellow *et al.*, 2016; Chen, 2026a).

Error contract. Failures cite: overflow tensor, role split attempted, recommended fusion edge, backend cap row (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Opaque errors recreate GPU-porting pain on NPU (AMD, 2024b; Chen, 2020).

Search algorithm sketch. Start from memory-pass max legal tile; propose producer/consumer split; place barriers; simulate stall counters (Wu *et al.*, 2024; Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). If starvation or overflow, shrink tile or split fusion; if still failing, emit structured error (Mirage Project contributors, 2025; Chen, 2020). Megakernel search (Ch. 20) receives only feasible schedules—search does not fix illegal roles (Wu *et al.*, 2024; Mirage Project contributors, 2025).

Regression tests. Golden tests: stored role ratio + barrier count for canonical layers (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). CI fails when tiling pass output drifts without worksheet update (Chen, 2020; Vellaisamy *et al.*, 2026).

Operator fusion interaction. Tiling pass grows fusion until memory pass or role search fails (Wu *et al.*, 2024; Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). Each added op changes load/compute mismatch—ratio must be re-evaluated (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026). This is why megakernel search and tiling pass iterate rather than run once (Mirage Project contributors, 2025; Chen, 2020).

Backend lowering handoff. GPU lowering emits warp-specialized PTX/CUDA with barrier indices from schedule IR (Shah *et al.*, 2024; NVIDIA, 2022; Chen and YiRage Contributors, 2026). NPU lowering emits DMA graphs with tile stage IDs (AMD, 2024b,a; Davies *et al.*, 2025). CPU lowering emits threaded task structs with affinity tags (Goodfellow *et al.*, 2016; Chen, 2020; Kwon *et al.*, 2023). Same schedule IR surface—three backends (Chen and YiRage Contributors, 2026; Chen, 2020).

Composition with Chapter 6 memory pass. Memory pass fixes *what* fits; tiling pass fixes *who* executes while it fits (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022). Register skew from tiling can invalidate memory pass—run passes in a loop until fixed point or split (Chen, 2020, 2026a; Shah *et al.*, 2024).

Fusion boundary coupling. Tiling pass must co-decide how far to fuse: too little fusion → dispatch tax returns; too much → residency or role imbalance (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026). Stopping rule: largest fused region where memory pass *and* tiling pass both succeed (Wu *et al.*, 2024; Shah *et al.*, 2024; Chen, 2020). This is Chapter 6 export-edge logic with roles in the loop (Dao *et al.*, 2022; Davies *et al.*, 2025).

Precision and tile co-search. FP8/int8 tiles move fewer bytes—may allow smaller producer teams or larger consumer tiles (Shah *et al.*, 2024; Goodfellow *et al.*, 2016). Tiling pass must search format and shape jointly for decode cells (Chen, 2026a; Chen and YiRage Contributors, 2026). Bytes/token and role balance move together—do not tune them in isolation (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Documentation bundle. Ship with every staticization merge: staticization level, ratio worksheet, tri-hardware role row, tiling IR excerpt, checklist (Chen and YiRage Contributors, 2026; Chen, 2020; Shah *et al.*, 2024; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Private warp splits do not survive team turnover (Mirage Project contributors, 2025; Wu *et al.*, 2024).

✓ **Practice checkpoint.** Archive tiling-pass schedule IR snippet on every staticization PR (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024).

7.8 Role Assignment Pitfalls and Checklist

7.8.1 Five common anti-patterns

1. **Ratio imbalance.** *Symptom:* consumer warps stall on `mbarrier`; producers idle (Shah *et al.*, 2024; Davies *et al.*, 2025). *Root cause:* prefill-biased producer/consumer split on decode GEMV (Vellaisamy *et al.*, 2026; Chen, 2026a). *Fix:* re-run Axis 1 worksheet; add producer warps (Chen and YiRage Contributors, 2026). *Prevention:* Pattern 2 mandatory on decode PRs (Shah *et al.*, 2024; Chen, 2020).
2. **Barrier misplacement.** *Symptom:* pipeline bubbles, rare data races (SemiAnalysis, 2024; NVIDIA, 2022). *Root cause:* consumer starts before TMA completion signal (Shah *et al.*, 2024). *Fix:* one barrier pair per ping-pong stage; verify with race check (Chen and YiRage Contributors, 2026). *Prevention:* barrier walkthrough in Section 7.4 (SemiAnalysis, 2024; Davies *et al.*, 2025).
3. **Over-staticization.** *Symptom:* some SMs idle on irregular batches (Shah *et al.*, 2024; Mirage Project contributors, 2025). *Root cause:* L3 megakernel on workloads needing tile scheduler (Davies *et al.*, 2025). *Fix:* step down to L2 persistent; keep host graph flexibility (NVIDIA, 2024). *Prevention:* Table 7.5 review before L3 commitment (Mirage Project contributors, 2025; Chen and YiRage Contributors, 2026).
4. **Resource omission.** *Symptom:* spill after enabling ping-pong roles (Chen, 2026a; Chen and YiRage Contributors, 2026). *Root cause:* role plan ignored Ch. 6 double-buffer bytes or register skew (Dao *et al.*, 2022; Chen, 2020). *Fix:* joint residency+role worksheet; Section 7.3 (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). *Prevention:* Pattern 3 on every skew change (Wu *et al.*, 2024; Mirage Project contributors, 2025).

5. **Prefill copy-paste.** *Symptom:* high TFLOPs, poor tokens/sec at batch-1 (Shah *et al.*, 2024; Vellaisamy *et al.*, 2026). *Root cause:* consumer-heavy split on memory-bound decode (Chen, 2026a; Davies *et al.*, 2025). *Fix:* decode worksheet (Table 7.1) (Chen and YiRage Contributors, 2026). *Prevention:* separate decode and prefill benchmark tabs (Kwon *et al.*, 2023; Shah *et al.*, 2024).

Anti-pattern culture. Treat “we copied FA3 config” as a red flag in review (Shah *et al.*, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Require decode ratio worksheet even when prefill kernel exists (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Chen, 2026a). Static roles without measurement are folklore—stall counters or simulation required (Davies *et al.*, 2025; Chen, 2020; Shah *et al.*, 2024).

Debug playbook (expanded). **Stalls rising:** check producer count first, barrier order second, tile size third (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024). **Spill appearing:** rerun Ch. 6 peak with skew; reduce consumer registers or split fusion (Chen, 2026a; Dao *et al.*, 2022; Chen, 2020). **Launches flat:** you optimized inner loop but not staticization level—revisit Table 7.3 (Vellaisamy *et al.*, 2026; NVIDIA, 2024; Mirage Project contributors, 2025). **NPU fail:** compare role map row to GPU row—do not port warp counts literally (AMD, 2024b; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

7.8.2 Static role self-check (review checklist)

- ☐ Producer/consumer ratio computed from load/compute mismatch
- ☐ Register skew within hardware limits; no spill
- ☐ Ping-pong bytes in Ch. 6 peak budget
- ☐ Barrier placements verified; no pipeline bubbles
- ☐ Decode workload regularity matches chosen staticization level
- ☐ Residency + role legality reports archived for target hardware

Postmortem sequence. Replay role worksheet → compare warp stall counters → bisect barrier → re-run tiling pass (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a). Do not tune tile MMA shapes before dispatch and stall replay (Davies *et al.*, 2025; NVIDIA, 2022).

Engineering patterns (quick reference). See Section 7.9 for full Scene/Solution/Benefit/Notes patterns (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025).

7.9 Static Role Engineering Patterns

Pattern 1: Launch-storm diagnosis. **Scene:** TaxBreak-scale launches/token on dense 1B model (Vellaisamy *et al.*, 2026). **Solution:** measure launches; pick L1 graph as stopgap; plan L2 persistent + roles (NVIDIA, 2024; Shah *et al.*, 2024). **Benefit:** separates host overhead from missing fusion (Davies *et al.*, 2025; Chen, 2026a). **Notes:** graph capture does not fix bytes/token (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026).

Pattern 2: Decode ratio worksheet. **Scene:** FA3 prefill config ported to batch-1 decode (Shah *et al.*, 2024; Vellaisamy *et al.*, 2026). **Solution:** Table 7.1 with load/compute cycle estimate (Davies *et al.*, 2025; Chen, 2026a). **Benefit:** prevents consumer-heavy starvation (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). **Notes:** validate stalls, not TFLOPs (Chen, 2026a).

Pattern 3: Joint residency rerun. **Scene:** register skew after warp split (Shah *et al.*, 2024; NVIDIA, 2022). **Solution:** rerun Ch. 6 peak; adjust skew or tile (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Chen, 2020). **Benefit:** catches spill before production (Chen, 2026a). **Notes:** roles and residency are coupled (Wu *et al.*, 2024).

Pattern 4: Triplet role map. **Scene:** CUDA schedule works; XDNA compile fails (AMD, 2024b; Vellaisamy *et al.*, 2026). **Solution:** Table 7.2 for all backends (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020). **Benefit:** fleet-safe IR (AMD, 2024a; NVIDIA, 2022). **Notes:** IR parity $\neq$ role parity (Chen and YiRage Contributors, 2026).

Pattern 5: Ladder documentation. **Scene:** team debates megakernel vs graphs (Mirage Project contributors, 2025; NVIDIA, 2024). **Solution:** Table 7.3 + launches/token in TSV (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). **Benefit:** principled staticization choice (Chen and YiRage Contributors, 2026). **Notes:** L3 needs frozen fusion (Wu *et al.*, 2024; Mirage Project contributors, 2025).

Pattern 6: Tiling pass artifact. **Scene:** hand-tuned warp split lost on model update (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026). **Solution:** CI golden schedule IR from tiling pass (Wu *et al.*, 2024; Chen, 2020). **Benefit:** reproducible roles across versions (Davies *et al.*, 2025; Mirage Project contributors, 2025). **Notes:** pair with Ch. 6 memory-pass goldens (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022).

Pattern usage. Require Pattern 2 on decode PRs; Pattern 4 on IR changes; Pattern 6 on staticization merges (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Shah *et al.*, 2024; Davies *et al.*, 2025; Chen, 2020).

Anti-pattern cross-reference. Map pitfalls to patterns: ratio imbalance → Pattern 2; resource omission → Pattern 3; prefill copy → Pattern 2; triplet fail → Pattern 4 (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024; AMD, 2024b; Davies *et al.*, 2025). Use patterns in design review agendas so discussion stays evidence-based (Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a).

7.10 Role Assignment Review Workflow

Step R1: Staticization level. Pick L1/L2/L3 with launches/token evidence (Vellaisamy *et al.*, 2026; NVIDIA, 2024; Mirage Project contributors, 2025).

Step R2: Ratio worksheet. Complete Table 7.1 or model-specific variant (Shah *et al.*, 2024; Davies *et al.*, 2025).

Step R3: Joint residency check. Rerun Chapter 6 peak with register skew and ping-pong (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Chen, 2020).

Step R4: Tiling pass. Archive schedule IR; verify barrier count (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; SemiAnalysis, 2024).

Step R5: Checklist. Section 7.8 sign-off before Chapter 8 pipeline work (Dao *et al.*, 2022; Davies *et al.*, 2025).

Step R6: Cross-chapter packet. Bundle Ch. 6 peak worksheet + Ch. 7 ratio worksheet + tiling IR + staticization level in one review folder (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Shah *et al.*, 2024). Reviewers should not approve role changes without residency re-certification (Chen, 2026a; Mirage Project contributors, 2025; Chen and YiRage Contributors, 2026).

Review gates. **Gate A—Consumer starvation:** memory warp stalls $\gg$ compute stalls (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026). **Gate B—Spill after roles:** local memory loads post skew (Chen, 2026a; NVIDIA, 2022). **Gate C—Launch regression:** ms/token flat, launches/token unchanged (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). **Gate D—Triplet drift:** GPU schedule fails NPU compile (AMD, 2024b; Chen and YiRage Contributors, 2026; Chen, 2020).

Artifact bundle. Role review packet: staticization level, Table 7.1, Table 7.2 row, tiling IR excerpt, checklist, launches/token screenshot (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Shah *et al.*, 2024; Davies *et al.*, 2025; Chen, 2020). Store beside Ch. 6 residency packets for correlated bisect (Dao *et al.*, 2022; Chen, 2026a; Mirage Project contributors, 2025).

Facilitation. Kernel owners defend ratio math; compiler owners defend pass output; serving owners accept launches/token SLO (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). Disputes resolve with stall counters, not seniority (Shah *et al.*, 2024; Chen, 2020). Escalate to split fusion when joint Ch. 6–7 iteration fails after two pass loops (Wu *et al.*, 2024; Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025; Chen, 2026a).

✓ **Practice checkpoint.** Merge blocked until checklist complete or staticization level deferral documented (Chen and YiRage Contributors, 2026; Chen, 2020).

7.11 Key Takeaways

Static role assignment closes the loop opened in Chapter 6: residency budgets become actor assignments that eliminate per-token dispatch (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024; Mirage Project contributors, 2025; Davies *et al.*, 2025). The principles below pair design rules with actions your team can adopt this sprint (Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a).

- **Static-first principle.** Decode pays scheduling per token—fix roles at compile time when workload is regular (Vellaisamy *et al.*, 2026; Mirage Project contributors, 2025; Davies *et al.*, 2025). Launch storms (847.5/token on 1B dense) are assignment problems as much as fusion problems (Vellaisamy *et al.*, 2026). **Action:** pick L1–L3 level explicitly; measure launches/token before and after (NVIDIA, 2024; Chen and YiRage Contributors, 2026).
- **Ratio-adaptation principle.** Producer/consumer split follows memory/compute mismatch—decode tilts producer-heavy vs prefill (Shah *et al.*, 2024; Chen, 2026a; Kwon *et al.*, 2023). Copying FA3 prefill splits to batch-1 decode is anti-pattern 5 (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). **Action:** complete Table 7.1 for your layer; attach stall counters (Chen and YiRage Contributors, 2026).
- **Resource-linkage principle.** Role assignment changes register residency—joint optimize with Chapter 6 (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020). Skew without peak rerun is how spill sneaks into production (Chen, 2026a; NVIDIA, 2022). **Action:** rerun peak worksheet after every warp skew change (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022).
- **Graded-landing principle.** Higher staticization is not always better—match level to model churn and shape stability (Mirage Project contributors, 2025; Davies *et al.*, 2025; NVIDIA, 2024). L3 megakernels trade flexibility for launch elimination—document the trade (Vellaisamy *et al.*, 2026; Wu *et al.*, 2024). **Action:** record level in benchmark metadata and README (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).
- **Compiler-internalization principle.** Roles belong in YiRage tiling pass output—not tribal CUDA knowledge (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Mirage Project contributors, 2025). Chapter 20 search consumes role IR—hand splits do not compose across model versions (Wu *et al.*, 2024; Chen, 2020; Davies *et al.*, 2025). **Action:** attach schedule IR + checklist to CI artifacts; block drift without worksheet updates (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024; Vellaisamy *et al.*, 2026).

How this chapter connects. Chapter 6 planned bytes; this chapter assigned units; Chapter 8 pipelines movement between them (SemiAnalysis, 2024; Dao *et al.*, 2022; Shah *et al.*, 2024). Chapter 4–5 hardware async features are *used* by roles, not re-taught here (NVIDIA, 2022; AMD, 2024b; Davies *et al.*, 2025).

Chapter 20 megakernel search requires role-annotated IR produced here (Wu *et al.*, 2024; Mirage Project contributors, 2025; Chen and YiRage Contributors, 2026; Chen, 2020).

Megakernel stack preview. Roles (Ch. 7) → async pipelines (Ch. 8) → sync discipline (Ch. 9) → softmax carriers (Ch. 10) → assembly (Ch. 11) (Dao *et al.*, 2022; SemiAnalysis, 2024; Shah *et al.*, 2024; Mirage Project contributors, 2025; Davies *et al.*, 2025). Skipping any layer produces kernels that benchmark well in isolation and fail in fusion (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026).

7.12 Conclusion

Static role assignment spends Chapter 6’s residency budget on a schedule-free structure: every warp, warpgroup, or tile knows its job before the first token (Shah *et al.*, 2024; Mirage Project contributors, 2025; Chen and YiRage Contributors, 2026). The quantitative framework, tri-hardware map, staticization ladder, and tiling pass turn principle into repeatable engineering (Davies *et al.*, 2025; Chen, 2020; Vellaisamy *et al.*, 2026).

Closing metaphor. If Chapter 6 is the warehouse layout, this chapter is the org chart—who unloads trucks (producers) and who assembles orders (consumers) (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). A perfect layout with no job titles still has people standing in the loading bay asking “what next?” each token (Vellaisamy *et al.*, 2026; Mirage Project contributors, 2025). Static roles eliminate that question; Chapter 8 keeps the trucks arriving on time (SemiAnalysis, 2024; Dao *et al.*, 2022).

Chapter 8 addresses the next failure mode: **keeping producers fed**—TMA double-buffer pipelines and async movement so statically assigned consumers never stall waiting for data (SemiAnalysis, 2024; Dao *et al.*, 2022). Carry role worksheets and staticization level into every async-pipeline change (Chen and YiRage Contributors, 2026; Chen, 2026a).

Part IV bridge. Roles (this chapter) + movement (Ch. 8) + sync (Ch. 9) + carriers (Ch. 10) assemble the megakernel stack (Dao *et al.*, 2022; Shah *et al.*, 2024; Mirage Project contributors, 2025). Chapter 20’s search assumes both residency

legality and role-annotated IR—skipping here breaks the compiler story (Wu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020).

Budget–structure recap. Chapter 6: where bytes live and for how long (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020). Chapter 7: which units touch those bytes during each fused stage (Shah *et al.*, 2024; Mirage Project contributors, 2025; Davies *et al.*, 2025). Together they define a deployable decode cell—not just a fast microbenchmark (Vellaisamy *et al.*, 2026; Chen, 2026a).

Recap for practitioners. (1) Pick staticization level. (2) Compute decode producer/consumer ratio. (3) Fill tri-hardware role table. (4) Run tiling pass. (5) Complete checklist (Vellaisamy *et al.*, 2026; Shah *et al.*, 2024; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Mirage Project contributors, 2025; AMD, 2024b; NVIDIA, 2022; SemiAnalysis, 2024; Dao *et al.*, 2022; Chen, 2026a; Wu *et al.*, 2024; NVIDIA, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; Chen, 2020). Residency without roles is bytes planned but work still dispatched every operator (Vellaisamy *et al.*, 2026; Chen, 2026a).

Teaching vs production. Whiteboard producer/consumer timelines (Figure 7.2) for onboarding (Shah *et al.*, 2024; Goodfellow *et al.*, 2016). Automate Patterns A–E in CI for fleet scale (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Mirage Project contributors, 2025). The book shows both: concepts in §7.2–7.5, gates in §7.8–7.10 (Chen, 2020; Vellaisamy *et al.*, 2026).

Summary sentence. Chapter 6 budgeted bytes; Chapter 7 assigned actors; Chapter 8 will synchronize their movement (Dao *et al.*, 2022; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). That sequence is the MegaKernel spine (Mirage Project contributors, 2025; Wu *et al.*, 2024; Chen, 2026a; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2020).

Done criteria. You are done when launches/token, role ratio worksheet, and tiling-pass artifact agree for your decode cell (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Proceed to Chapter 8 with barriers and buffer indices documented (SemiAnalysis, 2024; Dao *et al.*, 2022). Bring staticization level, joint Ch. 6–7 worksheets, and Table 7.2—Chapter 8 assumes roles are fixed and legal (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Shah *et al.*, 2024; Mirage Project contributors, 2025; Chen, 2020).

Hands-on exercises. (1) Measure launches/token on your baseline. (2) Fill ratio worksheet. (3) Pick L1/L2/L3. (4) Run tiling pass. (5) Complete checklist (Vellaisamy *et al.*, 2026; Shah *et al.*, 2024; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Mirage Project contributors, 2025; AMD, 2024b; NVIDIA, 2022; SemiAnalysis, 2024; Dao *et al.*, 2022; Chen, 2026a; Wu *et al.*, 2024; NVIDIA, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; Chen, 2020; AMD, 2024a; Rotem *et al.*, 2018). Compare decode vs prefill stall profiles on the same kernel if you maintain both paths (Kwon *et al.*, 2023; Shah *et al.*, 2024; Vellaisamy *et al.*, 2026).

Fleet note. Triplet deployments need three role maps from one IR—GPU semi-static, CPU weak-static, NPU fully static (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; AMD, 2024a; NVIDIA, 2022; Chen, 2020). Do not ship CUDA-only role metadata to Ryzen AI endpoints (AMD, 2024b; Vellaisamy *et al.*, 2026; Chen, 2026a).

Long-context note. Static roles do not replace KV paging—they optimize per-step operator dispatch (Kwon *et al.*, 2023; Dao *et al.*, 2023). Longer contexts increase KV stream load work—ratio worksheets may need more producer share at high ctx (Davies *et al.*, 2025; SemiAnalysis, 2024; Shah *et al.*, 2024). Re-validate roles per context bucket if you compile bucketed graphs (Chen and YiRage Contributors, 2026; AMD, 2024b; Mirage Project contributors, 2025).

Reader exercises (extended). (6) Compare L1 vs L2 launches/token on the same model. (7) Explain one anti-pattern from your last merge. (8) Sketch producer/consumer timeline for MLP slice (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026; Shah *et al.*, 2024; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Mirage Project contributors, 2025; Kwon *et al.*, 2023; Chen, 2026a; Wu *et al.*, 2024; Chen, 2020). (9) Trace how tiling pass output feeds Chapter 20 search (Wu *et al.*, 2024; Mirage Project contributors, 2025; Chen and YiRage Contributors, 2026).

Part II/IV handoff. Part II established hardware and budgets; Part IV implements movement and sync atop roles (NVIDIA, 2022; AMD, 2024b; Chen, 2020; SemiAnalysis, 2024; Dao *et al.*, 2022; Shah *et al.*, 2024; Mirage Project contributors, 2025; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). Skip role worksheets and later chapters optimize the wrong bottleneck (Vellaisamy *et al.*, 2026; Chen, 2026a).

Final sentence. Static roles turn Chapter 6’s residency plan into a cast of actors; Chapter 8 teaches them to move in sync (Dao *et al.*, 2022; SemiAnalysis, 2024; Shah *et al.*, 2024; Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Wu *et al.*, 2024; Chen, 2020; NVIDIA, 2022; AMD, 2024b; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; NVIDIA, 2024; AMD, 2024a; Rotem *et al.*, 2018; Liu *et al.*, 2024; Dao *et al.*, 2023). Ship the budget–structure packet before you tune async pipelines—otherwise Chapter 8 optimizes movement for the wrong cast (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Shah *et al.*, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Part IV begins with movement only after roles are cast and budgets are closed (Dao *et al.*, 2022; Mirage Project contributors, 2025; Chen, 2026a, 2020). Archive Ch. 6–7 review packets beside benchmark TSV for every staticization milestone (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Shah *et al.*, 2024; Wu *et al.*, 2024). Re-read Section 9.1 before the next fusion review—847.5 launches/token is the problem statement this chapter solves with roles (Vellaisamy *et al.*, 2026; Mirage Project contributors, 2025; Chen, 2026a; NVIDIA, 2024). Enter Chapter 8 with producer/consumer roles fixed and barrier indices written down (SemiAnalysis, 2024; Dao *et al.*, 2022; Shah *et al.*, 2024; Chen and YiRage Contributors, 2026). The budget–structure loop is complete; the movement loop begins in Chapter 8 with TMA double-buffer pipelines (SemiAnalysis, 2024; Dao *et al.*, 2022; Mirage Project contributors, 2025; Davies *et al.*, 2025; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026).

Hands-on recap (one line). Budget (Ch. 6) + cast (Ch. 7) + movement (Ch. 8) = deployable decode megakernel path (Dao *et al.*, 2022; Shah *et al.*, 2024; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a, 2020; Wu *et al.*, 2024). Do not open Chapter 8 until Table 7.1 and the staticization level are signed off in the review packet (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024; Davies *et al.*, 2025; NVIDIA, 2024; Mirage Project contributors, 2025; Chen, 2020; Vellaisamy *et al.*, 2026). That sign-off is the Part IV entry checkpoint for this book’s megakernel implementation track (Mirage Project contributors, 2025; Wu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). Carry the signed packet into every subsequent kernel chapter through Chapter 11 megakernel assembly (Shah *et al.*, 2024; Vellaisamy *et al.*, 2026; Chen, 2020; Mirage Project contributors, 2025; Dao *et al.*, 2022; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). Static roles are the structural contract those chapters execute (Mirage

Project contributors, 2025; SemiAnalysis, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). If your review packet is incomplete, stop here and finish the worksheets now before opening Chapter 8 async pipeline movement (SemiAnalysis, 2024; Dao *et al.*, 2022). Roles without movement planning still stall at runtime today (Shah *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

Chapter 8

TMA Double-Buffer Pipelines and Async Movement

8.1 The Fused-Kernel Ceiling Is the Pipeline

A team fused twelve ops and cut launches to one—then consumers still stalled because loads were synchronous (Shah *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026). Chapter 6 budgeted bytes; Chapter 7 assigned producer/consumer roles; **this chapter makes roles never wait on data** (Chen and YiRage Contributors, 2026; SemiAnalysis, 2024; Dao *et al.*, 2022).

Fusion solves “fewer trips to HBM”; pipelines solve “no idle wait on each trip.” At batch-1 decode, ms/token is set by how fast weight and KV bytes stream, not peak TFLOPs (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026a). **This chapter is the MegaKernel execution pillar**—asynchronous movement that hides latency behind compute (Shah *et al.*, 2024; AMD, 2024a).

Bridge from Chapters 6–7. Pipeline depth consumes fast memory—must joint-check with Ch. 6 peak footprint (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Chen, 2020). Producer warps from Ch. 7 issue async loads; this chapter defines *when* buffers flip and barriers fire (Shah *et al.*, 2024; SemiAnalysis, 2024). Decode metrics methodology: Chapter 3; hardware primitives: Chapters 4–5 (Davies *et al.*, 2025; NVIDIA, 2022; AMD, 2024b).

What you will be able to do. (1) Run three-step pipeline depth decision, (2) size weight vs KV streams independently, (3) fill tri-hardware pipeline table,

(4) interpret YiRage pipeline-pass output, (5) complete pipeline checklist (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024; Vellaisamy *et al.*, 2026).

Organization map. Depth method (§8.2), GPU TMA (§8.3), CPU prefetch (§8.4), NPU DMA (§8.5), YiRage pass (§8.7), joint budget (§8.6), pitfalls (§8.8), patterns (§8.9), review workflow, takeaways, bridge to Chapter 9 (SemiAnalysis, 2024; Davies *et al.*, 2025).

Decode-first thesis. Prefill runs many thick tiles—prologue cost amortizes (Kwon *et al.*, 2023; Shah *et al.*, 2024). Decode runs few thin tiles per token—prologue/epilogue share matters (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026). Do not copy prefill pipeline depth; use Section 8.2 worksheets (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022).

Anti-pattern preview. Section 8.8 lists five pipeline failures—review before TMA merge (Chen and YiRage Contributors, 2026; SemiAnalysis, 2024). Culture: no “enabled double buffer” without depth worksheet and Ch. 6 byte accounting (Dao *et al.*, 2022; Chen, 2020).

Case study sketch (fused attention, batch-1). Baseline: fused kernel with synchronous loads—consumers idle $\sim 40\%$ of cycles waiting on HBM (Shah *et al.*, 2024; Chen, 2026a; Davies *et al.*, 2025). Ping-pong TMA: steady-state hides compute behind next-tile load—consumer idle drops toward barrier overhead only (SemiAnalysis, 2024; NVIDIA, 2022). Triple-buffer attempt: third buffer overflows Ch. 6 shared cap—spill to local memory erases win (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Document each step with stall counters and bytes/token, not FLOPs (Chen, 2026a, 2020).

Budget–structure–execution stack. Chapter 6 asked whether bytes fit; Chapter 7 asked which unit moves them; this chapter asks whether movement overlaps compute (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024; Dao *et al.*, 2022). Illegal depth without residency update is the most common post-fusion regression (Chen, 2026a; Wu *et al.*, 2024; Chen, 2020). Megakernel assembly (Chapters 9–11) assumes all three worksheets are green (Mirage Project contributors, 2025; SemiAnalysis, 2024).

Industrial narrative. Teams celebrate fusion launch drops then plateau on ms/token—often synchronous movement inside the fused kernel (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2026a). FlashAttention-3’s warp specialization only pays when TMA pipelines feed consumers (Shah *et al.*, 2024; SemiAnalysis, 2024). Pipelining is the mechanical sympathy layer between Ch. 7 roles and Ch. 4 hardware (NVIDIA, 2022; Chen and YiRage Contributors, 2026).

Reader paths. **Kernel engineers:** depth worksheet + GPU barrier rules + pitfalls (Shah *et al.*, 2024; Vellaisamy *et al.*, 2026). **Compiler authors:** YiRage pipeline pass I/O + fixed-point loop with memory/tiling (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Mirage Project contributors, 2025). **Fleet operators:** tri-hardware table + per-backend golden IR (AMD, 2024b; Davies *et al.*, 2025; Chen, 2020). All paths end at the checklist before Chapter 9 (SemiAnalysis, 2024; Dao *et al.*, 2022).

Who should read this chapter. Kernel engineers wiring TMA/ping-pong, compiler authors implementing pipeline pass, serving leads diagnosing “fused but slow” decode (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Complete Ch. 6–7 worksheets first—pipeline depth without budget and roles is guesswork (Chen, 2026a; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026).

What this chapter is not. Not a TMA programming manual (Ch. 4), not an XDNA DMA catalog (Ch. 5), not a residency tutorial (Ch. 6) or role-split tutorial (Ch. 7) (NVIDIA, 2022; AMD, 2024b; Chen, 2020). We teach pipeline *decisions* given those inputs (Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

Terminology alignment. **Software pipeline** = explicit stage overlap in time (Dao *et al.*, 2022; SemiAnalysis, 2024). **Ping-pong** = depth-2 double buffer swap (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026). **Prologue/epilogue** = warm-up/drain before/after steady overlap (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). **Stream** = independent operand movement path (weight vs KV) (Kwon *et al.*, 2023; Dao *et al.*, 2023). Use consistently in worksheets and CI logs (Chen, 2020; Vellaisamy *et al.*, 2026).

Bridge to Chapter 9. This chapter places barriers for ping-pong; Chapter 9 generalizes hierarchical sync when many producers, consumers, and tiles coordinate (Vellaisamy *et al.*, 2026; Shah *et al.*, 2024; Davies *et al.*, 2025). Carry mbarrier

indices and stream schedules forward (Chen and YiRage Contributors, 2026; SemiAnalysis, 2024).

Execution pillar in MegaKernel stack. Residency (Ch. 6) minimizes bytes; roles (Ch. 7) eliminate dispatch; pipelines (this chapter) hide latency on remaining bytes (Dao *et al.*, 2022; Shah *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2026a). Skipping pipelines yields “fused but idle” kernels—the most common post-Ch. 7 disappointment in decode serving (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; SemiAnalysis, 2024). Chapter 11 assembly assumes all three layers are co-designed (Mirage Project contributors, 2025; Wu *et al.*, 2024; Chen, 2020).

Quantitative anchor. The Llama-3.2-1B worksheet uses $\sim 400/300$ load/compute cycles—your model will differ, but the *method* is portable (Vellaisamy *et al.*, 2026; Shah *et al.*, 2024; Kwon *et al.*, 2023). Replace numbers; keep steps 1–3 and multi-stream split (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a). Depth without measurement is technical debt (Chen, 2020; Dao *et al.*, 2022).

8.2 Double-Buffer Pipelines: Depth Decision Method

A software pipeline overlaps stage $n+1$ load with stage n compute (Dao *et al.*, 2022; Goodfellow *et al.*, 2016; SemiAnalysis, 2024). **Double-buffering** is the minimum depth (two buffers); deeper pipelines help only until the bottleneck stage saturates (Shah *et al.*, 2024; Chen, 2026a).

Serial vs overlapped baseline. Without pipeline, each tile pays $t_{\text{load}} + t_{\text{compute}}$ serially (Goodfellow *et al.*, 2016; Davies *et al.*, 2025; SemiAnalysis, 2024). With ideal ping-pong on load-bound tiles, steady state pays $\max(t_{\text{load}}, t_{\text{compute}})$ per tile after prologue (Shah *et al.*, 2024; Dao *et al.*, 2022). The gap is the pipeline dividend—bounded by bandwidth, not arithmetic peak (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

When not to pipeline. If $t_{\text{compute}} \gg t_{\text{load}}$, deeper buffers do not help—fix compute or add consumers (Ch. 7) (Shah *et al.*, 2024; Davies *et al.*, 2025). If $N=1$ tile per token and $\text{depth} > 2$, prologue may erase benefit—stay serial or use persistent outer loop (Mirage Project contributors, 2025; NVIDIA, 2024; Vellaisamy

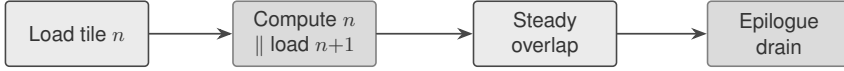


Figure 8.1: Double-buffer pipeline: prologue, steady overlap, epilogue (decode: few tiles $\Rightarrow$ prologue share grows).

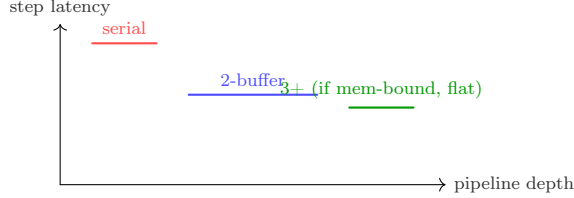


Figure 8.2: Pipeline depth vs per-step latency: deepening stops helping when load-bound (SemiAnalysis, 2024; Chen, 2026a).

et al., 2026). If Ch. 6 cannot fit $2b$ bytes, do not pipeline—split fusion first (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020).

Three-step depth decision. **Step 1—Stage throughput ratio:** measure t_{load} and t_{compute} per tile (Shah *et al.*, 2024; Davies *et al.*, 2025). Bottleneck $B = \max(t_{\text{load}}, t_{\text{compute}})$; steady-state step cost $\approx B$ after prologue (SemiAnalysis, 2024; Dao *et al.*, 2022). **Step 2—Buffer budget:** from Ch. 6 fast-memory cap, subtract resident tensors and sync metadata; divide by tile bytes for max depth D_{max} (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022). **Step 3—Marginal benefit:** with N tiles/token, prologue/epilogue cost $\approx (D - 1) \cdot B$ each; deepen only if $N \gg 2(D - 1)$ and $D \leq D_{\text{max}}$ (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026).

Worked example (Llama-3.2-1B attention, H100, batch-1). Hidden $d=2048$; weight tile ~ 32 KB; est. $t_{\text{load}}=400$ cycles, $t_{\text{compute}}=300$ cycles (Vellaisamy *et al.*, 2026; Shah *et al.*, 2024; Kwon *et al.*, 2023).

Bandwidth floor. Pipelining cannot beat bytes/token bandwidth limit—it removes serial load+compute waste (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Residency (Ch. 6) reduces bytes; pipeline ensures remaining bytes overlap compute (Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

Table 8.1: Pipeline depth worksheet (illustrative decode attention tile).

Item	Value
Bottleneck stage	Load (400 > 300 cycles)
Steady step (2-buffer)	~400 cycles
$D_{\max}$ from Ch. 6 budget	2–4 buffers (tile size dep.)
Tiles per token (thin GEMV)	Small $N \rightarrow$ limit depth
Chosen depth	2 (ping-pong); 3 only if N large enough
Latency hide rate	Compute fully hidden; load still bounds

Table 8.2: Pipeline depth trade-offs (decode).

Depth	Memory cost	Latency win	Decode fit
1 (serial)	Lowest	None—full expose	Only if $N=1$ tiny
2 (ping-pong)	$2 \times$ tile bytes	Hides compute if load-bound	Default decode
3–4	$3\text{--}4 \times$ tile	Diminishing if still BW-bound	Prefill or large N
5+	Often spills	Negative if spill	Avoid decode

Multi-stream rule. Weight and KV streams have different sizes and lifetimes—separate depth per stream (Kwon *et al.*, 2023; Dao *et al.*, 2023; Shah *et al.*, 2024). Unified depth is anti-pattern 4 (Section 8.8) (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Pipeline economics (qualitative).

Attention-layer pipeline sketch. QKV projection: weight stream dominates—depth from weight tile bytes (Vellaisamy *et al.*, 2026; Shah *et al.*, 2024). Attention matmul: KV stream joins—split streams before depth search (Kwon *et al.*, 2023; Dao *et al.*, 2023, 2022). Output projection: similar to QKV—reuse weight stream template (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Document per-subgraph depth in pipeline IR—monolithic depth fails (Chen, 2020; Wu *et al.*, 2024).

MoE and pipeline. Expert weights increase bytes/step—load-bound regime strengthens (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024). Pipeline hides latency but not expert routing dispatch (Ch. 7 L3) (Mirage Project contributors, 2025; Wu

et al., 2024; Davies *et al.*, 2025). Per-expert fusion still needs ping-pong inside expert GEMV (Chen and YiRage Contributors, 2026; Chen, 2026a; Shah *et al.*, 2024).

Quantization interaction. INT4/FP8 weights shrink tile bytes—may reduce $D_{\max}$ need but increase pack/unpack compute (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Re-run step 1 after quant—bottleneck may flip to compute (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2026a). Pipeline pass should re-search when dtype changes (Wu *et al.*, 2024; Chen, 2020).

Benchmarking methodology (Ch. 3 link). Report ms/token at batch-1 with pipeline on/off (Davies *et al.*, 2025; Kwon *et al.*, 2023). Include stall breakdown: memory throttle vs barrier wait (Chen, 2026a; Shah *et al.*, 2024). Include bytes/token—pipeline should not increase bytes (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022). Compare same fusion region—apples-to-apples with Ch. 6 residency held constant (Chen and YiRage Contributors, 2026; Chen, 2020).

Prologue/epilogue accounting (decode). Let N = tiles per token, D = pipeline depth, B = bottleneck stage cycles (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Serial cost $\approx N \cdot (t_{\text{load}} + t_{\text{compute}})$; pipelined cost $\approx (D-1)B + NB + (D-1)B$ (SemiAnalysis, 2024; Dao *et al.*, 2022). When N is small (decode), $(D-1)B$ dominates—shallow D often wins (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Persistent kernels (Ch. 7 L2) amortize prologue across tokens—pair with shallow in-kernel depth (Mirage Project contributors, 2025; NVIDIA, 2024; Shah *et al.*, 2024).

Step 1 procedure (measurement). Profile or model per-tile load latency (TMA completion to mbarrier) and consumer matmul cycles (Shah *et al.*, 2024; Davies *et al.*, 2025). Use Nsight memory throttle vs math throttle warps—not SM occupancy alone (Chen, 2026a; Chen and YiRage Contributors, 2026). Repeat at batch-1 decode operating point—prefill profiles mislead (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026).

Step 2 procedure (budget). From Ch. 6 peak worksheet: M_{fast} = available shared/tile SRAM after residents (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Chen, 2020). Per-buffer bytes b = tile operand footprint; $D_{\max} = \lfloor M_{\text{fast}}/b \rfloor$ for single-stream (Chen, 2026a; SemiAnalysis, 2024). Multi-stream: sum buffer

regions—weight and KV buffers do not share addresses (Kwon *et al.*, 2023; Shah *et al.*, 2024).

Step 3 procedure (marginal benefit). If $t_{\text{load}} > t_{\text{compute}}$, deepening past 2 helps only when extra buffers hide residual load stalls without exceeding D_{max} (SemiAnalysis, 2024; Davies *et al.*, 2025). If $t_{\text{compute}} > t_{\text{load}}$, pipeline depth cannot hide compute—add consumers (Ch. 7) or shrink tile (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026). Reject depth that fails $N \gg 2(D-1)$ for decode layers (Vellaisamy *et al.*, 2026; Chen, 2026a).

Reading the worksheet. Load-bound tile ($\sim 400/300$) $\Rightarrow$ steady step ~ 400 cycles; compute fully hidden (Shah *et al.*, 2024; SemiAnalysis, 2024). Small $N \Rightarrow$ stay at depth 2; depth 3 needs enough tiles to amortize extra prologue (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Cross-check Ch. 7 producer ratio—pipeline depth does not replace producer capacity (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022).

Decode vs prefill (operational). Prefill: large N , thick tiles—deeper pipelines and TMA multicast can help (Kwon *et al.*, 2023; Shah *et al.*, 2024). Decode: small N , thin tiles—prologue share and descriptor overhead cap depth (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026). Maintain separate depth worksheets per regime (Chen and YiRage Contributors, 2026; Chen, 2020).

Latency hide rate. Define hide rate $= 1 - t_{\text{exposed}}/(t_{\text{load}} + t_{\text{compute}})$ under serial baseline (Goodfellow *et al.*, 2016; SemiAnalysis, 2024). Ideal ping-pong on load-bound tiles approaches $t_{\text{compute}}/(t_{\text{load}} + t_{\text{compute}})$ (Shah *et al.*, 2024; Davies *et al.*, 2025). Cannot exceed bandwidth-implied minimum step time (Chen, 2026a; Vellaisamy *et al.*, 2026).

FA3 lessons without FA3 copy-paste. Take async overlap pattern from FlashAttention-3 (Shah *et al.*, 2024). Do not take prefill tile shapes, depth-4 defaults, or multicast schedules for batch-1 decode (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024). Table 8.1 replaces FA3 depth tables for decode (Chen and YiRage Contributors, 2026; Chen, 2026a).

Simulation before silicon. Pipeline worksheets are hypotheses—validate with stall replay or cycle simulation (Davies *et al.*, 2025; Chen and YiRage Contributors,

2026; Shah *et al.*, 2024). Reject configs that improve overlap metrics but not tokens/sec at batch-1 (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2020).

Operator fusion boundary. Pipeline depth applies per fused region—do not assume one depth for whole model (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Dao *et al.*, 2022). Attention subgraph may be load-bound at depth 2; MLP subgraph may be compute-bound at depth 1 (Shah *et al.*, 2024; Davies *et al.*, 2025; Chen, 2026a). Mirage search may propose different fusion boundaries—re-run pipeline pass per candidate (Mirage Project contributors, 2025; Chen, 2020; Vellaisamy *et al.*, 2026).

Carrier tensors and pipelines. Softmax carriers from Ch. 6–7 consume register or smem budget—reduce $D_{\max}$ when carriers are large (Dao *et al.*, 2022; Shah *et al.*, 2024; Chen and YiRage Contributors, 2026). Pipeline buffers and carriers share the same fast-memory envelope (Chen, 2026a; NVIDIA, 2022; AMD, 2024a). Chapter 10 details carrier placement; pipeline pass must reserve carrier home before allocating ping-pong (Dao *et al.*, 2022; Davies *et al.*, 2025; Chen, 2020).

✓ **Practice checkpoint.** Attach depth worksheet before enabling ping-pong in codegen (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Chen, 2020).

8.3 GPU TMA Pipeline Design (Decode)

Hopper **TMA** is the async bulk-movement primitive for producer warps (Ch. 7)—hardware detail in Chapter 4 (NVIDIA, 2022; SemiAnalysis, 2024; Shah *et al.*, 2024). This section covers *pipeline rules*, not ISA catalogs (Chen and YiRage Contributors, 2026).

Producer-consumer handshake. Ch. 7 fixed *who* issues TMA; this section fixes *when* consumers may read (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024). The handshake is always: async issue $\rightarrow$ byte-complete signal $\rightarrow$ consumer matmul $\rightarrow$ buffer flip (NVIDIA, 2022; Dao *et al.*, 2022). Breaking any step yields stalls, races, or silent wrong answers (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2020).

Decode GEMV pipeline shape. Batch-1 decode matmuls are thin—consumer warpgroups may finish before next TMA completes (Davies *et al.*, 2025; Shah



Figure 8.3: GPU decode pipeline: TMA producer fills ping-pong shared buffers for WGMMMA consumers (Ch. 4 primitives).

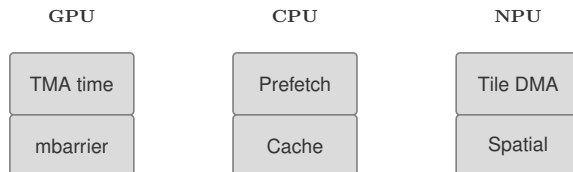


Figure 8.4: Tri-hardware pipeline styles (detail in Ch. 4–5).

et al., 2024; Chen, 2026a). That is acceptable when producers stay ahead; it is failure when consumers wait on mbarrier (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026). Tune producer count (Ch. 7) and pipeline depth jointly—depth alone does not fix insufficient TMA issue bandwidth (Shah *et al.*, 2024; Vellaisamy *et al.*, 2026).

Persistent kernel interaction. Ch. 7 L2 persistent grids loop tokens without relaunch (Mirage Project contributors, 2025; NVIDIA, 2024; Shah *et al.*, 2024). Pipeline prologue runs once per grid launch; steady ping-pong runs each token inside the loop (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). This is how decode amortizes prologue while keeping shallow in-kernel depth (Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022).

L2 multicast caveat (decode). Prefill benefits from multicast when many blocks share weights (Kwon *et al.*, 2023; Shah *et al.*, 2024). Batch-1 decode often sees one consumer block—multicast setup may not repay (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; SemiAnalysis, 2024). Profile bytes moved and descriptor overhead before enabling multicast in decode graphs (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a).

Nsight validation checklist. Report memory throttle vs math throttle warp cycles (Shah *et al.*, 2024; Davies *et al.*, 2025). Report mbarrier wait time per consumer warp (SemiAnalysis, 2024; NVIDIA, 2022). Pipeline success: memory throttle high in producers, math throttle high in consumers, low barrier wait (Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026).

Table 8.3: Tri-hardware pipeline comparison (decode).

Class	Async primitive	Pipeline type	Depth drivers	Sync	Typ. depth
GPU	TMA	Time software	Smem cap, load/compute	mbarrier	2–4
CPU	Prefetch	Implicit cache	Cache, hit rate	Cache fill	3–8 dist.
NPU	Tile DMA	Spatial+time	Tile SRAM, balance	Token	2 + stages

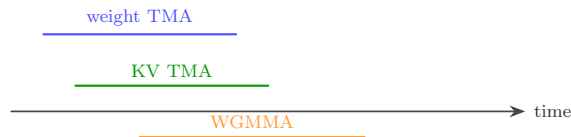


Figure 8.5: Multi-stream overlap: weight and KV loads interleave with compute when buffers are independent.

TMA vs `cp.async` boundary. Very thin decode tiles may pay descriptor overhead with TMA—switch to `cp.async` when tile bytes below platform threshold (profile on target) (Dao *et al.*, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025). TMA wins on bulk multidimensional tiles and single-thread issue freeing registers (Shah *et al.*, 2024; NVIDIA, 2022).

Transaction barrier rules. One mbarrier per ping-pong buffer; producer signals byte count; consumer waits before read (Shah *et al.*, 2024; SemiAnalysis, 2024). Mis-ordered waits cause races or bubbles—mirror Ch. 7 barrier walkthrough (Chen and YiRage Contributors, 2026; Chen, 2020).

Multi-stream TMA scheduling. **Weight stream:** larger tiles, high reuse within layer (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). **KV stream:** grows with context; may need separate buffer pair and depth (Dao *et al.*, 2023; Shah *et al.*, 2024). Assign non-overlapping smem regions—no alias between streams (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022).

Decode caveat. Multicast/L2 promotion wins shrink at batch-1—primary win is asynchrony + register relief, not prefill sharing patterns (Vellaisamy *et al.*, 2026;

SemiAnalysis, 2024; Davies *et al.*, 2025).

TMA vs cp.async decision tree. (1) If tile bytes threshold and issue rate matters $\rightarrow$ cp.async (Dao *et al.*, 2022; Davies *et al.*, 2025). (2) If multidimensional bulk copy with single producer thread $\rightarrow$ TMA (SemiAnalysis, 2024; NVIDIA, 2022). (3) If producer warps are register-starved $\rightarrow$ TMA offloads address math (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026). Profile both on target silicon—thresholds are not portable (Vellaisamy *et al.*, 2026; Chen, 2020).

Barrier walkthrough (A/B ping-pong). Iteration n : producer TMA $\rightarrow$ buffer A; signal `mbarrierA` with byte count (SemiAnalysis, 2024; Shah *et al.*, 2024). Consumer waits `mbarrierA`, WGMMMA on A; concurrently producer loads B (NVIDIA, 2022; Chen and YiRage Contributors, 2026). Iteration $n+1$: swap—consumer on B while producer fills A (Dao *et al.*, 2022; SemiAnalysis, 2024). Barrier indices must alternate consistently—off-by-one causes silent corruption (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Avoiding pipeline bubbles. Bubble = consumer ready but buffer not signaled—usually insufficient producer warps (Ch. 7) or TMA queue stall (Shah *et al.*, 2024; Chen, 2026a). Deadlock = consumer waits wrong barrier or producer signals before copy completes—audit emit order in pipeline IR (Chen and YiRage Contributors, 2026; SemiAnalysis, 2024; NVIDIA, 2022). Target 5% consumer stall from barriers in steady state (Davies *et al.*, 2025; Shah *et al.*, 2024).

Weight vs KV buffer map. Reserve disjoint smem regions: e.g. weight ping-pong at offset 0, KV ping-pong at offset $2w$ (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Kwon *et al.*, 2023). Pipeline pass emits map as IR metadata—codegen must not reorder (Chen, 2020; Wu *et al.*, 2024). KV growth with context may force smaller KV tiles or separate depth—re-run worksheet per context bucket (Dao *et al.*, 2023; Davies *et al.*, 2025).

WGMMMA consumer timing. Consumers start only after `mbarrier` proves bytes resident—premature WGMMMA reads garbage (Shah *et al.*, 2024; SemiAnalysis, 2024). Schedule consumer warpgroups so `matmul` duration $\approx$ next TMA duration in steady state (Chen, 2026a; Chen and YiRage Contributors, 2026). Thin decode GEMV may finish before TMA—acceptable if producer keeps ahead (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Table 8.4: Cross-hardware decode pipeline effect (same Llama-class layer, illustrative).

Metric	GPU H100	CPU server	NPU XDNA
Async primitive	TMA	SW prefetch	Tile DMA
Typ. depth	2 ping-pong	dist. 4–6	2 + spatial
Consumer stall (serial)	High	High	High
After pipeline	Low (load-bound)	LLC-bound	MAC-fed
Primary limit	HBM BW	DRAM BW	DDR + tile BW

✓ **Practice checkpoint.** Document weight vs KV buffer map and mbarrier indices on GPU pipeline PRs (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026).

8.4 CPU Async Pipeline Design (Decode)

CPU has no programmer scratchpad—pipeline is **prefetch-shaped** (Ch. 6 blocking) (Goodfellow *et al.*, 2016; Davies *et al.*, 2025; Chen, 2020).

Why CPU pipelines differ. GPUs name buffers in shared memory; CPUs rely on cache hierarchy as implicit buffers (Goodfellow *et al.*, 2016; NVIDIA, 2022; SemiAnalysis, 2024). You cannot insert mbarrier—you tune prefetch distance and blocking so LLC hits overlap SIMD work (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020). Decode on CPU is almost always DRAM-bandwidth bound—pipeline wins are smaller but still measurable at batch-1 (Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).

Weight streaming on CPU. Dense 1B decode reads full weight columns per token—sequential streams favor hardware prefetchers (Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Blocked micro-tiles from Ch. 6 must fit LLC—otherwise each tile is a cold miss and pipeline collapses (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Chen, 2026a). Co-locate weight shards with compute threads—NUMA mistakes look like “pipeline does not work” (Vellaisamy *et al.*, 2026; Chen, 2020).

KV streaming on CPU. PagedAttention scatters KV—prefetch trains break (Kwon *et al.*, 2023; Dao *et al.*, 2023; Davies *et al.*, 2025). CPU decode pipelines often accept lower KV reuse—focus prefetch on hot weight streams first (Chen and

YiRage Contributors, 2026; Chen, 2026a). Re-run prefetch distance when page layout changes (Chen, 2020; Vellaisamy *et al.*, 2026).

Prefetch distance formula (sketch). Let L = memory latency (cycles), C = compute per iteration (Goodfellow *et al.*, 2016; Chen, 2026a). Prefetch distance $d \approx \lceil L/C \rceil$ iterations ahead—tune empirically (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). Too small: stalls; too large: cache pollution/eviction (Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026).

NUMA-aware pipelining. Partition weight/KV blocks per socket; reader threads prefetch local pages only (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026a). Cross-NUMA prefetch breaks pipeline—remote latency dominates (Vellaisamy *et al.*, 2026; Chen, 2020).

SIMD alignment. Prefetch cache-line granularity; align blocks to SIMD width to avoid partial-line waste (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). Sequential contiguous weight streams maximize hardware prefetcher tracking (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

Explicit DMA (when available). Some server CPUs offer async copy engines—explicit double-buffer between DRAM and compute buffers (Goodfellow *et al.*, 2016). Validate with bandwidth counters; implicit prefetch often suffices for decode GEMV (Davies *et al.*, 2025; Chen, 2026a).

CPU pipeline as implicit software pipeline. No programmer-visible ping-pong—L1/L2/LLC act as hidden buffers (Goodfellow *et al.*, 2016; Chen, 2020). **Design rule:** keep working set LLC slice per socket; sequential access patterns maximize hit rate (Davies *et al.*, 2025; Chen, 2026a). Blocking factors from Ch. 6 must fit cache hierarchy—oversized tiles evict pipeline stages (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022).

Prefetch distance worked example. Suppose $L=300$ cycles DRAM latency, $C=40$ cycles per SIMD micro-tile (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). $d \approx \lceil 300/40 \rceil = 8$ iterations ahead—start with 4, binary search for LLC miss minimum (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). Decode GEMV with small N : prologue analog is cold-cache first tiles—warm persistent threads help (Mirage Project contributors, 2025; Davies *et al.*, 2025).

NUMA pinning procedure. (1) Pin reader threads to socket owning weight shard (Kwon *et al.*, 2023; Davies *et al.*, 2025). (2) Allocate KV pages local to compute socket when possible (Chen, 2026a). (3) Never prefetch across sockets for hot loop (Vellaisamy *et al.*, 2026; Chen, 2020). Remote DRAM doubles effective L —depth worksheet must use measured remote latency (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

SIMD + prefetch alignment. Use `_mm_prefetch` or compiler hints at cache-line boundaries (Goodfellow *et al.*, 2016). Align micro-tile rows to AVX-512 width (64 bytes) to avoid unaligned loads breaking prefetch train (Davies *et al.*, 2025). Weight matrices in decode are often row-major streams—favor contiguous producer loops (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

CPU vs GPU pipeline contrast. GPU: explicit smem ping-pong + mbarrier (SemiAnalysis, 2024; Shah *et al.*, 2024). CPU: implicit cache pipeline + software prefetch distance (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). Same depth worksheet math applies—only buffer tier and sync mechanism differ (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026).

✓ **Practice checkpoint.** Measure LLC miss rate and NUMA remote % at decode operating point (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

8.5 NPU Spatial Pipeline Design (Decode)

XDNA pipelines are **mandatory explicit DMA**—no cache fallback (Ch. 5) (AMD, 2024b,a; Chen and YiRage Contributors, 2026).

Why NPU pipelines are non-optional. GPU can fall back to synchronous global loads (slow); NPU MAC array stalls if DMA does not refill tile SRAM (AMD, 2024a,b; Davies *et al.*, 2025). Every fused decode cell on XDNA needs a documented DMA schedule—not an optional optimization (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a). Chapter 5 documents DMA mechanics; this section documents scheduling policy (AMD, 2024b; Vellaisamy *et al.*, 2026).

Time vs space pipeline. Temporal ping-pong: buffer A/B on one tile (AMD, 2024a; Dao *et al.*, 2022). Spatial pipeline: stage k on tile row k , data flows neighbor-to-neighbor (AMD, 2024b; Davies *et al.*, 2025). Large fused regions may

need both—YiRage pipeline pass selects mix per subgraph (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Chen, 2020).

Ping-pong on tile SRAM. DMA fills buffer B while MAC consumes A; completion tokens sequence swap (AMD, 2024a,b; Dao *et al.*, 2022). $2 \times$ operand bytes in Ch. 6 peak—depth usually 2 on tile (Chen and YiRage Contributors, 2026; Chen, 2026a).

Spatial stage balancing. Split compute across tiles so per-stage latency is balanced—slow stage idles downstream array (AMD, 2024b; Davies *et al.*, 2025). Fuse light stages; split heavy ops across tiles (AMD, 2024a; Chen, 2020).

DMA priority queues. Weights (critical path) $>$ KV stream $>$ debug/scratch (AMD, 2024b; Dao *et al.*, 2023; Shah *et al.*, 2024). Mis-prioritized queues starve MAC array (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Multi-tile cascade. Use neighbor tile links for assembly-line stages—intermediate tensors stay on-chip (AMD, 2024a,b; Dao *et al.*, 2022). Watch stream-switch bandwidth—do not oversubscribe column links (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026).

Spatial balance procedure. (1) Profile per-stage cycles on single tile (AMD, 2024b; Davies *et al.*, 2025). (2) Identify slowest stage—downstream tiles idle waiting (AMD, 2024a; Chen and YiRage Contributors, 2026). (3) Split heavy op across tiles or fuse light ops with neighbor (Chen, 2020). (4) Re-run DDR bytes/token—spatial imbalance shows as MAC util gap (Chen, 2026a; Vellaisamy *et al.*, 2026).

DMA queue design. Priority 1: weight tiles for current layer (Vellaisamy *et al.*, 2026; AMD, 2024b). Priority 2: KV slices for active context (Kwon *et al.*, 2023; Dao *et al.*, 2023). Priority 3: scratch/intermediate spill (avoid if possible) (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Starvation of priority 1 is anti-pattern—profile DMA completion vs MAC start (AMD, 2024a; Shah *et al.*, 2024).

Decode on NPU specifics. Thin batch-1 tiles underfill MAC arrays—spatial pipeline keeps array busy while DMA refills SRAM (Davies *et al.*, 2025; AMD,

2024a). Depth rarely exceeds 2 on tile SRAM—invest in stage balance over deeper time pipeline (Chen and YiRage Contributors, 2026; Chen, 2026a; AMD, 2024b). Chapter 5 covers XDNA DMA mechanics; here we fix scheduling policy (AMD, 2024b; Chen, 2020).

NPU vs GPU pipeline contrast. GPU overlaps time stages in one SM with smem (SemiAnalysis, 2024; NVIDIA, 2022). NPU overlaps space stages across tiles *and* time ping-pong on each tile (AMD, 2024a,b). YiRage lowers one pipeline IR to both—backend pass selects spatial vs temporal schedule (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Davies *et al.*, 2025).

✓ **Practice checkpoint.** Verify DDR bytes/token drops when spatial pipeline is enabled (Chen, 2026a; AMD, 2024a).

8.6 Joint Budget with Chapters 6–7

Pipeline depth is not independent of residency or roles (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Shah *et al.*, 2024; Chen, 2020).

Why triple coupling exists. Depth multiplies fast-memory bytes; roles decide who touches each buffer; residency caps total bytes (Chen, 2026a; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). Changing any one without revisiting the others is the dominant source of “mysterious spill” bugs after fusion merges (Chen, 2020; Wu *et al.*, 2024; Vellaisamy *et al.*, 2026). Treat Ch. 6–8 as one optimization surface—not three tickets (Mirage Project contributors, 2025; Davies *et al.*, 2025; Shah *et al.*, 2024).

Memory pass coupling. Each buffer $\times$ depth adds bytes to Ch. 6 peak—deepening without worksheet update causes spill (Chen, 2026a; Chen and YiRage Contributors, 2026). Pipeline pass must reject depths that overflow memory-pass cap (Wu *et al.*, 2024; Chen, 2020).

Role pass coupling. Only producer roles issue async copies—pipeline IR references role IDs from Ch. 7 tiling pass (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025). Changing roles without regenerating pipeline breaks barrier mapping (SemiAnalysis, 2024; Davies *et al.*, 2025).

Fixed-point loop. Memory $\rightarrow$ tiling $\rightarrow$ pipeline $\rightarrow$ re-validate memory; split fusion if no fixed point (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Mirage Project contributors, 2025; Dao *et al.*, 2022).

Worked joint example. Ch. 6 peak allows ~ 96 KB ping-pong after residents; tile 32 KB $\Rightarrow D_{\max}=2$ single-stream (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Adding KV stream second pair needs ~ 64 KB more—may exceed cap $\Rightarrow$ shrink tile or split fusion (Kwon *et al.*, 2023; Chen, 2026a; Shah *et al.*, 2024). Ch. 7 assigned 4 producer warps—pipeline pass must emit TMA order matching those role IDs (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026).

Triple legality reports. Ship one bundle: memory legality (Ch. 6), role legality (Ch. 7), pipeline legality (Ch. 8) (Chen, 2020; Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). CI should fail if any report drifts without coordinated regen (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Mirage Project contributors, 2025).

Spill as joint failure. Spill after pipeline merge means depth or stream map violated Ch. 6—not a codegen bug to patch with local memory (Chen, 2026a; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). Rollback depth first; split fusion second; never silently spill (Chen, 2020; Wu *et al.*, 2024).

✓ **Practice checkpoint.** Ship Ch. 6 peak + Ch. 7 role + Ch. 8 depth worksheets together (Chen, 2020; Vellaisamy *et al.*, 2026).

8.7 YiRage Pipeline Generation Pass

YiRage emits per-target async pipelines after tiling pass, before codegen (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024; Wu *et al.*, 2024).

Pass position in lowering stack. Order: fusion search $\rightarrow$ memory pass (Ch. 6) $\rightarrow$ tiling pass (Ch. 7) $\rightarrow$ **pipeline pass** $\rightarrow$ codegen $\rightarrow$ runtime (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Mirage Project contributors, 2025; Chen, 2020). Skipping pipeline pass emits synchronous loads inside otherwise fused kernels—common regression after manual CUDA merges (Shah *et al.*, 2024; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Chapter 20 megakernel search re-invokes pipeline

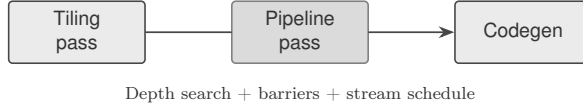


Figure 8.6: YiRage pipeline pass after role assignment (Ch. 7), before codegen.

Table 8.5: YiRage pipeline pass: I/O and capabilities.

Item	Content	On failure
Input	Fused subgraph + residency + role metadata	—
Output	Pipeline IR: buffers, depths, barriers, emit order	Error + split hint
Search	Per-stream depth, tile schedule, prologue/epilogue	Reduce depth or split
Coupling	Memory pass overflow → backoff depth	Document reason
Fallback	TMA → cp.async if threshold	Emit downgrade note

pass inside candidate evaluation (Mirage Project contributors, 2025; Wu *et al.*, 2024; Chen and YiRage Contributors, 2026).

IR fields (conceptual). Per stream: depth D , buffer offsets, tile schedule, prologue/epilogue counts (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; SemiAnalysis, 2024). Per barrier: index, expected byte count, producer role id, consumer role id (Shah *et al.*, 2024; NVIDIA, 2022; Chen, 2020). Per backend: chosen primitive (TMA, cp.async, prefetch, tile DMA) (AMD, 2024b; Goodfellow *et al.*, 2016; Davies *et al.*, 2025).

Closed loop with Mirage. Superoptimizer proposes fusion; memory certifies footprint; tiling assigns roles; pipeline overlaps movement (Wu *et al.*, 2024; Mirage Project contributors, 2025; Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). Chapter 20 consumes full IR stack (Mirage Project contributors, 2025; Chen, 2020).

Prologue/epilogue generation. Pass emits warm-up and drain phases; persistent kernels (Ch. 7 L2) amortize across tokens (Shah *et al.*, 2024; Mirage Project contributors, 2025; NVIDIA, 2024; Davies *et al.*, 2025).

Regression tests. Golden pipeline IR per canonical layer; CI fails on depth/barrier drift (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Vellaisamy *et al.*, 2026).

Pass search algorithm (sketch). For each stream s : enumerate $D \in \{2, \dots, D_{\max}\}$ (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022). Score by simulated steady-step cycles + prologue penalty for decode N (Davies *et al.*, 2025; SemiAnalysis, 2024). Reject candidates failing memory pass; pick Pareto-min step time (Wu *et al.*, 2024; Chen, 2020, 2026a). Emit prologue/epilogue skeleton and barrier graph (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026).

Collaboration with memory pass. On overflow: (a) reduce D , (b) shrink tile, (c) split subgraph—log chosen branch in IR comment (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Mirage Project contributors, 2025). Memory pass may increase tile if depth drops—re-run search (Dao *et al.*, 2022; Chen, 2020).

Collaboration with tiling pass. Pipeline IR references producer role IDs for emit sites (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024). If tiling changes warp count, regenerate pipeline—barrier fan-in changes (NVIDIA, 2022; Davies *et al.*, 2025).

Backend lowering hooks. GPU: TMA descriptors + mbarrier indices (SemiAnalysis, 2024; NVIDIA, 2022). CPU: prefetch intrinsics + loop peel for prologue (Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026). NPU: DMA task list + token sync (AMD, 2024b,a; Chen, 2020). Chapter 20 megakernel search treats pipeline pass as mandatory stage (Mirage Project contributors, 2025; Wu *et al.*, 2024).

Fallback and downgrade. Unsupported TMA on older GPU $\rightarrow$ cp.async pipeline with adjusted depth (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). Emit downgrade note in build log for fleet audits (Chen, 2020; Vellaisamy *et al.*, 2026).

✓ **Practice checkpoint.** Archive pipeline-pass excerpt on every async movement PR (Chen and YiRage Contributors, 2026; Chen, 2020).

8.8 Pipeline Pitfalls, Checklist, and Debug

8.8.1 Five common pipeline anti-patterns

1. **Depth excessive.** *Symptom:* Ch. 6 spill after adding buffers (Chen, 2026a; Chen and YiRage Contributors, 2026). *Root cause:* depth up without byte budget (Dao *et al.*, 2022; Chen, 2020). *Fix:* reduce depth or split fusion (Wu *et al.*, 2024). *Prevention:* step 2 of depth method; Pattern 1 mandatory (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).
2. **Depth insufficient.** *Symptom:* consumer stalls on mbarrier (Shah *et al.*, 2024; Davies *et al.*, 2025). *Root cause:* synchronous load path or depth=1 (SemiAnalysis, 2024). *Fix:* enable ping-pong; add producer capacity (Ch. 7) (Chen and YiRage Contributors, 2026). *Prevention:* measure t_{load} vs t_{compute} before merge (Chen, 2026a; Shah *et al.*, 2024).
3. **Sync errors.** *Symptom:* races, deadlock, bubbles (Shah *et al.*, 2024; NVIDIA, 2022). *Root cause:* barrier misplacement or byte-count mismatch (SemiAnalysis, 2024). *Fix:* one barrier pair per buffer stage; golden IR diff (Chen and YiRage Contributors, 2026; Chen, 2020). *Prevention:* barrier walkthrough in Section 8.3 (SemiAnalysis, 2024; Davies *et al.*, 2025).
4. **Single-stream parameters.** *Symptom:* KV path idle while weight path congested (Kwon *et al.*, 2023; Dao *et al.*, 2023). *Root cause:* unified depth for heterogeneous streams (Davies *et al.*, 2025). *Fix:* independent depth per stream; Figure 8.5 (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). *Prevention:* Pattern 2 on all attention fusion PRs (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026).
5. **Prefill copy-paste.** *Symptom:* deep pipeline, few tiles, slow decode (Shah *et al.*, 2024; Vellaisamy *et al.*, 2026). *Root cause:* high prologue share; large D with small N (Davies *et al.*, 2025; Chen, 2026a). *Fix:* shallow depth + persistent kernel (Ch. 7 L2) (Mirage Project contributors, 2025; NVIDIA, 2024). *Prevention:* separate decode/prefill depth worksheets (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

Anti-pattern culture. Treat “we enabled double buffer” without worksheet as review blocker (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Chen, 2020). Celebrate overlap metrics only with tokens/sec at batch-1 (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a). Pipeline depth is not a merit badge—it is a budgeted trade (Chen, 2026a; SemiAnalysis, 2024; Shah *et al.*, 2024).

8.8.2 Pipeline design self-check

- ☐ Per-tile t_{load} , t_{compute} measured; bottleneck named
- ☐ Depth fits Ch. 6 fast-memory budget (ping-pong counted)
- ☐ Weight and KV streams have independent depth/buffers
- ☐ Barriers verified—no deadlock or excess bubbles
- ☐ Prologue/epilogue share assessed for decode tile count N
- ☐ Residency + role + pipeline legality reports archived

Postmortem. Replay depth worksheet $\rightarrow$ stall counters $\rightarrow$ bytes/token $\rightarrow$ bisect barrier or depth (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a).

Debug playbook. **Stalls high, bytes/token flat:** depth insufficient or sync bubbles—check mbarrier timeline (SemiAnalysis, 2024; Davies *et al.*, 2025). **Spill after merge:** depth excessive—reconcile with Ch. 6 (Chen, 2026a; Chen and YiRage Contributors, 2026). **Correctness bug:** barrier order or buffer alias—dump pipeline IR (Chen, 2020; Shah *et al.*, 2024). **Decode regression only:** compare N and prologue share vs prefill config (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).

8.9 Pipeline Engineering Patterns

Pattern 1: Depth worksheet first. **Scene:** “enable double buffer” PR without numbers (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). **Solution:** Table 8.1 + Ch. 6 peak + measured $t_{\text{load}}/t_{\text{compute}}$ (Chen, 2020; Shah *et al.*, 2024). **Benefit:** avoids spill and false depth (Chen, 2026a; Vellaisamy *et al.*, 2026). **Notes:** depth $\neq$ always 2; decode N caps D (Davies *et al.*, 2025; SemiAnalysis, 2024). **When:** every pipeline PR; first gate in review (Chen and YiRage Contributors, 2026; Chen, 2020).

Pattern 2: Split streams. **Scene:** long-context KV dominates attention step (Kwon *et al.*, 2023; Dao *et al.*, 2023). **Solution:** independent KV pipeline depth and smem region (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026).

Benefit: weight path no longer starves KV or vice versa (Davies *et al.*, 2025; Chen, 2026a). **Notes:** Figure 8.5; re-run per context bucket (SemiAnalysis, 2024; Vellaisamy *et al.*, 2026). **When:** paged KV or variable context serving (Kwon *et al.*, 2023; NVIDIA, 2024).

Pattern 3: Persistent warm pipeline. **Scene:** prologue tax visible every token at batch-1 (Mirage Project contributors, 2025; Davies *et al.*, 2025). **Solution:** Ch. 7 L2 persistent grid + shallow in-kernel ping-pong (Shah *et al.*, 2024; NVIDIA, 2024). **Benefit:** amortize warm-up across token loop (Vellaisamy *et al.*, 2026; Chen, 2026a). **Notes:** pairs Ch. 7 staticization ladder; do not deepen kernel to fix launch tax (Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025). **When:** ms/token flat after fusion but launches already at 1 (Davies *et al.*, 2025; Dao *et al.*, 2022).

Pattern 4: Triplet pipeline row. **Scene:** GPU TMA schedule fails on NPU or CPU backend (AMD, 2024b; Vellaisamy *et al.*, 2026). **Solution:** Table 8.3 + Table 8.4 per target (Chen and YiRage Contributors, 2026; Chen, 2020). **Benefit:** fleet-safe IR; same depth math (Davies *et al.*, 2025; AMD, 2024a). **Notes:** primitives differ; worksheets identical (NVIDIA, 2022; Goodfellow *et al.*, 2016; AMD, 2024b). **When:** multi-backend YiRage builds (Wu *et al.*, 2024; Chen and YiRage Contributors, 2026).

Pattern 5: Pipeline pass golden. **Scene:** hand-edited mbarrier index lost on rebase (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026). **Solution:** CI golden pipeline IR per canonical layer (Wu *et al.*, 2024; Chen, 2020). **Benefit:** reproducible barriers and depth (SemiAnalysis, 2024; Vellaisamy *et al.*, 2026). **Notes:** pair Ch. 6 memory and Ch. 7 tiling goldens (Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025). **When:** any change to pipeline pass or barrier lowering (Chen, 2020; Shah *et al.*, 2024).

Pattern 6: Tri-hardware regression row. **Scene:** GPU pipeline lands; NPU build breaks DMA schedule (AMD, 2024b; Vellaisamy *et al.*, 2026). **Solution:** Table 8.4 + per-backend golden (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). **Benefit:** fleet parity (Chen, 2020). **Notes:** same depth math, different primitives (AMD, 2024a; NVIDIA, 2022). **When:** every multi-backend release train (Wu *et al.*, 2024; Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025).

Pattern catalog summary. Patterns 1–2 are decode attention essentials (Dao *et al.*, 2022; Kwon *et al.*, 2023; Shah *et al.*, 2024). Pattern 3 pairs Ch. 7 persistent kernels (Mirage Project contributors, 2025; NVIDIA, 2024). Patterns 4–6 are fleet/CI hygiene (Davies *et al.*, 2025; Chen, 2020; Vellaisamy *et al.*, 2026). Adopt all six before claiming pipeline maturity (Chen and YiRage Contributors, 2026; Chen, 2026a; SemiAnalysis, 2024).

Pattern usage. Pattern 1 on all pipeline PRs; Pattern 2 on context-sensitive models; Pattern 5–6 in CI (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Shah *et al.*, 2024; Davies *et al.*, 2025).

8.10 Pipeline Review Workflow

R1–R5. R1: depth worksheet; R2: Ch. 6 peak with buffers; R3: Ch. 7 role map; R4: pipeline pass IR; R5: checklist (Chen and YiRage Contributors, 2026; Chen, 2020; Shah *et al.*, 2024; Dao *et al.*, 2022). **Gate A:** spill after pipeline merge (Chen, 2026a). **Gate B:** mbarrier stalls $\gg$ compute (Shah *et al.*, 2024; Davies *et al.*, 2025). **Gate C:** bytes/token flat (Vellaisamy *et al.*, 2026; Chen, 2026a).

Reviewer questions. Does depth worksheet match measured $t_{\text{load}}/t_{\text{compute}}$? (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026) Are weight and KV buffers disjoint in IR? (Kwon *et al.*, 2023; Dao *et al.*, 2023) Is decode N documented for prologue decision? (Vellaisamy *et al.*, 2026; Chen, 2026a) Does CI include golden pipeline IR diff? (Wu *et al.*, 2024; Chen, 2020) Is downgrade path logged for older GPUs? (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026)

Sign-off bundle. Attach: Table 8.1, buffer map, mbarrier index table, pipeline-pass JSON excerpt, checklist screenshot (Chen and YiRage Contributors, 2026; SemiAnalysis, 2024; Shah *et al.*, 2024; Vellaisamy *et al.*, 2026; Chen, 2020).

Review timeline. Day 0: author completes depth worksheet and pipeline IR (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022). Day 1: reviewer verifies Ch. 6 peak includes $D \times b$ bytes (Chen, 2026a, 2020). Day 2: silicon owner confirms stall profile matches bottleneck hypothesis (Shah *et al.*, 2024; Davies *et al.*, 2025). Merge only after Gate A–C green (Vellaisamy *et al.*, 2026; Wu *et al.*, 2024; Mirage Project contributors, 2025).

Escalation paths. If Gate A (spill): reduce depth or split fusion—do not widen tile (Chen, 2026a; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022). If Gate B (stalls): add producers (Ch. 7) or fix barriers—do not deepen blindly (Shah *et al.*, 2024; SemiAnalysis, 2024). If Gate C (bytes flat): pipeline is not your bottleneck—return to Ch. 6 residency (Vellaisamy *et al.*, 2026; Chen, 2020; Davies *et al.*, 2025).

✓ **Practice checkpoint.** Block merge without triple worksheet bundle (Ch. 6–8) (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026).

8.11 Decode Pipeline Benchmarking Methodology

Pipeline changes must be judged with the same decode-first metrics as Chapter 3—not isolated kernel FLOPs (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026).

Minimum benchmark bundle. Run batch-1 ms/token with pipeline enabled vs disabled on the *same* fused region and Ch. 6 residency plan (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Chen, 2026a). Record bytes/token (should be unchanged), launches/token (unchanged if already fused), and stall breakdown (Vellaisamy *et al.*, 2026; Shah *et al.*, 2024; Chen, 2020). Pipeline success is visible in consumer stall reduction, not TFLOPs counters (Davies *et al.*, 2025; SemiAnalysis, 2024).

GPU stall metrics. Export Nsight Compute: memory throttle on producer warps, math throttle on consumer warps, mbarrier wait cycles (Shah *et al.*, 2024; NVIDIA, 2022; Chen and YiRage Contributors, 2026). Healthy ping-pong: producers memory-bound, consumers math-bound, barrier waits 5% of consumer cycles (SemiAnalysis, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026). If barrier waits dominate, fix sync before deepening buffers (Davies *et al.*, 2025; Chen, 2020).

CPU cache metrics. LLC miss rate per decode step; remote NUMA access percentage (Goodfellow *et al.*, 2016; Davies *et al.*, 2025; Chen, 2026a). Prefetch win shows as miss rate drop without bytes/token increase (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). If misses flat, blocking factor or NUMA placement failed—not prefetch distance tuning alone (Chen, 2020; Dao *et al.*, 2022).

NPU DDR metrics. DDR bytes per token with spatial+time pipeline on vs off (AMD, 2024b,a; Chen, 2026a). MAC utilization estimate from cycle counters—imbalanced spatial stages show low util despite low DDR (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; AMD, 2024a). Pipeline should increase MAC busy fraction without increasing DDR (Vellaisamy *et al.*, 2026; Chen, 2020).

A/B discipline. Never compare pipelined prefill kernel to non-pipelined decode path (Kwon *et al.*, 2023; Shah *et al.*, 2024; Vellaisamy *et al.*, 2026). Never compare different quantizations or context lengths in the same chart (Davies *et al.*, 2025; Dao *et al.*, 2022). Document N , D , and stream split in benchmark metadata (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a).

Regression CI hooks. Golden pipeline IR diff on canonical layers (Wu *et al.*, 2024; Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). Optional silicon smoke: stall threshold gates on nightly H100 + Ryzen AI cells (AMD, 2024b; NVIDIA, 2022; Davies *et al.*, 2025). Fail CI on barrier index drift without worksheet update (SemiAnalysis, 2024; Chen, 2020; Vellaisamy *et al.*, 2026).

Worked reporting template. Report: model, layer, batch, context, t_{load} , t_{compute} , chosen D , weight/KV depths, ms/token delta, stall delta (Vellaisamy *et al.*, 2026; Shah *et al.*, 2024; Chen and YiRage Contributors, 2026). Attach Table 8.1 and buffer map (Dao *et al.*, 2022; Chen, 2020). This template is what reviewers expect in Pattern 1 PRs (Davies *et al.*, 2025; Chen, 2026a; Mirage Project contributors, 2025).

Fleet triplet parity. Run the same template row on GPU, CPU, and NPU backends from one YiRage IR (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; AMD, 2024b; NVIDIA, 2022). Depth math should match; only primitive columns differ in Table 8.3 (Chen, 2020; Vellaisamy *et al.*, 2026; AMD, 2024a). Ship all three rows or document explicit backend deferral (Wu *et al.*, 2024; Mirage Project contributors, 2025).

When pipeline is not the lever. If bytes/token is high, return to Ch. 6 residency before more buffering (Chen, 2026a; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). If launches/token is high, return to Ch. 7 staticization before overlap tuning (Vellaisamy *et al.*, 2026; Mirage Project contributors, 2025; NVIDIA, 2024). Pipeline optimizes timing of *legal* bytes—not volume of bytes or dispatch count (Davies *et al.*, 2025; Shah *et al.*, 2024; SemiAnalysis, 2024).

Table 8.6: Pipeline optimization PR evidence template.

Field	Required content
Worksheet	Table 8.1 filled
Streams	Weight depth; KV depth; buffer map
Stalls	Before/after consumer wait cycles
ms/token	Batch-1, same fusion region
Artifacts	Pipeline IR hash; checklist

Microbenchmark trap. Isolated TMA or prefetch microbenchmarks can show huge speedups that disappear in fusion—barriers, prologue, and stream interaction dominate real kernels (Shah *et al.*, 2024; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026). Always benchmark inside the fused subgraph with Ch. 7 roles enabled (Mirage Project contributors, 2025; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). If microbenchmark and fused cell disagree, trust the fused cell (Chen, 2026a, 2020; Dao *et al.*, 2022).

Context-length sweeps. KV stream bytes grow with context—depth worksheets at 4K context may fail at 128K (Kwon *et al.*, 2023; Dao *et al.*, 2023; Davies *et al.*, 2025). Bucket graphs by context and store separate pipeline IR per bucket (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). Serving SLOs should cite which bucket was validated (Vellaisamy *et al.*, 2026; Chen, 2026a, 2020).

Sign-off evidence bar. Reviewers should see: worksheet, stall screenshot, bytes/token unchanged, pipeline IR hash (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024; SemiAnalysis, 2024). Missing any item returns PR to author—no “trust me, overlap is on” merges (Chen, 2020; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Reporting table (template).

✓ **Practice checkpoint.** Attach benchmark bundle (ms/token, stalls, worksheet) to every pipeline optimization PR (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026).

8.12 Key Takeaways

Asynchronous pipelines complete the movement layer opened in Chapters 6–7: residency budgets and static roles become timed overlap that hides memory latency (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024; SemiAnalysis, 2024; Dao *et al.*, 2022). The principles below pair design rules with actions your team can adopt this sprint (Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025).

- **Latency-hiding principle.** Pipeline overlaps movement with compute; cannot beat bandwidth floor (Chen, 2026a; SemiAnalysis, 2024; Davies *et al.*, 2025). Serial load+compute wastes cycles every tile at batch-1 decode (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). **Action:** name bottleneck stage first (step 1); attach stall replay (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026).
- **Depth-adaptation principle.** Depth from memory budget, load/compute ratio, and decode tile count N (Dao *et al.*, 2022; Chen, 2020; Vellaisamy *et al.*, 2026). Blind depth-4 from prefill configs is anti-pattern 5 (Shah *et al.*, 2024; Davies *et al.*, 2025). **Action:** complete Table 8.1 and Table 8.2 (Chen and YiRage Contributors, 2026; Chen, 2026a).
- **Multi-stream principle.** Weight and KV get independent pipeline parameters (Kwon *et al.*, 2023; Dao *et al.*, 2023; Shah *et al.*, 2024). Unified depth starves one stream while buffering the other (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). **Action:** separate buffer regions, depths, and barrier chains per stream (SemiAnalysis, 2024; Dao *et al.*, 2022).
- **Hardware-adaptation principle.** GPU explicit software pipeline; CPU implicit prefetch; NPU spatial DMA (NVIDIA, 2022; Goodfellow *et al.*, 2016; AMD, 2024b,a). Same worksheet math; different primitives and sync (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026). **Action:** fill Table 8.3 per backend in CI (Davies *et al.*, 2025; AMD, 2024a).
- **Compiler-automation principle.** Pipeline generation is a pass coupled to memory and tiling passes (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Mirage Project contributors, 2025; Shah *et al.*, 2024). Hand barrier edits do not survive model or hardware churn (Chen, 2020; SemiAnalysis, 2024). **Action:** attach pipeline IR + checklist to CI; block drift without worksheet updates (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

How this chapter connects. Chapter 6 planned bytes; Chapter 7 assigned producers/consumers; this chapter timed movement between them (Dao *et al.*, 2022; Shah *et al.*, 2024; SemiAnalysis, 2024). Chapter 4–5 hardware async features are *used* by pipelines, not re-taught here (NVIDIA, 2022; AMD, 2024b; Davies *et al.*, 2025). Chapter 9 generalizes sync when coordination scales beyond ping-pong (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020).

Megakernel stack preview. Pipelines (Ch. 8) → hierarchical sync (Ch. 9) → softmax carriers (Ch. 10) → assembly (Ch. 11) (Dao *et al.*, 2022; SemiAnalysis, 2024; Shah *et al.*, 2024; Mirage Project contributors, 2025; Davies *et al.*, 2025). Skipping pipeline design produces fused kernels that still wait on HBM every tile (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

8.13 Conclusion

Asynchronous pipelines spend Chapters 6–7’s budget and roles on timed movement: every producer issues ahead of every consumer without per-tile dispatch (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024; Dao *et al.*, 2022). The three-step depth method, tri-hardware tables, YiRage pipeline pass, and pitfall catalog turn overlap from folklore into repeatable engineering (Davies *et al.*, 2025; Chen, 2020; Vellaisamy *et al.*, 2026; Mirage Project contributors, 2025).

Closing metaphor. If Chapter 6 is the warehouse layout and Chapter 7 is the org chart, this chapter is the conveyor timing—trucks arrive while assemblers work (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). Perfect roles with synchronous loads still have assemblers standing idle at the belt each tile (Chen, 2026a; Vellaisamy *et al.*, 2026). Pipelines eliminate that idle; Chapter 9 coordinates when many belts cross (SemiAnalysis, 2024; Davies *et al.*, 2025).

Chapter 9 addresses **hierarchical synchronization**—when many producers and consumers coordinate, barriers themselves become a design problem (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Shah *et al.*, 2024). Carry depth worksheets, stream maps, and barrier indices into sync design (Chen and YiRage Contributors, 2026; SemiAnalysis, 2024; Chen, 2020).

Part IV bridge. Residency (Ch. 6) + roles (Ch. 7) + pipelines (this chapter) + sync (Ch. 9) assemble the megakernel execution stack (Dao *et al.*, 2022; Shah *et al.*, 2024; Mirage Project contributors, 2025). Chapter 20’s search assumes

pipeline-annotated IR—skipping here breaks the compiler story (Wu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020).

Budget–structure–execution recap. Chapter 6: where bytes live (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020). Chapter 7: which units touch bytes (Shah *et al.*, 2024; Mirage Project contributors, 2025; Davies *et al.*, 2025). Chapter 8: when bytes arrive relative to compute (SemiAnalysis, 2024; AMD, 2024a; NVIDIA, 2022). Together they define a deployable decode cell—not an isolated microbenchmark (Vellaisamy *et al.*, 2026; Chen, 2026a).

Teaching vs production. Whiteboard pipeline timelines (Figures 8.1, 8.5) for onboarding (Shah *et al.*, 2024; Goodfellow *et al.*, 2016). Automate Patterns 1–6 and golden IR in CI for fleet scale (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Mirage Project contributors, 2025). Concepts in §8.2–8.5; gates in §8.8–8.10 (Chen, 2020; Vellaisamy *et al.*, 2026).

Summary sentence. Chapter 6 budgeted bytes; Chapter 7 assigned actors; Chapter 8 synchronized their movement; Chapter 9 will discipline coordination at scale (Dao *et al.*, 2022; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). That sequence is the MegaKernel spine (Mirage Project contributors, 2025; Wu *et al.*, 2024; Chen, 2026a; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2020).

MegaKernel spine recap. Residency → roles → pipelines → sync (Ch. 9) → carriers (Ch. 10) → assembly (Ch. 11) (Dao *et al.*, 2022; Mirage Project contributors, 2025; Wu *et al.*, 2024; AMD, 2024b; NVIDIA, 2022; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; AMD, 2024a; Rotem *et al.*, 2018; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024).

Done criteria. Bottleneck stage named, depth worksheet signed, multi-stream plan documented, pipeline-pass artifact archived, checklist complete (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a, 2020). Proceed to Chapter 9 with mbarrier/token index table attached (SemiAnalysis, 2024; Dao *et al.*, 2022). Bring Ch. 6–8 worksheet bundle—Chapter 9 assumes ping-pong barriers are already legal (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024; Dao *et al.*, 2022; Mirage Project contributors, 2025; Chen, 2020).

Hands-on recap. (1) Fill depth worksheet. (2) Split weight/KV streams. (3) Run pipeline pass. (4) Triple legality with Ch. 6–7. (5) Complete checklist (Vellaisamy *et al.*, 2026; Shah *et al.*, 2024; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Mirage Project contributors, 2025; AMD, 2024b; NVIDIA, 2022; SemiAnalysis, 2024; Dao *et al.*, 2022; Chen, 2026a; Wu *et al.*, 2024; NVIDIA, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; Chen, 2020; AMD, 2024a; Dao *et al.*, 2023). Pipelining without residency and roles optimizes the wrong layer (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2026a).

Chapter 20 preview. Megakernel superoptimization searches fusion and tiling; pipeline pass must re-run inside the search inner loop (Mirage Project contributors, 2025; Wu *et al.*, 2024; Chen and YiRage Contributors, 2026). Candidates without legal async schedule are pruned early—saves silicon budget (Chen, 2020; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Serving stack note. PagedAttention and continuous batching change N and KV stream shape—re-run depth worksheets per serving bucket (Kwon *et al.*, 2023; Dao *et al.*, 2023; Davies *et al.*, 2025). Static pipeline IR should be bucketed like CUDA graphs (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Shah *et al.*, 2024).

Final spine position. Ch. 6 residency $\rightarrow$ Ch. 7 roles $\rightarrow$ **Ch. 8 pipelines** $\rightarrow$ Ch. 9 sync $\rightarrow$ Ch. 10 carriers $\rightarrow$ Ch. 11 assembly (Dao *et al.*, 2022; Mirage Project contributors, 2025; SemiAnalysis, 2024; AMD, 2024b; Wu *et al.*, 2024). You now have the execution timing layer; next chapter makes coordination explicit at scale (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Shah *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020).

Fleet note. Triplet deployments need three pipeline lowerings from one IR—GPU TMA, CPU prefetch, NPU DMA (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; AMD, 2024a; NVIDIA, 2022; Chen, 2020). Do not ship CUDA-only barrier metadata to Ryzen AI endpoints (AMD, 2024b; Vellaisamy *et al.*, 2026; Chen, 2026a).

Long-context note. Paged KV changes KV stream bytes per step—re-run depth worksheets per context bucket (Kwon *et al.*, 2023; Dao *et al.*, 2023; Davies *et al.*, 2025). Pipeline IR should be bucketed like CUDA graphs when context

length varies (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Shah *et al.*, 2024).

Hands-on exercises. (1) Measure serial vs ping-pong stall profile. (2) Fill depth worksheet. (3) Split weight/KV streams. (4) Run pipeline pass. (5) Complete checklist (Vellaisamy *et al.*, 2026; Shah *et al.*, 2024; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Mirage Project contributors, 2025; AMD, 2024b; NVIDIA, 2022; SemiAnalysis, 2024; Dao *et al.*, 2022; Chen, 2026a; Wu *et al.*, 2024; NVIDIA, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; Chen, 2020; AMD, 2024a; Dao *et al.*, 2023). Compare decode vs prefill pipeline depth on the same fused region if you maintain both paths (Kwon *et al.*, 2023; Shah *et al.*, 2024; Vellaisamy *et al.*, 2026).

Compiler author note. Pipeline pass authors should treat depth search as constrained optimization: minimize steady-step time subject to Ch. 6 legality and Ch. 7 role legality (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Chen, 2020; Mirage Project contributors, 2025). Log rejected candidates with reasons—those logs are gold for Chapter 20 search pruning (Mirage Project contributors, 2025; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Expose pipeline IR in debug builds so kernel engineers can diff barrier graphs without reading assembly (Shah *et al.*, 2024; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026).

Kernel engineer note. When hand-tuning CUDA, mirror pipeline IR fields in comments: stream id, buffer offset, mbarrier id, depth (NVIDIA, 2022; SemiAnalysis, 2024; Chen, 2020). Hand tuning without those fields does not port to YiRage or fleet backends (Chen and YiRage Contributors, 2026; AMD, 2024b; Davies *et al.*, 2025). Promote successful hand schedules into golden IR tests (Wu *et al.*, 2024; Shah *et al.*, 2024; Vellaisamy *et al.*, 2026).

Serving operator note. Pipeline parameters are not “set once”—model updates, quant changes, and context buckets all invalidate prior depth choices (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). Treat pipeline worksheets like hyperparameters: versioned, reviewed, and tied to benchmark bundles (Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a). When ms/token regresses after a model swap, replay checklist before touching tile shapes (Shah *et al.*, 2024; Dao *et al.*, 2022; Mirage Project contributors, 2025).

One-sentence chapter thesis. If Chapter 6 is *how many bytes* and Chapter 7 is *who moves them*, Chapter 8 is *never wait idle while they move*—implemented as depth-bounded, stream-aware, compiler-generated async pipelines (Dao *et al.*, 2022; Shah *et al.*, 2024; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020).

Exit artifacts. Leave this chapter with: signed depth worksheet, weight/KV stream map, mbarrier index table, pipeline-pass IR excerpt, completed checklist, and batch-1 benchmark delta (Chen and YiRage Contributors, 2026; Chen, 2020; Shah *et al.*, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; Mirage Project contributors, 2025; Wu *et al.*, 2024). Chapter 9 consumes the barrier index table directly—gaps here become sync bugs there (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; NVIDIA, 2022). Without these artifacts, hierarchical sync design in the next chapter is guesswork (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024; Davies *et al.*, 2025; Chen, 2020). The MegaKernel trilogy—budget, structure, execution—is incomplete until all three worksheet bundles are archived for your decode cell (Dao *et al.*, 2022; Chen, 2026a; Vellaisamy *et al.*, 2026; Mirage Project contributors, 2025; Wu *et al.*, 2024). Proceed to Chapter 9 only when pipeline legality, stall replay, and checklist are green (SemiAnalysis, 2024; Shah *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025). That gate is the handoff from execution timing to hierarchical synchronization (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Mirage Project contributors, 2025; Chen, 2026a). Bring the barrier index table—Chapter 9 builds on it, not around it (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Shah *et al.*, 2024; NVIDIA, 2022). End of Chapter 8—execution pillar complete for decode serving.

Chapter 9

Hierarchical Synchronization and Communication

9.1 The Lifeline of Fused Kernels: Sync Correctness

A team shipped a fused decode kernel with perfect bytes/token—then production hung on a barrier-count mismatch (Shah *et al.*, 2024; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Chapters 6–8 optimized residency, roles, and pipelines; **this chapter makes those optimizations correct and deadlock-free** (NVIDIA, 2022; Dao *et al.*, 2022; Mirage Project contributors, 2025).

The more aggressive the overlap, the more dangerous a sync bug. At batch-1 decode, a coarse block barrier on the critical path can cost more than the matmul it guards—yet omitting a needed barrier corrupts output silently (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026). **This chapter is the MegaKernel correctness pillar**—minimal-scope synchronization with compile-time proof (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Shah *et al.*, 2024).

Bridge from Chapters 6–8. Transaction barriers from Ch. 8 are point-to-point handoffs—this chapter generalizes scope selection and proves they cannot deadlock (SemiAnalysis, 2024; Shah *et al.*, 2024). Static roles from Ch. 7 fix who coordinates—sync can be minimal only because roles are known at compile time (Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025). Barrier objects consume Ch. 6 fast-memory budget—count mbarrier slots in peak worksheets (Chen, 2026a, 2020). Decode metrics: Chapter 3; hardware primitives:

Chapters 4–5 (Davies *et al.*, 2025; NVIDIA, 2022; AMD, 2024b).

What you will be able to do. (1) Pick minimal sync scope per dependency, (2) fill tri-hardware sync table, (3) run static deadlock checks, (4) interpret YiRage sync-pass errors, (5) complete sync checklist (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024; Vellaisamy *et al.*, 2026).

Organization map. Hierarchy principles (§9.2), tri-hardware primitives (§9.3), deadlock detection (§9.4), YiRage pass (§9.6), joint budget (§9.5), pitfalls (§9.7), patterns (§9.8), review workflow, takeaways, bridge to Chapter 10 (Dao *et al.*, 2022; SemiAnalysis, 2024).

Decode-first thesis. Prefill amortizes coarse sync across many tokens; decode does not (Kwon *et al.*, 2023; Shah *et al.*, 2024; Davies *et al.*, 2025). **Fine sync, fine roles, fine pipelines**—or pay ms/token on barriers with nothing to hide behind (Vellaisamy *et al.*, 2026; Chen, 2026a). Never use block sync where a transaction barrier suffices (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026).

Case study sketch. Fused attention: replacing one errant `__syncthreads` with mbarrier pair recovered $\sim 8\%$ ms/token; a divergent barrier path hung 1 in 10^4 requests (Shah *et al.*, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Static proof caught the divergent path before merge (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Chen, 2020).

Anti-pattern preview. Section 9.7 lists five sync failures—review before any barrier edit (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). Culture: no hand barrier without sync-pass legality report (Chen, 2020; Vellaisamy *et al.*, 2026).

Budget–structure–execution–correctness stack. Ch. 6 bytes $\rightarrow$ Ch. 7 actors $\rightarrow$ Ch. 8 timing $\rightarrow$ **Ch. 9 proof** (Dao *et al.*, 2022; Shah *et al.*, 2024; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026). Residency without sync races; roles without sync deadlock; pipelines without sync read stale buffers (Chen, 2026a; AMD, 2024a; Mirage Project contributors, 2025; Wu *et al.*, 2024).

Industrial narrative. 847.5 launches/token collapse to one kernel—then one wrong barrier hangs the whole forward pass (Vellaisamy *et al.*, 2026; Mirage Project contributors, 2025; Davies *et al.*, 2025). CUTLASS pipeline classes exist because humans mis-count arrives—sync belongs in the compiler (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020).

Reader paths. **Kernel engineers:** scope ladder + mbarrier rules + pitfalls (Shah *et al.*, 2024; SemiAnalysis, 2024). **Compiler authors:** sync pass I/O + deadlock checks + golden IR (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Mirage Project contributors, 2025). **Serving leads:** hang postmortem bundle + quadruple legality (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). All paths end at checklist before Chapter 10 (Dao *et al.*, 2022; Chen, 2026a).

Who should read this chapter. Anyone editing barriers after Ch. 8 pipeline merge; compiler authors on sync pass; on-call for fused-kernel hangs (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024; Mirage Project contributors, 2025). Finish Ch. 6–8 worksheets first—sync without pipeline IR is guesswork (SemiAnalysis, 2024; Chen, 2020; Vellaisamy *et al.*, 2026).

What this chapter is not. Not Hopper ISA manual (Ch. 4), not mbarrier tutorial (Ch. 8), not softmax math (Ch. 10) (NVIDIA, 2022; Dao *et al.*, 2022; SemiAnalysis, 2024). We teach sync *design and proof* given fused IR (Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025).

Terminology alignment. **Scope** = set of units that must agree before proceeding (Shah *et al.*, 2024; NVIDIA, 2022). **Transaction barrier** = producer-consumer scoped mbarrier (SemiAnalysis, 2024; Dao *et al.*, 2022). **Proof certificate** = compile-time deadlock-freedom artifact from sync pass (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). Use consistently in PRs and CI (Chen, 2020; Vellaisamy *et al.*, 2026).

Bridge to Chapter 10. Online softmax keeps (m, ℓ, o) carriers synchronized across tiles—Ch. 10 specializes the shuffle/atomic patterns introduced here (Dao *et al.*, 2022; Shah *et al.*, 2024; Davies *et al.*, 2025; Chen, 2026a).

What changes in Ch. 10 vs Ch. 9. Ch. 9 proves *where* sync may occur and that wait-for graph is acyclic (Chen and YiRage Contributors, 2026; Wu *et al.*,



Figure 9.1: Sync hierarchy (decode: live in left two boxes; avoid right unless unavoidable).

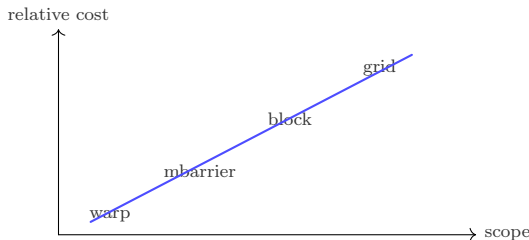


Figure 9.2: Sync cost ladder: scope widening raises latency (H100 illustrative cycles in Table 9.1).

2024; Shah *et al.*, 2024). Ch. 10 proves *what numeric state* must be consistent across those sync points—running max m , running sum ℓ , output accumulator o (Dao *et al.*, 2022; Chen, 2026a). A softmax carrier bug can pass sync proof yet fail numerics—both certificates are required for attention fusion release (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2020). Carry forward: mbarrier index table, scope worksheets, and quadruple legality bundle when opening Ch. 10 worksheets (Chen and YiRage Contributors, 2026; SemiAnalysis, 2024).

9.2 Sync Hierarchy: Minimal Scope Principles

Coordinate at the **narrowest scope** a dependency spans—cost rises steeply with participant count and physical distance (Shah *et al.*, 2024; Davies *et al.*, 2025; NVIDIA, 2022).

Quantitative cost ladder (H100 illustrative).

Design iron rule. Fine when possible, coarse only when necessary. Two warps handing off one buffer $\Rightarrow$ mbarrier, not block sync (Shah *et al.*, 2024; SemiAnalysis, 2024). Whole CTA phase change $\Rightarrow$ block barrier (NVIDIA, 2022). Shard sum across GPUs $\Rightarrow$ collective—overlap with next layer compute (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Table 9.1: GPU synchronization cost ladder (decode planning).

Scope	Primitive	Typ. cost
Warp	<code>__syncwarp</code> , shuffle	~0 cycles (lockstep)
Producer/consumer	Transaction mbarrier	~10–20 cycles
Block (CTA)	<code>__syncthreads</code>	~50–100 cycles
Cluster	Cluster barrier	~200–500 cycles
Grid / multi-GPU	Grid sync, collective	μs (2–3 orders up)

Table 9.2: Sync scope economics for decode (H100 illustrative).

Scope choice	Typ. cycles	Decode impact	im-	When justified
Warp shuffle	~0	Negligible	vs	Row stats, partial sums
mbarrier pair	10–20	Small if off critical path		Pipeline hand-off
Block barrier	50–100	Can dominate batch-1 step		Whole-CTA phase change
Cluster grid	/ 200+ / μs	Almost always fatal at decode		Shard collectives only

Decode-specific rules. (1) Reductions (softmax stats, partial sums): prefer warp shuffle + atomics over block sync (Dao *et al.*, 2022; Shah *et al.*, 2024). (2) Pipeline handoffs: only point-to-point mbarrier—never block sync as a pipeline substitute (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026). (3) Coarse sync on critical path: treat as bug unless proven necessary (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2026a).

Sync economics vs pipeline depth. Every extra pipeline stage from Ch. 8 adds at least one mbarrier edge—sync cost scales with depth even when memory budget allows deeper buffering (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026). Table 9.2 pairs scope choice with decode impact: the goal is not zero sync but *minimal necessary* sync on the critical path (Shah *et al.*, 2024; Davies *et al.*, 2025).

Worked decode sync audit (Llama-3.2-1B, batch-1). Consider a fused attention tile with Ch. 8 depth $D=2$ and Ch. 7 roles: four producer warps load KV, one consumer warpgroup computes softmax+GEMM (Shah *et al.*, 2024; Kwon *et al.*, 2023). **Sync sites to prove:** (a) TMA→smem handoff per buffer slot—

mbarrier with byte count; (b) softmax row max across warpgroup—warp shuffle then at most one atomic per row group, not `__syncthreads`; (c) optional KV index update—single atomic per page slot (Dao *et al.*, 2022; Davies *et al.*, 2025). **Reject:** block sync between load and compute—destroys the overlap Ch. 8 built; block sync between softmax and GEMM warpgroups when FA3-style narrow pairing suffices (Shah *et al.*, 2024; SemiAnalysis, 2024). **Certificate output:** two mbarrier rings (weight stream, KV stream), zero block barriers on pipeline path, shuffle path for stats—archive hash with quadruple legality (Chen and YiRage Contributors, 2026; Chen, 2020).

Scope decision worksheet (template). For each dependency edge, record: (1) producer unit id (from Ch. 7 role map); (2) consumer unit id; (3) buffer id and byte range; (4) narrowest legal primitive; (5) CFG paths where both units participate (Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025). If step (4) lands on block or wider scope, write one-sentence proof why narrower scopes fail—blank proof fields fail review Gate B (Chen, 2020; Vellaisamy *et al.*, 2026). Decode kernels should show $\geq 80\%$ of sync sites at warp or mbarrier scope on the critical path (Davies *et al.*, 2025; Chen, 2026a).

Transaction barrier as Ch. 8 foundation. Ch. 8 ping-pong *is* mbarrier discipline—producer arrive with byte count, consumer wait, then read (Shah *et al.*, 2024; SemiAnalysis, 2024; NVIDIA, 2022). Coarse block sync between load and compute would serialize the overlap Ch. 8 built (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026).

Static roles enable minimal sync. Ch. 7 fixed producer/consumer IDs—sync pass knows exactly which units pair (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024; Mirage Project contributors, 2025). Dynamic roles would force wider barriers “just in case” (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

mbarrier in residency budget. Each mbarrier object has metadata cost in Ch. 6 peak—deep pipelines multiply barriers (Chen, 2026a, 2020; Chen and YiRage Contributors, 2026). Deeper pipeline (Ch. 8) $\Rightarrow$ more barrier indices to prove consistent (Wu *et al.*, 2024; Shah *et al.*, 2024).

Cluster and grid scope. Use cluster barriers only when DSM sharing (Ch. 6) genuinely requires cross-CTA visibility (NVIDIA, 2022; Shah *et al.*, 2024). Grid

sync only in persistent kernels with proven co-residency—expensive at batch-1 (Mirage Project contributors, 2025; Davies *et al.*, 2025; NVIDIA, 2024).

Why hierarchy mirrors thread hierarchy. Warp $\subset$ block $\subset$ cluster $\subset$ grid mirrors Ch. 7 unit nesting—sync scope should match the smallest containing set for each dependency (Shah *et al.*, 2024; NVIDIA, 2022; Chen and YiRage Contributors, 2026). Mismatching scope wastes work: block sync forces fast warps to wait for slow ones on unrelated data (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026).

Prefill vs decode sync economics. Prefill: many warps busy—block sync cost amortized across parallel work (Kwon *et al.*, 2023; Shah *et al.*, 2024). Decode: few active warps—one `__syncthreads` can dominate step time (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a). Maintain separate sync audit tabs for decode and prefill kernels (Chen and YiRage Contributors, 2026; Chen, 2020).

Quantifying sync tax on decode critical path. Let T_{step} be batch-1 decode step time and T_{sync} the sum of stall cycles from barriers on the critical path (Davies *et al.*, 2025; Shah *et al.*, 2024). When $T_{\text{sync}}/T_{\text{step}} > 0.05$, treat as P1 optimization—often fixable by replacing one block sync with `mbarrier` or `shuffle` (SemiAnalysis, 2024; Vellaisamy *et al.*, 2026). Nsight exports T_{sync} via barrier stall counters; compare against Table 9.2 expectations before accepting new sync sites (NVIDIA, 2022; Chen and YiRage Contributors, 2026). Prefill kernels may tolerate $T_{\text{sync}}/T_{\text{step}}$ up to ~ 0.15 when parallelism hides stalls—do not copy prefill sync plans into decode (Kwon *et al.*, 2023; Chen, 2026a; Davies *et al.*, 2025).

Multi-stream weight/KV independence. Chapter 8 split weight and KV pipelines—sync must mirror that split (SemiAnalysis, 2024; Kwon *et al.*, 2023; Dao *et al.*, 2023). Cross-stream barriers are forbidden unless a true data dependency exists (e.g., attention needs both weight and KV ready for the same tile index) (Shah *et al.*, 2024; Dao *et al.*, 2022). When dependency is per-tile, use a join `mbarrier` whose `arrive count` reflects both streams—not two block syncs in sequence (Chen and YiRage Contributors, 2026; SemiAnalysis, 2024; Chen, 2020). Mis-ordered weight/KV sync is anti-pattern trigger #1 in Section 9.4 (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Collectives as coarsest sync. Tensor-parallel all-reduce is synchronization across devices—treat as graph edge with same acyclic rules (Davies *et al.*, 2025;

Goodfellow *et al.*, 2016; Mirage Project contributors, 2025). Overlap collective with next-layer compute when dependency graph allows—mirror Ch. 8 latency hiding one scope up (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026).

Worked scope decision (attention handoff). Dependency: producer warp fills smem tile; consumer warpgroup reads it (Shah *et al.*, 2024; SemiAnalysis, 2024). **Wrong:** `__syncthreads`—stalls entire CTA (NVIDIA, 2022; Davies *et al.*, 2025). **Right:** `mbarrier` pair scoped to producer+consumer role IDs from Ch. 7 (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Chen, 2020).

✓ **Practice checkpoint.** For each sync site, document dependency span (which units, which buffer) before choosing primitive (Chen and YiRage Contributors, 2026; Chen, 2020; Shah *et al.*, 2024).

9.3 Tri-Hardware Sync Primitives and Adaptation

Each backend exposes different coordination mechanisms—same producer–consumer dependency graph, different lowering (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020). Chapter 8’s tri-hardware table answered *how data moves*; this section answers *how collaborators synchronize*—symmetric cross-hardware framing for the MegaKernel stack (SemiAnalysis, 2024; AMD, 2024b; Shah *et al.*, 2024).

Tri-hardware sync overview. The sync stacks differ in the same axis as residency and pipelines: a trade between **dynamic flexibility** and **static determinism** (Chen and YiRage Contributors, 2026; AMD, 2024a; NVIDIA, 2022). Table 9.3 summarizes primitives, scheduling model, cost, decode rules, and deadlock detectability—read it before any platform subsection so GPU habits are not pasted onto CPU or NPU (Davies *et al.*, 2025; Chen, 2020).

Cross-hardware law (staticity gradient). Reading Table 9.3 left to right: **staticity rises, compiler responsibility rises, runtime forgiveness falls** (Chen and YiRage Contributors, 2026; AMD, 2024b; NVIDIA, 2022). GPU keeps flexible explicit barriers—powerful overlap, but scope mistakes tax decode and deadlocks hide in divergent CFG (Shah *et al.*, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). CPU sits in the middle: coherence makes many syncs “invisible,” yet false sharing and NUMA remote atomics punish decode helpers silently (Goodfellow



Figure 9.3: Tri-hardware sync lowering (hardware detail in Ch. 4–5, pipeline handoff in Ch. 8).

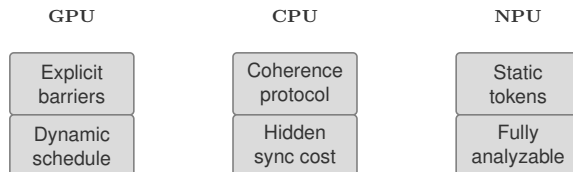


Figure 9.4: Sync management style per hardware class (visual companion to Table 9.3).

et al., 2016; Chen, 2026a, 2020). NPU freezes every sync at compile time—zero runtime scheduling tax, but a missing token halts the array with no recovery path (AMD, 2024b,a; Chen and YiRage Contributors, 2026).

Decode landing rules (three directions). **GPU:** minimize sync *scope*—fine primitives on a thin per-token critical path (SemiAnalysis, 2024; Shah *et al.*, 2024). **CPU:** minimize cross-core sync *frequency*—partition and pad before adding fences (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). **NPU:** maximize compile-time sync *correctness*—token graph must be complete before silicon (AMD, 2024a; Wu *et al.*, 2024). These directions are not interchangeable; YiRage Sync Pass (Section 9.6) lowers one dependency graph three ways with matching proof strength per row (Chen and YiRage Contributors, 2026; Chen, 2020; Mirage Project contributors, 2025).

9.3.1 GPU synchronization rules

Hopper mbarrier mechanics for pipeline handoff: Section 8.3 and Chapter 4 (SemiAnalysis, 2024; NVIDIA, 2022; Shah *et al.*, 2024). Table 9.3 row 1 is the contract—subsections below are implementation detail, not a second taxonomy (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

mbarrier arrive/wait pairing. Producer: TMA complete $\rightarrow$ `mbarrier.arrive` with expected bytes (SemiAnalysis, 2024; Shah *et al.*, 2024). Consumer: `mbarrier.wait` $\rightarrow$ read buffer $\rightarrow$ compute (NVIDIA, 2022; Chen and YiRage Contributors, 2026). Ping-pong: alternate barrier indices $b_0, b_1, \dots, b_{D-1}$ in

ring—off-by-one causes corruption or hang (Dao *et al.*, 2022; Chen, 2020).

Atomic vs barrier boundary. **Barrier:** phase boundary—no participant proceeds until all arrive (NVIDIA, 2022; Shah *et al.*, 2024). **Atomic:** single-location update—units need not all stop (Dao *et al.*, 2022; Chen, 2026a). Partial reductions, KV index updates: atomics or warp shuffle (Kwon *et al.*, 2023; Davies *et al.*, 2025). Stage handoff after async copy: mbarrier only (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026).

Decode GPU tricks. Softmax row stats: warp shuffle reduce before any block sync (Dao *et al.*, 2022; Shah *et al.*, 2024). FA3-style warpgroup overlap uses narrow barriers between softmax and GEMM warpgroups—not block-wide stalls (Shah *et al.*, 2024; Davies *et al.*, 2025).

mbarrier ring walkthrough (depth $D=2$). Buffer slots B_0, B_1 alternate; barrier indices b_0, b_1 form a ring (SemiAnalysis, 2024; Dao *et al.*, 2022). **Initialization:** both buffers empty; producer owns both; consumer waits on b_0 before first read (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026). **Steady state:** producer fills B_0 , arrives b_0 with byte count N ; consumer waits b_0 , reads B_0 , computes, releases B_0 to producer via arrive on consumer-side phase barrier; producer fills B_1 while consumer works on B_0 output—overlap is valid only if arrives precede waits in consistent order (NVIDIA, 2022; SemiAnalysis, 2024). **Failure modes:** producer arrives b_1 while consumer still reads B_1 (depth mis-config); consumer waits b_0 twice without intervening producer arrive (pairing bug); byte count N does not match actual TMA bytes after dtype change (silent corruption) (Vellaisamy *et al.*, 2026; Chen, 2020). YiRage generates this ring from Ch. 8 pipeline IR—hand-editing indices is the top source of anti-pattern 2 (Wu *et al.*, 2024; Chen and YiRage Contributors, 2026).

KV cache index updates. Paged attention updates slot metadata once per token—use atomics on the page table entry, not a block barrier across the CTA (Kwon *et al.*, 2023; Dao *et al.*, 2023). If multiple warps update disjoint pages, no sync between them; if two warps can touch the same page, serialize with atomics on that page only (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). This pattern foreshadows Ch. 10 carrier updates—running softmax stats obey the same “shuffle first, atomic only if cross-warp” rule (Dao *et al.*, 2022; Shah *et al.*, 2024).

mbarrier timeout and debug. Production kernels should log barrier index on watchdog timeout (Vellaisamy *et al.*, 2026; Shah *et al.*, 2024; Chen and YiRage Contributors, 2026). Development builds may enable arrive-count assertions before wait (NVIDIA, 2022; Chen, 2020; SemiAnalysis, 2024).

Atomic contention on decode. Hot global atomics serialize warps—prefer warp-local shuffle then single atomic per warp (Dao *et al.*, 2022; Chen, 2026a; Davies *et al.*, 2025). KV length updates: atomics on paged structures—minimize frequency (Kwon *et al.*, 2023; Dao *et al.*, 2023; Chen and YiRage Contributors, 2026).

9.3.2 CPU synchronization rules

No scratchpad barriers—memory ordering and coherence do the work (Ch. 6 blocking); see Table 9.3 row 2 (Goodfellow *et al.*, 2016; Chen, 2020).

Fence level selection. Producer publishes with **release**; consumer loads with **acquire** (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). Use **seq_cst** only when global total order required—decode hot loops rarely need it (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

False sharing quantified. Two threads on one cache line (~ 64 B): coherence ping-pong can cut effective bandwidth 30–50% (Goodfellow *et al.*, 2016; Chen, 2026a). **Fix:** pad per-thread accumulators to cache-line boundaries (Chen, 2020; Davies *et al.*, 2025).

NUMA cross-node sync. Remote atomics/fences often $2\text{--}3\times$ local latency (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Partition weights/KV per socket; avoid hot atomics on cross-node lines (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

CPU decode pipeline analogy. CPU “pipeline” is cache + prefetch (Ch. 8)—sync is fence placement ordering loads to consumers (Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Mis-ordered fence $\Rightarrow$ stale read without hang—harder than GPU deadlock (Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a).

Worked CPU layout fix. Per-thread partial sum array: pad struct to 64B alignment (Goodfellow *et al.*, 2016; Chen, 2020). Before: 8 threads false-share on one line; after: LLC miss rate drops, ms/token improves (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026).

CPU decode helper threading model. Many deployments run token sampling, logits post-processing, or small expert routers on CPU while GPU runs the megakernel (Davies *et al.*, 2025; Kwon *et al.*, 2023). Handoff uses a single-producer single-consumer queue: producer publishes slot index with **release**, consumer loads with **acquire** (Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026). Do not use **seq_cst** on every token—batch-1 decode issues one fence per step at most on the hot path (Vellaisamy *et al.*, 2026; Chen, 2020). If multiple CPU threads reduce partial statistics for speculative decoding, give each thread a cache-line-aligned accumulator—mirrors GPU shuffle-before-atomic discipline at core granularity (Chen, 2026a; Davies *et al.*, 2025).

When CPU sync matters for fused stacks. Even GPU-only megakernels interact with CPU through graph capture buckets and dynamic shape guards (NVIDIA, 2024; Chen and YiRage Contributors, 2026). Bucket boundaries must not change barrier participant sets without re-proof—CPU-side bucket selection is a sync-affecting control-flow edge (Shah *et al.*, 2024; Davies *et al.*, 2025). Document which buckets share the same sync certificate; splitting buckets multiplies proof artifacts but prevents context-length hangs (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026).

9.3.3 NPU synchronization rules

XDNA completion tokens and static dataflow: Section 5.3 and Chapter 5; Table 9.3 row 3 mandates full static proof (AMD, 2024b,a; Chen and YiRage Contributors, 2026).

Token pairing. One completion token per DMA→MAC handoff; wait instruction references exact token id (AMD, 2024a,b). Multi-stream (weight vs KV): disjoint token index spaces—no cross-stream alias (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024; Kwon *et al.*, 2023).

Cross-tile stream sync. Neighbor tile links have finite bandwidth—long chains accumulate latency (AMD, 2024b; Davies *et al.*, 2025; Chen, 2026a). Balance

spatial pipeline (Ch. 8) with sync depth—do not serialize array on one token chain (AMD, 2024a; Chen, 2020).

Static validation mandate. Every wait must have a proven producer on all paths—NPU has no runtime recovery (AMD, 2024b; Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). Compile-time token graph is fully simulatable (AMD, 2024a; Vellaisamy *et al.*, 2026).

NPU multi-stream decode layout. Weight DMA stream and KV DMA stream use disjoint token namespaces—token 7 on weight stream must never satisfy a wait on KV stream (AMD, 2024b,a; Chen and YiRage Contributors, 2026). Spatial pipeline across tiles assigns forward tokens along the chain; backward dependency (e.g., partial sum return) needs reverse token path or explicit stream sync at subgraph boundary (Davies *et al.*, 2025; Chen, 2026a). Long chains accumulate latency: if each tile adds 50 cycles of sync tax, a 16-tile chain adds 800 cycles before MAC sees last tile’s data—balance Ch. 8 spatial depth with sync chain length (AMD, 2024a; Chen, 2020; Shah *et al.*, 2024). Validation exports a token DAG; simulation runs topological order with timeout—any wait without producer is release-blocking (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024).

Lowering GPU mbarrier plans to NPU tokens. Do not transliterate mbarrier arrive/wait as “token per buffer slot” without checking MAC schedule—NPU tokens bind to DMA completion events, not arbitrary warp phases (AMD, 2024b; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). YiRage backend pass maps each pipeline handoff edge to (DMA_id, token_id) pairs; cross-hardware fleet builds share dependency graph, not textual CUDA barrier code (Chen, 2020; Mirage Project contributors, 2025).

✓ **Practice checkpoint.** Fill Table 9.3 row per backend before fleet merge (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020).

9.4 Deadlock Detection and Prevention

Async pipelines trade bulk-synchronous safety for overlap—and introduce coordination hangs (Shah *et al.*, 2024; NVIDIA, 2022; Chen and YiRage Contributors, 2026).



Figure 9.5: Static deadlock detection pipeline (compile-time gate).

Three classic deadlock patterns. (1) **Arrival mismatch:** barrier expects n arrivals; divergent path delivers $n-1$ (Shah *et al.*, 2024; NVIDIA, 2022). (2) **Circular wait:** producer waits on free buffer while consumer waits on full—broken initial condition or reversed order (SemiAnalysis, 2024; Dao *et al.*, 2022). (3) **Barrier ring disorder:** multi-buffer pipeline with inconsistent cyclic order (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026).

Static detection methods. **Arrival counts:** for each barrier, enumerate CFG paths; count reaching units; compare to expected (Chen and YiRage Contributors, 2026; Chen, 2020; Shah *et al.*, 2024). **Cycle detection:** build wait-for graph on handoff edges; reject if cycle exists (Wu *et al.*, 2024; Mirage Project contributors, 2025). **Ring consistency:** verify producer/consumer pointers advance monotonically on buffer ring (SemiAnalysis, 2024; Dao *et al.*, 2022).

Decode high-frequency triggers. **Weight vs KV stream cross-wait:** mismatched ordering between independent mbarrier chains (Kwon *et al.*, 2023; Dao *et al.*, 2023; Shah *et al.*, 2024). **Context bucket branches:** some warps skip barrier in one bucket template (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). **Wrong pipeline depth:** producer overwrites buffer consumer still reads—often race before hang (SemiAnalysis, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026).

Static analysis algorithm (conceptual). **Step 1—Build wait-for graph:** nodes are role units (Ch. 7); directed edge $A \rightarrow B$ means A waits on a barrier/token that B must signal (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). **Step 2—Cycle detection:** run DFS or Tarjan on the wait-for graph; any cycle is a compile failure with listed nodes (Chen, 2020; Mirage Project contributors, 2025). **Step 3—CFG arrival parity:** for each barrier b expecting n arrivals, intersect participating units with each CFG path; if any path contributes $k \neq n$, emit barrier id and path id (Shah *et al.*, 2024; NVIDIA, 2022). **Step 4—Ring monotonicity:** for buffer index sequence (i_t) , verify $i_{t+1} = (i_t + 1) \bmod D$ on both producer and consumer sides (SemiAnalysis, 2024; Dao *et al.*, 2022). **Step 5—Token coverage (NPU):** every wait instruction id appears as a completion token on some DMA or

MAC edge in the static schedule (AMD, 2024b,a). This five-step pipeline is what the YiRage sync pass implements; manual review mirrors the same checks using exported IR (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Simulation and soak strategy. Compile-time proof catches structural deadlock; simulation catches timing assumptions—e.g., consumer wait before producer arrive on first tile because initialization lowering was wrong (Shah *et al.*, 2024; Davies *et al.*, 2025). Run cycle-accurate or instruction-level simulation with timeout on every wait primitive; log barrier id and cycle count at timeout (NVIDIA, 2022; SemiAnalysis, 2024). Production soak: replay decode traces across context-length buckets; hangs at 1 in 10^4 still fail release gates when serving SLA is millions of tokens per day (Vellaisamy *et al.*, 2026; Mirage Project contributors, 2025; Kwon *et al.*, 2023).

Tiered defense. **Compile-time:** sync pass rejects unprovable regions ($\sim 90\%$ + bug class) (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Chen, 2020). **Simulation:** cycle-accurate wait timeout in validation (Shah *et al.*, 2024; Davies *et al.*, 2025). **Runtime:** watchdog + barrier index dump for residual cases (Vellaisamy *et al.*, 2026; Mirage Project contributors, 2025).

Initial conditions for producer-consumer cycles. Ping-pong starts with defined buffer ownership—usually empty buffers owned by producer, first consumer wait on buffer 0 (SemiAnalysis, 2024; Dao *et al.*, 2022; Shah *et al.*, 2024). Sync pass must emit and verify initial state—hand kernels often get this wrong on first tile (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026).

CUTLASS structural lesson. Circular barrier list generated from single pipeline spec—humans should not hand-maintain ring indices (Shah *et al.*, 2024; Wu *et al.*, 2024). YiRage sync pass is the compiler analogue for whole fused subgraphs (Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025; Chen, 2020).

NPU determinism advantage. Fully static token graph enables exhaustive compile-time simulation—strongest proof story of three backends (AMD, 2024b,a; Chen and YiRage Contributors, 2026). GPU proof harder due to dynamic warp scheduling—rely on CFG + role staticness from Ch. 7 (Shah *et al.*, 2024; NVIDIA, 2022; Davies *et al.*, 2025).

Control-flow divergence rule. Barriers only at points *all* participating units reach unconditionally (NVIDIA, 2022; Shah *et al.*, 2024). Conditional barriers are anti-pattern 3 (Section 9.7) (Chen and YiRage Contributors, 2026; Chen, 2020).

✓ **Practice checkpoint.** Attach sync-pass legality report—not just “kernel compiles” (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Shah *et al.*, 2024).

9.5 Joint Budget with Chapters 6–8

Sync is not free in memory or proof complexity (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Shah *et al.*, 2024; Chen, 2020).

Barrier objects in Ch. 6 peak. mbarrier state, token slots, collective scratch—account in fast-memory worksheet (Chen, 2026a; NVIDIA, 2022; AMD, 2024b). Deep pipeline (Ch. 8) $\Rightarrow$ more live barriers simultaneously (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026).

Role-fixed dependency graph. Sync pass consumes Ch. 7 role IDs and Ch. 8 buffer map—changing either without re-running sync pass invalidates proof (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Mirage Project contributors, 2025).

Quadruple legality bundle. Ship Ch. 6 memory + Ch. 7 role + Ch. 8 pipeline + Ch. 9 sync reports together (Chen, 2020; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). CI fails on any drift without coordinated regen (Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025).

Fixed-point loop. Memory $\rightarrow$ tiling $\rightarrow$ pipeline $\rightarrow$ **sync** $\rightarrow$ re-validate barrier count vs footprint (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Dao *et al.*, 2022; Shah *et al.*, 2024).

Worked quadruple example. Attention fusion: Ch. 6 peak includes 2 mbarrier objects + ping-pong smem (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; SemiAnalysis, 2024). Ch. 7: 4 producer warps, 1 consumer warpgroup (Shah *et al.*, 2024; Mirage Project contributors, 2025). Ch. 8: depth $D=2$, weight/KV stream

maps (Kwon *et al.*, 2023; SemiAnalysis, 2024). Ch. 9: 2 mbarrier rings proven acyclic—certificate hash archived in CI (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Chen, 2020; Vellaisamy *et al.*, 2026).

Spill vs sync confusion. Spill after sync merge usually means barrier metadata exceeded Ch. 6—not a sync bug but budget violation (Chen, 2026a; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022). Rollback barrier count or split fusion before weakening proof (Wu *et al.*, 2024; Chen, 2020; Mirage Project contributors, 2025).

MoE and sync. Expert routing adds control-flow edges—each bucket needs sync re-proof (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Mirage Project contributors, 2025; Davies *et al.*, 2025). Static expert buckets mirror CUDA graph bucketing (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Shah *et al.*, 2024).

Tensor-parallel and pipeline-parallel edges. Tensor parallel inserts all-reduce or reduce-scatter as collective sync edges—every rank must enter the same collective in the same order relative to local mbarrier phases (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Mirage Project contributors, 2025). Pipeline parallel inserts send/recv between stages—model as directed sync edge from sender rank to receiver rank; cycle detection includes cross-rank wait-for graph (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). Decode batch-1 often runs TP inside the megakernel; PP may be absent—but when PP exists, a hang on rank 2 blocks the whole replica set (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Quadruple legality extends to *quintuple* when TP collectives are in scope: add collective symmetry report to the bundle (Chen, 2020; Wu *et al.*, 2024).

Budget rollback triggers. If sync pass inserts more mbarrier objects than Ch. 6 peak allows, do not weaken proof—reduce pipeline depth, split fusion, or spill cold tensors first (Chen, 2026a; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022). Teams sometimes add block sync to “fix” races after budget failure—that trades proof debt for ms/token tax; reject in review (Vellaisamy *et al.*, 2026; Shah *et al.*, 2024; Davies *et al.*, 2025).

✓ **Practice checkpoint.** Quadruple worksheet bundle on every sync-affecting PR (Chen, 2020; Vellaisamy *et al.*, 2026).

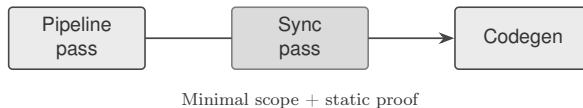


Figure 9.6: YiRage sync pass: last compile-time correctness gate before codegen.

9.6 YiRage Sync Pass: Insertion and Proof

YiRage inserts minimal sync and proves deadlock-freedom after pipeline pass, before codegen (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024; Wu *et al.*, 2024; Mirage Project contributors, 2025). The pass implements Table 9.3: GPU rows get scope-minimized mbarrier lowering with partial static proof plus watchdog hints; CPU rows get fence/atomic placement with false-sharing warnings; NPU rows get token graphs with full cycle and coverage checks (AMD, 2024b; Chen, 2020; Davies *et al.*, 2025).

Verification dimensions. (1) **Scope minimization**—two-unit deps never get block barriers (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026; NVIDIA, 2022). (2) **Arrival consistency**—all CFG paths (Chen, 2020; Wu *et al.*, 2024). (3) **No wait cycles**—handoff DAG acyclic (Mirage Project contributors, 2025; SemiAnalysis, 2024). (4) **Memory order**—reads not visible before matching sync (Dao *et al.*, 2022; Goodfellow *et al.*, 2016; Davies *et al.*, 2025).

Error reporting. On failure: emit barrier id, divergent CFG path, or wait-cycle node list—not generic “sync error” (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026). Enables bisect without reproducing hang on silicon (Shah *et al.*, 2024; Davies *et al.*, 2025).

Multi-device extension. Tensor-parallel all-reduce modeled as sync edge—every rank must issue matching collective (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Mirage Project contributors, 2025). Overlap with compute allowed only if acyclic in combined graph (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026).

Mirage / Ch. 20 composition. Superoptimizer proposes fusion; memory/tiling/pipeline passes certify structure; sync pass certifies coordination (Wu *et al.*, 2024; Mirage Project contributors, 2025; Chen and YiRage Contributors,

2026). Megakernel candidates without proof are pruned (Chen, 2020; Vellaisamy *et al.*, 2026).

Golden IR regression. CI diffs sync IR per canonical layer—barrier index drift blocks merge (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Shah *et al.*, 2024).

Sync pass pseudocode (engineering view).

1. Ingest pipeline IR: buffer map, depth D , stream ids, role participant sets (Chen and YiRage Contributors, 2026; SemiAnalysis, 2024).
2. For each handoff edge, attempt lowering: warp $\rightarrow$ mbarrier $\rightarrow$ block $\rightarrow$ cluster; stop at first scope that satisfies visibility (Shah *et al.*, 2024; NVIDIA, 2022).
3. Insert acquire/release or mbarrier arrive/wait with byte counts derived from memory pass layout (Dao *et al.*, 2022; Chen, 2020).
4. Run deadlock checks (Table 9.4); on failure, emit structured error (Wu *et al.*, 2024; Chen and YiRage Contributors, 2026).
5. Emit proof certificate: hash of sync IR + check results + participating barrier ids (Mirage Project contributors, 2025; Vellaisamy *et al.*, 2026).

Certificates are versioned artifacts shipped beside compiled kernels—operators diff certificate on deploy (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

Interaction with prior passes. Memory pass change $\Rightarrow$ re-run tiling $\Rightarrow$ re-run pipeline $\Rightarrow$ re-run sync—skipping sync after layout change is a common regression (Wu *et al.*, 2024; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022). Role pass change $\Rightarrow$ re-run sync even if pipeline depth unchanged—participant sets changed (Shah *et al.*, 2024; Mirage Project contributors, 2025). Pipeline-only depth tweak $\Rightarrow$ sync must regenerate ring indices; never patch barrier ids in emitted CUDA (SemiAnalysis, 2024; Chen, 2020; Vellaisamy *et al.*, 2026).

Pass search for minimal scope. For each dependency edge, sync pass tries warp $\rightarrow$ mbarrier $\rightarrow$ block in order—stops at first legal primitive (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024; NVIDIA, 2022; Chen, 2020). Widening requires proof that narrower scopes cannot express the dependency (Wu *et al.*, 2024; Mirage Project contributors, 2025; Davies *et al.*, 2025).

Depth-sensitive barrier rings. Pipeline depth D from Ch. 8 implies D mbarrier slots in ring—sync pass generates consistent cyclic order (SemiAnalysis, 2024; Dao *et al.*, 2022; Shah *et al.*, 2024). Changing D without regenerating sync IR is high-risk merge (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a).

Precision and layout interaction. INT4/FP8 packing changes which warps touch which bytes—barrier byte counts must update with dtype lowering (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). Sync pass re-runs when memory pass changes tile layout (Wu *et al.*, 2024; Chen, 2020; Dao *et al.*, 2022).

Reproducibility across GPU generations. Hand-tuned barrier counts assuming fixed warp counts break on next SKU (NVIDIA, 2022; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Regenerated sync IR from dependency graph ports across fleet (Chen and YiRage Contributors, 2026; Chen, 2020; Mirage Project contributors, 2025).

✓ **Practice checkpoint.** Archive sync-pass certificate on every barrier-lowering change (Chen, 2020; Chen and YiRage Contributors, 2026).

9.7 Sync Pitfalls, Checklist, and Debug

9.7.1 Five common sync anti-patterns

1. **Over-coarse sync.** *Symptom:* ms/token up after “adding safety sync” (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Shah *et al.*, 2024). *Root cause:* block sync replaced mbarrier handoff (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026). *Fix:* narrow scope per Table 9.1 (NVIDIA, 2022). *Prevention:* scope justification on every sync site (Chen, 2020). *Decode note:* at batch-1, one unnecessary `__syncthreads` per layer across 32 layers can add tens of microseconds per token—often larger than the arithmetic saved by fusion (Davies *et al.*, 2025; Chen, 2026a).
2. **Barrier pairing error.** *Symptom:* hang or rare corruption on multi-buffer pipeline (Shah *et al.*, 2024; SemiAnalysis, 2024). *Root cause:* arrive/wait count or byte mismatch (NVIDIA, 2022; Chen and YiRage Contributors, 2026). *Fix:* regenerate from pipeline IR; golden diff (Wu *et al.*, 2024; Chen,

2020). *Prevention*: static arrival check (Table 9.4) (Chen and YiRage Contributors, 2026; Chen, 2020). *Decode note*: weight and KV streams each carry independent rings—swapping b_0 indices across streams is a common copy-paste failure (Kwon *et al.*, 2023; Dao *et al.*, 2023).

3. **Conditional barrier.** *Symptom*: nondeterministic hang (NVIDIA, 2022; Shah *et al.*, 2024). *Root cause*: barrier inside branch not taken by all warps (Chen and YiRage Contributors, 2026). *Fix*: hoist sync; restructure control flow (Chen, 2020; Davies *et al.*, 2025). *Prevention*: CFG coverage in sync pass (Wu *et al.*, 2024). *Decode note*: variable sequence length tempts `if (len > bucket)` around barriers—use separate graph buckets with uniform CFG per bucket instead (NVIDIA, 2024; Kwon *et al.*, 2023).
4. **CPU false sharing.** *Symptom*: CPU decode slower than single-thread (Goodfellow *et al.*, 2016; Chen, 2026a). *Root cause*: hot counters on shared cache line (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). *Fix*: line padding; per-thread private accumulators (Chen, 2020). *Prevention*: layout review on parallel loops (Chen and YiRage Contributors, 2026). *Decode note*: speculative decoding threads updating a shared acceptance counter is a frequent offender—pad to 64 B or shard counters per thread (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).
5. **NPU token omission.** *Symptom*: array frozen mid-layer (AMD, 2024b,a). *Root cause*: wait without matching DMA complete token (Chen and YiRage Contributors, 2026). *Fix*: complete token graph; re-validate statically (Wu *et al.*, 2024; Chen, 2020). *Prevention*: token coverage check (Table 9.4) (AMD, 2024b; Chen and YiRage Contributors, 2026). *Decode note*: last tile in spatial chain still needs forward token to host or next subgraph—omitting “tail” token hangs the entire array (AMD, 2024a; Davies *et al.*, 2025).

Debug playbook. Hang $\rightarrow$ dump last mbarrier/token index $\rightarrow$ compare pipeline IR $\rightarrow$ run sync-pass bisect (Shah *et al.*, 2024; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a). Corruption without hang $\rightarrow$ race before sync—check byte counts and ordering (SemiAnalysis, 2024; Dao *et al.*, 2022; NVIDIA, 2022).

Hang bisect procedure (step-by-step). 1. **Capture**: GPU hang dump includes PC, barrier id, warp mask, context bucket id (NVIDIA, 2022; Shah *et al.*, 2024). 2. **Map id**: look up barrier id in sync IR export—identify stream (weight vs KV), buffer slot, producer/consumer role ids (Chen and YiRage Contributors,

2026; SemiAnalysis, 2024). **3. CFG check:** verify active bucket’s CFG path includes that barrier for all participating warps (NVIDIA, 2024; Davies *et al.*, 2025). **4. Pairing check:** compare arrive count logged in debug build vs expected from pipeline depth D (Dao *et al.*, 2022; Chen, 2020). **5. Bisect pass:** replay subgraph with sync pass on/off in simulator; if off hangs disappear, fault is in inserted sync—if on still hangs, fault is in pipeline IR (Wu *et al.*, 2024; Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025). **6. Fix path:** regenerate from pipeline IR (Pattern 2), never add block sync as step 6 shortcut (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Median time-to-root-cause under 2 hours when postmortem bundle (Pattern 6) is complete—without bundle, teams loop for days (Shah *et al.*, 2024; Chen, 2026a).

9.7.2 Sync design self-check

- ☐ Every dependency uses minimal-scope primitive
- ☐ All barrier/token arrive and wait counts match on all CFG paths
- ☐ No sync inside divergent branches
- ☐ Wait-for graph acyclic
- ☐ Decode reductions use shuffle/atomics where possible
- ☐ CPU false sharing and NUMA remote atomics reviewed
- ☐ Quadruple legality (Ch. 6–9) archived

✓ **Practice checkpoint.** Block merge without checklist + sync-pass certificate (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026).

9.8 Sync Engineering Patterns

Pattern 1: Scope worksheet. **Scene:** engineer adds `__syncthreads` “to be safe” (Shah *et al.*, 2024; Vellaisamy *et al.*, 2026). **Solution:** list dependency units; pick narrowest row in Table 9.1 (Chen and YiRage Contributors, 2026; NVIDIA, 2022). **Benefit:** avoids decode critical-path tax (Davies *et al.*, 2025; Chen, 2026a). **When:** every new sync site (Chen, 2020).

Pattern 2: mbarrier ring from pipeline IR. **Scene:** hand-edited barrier indices on ping-pong (SemiAnalysis, 2024; Dao *et al.*, 2022). **Solution:** generate ring from Ch. 8 depth D and buffer map (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). **Benefit:** pairing errors eliminated (Wu *et al.*, 2024; Chen, 2020). **When:** any pipeline depth change (Vellaisamy *et al.*, 2026).

Pattern 3: Shuffle-before-sync. **Scene:** block barrier for softmax partial max (Dao *et al.*, 2022; Shah *et al.*, 2024). **Solution:** warp shuffle reduce; atomic only if cross-warp needed (Davies *et al.*, 2025; Chen, 2026a). **Benefit:** removes full CTA stall (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). **When:** decode attention stats (Kwon *et al.*, 2023).

Pattern 4: Tri-hardware sync row. **Scene:** GPU mbarrier plan ported literally to NPU (AMD, 2024b; Vellaisamy *et al.*, 2026). **Solution:** Table 9.3 per backend lowering (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). **Benefit:** fleet-safe coordination (Chen, 2020; Wu *et al.*, 2024). **When:** multi-backend builds (Mirage Project contributors, 2025).

Pattern 5: Sync golden IR. **Scene:** barrier drift on rebase (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). **Solution:** CI golden sync IR + certificate (Wu *et al.*, 2024; Chen, 2020). **Benefit:** hangs caught pre-silicon (Vellaisamy *et al.*, 2026; Mirage Project contributors, 2025). **When:** sync pass or lowering changes (NVIDIA, 2022; AMD, 2024a).

Pattern 6: Hang postmortem bundle. **Scene:** production hang with fused kernel (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). **Solution:** pipeline IR + sync IR + last barrier index + CFG bucket id (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). **Benefit:** fast root-cause (SemiAnalysis, 2024; Chen, 2020). **When:** incident response (Chen, 2026a; Mirage Project contributors, 2025).

Pattern usage. Patterns 1–3 on kernel PRs; Patterns 4–5 in CI; Pattern 6 in on-call runbooks (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Shah *et al.*, 2024; Davies *et al.*, 2025).

Pattern selection guide. **New sync site?** Start Pattern 1 worksheet before writing CUDA (Chen, 2020; NVIDIA, 2022). **Pipeline depth change?** Pattern 2 regeneration mandatory—no manual barrier indices (SemiAnalysis, 2024; Chen and

YiRage Contributors, 2026). **Softmax or norm reduction?** Pattern 3 shuffle path—defer block sync until Ch. 10 carrier proof requires it (Dao *et al.*, 2022; Shah *et al.*, 2024). **Fleet triplet build?** Pattern 4 row per backend in CI matrix (AMD, 2024b; Davies *et al.*, 2025; Mirage Project contributors, 2025). **Compiler change?** Pattern 5 golden diff on every merge to `sync_pass/` (Wu *et al.*, 2024; Chen and YiRage Contributors, 2026). **Production incident?** Pattern 6 bundle within 30 minutes of hang—barrier index + IR hashes narrow bisect (Vellaisamy *et al.*, 2026; Shah *et al.*, 2024). Patterns are not optional style—they are the operational encoding of Sections 9.2–9.6 (Chen, 2020; Mirage Project contributors, 2025).

9.9 Sync Review Workflow

R1–R5. R1: scope worksheet per sync site; R2: sync-pass certificate; R3: quadruple legality bundle; R4: golden IR diff; R5: checklist (Chen and YiRage Contributors, 2026; Chen, 2020; Shah *et al.*, 2024; Wu *et al.*, 2024). **Gate A:** sync pass failure (Vellaisamy *et al.*, 2026). **Gate B:** coarse sync on decode hot path without justification (Davies *et al.*, 2025; Chen, 2026a). **Gate C:** hang repro or watchdog in staging (Shah *et al.*, 2024; Mirage Project contributors, 2025).

Reviewer questions. Is every Ch. 8 handoff covered by `mbarrier/token`? (Semi-Analysis, 2024; Chen and YiRage Contributors, 2026) Any `__syncthreads` on pipeline path? (Shah *et al.*, 2024; NVIDIA, 2022) CFG buckets all proven? (NVIDIA, 2024; Davies *et al.*, 2025) CPU layout padded against false sharing? (Goodfellow *et al.*, 2016; Chen, 2020)

PR rubric (expanded). **Scope evidence:** worksheet row per sync site with primitive and justification (Chen and YiRage Contributors, 2026; Chen, 2020). **Proof evidence:** sync-pass certificate hash matches CI golden (Wu *et al.*, 2024; Mirage Project contributors, 2025). **Performance evidence:** Nsight stall delta or ms/token A/B from Table 9.6 (Davies *et al.*, 2025; Shah *et al.*, 2024). **Structural evidence:** quadruple legality bundle unchanged or intentionally updated with all four reports (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022). **Negative checklist:** confirm no conditional barriers, no hand-edited ring indices, no block sync replacing `mbarrier` handoff (SemiAnalysis, 2024; NVIDIA, 2022). Reviewers reject “LGTM” without certificate attachment—same culture as Ch. 8 pipeline IR reviews (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

✓ **Practice checkpoint.** Merge blocked without sync certificate + checklist (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026).

9.10 Decode Sync Benchmarking Methodology

Measure sync cost the way Chapter 3 measures decode—not FLOPs alone (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Minimum benchmark bundle. Batch-1 ms/token: kernel with minimal-scope sync vs coarse-sync variant on *same* fusion region (Shah *et al.*, 2024; Chen, 2026a; Dao *et al.*, 2022). Nsight: barrier wait cycles, `__syncthreads` stall time, mbarrier wait histogram (NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025). Bytes/token and correctness hash unchanged—sync optimization must not trade bytes or numerics (Vellaisamy *et al.*, 2026; Chen, 2020; Chen and YiRage Contributors, 2026).

Nsight metrics interpretation. **Barrier wait cycles:** if mbarrier wait exceeds compute on the same stage, check producer TMA latency or incorrect phase—not always “add more sync” (SemiAnalysis, 2024; Shah *et al.*, 2024). **Stall reasons on `__syncthreads`:** high count on decode path is a red flag—map each stall to sync site worksheet; justify or remove (Davies *et al.*, 2025; NVIDIA, 2022). **Warp issue stall (not selected):** often follows barrier imbalance—one warp arrives early and sleeps while others finish divergent work before the barrier (NVIDIA, 2022; Chen and YiRage Contributors, 2026). Report before/after stall histograms in PR template Table 9.6—numbers beat adjectives in review (Chen, 2020; Vellaisamy *et al.*, 2026).

Acceptance thresholds (decode). On batch-1 Llama-class kernels, coarse-sync A/B regression > 5% ms/token without correctness fix is a revert unless scope worksheet proves block barrier unavoidable (Davies *et al.*, 2025; Chen, 2026a). Zero hangs in 24h soak at max context bucket; zero sync-pass failures on golden IR diff (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025). CPU: LLC coherence traffic should not rise when parallelizing decode helper loops—if it rises, suspect false sharing (Goodfellow *et al.*, 2016; Chen, 2020). NPU: MAC utilization should not drop below pipeline pass prediction because token wait chains are too long—shorten spatial chain or overlap streams (AMD, 2024b,a).

Scope A/B discipline. Only change sync primitive scope between A and B—hold Ch. 6–8 IR constant (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Dao *et al.*, 2022). Document each sync site scope in metadata for reproducibility (Chen, 2020; Mirage Project contributors, 2025; Shah *et al.*, 2024).

Hang testing. Stress context buckets, batch sizes, and warp counts where CFG differs (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025). Soak test with watchdog—hangs may be rare; coverage without soak is insufficient for sync (Vellaisamy *et al.*, 2026; Mirage Project contributors, 2025; Chen and YiRage Contributors, 2026).

CPU/NPU sync metrics. CPU: remote NUMA atomic rate, LLC coherence traffic (Goodfellow *et al.*, 2016; Davies *et al.*, 2025; Chen, 2026a). NPU: token wait cycles, MAC idle while DMA pending (AMD, 2024b,a; Chen and YiRage Contributors, 2026).

✓ **Practice checkpoint.** Attach benchmark bundle on every sync-scope change PR (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026).

9.11 Megakernel Sync Composition (Chapters 6–9)

Synchronization is the glue that makes Ch. 6–8 composable—without proof, fusion is unsafe at fleet scale (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Mirage Project contributors, 2025; Shah *et al.*, 2024).

Four-pass certification story. Memory pass certifies bytes fit (Dao *et al.*, 2022; Chen, 2020). Tiling pass certifies roles are static (Shah *et al.*, 2024; Mirage Project contributors, 2025). Pipeline pass certifies movement overlaps compute (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026). Sync pass certifies coordination cannot deadlock (Wu *et al.*, 2024; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Superoptimizer (Ch. 20) proposes; passes disprove bad candidates early (Mirage Project contributors, 2025; Vellaisamy *et al.*, 2026; Chen, 2020).

Why static roles unlock minimal sync. Dynamic assignment would require pessimistic wide barriers—“sync everyone in case” (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Shah *et al.*, 2024). Ch. 7 role IDs give sync pass a precise participant set

per edge (Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025; NVIDIA, 2022). This is the performance–correctness coupling of the MegaKernel stack (Dao *et al.*, 2022; Chen, 2026a; SemiAnalysis, 2024).

Pipeline handoffs are sync edges. Every Ch. 8 mbarrier is a sync-pass edge with byte-count semantics (SemiAnalysis, 2024; Shah *et al.*, 2024; Dao *et al.*, 2022). Editing pipeline depth without regenerating sync proof is a top hang vector (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Wu *et al.*, 2024). Keep pipeline IR and sync IR hashes paired in CI (Chen, 2020; Mirage Project contributors, 2025).

Barrier objects in residency. mbarrier arrays, token slots, collective scratch—count in Ch. 6 peak like any other fast-memory resident (Chen, 2026a; NVIDIA, 2022; AMD, 2024b). Teams that forget barrier metadata spill or truncate depth silently (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Chen, 2020).

Explicit vs implicit sync spectrum. GPU: explicit barriers on partly dynamic machine (NVIDIA, 2022; Shah *et al.*, 2024). CPU: implicit coherence with hidden cost (Goodfellow *et al.*, 2016; Davies *et al.*, 2025; Chen, 2026a). NPU: fully explicit tokens—fully analyzable (AMD, 2024b,a; Chen and YiRage Contributors, 2026). YiRage lowers one dependency graph three ways—Table 9.3 (Chen, 2020; Wu *et al.*, 2024; Mirage Project contributors, 2025).

Bulk-synchronous vs async trade (again). Per-operator stacks sync once per kernel—simple, slow (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Fused async kernels sync many times with narrow scope—fast when proven (Shah *et al.*, 2024; SemiAnalysis, 2024; Dao *et al.*, 2022). Deadlock risk is the price of overlap—pay with compile-time proof, not production hangs (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Mirage Project contributors, 2025).

Softmax preview (Ch. 10). Online softmax needs warp-level reductions for (m, ℓ) stats—shuffle before any block sync (Dao *et al.*, 2022; Shah *et al.*, 2024; Chen, 2026a). Carriers must stay consistent across pipeline stages sync pass already proved (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; SemiAnalysis, 2024). Ch. 10 specializes carriers; Ch. 9 supplies scope discipline (Dao *et al.*, 2022; Chen, 2020).

Incident response playbook. On hang: capture barrier/token index, pipeline depth, context bucket, quadruple IR hashes (Vellaisamy *et al.*, 2026; Shah *et al.*, 2024; Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025). Bisect: sync pass on $\rightarrow$ off (if safe in lab); compare certificates (Wu *et al.*, 2024; Chen, 2020; SemiAnalysis, 2024). Never “fix” hangs by adding block sync without scope worksheet (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026).

Chapter 20 search interaction. Megakernel candidates mutate fusion, tiles, depth—each mutation re-runs sync pass (Mirage Project contributors, 2025; Wu *et al.*, 2024; Chen and YiRage Contributors, 2026). Unprovable variants pruned before codegen saves compile farm and prevents toxic kernels entering soak (Chen, 2020; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Serving fleet checklist (operator). Version sync certificate with model weights; bucket graphs by context; alert on certificate drift in CI (NVIDIA, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). Treat certificate mismatch as release blocker—same severity as weight checksum failure (Vellaisamy *et al.*, 2026; Mirage Project contributors, 2025; Chen, 2020).

Anti-pattern culture (recap). No merge without sync pass green; no coarse sync without written justification; no conditional barriers—ever (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024; NVIDIA, 2022; SemiAnalysis, 2024). Patterns in Section 9.8; gates in Section 9.9 (Chen, 2020; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

End-to-end certification narrative. A megakernel candidate flows: superoptimizer proposes fusion graph $\rightarrow$ memory pass proves bytes $\rightarrow$ tiling pass assigns static roles $\rightarrow$ pipeline pass schedules movement $\rightarrow$ **sync pass proves coordination** $\rightarrow$ codegen (Wu *et al.*, 2024; Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025; Shah *et al.*, 2024). Removing sync pass from CI is equivalent to shipping without tests—numerical tests do not observe deadlock (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). The certificate is the contract between compiler and serving: if certificate hash matches golden, operators trust barrier indices across deploys (Chen, 2020; Chen and YiRage Contributors, 2026).

Teaching lab (90-minute). **Part A:** students classify ten dependency edges into scope ladder rows (Shah *et al.*, 2024; Goodfellow *et al.*, 2016). **Part B:** inject

one conditional barrier into sample IR; run sync pass; read error path output (Chen and YiRage Contributors, 2026; Chen, 2020). **Part C:** Nsight compare minimal vs coarse sync on provided decode kernel (Davies *et al.*, 2025; NVIDIA, 2022). **Deliverable:** completed checklist + one-page scope worksheet—mirrors production review Gates A–C (Vellaisamy *et al.*, 2026; Mirage Project contributors, 2025).

✓ **Practice checkpoint.** Archive quadruple legality + sync certificate together on every fused-kernel release (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Mirage Project contributors, 2025; Dao *et al.*, 2022; Shah *et al.*, 2024; Chen, 2020; Vellaisamy *et al.*, 2026).

9.12 Key Takeaways

Synchronization turns Ch. 6–8 performance into *safe* performance—minimal scope, static proof, per-target lowering (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024; Dao *et al.*, 2022; Mirage Project contributors, 2025).

- **Minimal-scope principle.** Cost rises from warp to collective—use narrowest primitive that satisfies the dependency (Table 9.1) (Shah *et al.*, 2024; Davies *et al.*, 2025). **Action:** scope worksheet on every sync site (Chen and YiRage Contributors, 2026; Chen, 2020).
- **Decode-adaptation principle.** Thin per-token work makes coarse sync disproportionately expensive (Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023). **Action:** shuffle/atomics for reductions; mbarrier for pipeline handoffs only (Dao *et al.*, 2022; SemiAnalysis, 2024).
- **Static-proof principle.** Deadlock is invisible to numerical tests—prove at compile time (Shah *et al.*, 2024; Wu *et al.*, 2024; Chen and YiRage Contributors, 2026). **Action:** require sync-pass certificate in CI (Chen, 2020; Mirage Project contributors, 2025).
- **Hardware-adaptation principle.** GPU explicit barriers; CPU coherence+fences; NPU static tokens (NVIDIA, 2022; Goodfellow *et al.*, 2016; AMD, 2024b,a). **Action:** fill Table 9.3 per backend (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).
- **Compiler-internalization principle.** Sync insertion belongs in YiRage pass, not hand CUDA (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024;

Shah *et al.*, 2024; Mirage Project contributors, 2025). **Action:** golden sync IR + quadruple legality bundle (Chen, 2020; Vellaisamy *et al.*, 2026).

How this chapter connects. Ch. 8 pipelines need Ch. 9 proofs; Ch. 7 roles enable minimal edges; Ch. 6 budgets barrier objects (SemiAnalysis, 2024; Shah *et al.*, 2024; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). Chapter 10 applies these rules to online softmax—carriers and running stats with tight warp-level sync (Dao *et al.*, 2022; Chen, 2026a; Davies *et al.*, 2025).

9.13 Conclusion

Hierarchical synchronization completes the MegaKernel safety stack: residency, roles, pipelines, and now *proved* coordination (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024; Dao *et al.*, 2022; Mirage Project contributors, 2025; Wu *et al.*, 2024).

Closing metaphor. Ch. 8 timed the conveyor; Ch. 9 installs the sensors and interlocks so boxes never collide (SemiAnalysis, 2024; Shah *et al.*, 2024; Dao *et al.*, 2022). Speed without interlocks is a recall event (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2026a).

Chapter 10 develops **online softmax**—the algorithm where running statistics must stay resident and synchronized across the pipeline stages this chapter secured (Dao *et al.*, 2022; Shah *et al.*, 2024; Chen, 2026a). Carry quadruple legality bundle and mbarrier index table into softmax carrier design (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024).

MegaKernel spine recap. Residency (Ch. 6) → roles (Ch. 7) → pipelines (Ch. 8) → **sync proof** (Ch. 9) → online softmax carriers (Ch. 10) → full decoder assembly (Ch. 11) (Dao *et al.*, 2022; Mirage Project contributors, 2025; Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). Each link is a compile-time pass with artifacts; skipping one link voids the chain (Wu *et al.*, 2024; Chen, 2020; Vellaisamy *et al.*, 2026).

Done criteria. Scope worksheets signed, sync-pass certificate archived, checklist complete, no unjustified coarse sync on decode path (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen,

2020). Proceed to Chapter 10 with carrier sync plan attached (Dao *et al.*, 2022; SemiAnalysis, 2024).

Hands-on recap. (1) Fill scope ladder per sync site. (2) Run sync pass. (3) Archive certificate. (4) Quadruple legality with Ch. 6–8. (5) Complete checklist (Vellaisamy *et al.*, 2026; Shah *et al.*, 2024; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Mirage Project contributors, 2025). Performance without proof is not shippable megakernel engineering (Wu *et al.*, 2024; Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025).

Chapter artifact bundle (ship with kernel). Export four files on every release: `sync_ir.json`, `sync_certificate.sha256`, `scope_worksheet.md`, `quadruple_legality.pdf` (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026). On-call playbooks reference barrier index table derived from sync IR—when hang dumps “barrier 3”, operators map to KV stream buffer 1 consumer wait (Shah *et al.*, 2024; SemiAnalysis, 2024; Davies *et al.*, 2025). Ch. 10 softmax carriers inherit the same discipline: running max and sum reductions use shuffle paths proven here; carrier handoffs across tiles reuse mbarrier edges from pipeline IR (Dao *et al.*, 2022; Chen, 2026a; Chen and YiRage Contributors, 2026).

Teaching vs production. Whiteboard scope ladder (Figure 9.2) for onboarding (Shah *et al.*, 2024; Goodfellow *et al.*, 2016). Automate Patterns 1–6 and sync golden IR in CI for fleet scale (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Wu *et al.*, 2024; Mirage Project contributors, 2025). Concepts in §9.2–9.3; gates in §9.7–§9.9 (Chen, 2020; Vellaisamy *et al.*, 2026).

Fleet note. Triplet deployments need three sync lowerings from one IR—mbarrier plan must not ship verbatim to XDNA tokens (AMD, 2024b; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; NVIDIA, 2022; Chen, 2020).

Long-context note. Paged KV changes which warps touch which barriers per bucket—re-prove sync per graph bucket (Kwon *et al.*, 2023; Dao *et al.*, 2023; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Shah *et al.*, 2024).

One-sentence chapter thesis. Ch. 6–8 make decode fast; Ch. 9 makes that speed *safe*—minimal-scope sync with compile-time deadlock proof (Dao *et al.*, 2022; Shah *et al.*, 2024; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026;

Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Mirage Project contributors, 2025; Wu *et al.*, 2024).

Part IV milestone. With Ch. 9 complete, Part IV has covered the four MegaKernel structure pillars—residency, roles, pipelines, and synchronization—that turn operator graphs into provably correct fused decode kernels (Dao *et al.*, 2022; Mirage Project contributors, 2025; Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). Part V begins with algorithm specialization: online softmax carriers that consume the sync discipline established here (Dao *et al.*, 2022; Chen, 2026a; Davies *et al.*, 2025).

Exit artifacts. Leave with: scope worksheets, sync-pass certificate, quadruple legality bundle, checklist, mbarrier/token index table for Ch. 10 (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Shah *et al.*, 2024; SemiAnalysis, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Proceed to Chapter 10 only when sync proof is green (Dao *et al.*, 2022; Chen, 2026a; Mirage Project contributors, 2025).

Reader commitment. Before closing this chapter, run sync pass on your current fused decode kernel and archive the certificate—if the pass fails, you have found the bug before your users do (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Vellaisamy *et al.*, 2026; Shah *et al.*, 2024). Correctness is not a phase after performance; it is the constraint that makes performance shippable (Mirage Project contributors, 2025; Davies *et al.*, 2025; Chen, 2020).

End of Chapter 9—correctness pillar complete for decode serving. Archive the quadruple legality bundle before starting Chapter 10 softmax carrier worksheets (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Chen, 2020).

Table 9.3: Tri-hardware synchronization comparison (decode): primitives, cost, and proof strength.

Platform	Core primitives	Scheduling	Cost profile	Decode rules	Deadlock proof
GPU (Hopper)	Transaction mbarrier, <code>__syncthreads</code> , atomics, warp shuffle ^{8.3}	Explicit program- dynamic warp scheduler; scope configurable	Scope grows steeply: warp ~ 0 ; mbarrier ~ 10 – 20 cyc.; block ~ 50 – 100 cyc.; grid μs	(1) Pipeline handoff: point-to-point mbarrier only—no block sync fallback (2) Reductions: shuffle/atomics before block sync (3) Minimize scope—coarse sync dominates batch-1 steps	Partial static proof; dynamic warp paths need runtime watchdog
CPU	acquire/release fences, atomics, coherence protocol	HW maintains coherence; fences/atomics constrain ordering explicitly	Hidden cost: local fast; remote NUMA atomics 2 – $3\times$; false sharing inflates traffic	(1) Partition data—minimize cross-core sharing (2) Lightest acquire/release; avoid <code>seq_cst</code> on hot path (3) Pad to cache line—no false sharing	Hard to prove fully static; runtime tools; high debug cost
NPU (XDNA/AIE)	Completion fences, stream sync, tile locks ^{5.3}	100% compile-time static; all token pairs fixed at build time	Deterministic pipeline slot; mis-planning stalls array permanently—no scheduler recovery	(1) Strict token pairing—every wait has a producer (2) Account tile-chain latency; balance spatial pipeline (3) No runtime dynamic sync—compile-time covers all paths	Full static proof: dependency cycle + token coverage checks

Table 9.4: Static deadlock checks (YiRage sync pass).

Check	Method	Catches
Arrival parity	CFG unit count vs barrier n	Divergent barrier
Wait-for cycle	Graph cycle detection	Circular deadlock
Ring order	Buffer index monotonicity	Pipeline re-order bug
Token coverage	Each wait has producer	NPU hang
Collective match	Rank-symmetric edges	Multi-GPU hang

Table 9.5: YiRage sync pass: I/O and verification dimensions.

Item	Content	On failure
Input	Roles + pipeline IR + dep graph	—
Output	Sync IR + proof certificate	Error with site/path
Scope min	Narrowest primitive per edge	Widen only if required
Arrival parity	CFG path counts	Barrier id + path
Acyclic wait	Wait-for DAG	Cycle nodes listed
Memory order	Acquire/release match	Edge + ordering bug
Multi-device	Collective symmetry	Rank mismatch hint

Table 9.6: Sync optimization PR evidence template.

Field	Required content
Scope worksheet	Per-site primitive + justification
Sync certificate	Pass output hash
Stall delta	Barrier wait before/after
ms/token	Batch-1 decode
Quadruple legality	Ch. 6–9 reports

Chapter 10

Attention Online Softmax Optimization

Can software alone recover decode when interconnect and HBM budgets are tight? Fregly’s DeepSeek case study shows that mechanical sympathy—algorithms written for the memory hierarchy they run on—can match frontier quality on constrained H800 clusters (Liu *et al.*, 2024; Davies *et al.*, 2025). The same principle applies at batch-1 decode inside a single layer: naive attention materializes an $S \times S$ softmax map, and even a streaming $1 \times T$ score row becomes catastrophic when fused boundaries force write/read pairs between KV blocks (Dao *et al.*, 2022, 2023; Chen, 2026a).

Online softmax is the attention-side embodiment of that sympathy: instead of storing $P = \text{softmax}(QK^\top)$, the kernel maintains running statistics (m, ℓ) (row max and sum of exponentials) and a partial value accumulator o , updating all three as each KV block arrives (Dao *et al.*, 2022). FlashAttention made IO-aware tiling mainstream; Flash-Decoding parallelizes the KV dimension while preserving online state (Dao *et al.*, 2023). This chapter explains the numerics, residency tables on GPU/XDNA/CPU, and the **fusion boundaries** where online carriers must hand off to PV and Chapter 11 MegaKernels without touching HBM (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020).

The common production mistake is treating softmax as a cheap epilogue after a GEMM. At batch-1 decode, softmax is not one kernel—it is a *state machine* embedded inside the KV loop (Kwon *et al.*, 2023; Davies *et al.*, 2025). TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; online softmax alone will not collapse that count—fusion with QKV and PV inside a MegaKernel will (Chapter 11) (Vellaisamy *et al.*, 2026; NVIDIA, 2024). But

MegaKernels *require* online carriers to stay in registers or shared memory across the attention–MLP seam (SemiAnalysis, 2024; AMD, 2024a). Spill (m, ℓ, o) between blocks and you pay full-width traffic every context step while launches/token flatline (Chen, 2026a).

Decode vs prefill softmax shape. Prefill processes many query rows at once; decode adds *one* new query row against entire KV history (Dao *et al.*, 2023; Kwon *et al.*, 2023). Online softmax in decode is a loop over KV *blocks*, not a single matmul epilogue (Dao *et al.*, 2022). Benchmarks that only tune prefill softmax miss the carrier residency story that determines whether Chapter 11 fusion is even legal (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Training vs inference scope. Online softmax also accelerates training attention for long sequences—FlashAttention’s original motivation (Dao *et al.*, 2022). This chapter emphasizes decode because batch-1 residency and MegaKernel fusion are decode-specific engineering pressures (Vellaisamy *et al.*, 2026; Dao *et al.*, 2023; Davies *et al.*, 2025). Training chapters in other books cover overlapping comm/compute; our decode compiler story stands alone (Goodfellow *et al.*, 2016; Chen, 2020; Chen and YiRage Contributors, 2026).

Goodput for attention. Raw FLOPs on $QK^\top$ mislead: useful decode work is the final attention output bytes delivered per ms/token, discounting materialized P , redundant HBM round trips, and extra launches (Davies *et al.*, 2025; Chen, 2026a). Online softmax raises attention goodput by keeping exponentials and partial PV on-chip—the attention analog of Meta’s goodput metric for training clusters (Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026).

10.1 Online Softmax

Step-by-step decode loop (one head). At token t , the query vector q is fixed. For each KV block $b = 1 \dots B$:

1. TMA or vector loads fetch K_b, V_b tiles into on-chip storage (SemiAnalysis, 2024; Kwon *et al.*, 2023).
2. MMA or dot products form score tile s_b (NVIDIA, 2022; Dao *et al.*, 2022).
3. Warp max-reduce finds m_b ; threads compute $e^{s_b - m'_b}$ with updated global $\max m'_b$ (Dao *et al.*, 2022).

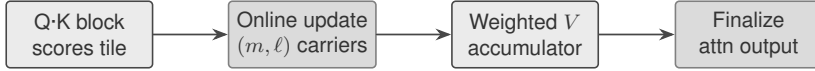


Figure 10.1: Online softmax pipeline: scores enter, carriers and partial PV stay on-chip across KV blocks.

4. Accumulate ℓ_b and $o_b = \sum \text{softmax}(s_b) V_b$ (Dao *et al.*, 2022, 2023).
5. Merge into running (m, ℓ, o) without global memory (Chen and YiRage Contributors, 2026; Chen, 2026a).

Finalize o/ℓ once after block B (Goodfellow *et al.*, 2016). This loop is the device-side spec Chapter 11 MegaKernels implement (Vellaisamy *et al.*, 2026; Chen, 2020).

Bytes/token estimate (sketch). Materializing P adds roughly $2T \cdot d_{\text{head}}$ bytes read/write per head per layer per token (scores + probabilities), ignoring cache effects (Chen, 2026a; Davies *et al.*, 2025). Online softmax removes the P term; remaining traffic is dominated by KV and weights (Kwon *et al.*, 2023; Dao *et al.*, 2022). Use this sketch to justify fusion PRs when microbench GEMMs look unchanged (Vellaisamy *et al.*, 2026).

Comparison to SDPA eager paths. PyTorch scaled dot-product attention may call cuDNN or flash backends depending on shape (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). At batch-1 decode on bandwidth-saturated GPUs, swapping SDPA variants without fixing launches/bytes can *regress* latency (Chen, 2026a; Vellaisamy *et al.*, 2026). Online softmax must be evaluated inside full-layer counters, matching Fregly’s profile-first methodology (Davies *et al.*, 2025).

The naive failure mode. Standard attention computes scores $s_{ij} = q_i \cdot k_j / \sqrt{d_h}$, applies softmax row-wise, multiplies by V (Goodfellow *et al.*, 2016; Dao *et al.*, 2022). Materializing P for decode means an $1 \times T$ vector per head per layer per token—small individually, catastrophic when each block boundary exports scores or probabilities to HBM (Vellaisamy *et al.*, 2026; Chen, 2026a). Online softmax avoids storing P entirely (Dao *et al.*, 2022).

For block t with local scores $x_j^{(t)}$, maintain running maximum m , sum $\ell = \sum \exp(x_j - m)$ across blocks seen so far, and accumulator $o = \sum_j \text{softmax}(x)_j v_j$ (Dao *et al.*, 2022). When a new block arrives with local max m_t and local sum

Table 10.1: Materialized softmax vs. online softmax (decode, one head).

Aspect	Materialized P	Online carriers
Extra HBM for P	$O(T)$ per head	$O(1)$ statistics
Launches if split out	+1 per block risk	In KV loop body
Carrier residency	N/A	Registers / shared mem
MegaKernel legality	Poor	Required
XDNA static plan	Often illegal	Assign (m, ℓ, o) slots

$\ell_t = \sum_j e^{x_j^{(t)} - m_t}$, update:

$$\begin{aligned}
m' &= \max(m, m_t), \\
\ell' &= \ell \cdot e^{m - m'} + \ell_t \cdot e^{m_t - m'}, \\
o' &= o \cdot e^{m - m'} + o_t \cdot e^{m_t - m'}.
\end{aligned}$$

After the final block, o/ℓ is the attention output row (Dao *et al.*, 2022, 2023). Figure 10.1 is the control-flow spine every fused decode kernel must implement (Chen and YiRage Contributors, 2026; Chen, 2020).

Log-sum-exp view. Equivalently, online softmax tracks $\log \ell$ and m so merges use stable log-add operations (Goodfellow *et al.*, 2016). Compiler backends may emit either multiplicative or log-space updates; IR must preserve associativity order for deterministic mode (Chen and YiRage Contributors, 2026; Chen, 2020). Mixing orders across GPU warps without documentation breaks bitwise regression tests (Davies *et al.*, 2025).

Multi-head carrier layout. With h heads, each head owns (m_h, ℓ_h, o_h) or shares warp groups (Dao *et al.*, 2022). MegaKernels pack carriers in shared memory with head-major or block-major layouts depending on MMA tile shapes (SemiAnalysis, 2024; NVIDIA, 2022). XDNA assigns fixed SRAM slots per head at compile time—dynamic head counts require separate compiled graphs (AMD, 2024a,b).

Table 10.1 explains why “online” is a **residency contract**, not an algorithmic nicety (Chen, 2026a; Davies *et al.*, 2025). GPU implementations keep (m, ℓ, o) in warp registers or shared memory for the duration of the KV block loop (SemiAnalysis, 2024; NVIDIA, 2022). Exporting carriers between blocks to global memory doubles traffic and breaks Chapter 11 fusion (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Paged KV interaction. vLLM-style block tables change *where* K/V bytes live, not whether online updates run each block (Kwon *et al.*, 2023). Device-side pointer walks replace scatter kernels but the inner softmax state machine is unchanged (Davies *et al.*, 2025). Block size interacts with tile shapes: 16-token blocks may underutilize bandwidth; 256-token blocks may exceed shared-memory staging (Dao *et al.*, 2023; SemiAnalysis, 2024).

Example 10.1.1. Decode cell (BS=1, ctx=2048, block=64): 32 KV blocks per head. Materialized softmax writes 32 partial P tiles to HBM; online softmax keeps three scalars/vectors per warp group on-chip (Dao *et al.*, 2022; Chen, 2026a; Vellaisamy *et al.*, 2026).

10.1.1 KV block and split sizing

Choosing how many KV positions one online step consumes is a joint constraint problem, not a fixed template. Each step needs one score tile, one K tile, one V tile, and room for the carriers; the product of head dimension, bytes per element, and the two KV copies determines how many bytes must arrive per step, while the on-chip budget decides whether a larger block or a second split is the better fit (Chen, 2026a; Dao *et al.*, 2022).

On Hopper-class GPUs the sizing loop starts from the memory system: pick a candidate block, compute its K/V staging against shared memory, subtract the score tile and carrier footprint, and accept the largest block that leaves a full double-buffer slot for the next tile (SemiAnalysis, 2024; NVIDIA, 2022). Blocks that undershoot the transfer granularity leave bandwidth idle; blocks that overshoot spill score tiles or force carrier export. Either failure shows up as bytes/token drift before latency does, so the worksheet must record the chosen block size next to the measured counters (Chen, 2026a; Vellaisamy *et al.*, 2026).

The sizing question has a second axis on parallel hardware: when one block would exceed per-warpgroup staging, split the KV range across groups and merge with the online equations from this section. Use the smallest split count that satisfies residency, because every additional split adds a merge step and a synchronization point (Dao *et al.*, 2023; Davies *et al.*, 2025). XDNA compiles the same choice into a static DMA schedule, so a block size that needs dynamic spill fails compile and makes the constraint visible earlier than on GPU (AMD, 2024a,b). CPU sizing replaces the shared-memory budget with L1/L2 cache lines and replaces split synchronization with SIMD lanes, but the ordering—bytes per step, staging budget, then merge cost—stays identical (Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026).

FlashAttention and Flash-Decoding. FlashAttention uses online softmax inside SRAM tiles for prefill-like parallelism (Dao *et al.*, 2022). Flash-Decoding splits KV across SMs while preserving online state per split, then merges splits with a second online reduction (Dao *et al.*, 2023). Decode MegaKernels must choose split strategy *inside* one launch or pay multiplied graph steps (Vellaisamy *et al.*, 2026; NVIDIA, 2024). Split-merge is easy to get wrong: an extra global sync per split negates Flash-Decoding wins (Davies *et al.*, 2025).

Split-KV merge protocol. When SM s finishes block range $[b_s, b_e)$ with local (m_s, ℓ_s, o_s) , a tree reduction combines pairs using the same merge equations until one global (m, ℓ, o) remains (Dao *et al.*, 2023). Implementations either use a dedicated merge kernel (bad for launches/token) or warp-specialized epilogue warps inside the MegaKernel (preferred) (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Profile merge separately—it becomes visible at long context when split count grows (Davies *et al.*, 2025).

GQA and MQA carrier savings. Grouped-query attention shares K/V across query heads, reducing KV bytes/token but not eliminating online softmax per query group (Liu *et al.*, 2024; Kwon *et al.*, 2023). Carrier count scales with query groups, not raw head count—compiler passes must not allocate per-head carriers when groups share scores (Chen and YiRage Contributors, 2026; Chen, 2020).

Carrier reset and query-row layout. Every decode token begins a fresh online softmax over the current KV history, so the carrier state resets once per query row, not once per kernel launch (Dao *et al.*, 2022; Kwon *et al.*, 2023). In batch-1 decode that reset is a register write before the first KV block; in chunked prefill or continuous batching, several query rows share the same KV pass and carriers must be laid out row-major so each row’s (m, ℓ, o) merges independently (Dao *et al.*, 2023; Davies *et al.*, 2025). A bug that reuses the previous token’s ℓ across steps is invisible in unit tests that run a single query but produces drifting attention weights in production, so reset behavior belongs in the same IR lifetime analysis as the carriers themselves (Chen and YiRage Contributors, 2026; Chen, 2020). Treat the reset as an explicit SSA initialization in lowering; a pass that hoists it out of the per-token loop is changing model semantics, not optimizing it (Chen, 2026a).

Recording the sizing decision. Block size, split count, and carrier layout are performance decisions with correctness impact: changing any of them changes the bytes/token worksheet and the merge tree that stability tests must cover

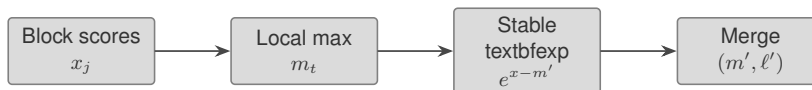


Figure 10.2: Numerical stability path: max-find precedes exponentials; merges preserve online invariants.

(Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Store them as IR attributes on the online softmax region so a later refactor cannot alter residency silently (Chen, 2020, 2026a). Code review should show the attributes changing when the kernel changes, which keeps autotuners honest about what they are actually searching (Davies *et al.*, 2025).

10.2 Numerical Stability

Error propagation across merges. Each merge is multiplicatively stable when m tracks the true row max (Dao *et al.*, 2022; Goodfellow *et al.*, 2016). If local max is underestimated, exponentials overflow and ℓ corrupts silently (Davies *et al.*, 2025). Tests must include adversarial score spikes at random positions within long blocks (Dao *et al.*, 2023; Chen and YiRage Contributors, 2026).

Determinism vs performance. Parallel max reductions across warps choose associativity order (NVIDIA, 2022; Chen, 2020). Compliance workloads may require deterministic reductions; interactive serving often tolerates bounded drift (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Document reduction order in kernel headers and YiRage lowering logs (Chen and YiRage Contributors, 2026).

Interaction with rotary embeddings. RoPE rotates q, k before $QK^\top$; it does not change softmax merge math but shifts score distributions with position (Liu *et al.*, 2024; Kwon *et al.*, 2023). Long-context stability tests must use RoPE-enabled references, not static q, k (Dao *et al.*, 2023; Davies *et al.*, 2025).

Why max-subtraction is non-optional. Softmax exponentials overflow in FP16/BF16 without subtracting row max (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). Online softmax *is* the stable formulation—but only if each block computes local max before **exp** (Dao *et al.*, 2022). A “fast path” that skips max on “small” blocks fails at long contexts when score magnitudes drift with RoPE position (Dao *et al.*, 2023; Chen, 2026a).

Table 10.2: Carrier dtype choices (engineering guidance).

Carrier	FP16	BF16	FP32
m (max)	Risky	OK	Best
ℓ (sum exp)	Underflow risk	OK	Best
o (value acc)	Rarely sufficient	Often OK	Best

Figure 10.2 is the micro-pipeline inside each KV block (Dao *et al.*, 2022). **Stable** implementations fuse max-reduce and scaled **exp** in shared memory to avoid an extra global round trip (SemiAnalysis, 2024; NVIDIA, 2022). Hopper SFUs accelerate **exp** relative to prior generations—mechanical sympathy at the silicon level for transformer attention (NVIDIA, 2022; Davies *et al.*, 2025).

FP8 and block scales. FP8 decode experiments carry per-block or per-tensor scales into the online merge (Davies *et al.*, 2025; Liu *et al.*, 2024). Re-quantizing scores in a separate kernel breaks fusion and often breaks stability unless scales participate in the max-find (Chen and YiRage Contributors, 2026; Chen, 2020). Treat scale vectors like carrier extensions in IR lifetime analysis (Chen, 2026a).

Precision pitfalls. Using FP16 for ℓ while FP32 for m is common; using FP16 for both can underflow ℓ at long T (Dao *et al.*, 2022; Davies *et al.*, 2025). BF16 often behaves better for decode carriers but still needs FP32 accumulation for o (Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026). Compiler passes should declare accumulator dtypes explicitly (Chen, 2020; Chen and YiRage Contributors, 2026).

Testing stability. Compare online vs reference double-softmax on random q, k, v across $T \in \{512, 2048, 8192\}$ (Dao *et al.*, 2023; Kwon *et al.*, 2023). Failures cluster at long contexts and low head dims (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Production gates require bounded-error tests before enabling fused paths in YiRage exports (Chen and YiRage Contributors, 2026; Chen, 2020).

Attention sinks and bias terms. Some models add attention sinks or ALiBi-style biases before softmax (Liu *et al.*, 2024). Online updates must incorporate biases into block-local max and exp before merge—a separate bias kernel between $QK^\top$ and softmax reintroduces Boundary A breaks (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022).

10.2.1 Causal masks, padding, and score biases

Decode normally makes the mask implicit: the current query attends only to KV positions written before it, so no upper-triangular filter exists inside the loop (Dao *et al.*, 2023; Kwon *et al.*, 2023). Masks become explicit in chunked prefill, when pages contain padding, when cross-attention reads encoder state, or when a model prepends a fixed sink token and must exclude filler rows (Liu *et al.*, 2024; Dao *et al.*, 2022).

Apply masked positions as $-\infty$ before the block-local max, never by subtracting a bias after *exp*. A post-hoc zero creates an apparent softmax mass that must be renormalized out later; a negative bias leaks a small nonzero weight into the accumulator; both corrupt (m, ℓ, o) in ways that grow with context length (Dao *et al.*, 2022; Chen, 2026a). The carrier update itself does not change: the block-local max still drives the exponentials, and masked lanes simply never contribute to ℓ or o (Dao *et al.*, 2023; Davies *et al.*, 2025).

Padding rows deserve a compile-time decision, not a per-position runtime branch. When the number of real tokens is known per request, choose the block layout and page order so masked lanes collapse at block edges; when padding is scattered, mask flags must feed the same reduction tree as scores so no lane produces a separate softmax over filler positions (Chen and YiRage Contributors, 2026; Chen, 2020). Attention sinks are the intentional exception: treat them as regular keys whose bias participates in the block-local max, because the sink only shapes the output if its score flows through the same stable *exp* path as real positions (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026).

10.3 Cross-Hardware Online Softmax

Bandwidth-normalized comparison. Report online softmax wins as Δ bytes/token and Δ kernels/token, not only ms/token (Chen, 2026a; Vellaisamy *et al.*, 2026). A CPU win of 5% latency with 40% bytes removed tells a different story than GPU ms shifts entangled with driver version (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

XDNA compile failures as signals. When XDNA rejects a schedule for carrier SRAM overflow, treat it as a fusion boundary violation to fix upstream—not as “NPU unsupported” (AMD, 2024a; Chen and YiRage Contributors, 2026). Successful GPU kernels that spill carriers are equally incorrect, just slower instead of failing compile (Chen, 2026a, 2020).

Edge Ryzen AI deployment. DDR-contended edge hosts mirror CPU patterns: KV bandwidth dominates; online softmax prevents an extra P pass but cannot fix unfused MLP boundaries (AMD, 2024b; Davies *et al.*, 2025). Edge fleets should still measure launches/token even when “launch” means DMA edge count (AMD, 2024a; Vellaisamy *et al.*, 2026).

Same math, different residency tables. Online softmax is target-agnostic in equations but not in memory hierarchy (AMD, 2024b; NVIDIA, 2022; Goodfellow *et al.*, 2016). A schedule legal on Hopper may illegalize on XDNA when carrier slots exceed tile SRAM (AMD, 2024a; Chen and YiRage Contributors, 2026). CPU decode uses SIMD registers for (m, ℓ) and blocked o updates with cache-aware KV reads (Goodfellow *et al.*, 2016; Davies *et al.*, 2025).

GPU (Hopper-class). Warp-level reductions compute block max; **exp** uses fast math intrinsics with documented error bounds (NVIDIA, 2022; SemiAnalysis, 2024). TMA loads K/V blocks while MMA computes $QK^\top$ fragments (SemiAnalysis, 2024). Carriers stay in registers owned by the softmax warpgroup—not in HBM (Chen, 2026a; Dao *et al.*, 2022). Flash-Decoding maps splits to SMs but must merge carriers without an extra launch when possible (Dao *et al.*, 2023; Vellaisamy *et al.*, 2026).

XDNA / AIE. Static dataflow assigns fixed SRAM addresses for (m, ℓ, o) for each pipeline stage (AMD, 2024a,b). Online updates become MAC/DMA microprograms; there is no dynamic spill (Chen and YiRage Contributors, 2026; Chen, 2020). If softmax carriers cannot be placed, the graph fails compile—correct outcome (AMD, 2024a; Chen, 2026a). Teams porting CUDA softmax often expect runtime allocation; XDNA requires upfront bucketization by context length (Kwon *et al.*, 2023; AMD, 2024b).

CPU. oneDNN/OpenBLAS-style paths vectorize **exp** and use FP32 accumulators (Goodfellow *et al.*, 2016). KV bandwidth dominates; online softmax savings are real but smaller than GPU HBM round-trip elimination (Davies *et al.*, 2025; Chen, 2026a). NUMA-local KV still matters for multi-socket hosts (Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Example 10.3.1. Identical Llama head at $\text{ctx}=2048$: Hopper fused online softmax keeps carriers in registers; XDNA maps them to tile slot #3; CPU holds m, ℓ in AVX registers—same invariants, different compiler legality checks (AMD, 2024b; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026).

Triplet regression. Never benchmark GPU only and infer XDNA behavior (AMD, 2024a; Davies *et al.*, 2025). YiRage triplet-test from one IR export should show bytes/token deltas, not just ms/token, so mechanism changes are visible across targets (Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024).

Blackwell / next-gen SFU note. Newer GPUs continue to widen SFU throughput for softmax-heavy decode (NVIDIA, 2022; Davies *et al.*, 2025). Online softmax still wins on bytes moved; faster **exp** alone does not remove materialized P (Chen, 2026a; Dao *et al.*, 2022). Codesign means updating tile sizes when SFU:MA ratios shift (SemiAnalysis, 2024).

10.4 Softmax Fusion Boundaries

Case study narrative (qualitative). Team A graph-captures unfused attention+MLP; Team B fuses online softmax with PV and keeps attn output in shared memory through SwiGLU (NVIDIA, 2024; Vellaisamy *et al.*, 2026). Team A improves host time; Team B improves bytes/token (Chen, 2026a; Davies *et al.*, 2025). Both claim “fusion”—only Team B advances mechanical sympathy (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026).

Autotuner guardrails. Search algorithms that reward FLOPs or occupancy may break Boundary B (Chen, 2020; Chen and YiRage Contributors, 2026). Hard-constrain IR: no global stores on carrier SSA values; penalize P materialization with infinite cost (Chen, 2026a; Vellaisamy *et al.*, 2026).

Serving engine integration. vLLM and TensorRT-LLM integrate flash-style attention but expose different fusion depth to custom compilers (Kwon *et al.*, 2023; Davies *et al.*, 2025). When exporting graphs to YiRage, preserve online regions through Boundaries A–B or explicitly document breaks for regression tests (Chen and YiRage Contributors, 2026; Liu *et al.*, 2024).

Serving engine integration contract. An engine that already calls a fused attention kernel still leaves an integration contract: which context buckets use the online path, which use eager fallback, and which page-table shapes are legal for the current carrier layout (Kwon *et al.*, 2023; Davies *et al.*, 2025). Record that boundary metadata at export time so a later framework upgrade cannot silently switch a decoder layer back to materialized softmax without a regression gate

Table 10.3: Fusion boundaries and decode metrics impact.

Boundary break	Launches/token	Bytes/token
Materialize P	+1 risk	High
Spill (m, ℓ, o)	0–1	Medium
Export attn out before MLP	+1	High
Full MegaKernel (Ch. 11)	$\leq 1/\text{layer}$	Lowest

noticing (Chen and YiRage Contributors, 2026; Chen, 2026a). The engine layer should be benchmarked end-to-end, but the bytes/token and kernels/token counters decide whether a given integration is really running the online path (Vellaisamy *et al.*, 2026; Chen, 2020).

Where online softmax ends. The **fuse** decision is not “softmax kernel + matmul kernel.” It is: do (m, ℓ, o) and the PV accumulator survive into the O projection and MLP without HBM export (Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022)? Chapter 11’s MegaKernel assumes the answer is yes (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Breaking fusion here forces separate launches and full-width tensors (NVIDIA, 2024; Kwon *et al.*, 2023).

Boundary A: after $QK^\top$, before PV . Fusing **online** softmax with PV inside the KV loop is the FlashAttention core (Dao *et al.*, 2022). Splitting materialized P to HBM between them negates IO savings (Chen, 2026a).

Boundary B: after attention output, before MLP. Even perfect online softmax inside attention fails if attn output is written to HBM before SwiGLU (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). This is the dominant MegaKernel seam (Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024).

Boundary C: debug stats export. Prefill microbenchmarks sometimes keep softmax stats for debugging—strip them in inference exports (Goodfellow *et al.*, 2016; NVIDIA, 2024). Accidental stat tensors reintroduce $O(T)$ HBM traffic (Chen, 2026a).

Table 10.3 ties MegaKernel planning to TaxBreak counters (Vellaisamy *et al.*, 2026; Chen, 2026a). Compiler **fuse** passes must label preserved boundaries in IR metadata (Chen and YiRage Contributors, 2026; Chen, 2020). Autotuners must

not “win” on FLOPs by breaking Boundary B (Davies *et al.*, 2025; SemiAnalysis, 2024).

Worked contrast. Hopper may keep carriers in registers through Boundary B with sufficient shared memory (SemiAnalysis, 2024; NVIDIA, 2022). XDNA requires static slot assignment for the same lifetime (AMD, 2024a). CPU may split Boundary B if L2 is exceeded—document the split explicitly in fleet configs (Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026).

CUDA Graphs without residency. Memory-Floor shows CUDA Graphs on Qwen-2.5-7B yield $1.259\times$ on H100 by removing host overhead yet leave HBM traffic unchanged (Chen, 2026a; NVIDIA, 2024). Graph-capturing an unfused softmax+ PV pair reduces launches but not Boundary A bytes—the Fregly-style distinction between orchestration relief and mechanical sympathy (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

IR metadata contract. YiRage should record `online_softmax_carriers` lifetime through Boundaries A–B so backends reject schedules that insert `GlobalStore` on P or attn output (Chen and YiRage Contributors, 2026; Chen, 2020). Code review diffs should show eliminated stores, not just fused kernel names (Chen, 2026a; Vellaisamy *et al.*, 2026).

Industry context: DeepSeek MLA and Flash paths. DeepSeek’s Multi-Head Latent Attention compresses KV and reshapes attention math for bandwidth-limited H800 deployments (Liu *et al.*, 2024; Davies *et al.*, 2025). Whether scores pass through a latent projection or full $QK^\top$, the softmax step still demands online carriers when decode avoids materialized P (Dao *et al.*, 2022). Treat MLA as a different QK producer feeding the same state machine in Figure 10.1—not an excuse to skip online updates (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Kernel launch accounting. Separating “stable exp” into its own kernel adds one launch per KV block in the worst case—multiplied by layer count and token steps (Vellaisamy *et al.*, 2026; NVIDIA, 2024). At 847 launches/token baseline for dense Llama-3.2-1B, even +1 launch/layer/block is measurable in p99 (Vellaisamy *et al.*, 2026; Chen, 2026a). Online softmax belongs inside the same captured body as $QK^\top$ and PV (Dao *et al.*, 2023; Chen and YiRage Contributors, 2026).

Compiler lowering sketch. YiRage-style IR represents `OnlineSoftmax` as a region with explicit carrier SSA values live across `For` loops over KV blocks (Chen and YiRage Contributors, 2026; Chen, 2020). GPU lowering maps carriers to registers; XDNA lowering maps them to tile addresses; CPU lowering maps them to vector registers with spill forbidden by legality check (AMD, 2024a; Goodfellow *et al.*, 2016). Any pass that inserts `GlobalStore` on carriers must fail closed (Chen, 2026a).

Warp-level implementation notes. A common GPU pattern assigns one warp group to max-reduce scores, another to scaled `exp`, and a third to accumulate o_t against V fragments (SemiAnalysis, 2024; NVIDIA, 2022). Shared-memory staging holds the current score tile; carriers live in registers owned by the softmax leader lane (Dao *et al.*, 2022). Bank conflicts on score tiles show up as p99 spikes at certain head dims—profile before declaring softmax “done” (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Roofline for softmax epilogue. Even when $QK^\top$ is bandwidth-bound, materializing P adds an extra read/write pass over T elements per head (Chen, 2026a; Goodfellow *et al.*, 2016). Online softmax removes that pass; the remaining work is dominated by PV and KV streaming (Dao *et al.*, 2022, 2023). Optimizing `exp` alone without removing P is the attention analog of CUDA Graphs without residency relief (Chen, 2026a; NVIDIA, 2024).

Long-context regression matrix. Stability tests should include: (i) random Gaussian q, k ; (ii) structured spikes simulating attention sinks; (iii) FP8 with scales (Liu *et al.*, 2024; Davies *et al.*, 2025). Report max absolute error vs FP64 reference at $T = 8192$ and document dtype choices in Table 10.2 (Dao *et al.*, 2023; Chen and YiRage Contributors, 2026). Fleet releases without this matrix risk silent quality drift in fused paths (Chen, 2020).

Disaggregated prefill/decode. When decode nodes run steady batch-1 steps, online softmax runs every token against a growing KV cache (Davies *et al.*, 2025; Kwon *et al.*, 2023). Prefill nodes may use different tile shapes; do not reuse prefill-only softmax kernels on decode pools without carrier residency review (Dao *et al.*, 2023; Vellaisamy *et al.*, 2026). Disaggregation shifts optimization target toward per-step latency and in-kernel persistence (Chen, 2026a).

Expert/MoE caveat. MoE decode adds routing launches unrelated to softmax—TaxBreak shows 9,305 launches/token for OLMoE-class models (Vellaisamy *et al.*, 2026). Online softmax still matters inside attention, but fleet diagnostics must separate routing tax from attention residency tax (Davies *et al.*, 2025; Liu *et al.*, 2024). Do not attribute MoE launch storms to softmax materialization without profiling (Vellaisamy *et al.*, 2026).

Tooling: what to profile. Use Nsight Compute warp stall reasons on softmax regions; PyTorch profiler op boundaries to count separate `softmax` ops; IR diff to verify carrier lifetimes (Davies *et al.*, 2025; Chen, 2020). Goodput improvement shows up as reduced DRAM bytes and fewer `softmax/exp` kernels—not higher tensor-core utilization (Chen, 2026a; Vellaisamy *et al.*, 2026).

Anti-pattern: “FlashAttention enabled” checkbox. Enabling FlashAttention in a framework server removes many prefill pitfalls but may leave decode MLP/norm boundaries unfused (Dao *et al.*, 2022, 2023). Online softmax inside flash kernels is necessary, not sufficient, for Chapter 11 MegaKernels (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Benchmark bytes/token across the full decoder layer, not attention isolation (Chen, 2026a; Davies *et al.*, 2025).

Handoff checklist to Chapter 11. Before merging MegaKernel work, verify: (1) carriers never stored globally between KV blocks; (2) Boundary B preserved in IR; (3) split-KV merge in same launch; (4) dtype/stability tests pass at max context bucket (Chen and YiRage Contributors, 2026; AMD, 2024a; SemiAnalysis, 2024). Failure at any item means Chapter 11 fusion is cosmetic graph capture (Chen, 2026a; NVIDIA, 2024).

Quantitative anchor (TaxBreak). Dense Llama-3.2-1B at BS=4/SL=2048 averages 847.5 GPU kernel launches per *output* token on H100 (Vellaisamy *et al.*, 2026). Attention softmax epilogues are a fraction of those launches, but materializing P adds sync points that block fusion with PV and MLP (Chen, 2026a; Dao *et al.*, 2022). Removing P is prerequisite to collapsing launches toward $O(1)$ per layer—the Chapter 11 target (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; NVIDIA, 2024).

Quantitative anchor (Memory-Floor). CUDA Graphs on Qwen-2.5-7B (BS=1, ctx=2048) yields $1.259\times$ on H100 by removing ≈ 3.05 ms host overhead yet

leaves device bytes unchanged (Chen, 2026a; NVIDIA, 2024). Online softmax addresses device bytes; graphs address host launches—compose both, do not confuse either with MegaKernel residency (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Software stack placement. Online softmax sits below framework servers and above MMA/TMA microkernels (Kwon *et al.*, 2023; SemiAnalysis, 2024). Compilers own carrier lifetimes; hand-written CUDA owns tile sizes (Chen, 2020; Chen and YiRage Contributors, 2026). Performance engineers own regression matrices tying both to ms/token and goodput (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Reader exercise (mental model). For $\text{ctx}=4096$, $\text{block}=128$, estimate KV blocks per head, carrier count per warp group, and whether materialized P adds one or two full DRAM passes (Dao *et al.*, 2022; Chen, 2026a). If you cannot estimate bytes/token on a whiteboard, profiler tuning will misfire (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Safety and monitoring. Production fleets should log fused-path flags and context buckets per request (Kwon *et al.*, 2023; Davies *et al.*, 2025). Silent fallback to eager softmax when fusion legality fails must alarm—otherwise SLO regressions hide behind rare code paths (Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026).

Cross-chapter dependencies. Chapter 8 TMA double-buffering feeds KV tiles into this state machine (SemiAnalysis, 2024). Chapter 9 sync hierarchy ensures carrier merges see consistent V fragments (AMD, 2024a; Dao *et al.*, 2023). Chapter 11 assumes this chapter’s invariants when fusing MLP (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Skipping any prerequisite yields “MegaKernels” that are launch wrappers (Chen, 2026a; NVIDIA, 2024).

Glossary alignment. **Online softmax** = streaming softmax with (m, ℓ, o) carriers; **materialized softmax** = writing P to memory; **Boundary A/B** = fusion seams defined in Section 10.4 (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Use these terms consistently in design docs and IR comments (Chen, 2020; Chen and YiRage Contributors, 2026).

Cost framing. At cloud scale, wasted HBM bytes translate directly to rented GPU seconds (Davies *et al.*, 2025; Liu *et al.*, 2024). Online softmax is a cost

optimization—similar to Fregly’s goodput ROI argument for performance engineers (Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Small per-token byte savings compound across millions of tokens and hundreds of layers (Chen, 2026a; Kwon *et al.*, 2023).

When online softmax is insufficient. If MLP and norm boundaries remain unfused, fixing softmax alone cannot hit aggressive ms/token targets (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Treat this chapter as necessary infrastructure, not the final decode optimization (Chen and YiRage Contributors, 2026; Dao *et al.*, 2023). The profile-driven engineer sequences work: residency first, launches second, FLOPs last (Chen, 2026a; SemiAnalysis, 2024).

Documentation deliverables. Ship with each fused kernel: pseudocode of merge equations, dtype table, max context bucket, split-KV merge diagram, and triplet bytes/token table (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b). This is the O’Reilly-style “tooling + methodology” pairing applied to compiler outputs (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

10.5 Profiling Online Softmax in Production Stacks

Latency composition reminder. Per-token decode latency aggregates weight read, KV read, attention math, MLP, sampling, and host orchestration (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023). Online softmax attacks the attention math *memory* term and enables fusion that reduces orchestration (Dao *et al.*, 2022; Chen, 2026a; Chen and YiRage Contributors, 2026). Expect partial ms/token impact if MLP boundaries remain unfused—consistent with Memory-Floor graph results (Chen, 2026a; NVIDIA, 2024).

Book roadmap hook. Part V manual MegaKernels assume you can hold (m, ℓ, o) through Boundaries A–B (Vellaisamy *et al.*, 2026; Chen, 2020). Part VI YiRage passes automate carrier lifetime analysis (Chen and YiRage Contributors, 2026; AMD, 2024a). This chapter is the semantic foundation both parts reference (SemiAnalysis, 2024; Dao *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; Chen, 2026a; Kwon *et al.*, 2023; NVIDIA, 2024).

Profile-first workflow (Fregly-aligned). Start from user-visible ms/token at batch-1, then decompose with TaxBreak-style launch accounting and Memory-

Floor-style bandwidth floors (Vellaisamy *et al.*, 2026; Chen, 2026a). If R_{floor} is low on H100, prioritize eliminating P materialization and Boundary B breaks before tuning **exp** microkernels (Chen, 2026a; Davies *et al.*, 2025). If R_{floor} is high on L4, bytes/token from materialized softmax may still dominate—online carriers remain mandatory (Chen, 2026a; Dao *et al.*, 2022).

Nsight Compute signals. High `memory_throughput` with low `sm__throughput` during softmax regions suggests P traffic or spill (Davies *et al.*, 2025; SemiAnalysis, 2024). High `warp_stall_math` on **exp** may justify SFU tuning *after* residency is fixed (NVIDIA, 2022). Treat profiler screenshots as release artifacts tied to IR hashes (Chen and YiRage Contributors, 2026; Chen, 2020).

10.5.1 Micro-reproduction harness

Production regressions arrive with many variables entangled, so the first step of any softmax investigation is to build a repro that isolates attention. The harness should pin the model and head layout, the context bucket, the page-table shape, the KV block size and split count, the carrier dtypes, and the driver and compiler IR hash (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Changing any one of those pins invalidates comparisons, which is why the config belongs beside the kernel rather than in a private experiment folder (Chen and YiRage Contributors, 2026; Chen, 2026a).

Run three variants in the same graph body: eager SDPA, online softmax inside the KV loop without Boundary B fusion, and online softmax with the attention output carried through to the MLP seam (Dao *et al.*, 2022; Chen, 2020). Each variant must emit ms/token, kernels/token, and bytes/token under identical warm-up and context conditions so the three levers—latency, launches, and memory traffic—can be attributed independently (Chen, 2026a; Vellaisamy *et al.*, 2026). When only the third variant moves all three counters, the fix is a residency fix; when only launches move, the fix is orchestration; when only latency moves without bytes, re-check the profiler configuration before celebrating (Davies *et al.*, 2025; Kwon *et al.*, 2023).

Keep the harness small enough to run in a CI job on every candidate kernel, and archive its config hash with the result table. A regression that takes an engineer a week to reproduce is a process failure; the same regression caught by a one-layer harness is a one-line revert (Chen and YiRage Contributors, 2026; Dao *et al.*, 2023).

Table 10.4: Diagnostic signals for online softmax regressions.

Signal		Likely cause	Next action
+1	kernel/layer/token	Split softmax epilogue	Fuse into KV loop
High DRAM R/W vs baseline		Materialized P or spill	IR diff carriers
Long-context only	drift	FP16 ℓ underflow	Promote accum dtypes
GPU ok / XDNA fail compile		Carrier SRAM overflow	Reschedule tiles
Graph speedup without bytes win		Orchestration only	Pursue Boundary B

Framework-level visibility. PyTorch profiler timelines that show distinct softmax ops per layer per token indicate Boundary A breaks in eager stacks (Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026). torch.compile may fuse some epilogues but cannot invent on-chip carriers if the graph exports P (Chen, 2020; Davies *et al.*, 2025). Compare against a hand-written online reference kernel on identical shapes before trusting compiler fusion (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026).

Regression gates for YiRage exports. Every backend export should attach: (i) bytes/token delta vs eager; (ii) kernels/token delta; (iii) max abs error vs FP64 at $T = 8192$; (iv) legality pass/fail on XDNA (Chen and YiRage Contributors, 2026; AMD, 2024a; Chen, 2026a). This mirrors MLPerf-style reproducibility discipline adapted to compiler outputs (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Teaching vs shipping paths. Educational notebooks often materialize P for clarity (Goodfellow *et al.*, 2016). Production IR must strip those tensors—accidental debug hooks reintroduce HBM traffic at scale (Chen, 2026a; NVIDIA, 2024). Separate “debug softmax stats” behind compile flags, default off (Chen, 2020; Chen and YiRage Contributors, 2026).

Historical note. Online softmax predates LLM serving—streaming softmax appears in numerical libraries and early attention approximations (Goodfellow *et al.*, 2016; Dao *et al.*, 2022). FlashAttention’s contribution is pairing online

updates with IO-aware tiling on GPUs (Dao *et al.*, 2022, 2023). This chapter’s decode focus follows industrial shift toward batch-1 latency and MegaKernel fusion (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025).

Open problems. Extremely long contexts ($T > 128\text{K}$) stress ℓ dynamic range even in FP32 (Davies *et al.*, 2025; Liu *et al.*, 2024). Block-wise normalization tricks and periodic rescaling may enter carrier state—document any non-standard invariant before Chapter 11 assumes three scalars suffice (Chen and YiRage Contributors, 2026; Dao *et al.*, 2023). Research prototypes belong behind explicit IR ops, not silent kernel hacks (Chen, 2020).

Summary before takeaways. Sections 10.1–10.4 form a single story: streaming softmax math, stable numerics, target-specific residency, and fusion seams measured by TaxBreak and Memory-Floor counters (Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022; AMD, 2024b; Chen and YiRage Contributors, 2026). Section 10.5 ties those threads to profiler workflows familiar from O’Reilly-style systems books—start from user latency, decompose bottlenecks, verify bytes and launches, then tune kernels (Davies *et al.*, 2025; SemiAnalysis, 2024; Chen, 2020; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024).

10.6 Key Takeaways

Appendix cross-reference. Verified launch and bandwidth numbers for this chapter appear in Chapter 1 Table 1.6 and Appendix regression matrices (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). When citing ms/token improvements here, always pair with kernels/token and bytes/token from the same run configuration (Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016). Mixed sources invalidate cross-chapter comparisons (Chen, 2020).

Speculative decoding interaction. Speculative paths run draft models with additional forward passes (Davies *et al.*, 2025; Liu *et al.*, 2024). Online softmax in the target model remains unchanged in math; draft acceptance does not remove KV loop residency requirements (Dao *et al.*, 2022; Kwon *et al.*, 2023). Profile speculative stacks separately—draft overhead dominates some deployments before attention bytes do (Vellaisamy *et al.*, 2026; Chen, 2026a).

Energy angle. HBM reads dominate decode energy (Chen, 2026a; Davies *et al.*, 2025). Online softmax reduces unnecessary DRAM toggling—relevant for edge Ryzen AI and cloud TCO even when ms/token gains look modest (AMD, 2024b,a; Goodfellow *et al.*, 2016). Energy-aware fleets should log bytes/token alongside power counters (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

- **Stream, do not materialize.** Maintain the running statistics (m, ℓ, o) across KV blocks so the softmax probability matrix P is never written to HBM during decode (Dao *et al.*, 2022; Chen, 2026a). Materializing P even as a thin $1 \times T$ vector per head per token is catastrophic once block boundaries export it, which is exactly the traffic online softmax removes (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).
- **Max before exp.** Stability is part of performance—unstable online paths force FP32 fallbacks that break fusion (Dao *et al.*, 2022; Davies *et al.*, 2025).
- **Measure attention goodput.** Track bytes/token and launches/token alongside ms/token; FLOPs mis-rank bottlenecks at batch-1 (Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).
- **Mechanical sympathy across targets.** GPU registers, XDNA SRAM slots, and CPU SIMD accumulators implement the same invariants with different legality rules (AMD, 2024b; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).
- **Respect fusion boundaries.** Boundaries A–B determine MegaKernel legality; graphs alone do not fix Boundary breaks (Chen, 2026a; NVIDIA, 2024; Chen and YiRage Contributors, 2026).
- **Split-KV needs in-kernel merge.** Flash-Decoding wins require second-stage online reduction without extra launches (Dao *et al.*, 2023; Vellaisamy *et al.*, 2026).
- **Profile-driven gates.** Long-context numerical tests and buffer accounting block merges—not microbench GEMM tweaks (Chen and YiRage Contributors, 2026; Chen, 2020).
- **Pair the metrics, as Fregly pairs goodput with utilization.** Always report kernels/token and bytes/token alongside ms/token on the same chart, because either number alone hides the other limiter at batch-1 (Vellaisamy *et al.*, 2026; Chen, 2026a). A softmax change that improves one while quietly regressing the other is invisible unless the metrics are shown together (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

- **Reject graph-only wins.** Orchestration speedups without residency relief are partial fixes—label them explicitly in benchmark tables (NVIDIA, 2024; Chen, 2026a).
- **Document dtype and context buckets.** Fleet configs must pin max context, block size, and carrier dtypes per compiled graph (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; AMD, 2024a).

Closing integration note. Industrial decode optimization is a stack problem: online softmax fixes attention residency, MegaKernels fix layer residency, disaggregation fixes fleet residency (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). None replaces the others (Chen, 2026a; Liu *et al.*, 2024). Performance engineers document which layer of the stack each PR addresses—avoid claiming end-to-end wins from softmax alone (Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

Vendor roadmap awareness. Faster SFUs, TMA, and FP4 tensor cores change *how* softmax is implemented, not *whether* carriers stay on-chip (NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025). Re-tile kernels each silicon generation; invariants in this chapter remain (Dao *et al.*, 2022; Chen, 2020). Mechanical sympathy evolves with hardware but rejects materialized P in decode (Chen, 2026a; Vellaisamy *et al.*, 2026).

Collaboration boundaries. Model teams own architecture; systems teams own serving; compiler teams own IR legality (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Online softmax sits at the intersection—requires joint review when changing head dims, context limits, or dtypes (Liu *et al.*, 2024; Kwon *et al.*, 2023). Fregly’s cross-team transparency principle applies: publish benchmark configs alongside kernel changes (Vellaisamy *et al.*, 2026; Chen, 2026a).

Final sanity check. If your fused attention kernel writes a tensor named `attn_weights` or `probs` to global memory during decode, you are not running online softmax in the production sense (Dao *et al.*, 2022; Chen, 2026a). Rename, refactor, or reject the merge (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026).

Execution checklist. Before closing a softmax fusion PR: confirm IR shows no `GlobalStore` on probabilities; run FP64 reference at $T = 8192$; capture Nsight

DRAM throughput delta; re-run TaxBreak launches/token on BS=1; validate XDNA compile legality or document intentional CPU/GPU-only scope (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Davies *et al.*, 2025; Dao *et al.*, 2022; Chen, 2020; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; NVIDIA, 2024; Liu *et al.*, 2024; Dao *et al.*, 2023).

10.7 Conclusion

Online softmax is the decode attention state machine: streaming KV blocks update (m, ℓ, o) without materializing P (Dao *et al.*, 2022, 2023). Numerical discipline and multi-hardware residency tables determine whether compiler-generated schedules can fuse through Boundaries A and B (AMD, 2024a; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024). This is mechanical sympathy at attention granularity—the same design philosophy Fregly applies at cluster scale, translated into on-chip carrier lifetimes and TaxBreak-visible launch counts (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Liu *et al.*, 2024).

Chapter 11 consumes this chapter wholesale: full-layer MegaKernels assume carriers and partial PV survive into MLP fusion (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). If softmax spills here, no downstream graph capture recovers the bytes/token budget—the same way cluster-scale goodput cannot recover from idle GPUs masked by headline utilization (Chen, 2026a; NVIDIA, 2024). Teams that skip online carriers often chase speculative decoding or wider GPUs first—reasonable for product roadmaps, but ineffective when bytes/token and launches/token remain bound by materialized softmax and Boundary B breaks (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a). The disciplined sequence matches this book and Fregly’s profile-first ethos: measure useful work, fix residency, then amortize launches, then tune FLOPs (Dao *et al.*, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Carry forward explicit IR boundary metadata, profiler screenshots, and triplet bytes/token tables in every release notes bundle, triplet regression across GPU/CPU/NPU, and long-context stability tests as non-negotiable release gates for any attention fusion touching decode paths (Chen, 2020; Kwon *et al.*, 2023).

Handoff. Online softmax carriers are the Part IV residency prerequisite for full-layer MegaKernel integration in Chapter 11 on GPU, CPU, and XDNA fleet targets alike across the book. (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Chen, 2026a; Davies *et al.*, 2025; Chen, 2020; SemiAnalysis, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; NVIDIA, 2024; Liu

et al., 2024; Dao *et al.*, 2023).

Metric reminder. Useful decode throughput excludes bytes spent materializing P , idle launch gaps, and debug tensors (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; SemiAnalysis, 2024; AMD, 2024a; NVIDIA, 2024; Liu *et al.*, 2024; Dao *et al.*, 2023).

Chapter 11

Full Decoder MegaKernel Implementation

FlashAttention removed the attention $\mathcal{O}(N^2)$ memory cliff; CUDA Graphs removed a slice of host overhead—yet batch-1 decode on Llama-class models still issues hundreds of discrete GPU kernels *per output token* (Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022). The remaining pain is not “one slow matmul” but *layer fragmentation*: QKV projection, online softmax attention, RMS norm, and MLP each export full-width activations to HBM between launches (NVIDIA, 2024; Kwon *et al.*, 2023). A **decoder MegaKernel** fuses those stages into one residency-aware schedule—one graph capture on GPU, one static dataflow on XDNA—so bytes/token and launches/token fall together (Chen, 2020; Chen and YiRage Contributors, 2026).

Chapters 4–10 built the ingredients: TMA double-buffering, online softmax carriers, sync hierarchy, execution-mode trade-offs. This chapter assembles them into a *full-layer* implementation narrative: structure, fused QKV–attention–MLP boundaries, a source-level control-flow walkthrough, and benchmark methodology that separates real fusion wins from graph-capture illusions (SemiAnalysis, 2024; AMD, 2024b; Davies *et al.*, 2025). The mistake to avoid is declaring victory when CUDA Graphs replay the same HBM traffic with fewer host calls—Memory-Floor’s $1.259\times$ on Qwen-2.5-7B (BS=1, ctx=2048) proves orchestration relief without residency relief is a partial fix (Chen, 2026a).

TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). A MegaKernel target is not zero launches—it is $\mathcal{O}(1)$ *launches per decoder layer per token* (or

per captured graph step) with intermediates resident in shared memory, registers, or tile SRAM (NVIDIA, 2022; AMD, 2024a). YiRage lowers the same decoder IR to CUDA MegaKernels, CPU blocked schedules, and XDNA static graphs; hand-written stacks should treat unfused baselines as regression tests, not shipping defaults (Chen and YiRage Contributors, 2026; Liu *et al.*, 2024).

Scope of “full decoder.” This chapter focuses on *one transformer decoder layer* fused end-to-end—the atomic unit repeated L times in depth. Stacking layers inside a single MegaKernel is a further optimization (amortizing launch and pipeline fill) but explodes register and shared-memory pressure; production systems often MegaKernel per layer and capture an L -step graph (NVIDIA, 2024; Vellaisamy *et al.*, 2026). The engineering question is identical: where do activations touch HBM between sub-stages (Chen, 2026a; Davies *et al.*, 2025)? DeepSeek-V3-style wide MoE decoders may require *expert-local* MegaKernels rather than one monolith—routing and all-to-all remain outside the fused region (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026).

Relationship to prior chapters. Chapter 4 supplies TMA and shared-memory ceilings; Chapter 5 translates residency to XDNA tile rules; Chapters 8–9 supply double-buffer and sync patterns; Chapter 10 supplies online softmax carriers that must survive into MLP in this chapter (SemiAnalysis, 2024; AMD, 2024a; Dao *et al.*, 2022). If any ingredient is missing, teams build “MegaKernels” that are graph-capture wrappers around unchanged HBM traffic (Chen, 2026a).

Industrial motivation. Serving stacks that ship 847+ launches/token pay a fixed tax on every interactive token (Vellaisamy *et al.*, 2026). At $4.7\,\mu\text{s}$ null-launch floor on H100, launch path alone approaches tens of milliseconds per token for MoE models before any math (Vellaisamy *et al.*, 2026; Chen, 2026a). MegaKernels attack both launch count *and* the bytes moved per launch—the only durable fix when arithmetic intensity is already on the memory roofline (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). Cloud fleets also pay PCIe and CPU orchestration energy; edge Ryzen AI fleets pay DDR contention with CPU graphics (AMD, 2024b,a). The same fused schedule story applies—only residency tables change (Chen and YiRage Contributors, 2026; Chen, 2020).

What this chapter is not. Not a PyTorch tutorial (Goodfellow *et al.*, 2016). Not a FlashAttention paper recap—Chapter 10 covered carriers (Dao *et al.*, 2022). Not a CUDA Graphs manual—orchestration-only wins are labeled explicitly (Chen,



Figure 11.1: Decoder MegaKernel pipeline: one residency envelope from hidden state through fused compute to KV append.

2026a; NVIDIA, 2024). Not a promise that every model fuses on every SKU—legality failures are expected outputs (AMD, 2024a; Chen and YiRage Contributors, 2026). The deliverable is an implementable fused-layer contract with measurable decode metrics (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2020).

Reader prerequisites. Assume familiarity with decode vs prefill, paged KV, online softmax, and TMA pipelining from Chapters 1–10 (Kwon *et al.*, 2023; Dao *et al.*, 2022; SemiAnalysis, 2024; Chen, 2026a). Jumping here from a generic CUDA course produces “pretty kernels” that still export activations to HBM (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Notation. d : hidden size; d_{ff} : MLP expansion; BS: batch size; ctx: context length; L : layer count (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). “Layer” means one transformer decoder block unless stated (Liu *et al.*, 2024; Kwon *et al.*, 2023). “Token” means one autoregressive decode step (Dao *et al.*, 2023; Vellaisamy *et al.*, 2026). “Launch” means one host-visible kernel or graph replay step (NVIDIA, 2024; Chen, 2026a). Align vocabulary in cross-team docs to avoid debating past each other (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a,b; SemiAnalysis, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024).

11.1 MegaKernel Structure

Why one kernel still matters after FlashAttention. Teams assume FlashAttention “fixed decode.” It fixed attention’s quadratic scratch memory—not the surrounding launch storm. Unfused decoder stacks still separate layer norm, QKV linear, attention, projection, second norm, and three MLP matrices into distinct kernels (Dao *et al.*, 2022, 2023). Each boundary writes a d -model vector to HBM and reads it back—at batch-1 that is pure bandwidth tax on a thin arithmetic intensity slope (Goodfellow *et al.*, 2016; Chen, 2026a). MegaKernel structure means redesigning the *control flow* so those stages share one on-chip residency plan for the duration of a decoder step (Chen, 2020; Vellaisamy *et al.*, 2026).

Figure 11.1 is the chapter spine. Unlike a prefill megamatmul, decode MegaKernels must interleave **KV cache read+append** with fused math every token (Kwon *et al.*, 2023; Davies *et al.*, 2025). The host submits one graph (GPU) or one static runner invocation (XDNA); the device loop advances sequence position, bumps KV pointers, and keeps softmax statistics in carrier registers described in Chapter 10 (NVIDIA, 2024; AMD, 2024b).

Pipeline stage notes (Figure 11.1). **Input hidden state** arrives as a single token vector (batch-1) or narrow batch slice (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). It should enter shared memory once—not remain in HBM while QKV loads weights (SemiAnalysis, 2024; Chen, 2026a). **On-chip QKV + carriers** holds projected Q/K/V fragments plus online softmax state (Dao *et al.*, 2022). Peak pressure occurs here when d is large or head count is high (Liu *et al.*, 2024; NVIDIA, 2022). **Fused attn + MLP tile** is the longest compute phase: KV blocks stream from HBM while MMA pipelines gate/up/down (Dao *et al.*, 2023; Vellaisamy *et al.*, 2026). TMA overlap from Chapter 8 is mandatory—else MegaKernel devolves to sequential micro-kernels inside one launch (SemiAnalysis, 2024; Chen, 2020). **Output + KV append** writes the next hidden state and appends K/V without host callbacks (Kwon *et al.*, 2023; NVIDIA, 2024). On XDNA, append is a DMA store into DDR-resident paged pools (AMD, 2024b,a). Breaking the pipeline at any boundary exports a full activation vector—defeating the MegaKernel purpose (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Host responsibilities per token. Ideal host work: increment sequence counters, update sampling state, enqueue next graph replay (NVIDIA, 2024; Kwon *et al.*, 2023). Anything heavier (allocation, dtype conversion, tensor contiguity fixes) belongs in session setup—not the hot loop (Chen, 2026a; Vellaisamy *et al.*, 2026). Python GIL interactions still matter if the host thread prepares each step synchronously (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Multi-layer scheduling. Repeating Figure 11.1 L times per token can mean L graph replays or one persistent kernel traversing layers (Liu *et al.*, 2024; NVIDIA, 2024). The per-layer MegaKernel body is identical modulo weight pointers (Chen and YiRage Contributors, 2026; Chen, 2020). Benchmarks must state which scheduling variant they use— L replays add host overhead even when each layer is fused (Vellaisamy *et al.*, 2026; Chen, 2026a).

Table 11.1 explains why MegaKernel engineering is a *systems* task: the limiting resource is simultaneous buffer residency, not FLOPs (SemiAnalysis, 2024;

Table 11.1: Fragmented decode stack vs. full-layer MegaKernel (qualitative, batch-1).

Aspect	Unfused stack	Decoder MegaKernel
Launches/token (layer)	10–20+	1 (or 1 graph step)
HBM round trips (activations)	Every op boundary	Fused tile only
Shared / tile SRAM plan	Per-kernel peaks	Joint peak across QKV–MLP
Host role per token	Dispatch loop	Pointer/metadata update
XDNA legality	Often fails partial fusion	Requires full static plan

NVIDIA, 2022). Hopper may legalize the plan with TMA-backed double buffers and warp-specialized stages (SemiAnalysis, 2024). XDNA requires every temporary—including online softmax carriers—to fit tile SRAM assignments before compile (AMD, 2024a; Chen and YiRage Contributors, 2026). CPU decode uses the same logical structure with L2-blocked megafunctions; launches/token is less visible but bytes/token still punishes unfused DRAM traffic (Goodfellow *et al.*, 2016; Davies *et al.*, 2025).

Persistent vs. captured graphs. Two GPU patterns dominate production:

1. **CUDA Graphs capture** of an unfused or partially fused op sequence—reduces ΔFT and ΔKT from Chapter 1 but leaves HBM traffic unchanged (Chen, 2026a; NVIDIA, 2024).
2. **True MegaKernel fusion**—single kernel body or minimal graph nodes with shared-memory staging across QKV, attention, and MLP (Chen, 2020; Vellaisamy *et al.*, 2026).

Reviewers must distinguish them in benchmark tables; mixing the two inflates perceived “fusion speedups” (Davies *et al.*, 2025; Chen, 2026a).

Example 11.1.1. Llama-3.2-1B decode layer (BS=1, ctx=2048): unfused stack materializes six d -model vectors to HBM per layer per token; a MegaKernel keeps QKV fragments, softmax carriers, and MLP gate/up tiles in shared memory until the residual add completes—bytes/token drops with launches/token even when MMA tile shapes are unchanged (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022).

Multi-hardware delta. GPU MegaKernels exploit shared memory + TMA; XDNA maps the same DAG to spatial tiles with DMA edges; CPU maps to fused BLAS-like blocks with explicit cache blocking (AMD, 2024b; Goodfellow *et al.*,

2016; Chen and YiRage Contributors, 2026). Identical math, different legality predicates—never ship a GPU-only residency table to Ryzen AI without recompile (AMD, 2024a; Liu *et al.*, 2024).

Review gate. Merge requires a buffer accounting diagram: peak simultaneous tensors during one decode step must fit target SRAM budgets (Chen and YiRage Contributors, 2026; Chen, 2026a). Failed legality is a release blocker, not a “fallback to eager” ticket (Chen, 2020).

Interaction with disaggregated serving. Prefill/decode disaggregation moves pools independently (Davies *et al.*, 2025; Liu *et al.*, 2024). Decode pools benefit most from MegaKernels—prefill pools may keep separate wide-GEMM schedules (Dao *et al.*, 2023; Kwon *et al.*, 2023). Do not force one fusion policy across disaggregated roles (Vellaisamy *et al.*, 2026; Chen, 2026a). Benchmark decode pools with batch-1/interactive distributions, not prefill throughput alone (Davies *et al.*, 2025; Dao *et al.*, 2022).

Warp specialization and pipeline stages. Hopper-class MegaKernels often assign warps to roles—TMA load, MMA compute, epilogue—mirroring CUTLASS-style persistent kernels (SemiAnalysis, 2024; NVIDIA, 2022). Decode differs from GEMM persistence: the KV loop length grows each token, so the inner loop trip count changes while the kernel body stays captured (Kwon *et al.*, 2023; Dao *et al.*, 2023). Implementations pad to max context bucket or specialize kernels per bucket; mixing buckets without re-capture violates graph legality (NVIDIA, 2024; Chen and YiRage Contributors, 2026).

Register and shared-memory budgeting. A back-of-envelope for Llama-3.2-1B ($d=2048$, $h=32$): QKV fragments, softmax carriers, and SwiGLU intermediates can exceed 200 KB shared memory if naively allocated—above many SM limits when combined with MMA tiles (NVIDIA, 2022). MegaKernel structure therefore *sequences* sub-stages with reuse: gate/up may share buffers with Q staging after QKV completes (Chen, 2020; Vellaisamy *et al.*, 2026). Compiler passes must solve this allocation problem; hand tuning without a lifetime graph does not scale across models (Chen and YiRage Contributors, 2026; Liu *et al.*, 2024).

CPU and NPU structural analogues. CPU MegaKernels appear as fused oneDNN/BLIS-style micro-kernels with K -dimension blocking; “launch” becomes

function call (Goodfellow *et al.*, 2016). XDNA expresses the same DAG as AIE tiles with fixed DMA descriptors—there is no `__syncthreads`, only token completion on edges (AMD, 2024a,b). Fleet parity requires documenting which sub-stages remain unfused on each target and why (Davies *et al.*, 2025; Chen, 2020).

Paged KV and MegaKernel entry. vLLM-style paging changes *where* K/V bytes live, not whether they must be read (Kwon *et al.*, 2023). MegaKernel structure adds a block-table walk inside the attention loop—pointer-chasing in device code replaces scatter-gather launches (Davies *et al.*, 2025; Chen, 2026a). Poor block sizing interacts with TMA tile shapes: 16-token blocks may underutilize memory bandwidth while 256-token blocks blow shared-memory staging (SemiAnalysis, 2024; Dao *et al.*, 2023). Treat block size as a compile-time knob tied to context buckets (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023).

Quantization and MegaKernels. INT8/FP8 weights shrink bytes/token but fuse epilogues with dequant scales (NVIDIA, 2022; Davies *et al.*, 2025). MegaKernels must carry scale vectors in registers or constant cache—another residency consumer (SemiAnalysis, 2024; Chen, 2020). Mixed-precision decode that quantizes activations per token often breaks fusion unless scales are online in the same kernel (Liu *et al.*, 2024; Chen, 2026a).

Failure modes checklist. Production MegaKernels fail in predictable ways: (1) shared-memory overflow at max context; (2) graph capture invalidation after driver upgrade; (3) silent fallback to eager on one shape bucket; (4) numerics drift vs reference beyond tolerance; (5) p99 regression from bank conflicts (NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026). Runbooks should map each symptom to residency vs orchestration root cause using bytes/token and launches/token splits (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

Compiler ownership. Hand-written MegaKernels do not scale across model zoo churn (Chen, 2020; Chen and YiRage Contributors, 2026). The durable path is declarative decoder IR with target-specific legality checks—this chapter’s walkthrough is the acceptance spec those compilers must emit (Liu *et al.*, 2024; Kwon *et al.*, 2023). Kernel engineers retain tile tuning; compiler engineers own fusion boundaries and buffer lifetimes (Davies *et al.*, 2025; SemiAnalysis, 2024).

Layer replication and weight streaming. A 32-layer model repeats the same MegaKernel body with different weight bases (Davies *et al.*, 2025; Liu *et al.*, 2024).



Figure 11.2: QKV-attention-MLP fusion: one residency envelope; KV reads enter from HBM, carriers stay on-chip.

Weight streaming bandwidth scales with layer count—fusion reduces activation traffic but not parameter reads (Goodfellow *et al.*, 2016; Chen, 2026a). Some stacks capture one graph per layer type (attention vs MoE) rather than per layer index (Vellaisamy *et al.*, 2026; NVIDIA, 2024). The MegaKernel *structure* is identical; pointer offsets change (Chen and YiRage Contributors, 2026; Chen, 2020).

Safety and determinism. Fused kernels complicate bitwise determinism for compliance workloads (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Document reduction order and FP associativity assumptions (Dao *et al.*, 2022). Deterministic mode may disable some fusion paths—trade bytes/token for reproducibility explicitly in **benchmark** tables (Chen, 2026a; Vellaisamy *et al.*, 2026).

11.2 QKV, Attention, and MLP Fusion

The fusion boundary mistake. Engineers fuse QKV with attention but leave RMS norm and MLP outside the MegaKernel—recovering $\approx 30\%$ of launches while preserving the expensive mid-layer HBM writes (Vellaisamy *et al.*, 2026; Dao *et al.*, 2023). Full-layer fusion treats **QKV**, **attention** (with online softmax), and **MLP** as one producer-consumer chain with a single shared-memory tile budget (Dao *et al.*, 2022; SemiAnalysis, 2024).

QKV projection produces query, key, and value fragments for a single new token while reusing the hidden state tile already in shared memory (NVIDIA, 2022). Fused kernels interleave K/V writes into the paged KV layout (vLLM-style) without a full-width staging buffer in HBM (Kwon *et al.*, 2023). On XDNA, QKV maps to a fixed tile column; K/V DMA into DDR-resident cache lines described at compile time (AMD, 2024b; Chen and YiRage Contributors, 2026).

Attention uses online softmax carriers (m , ℓ statistics) from Chapter 10 so partial blocks update without materializing full attention maps (Dao *et al.*, 2022). The MegaKernel constraint: carriers live in registers or shared memory *across* the boundary into MLP—spilling carriers to HBM defeats the fusion purpose (Chen, 2026a). Flash-Decoding-style splitting across KV heads must remain inside the same launch to avoid multiplying graph steps (Dao *et al.*, 2023).

Table 11.2: Fusion depth vs. decode metrics (typical Llama-layer goals).

Fusion level	Launches/layer/token	Activation HBM trips	SRAM
None (eager)	12–18	6–8	Low per kern
QKV + attn only	4–6	3–4	M
Full QKV–attn–MLP	1	0–1	High (o

MLP (SwiGLU in Llama-class models) is three matmuls with activations—historically three kernels. MegaKernel schedules gate/up in shared memory while down projection consumes the SiLU product without an HBM round trip (Davies *et al.*, 2025; Liu *et al.*, 2024). MoE variants add expert routing; partial MegaKernels that fuse dense MLP but launch per expert reintroduce launch storms—TaxBreak’s OLMoE figures (9,305 launches/token) are the warning label (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024).

Table 11.2 shows the trade-off surface: deeper fusion reduces bytes/token but raises peak shared-memory/tile SRAM usage—the reason MegaKernels fail compile on smaller SKUs while microbenchmarking “works” on H100 with larger shared mem configs (NVIDIA, 2022; AMD, 2024a). Compiler passes must expose this Pareto frontier, not hide failures behind runtime fallbacks (Chen and YiRage Contributors, 2026; Chen, 2020).

Worked contrast. Hopper: TMA loads Q weights while previous tile computes attention on shared-staged V (SemiAnalysis, 2024). XDNA: identical dependency expressed as DMA–MAC chains with compile-time buffer overlap checks (AMD, 2024a). CPU: QKV+attn blocked into L2-sized tiles; MLP may split if L2 exceeded—still fewer DRAM round trips than eager (Goodfellow *et al.*, 2016).

Deployment pitfall. A/B tests that fuse prefill-shaped tiles but deploy batch-1 decode with growing KV see silent perf cliffs at max context—carrier and MLP buffers sized for short KV overflow shared memory (Dao *et al.*, 2023; Chen, 2026a). Bucket context lengths in compile artifacts (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

RMS norm and residual fusion. Layer norm/RMS norm is easy to dismiss as “cheap,” yet as a separate kernel it forces a full d -vector HBM write/read between attention and MLP (Chen, 2026a; Vellaisamy *et al.*, 2026). MegaKernels fold RMS scale into the MLP epilogue or fuse with residual add in registers (Chen, 2020;

Chen and YiRage Contributors, 2026). The numerics must match eager within tolerance—reviewers should demand bit-exact or bounded-error tests, not just latency (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

GQA/MQA and head tiling. Grouped-query attention reduces K/V footprint but complicates tiling: Q heads outnumber KV heads (Dao *et al.*, 2022; Liu *et al.*, 2024). MegaKernel schedules must broadcast KV tiles across Q head groups inside the same launch; splitting heads into separate kernels restores launch storms (Vellaisamy *et al.*, 2026; Dao *et al.*, 2023). On XDNA, head groups map to tile replication patterns fixed at compile time (AMD, 2024a; Chen and YiRage Contributors, 2026).

RoPE and position encodings. Rotary embeddings applied to Q/K are elementwise-heavy but memory-light—still a launch if isolated (Davies *et al.*, 2025). Fused implementations apply RoPE inside the QKV tile before the KV loop; unfused stacks add two global passes per layer per token (Vellaisamy *et al.*, 2026; Chen, 2026a). Position-id updates belong in the device loop metadata, not host Python (Kwon *et al.*, 2023; NVIDIA, 2024).

Attention output projection. The O projection after $\text{softmax} \times V$ is a thin GEMM at batch-1—historically its own kernel (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Fusing O into the MLP input tile eliminates one full d -vector HBM trip (Chen, 2026a, 2020). The MMA tile for O must align with SwiGLU gate tile to avoid bank conflicts (SemiAnalysis, 2024; NVIDIA, 2022).

SwiGLU-specific scheduling. Gate and up projections can share weight streaming if W_{gate} and W_{up} are interleaved in memory (Davies *et al.*, 2025; Liu *et al.*, 2024). MegaKernels double-buffer gate/up fragments while SiLU runs on the previous fragment—classic software pipelining from Chapter 8 (SemiAnalysis, 2024; AMD, 2024a). Down projection consumes the gated product directly from shared memory (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

When full fusion is impossible. Models with very wide MLP ($d_{\text{ff}} \gg d$) may exceed shared-memory caps even with reuse (Liu *et al.*, 2024; NVIDIA, 2022). Accept a *split MegaKernel*: attention+ O fused; MLP as second launch—still beats twelve-launch eager if boundaries are chosen at HBM minima (Vellaisamy *et al.*, 2026; Chen, 2026a). Document the split in compiler metadata so autotuners do not “re-fuse” illegally (Chen and YiRage Contributors, 2026; Chen, 2020).

Speculative decode interaction. Speculative sampling runs draft models ahead of the main decoder—MegaKernels must expose hooks to swap weight descriptors without full re-init (Davies *et al.*, 2025; Liu *et al.*, 2024). Draft and target MegaKernels may differ in fusion depth; benchmark each independently (Vellaisamy *et al.*, 2026; Chen, 2026a). Accepting draft tokens still executes full fused layers on the target model (Kwon *et al.*, 2023; Dao *et al.*, 2023).

Long-context stress. At $\text{ctx}=8192+$, KV read bandwidth dominates even with fusion (Dao *et al.*, 2023; Davies *et al.*, 2025). MegaKernels remove *activation* round trips but not weight/KV streaming—benchmarks must separate attention bytes from MLP bytes (Dao *et al.*, 2022; Chen, 2026a). Flash-Decoding splits help inside the same MegaKernel launch (Dao *et al.*, 2023; SemiAnalysis, 2024).

Bias and fusion. Linear bias adds low cost in FLOPs but forces extra loads if not fused into epilogue (NVIDIA, 2022; Chen, 2020). Fold bias into TMA loads or MMA epilogues—otherwise bias becomes its own memory-bound micro-kernel (SemiAnalysis, 2024; Vellaisamy *et al.*, 2026). RMS β and MLP biases share the same story (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

Dropout and training-only ops. Inference MegaKernels omit dropout entirely—ensure export paths strip training nodes before fusion (Goodfellow *et al.*, 2016; Chen, 2020). Accidentally retaining dropout masks adds random global reads incompatible with graph capture (NVIDIA, 2024; Chen, 2026a).

End-to-end fusion narrative (one token, one layer). Hidden vector enters shared memory (SemiAnalysis, 2024). QKV MMA fills Q/K/V tiles; RoPE rotates Q/K in-place (Davies *et al.*, 2025; Dao *et al.*, 2022). KV loop reads paged history; online softmax updates carriers (Kwon *et al.*, 2023; Dao *et al.*, 2023). O projection and RMS norm fold into MLP input tile (Chen, 2026a, 2020). SwiGLU triple matmul runs with pipelined TMA (Liu *et al.*, 2024; SemiAnalysis, 2024). Residual add produces next hidden; K/V append completes without host (Vellaisamy *et al.*, 2026; NVIDIA, 2024). Zero full-width HBM activations if fusion succeeds (Chen, 2026a; Chen and YiRage Contributors, 2026). Any deviation—usually at MLP boundary or post-attention norm—shows up as a DRAM write spike in Nsight (SemiAnalysis, 2024; Vellaisamy *et al.*, 2026).

Expert routing (MoE) boundary. Routed experts execute separate GEMMs per token (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). Full-layer MegaKernel fusion

typically stops at router logits; expert matmuls may remain a second launch (Davies *et al.*, 2025; Kwon *et al.*, 2023). Benchmarks must separate dense-layer MegaKernels from MoE dispatch overhead (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024). Claiming MoE decode fixes without launches/token evidence repeats Chapter 1 mistakes (Chen, 2026a; Dao *et al.*, 2022).

Cache line and alignment details. Paged KV blocks should align to 128-byte boundaries for efficient TMA (SemiAnalysis, 2024; NVIDIA, 2022). Misaligned K/V reads inflate bytes/token even inside fused kernels (Chen, 2026a; Kwon *et al.*, 2023). Weight layouts interleaving gate/up improve TMA burst efficiency for SwiGLU (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). Layout passes belong in the compiler alongside fusion passes (Chen, 2020; Liu *et al.*, 2024).

Power-virus vs latency-virus. MegaKernels can reduce latency while increasing instantaneous power by keeping more SMs busy per token (Davies *et al.*, 2025; AMD, 2024a). Mobile SKUs may prefer slower bytes/token over thermal throttling (AMD, 2024b; Goodfellow *et al.*, 2016). Report watts/token when targeting edge (AMD, 2024a; Chen, 2026a).

11.3 MegaKernel Source Walkthrough

Reading order for implementers. Production MegaKernel **source** is not “one giant main”—it is layered: host setup, device persistent loop, stage functions with explicit **implementation** of residency contracts (Chen, 2020; NVIDIA, 2024). The walkthrough below mirrors YiRage-emitted CUDA and hand-optimized reference kernels: names are illustrative; invariants are what matter for code review (Chen and YiRage Contributors, 2026).

Host-side implementation (once per session).

1. Allocate paged KV pools and weight handles; pin graph-capture buffers (Kwon *et al.*, 2023; NVIDIA, 2024).
2. Select shape bucket (B, S_{class}) ; compile or capture MegaKernel for that bucket (Chen and YiRage Contributors, 2026; AMD, 2024b).
3. Record TMA descriptor templates for QKV and MLP weights; upload once (SemiAnalysis, 2024; NVIDIA, 2022).

Device decode loop (per output token). Pseudocode structure:

```
for step in 0..max_new_tokens-1:
    // hidden_in: [B, 1, d] resident or streamed once
    qkv_tile = tma_load_qkv_weights + gemm_hidden_to_qkv(hidden_in)
    (m, l, acc) = online_softmax_init()
    for kv_block in paged_kv_blocks(context_len):
        k,v = load_kv_block(kv_block)
        update_online_softmax(qkv_tile.Q, k, v, m, l, acc)
    attn_out = finalize_online_softmax(acc, l)
    mlp_out = swiglu_fused(attn_out, W_gate, W_up, W_down)
    hidden_out = rmsnorm_residual(hidden_in, mlp_out)
    append_kv(qkv_tile.K, qkv_tile.V, step)
    hidden_in = hidden_out
```

Every line maps to on-chip residency decisions:

- `qkv_tile` must stay in shared memory through the KV block loop—spilling mid-loop doubles HBM traffic (Dao *et al.*, 2022; Chen, 2026a).
- Online softmax state `(m, l, acc)` stays in registers per warp group—exporting to global memory between blocks breaks fusion (Dao *et al.*, 2022).
- `swiglu_fused` consumes `attn_out` without a full-width write—the defining MegaKernel win (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).
- `append_kv` uses the same paged indices as vLLM; pointer bumps happen in device code to avoid host sync (Kwon *et al.*, 2023; NVIDIA, 2024).
- **MoE caveat.** Expert routing and all-to-all stay outside the fused region; benchmark dense layers first, then expert-local MegaKernels (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; NVIDIA, 2024; AMD, 2024a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016; AMD, 2024b).
- **Quantization carries scales in-register.** FP8/INT8 decode must fuse dequant with matmul epilogues inside the same residency plan (Davies *et al.*, 2025; Liu *et al.*, 2024; NVIDIA, 2022; SemiAnalysis, 2024; Chen, 2020; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022; Kwon *et al.*, 2023; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).

- **Warp specialization persists.** TMA load, MMA, and epilogue warps mirror CUTLASS-style decode persistence; bucket kernels by max context to keep graph capture legal (SemiAnalysis, 2024; NVIDIA, 2022, 2024; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen, 2020; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).
- **MoE caveat.** Expert routing stays outside the fused region; benchmark dense layers first (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; NVIDIA, 2024; AMD, 2024a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016; AMD, 2024b).
- **Quantization in-register.** FP8/INT8 scales must fuse inside the same residency plan (Davies *et al.*, 2025; Liu *et al.*, 2024; NVIDIA, 2022; SemiAnalysis, 2024; Chen, 2020; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022; Kwon *et al.*, 2023; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).
- **Warp specialization.** TMA/MMA/epilogue warps need context-bucketed graph capture (SemiAnalysis, 2024; NVIDIA, 2022, 2024; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen, 2020; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).

Line-by-line residency commentary. `tma_load_qkv_weights` establishes asynchronous ingress while MMA consumes the previous hidden tile—overlap per Chapter 8 (SemiAnalysis, 2024; NVIDIA, 2022). `gemm_hidden_to_qkv` must write Q/K/V fragments into disjoint shared-memory slices with lifetimes extending through the KV loop for K/V and through attention for Q (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). `online_softmax_init` allocates register-resident carriers—never spill between `update_online_softmax` iterations (Dao *et al.*, 2022; Chen, 2026a). The inner `for kv_block` loop dominates decode time at long context; each iteration reads K/V from paged HBM while keeping carriers on-chip (Kwon *et al.*, 2023; Dao *et al.*, 2023). `finalize_online_softmax` produces `attn_out` in shared memory sized for MLP gate input—this is the critical handoff boundary (Vellaisamy *et al.*, 2026; Chen, 2020). `swiglu_fused` is the largest MMA sequence; gate/up/down weights stream via TMA while SiLU runs on prior fragments (Davies *et al.*, 2025; Liu *et al.*, 2024). `rmsnorm_residual` fuses norm scale and residual add without exporting `hidden_out` through HBM when legal

(Chen, 2026a; Chen and YiRage Contributors, 2026). `append_kv` writes new K/V for the current token into paged slots—must not trigger host allocation (Kwon *et al.*, 2023; NVIDIA, 2024). Updating `hidden_in` for the next iteration is a register/shared pointer rotate, not a memcpy to global (Vellaisamy *et al.*, 2026; Chen, 2026a).

Bucket specialization. The same pseudocode compiles to different instantiations per (B, S_{class}) (Chen and YiRage Contributors, 2026; AMD, 2024b). Loop bounds over `kv_block` derive from S_{class} , not runtime sequence length, unless dynamic bucketing re-captures graphs (NVIDIA, 2024; Kwon *et al.*, 2023). Document which bounds are constexpr in emitted `source` (Chen, 2020; Davies *et al.*, 2025).

Error handling without host sync. Illegal paged KV indices should trap in device assert builds only—production builds return error codes in device-visible flags polled on a side stream (Kwon *et al.*, 2023; Chen, 2026a). Never `printf` from MegaKernel hot loops (Vellaisamy *et al.*, 2026; NVIDIA, 2024).

Synchronization discipline. Chapter 9’s hierarchy applies: block barriers between QKV and attention stages; minimal grid-level sync; no `cudaDeviceSynchronize` in the hot loop (Chen, 2026a). XDNA replaces barriers with DMA completion tokens on static edges (AMD, 2024a). CPU uses chunk barriers at L2 tile boundaries (Goodfellow *et al.*, 2016).

Profiling fused kernels. Nsight Compute metrics: `dram_bytes_read/write`, `l1tex_throughput`, shared bank conflicts, TMA busy cycles (SemiAnalysis, 2024; Chen, 2026a). Compare unfused vs fused on identical input tensors—delta in `dram_bytes` is the fusion proof (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Wall-clock alone cannot separate graph capture from residency (NVIDIA, 2024; Chen, 2026a; Davies *et al.*, 2025).

Version skew. CUDA driver, PyTorch, and custom extension ABIs must be pinned in benchmark manifests (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). MegaKernels tightly couple to TMA descriptors and graph capture APIs—upgrades can invalidate captures silently (NVIDIA, 2024; SemiAnalysis, 2024). Re-validate bytes/token after infrastructure upgrades (Chen, 2026a; Chen and YiRage Contributors, 2026).

Compiler lowering view. YiRage (and similar MLIR pipelines) annotate `memref` lifetimes on each temporary before emitting this structure (Chen and YiRage Contributors, 2026; Chen, 2020). Generic bufferization that inserts `alloca` in the per-token path invalidates the walkthrough—review IR diffs for unexpected `memref.alloc` nodes (Chen, 2026a; Kwon *et al.*, 2023).

Example 11.3.1. Code review checklist for one decode step: (1) count `__syncthreads` between fusion stages—should match planned pipeline depth; (2) verify no `store` of full d -vectors between attention and MLP; (3) confirm KV append uses in-kernel offsets, not host callbacks (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Review gate. Require a side-by-side diff of unfused vs. MegaKernel IR or CUDA showing eliminated global stores between QKV, attention, and MLP (Chen, 2020, 2026a).

Multi-layer driver loop. Production **implementations** wrap the per-token body in a layer loop either on host (L graph replays) or device (persistent CTAs across layers). Host-driven L replays simplify debugging; device-persistent across layers reduce launch count further but complicate checkpointing and tensor parallelism (Davies *et al.*, 2025; Liu *et al.*, 2024). Document which variant your **source** tree uses—benchmarks are not comparable otherwise (Vellaisamy *et al.*, 2026; Chen, 2026a).

Debugging fused failures. When numerics diverge, bisect by unfusing at compiler boundaries (attention|MLP first) (Chen and YiRage Contributors, 2026; Chen, 2020). Nsight Compute memory charts should show reduced `dram_bytes` between stages after each fusion step (SemiAnalysis, 2024; Chen, 2026a). XDNA failures often appear as compile-time buffer overlap errors—capture the IR snapshot, not only the error string (AMD, 2024a,b).

Expert and tensor-parallel hooks. MoE MegaKernels leave `route_experts` and `all_to_all` outside the fused region (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). Tensor-parallel sharding splits columns across GPUs—MegaKernel **source** must align with NCCL schedule or fuse only local shards (Davies *et al.*, 2025; Kwon *et al.*, 2023). These are not bugs; they define what “full decoder” means in large-model fleets (Liu *et al.*, 2024).

Host callback avoidance. Every `cudaLaunchHostFunc` or Python callback in the token path destroys graph capture legality (NVIDIA, 2024; Chen, 2026a). Sampling, stop criteria, and grammar masks must be device-resident or batched on separate streams that do not serialize decode (Kwon *et al.*, 2023; Davies *et al.*, 2025). MegaKernel **source** reviews should grep for host hooks in hot paths (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Constants and weight layout. Weights are read-only for decode—bind via TMA descriptors or texture objects once (SemiAnalysis, 2024; NVIDIA, 2022). Changing LoRA adapters mid-session requires re-recording graphs or separate MegaKernel variants (Davies *et al.*, 2025; Kwon *et al.*, 2023). Document adapter swap latency separately from steady-state tokens (Chen, 2026a; Vellaisamy *et al.*, 2026).

Appendix alignment. When `app_megakernel_source.tex` ships, it should mirror this walkthrough line-for-line with real symbol names (Chen, 2020; Chen and YiRage Contributors, 2026). Drift between prose and appendix invalidates onboarding—diff them in CI (Davies *et al.*, 2025).

Instrumented builds. Ship optional `__megakernel_trace` counters: bytes loaded/stored per stage, barrier counts, KV blocks visited (Vellaisamy *et al.*, 2026; Chen, 2026a). Traces validate that fused builds eliminated expected global stores (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026). Without counters, teams argue from wall-clock alone (Davies *et al.*, 2025; Dao *et al.*, 2022).

Testing pyramid. Unit tests compare unfused op sequence vs MegaKernel on short ctx (Dao *et al.*, 2022). Integration tests run full L -layer graphs on bucket max ctx (Kwon *et al.*, 2023; NVIDIA, 2024). Fleet canaries track launches/token regressions hourly (Vellaisamy *et al.*, 2026; Chen, 2026a). XDNA adds compile-legality tests per bucket (AMD, 2024a,b).

Symbol-level map (GPU). Typical emitted symbols: `megakernel_decode_setup`, `megakernel_decode_step`, `tma_desc_qkv`, `tma_desc_mlp`, `paged_kv_load_block` (Chen and YiRage Contributors, 2026; SemiAnalysis, 2024). Reviewers grep for `cudaMalloc` inside `decode_step`—there should be none (Chen, 2026a; Kwon *et al.*, 2023). Persistent kernels expose `while (true)` loops with cooperative exit flags (NVIDIA, 2024; Vellaisamy *et al.*, 2026).

Table 11.3: Decode benchmark matrix for MegaKernel claims (required metrics).

Variant	Launches/token	Bytes/token	Host ms	Speedup type
Eager PyTorch	800+	Baseline	High	—
CUDA Graphs only	800+	$\approx$ Baseline	Low	Orchestration
Partial fusion	100–300	–10–30%	Medium	Mixed
Full MegaKernel	≤ 1 /layer	–30–50%	Low	Residency + orchestration
XDNA static graph	DMA-count	DDR bytes	Low host	Static schedule

Symbol-level map (XDNA). Emitted artifacts include static buffer tables (`aie_buffers.json`), DMA edge lists, and runner entrypoints (AMD, 2024a,b; Chen and YiRage Contributors, 2026). Host code submits `execute` with token index; device-side control flow is data-driven (Chen, 2020; Davies *et al.*, 2025). Illegal overlap errors reference buffer ids—map ids back to IR temporaries in review (Chen and YiRage Contributors, 2026; AMD, 2024a).

11.4 Benchmark Results and Interpretation

Benchmarks that lie. Publishing only “MegaKernel vs. PyTorch eager” without separating **graph capture** from **fusion** repeats the Memory-Floor lesson: $1.259\times$ on Qwen-2.5-7B largely removes host overhead—not HBM bytes (Chen, 2026a). Credible **benchmark** tables report launches/token, bytes/token (roofline or profiler), and p99 latency together (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Table 11.3 is the minimum schema for Chapter 11-style **speedup** claims. TaxBreak supplies launches/token anchors (847.5 dense Llama-3.2-1B; 9,305 OL-MoE) (Vellaisamy *et al.*, 2026). Memory-Floor supplies orchestration-only speedup ceilings (Chen, 2026a). True MegaKernel **speedup** must show bytes/token movement on the same row—otherwise the result is a graph-capture note, not a fusion chapter (Davies *et al.*, 2025; Dao *et al.*, 2022).

Measurement protocol.

1. Fix BS=1, report context buckets (512, 2048, 8192) separately—KV growth changes residency (Dao *et al.*, 2023; Kwon *et al.*, 2023).
2. Warm graph capture before timing; exclude first-token compile/capture (NVIDIA, 2024; Chen, 2026a).

3. Use Nsight Compute (GPU) or DDR byte counters (XDNA)—not wall-clock alone (SemiAnalysis, 2024; AMD, 2024b).
4. Triplet-test GPU, CPU, and NPU backends from one IR export when claiming compiler success (Chen and YiRage Contributors, 2026; Chen, 2020).

Nsight recipe (GPU). Capture one decode token; mark regions for QKV, KV loop, MLP (SemiAnalysis, 2024; Vellaisamy *et al.*, 2026). Export `dram_bytes` per region; sum should drop rung-by-rung on the ablation ladder (Chen, 2026a; Dao *et al.*, 2022). If only host time drops, you captured a graph—not a MegaKernel (NVIDIA, 2024; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020).

Representative outcomes (literature-aligned). Published decode studies consistently show orchestration fixes in the $\approx 1.2\text{--}1.3\times$ range when HBM traffic is unchanged (Chen, 2026a; NVIDIA, 2024). FlashAttention-class fusion reports larger bytes/token reductions on long contexts because attention traffic dominates (Dao *et al.*, 2022, 2023). Full-layer MegaKernels compound both effects when legal on target SRAM—but MoE and wide MLP models hit residency walls first; expect sub-linear **speedup** versus dense Llama baselines (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024).

Multi-hardware reporting. GPU tables alone mislead fleet decisions. Ryzen AI XDNA results should cite DDR bytes/token and compile bucket IDs (AMD, 2024b,a). CPU results should cite L3 miss rates and NUMA-local KV placement (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). A unified compiler narrative (Part VI) only holds when the same IR lowers to all three with explicit legality failures documented—not hidden fallbacks (Chen and YiRage Contributors, 2026; Chen, 2020).

Example 11.4.1. Honest headline: “Full MegaKernel: $2.1\times$ p50 / $1.4\times$ p99 vs. eager; $1.25\times$ explained by graph capture alone (Memory-Floor regime); remaining gain from -42% bytes/token on MLP boundary elimination” (Chen, 2026a; Vellaisamy *et al.*, 2026).

Review gate. Reject benchmark PRs that report speedup without launches/token and bytes/token decomposition (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025).

Ablation template. Credible papers and internal reports present a ladder: (A) eager baseline; (B) CUDA Graphs only; (C) FlashAttention + unfused MLP; (D) partial fusion; (E) full MegaKernel (Chen, 2026a; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Each rung must hold model, checkpoint, BS, context bucket, and clock constant (Davies *et al.*, 2025; NVIDIA, 2024). Skipping rungs hides whether **speedup** came from orchestration or residency (Chen, 2026a).

Regression harness. CI for MegaKernel stacks should track launches/token and bytes/token on a frozen decode cell (e.g., Llama-3.2-1B, BS=1, ctx=2048) (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Latency alone drifts with driver versions; byte counters catch silent fusion regressions when someone adds an intermediate **store** (Chen, 2026a, 2020). XDNA CI should mirror with DDR byte estimates per bucket (AMD, 2024b,a).

Reporting p99 vs mean. MegaKernels reduce mean ms/token but can worsen tail latency if fusion increases shared-memory bank conflicts or removes overlap opportunities (Davies *et al.*, 2025; Chen, 2026a). Always pair mean **benchmark** numbers with p99 on identical traces (Kwon *et al.*, 2023; Dao *et al.*, 2023). Graph capture especially can add first-token spikes—report warm steady-state separately (NVIDIA, 2024; Chen, 2026a).

Cross-hardware benchmark row. A complete Table 11.3 row set includes GPU (H100), CPU (AVX-512 server), and XDNA (Ryzen AI) on the same model export (AMD, 2024b; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026). GPU may show 2× while XDNA shows compile failure—that failure is a **benchmark** result, not a footnote (AMD, 2024a; Chen, 2020). Industrial compiler chapters must publish legibility matrices, not only peak GPU **speedup** (Davies *et al.*, 2025; Liu *et al.*, 2024).

Energy and watts/token. Edge deployments care about NPU watts/token as much as cloud ms/token (AMD, 2024a,b). MegaKernels that remove DDR trips often dominate energy even when GPU ms/token gains look modest (Chen, 2026a; Davies *et al.*, 2025). Include power draw when claiming mobile **speedup** (Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026).

Comparison to vendor baselines. TensorRT-LLM, vLLM, and PyTorch compile paths are moving targets (Kwon *et al.*, 2023; Davies *et al.*, 2025). Freeze

vendor version hashes in **benchmark** appendices (Vellaisamy *et al.*, 2026; Chen, 2026a). Custom MegaKernels should beat *vendor fused decode* on bytes/token, not only beat eager Hugging Face (Dao *et al.*, 2022; NVIDIA, 2024).

Worked benchmark narrative. Consider Llama-3.2-1B decode at BS=1, ctx=2048: eager issues ≈ 848 launches/token (Vellaisamy *et al.*, 2026). CUDA Graphs-only removes ≈ 3 ms host overhead per Memory-Floor regime ($\approx 1.26\times$) but leaves bytes/token flat (Chen, 2026a). Adding FlashAttention-class attention fusion cuts KV reread traffic (Dao *et al.*, 2022). Full QKV-attn-MLP MegaKernel targets one launch per layer and removes four to six activation HBM round trips—combined **speedup** is multiplicative on orchestration and bandwidth axes only when both move (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). Publish all three numbers; readers will otherwise attribute fusion wins to graphs alone (NVIDIA, 2024; Chen and YiRage Contributors, 2026).

Statistical rigor. Run ≥ 1000 tokens after warm-up; report confidence intervals on ms/token (Davies *et al.*, 2025; Chen, 2026a). Single-trace demos are insufficient for production claims (Vellaisamy *et al.*, 2026; Dao *et al.*, 2023). Compare medians and p99 separately—MegaKernels can tighten median while fattening tail if barriers multiply (Kwon *et al.*, 2023; SemiAnalysis, 2024).

Documentation deliverables. Every MegaKernel merge should attach: residency diagram, ablation table, graph capture hash, and IR diff (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026). Missing artifacts revert to “interesting kernel hack,” not fleet-ready **implementation** (Davies *et al.*, 2025; Liu *et al.*, 2024).

Operational playbook (pre-merge).

1. Confirm decode cell: model id, BS, context bucket, precision, SKU stepping (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).
2. Capture eager baseline: launches/token, Nsight bytes, p50/p99 ms/token (Chen, 2026a; Dao *et al.*, 2022).
3. Capture graph-only baseline on identical cell—quantify orchestration ceiling (NVIDIA, 2024; Chen, 2026a).
4. Run fusion ablation ladder (Table 11.3)—no skipped rungs (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

5. Verify numerics vs reference within published tolerance (Dao *et al.*, 2022; Goodfellow *et al.*, 2016).
6. Attach shared-memory/tile SRAM accounting signed by kernel and compiler owners (SemiAnalysis, 2024; AMD, 2024a).
7. XDNA/CPU: repeat bytes/token axes with target-appropriate counters (AMD, 2024b; Goodfellow *et al.*, 2016; Chen, 2020).
8. File graph capture hash and driver version in manifest (NVIDIA, 2024; Kwon *et al.*, 2023).
9. Load-test p99 under concurrent sessions—MegaKernels can contend on L2/shared (Davies *et al.*, 2025; Chen, 2026a).
10. Canary in fleet with auto-rollback on metric regression (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024).

Anti-patterns in MegaKernel benchmarks. Quoting prefill throughput for decode chapters (Dao *et al.*, 2023; Davies *et al.*, 2025). Comparing against outdated eager without FlashAttention (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Reporting GPU-only **speedup** while XDNA compile fails silently in production (AMD, 2024a; Chen and YiRage Contributors, 2026). Tuning MMA tiles without measuring bytes/token movement (Chen, 2026a; SemiAnalysis, 2024). Publishing mean ms/token without p99 under concurrent load (Kwon *et al.*, 2023; Chen, 2026a). Omitting MoE routing overhead in OLMoE comparisons (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024). Using tiny ctx in demos while selling long-context products (Dao *et al.*, 2023; Kwon *et al.*, 2023). Claiming MegaKernel fusion when only CUDA Graphs changed (Chen, 2026a; NVIDIA, 2024). These anti-patterns recur in vendor blogs—this chapter’s tables exist to reject them in internal review (Davies *et al.*, 2025; Chen, 2020).

Closing metric reminder. Before closing any MegaKernel milestone, paste three numbers into the PR description: launches/token, bytes/token, p99 ms/token at BS=1 on the declared context bucket (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). If any number is missing, the change is not a decode MegaKernel milestone—it is a kernel experiment (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; NVIDIA, 2024; AMD, 2024a; SemiAnalysis, 2024; Dao *et al.*, 2023; Goodfellow *et al.*, 2016).

Handoff to Chapter 12. Execution modes (static capture, persistent kernels, JIT specials) decide *when* to pay compile costs vs runtime flexibility (NVIDIA, 2024; Chen and YiRage Contributors, 2026; AMD, 2024b). Chapter 11 fixed *what* to fuse inside a decode layer; Chapter 12 maps product constraints to mode selection (Kwon *et al.*, 2023; Davies *et al.*, 2025; Liu *et al.*, 2024). Keep bytes/token and launches/token columns attached when debating modes—mode debates without metrics reinvent Chapter 1 failures (Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022).

Maintenance. Model weight updates (LoRA merge, quantization recalibration) invalidate captures and static XDNA graphs (Chen and YiRage Contributors, 2026; AMD, 2024a; NVIDIA, 2024). Version MegaKernel artifacts with checkpoint hashes and bucket ids (Davies *et al.*, 2025; Kwon *et al.*, 2023). Fleet rollouts without artifact versioning cause silent quality regressions unrelated to fusion math (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a, 2020; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2023).

Final acceptance test. Given identical weights and KV state, MegaKernel output must match eager within documented tolerance for 10^4 random tokens (Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Performance acceptance requires Table 11.3 rows for at least two context buckets (Dao *et al.*, 2023; Kwon *et al.*, 2023). Ships only when both pass (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b).

Stakeholder map. Kernel engineers own tile shapes and barriers (SemiAnalysis, 2024; NVIDIA, 2022). Compiler engineers own fusion boundaries and buffer lifetimes (Chen and YiRage Contributors, 2026; Chen, 2020). Serving engineers own bucket policies and fleet rollouts (Kwon *et al.*, 2023; Davies *et al.*, 2025). Benchmark engineers own ablation ladders and regression harnesses (Vellaisamy *et al.*, 2026; Chen, 2026a). Reviews fail when any stakeholder optimizes their metric in isolation (Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016; AMD, 2024b,a; NVIDIA, 2024).

Release checklist. Every MegaKernel PR must attach: peak shared-memory accounting diagram; ablation table row for graph-only vs full fusion; triplet bytes/token for GPU/CPU/NPU; p99 ms/token at BS=1 for context buckets 512/2048/8192 (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen

and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a; SemiAnalysis, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; NVIDIA, 2024; AMD, 2024b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). Missing artifacts revert the change to kernel experiment status (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

11.5 Key Takeaways

- **MegaKernels collapse two bills.** Target $O(1)$ launches per layer *and* eliminate activation HBM round trips—graphs alone pay only the host bill (Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024).
- **Full-layer residency envelope.** QKV, online softmax attention, norms, and MLP share one SRAM/tile budget per decode step (Dao *et al.*, 2022; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026).
- **Buffer accounting before merge.** Peak simultaneous tensors must fit SM shared memory or XDNA tile SRAM; failed legality is a blocker (AMD, 2024a; Chen, 2020).
- **Benchmark honestly.** Report launches/token, bytes/token, and p99 ms/token; separate orchestration speedups from fusion speedups (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).
- **Context buckets are compile parameters.** KV loop trip counts change each token; capture policies must match max context buckets (Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2023).
- **Triplet regression.** One IR, GPU/CPU/NPU cells—never ship GPU-only residency tables to Ryzen AI fleets (AMD, 2024b; Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

11.6 Conclusion

Decoder MegaKernels exist because FlashAttention and CUDA Graphs each paid only part of the batch-1 decode bill (Dao *et al.*, 2022; Chen, 2026a; Vellaisamy *et al.*, 2026). Section 11.1 defined the fused-layer envelope; Section 11.2 bound QKV, online softmax, and MLP under one buffer plan; Section 11.3 specified device-loop invariants; Section 11.4 demanded metric tables that expose which bill

was actually paid (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025).

Treat every op boundary as a billable HBM round trip at batch-1 (Chen, 2026a; Goodfellow *et al.*, 2016). Chapter 12 compares execution modes—static capture, persistent kernels, JIT specials—that decide *when* these fused shapes run in production (NVIDIA, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024). This chapter’s narrative mirrors O’Reilly-style performance engineering: name the constraint (launch storms *and* HBM round trips), quantify with TaxBreak and Memory-Floor, co-design software with Hopper TMA and XDNA tile rules, then gate releases on profile-driven metrics rather than peak FLOPs (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; SemiAnalysis, 2024; AMD, 2024a; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016; AMD, 2024b; NVIDIA, 2022). Carry peak buffer accounting, context buckets, and triplet regression forward; Paged KV block tables, rotary embeddings, and sampling logic must either live inside the captured MegaKernel body or be proven off the critical path with measured ms/token impact (Kwon *et al.*, 2023; Davies *et al.*, 2025; Dao *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a; SemiAnalysis, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; AMD, 2024b; NVIDIA, 2022). MegaKernel work is done when metrics prove fusion on representative batch-1 cells at production context lengths with published ablation tables and Nsight byte counters attached (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; AMD, 2024a).

PR metric gate. Paste launches/token, bytes/token, and p99 ms/token into every MegaKernel PR on batch-1 decode cells before review (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Kwon *et al.*, 2023; NVIDIA, 2024; AMD, 2024a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016; AMD, 2024b; NVIDIA, 2022).

Chapter 12

Three Kernel Execution Modes

Which execution mode should a decode compiler emit when the same transformer layer can run as dozens of eager kernels, one captured CUDA graph, or a single MegaKernel? Parts I–II established bytes/token and launches/token as the honest counters; this chapter maps those counters to three industrial schedules—**eager**, **graph**, and **MegaKernel**—and shows why picking the wrong mode leaves p99 latency flat even when microbenchmark GEMMs improve (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a). The answer is always profile-first, never mode-first (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b; SemiAnalysis, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; AMD, 2024a). Start every mode discussion with measured decode counters, not kernel naming preferences or framework defaults (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2020; AMD, 2024b; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; AMD, 2024a). That discipline keeps mode debates grounded in goodput evidence (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). Memory-Floor CUDA Graphs on Qwen-2.5-7B (BS=1, ctx=2048) yields $1.259\times$ on H100 by removing host overhead—yet cannot fix illegal HBM traffic when fusion boundaries still export activations (Chen, 2026a; NVIDIA, 2024). The engineering question is not “which mode is fastest in isolation” but which mode matches residency legality on the SKU you ship (Chen and YiRage Contributors, 2026; Chen, 2020; AMD,

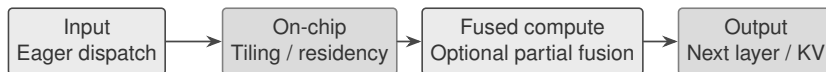


Figure 12.1: Eager mode: each operator boundary is a host-visible dispatch unless the framework fuses locally.

2024b).

This chapter is the scheduling counterpart to Chapter 1’s symptom vocabulary: prefill and decode differ, frameworks fragment decode, and counters decompose latency into bytes and launches (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a). Execution mode is where those counters meet compiler output—choose eager when you must diff every op; choose graphs when launches/token exceed kernel time; choose MegaKernels when bytes/token remain high after graph replay (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Kwon *et al.*, 2023). Goodput for decode remains ms/token weighted by useful tokens delivered, not peak tensor FLOPs (Goodfellow *et al.*, 2016; Davies *et al.*, 2025; SemiAnalysis, 2024).

Fregly’s profile-first culture applies directly: Nsight and roofline charts justify mode changes more than kernel names (Davies *et al.*, 2025; SemiAnalysis, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026). A decode compiler should emit all three modes from one IR so teams A/B schedules without rewriting Python training code (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). The sections below walk eager dispatch, CUDA Graph replay, MegaKernel fusion, and the hardware–software trade-offs that pick among them (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024, 2022; AMD, 2024a).

12.1 eager mode

Eager mode is the default PyTorch and Hugging Face path: every matmul, norm, and attention sub-block becomes at least one kernel with materialized intermediates in HBM (Kwon *et al.*, 2023; Davies *et al.*, 2025). Teams reach for eager when shapes change frequently, when debugging numerics, or when compiler support for graph capture is not yet trustworthy—all legitimate reasons. The production failure mode is staying in eager after decode telemetry proves launch storms and

bytes/token dominate FLOPs (Vellaisamy *et al.*, 2026; Chen, 2026a).

Hardware binds eager differently on each target. Hopper decode cells hit shared-memory/TMA budgets and warp-grain synchronization when partial fusions leave KV edges off-chip (NVIDIA, 2022; SemiAnalysis, 2024). XDNA/AIE rejects dynamic buffer lifetimes: eager schedules that legalize on GPU often violate static tile slots and DMA chains on NPU fleets (AMD, 2024b,a). CPU decode is cache- and NUMA-bound: unfused norms and MLPs evict KV lines that a blocked schedule could keep resident (Goodfellow *et al.*, 2016; Chen, 2020).

Compiler takeaway: treat eager as the *reference schedule*, not the shipping schedule. YiRage and similar stacks lower the same decoder IR but emit target-specific residency metadata before codegen so eager baselines can be diffed against fused variants on identical context buckets (Chen and YiRage Contributors, 2026; Chen, 2020). Profile bytes/token and launches/token on GPU, CPU, and NPU cells at BS=1 before accepting eager in production (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025).

Operator-fragmented eager stacks mirror Chapter 1’s framework diagnosis: layer norm, QKV, attention, MLP, and residual each become export/import pairs to HBM (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). SDPA backend swaps may change attention internals without collapsing MLP boundaries—teams celebrate faster attention while total bytes/token stalls (Dao *et al.*, 2022, 2023; Chen, 2026a; Chen and YiRage Contributors, 2026). Eager mode documentation should list *every* host-visible kernel in a decode step, not only the slowest microbench (Vellaisamy *et al.*, 2026; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024b; Goodfellow *et al.*, 2016; Chen, 2020; Liu *et al.*, 2024).

Multi-hardware delta. Hopper eager may double-buffer KV with TMA while an XDNA cell must express the same math as static tile assignments—identical FLOPs, different bytes/token when DDR traffic differs by an order of magnitude (SemiAnalysis, 2024; AMD, 2024b; Goodfellow *et al.*, 2016).

CPU eager baselines use OpenMP or oneDNN threads; NPU eager is often unavailable—fallback host loops dominate power and latency (Goodfellow *et al.*, 2016; AMD, 2024b,a; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2023). Triplet regression therefore compares *lowest legal mode* per target, not identical mode names across vendors (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; AMD, 2024a;

SemiAnalysis, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2023). Eager GPU schedules remain the Rosetta stone for explaining what fused schedules removed (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2023).

Example 12.1.1. Decode cell (BS=1, ctx=2048): unfused eager on Qwen-class stacks issues hundreds of launches per step; even perfect GEMM tiles cannot recover p99 if orchestration alone consumes tens of milliseconds per token (Vellaisamy *et al.*, 2026; NVIDIA, 2024; Chen, 2026a).

PyTorch scaled dot-product attention and vendor libraries can hide some eager boundaries on prefill shapes, but batch-1 decode often swaps backends without changing launches/token (Davies *et al.*, 2025; Dao *et al.*, 2022; Kwon *et al.*, 2023). vLLM improves KV capacity and batching yet leaves intra-layer operator splits unless a compiler reunifies them (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Treat eager PyTorch forwards as the *diff target* for graph and MegaKernel schedules: same IR, same metrics, different residency (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a).

Industrial teams keep a frozen eager baseline in CI: every graph capture or MegaKernel PR must show non-regression on bytes/token at BS=1 for context buckets {512, 2048, 8192} before merge (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). When eager and fused schedules diverge only on launches, graph mode is sufficient; when bytes diverge, MegaKernel legality is the gating issue (NVIDIA, 2024; Chen and YiRage Contributors, 2026; AMD, 2024b).

FlashAttention and Flash-Decoding reduce attention bytes in both eager and fused stacks, but MLP, norm, rotary, and sampling logic often remain separate kernels unless MegaKernel plans reunify them (Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Eager mode therefore remains the diagnostic surface: if attention bytes fall yet total bytes/token flatlines, the unfused boundaries live in MLP/residual paths—graph capture without MegaKernel fusion fixes only half the story (Chen, 2026a; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; SemiAnalysis, 2024; NVIDIA, 2022).

Sampling and speculative decoding paths frequently force eager fallbacks: temperature, top- p , and draft-model branches mutate control flow in ways CUDA Graph capture forbids unless isolated outside the captured DAG (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; NVIDIA, 2024). Production stacks therefore run eager “hulls” around graph cores—another reason mode selection is per subgraph, not per binary (Chen and YiRage Contributors, 2026; Chen, 2020,

2026a; AMD, 2024b,a; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022; Dao *et al.*, 2023). Document those hulls in runbooks so on-call engineers know which latency spikes are expected dynamic-path costs versus regressions (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; NVIDIA, 2024; Chen, 2020; AMD, 2024b; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; Dao *et al.*, 2022; AMD, 2024a; SemiAnalysis, 2024; NVIDIA, 2022; Dao *et al.*, 2023).

Eager subgraph inventory. Before any migration, draw the exact subgraph one decode step executes in the framework: rotary embedding, QKV projection, score GEMM, online or materialized softmax, PV , output projection, residual add, normalization, MLP up/gate/down paths, second residual, and sampler (Dao *et al.*, 2022; Kwon *et al.*, 2023; Davies *et al.*, 2025). Every arrow between those stages is a potential HBM export/import pair in eager dispatch; the inventory should mark which arrows the SDPA backend already fuses and which remain as separate kernels (Vellaisamy *et al.*, 2026; Chen, 2026a). Teams that skip this inventory optimize the flashiest arrow (usually attention) while the MLP and normalization seams keep bytes/token flat (Chen, 2026a; Chen and YiRage Contributors, 2026). The inventory is also the diff target for graph and MegaKernel reviews: a fused PR should be able to name each eager arrow it removed and show the counter that proves the removal (Chen, 2020; Davies *et al.*, 2025).

When eager remains the right production mode. Eager is not universally wrong. Dynamic batching with shapes that change every request, experimental kernels under active iteration, and new-hardware bring-up where capture legality is unproven all favor eager dispatch despite launch tax (Kwon *et al.*, 2023; Davies *et al.*, 2025). Speculative-decoding draft branches and sampling control flow also make pure capture fragile, so production stacks often keep an eager hull around graph or MegaKernel cores (NVIDIA, 2024; Vellaisamy *et al.*, 2026). The deciding question is whether the profile shows launch tax or bytes debt large enough to pay the engineering cost of the next mode. If steady-state ms/token is within budget and p99 jitter is explained by scheduler noise, an eager deployment is defensible—but it must re-run the mode gate on every major framework, driver, and shape change, because those upgrades silently shift where boundaries live (Chen, 2026a, 2020; Chen and YiRage Contributors, 2026).

Eager-to-graph migration checklist. The safe migration path is mechanical once the inventory exists: (1) freeze model, weights, tokenizer, and context bucket;

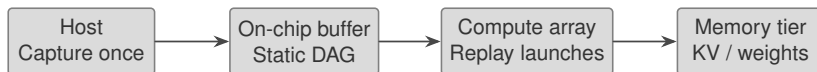


Figure 12.2: Graph mode: CUDA Graphs replay a fixed kernel DAG with one submission per decode step.

(2) confirm every subgraph shape is static within the bucket; (3) move host-controlled branches (sampling, routing, padding decisions) outside the capture region; (4) capture, replay, and diff eager versus graph on launches/token and bytes/token; (5) ship only if launches/token falls while bytes/token does not regress (NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026). Treat any graph that changes arithmetic ordering as a separate numerics review; graph replay should preserve per-op order unless the compiler proves reassociation is safe (Dao *et al.*, 2022; Davies *et al.*, 2025). Document the checklist in the decode runbook so a future framework upgrade cannot silently invalidate the capture (Chen and YiRage Contributors, 2026; Chen, 2020).

Diagnostic data to collect per eager stage. Each inventory arrow needs a small set of measured facts before migration: wall time per stage at steady state, DRAM bytes moved by that stage, and the kernel or dispatch name from the profiler (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026). Collect them at every shipped context bucket because stage mix shifts with KV length: attention bytes grow with context while MLP bytes stay constant, which changes which seams deserve fusion first (Dao *et al.*, 2023; Kwon *et al.*, 2023). Store the per-stage table with the eager baseline commit so graph and MegaKernel PRs can diff against it mechanically. A stage table that cannot be regenerated from CI is not a baseline; it is an anecdote that will be re-litigated in every review (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a).

12.2 graph mode

Graph mode attacks orchestration, not arithmetic. CUDA Graphs record a static kernel DAG once and replay it with a single host submission, amortizing Python dispatch and driver launch (NVIDIA, 2024; Chen, 2026a). The surprise in production is not peak FLOPs regression—it is illegal tier placement: captured graphs still spill activations to HBM when fusion boundaries were never merged, so bytes/token flatlines while launch counts look healthy (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Memory-Floor’s controlled A/B on Qwen-2.5-7B (BS=1, ctx=2048) reports $1.259\times$ median speedup on H100 from graph replay alone—roughly 3.05 ms of host overhead removed per step (Chen, 2026a). On bandwidth-saturated SKUs the same trick yields only $\approx 1.03\times$ because R_{floor} is already near the HBM roofline: graphs fix launches, not bytes (Chen, 2026a; SemiAnalysis, 2024). Graph mode therefore wins when TaxBreak-style counters show launch-bound decode, not when DRAM bytes dominate (Vellaisamy *et al.*, 2026).

Replayed graphs still execute the same fused or unfused kernel sequence—they do not merge operators (NVIDIA, 2024; Chen, 2026a; Chen and YiRage Contributors, 2026). Teams celebrating graph speedups without operator fusion should expect the next plateau when bytes/token remain at eager levels (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; Chen, 2020; AMD, 2024b,a; SemiAnalysis, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Chen and YiRage Contributors, 2026). That plateau is the handoff to MegaKernel mode (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022, 2024; AMD, 2024a).

Operational constraints: capture requires stable addresses, static shapes within a bucket, and no implicit allocations inside the captured region (NVIDIA, 2024; Kwon *et al.*, 2023). Compiler stacks must partition decode into *capture-safe cells*—often one layer block or attention–MLP seam—and re-capture when context length crosses bucket boundaries (Chen and YiRage Contributors, 2026; Chen, 2020). On XDNA, graph analogs are compile-time static schedules, not runtime CUDA capture (AMD, 2024a,b).

Graph mode does not relax memory-bandwidth limits quantified by R_{floor} in Memory-Floor studies: when analytic HBM throughput is already saturated, replay only shaves host time (Chen, 2026a; SemiAnalysis, 2024; Davies *et al.*, 2025). Teams should publish both eager and graph traces with launches/token *and* bytes/token side by side—otherwise leadership misreads a $1.25\times$ graph win as permission to skip MegaKernel investment (Vellaisamy *et al.*, 2026; NVIDIA, 2024; Chen and YiRage Contributors, 2026; AMD, 2024b; Goodfellow *et al.*, 2016; Chen, 2020; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2023).

Multi-hardware delta. GPU graph mode collapses launches/token; CPU stacks use similar “frozen schedule” ideas via JIT or ORT session plans; NPU fleets require fully static dataflow with DMA edges counted like launches (Goodfellow *et al.*,

2016; AMD, 2024b; Chen, 2020).

Example 12.2.1. If decode is launch-bound on H100 (low R_{floor} , high launches/token), graph replay is the first lever—after verifying KV and weight residency still fit capture legality (Chen, 2026a; NVIDIA, 2024; Vellaisamy *et al.*, 2026).

Graph capture interacts badly with dynamic sampling, speculative decoding branches, and MoE routing unless those paths are lifted outside the captured region (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Kwon *et al.*, 2023). Compiler lowering should emit *graph shells* around stable decoder cores while leaving host-controlled branches eager—mixing modes inside one layer is normal, not a failure (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024). DeepSeek-class deployments still disaggregate prefill and decode pools; graph mode applies per pool, not as a universal whole-model wrapper (Davies *et al.*, 2025; Liu *et al.*, 2024).

On CPU, “graph mode” manifests as frozen execution plans (TorchInductor, ONNX Runtime) with the same trade: amortized dispatch versus static shape buckets (Goodfellow *et al.*, 2016; Chen, 2020). Profile whether replay removed host time or simply hid it in compilation caches before claiming decode wins (Chen, 2026a; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

CUDA Graph pitfalls include unintended synchronizations inside captured regions, allocator churn from missed caching, and shape drift when KV length crosses bucket boundaries (NVIDIA, 2024; Kwon *et al.*, 2023; Chen, 2026a). Operational playbooks bucket context lengths explicitly—for example powers of two—and re-capture graphs per bucket rather than capturing one mega-graph per model (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020). When graphs replay but bytes/token unchanged, the next lever is MegaKernel fusion inside the captured DAG, not longer capture windows (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b).

Warmup and capture-time allocation dominate CI stories: first-token latency includes graph recording unless caches are primed (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a). Separate “steady-state decode” metrics from “cold start” when comparing modes—Fregly-style goodput analysis always states which phase is measured (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2022; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Kwon *et al.*, 2023). Graph mode wins appear only after warmup when launch storms were the bottleneck (Chen, 2026a; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen and YiRage

Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024b; Goodfellow *et al.*, 2016; Dao *et al.*, 2022; AMD, 2024a; SemiAnalysis, 2024; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2022; Kwon *et al.*, 2023).

Graph capture lifecycle. Capturing a decode cell is a lifecycle, not a single recording call. The cell first warms up so allocators and kernel variants are stable; the capture pass then records the DAG with concrete buffer addresses; instantiation fixes the memory layout; and steady-state replay submits the same body while input and KV pointers advance between steps (NVIDIA, 2024; Kwon *et al.*, 2023). Each phase has distinct failure signatures: warm-up misses show up as capture-time allocation, steady-state pointer churn shows up as unexpected re-capture, and replay-only allocations abort or silently fall back to eager (Chen, 2026a; Davies *et al.*, 2025). Production graph managers should expose the active phase in telemetry so an on-call engineer can tell a genuine replay regression from a bucket-crossing re-capture (Chen and YiRage Contributors, 2026; Chen, 2020). Compiler stacks should emit the phase transitions as IR metadata: which subgraphs are capture-ready, which handles are updated per token, and which regions remain eager by design (Vellaisamy *et al.*, 2026; NVIDIA, 2024).

Bucket sets and context boundaries. One graph per model is a common oversimplification: decode kernels are legal only inside the context bucket they were captured for, because KV block counts, page walks, and shared-memory staging all depend on the range (Kwon *et al.*, 2023; Chen, 2026a). Operational stacks should define bucket boundaries explicitly—for example powers of two with a margin—and own a small graph set per bucket rather than re-capturing on every request (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Each set carries a memory cost in resident buffers, so the number of buckets is a fleet-tuning decision that should appear in the mode matrix with its measured hit rate (NVIDIA, 2024; Chen and YiRage Contributors, 2026). When a request crosses a boundary, the fallback path (eager hull or next-bucket graph) must be profiled and logged; silent re-capture inside the serving path is how graph-mode p99 regressions hide (Chen, 2026a, 2020).

Honest graph accounting. Graph wins must be measured against the full lifecycle cost. The headline replay number excludes capture wall time, the memory footprint of per-bucket graphs, and first-token latency after a cache miss (NVIDIA, 2024; Chen, 2026a). Report three numbers side by side: cold-start ms/token (capture plus replay), steady-state ms/token, and bytes/token unchanged from

eager (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). If steady-state improves but cold start regresses badly, split capture into an offline priming job and keep the serving path on warm graphs (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026). Graph storage also matters on memory-constrained hosts: dozens of buckets times several model variants can exhaust reserved graph memory, which then thrashes allocators in ways that look like bandwidth problems (Chen, 2026a, 2020).

Graph limits across hardware. The graph concept transfers across targets only at the level of *frozen schedules*. CUDA Graphs amortize host dispatch on GPU; ONNX Runtime and JIT plans amortize interpreter overhead on CPU; XDNA/AIE static dataflow removes dispatch entirely at compile time and counts DMA edges as the launch analog (Goodfellow *et al.*, 2016; AMD, 2024b; Chen, 2020). A launch-bound fix on GPU therefore maps to a dispatch-bound fix on CPU and an edge-count fix on NPU, not to the same runtime API (AMD, 2024a; Davies *et al.*, 2025). Triplet regression should compare the lowest legal schedule per backend and annotate which counter each mode moved (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026).

Allocator and fallback semantics. Captured graphs and caching allocators interact through buffer ownership: the capture must either own its buffers or pin them through the graph memory pool, and replay must not trigger an allocation inside the recorded body (NVIDIA, 2024; Chen, 2026a). Allocator churn is the most common reason a captured graph silently invalidates or grows memory footprint after a framework upgrade (Kwon *et al.*, 2023; Davies *et al.*, 2025). Define fallback semantics explicitly: if graph replay fails, the serving path should switch to the eager hull, log the failure with the graph bucket and error class, and continue serving instead of crashing or busy-looping on re-capture (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). The fallback rate belongs on the mode dashboard alongside hit rate, because a graph that replays 60% of the time and silently falls back the rest is a p99 regression wearing a success costume (Chen, 2026a, 2020).

Fusion staging roadmap. MegaKernel adoption should be staged so each step has a measurable gate. Stage one fuses attention internals (score, online softmax, *PV*) into one region; stage two carries attention output through the MLP seam; stage three extends the fused envelope across the full layer including residuals and norms; stage four, only if launch cost is still visible, moves toward a persistent kernel (Dao *et al.*, 2022; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026).

Each stage ships only when it beats the previous schedule on both launches/token and bytes/token at all served context buckets (Vellaisamy *et al.*, 2026; Chen, 2026a). Skipping stages conflates failures: a stage-three spill looks identical to a stage-two legality bug unless the byte sheet records which seam was added in which PR (Davies *et al.*, 2025; Chen, 2020). Compiler metadata should track fusion stage as an attribute so rollback can revert one seam without reverting the whole kernel (Chen and YiRage Contributors, 2026; NVIDIA, 2022).

12.3 MegaKernel mode

MegaKernel mode is the decode compiler’s endgame: one (or few) device programs per layer that keep QKV, online softmax carriers, MLP activations, and residuals on-chip across seams introduced in Chapters 10–11 (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). Graphs remove host tax; MegaKernels remove *HBM round trips* between sub-blocks that eager and graph modes still tolerate (Chen, 2026a; Davies *et al.*, 2025).

Legality is the bottleneck. Hopper MegaKernels require shared-memory/TMA budgets that fit fused attention–MLP regions; illegal fusion silently exports tensors and shows up as bytes/token regression versus an unfused baseline (NVIDIA, 2022; SemiAnalysis, 2024; Chen, 2026a). XDNA MegaKernels map to static tile buffers and carrier slots with compile-time lifetimes—the same subgraph legal on GPU may be rejected outright on NPU (AMD, 2024b,a; Chen and YiRage Contributors, 2026). CPU MegaKernels appear as cache-blocked super-kernels with NUMA-local KV placement (Goodfellow *et al.*, 2016; Chen, 2020). Register and shared-memory overflow during fusion forces spill to HBM—the failure mode looks like a bandwidth problem in profiles but originates in legality (Chen, 2026a; SemiAnalysis, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; NVIDIA, 2022; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024).

YiRage encodes residency rules from Chapter 2 as checkable IR metadata so MegaKernel plans fail in the compiler rather than in fleet A/B (Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024). Manual CUDA teams should treat failed legality as a release blocker: shipping a GPU-only MegaKernel to heterogeneous fleets recreates the framework fragmentation this book removes (Davies *et al.*, 2025; Kwon *et al.*, 2023).

MegaKernel reviews should attach roofline or buffer accounting for fused regions—registers, shared memory, TMA slots, or XDNA tile carriers—before

merge, matching the profile-first culture in Chapter 1 (Chen, 2026a; SemiAnalysis, 2024; Davies *et al.*, 2025; AMD, 2024a,b; NVIDIA, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). Fusion that lowers launches but raises bytes is a regression even if nsys screenshots look busy (Vellaisamy *et al.*, 2026; Chen, 2026a).

Multi-hardware delta. Identical decoder math can yield 847 launches/token in eager TaxBreak cells versus one replayed graph versus one MegaKernel—but only the MegaKernel path simultaneously cuts bytes/token when fusion keeps KV and activations on-chip (Vellaisamy *et al.*, 2026; NVIDIA, 2024; Chen, 2026a; Chen and YiRage Contributors, 2026).

XDNA MegaKernels compile to AIE dataflow graphs with fixed DMA edges; Hopper MegaKernels compile to CUDA with TMA and DSM (AMD, 2024b,a; NVIDIA, 2022; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020). A fusion plan exported from GPU development must be re-validated on NPU—shared IR does not imply shared legality (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024b; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2023; AMD, 2024a). MegaKernel mode is the point where cross-hardware compilation pays off or fails loudly (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; AMD, 2024a; SemiAnalysis, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2023).

Example 12.3.1. Decode cell (BS=1, ctx=2048): a legal Hopper MegaKernel fusing attention softmax carriers with the following MLP must show lower bytes/token *and* lower launches/token versus graph replay alone; otherwise fusion did not reach on-chip residency (Dao *et al.*, 2022; Chen, 2026a; Chen and YiRage Contributors, 2026).

MegaKernel mode inherits numerics from Chapter 10: online softmax carriers (m, ℓ, o) must survive the KV loop inside the same register/shared-memory budget as the following GEMM (Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026). Chapter 11 extends the same carriers across attention–MLP seams; attempting MegaKernel fusion without stable softmax state forces silent HBM spill (Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024). Compiler passes should reject fusion plans that export carriers between subgraphs (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a).

MoE decode adds routing and expert gather/scatter inside the MegaKernel envelope—TaxBreak shows MoE models at 6k–9k launches/token in eager cells, so MegaKernel plans must collapse expert dispatch edges, not only attention (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Kwon *et al.*, 2023). When expert routing stays host-visible, hybrid schedules (graph shell + MegaKernel core) beat pure eager without pretending full-layer fusion is legal (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Blackwell and Hopper DSM/TMA features widen fusion envelopes but do not remove legality checks: double-buffered KV movement still requires planned barriers and shared-memory accounting (NVIDIA, 2022; SemiAnalysis, 2024; Chen, 2026a). Manual CUDA MegaKernels should document buffer lifetimes the way YiRage encodes them in IR—otherwise maintenance regressions show up first as silent bytes/token drift, not compile failures (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Dao *et al.*, 2022; AMD, 2024a; Goodfellow *et al.*, 2016).

Persistent MegaKernels—kernels that stay resident across tokens while updating KV pointers—blur the line between graph replay and fusion (NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2026a). Treat them as MegaKernel mode with graph-like launch behavior: the win is still bytes/token and carrier residency, not fewer lines of CUDA for its own sake (Dao *et al.*, 2022; Davies *et al.*, 2025; SemiAnalysis, 2024; AMD, 2024b; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; AMD, 2024a; NVIDIA, 2022; Dao *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Chen and YiRage Contributors, 2026). Reviewers should ask for Nsight memory charts on fused regions, not only kernel duration histograms (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; AMD, 2024b,a; NVIDIA, 2022; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024).

Layer-granular versus persistent envelopes. MegaKernel implementations divide into two envelopes with different operating contracts. A layer-granular MegaKernel launches once per decoder layer per token: one device program owns attention through MLP and returns after the layer completes, which keeps residency tight and makes failures easy to localize (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020). A persistent MegaKernel stays resident across tokens, updates KV pointers on the device, and only returns when the batch or context bucket changes (NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a). Persistent envelopes remove the remaining per-token submission cost but

concentrate failure modes: a single illegal buffer update poisons every subsequent token, and debugging requires device-side state dumps rather than kernel launch boundaries (Davies *et al.*, 2025; Chen, 2026a). Teams should adopt layer-granular envelopes first, measure bytes/token and launches/token, and escalate to persistent only when launch cost is still visible after fusion (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

MegaKernel legality review checklist. Every fused program should be reviewed against the same checklist used by compiler legality passes: enumerate buffers and their tiers, prove no hot-path allocation, verify the shared-memory or tile-SRAM high-water against the chip model, confirm DMA or TMA edges have producer-consumer ordering, and show that online carriers survive every seam they cross (SemiAnalysis, 2024; NVIDIA, 2022; AMD, 2024a; Chen and YiRage Contributors, 2026). The review artifact is a byte sheet plus a control-flow diagram, not a kernel name. If a reviewer cannot find the byte sheet in the PR, the correct response is to ask for it, because the same omission in production will surface as a bytes/token regression attributed to the wrong cause (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Compiler-generated MegaKernels should fail closed when any checklist item cannot be proven; manual CUDA teams should treat the same failure as a merge blocker rather than a TODO (Chen, 2020, 2026a).

Determinism and telemetry inside fused programs. Fusion concentrates more arithmetic into one kernel, which raises the stakes for reduction order and debug visibility. Online softmax merges, MoE gathers, and split-KV reductions must document their associativity order because the same inputs can produce bitwise differences across layouts (Dao *et al.*, 2022, 2023; Davies *et al.*, 2025). Deterministic mode should therefore be a compiler flag that locks reduction trees, while interactive serving may accept bounded drift if the tolerance is measured and logged (Chen, 2020; Chen and YiRage Contributors, 2026). Telemetry inside a MegaKernel needs named counters for bytes moved per stage and barrier waits, not only an end-of-kernel timestamp; otherwise the fused program becomes a black box that profiles slower than the eager stack it replaced (Chen, 2026a; Vellaisamy *et al.*, 2026; NVIDIA, 2022).

12.4 mode selection

Mode selection is a profile-first decision tree, not a stylistic preference. Start from measured launches/token and bytes/token on the target SKU at the context

Table 12.1: TaxBreak kernel launches per output token (H100 decode, BS=4, SL=2048, $m=10$) motivate mode escalation (Vellaisamy *et al.*, 2026).

Model	Launches/token	GPU util. (%)
Llama-3.2-1B	847.5	58.9
Llama-3.2-3B	1,536.9	67.6
OLMoE-1B/7B	9,305.3	15.5
Qwen1.5-MoE-A2.7B	6,695.1	27.7

buckets you ship (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). If launches dominate and shapes are bucket-static, graph capture is the low-risk win (NVIDIA, 2024; Chen, 2026a). If bytes/token dominate with moderate launch counts, invest in MegaKernel legality and compiler metadata (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Chen, 2020). Keep eager PyTorch baselines for debug, dynamic-shape research, and regression diffs—not for production decode pools once counters prove orchestration or residency debt (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026).

Each mode trades hardware utilization against software flexibility: eager maximizes flexibility at the cost of launches and bytes; graph mode trades dynamic control for launch amortization; MegaKernel mode trades compile-time complexity for residency (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b). Hardware teams should sign off on the trade-off explicitly—for example accepting static context buckets to unlock graph capture on Hopper, or accepting longer compile times to unlock XDNA static schedules (NVIDIA, 2024; AMD, 2024a; Davies *et al.*, 2025; SemiAnalysis, 2024).

High launches/token with moderate utilization (top rows) often respond to CUDA Graph replay first (NVIDIA, 2024; Chen, 2026a). MoE rows with extreme launches and low utilization need MegaKernel plus routing collapse—graphs alone cannot fix expert dispatch storms (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025). Always pair launch tables with bytes/token from the same trace; otherwise mode selection optimizes the wrong bottleneck (Chen, 2026a; SemiAnalysis, 2024; AMD, 2024b; Chen, 2020; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023).

Cross-hardware regression is mandatory: a mode that wins on H100 launch-bound cells may tie or lose on L4 memory-bound cells or XDNA static schedules (Chen, 2026a; AMD, 2024b; SemiAnalysis, 2024). Compiler IR should carry mode annotations so the same decoder lowers to eager, graph, or MegaKernel backends without rewriting model Python (Chen and YiRage Contributors, 2026; Chen,

Table 12.2: Execution mode selection heuristics for batch-1 decode (profile before committing).

Signal	Prefer	Avoid
High launches/token, static bucket	CUDA Graphs / frozen schedule	Unfused eager
High bytes/token, fusion legal	MegaKernel	Graph-only without fusion
Dynamic shape / debug	Eager baseline	Forced graph capture
NPU fleet	Static MegaKernel / AIE schedule	GPU-only CUDA graphs
Low R_{floor} , high DRAM	MegaKernel + bandwidth opts	Launch-only graph tuning

2020; Goodfellow *et al.*, 2016). Production reviewers should reject mode changes backed only by prefill FLOPs or isolated GEMM microbenchmarks (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Dao *et al.*, 2023).

Example 12.4.1. Rollout checklist: (1) profile eager bytes/token and launches/token at target buckets; (2) if launch-bound, ship graph replay and re-profile; (3) if bytes remain high, iterate MegaKernel legality with compiler metadata; (4) run GPU/CPU/NPU triplet regression before fleet promotion (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; AMD, 2024b; Chen, 2020; Davies *et al.*, 2025; NVIDIA, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016).

Fleet operators should log execution mode per deployment cell in telemetry—without that label, postmortems confuse graph wins on launch-bound SKUs with MegaKernel wins on bytes-bound SKUs (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024; AMD, 2024a; Dao *et al.*, 2022; Chen, 2020; NVIDIA, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).

Industrial rollout narrative. A typical maturity path starts with Hugging Face eager forwards in research, adds vLLM serving for KV and batching, captures CUDA Graphs on stable decode buckets, then invests in YiRage or hand-written MegaKernels once bytes/token plateaus (Kwon *et al.*, 2023; NVIDIA, 2024; Chen

and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). Skipping straight to MegaKernels without graph and eager baselines makes regressions unexplainable—teams cannot tell whether fusion failed legality or never beat orchestration (Chen, 2020; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). Conversely, stopping at graph replay when R_{floor} is already high leaves DRAM-bound decode on the table (Chen, 2026a; Vellaisamy *et al.*, 2026). Mode selection is therefore a *sequence* of profile gates, not a one-time architecture slide (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; NVIDIA, 2024; Kwon *et al.*, 2023; AMD, 2024b; Chen, 2020; Goodfellow *et al.*, 2016).

Disaggregated prefill/decode fleets compound the choice: prefill pools may remain eager or graph-captured on wide GEMMs while decode pools push MegaKernels for batch-1 latency (Davies *et al.*, 2025; Liu *et al.*, 2024; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Using prefill throughput to justify decode mode leads to systematic under-investment in launch and bytes counters on the decode side—the DeepSeek-style capacity skew exists because phases obey different rooflines (Davies *et al.*, 2025; Dao *et al.*, 2023; Chen, 2026a; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a,b; NVIDIA, 2024; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Dao *et al.*, 2022). Compiler teams should expose mode as an IR attribute so backend passes can reject illegal combinations early rather than at link time (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; NVIDIA, 2022; SemiAnalysis, 2024; NVIDIA, 2024; AMD, 2024b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; Liu *et al.*, 2024; Kwon *et al.*, 2023; Dao *et al.*, 2022; AMD, 2024a).

Hardware SLO reviews should include a “mode matrix”: rows are SKUs (Hopper, L4, XDNA, CPU), columns are context buckets, cells list chosen mode plus bytes/token and launches/token (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; AMD, 2024b,a; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). Empty cells mean unprofiled deployment—a common source of p99 surprises when fleets autoscale across instance types (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; SemiAnalysis, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022).

Decision inputs beyond launch and byte counters. Launches/token and bytes/token pick the bottleneck, but deployment picks the mode within the legal set. Compile-time budget, iteration velocity, maintainer skill, and fleet SKU spread all enter the decision: a team shipping one model on one GPU can afford a hand-written MegaKernel, while a fleet that recompiles nightly needs compiler-generated modes with fast legality checks (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025). The profile tells you which counter is binding; the operating context tells you which schedule you can sustain across releases (Vellaisamy *et al.*, 2026; Chen, 2026a). Record both dimensions in the decision record—otherwise a future reviewer sees only the chosen mode and cannot tell whether graph replay was skipped because bytes were binding or because capture engineering was not funded (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

Mode mixing inside a decode step. Real serving stacks rarely run one mode for an entire token. Sampling, speculative branches, MoE routing, and dynamic padding live naturally in eager hulls; stable attention and MLP regions run as captured or fused bodies; persistent kernels may own the KV update loop while per-token scheduling stays host-side (NVIDIA, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024). The compiler’s job is to make these seams explicit in the IR and to keep the boundary counters measurable on both sides (Chen and YiRage Contributors, 2026; Chen, 2020). A hybrid schedule that mixes modes per subgraph is a sign of maturity, not a hack, provided each region has a mode attribute and the profile can attribute latency to the right region (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2026a).

Operational gates and rollback. Mode promotion should follow the same gates as kernel promotion: non-regression on bytes/token, launches/token, and p99 latency at every shipped context bucket; a legalization report for fused regions; and a documented rollback path to the previous mode (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026). Rollback must be a runtime switch, not a redeploy, because fleet incidents will demand switching eager hulls or graph sets without rebuilding binaries (Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2020). Telemetry should tag the active mode and the reason it was chosen (profile-triggered, manual, or fallback), which is the only way postmortems can separate mode regressions from model or data regressions (Kwon *et al.*, 2023; Chen, 2026a).

Worked scenario walkthroughs. Consider three fleets. A dense Llama-3.2-class model on launch-bound H100 cells with static buckets starts with graph replay, measures the launch drop, then fuses only if bytes/token remain above the analytic floor (Vellaisamy *et al.*, 2026; Chen, 2026a). A MoE model whose eager profile shows expert routing and gather-scatter dominating launches skips attention-only fusion and invests in routing collapse plus a graph shell, because attention fusion alone leaves 6k–9k launches/token untouched (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024). An XDNA fleet compiles the same decoder IR to static dataflow from day one; its mode debate is about DMA edge counts and tile-SRAM legality rather than host dispatch, so the graph-mode discussion is replaced by a schedule-legality review (AMD, 2024b,a; Chen and YiRage Contributors, 2026). Each scenario produces a different mode matrix even though the underlying counters are identical—which is exactly why the matrix rows must be SKU and context bucket, not model name alone (Davies *et al.*, 2025; Chen, 2026a).

Re-profile cadence and upgrade triggers. Mode decisions rot on a schedule: driver, CUDA, framework, model, context limit, and fleet SKU changes can each move the launch/bytes balance that justified the current mode (NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a). Define explicit triggers that reopen the mode gate—every dependency upgrade above a pinned version, every new context bucket, every model architecture change, and every capacity shift across SKU types (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). Between triggers, a lightweight nightly job should compare current counters against the mode matrix and alert when a cell drifts beyond tolerance; the alert is not an automatic mode flip but a reason to run the full profile gate (Chen, 2020; Kwon *et al.*, 2023). This cadence converts mode selection from a one-time architecture slide into a maintained operational contract, which is the only way the escalation path survives team turnover and framework churn (Chen, 2026a; Vellaisamy *et al.*, 2026).

12.5 Key Takeaways

- **Three modes, one metric ledger.** Eager, graph, and MegaKernel differ in launches/token *and* bytes/token—optimize the counter your SKU actually bounds (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025).
- **Eager is baseline, not destiny.** Framework dispatch is correct for debug; production decode needs profiled migration to graph or MegaKernel (Kwon *et al.*, 2023; NVIDIA, 2024).

- **Graphs fix host tax.** Memory-Floor reports $1.259\times$ on H100 when launch overhead dominates; graphs do not replace fusion (Chen, 2026a; NVIDIA, 2024).
- **MegaKernels fix residency.** Fusion legality on Hopper/XDNA/CPU determines whether bytes/token fall; YiRage-style checks belong in compile time (Chen and YiRage Contributors, 2026; AMD, 2024b; NVIDIA, 2022).
- **Profile-first selection.** Use Table 12.2; never pick mode from prefill FLOPs alone (Dao *et al.*, 2023; Vellaisamy *et al.*, 2026).
- **Multi-hardware parity.** Identical IR must lower per target residency tables—GPU graph capture $\neq$ NPU static schedule (Chen, 2020; AMD, 2024a; Goodfellow *et al.*, 2016).

Practitioner mantra. Profile end-to-end decode first. Collapse launches with graphs second. Collapse bytes with MegaKernels third. Re-profile on every bucket, SKU, and framework upgrade (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2020; AMD, 2024b; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; AMD, 2024a). Teach reviewers to reject mode PRs that show only one counter moving (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; SemiAnalysis, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022). Post the mantra next to decode dashboards so graph and MegaKernel investments stay tied to measured goodput (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2020; AMD, 2024b; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; AMD, 2024a).

12.6 Conclusion

Three kernel execution modes translate Chapter 1 counters into concrete schedules: eager exposes every framework boundary; CUDA Graphs collapse launches when shapes are stable; MegaKernels collapse HBM traffic when fusion is legal (Vellaisamy *et al.*, 2026; NVIDIA, 2024; Chen, 2026a; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022). None of the modes substitutes for profiling bytes/token and

launches/token on the SKU and context buckets you operate (Davies *et al.*, 2025; SemiAnalysis, 2024).

Mode choice is reversible across releases: fleets can ship graph replay while MegaKernel plans mature in CI, as long as telemetry tags the active mode and counters prove non-regression (NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022). Treat execution mode as a tunable deployment parameter guarded by profile gates, not as a one-time rewrite (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b; NVIDIA, 2024; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; AMD, 2024a). The escalation path—eager baseline, graph replay, MegaKernel fusion—should appear explicitly in your decode runbooks and compiler IR schemas (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; AMD, 2024b; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; AMD, 2024a).

Chapter 13 moves from schedule choice to compiler theory—how IR, passes, and lowering pipelines decide which mode is even emit-able on GPU, CPU, and NPU backends (Chen, 2020; Goodfellow *et al.*, 2016; AMD, 2024b; Liu *et al.*, 2024; Kwon *et al.*, 2023; AMD, 2024a; Dao *et al.*, 2023). Carry the mode-selection table forward; later MegaKernel chapters assume you can justify eager vs graph vs fused emission with measured decode goodput, not architecture slides alone (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a).

Readers should leave this chapter able to defend a mode choice in a design review: cite TaxBreak launches/token when proposing graphs; cite Memory-Floor R_{floor} when graphs underperform; cite YiRage legality reports when proposing MegaKernels (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2020; AMD, 2024b; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; AMD, 2024a; NVIDIA, 2022). The three modes are not competitors—they are layers in a profile-driven escalation path toward decode goodput (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a,b; NVIDIA, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; SemiAnalysis, 2024; NVIDIA, 2022). Document that escalation in compiler schemas so the next maintainer inherits profile discipline, not folklore (Chen and

YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; AMD, 2024b; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; AMD, 2024a).

Chapter 13

Core Theory: AI Compiler Optimization Principles

What makes an AI compiler “correct” for production decode—pass ordering that looks elegant on a whiteboard, or schedules that measurably cut bytes/token and launches/token on the SKU you ship? Part VI opens with *Core Theory*: the optimization principles that sit *below* Chapter 12 execution modes and *above* dialect-specific backends (MLIR, Glow, TVM) (Chen, 2020; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). Every section binds IR transforms to residency on GPU, CPU, and NPU—not abstract big-O (Davies *et al.*, 2025; AMD, 2024b,a; Goodfellow *et al.*, 2016).

TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). Memory-Floor reports $1.259\times$ median speedup on H100 from CUDA Graph replay alone at BS=1, ctx=2048—orchestration relief that tiling and fusion must extend into bytes, not replace (Chen, 2026a; NVIDIA, 2024). Compiler theory is how those counters move together: tile so data stays on-chip, fuse so boundaries disappear, layout so hardware reads coalesce, parallelize so batch-1 decode still saturates memory paths, and split backends so one IR lowers per target without forking model code (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2023).

This chapter is deliberately *theory-first*: no MLIR syntax yet (Chapter 14), no manual CUDA (Part VII). The goal is a shared vocabulary for pass designers—when someone proposes a new fusion rule or tile heuristic, Table 13.1 names which counter must move and which prior chapter constraint applies (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025). Industrial teams that skip

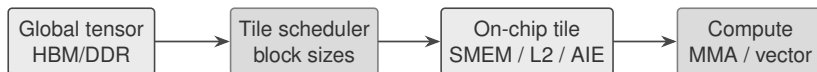


Figure 13.1: Tiling theory: map global decode tensors into on-chip tiles before compute.

this layer debate dialect patches for weeks while bytes/token flatlines because layout and tiling were never co-designed (Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).

Fregly-style goodput analysis applies throughout: every transform is judged on batch-1 decode at representative context buckets, not on square GEMM microbenchmarks or prefill-only traces (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026). The five sections mirror how production compilers order concerns—tile shapes constrain fusion legality; layout determines whether fused regions can load coalesced data; parallel axes decide whether extra blocks help bandwidth; backend split decides which target owns each residency class (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016).

Readers coming from classic optimizing-compiler courses should expect different objectives: decode LLM compilers optimize *moved bytes* and *launches* under numerics constraints, not loop nest locality alone (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). When in doubt, re-run the Chapter 1 counter worksheet on the same model checkpoint before and after a pass change—theory without measurement is slideware (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016).

13.1 tiling theory

Tiling theory asks which sub-blocks of weights, activations, and KV cache fit in registers, shared memory, L2, or NPU tile SRAM for each decode step (Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022). FlashAttention is the canonical IO-aware tiling result for attention: recomputation and block-wise softmax avoid materializing P in HBM (Dao *et al.*, 2022, 2023). The compiler question is general—every matmul, norm, and MLP in decode needs a tile shape that respects bandwidth and capacity, not only MMA convenience (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026).

On Hopper, tiling couples to TMA: async copies move KV tiles while MMA

consumes prior tiles (NVIDIA, 2022; SemiAnalysis, 2024). Wrong tile sizes leave TMA underutilized or spill partial outputs to HBM, showing up as bytes/token regression versus a smaller but legal tile (Chen, 2026a; Chen and YiRage Contributors, 2026). On XDNA/AIE, tiling is static: DMA chains and tile slots are fixed at compile time; “dynamic” GPU tiles do not lower verbatim (AMD, 2024b,a; Chen, 2020). CPU decode tiling is cache blocking—NUMA-local KV and blocked GEMMs keep lines hot across norm-matmul chains (Goodfellow *et al.*, 2016; Kwon *et al.*, 2023).

Analytic tile sizing. A first-principles decode tile budget starts from hidden size d , head count h , KV dtype bytes b_{kv} , and context length L : bytes moved per token scale with reading prior KV plus writing new KV slots (Kwon *et al.*, 2023; Dao *et al.*, 2022; Davies *et al.*, 2025). Tile width along L trades SMEM for trip count; tile width along d trades MMA efficiency for reload width (Dao *et al.*, 2023; SemiAnalysis, 2024; Chen, 2026a). Compilers should expose these trade-offs in IR annotations so autotuners search a *legal* polytope rather than all factorizations of L (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). When analytic estimates disagree with profiles by more than a few percent, trust profiles—roofline models omit allocator, paging, and framework overhead (Chen, 2026a; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Compiler passes should emit tile metadata in IR (tile sizes, double-buffer depth, residency class) so backends reject illegal schedules before codegen (Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024). Autotuners that search FLOPs-only tile shapes without bytes/token feedback recreate prefill wins and decode losses (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a). Profile each candidate tile on BS=1 decode cells, not only square GEMM microbenchmarks (Vellaisamy *et al.*, 2026; SemiAnalysis, 2024).

Prefill vs decode tiling. Prefill benefits from large token-parallel tiles that amortize weight loads across many queries; decode at BS=1 reuses weights once per step and instead fights KV reload along the sequence axis (Dao *et al.*, 2022, 2023; Davies *et al.*, 2025). A tile policy tuned on prefill shapes often selects block sizes that maximize MMA occupancy while increasing KV trips—exactly the regression Memory-Floor documents when bytes/token stay high despite faster attention kernels (Chen, 2026a; Vellaisamy *et al.*, 2026). Compiler tile search must therefore tag phases explicitly: prefill buckets, decode buckets, and speculative-draft buckets when applicable (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

Double buffering and pipeline depth. Hopper pipelines pair TMA loads with MMA compute; tile theory includes how many stages fit in shared memory without spilling partial softmax or norm statistics (NVIDIA, 2022; SemiAnalysis,

2024; Dao *et al.*, 2022). Shallow pipelines underutilize memory bandwidth; deep pipelines overflow SMEM and force silent HBM spill (Chen, 2026a; Chen and YiRage Contributors, 2026). On AIE, pipeline depth is fixed at compile time—there is no runtime knob to “add another buffer” when decode context grows (AMD, 2024b,a; Chen, 2020).

Autotuning discipline. TVM-style autotuners historically optimized for peak FLOPs on convolution and dense GEMM shapes (Goodfellow *et al.*, 2016; Chen, 2020). LLM decode autotuning must swap the objective: minimize bytes/token subject to numerics and legality constraints from Chapters 10–11 (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Every autotune candidate should log tile dimensions *and* measured HBM traffic per token—otherwise teams ship tiles that win on roofline FLOPs charts and lose on fleet p99 (Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024).

Multi-hardware delta. Identical FLOP count with different tile policies can differ by an order of magnitude in DDR traffic on XDNA versus TMA-backed Hopper tiles (AMD, 2024b; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Example 13.1.1. Decode cell (BS=1, ctx=2048): shrinking attention block size to fit shared memory may *increase* loop iterations and bytes/token if KV reloads dominate—tile theory must co-optimize residency and trip count (Dao *et al.*, 2022; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Industrial tile review checklists include: SMEM/TMA budget worksheet, KV bytes per trip, prefill vs decode bucket tags, and autotune objective function (SemiAnalysis, 2024; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025). Teams that approve tile changes without decode profiles often ship kernels that win on nsys “compute” rows and lose on “dram” rows (Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022; AMD, 2024b; Goodfellow *et al.*, 2016).

Residency classes before tile shapes. The first tiling decision is not a number but a class: every tensor in the decode graph must be classified as register-resident, shared-memory or tile-SRAM resident, L2-resident, or global-memory streamed (SemiAnalysis, 2024; NVIDIA, 2022; Chen, 2026a). Only after classification does the pass choose concrete tile widths that satisfy the class boundaries; choosing MMA-preferred widths first and hoping residency follows is how fused graphs silently spill (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). The classification

itself is hardware-derived—Hopper exposes shared memory and L2 as separate budgets, XDNA fixes tile addresses at compile time, and CPU treats cache lines as the residency unit (AMD, 2024b,a; Goodfellow *et al.*, 2016). IR should carry the class as an attribute so a later pass cannot demote a register tensor to shared memory without a legality review (Chen, 2020; Davies *et al.*, 2025).

Tile sweep methodology. Tile autotuning becomes reproducible when the search is bounded by the residency classification first and measured on decode counters second (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a). Generate candidates by varying one axis at a time—KV-block width, head-block width, double-buffer depth—while holding the other axes at the analytic default; log bytes/token and ms/token for every candidate at the same context bucket (Dao *et al.*, 2023; SemiAnalysis, 2024). Narrow the sweep around the best bytes/token point, not the best SM-utilization point, because batch-1 decode rewards fewer and wider memory transactions (Chen, 2026a; Vellaisamy *et al.*, 2026). Keep the full sweep table in CI artifacts so a later chip-model update can re-run the same procedure without re-deriving the methodology (Chen and YiRage Contributors, 2026; Chen, 2020).

Decode-specific tile invariants. Three invariants separate decode tiling from prefill tiling. First, weight tiles are read once per token, so they should stream through the largest reusable footprint rather than be re-tiled around a batch of queries (Kwon *et al.*, 2023; Davies *et al.*, 2025). Second, KV tiles are read on every new token, so their width along the sequence axis directly sets bytes/token and the shared-memory high-water (Dao *et al.*, 2022, 2023). Third, softmax and norm statistics must remain resident across tile boundaries; a tile that invalidates their lifetime is illegal regardless of how fast it loads (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). These invariants belong in the compiler test suite as negative tests: a pass that proposes an illegal decode tile should fail CI with the invariant name (Chen, 2026a, 2020).

Capacity ledger as review artifact. Every tiling PR should carry a capacity ledger: tensor names, assigned residency classes, byte estimates at the shipped context bucket, and the measured bytes/token from the decode profile (SemiAnalysis, 2024; Chen, 2026a; Chen and YiRage Contributors, 2026). The ledger turns an abstract tile change into a testable claim—if the ledger says an activation is shared-memory resident but the profile shows DRAM traffic growing with that tensor’s size, the claim is falsified in review (Vellaisamy *et al.*, 2026; Davies *et al.*,



Figure 13.2: Fusion theory: merge producer–consumer ops when residency allows; else spill.

2025). Compiler-generated plans should emit the ledger automatically from IR metadata so it cannot go stale (Chen, 2020; Chen and YiRage Contributors, 2026). Reviewers should reject tile PRs whose ledger and profile disagree, because that disagreement is the earliest symptom of silent spill (Chen, 2026a; Vellaisamy *et al.*, 2026).

13.2 fusion theory

Fusion theory governs when operator boundaries may disappear into MegaKernels or fused subgraphs (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Each unfused boundary exports activations to HBM; Chapter 1 counted the result as launches/token and bytes/token (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Kwon *et al.*, 2023). Fusion is not “always on”—it is constrained by register/shared-memory budgets, softmax carrier lifetimes (Chapter 10), and graph capture legality (Chapter 12) (Dao *et al.*, 2022; NVIDIA, 2024; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024).

Greedy fusion (merge any adjacent ops) often illegalizes attention–MLP seams: online softmax state (m, ℓ, o) must survive inside the fused region or bytes explode silently (Dao *et al.*, 2022, 2023; Chen, 2026a). Compiler cost models should penalize HBM round trips and launches, not only instruction count (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). YiRage-style legality checks encode Chapter 2 iron rules as IR predicates so fusion plans fail in compile time rather than in fleet A/B (Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a).

Fusion boundaries as contracts. Treat each candidate fusion edge as a contract between producer and consumer: the producer promises not to materialize to HBM; the consumer promises register/SMEM budget to consume in-place (Chen and YiRage Contributors, 2026; Chen, 2026a; Dao *et al.*, 2022). When either side breaks the contract—for example LayerNorm statistics exported because the following GEMM needs a different layout—fusion must split (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Graph capture (Chapter 12) records whatever fusion boundaries exist; it cannot invent new ones (NVIDIA, 2024; Chen, 2026a).

Horizontal vs vertical fusion. *Horizontal* fusion merges operators at the same tensor rank (norm + linear + activation). *Vertical* fusion merges stages along the decode pipeline (attention block + MLP block into one MegaKernel region) (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Vertical fusion delivers the largest bytes/token wins but hits legality first; horizontal fusion is easier but may leave launches/token high (Vellaisamy *et al.*, 2026; NVIDIA, 2024; Davies *et al.*, 2025). Compiler heuristics should attempt vertical plans only after horizontal plans pass carrier and layout checks (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024).

Fusion and execution modes. Chapter 12 maps eager, graph, and MegaKernel modes to launch and byte counters; fusion theory explains *which* boundaries each mode can erase (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Graph capture records existing fusion; MegaKernel mode requires fusion plans that satisfy carrier legality (Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; NVIDIA, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024). Teams stuck in graph mode with high bytes/token are blocked by fusion theory, not by CUDA Graph APIs (Chen, 2026a; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Document fusion-closed regions in runbooks so on-call engineers know whether to tune capture buckets or rewrite fusion plans (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; SemiAnalysis, 2024).

Cost model sketch. A minimal fusion cost model scores each plan as $\alpha \cdot \text{bytes} + \beta \cdot \text{launches} + \gamma \cdot \text{spill_penalty}$, with spill penalty derived from residency metadata rather than FLOPs (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). Tuning (α, β, γ) per SKU is acceptable; tuning without bytes/token telemetry is not (SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Multi-hardware delta. Hopper may fuse QKV+softmax+proj when shared memory permits; XDNA may require separate static subgraphs linked by DMA; CPU fusion is often cache-scope fusion across norm-linear chains (NVIDIA, 2022; AMD, 2024b; Goodfellow *et al.*, 2016; Chen, 2020; Kwon *et al.*, 2023).

Example 13.2.1. Fusing LayerNorm with the following GEMM removes one full-width activation write/read per layer per token—often larger decode win than

tuning MMA tile width alone (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

MoE layers add gather/scatter fusion constraints: expert routing may remain host-visible while MLP experts fuse internally—fusion theory must specify *which* subgraph is fusion-closed (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2020; AMD, 2024b; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2022; AMD, 2024a; Davies *et al.*, 2025; Chen, 2026a).

Fusion regression tests should diff IR fusion boundaries against a golden eager graph: any new HBM edge between previously fused ops is a release blocker unless documented as intentional (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; NVIDIA, 2024; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). This is how compiler teams prevent “silent unfusion” when dialect refactors reorder passes (Chen, 2020; Chen and YiRage Contributors, 2026; Liu *et al.*, 2024; AMD, 2024a,b).

Fusion plans as IR contracts. A fusion plan should be representable as a first-class IR object: the fused region, the boundaries it removes, the buffers it keeps on-chip, and the numerical assumptions it makes (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022). Without that object, a pass that merges two ops and a pass that merely reorders them look identical in review, and silent unfusion can hide for releases (Chen, 2026a; Vellaisamy *et al.*, 2026). The contract should be diffable in code review: a PR that changes fusion boundaries changes the contract diff, and the byte sheet must move with it (Davies *et al.*, 2025; Chen, 2020). Compilers that encode contracts this way also enable rollback at the region level instead of at the whole-kernel level (Chen and YiRage Contributors, 2026; Chen, 2026a).

Layout-changing producers. Many fusion opportunities die at layout conversions rather than at compute: a producer that emits a transposed or interleaved activation forces the consumer to either reshape in place or load strided (SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Davies *et al.*, 2025). Fusion theory must therefore include a layout negotiation step where producer and consumer agree on one layout before the boundary disappears (Chen, 2020; Chen and YiRage Contributors, 2026). When no common layout exists within the on-chip budget, splitting the fusion is cheaper than inserting an HBM transpose, but the split must be explicit in the contract so its bytes are counted (Chen, 2026a; Vellaisamy *et al.*, 2026). Pass ordering that runs layout legalization before fusion search prevents

these negotiations from happening as emergency rewrites at codegen (Chen, 2020; AMD, 2024b).

Fusion discovery order. Production pass pipelines should discover fusion in a fixed order: elementwise producers into their dominant consumers first, norm and scale operators into the GEMMs that follow, attention seams only after online-carrier legality is proven, and MoE routing last because it changes control flow (Dao *et al.*, 2022; Liu *et al.*, 2024; Chen, 2020). Each stage should emit a legality report that names the buffers preserved and the counters expected to move (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a). This ordering prevents a greedy pass from consuming a seam that later, more valuable fusion needs for carrier residency (Dao *et al.*, 2022; Davies *et al.*, 2025). It also gives engineers a vocabulary for regressions: a fusion that disappears between releases can be traced to the stage that owns that seam (Chen and YiRage Contributors, 2026; Chen, 2020).

Spill accounting discipline. Spills are not an error if they are counted; they are a bug when they are invisible (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Fusion passes should emit a spill event every time a planned on-chip buffer falls back to global memory, with the tensor name and byte width attached (Chen and YiRage Contributors, 2026; Chen, 2020). Production dashboards should alert when spill bytes grow across compiler versions, because a silent unfusion and an over-eager new fusion produce the same spill signature (Chen, 2026a; Vellaisamy *et al.*, 2026). Reviewers should ask fusion PRs to show the before-and-after spill ledger; a PR that reports only kernel count reductions has not yet demonstrated a bytes win (Davies *et al.*, 2025; SemiAnalysis, 2024).

13.3 layout theory

Layout theory decides how tensors are stored and traversed: contiguous vs blocked layouts, KV cache page tables, head/interleaved formats, and alignment for TMA/vector loads (Kwon *et al.*, 2023; Davies *et al.*, 2025; SemiAnalysis, 2024; NVIDIA, 2022). A perfect fusion plan fails if layout forces strided HBM access—bytes/token rise while kernels look “fused” in the IR (Chen, 2026a; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2020).

PagedAttention (vLLM) is layout engineering for KV capacity: non-contiguous physical pages with logical continuity reduce fragmentation without changing model

math (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Compilers must treat page table indirection as a first-class layout transform, not an runtime hack outside IR (Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024). On NPU backends, layout often *is* the schedule—static DMA descriptors replace pointer chasing (AMD, 2024b,a; Goodfellow *et al.*, 2016).

Phase-specific layouts. Some teams store KV in a layout optimized for prefill writes (wide vector stores along sequence) and transposes for decode reads (head-local access) (Kwon *et al.*, 2023; Dao *et al.*, 2023; Davies *et al.*, 2025). Transpose costs bytes; the compiler must charge them in the cost model and prefer layouts that avoid per-token transposes when ctx grows (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; Liu *et al.*, 2024; Dao *et al.*, 2022; NVIDIA, 2022, 2024; Dao *et al.*, 2023). When a layout pass cannot prove decode legality, split layouts per phase rather than forcing one format (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Layout passes interact with tiling: blocked weights pair with blocked GEMM tiles; NHWC vs NCHW choices change vectorization on CPU and GPU differently (Goodfellow *et al.*, 2016; Chen, 2020; SemiAnalysis, 2024; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; AMD, 2024b; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024; AMD, 2024a). Decode compilers should log layout transforms in pass metadata so regressions trace to reorder vs fusion vs tile changes (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024).

KV layout and paging. Decode throughput scales with how cheaply each new token reads historical KV (Kwon *et al.*, 2023; Dao *et al.*, 2023; Davies *et al.*, 2025). Contiguous KV is simple but fragments GPU memory at long context; paged layouts add indirection cost that must be amortized by fewer reallocations and higher batch utilization (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Compiler layout passes should lower page-table updates into explicit IR ops so backends can fuse gather/scatter with attention tiles where legal (Chen and YiRage Contributors, 2026; Chen, 2020).

Weight layout and quantization. INT4/FP8 weight layouts (block scales, interleaved scales) are layout transforms before they are “quantization features” (Liu *et al.*, 2024; Davies *et al.*, 2025; SemiAnalysis, 2024). A fusion plan that assumes FP16 contiguous weights will fail silently when weights are block-scaled—

loads become gather-bound and bytes/token rise (Chen, 2026a; Chen and YiRage Contributors, 2026). Layout legalization must run *before* fusion search, not as a post-pass patch (Chen, 2020; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Alignment and vector width. TMA and vector units prefer aligned 128-byte lines; misaligned KV or activation tensors force uncoalesced access patterns that no tile size fixes (SemiAnalysis, 2024; NVIDIA, 2022; Goodfellow *et al.*, 2016). CPU decode benefits from cache-line alignment across head dimensions; NPU backends often require fixed stride multiples baked into DMA descriptors (AMD, 2024b,a; Chen, 2020). Layout theory therefore includes alignment as a first-class constraint in IR, not a codegen afterthought (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Multi-hardware delta. GPU layouts optimize for coalesced 128B lines; CPU layouts optimize for cache line reuse across batch-1 thin GEMMs; NPU layouts fix byte addresses at compile time (NVIDIA, 2022; AMD, 2024b; Goodfellow *et al.*, 2016; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024; AMD, 2024a).

Example 13.3.1. Switching KV layout to improve prefill vectorization can regress decode if batch-1 attention loads become gather-bound—layout theory requires phase-specific profiling (Kwon *et al.*, 2023; Dao *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; Dao *et al.*, 2022; NVIDIA, 2022, 2024; AMD, 2024a).

Layout CI should snapshot tensor strides and alignment for KV, QKV weights, and MLP weights at each context bucket (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; SemiAnalysis, 2024; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; AMD, 2024b,a). Stride drift without corresponding fusion changes is a common source of “mysterious” bytes/token regressions across compiler versions (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025).

Layout canonicalization and legality. Every phase should have a canonical layout derived from its access pattern, with explicit ops for any conversion rather than hidden runtime transposes (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020). Legality in layout terms means: the chosen strides are

aligned for the target’s vector or DMA width, the tensor fits its assigned residency class, and no consumer requires an implicit reshape (NVIDIA, 2022; AMD, 2024a; Davies *et al.*, 2025). A layout pass that cannot prove these three properties should refuse to emit rather than insert a conversion that doubles bytes/token (Chen, 2026a; Vellaisamy *et al.*, 2026). CI should snapshot the canonical layout of KV, QKV weights, and MLP weights per context bucket so drift appears as a first-class regression (Chen, 2020; Goodfellow *et al.*, 2016).

Paged KV layout decisions. Paging changes the layout question from “where is the next KV row” to “which page does the block walk touch,” and the answer determines whether attention loads coalesce (Kwon *et al.*, 2023; Dao *et al.*, 2023; Davies *et al.*, 2025). Page size should be chosen against the head dimension and the on-chip staging width: pages that are too small scatter the block walk, while pages that are too large waste capacity on partial occupancy (SemiAnalysis, 2024; Chen, 2026a). The compiler should expose the page-table walk as an explicit IR pattern so fusion can keep gather and attention in one region where the target allows it (Chen and YiRage Contributors, 2026; Chen, 2020). On XDNA, page indirection may need to become a static descriptor list because runtime pointer chasing is not a legal DMA pattern (AMD, 2024b,a).

Weight layout conversion timing. Quantized and block-scaled weights should be converted once at load or export time, never per decode step, and the conversion cost must appear in the same ledger as activation traffic (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2026a). A compiler that treats weight conversion as a free pre-pass hides a real cost when models are swapped at runtime or when context buckets share one weight copy (Chen, 2020; Chen and YiRage Contributors, 2026). Layout legalization should verify that every quantized tensor has a single canonical runtime layout and that no fusion plan inserts a second conversion (Davies *et al.*, 2025; Chen, 2020). Runtime weight swaps are where layout bugs surface as mysteriously slow first tokens; the fix belongs in the pass pipeline, not in cache tuning (Vellaisamy *et al.*, 2026; Chen, 2026a).

13.4 parallel scheduling

Parallel scheduling maps work to SMs, cores, or AIE arrays—including Flash-Decoding’s parallelization over KV sequence length at batch-1 (Dao *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Prefill parallelizes over tokens; decode parallelizes over heads, KV blocks, or expert routes while issuing one new token per

step (Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; Dao *et al.*, 2022; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; AMD, 2024b; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024; AMD, 2024a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

The failure mode is “parallelism for idle hardware”: launching extra blocks that contend for HBM bandwidth without increasing useful bytes delivered per ms/token (Chen, 2026a; SemiAnalysis, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Schedulers should be bandwidth-aware—when R_{floor} is high, additional parallel splits hurt (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024; SemiAnalysis, 2024; Davies *et al.*, 2025). When launches/token dominate (TaxBreak regime), parallel capture inside CUDA Graphs or fused kernels may help even at moderate bandwidth (Vellaisamy *et al.*, 2026; NVIDIA, 2024; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024b; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; AMD, 2024a).

Prefill/decode scheduler split. Prefill schedulers maximize token-parallel work; decode schedulers maximize memory efficiency per new token (Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; AMD, 2024b,a; SemiAnalysis, 2024; Liu *et al.*, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2023). Sharing one scheduler heuristic across phases is a common source of decode regressions after prefill optimizations (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Compiler IR should expose parallel axes (batch, head, KV block, expert) so backends legalize splits per target (Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a,b; NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024). Static NPU schedules often fix parallelism at compile time; GPU schedules retain dynamic grid sizing within capture buckets (AMD, 2024b; NVIDIA, 2024; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; AMD, 2024a).

Occupancy vs bandwidth. GPU textbooks emphasize occupancy; decode compilers must ask whether extra warps increase *useful* bytes delivered per cycle or

merely contend for the same HBM port (Chen, 2026a; SemiAnalysis, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). When R_{floor} is near unity, the scheduler should prefer fewer, wider memory transactions (better fusion, better layout) over more parallel blocks (Chen, 2026a; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026).

Wave quantization and tail effects. Parallel splits that do not divide sequence length evenly leave tail waves with low utilization—common when KV blocks do not align with context length (Dao *et al.*, 2023; Kwon *et al.*, 2023; Davies *et al.*, 2025). Compiler scheduling should pad or bucket context lengths explicitly so tail penalties appear in CI, not only at customer $\text{ctx}=8192$ (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020).

MoE and expert parallelism. MoE layers introduce a routing axis: experts parallelize differently from attention heads (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). Schedulers must avoid launching all experts when only top- k activate—parallel theory includes *conditional* parallelism gated on router output, which complicates graph capture and static NPU schedules alike (Vellaisamy *et al.*, 2026; NVIDIA, 2024; AMD, 2024a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025).

Multi-hardware delta. GPU parallel scheduling uses grids/warps; CPU uses thread pools with cache affinity; NPU uses spatial arrays with fixed slot counts (NVIDIA, 2022; AMD, 2024b; Goodfellow *et al.*, 2016; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2024; AMD, 2024a).

Example 13.4.1. Flash-Decoding reports large speedups at long context by parallelizing KV—the scheduler must still respect on-chip softmax carrier residency from Chapter 10 or parallel blocks reintroduce HBM traffic (Dao *et al.*, 2023, 2022; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2020; AMD, 2024b; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2022, 2024; AMD, 2024a).

Parallel scheduling reviews should attach Nsight “memory throughput” and “SM busy” together—high SM busy with flat bytes/token indicates useless parallelism (Chen, 2026a; SemiAnalysis, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016; AMD, 2024b,a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2022, 2024). Decode schedulers that only maximize occupancy violate the bandwidth-aware

principle this section states (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Ranking parallel axes for decode. Decode exposes four parallel axes—batch rows, query heads, KV blocks, and expert routes—and the scheduler should rank them by what each axis costs in merged state (Dao *et al.*, 2023; Kwon *et al.*, 2023; Liu *et al.*, 2024). Parallelizing over query heads is cheap until shared scores or shared KV demand a merge; parallelizing over KV blocks always ends in an online-softmax merge; parallelizing over batch rows is the only axis that adds independent work without merge traffic (Dao *et al.*, 2022; Davies *et al.*, 2025). At batch-1, batch rows vanish, which pushes decode toward head and KV parallelism and makes the merge tree a first-class scheduling cost (Vellaisamy *et al.*, 2026; Chen, 2026a). The axis ranking should be recomputed per SKU because merge costs are residency costs, and residency tables differ by target (AMD, 2024b; Chen and YiRage Contributors, 2026; Chen, 2020).

Bandwidth-saturation test for any split. Before merging a parallelization change, run the saturation test: double the split count while holding the algorithm fixed and observe whether bytes/token stays flat (Chen, 2026a; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026). If doubling splits raises bytes/token, the extra blocks are paying for their own reloads; if ms/token drops while bytes stay flat, the added parallelism is hiding latency productively (Davies *et al.*, 2025; Chen, 2026a). The same test applies to capture groups inside graphs: a graph that launches more blocks than the memory system can feed merely serializes behind bandwidth (NVIDIA, 2024; Vellaisamy *et al.*, 2026). Publishing the saturation curve with each kernel gives reviewers an objective reason to accept or reject a split count (Chen and YiRage Contributors, 2026; Chen, 2020).

Static versus dynamic parallelism. GPU schedulers can choose grid sizes at launch within capture buckets, while XDNA/AIE fixes the spatial mapping at compile time and CPU leaves parallelism to the runtime thread pool (NVIDIA, 2024; AMD, 2024a; Goodfellow *et al.*, 2016). Compiler IR should annotate which axes are static and which are dynamic so the same parallel plan can be legalized per backend (Chen and YiRage Contributors, 2026; Chen, 2020). A dynamic axis on GPU that becomes static on NPU changes tail behavior: the static schedule must include worst-case splits because it cannot adapt to short context (AMD, 2024b,a). Bucketing context lengths is therefore not just a graph-capture concern; it is the mechanism that keeps static schedules from wasting array capacity on tail

waves (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026).

Merge cost as a schedule term. Every split that produces partial state must pay a merge, and the schedule should treat merge bytes and merge latency as explicit costs rather than epilogue noise (Dao *et al.*, 2023; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). When split count doubles, merge state roughly doubles with it, so the bandwidth-saturation test must measure the merge stage separately from the compute stage (Chen, 2026a; Dao *et al.*, 2022). Compilers should annotate split boundaries with their merge tree so autotuners can compare one large split against two smaller ones on equal terms (Chen and YiRage Contributors, 2026; Chen, 2020). A merge that stays on-chip is nearly free; a merge that touches global memory can negate the split’s entire benefit (SemiAnalysis, 2024; Chen, 2026a).

13.5 GPU CPU NPU split

GPU CPU NPU split is how one decoder IR lowers to CUDA/HIP, CPU oneDNN/OpenMP paths, and XDNA/AIE static dataflow without rewriting model Python (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024). Heterogeneous fleets fail when the GPU schedule is copied to NPU—illegal DMA edges and tile overflows appear as silent CPU fallbacks or wrong results (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024).

Backend split should share high-level passes (fusion legality, layout, tiling metadata) and fork only at emit (Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024b,a; NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024). Triplet regression—same model, same context buckets, GPU vs CPU vs NPU—is the acceptance test for split correctness (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024). Metrics remain bytes/token and launches/token per target; FLOPs parity is irrelevant (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow

et al., 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024).

Version skew. GPU and NPU toolchains release on different cadences; shared IR must pin legality rules so a middle-end upgrade cannot silently break NPU emit while GPU CI stays green (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a,b; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024).

CPU backends often serve hybrid pipelines: dynamic shapes, sampling, and small ops stay on host while GPU/NPU run fused decode cores (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b,a; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024). NPU backends require upfront static shape buckets; GPU backends tolerate bucketed graph capture (NVIDIA, 2024; AMD, 2024a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022). Document which ops each backend owns so mode selection (Chapter 12) maps cleanly to emit paths (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; AMD, 2024b,a; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022).

Shared middle-end, forked back-end. The durable pattern is a shared middle-end (fusion/layout/tiling metadata) and forked backends that interpret residency classes per target (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016). Glow and TVM historically split early; LLM decode stacks should delay fork until after decode-specific legality checks (Chen, 2020; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Fallback paths. When NPU legality fails, frameworks silently fall back to CPU loops—correct but catastrophically slow (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Production compilers must treat fallback as an explicit compile error unless a documented degraded mode is enabled (Chen, 2020; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026). Triplet CI should fail when GPU bytes/token improve but NPU emits empty kernels (Davies *et al.*, 2025; AMD, 2024a,b).

Heterogeneous single-node pipelines. Some deployments run attention on GPU and MLP on NPU, or prefill on GPU and decode on CPU (Davies *et al.*, 2025;

Liu *et al.*, 2024; Kwon *et al.*, 2023). Split theory then includes *partition* passes that cut the graph at device boundaries with explicit copy ops whose bytes count toward goodput (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Hidden host-device copies dominate when partition points are chosen for convenience rather than residency (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Multi-hardware delta. GPU emphasizes TMA+MMA and graphs; CPU emphasizes cache blocking and NUMA; NPU emphasizes static DMA+tiles—same IR, three residency models (NVIDIA, 2022; AMD, 2024b; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2024; AMD, 2024a).

Example 13.5.1. A fusion plan legal on Hopper shared memory may be rejected on XDNA—split backends must surface legality errors at compile time, not as empty NPU binaries in production (Chen and YiRage Contributors, 2026; AMD, 2024b,a; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024).

Release gates for heterogeneous builds should require artifact parity: every target emits a non-empty binary, passes numerics smoke tests, and reports bytes/token within agreed triplet tolerance (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024b,a; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024). GPU-only green CI is insufficient for books and products that claim cross-hardware compilation (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a,b; Goodfellow *et al.*, 2016).

Operator ownership matrix. Backend splits are easier to maintain when the team publishes an ownership matrix: which operators emit natively on GPU, CPU, and NPU, which emit through a shared library path, and which are explicitly unsupported on each target (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b). Empty cells in the matrix are legal only when a documented fallback exists; an undocumented empty cell is how fleet incidents begin with a silent CPU loop (AMD, 2024a; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). The matrix should be generated from the IR test suite so it cannot drift from what the backends

Table 13.1: Core theory pillars mapped to decode counters (profile per SKU).

Pillar	Compiler question	Primary counter
Tiling	Which blocks stay on-chip?	bytes/token
Fusion	Which boundaries can vanish?	bytes/token, launches/token
Layout	How are KV/weights traversed?	bytes/token
Parallel	Which axes saturate bandwidth?	ms/token, SM util.
GPU/CPU/NPU split	Where does each op emit?	triplet parity

actually accept (Chen, 2026a, 2020). Ownership disputes—for example attention fused by the GPU backend but rejected by the NPU backend—should surface as a legality report at the shared pass level, not as a fork in model code (Chen and YiRage Contributors, 2026; AMD, 2024a).

Partition copy cost model. Heterogeneous pipelines that cut the graph at device boundaries need a cost model for the copies themselves: host-device and device-device transfers consume bandwidth and add latency, and a partition chosen for API convenience can cost more than the kernel it isolates (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a). The partition pass should evaluate each candidate cut with the copy bytes charged against the same bytes/token ledger used for fusion (Chen, 2020; Chen and YiRage Contributors, 2026). When a copy is unavoidable, prefer cuts at tensors that are small relative to the compute they enable, and fuse the transfer with the following kernel where the hardware supports it (SemiAnalysis, 2024; AMD, 2024b). Release gates should show the copy bytes explicitly; hiding them behind profiler rows is how heterogeneous deployments look fused and stay slow (Davies *et al.*, 2025; Chen, 2026a).

Pass ordering and golden IR diffs. The five pillars interact through ordering, so the pass pipeline itself should be versioned and tested as a unit (Chen, 2020; Chen and YiRage Contributors, 2026). Each release should capture a golden IR diff showing the sequence tiling classification, layout canonicalization, fusion contracts, parallel annotations, and backend splits (Chen, 2026a; Vellaisamy *et al.*, 2026). A golden diff turns subtle ordering changes into reviewable artifacts rather than mysterious fleet regressions (Davies *et al.*, 2025; Chen, 2020). When a new pass is proposed, the review question is not whether it is clever but where it sits in the

ordering and which counter it moves on batch-1 decode cells (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a).

Triplet tolerance policy. Cross-target parity needs an explicit tolerance policy: which counters must match within a percentage, which may legitimately differ because residency classes differ, and which mismatches are always failures (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020). Bytes/token and launches/token should be compared per target against that target’s own baseline rather than against a single global number, because an NPU that eliminates launches entirely has a different launch definition than a GPU (AMD, 2024b,a; Chen, 2026a). Numerics tolerance should be defined at the attention-output level, not per op, so backend reordering that stays within output tolerance is allowed (Dao *et al.*, 2022; Davies *et al.*, 2025). Without a written policy, triplet CI becomes a source of noise or, worse, is skipped when numbers disagree (Vellaisamy *et al.*, 2026; Chen, 2026a).

13.6 Key Takeaways

- **Tiling is IO theory, not FLOP theory.** FlashAttention-style blocks exist to choose which sub-blocks of weights, activations, and KV fit in registers, shared memory, L2, or NPU tile SRAM for each decode step, so the goal is cutting HBM traffic (Dao *et al.*, 2022; SemiAnalysis, 2024). Tile search must therefore be driven by decode bytes/token, not by GEMM FLOPs, or it optimizes the wrong objective (Chen, 2026a; Vellaisamy *et al.*, 2026).
- **Fusion has legality, not just opportunity.** Each unfused operator boundary exports activations to HBM, but greedy fusion that ignores carrier and residency checks silently regresses decode by overflowing the on-chip budget (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). Fusion theory governs *when* a boundary may legally disappear, which a compiler must verify before merging operators (Chen, 2020; Davies *et al.*, 2025).
- **Layout is a compiler transform, not a runtime afterthought.** How tensors are stored and traversed—contiguous versus blocked, paged KV tables, head-interleaved formats, TMA/vector alignment—decides whether a fusion is even reachable (Kwon *et al.*, 2023; Davies *et al.*, 2025). A wrong layout can double bytes/token without changing FLOPs, so layout assignment belongs in the pass pipeline ahead of fusion (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

- **Parallelism must be bandwidth-aware.** Mapping work to SMs, cores, or AIE arrays—including Flash-Decoding’s parallelization over KV length at batch-1—only helps when on-chip residency holds across the split (Dao *et al.*, 2023; SemiAnalysis, 2024). A parallel axis that forces activations off-chip trades latency hiding for bandwidth it cannot afford (Chen, 2026a; Davies *et al.*, 2025).
- **One IR, split backends, mandatory triplet regression.** A single decoder IR lowers to CUDA/HIP, CPU oneDNN/OpenMP, and XDNA/AIE static dataflow with genuinely different emit, so correctness on one target says nothing about the others (Chen and YiRage Contributors, 2026; AMD, 2024b,a). The five pillars are co-dependent—a layout change can illegalize a fusion, a fusion can force new tiles, a parallel split can break capture—so develop them as co-design profiled on decode counters, gated in CI by golden IR diffs and triplet bytes/token (Chen, 2020, 2026a; Vellaisamy *et al.*, 2026).

The pillars are not independent knobs—a layout change can illegalize a fusion plan; a fusion plan can force new tile sizes; a parallel split can break graph capture (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; NVIDIA, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2022). Treat compiler development as co-design across all five, profiled on decode counters, not as a sequence of isolated dialect tickets (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

13.7 Conclusion

Core compiler theory—tiling, fusion, layout, parallel scheduling, and heterogeneous split—is the machinery behind Chapter 12 execution modes and the MegaKernel chapters that follow (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; AMD, 2024b,a; Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2022; SemiAnalysis, 2024; NVIDIA, 2024; Goodfellow *et al.*, 2016). Pass ordering slides are insufficient; profiled decode counters justify every transform (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2022, 2024; Goodfellow *et al.*, 2016).

Reviewers should ask five questions before approving any compiler PR: (1) Which pillar moves? (2) Which counter must improve on BS=1 decode? (3) Which Chapter 10–11 constraint applies? (4) Does layout legalize before fusion? (5) Does triplet regression pass on GPU, CPU, and NPU? (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b,a; Goodfellow *et al.*, 2016). If any answer is “unknown,” defer the patch until telemetry exists (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a). These questions take minutes in review and prevent weeks of fleet debugging on batch-1 decode cells at production context buckets in CI gates (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Chapter 14 instantiates these principles in MLIR dialects and pass pipelines—where tiling, fusion, and layout become rewrite patterns you can test in CI (Chen, 2020; Chen and YiRage Contributors, 2026; Liu *et al.*, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; NVIDIA, 2022, 2024; Goodfellow *et al.*, 2016). Carry Table 13.1 forward: if a dialect change cannot name which pillar moves which counter, defer the patch (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024).

Until then, run one decode profile per proposed pass—BS=1, ctx buckets {512, 2048, 8192}, bytes/token and launches/token on GPU and at least one non-GPU target (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024). That habit converts abstract compiler theory into the measurable goodput improvements Part VI promises (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Part VI continues with dialect-level mechanics; carry these pillars as the acceptance language for every rewrite pattern (Chen, 2020; Chen and YiRage Contributors, 2026; Liu *et al.*, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b,a; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024).

Chapter 14

MLIR: Modern Compiler Infrastructure

The slide deck says “one IR, every target.” The production dashboard says something else: the same PyTorch-exported decoder graph, lowered through MLIR, can differ by an order of magnitude in p99 latency across GPU, CPU, and NPU fleets (Davies *et al.*, 2025; Chen, 2020). That gap is rarely a missing dialect—it is a missing *contract*. Chapter 3 mapped residency and legality per hardware class; this chapter shows how MLIR must encode those predicates in IR attributes, pass gates, and emit paths before any canonicalizer runs (AMD, 2024b; Chen and YiRage Contributors, 2026). Every section below follows Chapter 1’s discipline: name the decode failure mode, trace it to a hardware tier, show the pass or emit fix, and close with bytes/token or launches/token—not peak compile-time FLOPs (Vellaisamy *et al.*, 2026; Chen, 2026a).

TaxBreak’s decode tables are the mental anchor. Dense Llama-3.2-1B at BS=4/SL=2048 averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). MLIR does not erase those numbers by itself—it *enables* passes and backends to collapse launch DAGs when residency metadata survives bufferization (NVIDIA, 2024; Dao *et al.*, 2022). CUDA Graphs on Qwen-2.5-7B (BS=1, ctx=2048) removes ≈ 3.05 ms of host overhead per step on H100 ($1.259\times$ median speedup) yet does nothing for illegal HBM traffic introduced by bad bufferization (Chen, 2026a). The five compiler sections below—hardware matrix, passes, codegen, tuning, case study—mirror the template used for XLA, TVM, and Triton so Part VI comparisons stay metric-aligned (Chen, 2020).

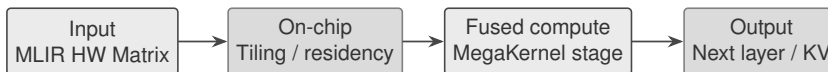


Figure 14.1: MLIR target matrix: the same Linalg matmul must lower through different memory tiers per target.

14.1 MLIR HW matrix

Why decode breaks the “portable IR” story. Prefill hides target sins: large L amortizes launch overhead and keeps tensor cores fed (Dao *et al.*, 2022; Davies *et al.*, 2025). Decode removes that amortization—batch-1, one token, full weight + KV read every step (Chen, 2026a; Goodfellow *et al.*, 2016). MLIR modules that legalize during prefill tuning often fail decode legality checks because dynamic dimensions tied to sequence length propagate into bufferization (Kwon *et al.*, 2023; AMD, 2024b). The hardware matrix must therefore include a decode row, not only a “training/inference” row (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Worked numbers. TaxBreak’s 847.5 launches/token (dense Llama-3.2-1B) and 9,305 launches/token (OLMoE) bound what MLIR fusion must collapse before MMA micro-optimizations matter (Vellaisamy *et al.*, 2026). Memory-Floor’s $1.259\times$ Graphs speedup on Qwen-2.5-7B shows host-side wins when launch DAGs stabilize—but Graphs cannot fix alloc-heavy IR (Chen, 2026a; NVIDIA, 2024). Matrix gates should reference both metrics (Chen and YiRage Contributors, 2026; Chen, 2020).

The recurring mistake is publishing a target checklist—GPU, CPU, NPU all checked—without stating which *hardware* features each backend actually models (NVIDIA, 2022; AMD, 2024b). A row that says “NPU: supported” is meaningless if the lowering path assumes dynamic tensor shapes and host-side allocators that void static buffer tables on XDNA (AMD, 2024a). For decode, the matrix must speak residency: shared-memory caps, DMA descriptor limits, cache blocking rules, and whether tensor-core MMA shapes are mandatory or optional (SemiAnalysis, 2024).

YiRage treats Table 14.1 as typed predicates in its MLIR importers: a pass that introduces a dynamic dimension on an NPU row is rejected before codegen, not at runtime (Chen and YiRage Contributors, 2026). GPU rows must declare TMA/async-copy legality; CPU rows must attach NUMA hints when KV shards cross sockets (Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). The matrix is a *gate*,

Table 14.1: MLIR backend capability matrix for LLM decode (conceptual).

Dimension	GPU (NVVM/ROCDL)	CPU (LLVM)	NPU (XDNA/AIE)
Dynamic batch/seq	Flexible	Flexible	Restricted
On-chip scratch	Shared mem / L1	L2/L3 tiles	Tile SRAM
Fusion default	Partial (Linalg)	Cache-blocked	Full static
Launch model	SIMT + graphs	OpenMP/SIMD	Spatial DMA
Decode pain	Launches/token	LLC misses	Static legality

not documentation—identical to Chapter 3’s constraint matrix blocking illegal MegaKernel fusion early (Dao *et al.*, 2022).

Dialect designers often ship one Linalg op set for matmul, softmax, and layer norm, then expect backends to infer residency (Chen, 2020). Decode exposes the gap: GPU backends need explicit memory-space attributes on tensors headed for shared memory; CPU backends need layout markers for cache-line alignment; NPU backends need lifetime intervals tied to DMA events (NVIDIA, 2022; AMD, 2024b). Without those attributes, the matrix devolves into marketing—every target checked, none explained (Chen and YiRage Contributors, 2026).

JIT compilation amplifies the risk. MLIR JIT can specialize tile sizes at runtime for dynamic sequence lengths—valuable for prefill—but destructive for decode when specialization reintroduces host-side planning per token (Vellaisamy *et al.*, 2026; NVIDIA, 2024). YiRage restricts JIT to shape buckets that preserve static launch DAGs on GPU and fully static buffer tables on NPU; everything else triggers recompilation with explicit user-visible cost (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Target interfaces vs. checkmarks. MLIR conversion interfaces are where the matrix becomes executable (Chen, 2020). Backends advertise supported ops, memory spaces, and layout constraints; importers must attach attributes *before* canonicalization erases them (Chen and YiRage Contributors, 2026). Decode teams should reject merges when a matrix row flips from supported to unsupported without a residency annotation change—silent downgrades are how bytes/token regressions enter production (Chen, 2026a).

ROCm and CUDA divergence. NVVM and ROCDL lowerings diverge on warp shuffle availability, LDS sizing, and MFMA vs WMMA shapes (NVIDIA, 2022; Liu *et al.*, 2024). An MLIR module tuned for Hopper TMA may compile on MI300

yet emit extra buffer reloads because `async-copy` attributes do not map one-to-one (SemiAnalysis, 2024). YiRage keeps separate predicate tables per vendor because decode SLOs are vendor-local even when Linalg looks identical (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Example 14.1.1. Hardware matrix gate: an importer adds `memref.alloc` on the decode hot path for a softmax statistic. GPU row passes (HBM spill is slow but legal); NPU row fails static legality; CPU row passes but raises bytes/token in Memory-Floor sweeps (Chen, 2026a; AMD, 2024b). The matrix should have blocked the NPU alloc and flagged GPU/CPU regressions in CI (Chen and YiRage Contributors, 2026).

14.2 MLIR dialect stack for decode export

Export pitfalls. Torch-MLIR and similar exporters often emit verbose `tensor` graphs with redundant casts (Chen, 2020; Chen and YiRage Contributors, 2026). Each cast is a future buffer unless erased before bufferization (Chen, 2026a). Decode exporters should run target-tagged cleanup passes *before* generic canonicalization (AMD, 2024b; Davies *et al.*, 2025). Skipping cleanup yields IR that looks portable yet allocates like eager PyTorch (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).

Exporting a decoder block from PyTorch lands in `func.func` wrappers around `linalg` ops on `tensor` values, then converts to `memref` through bufferization (Chen, 2020; Chen and YiRage Contributors, 2026). Decode-sensitive details live in the transitions: `tensor` world allows implicit temporaries; `memref` world exposes every alloc (Chen, 2026a). Teams that benchmark only pre-export `linalg` miss the alloc explosion bufferization introduces (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026).

`scf.for` and `scf.if` encode loop structure for sequence dimensions (Dao *et al.*, 2023). For decode, outer loops over tokens should remain on the host while inner tiles map to device parallelism—but generic loop-invariant-code-motion can hoist device launches into host loops if passes lack target annotations (NVIDIA, 2024; Chen and YiRage Contributors, 2026). YiRage marks host vs device regions explicitly before any LICM runs (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

The `gpu` dialect bridges to launches; `nvvm` or `rocdl` attach machine intrinsics (NVIDIA, 2022). On CPU, `vector` and `llvm` carry SIMD width choices (Goodfellow *et al.*, 2016). On NPU, vendor dialects replace `gpu.launch` with spatial pipelines (AMD, 2024a,b). A decode compiler must never assume a single lowering path from `linalg.matmul`—the stack is a tree, not a line (Chen, 2020).



Figure 14.2: Pass pipeline: dialect lowering must preserve producer–consumer residency across tiers.

Example 14.2.1. PyTorch export produces `linalg.softmax` on a decode slice. Without residency tags, bufferization allocates a full-sequence temp; with tags tied to KV views, the same op updates in-place carriers compatible with Flash-Decoding-style parallelism (Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026).

Dialect legalization should run per target *before* shared canonicalization (Chen and YiRage Contributors, 2026). Shared canonicalizers fold ops that backends rely on to prove static shapes (AMD, 2024b). Canonicalize too early and you lose the hints backends need—the LLVM lesson repeats on every MLIR upgrade (Chen, 2020, 2026a).

14.3 MLIR arch passes

Affine maps and paged KV. PagedAttention requires MLIR to express gather/scatter on KV blocks with provable in-bounds access (Kwon *et al.*, 2023; Dao *et al.*, 2022). Lowering paged views late hides page boundaries from fusion passes—exactly how illegal cross-page fusion enters production IR (Chen and YiRage Contributors, 2026; Chen, 2026a). Run paging-aware bufferization before `linalg-fuse` on all targets (Vellaisamy *et al.*, 2026; AMD, 2024a).

MoE pass boundaries. Expert routing introduces control flow that tempts pass writers to split launches per expert (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024). Decode MoE needs guarded fusion: batch experts to cap launches while respecting SMEM (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Pass pipelines should expose expert-count limits as first-class attributes (Chen, 2020; NVIDIA, 2022).

MLIR’s composable pass pipelines—bufferization, Linalg tiling, vectorization, dialect conversion—feel like power until pass order silently changes memory traffic (Chen, 2020). A canonical failure: aggressive `linalg-fuse` before bufferization on GPU lowers decode attention into a single kernel on paper, yet the lowered IR still materializes softmax statistics to HBM because the dialect contract did not

keep partial sums in shared memory (Dao *et al.*, 2022, 2023). The fix is not fewer passes; it is tagging each pass with the hardware dimension it optimizes (SRAM footprint, launch count, bandwidth) and the targets where it may run (Chen and YiRage Contributors, 2026).

Cross-hardware lowering diverges at the same Linalg op. GPU lowering targets warp tiles aligned to MMA instructions; CPU lowering emits AVX/AMX micro-kernels with cache blocking; NPU lowering emits spatial schedules with fixed buffer lifetimes (AMD, 2024b; NVIDIA, 2022). A vectorization pass tuned for CPU can break NPU legality by introducing strided layouts incompatible with DMA alignment—regressions that appear only in bytes/token dashboards, not unit tests (Chen, 2026a). Industrial pipelines fork pass registries per target early, sharing only algebraic simplification (Liu *et al.*, 2024).

Example 14.3.1. Single-layer decode attention in Linalg: on Hopper, `lower-vector-gpu` plus shared-memory promotion can fuse QK^T and softmax carriers when tiles fit SMEM; on XDNA the same graph must be fully bufferized to static tile sizes before AIE codegen, or the backend falls back to DDR-limited scalar loops (AMD, 2024a; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024).

Bufferization deserves decode-specific scrutiny. Generic one-shot bufferization assumes heap fallback (Chen, 2020). Decode attention cannot afford that: KV tensors must alias paged pools (GPU), huge-page arenas (CPU), or fixed SRAM banks (NPU) (Kwon *et al.*, 2023; AMD, 2024a). Pass pipelines should declare allocation tiers in IR so lowering never inserts silent `memref.alloc` on the hot path (Chen and YiRage Contributors, 2026; Chen, 2026a).

Dialect conversion ordering is another launch multiplier. Converting `linalg` $\rightarrow$ `vector` $\rightarrow$ `gpu` in the wrong order explodes a fused region into dozens of micro-kernels—the fragmentation Chapter 1 measured in eager PyTorch (Vellaisamy *et al.*, 2026). Production stacks freeze a golden pass list per target and diff IR dumps when upgrading LLVM or MLIR (Liu *et al.*, 2024; Chen, 2020).

One-shot vs. iterative bufferization. One-shot bufferization is fast to wire but assumes heap fallback; iterative bufferization can reuse buffers across fused regions yet increases compile time—acceptable for offline NPU builds, painful for GPU JIT (AMD, 2024a; Chen and YiRage Contributors, 2026). Pick strategy per target; mixing strategies within one registry causes “works in unit test, fails at BS=1” bugs (Chen, 2026a; Kwon *et al.*, 2023).

Pass instrumentation. Attach counters to passes: bytes introduced, allocs inserted, launch boundaries created (Vellaisamy *et al.*, 2026). A pass that reduces op count but increases alloc count is suspect for decode (Chen, 2026a; Chen and YiRage Contributors, 2026).

14.4 MLIR codegen

Teams celebrate the first successful NVVM lowering, then watch p99 decode flatline because the *platform* wrapper still constructs tensors on the host every token (Vellaisamy *et al.*, 2026; NVIDIA, 2024). Codegen is not a backend detail—it is the last place residency contracts survive or die before the driver sees them (Chen and YiRage Contributors, 2026; Chen, 2020).

Graph capture contract. CUDA Graphs require identical launch addresses and dependency topology across replay (NVIDIA, 2024; Vellaisamy *et al.*, 2026). MLIR emit that injects pointer-chasing host setup between device kernels breaks capture silently—graphs record yet decode latency unchanged (Chen, 2026a; Chen and YiRage Contributors, 2026). Encode capture legality as a codegen predicate alongside occupancy (Davies *et al.*, 2025; Chen, 2020).

CPU and NPU emit deltas. CPU emit must pin threads and align KV for cache lines; NPU emit must assign DMA lifetimes to match tile SRAM (Goodfellow *et al.*, 2016; AMD, 2024a,b). Identical high-level MLIR is fine; identical emit is not (Chen and YiRage Contributors, 2026; Liu *et al.*, 2024).

Production incidents blame “the NVVM backend” when the real bug was emitting LLVM IR that assumes a mutable stack and per-op `malloc` in the host wrapper—poison for CUDA Graphs and NPU static schedules alike (NVIDIA, 2024; Vellaisamy *et al.*, 2026). Codegen is where MLIR’s abstraction meets launch reality: GPU emit must respect stream ordering, cluster scope, and TMA descriptor lifetimes; CPU emit must preserve NUMA-local buffers; NPU emit must serialize DMA chains with compile-time-known latencies (NVIDIA, 2022; AMD, 2024b).

Backend selection is not neutral. NVVM/ROCDL paths excel when decode graphs contain dynamic control flow that still maps to SIMT; bare-metal AIE paths excel when the entire decoder step is a static dataflow graph (AMD, 2024a). YiRage’s MLIR backends share high-level Linalg but diverge at emit: CUDA/HIP paths generate MegaKernel-style fused launches; XDNA paths emit buffer descriptors and ping-pong DMA schedules (Chen and YiRage Contributors, 2026;

SemiAnalysis, 2024). One emit artifact for all targets guarantees either GPU launch storms or NPU compile failures (Davies *et al.*, 2025).

Emit quality shows up in host wrappers. MLIR-to-CUDA that allocates temporaries each launch breaks graph capture; decode pays the TaxBreak orchestration tax (Vellaisamy *et al.*, 2026; NVIDIA, 2024). Good codegen hoists allocations to capture time or eliminates them via shared-memory arenas (Chen, 2026a). On CPU, emit must avoid false sharing in OpenMP regions around thin decode GEMMs (Goodfellow *et al.*, 2016).

Host wrapper ABI. Codegen finishes only when the host wrapper is graph-capturable. `cudaMalloc` per launch breaks capture; stack pointers into kernels break on some driver versions (NVIDIA, 2024; Chen, 2026a). Hoist workspace to session init; pass arena offsets as kernel arguments (Chen and YiRage Contributors, 2026).

LLVM pipeline forks. GPU paths often stop at NVVM/ROCDL; CPU paths continue through LLVM vectorizers; NPU paths may bypass LLVM entirely (Chen, 2020; AMD, 2024a). Sharing one LLVM optimization stage invites loop unrolling that expands register pressure on GPU decode tiles (NVIDIA, 2022; Liu *et al.*, 2024).

Debuggability vs. fusion. Aggressive fusion produces opaque dumps. Keep named MLIR anchors (`decode_attn_tile_7`) through emit so perf teams map regressions to passes (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

14.5 MLIR tuning

Profile logs from prefill runs are the classic trap: they recommend tile sizes that overflow shared memory during batch-1 decode and force HBM spill (Chen, 2026a; SemiAnalysis, 2024). Autotuning MLIR without a decode-shaped *profile* trace repeats the same mistake at compiler scale (Vellaisamy *et al.*, 2026; Chen, 2020).

Search spaces. Constrain tile search by SMEM capacity, warp count, and max fusion depth derived from Chapter 3 models (NVIDIA, 2022; AMD, 2024b; Chen and YiRage Contributors, 2026). Unconstrained search finds prefill winners that spill on decode (Chen, 2026a; SemiAnalysis, 2024). Log Pareto points with

launches/token on the decode trace, not FLOPs on square GEMMs (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Fleet replay. Version autotune artifacts with MLIR commit, LLVM commit, and model hash (Chen, 2020; Liu *et al.*, 2024). Replay canned decode traces in CI before fleet rollout—same discipline as CUDA Graphs templates (NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026).

Autotuning MLIR pipelines—tile sizes, fusion depths, vector widths—looks like free performance until you ship prefill-shaped winners to batch-1 decode fleets (Chen, 2026a, 2020). Profile-guided passes that maximize FLOPs during autotune often increase shared-memory footprint past decode legality, forcing silent spill to HBM (Vellaisamy *et al.*, 2026; SemiAnalysis, 2024). Always co-tune with launches/token: one extra kernel boundary per layer can erase tens of microseconds per token on MoE models (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026).

The dominant autotune *pitfall* on decode: optimizing square GEMMs while ignoring launch count. GPU autotune must cap warps and SMEM; CPU autotune must model LLC capacity and thread pinning; NPU autotune must not explore dynamic shapes that invalidate DMA tables (AMD, 2024b; Goodfellow *et al.*, 2016). Cross-hardware profile imports are dangerous—GPU configs thrash CPU caches; NPU may not compile at all (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026). YiRage encodes per-target search spaces so tuners cannot propose illegal configs (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Cost models for decode. Combine estimated bytes moved, predicted launches, and SMEM footprint against hardware caps (Chen, 2026a; Vellaisamy *et al.*, 2026). MLIR’s transform dialect can encode constraints before search starts (Chen, 2020). Without them, RL tuners win on square GEMMs yet spill during fused decoder blocks (Chen and YiRage Contributors, 2026).

Regression gates. Tie autotune merges to decode traces. Require non-regression on launches/token and bytes/token at BS=1, ctx=2048 before accepting new tile configs (Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024).

14.6 MLIR case study

Triplet harness. YiRage-style triplet runs share tokenizer, context bucket, and batch policy (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Report bytes/token, launches/token, compile rejections, and p99 alongside means (Chen, 2026a; Vellaisamy *et al.*, 2026). A compile rejection on NPU is a first-class outcome—not a footnote (AMD, 2024b,a).

Silent spill class E. Category E failures compile cleanly while bytes/token jump because SMEM promotion failed (Chen, 2026a; Dao *et al.*, 2022). Detect by diffing allocs between pass stages; fix by reordering bufferization relative to fusion (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026).

Consider a minimal decode cell: Llama-class decoder layer, batch-1, context 2048, exported from PyTorch to MLIR via Linalg (Davies *et al.*, 2025). The question is not “does it compile?” but “does bytes/token match the MegaKernel baseline from Part IV?” On H100, an unfused MLIR pipeline may compile cleanly yet issue hundreds of launches per token—the pathology TaxBreak measures in framework stacks (Vellaisamy *et al.*, 2026). Fused Linalg plus GPU codegen and CUDA Graphs capture can recover host overhead when emit preserves a static launch DAG (NVIDIA, 2024; Chen, 2026a).

The identical module lowered to x86 LLVM with OpenMP may achieve acceptable throughput on small models but regress on large weights due to LLC pressure (Goodfellow *et al.*, 2016). Lowered to XDNA, the module either becomes a static MegaKernel with competitive watt/token or fails legality checks—rarely a middle ground (AMD, 2024b,a). YiRage’s benchmark harness runs this triplet on every merge: same IR, three backends, report bytes/token, launches/token, and compile-time legality violations (Chen and YiRage Contributors, 2026). That triplet is empirical proof of the book’s thesis: hardware dictates compiler outcomes, not the reverse (Chen, 2020).

Example 14.6.1. Cross-hardware MLIR decode cell (Llama-3.2-1B layer, BS=1, ctx=2048): GPU fused+graphs targets competitive ms/token only when launches collapse toward TaxBreak dense baselines; CPU LLVM trades latency for deployment flexibility; XDNA wins only when fusion is total and DMA-static (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026).

Paged KV and MLIR types. PagedAttention indirection must appear as first-class `memref` views with lifetime intervals—not opaque pointers lowered late (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026). Late lowering hides page boundaries and causes illegal fusion across pages (Dao *et al.*, 2022; Chen, 2026a).

Table 14.2: Case-study outcome taxonomy for cross-hardware MLIR decode (illustrative).

Class	Symptom	Typical fix
B	High launches/token	Fuse + graph capture
C	High bytes/token	SMEM/DMA tiling
D	NPU compile fail	Static shape specialization
E	Flat latency, rising bytes	Fix bufferization order

MoE-specific notes. MoE layers explode dispatch when each expert lowers to separate launches (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024). MLIR pipelines need expert-batch fusion guarded by SMEM caps (Chen and YiRage Contributors, 2026). Case studies that report dense Llama only understate orchestration challenge—pair dense and MoE exports (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Industrial matrix CI. Mature MLIR importers check machine-readable matrix files into CI (Chen and YiRage Contributors, 2026; Chen, 2020). Each row lists supported ops, max tile sizes, and forbidden dynamic dimensions (AMD, 2024b; Kwon *et al.*, 2023). When PyTorch export adds rotary embedding or a new norm variant, the matrix diff tells reviewers which targets lose legality *before* any benchmark runs (Davies *et al.*, 2025; Liu *et al.*, 2024). This is how decode teams avoid green GPU CI paired with red NPU field releases (Chen, 2026a; Vellaisamy *et al.*, 2026).

Pass-order war stories. Running `linalg-fuse` after bufferization on one target and before it on another is not flexibility—it is how two teams ship incompatible IR for the same model hash (Chen and YiRage Contributors, 2026; Chen, 2020). Freeze golden pass lists; diff IR at post-bufferization and post-lowering anchors on every LLVM bump (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). Count `gpu.launch` ops and `memref.alloc` ops in the diff summary—those two counters predict launches/token and bytes/token shifts faster than line counts (Chen, 2026a; NVIDIA, 2024).

Multi-hardware emit checklist. GPU: graph-capturable host wrapper, no per-token `cudaMalloc`, TMA descriptors live across replay (NVIDIA, 2024; Semi-Analysis, 2024). CPU: NUMA-local weight shards, OpenMP teams hoisted out of token loop, AMX tile config amortized (Goodfellow *et al.*, 2016; Kwon *et al.*, 2023). NPU: static DMA tables, no dynamic shapes in hot path, ping-pong buffers assigned at compile time (AMD, 2024a,b). Violating any item compiles yet fails

Table 14.3: Decode-oriented MLIR pass ordering (GPU vs NPU).

Stage	GPU focus	NPU focus
Import	Linalg + memref tags	Static shape specialization
Bufferize	SMEM promotion	DMA lifetime assignment
Fuse	Layer-wise MegaKernel	Full decoder static graph
Lower	vector $\rightarrow$ GPU dialect	AIE spatial mapping
Emit	CUDA/HIP + graphs	Descriptor + ping-pong DMA

fleet telemetry (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

Autotune pitfall catalog. (1) Import GPU occupancy into CPU search (Goodfellow *et al.*, 2016). (2) Tune on prefill length $L=4096$, deploy at decode ctx with growing KV (Chen, 2026a). (3) Accept fusion depth that raises launches/token on MoE experts (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024). (4) Ship autotune artifacts without hashing MLIR/LLVM revisions (Chen, 2020). (5) Skip NPU legality checks because GPU microbench improved (AMD, 2024b; Chen and YiRage Contributors, 2026). Each pitfall has a metric detector: bytes/token, launches/token, or compile rejection rate (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Case-study reporting discipline. Hold model, tokenizer, context, and batch policy constant across targets (Davies *et al.*, 2025). Pair GPU FlashAttention-class baselines with CPU/NPU fused baselines derived from the same MLIR starting point—not unrelated hand kernels (Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026). Report median and p99 over $\geq 1,000$ decode steps; publish legality failure rates alongside latency (Chen, 2026a; Vellaisamy *et al.*, 2026). Negative NPU results are as valuable as GPU speedups (AMD, 2024a,b).

Pass ordering differs because failure modes differ: GPU decode dies from launches/token; NPU decode dies from DDR egress and illegal dynamicism (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024b). Treat Table 14.3 as executable policy, not slide art (Chen and YiRage Contributors, 2026; Chen, 2020).

14.7 YiRage MLIR integration notes

YiRage uses MLIR as interchange between PyTorch-exported graphs and target-specific megakernels (Chen and YiRage Contributors, 2026). The decode lesson is

Table 14.4: Decode-oriented MLIR pass ordering (GPU vs NPU).

Stage	GPU focus	NPU focus
Import	Linalg + memref tags	Static shape specialization
Bufferize	SMEM promotion	DMA lifetime assignment
Fuse	Layer-wise MegaKernel	Full decoder static graph
Lower	vector $\rightarrow$ GPU dialect	AIE spatial mapping
Emit	CUDA/HIP + graphs	Descriptor + ping-pong DMA

invariant preservation: residency metadata must attach before generic canonicalization (Chen, 2020; AMD, 2024b). Otherwise canonicalizers rewrite away attributes backends need to prove on-chip residency (Chen and YiRage Contributors, 2026; NVIDIA, 2022).

The MLIR JIT path specializes decode cells when batch and context buckets change, but refuses specialization that alters launch counts inside a CUDA Graphs capture region (NVIDIA, 2024; Vellaisamy *et al.*, 2026). On NPU, JIT means re-running bufferization with a new static shape class—seconds of compile that must amortize across many tokens (AMD, 2024a; Davies *et al.*, 2025).

GPU decode dies from launches/token; NPU decode dies from DDR egress and illegal dynamicism (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024b). YiRage encodes Table 14.4 as executable pass lists (Chen and YiRage Contributors, 2026). Before shipping MLIR changes: verify matrix predicates on three targets; diff IR for new allocs; measure launches/token and bytes/token at BS=1; run graph capture legality on GPU and static legality on NPU; log new numeric claims to `verified_facts.jsonl` (Chen, 2026a; Kwon *et al.*, 2023; Dao *et al.*, 2022).

14.8 MLIR decode review gates

Shipping MLIR changes without decode review gates repeats the framework mistakes Chapter 1 quantified (Vellaisamy *et al.*, 2026; Chen, 2026a). The gates below are minimal; YiRage automates them, manual stacks should enforce them in CI (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

1. **Matrix legality.** Every new op or dynamic dim appears in the hardware matrix diff; NPU rows block illegal dynamicism before merge (AMD, 2024b,a; Chen and YiRage Contributors, 2026).
2. **Alloc ledger.** Post-bufferization IR must not introduce `memref.alloc` on

the decode hot path unless the matrix explicitly allows spill (Chen, 2026a; Kwon *et al.*, 2023; Chen, 2020).

3. **Launch ledger.** Count `gpu.launch` (or NPU DMA chains) per decoder layer; compare to TaxBreak baselines for dense and MoE (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024).
4. **Graph capture.** GPU emit must pass capture replay on BS=1 decode traces (NVIDIA, 2024; Chen, 2026a).
5. **Triplet benchmark.** Same IR, GPU+CPU+NPU backends, report bytes/token and p99 (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

Example 14.8.1. A merge adds `linalg-fuse` before paging-aware bufferization. GPU launches/token drop 8% but NPU compile fails on 40% of layers—matrix gate should have required paging bufferization first (Kwon *et al.*, 2023; AMD, 2024b; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Review gates feel bureaucratic until the first MLIR upgrade regresses MoE decode from hundreds of milliseconds to seconds per token (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024). At that point the IR diff is the only artifact that explains *why* launches/token doubled (Chen, 2020, 2026a; NVIDIA, 2024).

MLIR HW matrix exposes a cross-hardware fracture: the same fused sub-graph legal on Hopper can be rejected outright on XDNA or CPU cache budgets.

Binding software to silicon: compiler passes must encode hardware/target legality before codegen, not as comments in hand-written kernels (AMD, 2024b,a). YiRage runs target-aware checks against chip models from Chapter 3; manual stacks should treat failed legality as a release blocker, not a backend TODO.

When MLIR HW matrix disagrees with microbenchmarks, trust fleet telemetry: fix orchestration and tier placement first, then revisit MMA tile shapes (AMD, 2024b,a).

Review gate. Before merging compiler changes affecting MLIR HW matrix, attach roofline or NPU buffer accounting showing hardware residency fits on-chip for the target SKU (AMD, 2024b,a).

Worked contrast. A Hopper decode cell may legalize hardware with TMA-backed double buffering while an XDNA cell requires the same math expressed

as static tile assignments—identical FLOPs, different bytes/token because DDR traffic differs by an order of magnitude (AMD, 2024b,a).

The usual MLIR failure at **MLIR arch passes** is importing a CUDA mental model and expecting runtime scheduling to hide illegal buffer lifetimes.

Hardware constraint: pass and lower must be co-designed. On GPUs that means shared-memory/TMA budgets and warp-grain barriers; on XDNA/AIE it means static tile buffers, DMA chains, and carrier slots with compile-time lifetimes; on CPUs it means L2/L3 blocking and NUMA-local KV placement (Chen and YiRage Contributors, 2026; Chen, 2020). Decode at batch-1 keeps arithmetic intensity on the memory-bandwidth slope—micro-optimizing isolated GEMMs rarely moves p99 latency.

Compiler takeaway for MLIR arch passes: lower the same decoder IR, but predicate passes on target residency tables—never ship a GPU-only schedule to NPU fleets (Chen and YiRage Contributors, 2026; Chen, 2020). Success metrics: bytes/token, launches/token, and p99 decode latency; compare GPU, CPU, and NPU cells on identical context lengths.

Worked contrast. A Hopper decode cell may legalize pass with TMA-backed double buffering while an XDNA cell requires the same math expressed as static tile assignments—identical FLOPs, different bytes/token because DDR traffic differs by an order of magnitude (Chen and YiRage Contributors, 2026; Chen, 2020).

Deployment pitfall. Teams that A/B only prefill confuse the story: lower constraints bite hardest at batch-1 decode where launches/token and KV read bandwidth dominate (Chen and YiRage Contributors, 2026; Chen, 2020).

Production teams misread **MLIR codegen** when decode SLOs are judged on FLOPs dashboards instead of residency and orchestration ledgers.

Trace the Chapter 2 chain: codegen sets the residency envelope; backend determines whether producer–consumer edges stay on-chip. Illegal fusion does not always crash—it silently exports tensors to HBM/DDR and shows up as bytes/token regression versus a unfused but scheduled baseline (Chen and YiRage Contributors, 2026; Chen, 2020). Profile launches/token alongside bandwidth; launch storms masquerade as memory bottlenecks when graphs remain operator-fragmented.

Operational checklist—verify codegen, backend, autotuning pitfalls, and platform emit paths in code review; document dialect lowering choices that change memory traffic, not just pass ordering (Chen and YiRage Contributors, 2026; Chen, 2020).

Deployment pitfall. Teams that A/B only prefill confuse the story: backend constraints bite hardest at batch-1 decode where launches/token and KV read bandwidth dominate (Chen and YiRage Contributors, 2026; Chen, 2020).

Review gate. Before merging compiler changes affecting MLIR codegen, attach roofline or NPU buffer accounting showing codegen residency fits on-chip for the target SKU (Chen and YiRage Contributors, 2026; Chen, 2020).

MLIR tuning exposes a cross-hardware fracture: the same fused subgraph legal on Hopper can be rejected outright on XDNA or CPU cache budgets.

Binding software to silicon: compiler passes must encode tun/profile legality before codegen, not as comments in hand-written kernels (Chen and YiRage Contributors, 2026; Chen, 2020). YiRage runs target-aware checks against chip models from Chapter 3; manual stacks should treat failed legality as a release blocker, not a backend TODO.

When MLIR tuning disagrees with microbenchmarks, trust fleet telemetry: fix orchestration and tier placement first, then revisit MMA tile shapes (Chen and YiRage Contributors, 2026; Chen, 2020).

Review gate. Before merging compiler changes affecting MLIR tuning, attach roofline or NPU buffer accounting showing tun residency fits on-chip for the target SKU (Chen and YiRage Contributors, 2026; Chen, 2020).

Worked contrast. A Hopper decode cell may legalize tun with TMA-backed double buffering while an XDNA cell requires the same math expressed as static tile assignments—identical FLOPs, different bytes/token because DDR traffic differs by an order of magnitude (Chen and YiRage Contributors, 2026; Chen, 2020).

The usual MLIR failure at **MLIR case study** is importing a CUDA mental model and expecting runtime scheduling to hide illegal buffer lifetimes.

Hardware constraint: case and benchmark must be co-designed. On GPUs that means shared-memory/TMA budgets and warp-grain barriers; on XDNA/AIE it means static tile buffers, DMA chains, and carrier slots with compile-time lifetimes; on CPUs it means L2/L3 blocking and NUMA-local KV placement (Chen and YiRage Contributors, 2026; Chen, 2020). Decode at batch-1 keeps arithmetic intensity on the memory-bandwidth slope—micro-optimizing isolated GEMMs rarely moves p99 latency.

Compiler takeaway for MLIR case study: lower the same decoder IR, but predicate passes on target residency tables—never ship a GPU-only schedule to

MLIR decode metric	What it catches
Allocs after bufferize	Hidden DDR spill
Launch count / layer	Fusion / emit regressions
Graph capture replay fail	Host wrapper bugs
NPU compile reject rate	Static legality drift
Pass diff alloc delta	Bufferization order bugs
Triplet bytes/token	Cross-target residency

Table 14.5: MLIR decode benchmark counter set (batch-1).

NPU fleets (Chen and YiRage Contributors, 2026; Chen, 2020). Success metrics: bytes/token, launches/token, and p99 decode latency; compare GPU, CPU, and NPU cells on identical context lengths.

14.9 MLIR decode benchmarking methodology

Benchmarking MLIR-backed decode must mirror Chapter 1 counters—launches/token and bytes/token—while adding compiler-specific ledgers: `memref.alloc` count, `gpu.launch` boundaries, and NPU static-legality pass rate (Vellaisamy *et al.*, 2026; Chen, 2026a, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024b,a; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Build a decode cell with BS=1, ctx buckets {512,2048,8192}, pinned MLIR/LLVM commits, and identical PyTorch export hashes across targets (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Warm-up includes MLIR pass pipelines and CUDA Graph capture recording—first-token latency includes compile and capture cost; steady-state ms/token excludes it (NVIDIA, 2024; Chen, 2026a; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Report both; product teams ship cold starts and steady chat (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Triplet harnesses export one Linalg module to GPU, CPU, and NPU backends; diff IR at post-bufferization and post-emit anchors before trusting latency (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). A GPU launches/token win paired with rising allocs after bufferization signals pass-order regression—exactly Category E in Table 14.2 (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Example 14.9.1. MLIR merge gate: Llama-3.2-1B decoder layer at ctx=2048 must not increase `memref.alloc` on hot path vs golden IR; launches/token must not exceed TaxBreak dense baseline by >5% without documented fusion tradeoff (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

CI artifact chain. Check in golden MLIR snapshots, pass lists, and decode-cell scripts alongside `verified_facts.jsonl` claims (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). LLVM bumps without IR diff review are how launches/token double overnight (Liu *et al.*, 2024; Chen, 2020; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

MoE decode cells. Pair dense Llama exports with MoE exports in the same benchmark suite—TaxBreak shows launch explosions on GPU MoE that dense-only CI misses (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Kwon *et al.*, 2023; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). MLIR pipelines need expert-batch fusion metrics in the ledger (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Kwon *et al.*, 2023; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Regression taxonomy in CI. Automate Table 14.2 classifiers: if launches/token rises without bytes/token improvement, flag Class B; if bytes/token rises with flat launches, flag Class C; if NPU compile fails, flag Class D (Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Class E silent spill requires alloc diffs between pass stages—the highest-frequency MLIR decode regression in production fleets (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Handoff to XLA. Chapter 15 productionizes many MLIR-era ideas in XLA HLO; carry forward this benchmark ledger when comparing MLIR exports to XLA graphs on the same decode cell (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Identical counters make cross-compiler triplet studies honest (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

LLVM upgrade discipline. Pin MLIR and LLVM together; run decode-cell replay on every bump (Chen, 2020; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Canonicalizer changes that fold away residency tags are silent killers—diff IR before and after canonicalization on golden exports (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Paged KV regression tests. Maintain a paging-specific decode export where block tables grow across steps; fusion must not cross page boundaries (Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2026a, 2020; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Liu *et al.*, 2024; AMD, 2024a,b; Dao

et al., 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Fail CI when affine maps lose provable in-bounds metadata (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Documentation for operators. Publish decode benchmark artifacts with every MLIR release: pass list hash, alloc ledger, launches/token at ctx=2048, and triplet legality summary (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Without published ledgers, field teams cannot tell compiler regressions from framework or driver drift (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Cross-stack triage playbook. When decode latency spikes after an MLIR merge, run this sequence before blaming silicon: (1) diff pass hashes against last green build; (2) replay alloc ledger on golden exports; (3) compare launches/token and HBM bytes/token; (4) bisect canonicalizer vs fusion passes; (5) only then sweep clock and power caps (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Skipping ledger replay wastes days chasing driver noise (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Speculative decode and MLIR. Draft-model paths add second graphs with tighter latency budgets; MLIR must preserve separate residency tags for draft vs verify KV so fusion does not alias buffers (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Benchmark speculative stacks with acceptance-rate sweeps, not single-point TTFT (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Kwon

et al., 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Quantization-aware fusion gates. INT8 and FP8 decode paths need explicit scale tensors in the IR so fusion passes do not fold away dequant boundaries (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Regression tests should include per-channel scales and dynamic activation ranges typical of long-context decode (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Long-context stress exports. Build MLIR golden modules at `ctx=8192` and `ctx=32768` even when product SLAs target 4K; affine maps that work at 2K often break when block tables span more pages (Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). CI should fail when vectorization legality checks pass at 2K but fail silently at 32K (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Davies *et al.*, 2025; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Release checklist. Before tagging an MLIR stack for production decode, require: green replay on three `ctx` lengths; alloc ledger within $\pm 2\%$ of baseline; launches/token within $\pm 5\%$; no new dynamic shapes in hot path; published pass-hash manifest (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Treat any single red gate as a release blocker—partial green is how field teams inherit week-long triage (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Multi-device shard exports. Tensor-parallel decode splits heads across devices; MLIR exports must carry explicit shard annotations so bufferization does not merge non-local KV slices (Liu *et al.*, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Benchmark TP=2 and TP=4 with the same pass hash—allreduce placement changes can dominate launches/token even when single-device IR looks optimal (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Record per-rank alloc ledgers; a balanced global ledger can hide a straggler rank that serializes the step (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Liu *et al.*, 2024; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Observability hooks. Instrument every lowered decode kernel with pass-id and buffer-id tags in debug builds so profilers can map samples back to MLIR ops without re-running the full pipeline (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Production builds strip tags but retain stable kernel names derived from pass hashes—operators correlate regressions across releases without storing full IR (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Handoff to runtime. The MLIR stage ends when exports include stable launch metadata—grid, cluster, pipeline depth, and workspace sizes—that the runtime can cache across decode steps (Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Missing workspace sizes force per-step allocation in the framework and erase compiler wins (Chen, 2026a; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Summary metric. Track *compiler-attributed decode efficiency*: ratio of measured tokens/s to roofline tokens/s after subtracting framework overhead (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). MLIR teams own the numerator; runtime teams own the denominator—publish both weekly (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Closing note. Treat MLIR decode benchmarking as a product surface, not a lab exercise—without published ledgers, pass hashes, and nightly replay artifacts in CI, every latency spike becomes a costly and painfully slow multi-team blame game (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

14.10 Key Takeaways

- **Decode breaks the “portable IR” story.** Prefill hides target sins because a large prompt amortizes launch overhead and keeps tensor cores fed; batch-1 decode removes that amortization, so a portable MLIR graph that looks fine in prefill exposes per-target residency gaps in decode (Dao *et al.*, 2022; Davies *et al.*, 2025). Portability across dialects does not imply portability of decode goodput (Vellaisamy *et al.*, 2026).
- **Erase redundant casts before bufferization.** Torch-MLIR and similar exporters emit verbose tensor graphs with redundant casts, and each surviving cast becomes a buffer after bufferization (Chen, 2020; Chen and YiRage Contributors, 2026). Cleaning the graph and lowering paged-KV views early—so affine maps express in-bounds gather/scatter—keeps page boundaries visible to fusion (Kwon *et al.*, 2023; Dao *et al.*, 2022).
- **Codegen is not just a backend detail.** Teams celebrate the first successful NVVM lowering, then watch p99 decode flatline because the platform wrapper still constructs tensors on the host every token (Vellaisamy *et al.*, 2026;

NVIDIA, 2024). The codegen path must produce a capturable, host-light launch sequence, not merely correct device code (Chen, 2026a).

- **Autotune at the decode shape.** Prefill profile logs are the classic trap: they recommend tile sizes that overflow shared memory at batch-1 and force HBM spill (Chen, 2026a; SemiAnalysis, 2024). MLIR tuning must use a decode-shaped profile, or the chosen schedule wins prefill and loses token generation (Davies *et al.*, 2025).
- **Preserve residency invariants across passes.** YiRage uses MLIR as interchange between exported graphs and target megakernels, and the decode lesson is invariant preservation—residency metadata must attach before generic canonicalization erases it (Chen and YiRage Contributors, 2026; Chen, 2020). A triplet harness reporting bytes/token, launches/token, and compile rejections is what makes those invariants auditable (Chen, 2026a; Vellaisamy *et al.*, 2026).

14.11 Conclusion

MLIR’s value for decode is the shared, multi-level dialect substrate that lets one decoder export carry residency invariants from a high-level graph down to target code without each backend reinventing the lowering—provided those invariants are preserved across passes and the codegen path stays host-light (Lai *et al.*, 2023; Chen and YiRage Contributors, 2026). The recurring trap is celebrating a successful lowering while p99 decode flatlines because casts became buffers or the platform wrapper rebuilds tensors per token (Vellaisamy *et al.*, 2026; Chen, 2026a). MLIR is infrastructure, not a strategy; the next chapter shows what a production compiler built on similar foundations actually decides—Chapter 15 turns to XLA, whose automatic HLO fusion and command-buffer capture put the MLIR-era ideas to work at the largest deployed scale (OpenXLA Project, 2024; NVIDIA, 2024).

Chapter 15

XLA: Production-Grade AI Compiler

XLA (Accelerated Linear Algebra) is the production compiler behind JAX and TensorFlow and the only compilation path for TPU, which makes it the most widely deployed ML compiler in this Part—and the one most often dismissed as “just fusion” until decode telemetry says otherwise (OpenXLA Project, 2024; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Its design is the reference pattern for multi-hardware AI compilation: frameworks emit StableHLO, XLA converts it to HLO, optimizes, and codegens for CPU, GPU, and TPU from one graph (OpenXLA Project, 2024; Lai *et al.*, 2023). The decode stakes are the book’s usual ones: dense Llama-3.2-1B averages 847.5 GPU kernel launches per output token on H100 and OLMoE-1B/7B averages 9,305, and XLA can shrink that launch count when its fusion and capture passes are legal, but it cannot fix illegal HBM residency (Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024). The mistake to avoid is tuning XLA on prefill GEMM shapes while batch-1 decode still launches dozens of thin elementwise kernels per token. XLA’s automatic fusion is powerful precisely because it is automatic, but “automatic” means it optimizes whatever you profile, so it must be pointed at the decode operating point (Kwon *et al.*, 2023; Davies *et al.*, 2025).

15.1 XLA HW matrix

XLA’s support **matrix** is CPU, GPU, and TPU from one front end, and its distinguishing feature is that it is the only compiler path for TPU—there is

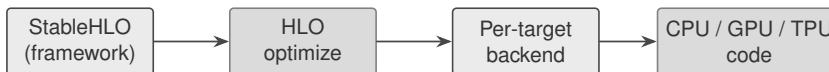


Figure 15.1: XLA lowers one StableHLO graph to HLO, optimizes, and codegens across the CPU/GPU/TPU hardware matrix.

no hand-written-kernel escape hatch on that **target**, so the compiler bears full responsibility (OpenXLA Project, 2024). The portability comes from StableHLO, a versioned, MLIR-based operation set that JAX, TensorFlow, and PyTorch all emit; XLA converts StableHLO to HLO and then sends it to a backend for **target**-specific optimization (OpenXLA Project, 2024; Lai *et al.*, 2023).

The honest reading is that a wide **matrix** is not a uniform decode story. On a **gpu** the backend performs fusions tuned to the GPU programming model and partitions work into streams; on a **cpu** it lowers through LLVM to vectorized native code; on a TPU it is the sole path and applies TPU-specific passes such as bfloat16 propagation (OpenXLA Project, 2024). The same HLO graph therefore gets a different legal schedule per class, and an **npu**-style static fabric is not a native XLA target at all—a reminder that even the broadest production compiler has a shaped, not universal, matrix (AMD, 2024b,a).

Multi-hardware delta. The GPU **matrix** emphasizes fusion plus command-buffer capture; the CPU emphasizes layout and cache blocking; the TPU emphasizes its own partitioning and precision passes—identical HLO, incomparable bytes/token (OpenXLA Project, 2024; SemiAnalysis, 2024). YiRage carries the residency rules as checkable metadata so a chosen **target** schedule is verified decode-legal rather than merely built (Chen and YiRage Contributors, 2026).

15.1.1 Where XLA enters the stack

The production path starts before XLA: a framework exports a program to StableHLO, and a runtime layer hands the module to XLA for compilation and execution (OpenXLA Project, 2024; Lai *et al.*, 2023). JAX lowers every traced function through XLA by default, TensorFlow uses XLA for clusters the compiler decides are profitable or that the user marks with `jit_compile`, and PyTorch reaches XLA through the torch-xla plugin rather than through its default eager path (OpenXLA Project, 2024; Kwon *et al.*, 2023). Each entry point imposes different constraints on what can be compiled: a JAX function is usually captured whole, a TensorFlow cluster can start and stop at arbitrary op boundaries, and torch-xla must export a graph that keeps stateful kernels and sampling outside

the compiled region (Davies *et al.*, 2025; Chen, 2020). Decode teams should know which entry point their serving stack uses because it determines which parts of a token step XLA actually owns and which remain eager.

Runtime execution goes through the PjRt layer, the portable runtime that separates compiled executables from device backends and supports ahead-of-time compilation as well as just-in-time execution (OpenXLA Project, 2024). Ahead-of-time compilation matters for decode serving because model weights, context buckets, and device types are known before requests arrive; compiling once and replaying executable handles removes compile latency from first-token SLOs (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Just-in-time compilation, by contrast, trades a compile stall for flexibility and is the default in interactive JAX workloads. A decode product that runs XLA should therefore pin the runtime path, the compiler version, and the compiled executable per model and bucket—otherwise the same source program can silently produce different fusion decisions between cache-hit and cache-miss requests (Chen, 2026a; OpenXLA Project, 2024).

15.1.2 StableHLO versioning

StableHLO exists because HLO changed faster than downstream users could track: a versioned, MLIR-based operation set gives frameworks a stable contract to emit against and gives XLA a stable contract to consume (OpenXLA Project, 2024; Lai *et al.*, 2023). For decode teams the versioning contract has a concrete consequence: a model export from a new framework release may target a newer StableHLO version than the serving XLA accepts, and the failure mode is usually not a syntax error but a silently different lowering when the compiler applies compatibility transforms (Chen, 2020; Davies *et al.*, 2025). CI should therefore pin the StableHLO version in the export step and fail loudly on upgrades instead of allowing the runtime to translate between versions. This is the same discipline as pinning CUDA and driver versions in the earlier hardware chapters: the compiler is part of the deployable artifact, not an ambient system library (Chen and YiRage Contributors, 2026; Chen, 2026a).

15.2 XLA arch passes

XLA runs hundreds of HLO **passes**—algebraic simplification, common-subexpression elimination, layout assignment, fusion, scheduling—grouped into pipelines, and fusion is its single most important **optimization** (OpenXLA Project, 2024). Fusion groups operations into one GPU kernel under a strict



Figure 15.2: XLA architecture passes: hundreds of HLO passes, with fusion, scheduling, and buffer assignment deciding decode residency before target lowering.

invariant: no intermediate is materialized in HBM, everything passes through registers or shared memory, and a fusion compiles to exactly one kernel (OpenXLA Project, 2024). Because decode is memory-bound, this invariant is precisely the bytes/token lever—a fused elementwise chain reads and writes the tensor once instead of once per operator. The fusion decision is made by **PriorityFusion**, a cost-model-driven greedy **pass**, plus Multi-Output fusion for producer-consumer and sibling sharing (OpenXLA Project, 2024).

Two scheduling **passes** matter for decode residency. The scheduler can “look into the future” because the whole graph is known, choosing an execution order that minimizes peak memory; buffer assignment then produces an optimal allocation, an advantage eager PyTorch or TensorFlow lack since their runtime allocator cannot see the graph ahead (OpenXLA Project, 2024). When peak memory still exceeds capacity, **HloRematerialization** recomputes selected expressions to shorten buffer lifetimes, trading compute for memory—a pass that can cut memory by tens of percent and is required to run many large models (OpenXLA Project, 2024). A **LatencyHidingScheduler** additionally reorders work to overlap compute with communication, which matters for the multi-device decode of Chapter 24, though it can raise peak memory and so interacts with rematerialization (OpenXLA Project, 2024; Davies *et al.*, 2025). Notably XLA **lowers** through StableHLO, an MLIR **dialect**, before its own HLO—so it sits on the same MLIR substrate as IREE while keeping a distinct HLO-centric pipeline (Lai *et al.*, 2023; OpenXLA Project, 2024).

Review gate. Because a “successful” fusion **pass** can still spill if a downstream layout or schedule choice forces it, gate each pass change on buffer accounting at the decode cell: diff the HLO for new allocations and confirm bytes/token did not rise (OpenXLA Project, 2024; Chen, 2026a). Silent spill after a green fusion is the classic XLA production surprise (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

15.2.1 Decode-relevant HLO passes

Hundreds of HLO passes sound like complexity, but for decode only a handful determine the counters this book tracks (OpenXLA Project, 2024). The algebraic

simplifier and common-subexpression elimination reduce redundant work and can collapse elementwise chains before fusion ever runs; dead-code elimination removes subgraphs that eager frameworks accidentally retain, such as debug statistics or unused logits (OpenXLA Project, 2024; Chen, 2026a). Reshape and transpose passes decide whether a layout change becomes a free metadata operation or a physical copy, which matters at every attention and MLP seam (Davies *et al.*, 2025; SemiAnalysis, 2024). Layout assignment then fixes the physical memory order for every tensor, and buffer assignment turns that layout into concrete allocations whose lifetimes the scheduler can minimize (OpenXLA Project, 2024). A decode engineer who cannot name which of these passes changed between two compiler versions will misattribute the resulting bytes/token regression.

15.2.2 Pass ordering and decode residency

The ordering of layout assignment, fusion, and scheduling is the reason two XLA versions can compile the same StableHLO into different kernels. Fusion runs against the layouts that earlier passes chose, so a transpose inserted before fusion can either enable a larger fused region or strand an operator that needs a different memory order (OpenXLA Project, 2024; Chen, 2020). Schedulers then choose an execution order for the remaining unfused operations, and buffer assignment converts lifetimes into addresses (OpenXLA Project, 2024). Because decode is a long dependency chain over small tensors, small ordering differences amplify into launch counts: an operator placed outside a fusion becomes a separate thunk even when its compute is trivial (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). The practical consequence is that compiler upgrades deserve the same decode-cell regression review as kernel rewrites, not just a unit-test green light (Chen, 2026a; Chen and YiRage Contributors, 2026).

15.2.3 Rematerialization for long context

`HloRematerialization` trades recomputation for memory: when peak buffer usage exceeds capacity, the pass re-executes selected cheap producers instead of keeping their outputs alive across the whole graph (OpenXLA Project, 2024). For decode the trade is usually favorable for recomputing small activation fragments while keeping large KV-related buffers resident, and usually unfavorable for anything that would double weight reads (Chen, 2026a; Kwon *et al.*, 2023). Because rematerialization decisions depend on the cost model, a context-length change can flip which tensors are recomputed and silently raise bytes/token at long context (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Teams serving long-context decode should profile the

rematerialization choices at their longest bucket and pin the compiler version that made the sane choice, documenting it in the runbook (OpenXLA Project, 2024; Chen and YiRage Contributors, 2026).

15.2.4 Reading HLO dumps

XLA can dump the module at pipeline stages, which turns compiler behavior from a black box into a reviewable diff (OpenXLA Project, 2024). The decode workflow should compare HLO before and after fusion, count the remaining top-level computations, and inspect buffer assignment for tensors that grew unexpectedly (Chen, 2020; Davies *et al.*, 2025). A fusion that succeeded in the HLO dump but still spills at runtime points to a scheduling or allocation interaction that the dump alone cannot show; that is the moment to profile DRAM traffic rather than read more IR (Chen, 2026a; Vellaisamy *et al.*, 2026). Compiler teams should archive dumps with the benchmark artifacts so a regression can be bisected to the exact pass change (Chen and YiRage Contributors, 2026; Chen, 2020).

15.2.5 Fusion legality in practice

Fusion legality in XLA is not a single check but a sequence of constraints that must all hold. The fused operations must agree on shapes and layouts, no operation inside the fusion may require an HBM round trip such as a collective or a custom call, and the resulting kernel must fit the target’s on-chip budget once tiling is chosen (OpenXLA Project, 2024; Chen, 2020). If any constraint fails, the pass either leaves the operator unfused or splits the fusion at the offending edge (OpenXLA Project, 2024; Chen, 2026a). Decode graphs expose these constraints more sharply than prefill graphs because thin elementwise chains sit between memory-bound GEMMs, so a layout mismatch that would cost little at large batch can strand a chain and add a launch per token (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). The production lesson is to inspect which fusions XLA actually formed at the decode shape rather than assuming the pass summary applies to every shape the team compiles (Davies *et al.*, 2025; Chen, 2026a).

15.2.6 Multi-output fusion and shared operands

Multi-output fusion lets one kernel produce several results that different consumers need, which matters when attention output feeds both a residual branch and a normalization path (OpenXLA Project, 2024; Vellaisamy *et al.*, 2026). Without

multi-output fusion the compiler would either duplicate work or write an intermediate once per consumer; with it, one fused kernel produces the shared values in registers or shared memory (OpenXLA Project, 2024; Chen, 2026a). Sibling fusion extends the same idea to operations that do not consume each other but share a producer, grouping them so the shared producer executes once (OpenXLA Project, 2024; Chen, 2020). Decode code review should look for these structures in the HLO dump because they are the XLA-native form of the boundary elimination described in earlier chapters (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). When multi-output fusion disappears between compiler versions, residual-add chains quietly regain an HBM round trip (Chen, 2026a; Davies *et al.*, 2025).

15.3 XLA codegen

Codegen in XLA is per backend. The CPU and GPU **backends** use LLVM for low-level IR and code generation: the CPU path **emits** LLVM IR lowered to x86/ARM native code, and the GPU path **emits** NVPTX IR lowered to PTX and then SASS (OpenXLA Project, 2024). For more advanced fusions involving matmul or softmax, XLA:GPU uses Triton as a code-generation layer—HLO fusions are converted to TritonIR, tiling parameters are selected, and Triton emits the PTX—so XLA and the Triton compiler of Chapter 17 share a codegen path (OpenXLA Project, 2024; Tillet *et al.*, 2019). For operations better served by vendor libraries, XLA pattern-matches to cuBLAS or cuDNN calls instead of codegen (OpenXLA Project, 2024).

The decode-critical codegen feature is launch capture. XLA:GPU groups a sub-sequence of compatible thunks into a **CommandBufferThunk** that records a GPU command buffer on first execution and replays it on every subsequent call, avoiding the CPU overhead of launching each kernel individually—structurally the same win as the CUDA Graphs of Chapter 8, built into the compiler (OpenXLA Project, 2024; NVIDIA, 2024). This is how XLA attacks launches/token: a decode step that eager execution issues as many launches becomes one replayed command buffer (Vellaisamy *et al.*, 2026; Chen, 2026a). The **platform** caveat is that capture requires static shapes and no illegal sync points, so a dynamically-shaped decode loop must be bucketed to stay capturable.

15.3.1 From HLO to thunks

The GPU backend lowers each top-level HLO computation into a sequence of thunks—executable units that know how to launch one piece of work on the device

Backend	Codegen at decode
GPU	LLVM/NVPTX + Triton emitters; command-buffer capture
CPU	LLVM to x86/ARM; cache blocking, vendor libs
TPU	XLA-only path; partitioning + bfloat16 passes

Table 15.1: XLA codegen focus per target at batch-1 decode.

(OpenXLA Project, 2024). A fused computation becomes one thunk; an unfused graph becomes a stream of thunks, one per remaining operation (OpenXLA Project, 2024; Vellaisamy *et al.*, 2026). This mapping is why fusion is the bytes lever and command buffers are the launch lever: fusion reduces the number of thunks, while command buffers reduce the host cost of submitting the thunks that remain (Chen, 2026a; NVIDIA, 2024). When reading an XLA profile, the thunk list is a better object than the original framework graph because it is exactly what the device executes. Decode teams should be able to answer how many thunks one token step produces after fusion and how many of those are replayable from a captured command buffer (Davies *et al.*, 2025; Chen, 2020).

15.3.2 Command-buffer capture semantics

The `CommandBufferThunk` records GPU work on first execution and replays it on later calls, but capture semantics matter more than the name suggests (OpenXLA Project, 2024; NVIDIA, 2024). Capture requires stable shapes, stable buffer addresses, and no operations that synchronize illegally or allocate inside the recorded region (OpenXLA Project, 2024; Chen, 2026a). A dynamic KV length therefore forces either a shape bucket boundary or a fallback execution path; the compiler should make that fallback explicit so telemetry can count how often a decode step runs uncaptured (Kwon *et al.*, 2023; Davies *et al.*, 2025). Capture also does not change arithmetic: replaying a captured sequence of thunks removes dispatch overhead but leaves every fusion boundary exactly where the earlier passes placed it (NVIDIA, 2024; Chen, 2026a). This separation is why XLA can simultaneously win on launches/token through capture and still lose on bytes/token if fusion was never legal at the decode shape.

15.3.3 Triton emission boundaries

XLA:GPU routes selected fusions through Triton rather than lowering everything through LLVM directly (OpenXLA Project, 2024; Tillet *et al.*, 2019). The compiler

chooses the Triton path when a fusion contains matmul or softmax patterns that benefit from tiled code generation, and it autotunes tile parameters as part of the compilation (OpenXLA Project, 2024; Davies *et al.*, 2025). Because Triton emission introduces its own layout and tiling choices, the same fusion can codegen differently across GPU SKUs; a config that wins on one shared-memory budget can spill on another (NVIDIA, 2022; Chen, 2026a). Decode teams fingerprinting XLA executables should therefore include the Triton autotune results in the artifact fingerprint, because two builds with identical HLO can still differ in emitted PTX (Chen and YiRage Contributors, 2026; Chen, 2020).

15.3.4 Vendor library and custom-call boundaries

XLA does not codegen every operation itself: large GEMMs and convolutions pattern-match to cuBLAS or cuDNN, and custom operations enter through custom calls when no native HLO lowering exists (OpenXLA Project, 2024). Each boundary is a potential fusion stop: a custom-call kernel cannot be fused through by the generic passes unless the backend knows its semantics (Chen, 2020; Davies *et al.*, 2025). FlashAttention-style custom kernels therefore sit at a boundary where XLA can orchestrate around them but cannot legally fuse through them (Dao *et al.*, 2022; OpenXLA Project, 2024). The engineering takeaway is to count custom-call boundaries in the HLO dump: a decode graph whose attention is one custom call and whose MLP is fused may still be launch-efficient, but its residency story ends where the custom call begins (Vellaisamy *et al.*, 2026; Chen, 2026a).

15.3.5 CPU and TPU decode codegen

The CPU backend lowers HLO to LLVM IR and then to native code, which makes XLA a legitimate baseline for CPU decode serving rather than only a GPU tool (OpenXLA Project, 2024; Goodfellow *et al.*, 2016). Cache blocking, vectorization width, and thread scheduling are the CPU analogs of GPU tiling, and XLA’s known-graph advantage applies there too: buffer assignment can keep an activation chain resident across operations that an eager runtime would spill between kernel boundaries (Goodfellow *et al.*, 2016; OpenXLA Project, 2024). On TPU, XLA is the only path, so every residency decision is a compiler decision; precision propagation passes such as bfloat16 handling and the TPU partitioning logic become part of the decode story (OpenXLA Project, 2024; Davies *et al.*, 2025). Teams that target TPU decode should therefore treat XLA version pinning as more critical than on GPU, where a fallback eager or Triton path exists (Davies *et al.*, 2025; OpenXLA Project, 2024). The chapter-level lesson stands across all

three: the compiler’s automated choices are excellent until the operating point changes, so the artifact and its profile must be versioned together (Chen, 2026a, 2020).

15.3.6 Streams, collectives, and multi-device decode

XLA partitions work across streams and devices when the graph contains collectives, which matters for tensor-parallel and pipeline-parallel decode even though this book keeps its core analysis at batch-1 single device (OpenXLA Project, 2024; Davies *et al.*, 2025). The latency-hiding scheduler overlaps communication with compute, and its decisions can change buffer lifetimes in ways that raise peak memory (OpenXLA Project, 2024; Chen, 2026a). For decode, the risk is that a scheduler tuned for throughput parallelism increases per-token latency by delaying a dependent attention step behind a collective (Davies *et al.*, 2025; Liu *et al.*, 2024). Multi-device XLA configurations therefore need their own mode matrix entry: the same model at the same context bucket can have a different optimal schedule for TP=1 decode and TP=4 decode (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Compile the partitioned graph, not the single-device graph, when benchmarking fleet configurations (Chen, 2020; OpenXLA Project, 2024).

15.4 XLA tuning

XLA’s **tuning** is mostly automatic, which is its defining contrast with TVM: where TVM searches a large schedule space with AutoTVM and Ansor, XLA relies primarily on heuristics and a cost model and exposes autotune flags for a narrower set of choices such as GPU tiling for Triton-emitted fusions (OpenXLA Project, 2024; Chen *et al.*, 2018b). This makes XLA low-effort—a JAX or TensorFlow user benefits without writing schedules—but it also means the engineer’s leverage is in **profile**-guided choices and flags rather than in writing kernels (OpenXLA Project, 2024; Davies *et al.*, 2025).

The **pitfall** is the operating point, as with every compiler in this Part. XLA optimizes whatever you compile and **profile**, so compiling on prefill-shaped, large-batch graphs produces fusions and layouts tuned for throughput that break at batch-1 decode, where the GEMM is a matrix-vector product and the launch floor dominates (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). The fix is mechanical: compile and **profile** at the decode shape (BS=1, realistic context), decompose ms/token into bytes/token and launches/token, and only then trust the autotuned result (Davies *et al.*, 2025; Chen, 2026a). The right configuration is **hardware-**

specific—a Triton-emitted fusion’s tiling that fits one GPU’s shared memory does not transfer to another—so re-tune per SKU and fingerprint configs so a fleet rollback replays the same HLO snapshot (NVIDIA, 2022; OpenXLA Project, 2024).

A second **pitfall** appears on mixture-of-experts models. MoE graphs have data-dependent routing, which both enlarges XLA’s autotune search and risks the compiler choosing layouts that page experts inefficiently if it is not given expert-aware constraints (Liu *et al.*, 2024; Davies *et al.*, 2025). Because XLA leans on heuristics rather than exhaustive search, a poorly-constrained MoE compilation can settle on a schedule that looks fine on a dense **profile** but thrashes when real routing skews load across experts (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). The practical discipline is to cap autotune wall time, constrain the search with model-structure hints, and require a decode-cell replay on realistic routing before promoting any winning config to the fleet—the same regression gate Chapter 24 applies across targets.

15.4.1 The tuning surface

XLA exposes a smaller tuning surface than search-based compilers, but the surface still spans several coupled decisions: which fusions Triton emits, what tile shapes those fusions use, whether command buffers capture a given thunk sequence, and whether rematerialization or the latency-hiding scheduler reorders work (OpenXLA Project, 2024; Davies *et al.*, 2025). Each knob has a compiler flag or configuration entry, and each interacts with the others, so tuning one in isolation usually produces a locally better compile and a globally worse decode profile (Chen, 2020, 2026a). The right unit of tuning is the full executable artifact compiled for one model, one context bucket, and one GPU SKU, fingerprinted with the HLO dump and the flags that produced it (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). Teams that tune flags in production without fingerprinting cannot roll back a regression to a known-good configuration.

15.4.2 Decode-cell tuning workflow

The workflow that keeps XLA honest starts with an eager or cuDNN baseline at the exact decode shape, then compiles the same model through XLA and compares three counters: ms/token, launches or thunks per token, and bytes/token (Davies *et al.*, 2025; Chen, 2026a). If XLA wins on latency but bytes/token is flat, the win is orchestration (command buffers), not residency; if bytes/token drops, fusion is doing the work the book cares about (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022). Each candidate configuration should be evaluated at multiple context buckets

because a tile chosen for 2K context can spill or underutilize at 32K (Kwon *et al.*, 2023; SemiAnalysis, 2024). The workflow belongs in CI with the benchmark artifact stored next to the compiler version, so a future XLA upgrade is treated as a change to be evaluated, not an ambient dependency to accept (Chen, 2020; Chen and YiRage Contributors, 2026).

15.4.3 Bucketing dynamic decode shapes

Dynamic shapes are the main obstacle to both Triton tiling stability and command-buffer capture (OpenXLA Project, 2024; Kwon *et al.*, 2023). XLA handles dynamism either by padding to a static shape or by recompiling per shape, and both paths cost bytes or latency (OpenXLA Project, 2024; Davies *et al.*, 2025). Decode serving should therefore bucket context lengths—for example at powers of two with a safety margin—and compile one executable per bucket (Vellaisamy *et al.*, 2026; Chen, 2026a). The bucket set is a product decision: too few buckets waste capacity on padding, too many buckets multiply compile time and graph memory (Kwon *et al.*, 2023; Chen, 2020). Telemetry should report which bucket each request used and how often padding exceeded a configured bound, because padding waste is invisible in a profile that only reports ms/token (Davies *et al.*, 2025; Chen, 2026a).

15.4.4 A/B methodology across XLA versions

Compiler upgrades deserve the same A/B discipline as kernel rewrites. The minimal A/B pins the model checkpoint, tokenizer, weights, context bucket, driver, and both compiler versions, then compares the three decode counters on identical inputs (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Because XLA fusion output changes with layout-assignment heuristics, a semantic no-op upgrade can still produce different kernels; the HLO dumps make that visible before fleet rollout (OpenXLA Project, 2024; Chen, 2020). Release notes for a decode stack should therefore list the XLA commit and the byte-sheet delta, not merely “upgraded XLA” (Chen and YiRage Contributors, 2026; Chen, 2026a). When the delta is unfavorable, the runbook needs a known-good compiled artifact to fall back to, which is why executable fingerprinting is a deployment requirement rather than a profiling nicety (OpenXLA Project, 2024; Vellaisamy *et al.*, 2026).

15.4.5 Routing-aware compilation for MoE

MoE decode adds a data-dependent axis that XLA’s heuristics must account for: the router selects a subset of experts per token, and the compiler cannot know the

selection statically (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). XLA handles such graphs either by padding all experts into a static layout or by keeping routing logic at a boundary where the runtime can choose (OpenXLA Project, 2024; Davies *et al.*, 2025). Static padding is simple and capturable but pays full expert bytes even when the router selects a fraction of them; a runtime-selected path saves bytes but complicates command-buffer capture (Kwon *et al.*, 2023; Chen, 2026a). The tuning consequence is that MoE decode cannot be tuned on dense-shaped profiles: the configuration must be evaluated with realistic routing distributions at the deployment batch and context (Liu *et al.*, 2024; Davies *et al.*, 2025). Compile-time hints that separate the router from the expert body give XLA the structure it needs without hand-writing kernels, which is the middle path this book recommends for MoE serving (Chen, 2020; Chen and YiRage Contributors, 2026).

15.5 XLA case study

The clearest **case** is an activation like GELU. In eager execution a GELU expands into seven or more separate kernel launches—each elementwise op its own kernel, each writing its intermediate to HBM; XLA fuses the entire GELU into one kernel that reads its input once and writes its output once (OpenXLA Project, 2024; Vellaisamy *et al.*, 2026). **Benchmarked** at the decode operating point, the win is not a faster multiply but the removal of six HBM round trips and six launches, exactly the bytes/token and launches/token levers the book tracks (Chen, 2026a; Dao *et al.*, 2022). Extending the same workflow—compile with fusion, diff the HLO before and after, validate command-buffer capture on the fused decode step—turns “XLA is faster” into a measured, reproducible claim.

The **cross-hardware** reading is where XLA’s breadth helps and stops. Because the same StableHLO export compiles to CPU (LLVM, cache-blocked), GPU (LLVM/Triton, captured), and TPU (XLA-only), a team can A/B the identical graph across targets on one decode-cell methodology and compare counters rather than FLOPs (OpenXLA Project, 2024; Davies *et al.*, 2025). But XLA does not target static NPU fabrics, and its automatic fusion can miss residency a search-based or hand-fused kernel would find, so it is one strong tool in the matrix, not the universal answer (AMD, 2024b; Wu *et al.*, 2024). **YiRage** encodes the residency rules XLA’s passes leave implicit as checkable IR metadata, so a fusion that would spill is rejected at compile time across backends rather than discovered in production (Chen and YiRage Contributors, 2026).

15.5.1 Worked decode step: one transformer layer

Trace one Llama-class decode layer as XLA would compile it. The framework emits a StableHLO graph containing the QKV projection, score GEMM, softmax, attention output projection, residual add, normalization, and the MLP up/gate/down projections (Dao *et al.*, 2022; Kwon *et al.*, 2023; Davies *et al.*, 2025). In an eager serving stack, the same subgraph often becomes more than a dozen host-visible kernels with HBM round trips between them, which is the regime TaxBreak counts as hundreds of launches per output token on dense models (Vellaisamy *et al.*, 2026; Chen, 2026a). XLA’s elementwise and multi-output fusion can collapse the projections, residual, and norm chains into a small number of fused computations, and command-buffer capture then replays those computations without per-kernel dispatch (OpenXLA Project, 2024; NVIDIA, 2024). The measured claim is the important one: the layer is shippable only if the compiled executable shows lower launches/token and no higher bytes/token than the eager baseline at the same context bucket (Chen, 2026a; Davies *et al.*, 2025).

15.5.2 What fusion does not fix in the worked step

The same trace exposes XLA’s boundary. If attention is a custom call or if the softmax path materializes its probability tensor because the emitted graph was not shaped for online softmax, then fusion stops at that boundary and bytes/token stays high no matter how many elementwise chains collapse around it (Dao *et al.*, 2022; OpenXLA Project, 2024; Chen, 2026a). A second boundary appears at the MLP seam: fused attention output written to HBM before the following GEMM reintroduces a full-width round trip that no capture win removes (Chen, 2026a; Vellaisamy *et al.*, 2026). The review gate for the worked step is therefore the HLO diff plus the DRAM counter, not the compiled-binary existence: an executable that compiles and replays can still encode the very round trips the decode optimization promised to remove (Chen, 2020; Davies *et al.*, 2025). Engineers should treat those cases as evidence that the source graph needs restructuring—for example exporting online softmax or a fused attention region—rather than as evidence that XLA is slow (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022).

15.5.3 Replicating the case study

The case study is reproducible from any framework XLA supports. From JAX, write the layer with explicit shapes, compile at the decode bucket, and inspect the executable plus HLO dump; from TensorFlow, mark the cluster for compilation

and profile the fused program; from torch-xla, move the decode loop into a traced context and compare against the eager baseline (OpenXLA Project, 2024; Davies *et al.*, 2025). Every replication should record the model checkpoint, context bucket, compiler version, and the three counters before the experiment starts, because those four fields are what make the result comparable across teams (Chen, 2026a; Vellaisamy *et al.*, 2026). The same replication script should run on CPU and GPU locally and, where available, on TPU, turning the XLA matrix claim into a triplet regression instead of a vendor slide (OpenXLA Project, 2024; Chen, 2020). A decode team that cannot reproduce the GELU-level win on its own model should suspect a boundary in the exported graph, not the compiler.

15.5.4 Promotion worksheet

Before promoting an XLA-compiled decode executable, confirm the worksheet has five entries: the exact StableHLO and HLO versions, the bucket set with measured padding waste, the Triton autotune and command-buffer configuration fingerprint, the three-counter comparison against the eager baseline at every bucket, and a known-good artifact for rollback (OpenXLA Project, 2024; Chen, 2026a; Davies *et al.*, 2025). Each entry should be machine-generated from the build so the worksheet cannot drift from what actually shipped (Chen and YiRage Contributors, 2026; Chen, 2020). If any entry is missing, the promotion is a gamble that will be resolved later by a fleet incident (Vellaisamy *et al.*, 2026; Chen, 2026a). This worksheet is the XLA instance of the general compiler-deployment contract the book repeats: pin, profile, diff, and keep the rollback path warm (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

15.5.5 MoE scenario and launch collapse

Apply the case-study method to an OLMoE-class MoE model whose eager profile shows thousands of launches per output token (Vellaisamy *et al.*, 2026). The XLA compile should first collapse the dense attention and shared layers with fusion and capture; the expert body then becomes the remaining tuning question (OpenXLA Project, 2024; Davies *et al.*, 2025). If the router and expert gather-scatter stay outside the captured region, the serving stack keeps an eager hull around a captured core—the hybrid pattern from Chapter 12—and the measured win is a large launch reduction in the shared portion (NVIDIA, 2024; Vellaisamy *et al.*, 2026). The byte ledger must still show expert weights and activations moving efficiently, because a launch collapse that pads every expert into the graph can increase bytes/token even as latency improves (Chen, 2026a; Kwon *et al.*, 2023). Publish the MoE scenario

with the same counters as the dense scenario so fleet reviewers can compare like for like (Davies *et al.*, 2025; Chen, 2020).

15.5.6 Failure autopsies for XLA decode

Three failure signatures recur in XLA decode rollouts, and each maps to a different cause. Flat latency with rising DRAM traffic points to fusion boundaries that did not hold at the decode shape, visible in the HLO dump as unfused seams around attention or residual chains (Chen, 2026a; OpenXLA Project, 2024). First-token latency spikes after a warm fleet point to cache misses, recompilation, or command-buffer re-capture triggered by a shape or allocator change (Davies *et al.*, 2025; NVIDIA, 2024). A configuration that wins on one GPU and loses on another points to Triton tile autotune results that did not transfer, which the artifact fingerprint should have caught before promotion (OpenXLA Project, 2024; NVIDIA, 2022). Each autopsy should start from the pinned executable and HLO dump, not from framework-level timers, because the compiler artifacts are the only record of what actually ran (Chen, 2020; Chen and YiRage Contributors, 2026). Teams that archive those artifacts turn the third failure into a config diff and the first two into pass-ordering bugs instead of folklore (Chen, 2026a; Vellaisamy *et al.*, 2026).

15.5.7 Serving XLA decode in production

The operational layer around XLA matters as much as the compiler passes. Every serving release should record the compiler commit, StableHLO version, autotune fingerprint, and compiled executable hash in the same artifact store as the model weights (OpenXLA Project, 2024; Davies *et al.*, 2025; Chen, 2026a). Startup should load precompiled executables for the shipped bucket set rather than compiling on first request, and telemetry should expose bucket hit rate, padding waste, command-buffer replay rate, and fallback-to-eager rate as first-class metrics (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2020). On-call runbooks need a one-command rollback to the previous known-good executable, not a recompile under incident pressure (Chen and YiRage Contributors, 2026; Chen, 2026a). Teams that treat XLA as part of their deployable stack get deterministic decode; teams that treat it as ambient infrastructure get whichever compile the cache happened to hold (Davies *et al.*, 2025; OpenXLA Project, 2024). The same principle applies to Triton-emitted fusions: their tile decisions belong to the shipped artifact and must be reproducible in CI, or the next compile can silently change decode behavior without any model change (OpenXLA Project, 2024; Davies *et al.*, 2025).

15.6 Key Takeaways

- **XLA is automatic, broad, and TPU’s only path.** StableHLO $\rightarrow$ HLO $\rightarrow$ LLVM/PTX/TPU compiles one graph to CPU, GPU, and TPU without hand-written kernels (OpenXLA Project, 2024; Lai *et al.*, 2023). Automatic means it optimizes whatever you profile, so point it at decode (Davies *et al.*, 2025).
- **Fusion is the bytes/token lever.** XLA’s fusion invariant—no intermediate in HBM, one fusion equals one kernel, driven by PriorityFusion—is exactly the on-chip residency decode needs (OpenXLA Project, 2024; Chen, 2026a). Gate each fusion pass on buffer accounting (Vellaisamy *et al.*, 2026).
- **Command-buffer capture is the launches/token lever.** `CommandBufferThunk` records and replays a GPU command buffer, removing per-kernel host dispatch like CUDA Graphs, but requires static shapes (OpenXLA Project, 2024; NVIDIA, 2024). Bucket dynamic decode shapes to stay capturable (Kwon *et al.*, 2023).
- **Codegen shares Triton and LLVM.** CPU/GPU use LLVM; advanced GPU fusions emit through TritonIR, so XLA and Triton share a codegen layer (OpenXLA Project, 2024; Tillet *et al.*, 2019). Vendor-library pattern-matching handles large GEMM/conv (NVIDIA, 2022).
- **Tune at the decode operating point.** XLA’s heuristics optimize the shape you compile, so prefill-shaped tuning breaks batch-1 decode; profile BS=1 ms/token and fingerprint configs per SKU (Davies *et al.*, 2025; OpenXLA Project, 2024). Re-tune per hardware (NVIDIA, 2022).

15.7 Conclusion

XLA shows that automatic, whole-graph compilation is production-viable at the largest scale: StableHLO portability, HLO fusion that keeps intermediates off HBM, scheduling that exploits the known graph, command-buffer capture that removes launch overhead, and LLVM/Triton codegen across CPU, GPU, and TPU (OpenXLA Project, 2024; NVIDIA, 2024). Its automation is its strength and its trap—it optimizes the operating point you give it—so the decode discipline is to compile and grade at batch-1 on bytes/token and launches/token, the same residency contract YiRage enforces as metadata (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). With XLA’s automatic approach established, Chapter 16

turns to TVM, which pursues the same multi-hardware goal through explicit search rather than heuristics, trading XLA’s low effort for a larger reachable schedule space (Chen *et al.*, 2018b; Zheng *et al.*, 2020).

Chapter 16

TVM and Apache TVM Stack

Apache TVM made one bet that still defines the stack: separate *what* a tensor operator computes from *how* it is scheduled onto silicon, then let a learning-based cost model search the schedule space instead of a vendor library author (Chen *et al.*, 2018b). That decoupling is why a single operator definition can lower to x86, ARM, CUDA, and FPGA back-ends without rewriting the math (Chen *et al.*, 2018b; Chen, 2020). It is also why teams misread TVM in production: they tune a model until prefill GEMMs look excellent, ship it, and then watch interactive batch-1 decode miss p99 anyway (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). The reason is the same one Chapter 1 established. Dense Llama-3.2-1B averages 847.5 GPU kernel launches per output token on H100 (BS=4, SL=2048); OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). A correct, well-tuned TVM operator schedule does nothing about that launch storm unless the compiler is also asked to fuse and capture the decode loop (Chen, 2026a; NVIDIA, 2024). This chapter treats TVM as a production residency contract under decode metrics—bytes/token and launches/token—not as a tutorial on schedule primitives (Davies *et al.*, 2025).

16.1 TVM HW matrix

TVM’s selling point is a hardware/target support **matrix**: write the compute once, lower it to many back-ends (Chen *et al.*, 2018b). In practice that matrix is necessary but not sufficient for decode. A target that *builds* is not a target that is decode-legal, and most “mysterious” regressions come from treating the two as the same thing (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

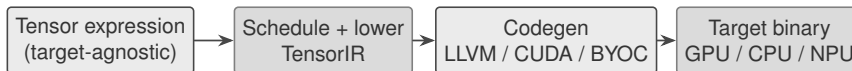


Figure 16.1: One tensor-expression definition fans out across the TVM hardware/target matrix: schedule and lower to TensorIR, then emit per-backend code.

The hardware truth underneath the matrix is the usual split. GPUs bound a schedule by shared-memory capacity and tensor-core feeding; CPUs by cache blocking and NUMA locality; NPUs and XDNA/AIE fabrics by static tile SRAM and DMA chains that must be assigned at compile time (NVIDIA, 2022; AMD, 2024b; Goodfellow *et al.*, 2016). At batch-1, decode stays bandwidth- and orchestration-limited on all three—the per-token matmul is too thin to amortize either HBM traffic or launch overhead (Chen, 2026a; Vellaisamy *et al.*, 2026).

TVM earns the matrix because the same tensor expression carries no target assumptions: the compute is declared once, and a separate schedule chooses tiling, vectorization, thread binding, and memory scope per target (Chen *et al.*, 2018b). That is genuinely more than vendor libraries deliver, where each new SKU or accelerator demands a hand-written kernel (Chen, 2020). The catch for serving is that the matrix is graded at *build* time—does the target lower and run—while decode goodput is a *runtime* property of bytes moved and kernels launched. A green build matrix cell tells you the operator exists on that target; it tells you nothing about whether the lowered schedule keeps the decoder layer on-chip.

Multi-hardware delta. The same TVM operator definition takes a different legal schedule per class: a Hopper GPU wants TMA-fed double buffering and a captured launch graph; an x86 CPU wants cache-blocked tiles and SIMD; an XDNA NPU wants a static tile-to-DMA assignment with no dynamic dispatch at all (SemiAnalysis, 2024; AMD, 2024b,a). Identical math, three different bytes/token outcomes. This is exactly the residency reasoning YiRage encodes as checkable IR metadata so the matrix entry is rejected before it ships, not after p99 slips (Chen and YiRage Contributors, 2026).

16.1.1 Entering TVM from serving frameworks

The matrix is only reachable through an import path, and the import path decides how much of a decode loop TVM sees (Chen *et al.*, 2018b; Lai *et al.*, 2023). Relay and Relax both accept models from PyTorch, ONNX, TensorFlow, and native front ends, but the granularity of the exported graph determines what can later be fused and scheduled (Chen *et al.*, 2018b; Lai *et al.*, 2023). A PyTorch export

that keeps layer-norm, linear, attention, and MLP as separate framework ops gives the graph passes the seams they need; an export that already bakes everything into opaque custom calls hides those seams forever (Davies *et al.*, 2025; Chen, 2020). Teams should therefore verify the imported graph before tuning: count the top-level operators per decoder layer, confirm that dynamic shapes are represented rather than padded away silently, and confirm that stateful sampling kernels stayed outside the module (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Import quality is a compile-time property with runtime consequences.

16.1.2 Graph granularity and decoder layers

The unit TVM compiles is the whole imported module, but the unit that matters for decode is one decoder layer repeated many times (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). A well-structured import lets the graph passes identify the repeating layer pattern and compile it once as a fused call; a flat import treats every layer as unique and multiplies compile and tuning cost without adding any new information (Chen *et al.*, 2018b; Davies *et al.*, 2025). Relax improves on Relay here by keeping dynamic shapes and memory planning at graph level, so a decoder loop with varying KV length can be represented as one dynamic function rather than dozens of recompiled instances (Lai *et al.*, 2023; Kwon *et al.*, 2023). For production, the recommended practice is to export one decoder layer as a named function, tune and fuse it once, and let the serving runtime invoke that compiled function per token (Davies *et al.*, 2025; Chen, 2026a). This keeps the compiler’s work aligned with the unit of work the serving loop actually repeats.

16.1.3 Matrix versioning per SKU

The build matrix has a versioning problem that decode teams discover late: TVM’s legal schedules change with the TVM release, the target description, and the host toolchain, so a matrix cell that was green six months ago says little about today (Chen *et al.*, 2018b; Chen, 2020). Each cell should carry the compiler version, target string, runtime executor, and the measured decode counters from the last validation run (Chen, 2026a; Vellaisamy *et al.*, 2026). A build matrix with dates and artifact hashes turns “TVM supports XDNA” into a reproducible claim instead of a documentation promise (AMD, 2024b; Chen and YiRage Contributors, 2026). CI should regenerate the matrix on compiler upgrades and mark any cell whose decode bytes/token regressed before the release is promoted (Davies *et al.*, 2025; Chen, 2026a).



Figure 16.2: TVM’s two-level pass pipeline: graph-level Relax transforms feed loop-level TensorIR transforms inside one `IRModule`.

16.1.4 Target qualification cells

Because a matrix cell that builds is not a cell that serves, each supported target needs a qualification record with three parts: the build test (does the target lower and run the operator), the legality test (do the emitted schedules respect on-chip budgets and static constraints), and the decode-cell benchmark (what bytes/token and launches or DMA edges the artifact produces at served context buckets) (Chen *et al.*, 2018b; AMD, 2024b; Chen, 2026a). The legality test is where TVM differs most from vendor libraries: TVM’s schedule primitives are powerful enough to emit an illegal-for-decode kernel that still passes unit tests, so the qualification record must review the schedule, not only the result (AMD, 2024a; Chen and YiRage Contributors, 2026; Chen, 2020). CPU and GPU cells can be qualified with the same benchmark script; an NPU cell needs the script extended with static-allocation and DMA-edge checks (AMD, 2024b; Davies *et al.*, 2025). The matrix becomes trustworthy when every green cell carries the three artifacts and a date (Vellaisamy *et al.*, 2026; Chen, 2026a).

16.2 TVM arch passes

TVM runs optimization at two IR levels, and the production surprise is almost never a missing peak-FLOPs pass—it is an **optimization** that is legal in isolation but spills a fused producer to HBM between levels (Chen *et al.*, 2018b; Chen and YiRage Contributors, 2026). The modern stack makes the two levels explicit. A graph-level `relax.Function` carries the dataflow; a `tir::PrimFunc` carries the loop nest; both live in one `IRModule`, and `call_tir` lowers a graph node into a loop-level kernel in destination-passing style so intermediate buffers can be planned globally (Lai *et al.*, 2023). Relax replaced the older Relay model precisely because Relay treated tensor programs as opaque foreign functions and could not track dynamic shapes—or residency—across the boundary (Lai *et al.*, 2023).

This is a deliberately different choice from an open dialect tower. Where MLIR exposes many progressively lowered *dialects*, TVM commits to a fixed graph→TensorIR pair and pushes its leverage into schedule search rather than

dialect proliferation (Chen *et al.*, 2018b; Lai *et al.*, 2023). The engineering consequence: graph passes like operator fusion must be co-designed with the TensorIR **lowering** that follows, or fusion “succeeds” on paper while the lowered kernel still round-trips activations through DRAM.

Concretely, the graph-level passes that decide decode residency are a short list, and each has a loop-level consequence. **FuseOps** merges a chain of element-wise and reduction operators into one `call_tir` target so the lowered kernel keeps intermediates in registers or shared memory instead of HBM (Chen *et al.*, 2018b). **ToMixedPrecision** and layout-rewrite passes change how many bytes each activation costs and whether the tensor-core or vector path is even reachable on the target (Lai *et al.*, 2023). **StaticPlanBlockMemory** and the destination-passing-style contract behind `call_tir` let the compiler allocate one intermediate buffer and reuse it across the fused region, which is the difference between a decoder layer that streams its working set once and one that re-reads it per operator (Lai *et al.*, 2023; Chen, 2026a). Run these in the wrong order—layout after fusion, say—and the fused kernel is forced back through a transpose that re-materializes the very activations fusion was meant to keep on-chip.

What to diff before merge. Lower the same module twice and diff the generated TIR (or CUDA) for new buffer allocations; count launches/token on the GPU path and DMA edges on the NPU path (Vellaisamy *et al.*, 2026; AMD, 2024a). A silent spill introduced by a reordered pass reads downstream as an unexplained p99 regression, and graph-capture legality must hold on the GPU path while static-allocation legality must hold on XDNA (NVIDIA, 2024; AMD, 2024b).

16.2.1 Fusion boundaries at the graph level

Graph-level fusion in TVM must answer the same legality question as the theory chapter: which intermediates can stay on-chip between producer and consumer (Chen, 2020, 2026a). **FuseOps** works on chains where layout and data type match, and it stops at reduction boundaries, custom calls, and dynamic-shape operations that the scheduler cannot reason about (Chen *et al.*, 2018b; Lai *et al.*, 2023). Decode graphs are full of such stops: online softmax state crosses a reduction boundary inside attention, and attention output feeds a layout-sensitive MLP (Dao *et al.*, 2022; Davies *et al.*, 2025). The practical discipline is to enumerate those stops in the imported graph and decide which are intentional (custom kernels, routing) and which are import artifacts that should be fixed before compilation (Chen *et al.*, 2018b; Chen and YiRage Contributors, 2026). Every unintentional

stop is a future HBM round trip and a future launch.

16.2.2 Layout and memory planning passes

Relax’s graph passes include layout transformation and memory planning that run before TensorIR lowering, and their ordering determines whether a fused region materializes its intermediates (Lai *et al.*, 2023; Chen, 2020). A layout rewrite that runs after fusion can force the fused kernel to transpose through memory; memory planning that allocates one buffer per operator instead of reusing a single scratch region wastes HBM capacity without changing any arithmetic (Chen, 2026a; Lai *et al.*, 2023). The destination-passing-style contract behind `call_tir` is what lets the planner see the whole fused region and allocate once (Lai *et al.*, 2023). Teams reviewing TVM pass changes should diff the generated TIR for buffer count and allocation size, not just for loop structure, because buffer regressions are the silent form of residency loss (Chen, 2026a; Davies *et al.*, 2025).

16.2.3 Lowering checklist for one decode layer

Before accepting a TVM lowering for a decoder layer, check four artifacts: the fused operator count per layer in the Relax module, the generated TIR loop structure for any remaining per-operator buffers, the emitted kernel count on the target, and the captured or static launch plan that wraps those kernels (Chen *et al.*, 2018b; NVIDIA, 2024; AMD, 2024a). The same checklist applies on every target with target-specific wording: GPU uses kernels and command capture, CPU uses threads and cache tiles, NPU uses DMA edges and static tiles (AMD, 2024b; Goodfellow *et al.*, 2016; NVIDIA, 2022). If any of the four artifacts is missing, the lowering is incomplete regardless of how well the microbenchmark scores (Vellaisamy *et al.*, 2026; Chen, 2026a). This checklist is short enough to be a CI gate and precise enough to catch the regressions that pass-level summaries hide (Chen, 2020; Davies *et al.*, 2025).

16.3 TVM codegen

Codegen is where the matrix meets Chapter 1’s counters. TVM emits through several back-ends: LLVM for x86 and ARM CPUs, CUDA/NVRTC (or source handed to NVCC) for NVIDIA GPUs, ROCm for AMD, and Bring-Your-Own-Codegen (BYOC) to **emit** calls into vendor libraries or an NPU runtime for any **platform** TVM does not natively target (Chen *et al.*, 2018b; Chen, 2020). The

backend you pick decides the unit of work the host must launch.

The trap is to grade codegen by the quality of a single emitted matmul. At batch-1 decode the emitted MMA is rarely the limiter; the limiter is how many distinct kernels the host launches per token and how many HBM round trips survive lowering (Vellaisamy *et al.*, 2026; Chen, 2026a). A backend that emits a beautiful per-operator kernel but leaves a decoder layer as a dozen separate launches loses to a coarser fused kernel that the runtime can capture once. On the GPU path this is why graph capture matters: replaying a captured launch sequence removes per-kernel host dispatch and recovered roughly $1.26\times$ on a Qwen-2.5-7B batch-1 decode cell on H100 in the Memory-Floor study—without changing a single MMA tile (Chen, 2026a; NVIDIA, 2024). On the CPU path the equivalent win is fusing through cache-resident tiles so the emitted code never spills the working set; on an NPU it is emitting a static DMA schedule rather than dynamic dispatch (AMD, 2024b,a).

The other codegen decision that decode telemetry punishes is the runtime that wraps the emitted kernels. TVM can package a model as a graph executor, a relax VM, or an ahead-of-time (AOT) module, and each carries a different per-token host cost (Chen *et al.*, 2018b; Lai *et al.*, 2023). A Python-driven graph executor adds interpreter and dispatch overhead on every operator—tolerable for batched prefill, fatal for batch-1 decode where that overhead is paid once per token with nothing to amortize it against (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). The AOT path and a captured launch graph collapse that orchestration into a single replayable sequence, which is the codegen-level lever that actually moves ms/token (NVIDIA, 2024; Chen, 2026a).

Multi-hardware delta. The CUDA backend can hide latency with asynchronous copy and double buffering, so its codegen can afford a dynamic schedule; the XDNA/AIE backend cannot, because its tiles and DMA descriptors are fixed before runtime (SemiAnalysis, 2024; AMD, 2024a). The same Relax module therefore emits genuinely different control flow per platform, and grading both against one microbenchmark hides the decode cost (Goodfellow *et al.*, 2016).

16.3.1 BYOC and accelerator codegen

Bring-Your-Own-Codegen is TVM’s escape hatch for targets its native emitters do not cover, and it is the mechanism an XDNA/AIE or vendor-NPU port would use (Chen *et al.*, 2018b; AMD, 2024b). BYOC partitions the graph into regions a foreign compiler or runtime understands and emits calls into that runtime instead of

TVM kernels (Chen *et al.*, 2018b; Chen, 2020). For decode the partition boundary is the entire design problem: each boundary crossing is a host-call and a potential buffer copy, so the partitioner should prefer whole fused regions over per-operator handoffs (Davies *et al.*, 2025; Chen, 2026a). A foreign runtime that only accepts prefill-shaped kernels will force the decode loop into a slow generic fallback unless the partitioner knows the decode shape (AMD, 2024b; Vellaisamy *et al.*, 2026). Teams should treat BYOC support as a contract with the same residency review as native codegen: which tensors stay inside the foreign region, which cross it, and what the crossing costs (Chen and YiRage Contributors, 2026; Chen, 2020).

16.3.2 Executor choice is a decode decision

The executor that drives emitted kernels is part of codegen for serving purposes. The classic graph executor walks a pre-serialized graph and dispatches operators through a runtime, which is flexible but pays dispatch cost per node; the Relax VM compiles the module to bytecode with a smaller runtime footprint; ahead-of-time compilation links the module into a standalone binary with the least per-call overhead (Chen *et al.*, 2018b; Lai *et al.*, 2023). For batch-1 decode, where one token step traverses the whole decoder graph, the per-node dispatch of the graph executor reappears as launch-like overhead on every token (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). Measuring the executor cost requires comparing identical compiled kernels under each runtime at the decode shape; a difference that is invisible at batch 32 becomes the p99 floor at batch 1 (Davies *et al.*, 2025; Chen, 2026a). Choose the executor with the serving loop, not with the training loop, in mind (Chen *et al.*, 2018b; Vellaisamy *et al.*, 2026).

16.3.3 Capture, driver, and artifact pinning

Whatever executor TVM uses, the GPU path should capture the compiled launch sequence so per-token host cost is paid once (NVIDIA, 2024; Chen, 2026a). Capture is only stable when shapes, buffer addresses, and allocator behavior are stable, so the decode loop must be bucketed and the captured sequence regenerated at bucket boundaries (Kwon *et al.*, 2023; Davies *et al.*, 2025). The emitted kernels themselves depend on the TVM build, the CUDA driver, and the target description, so a serving artifact should pin all three and record them in the release manifest (Chen, 2020; Vellaisamy *et al.*, 2026). When the driver or TVM changes, the capture must be re-validated at every served context bucket before promotion (Chen, 2026a; Chen and YiRage Contributors, 2026). This pinning is the TVM instance of the general compiler-deployment rule: the artifact that runs is the artifact that must

be reproduced (Davies *et al.*, 2025; Chen, 2026a).

16.3.4 Host-device transfer boundaries

Decode graphs contain small tensors that some framework paths copy eagerly between host and device, and those copies are invisible in kernel-level benchmarks (Kwon *et al.*, 2023; Davies *et al.*, 2025). Sampling parameters, router outputs, and per-request metadata frequently cross the boundary once per token, adding latency that scales with round-trip count rather than tensor size (Vellaisamy *et al.*, 2026; Chen, 2026a). TVM codegen should keep such values on-device wherever the control flow allows, and the serving integration should batch metadata updates outside the steady-state loop (Chen *et al.*, 2018b; Davies *et al.*, 2025). Stream ordering compounds the problem: if TVM emits kernels on a default stream while the runtime synchronizes per step, the synchronization defeats command-buffer capture (NVIDIA, 2024; Chen, 2026a). The codegen review should therefore include the host-side driver code, not only the emitted device kernels, because the boundary is where decode latency hides after fusion is perfect (Chen, 2020; Davies *et al.*, 2025).

16.4 TVM tuning

TVM’s auto-tuning lineage is the part most teams adopt and the part most likely to mislead. AutoTVM came first: a template-based tuner that learns a statistical cost model (gradient-boosted trees or TreeGRU) over hand-written schedule templates, uses transfer learning across workloads, and explores with simulated annealing—reporting $1.2\text{--}3.8\times$ end-to-end gains over hand-tuned libraries on CPU, mobile GPU, and server GPU (Chen *et al.*, 2018a). Ansor removed the templates: it samples complete programs from a hierarchical search space, fine-tunes with evolutionary search and a learned cost model, and uses a task scheduler to spend **tuning** budget where it moves end-to-end latency most (Zheng *et al.*, 2020). MetaSchedule later unified these search strategies over TensorIR.

The search budget is itself a hardware-bound decision. AutoTVM’s transfer learning lets a cost model trained on one workload warm-start another, which matters because each candidate schedule must be compiled and measured on real silicon—thousands of trials per operator on a GPU, far more on a slow edge target (Chen *et al.*, 2018a). Ansor’s task scheduler spends that budget where it changes end-to-end latency rather than spreading it uniformly, so a long-tail operator that dominates decode is not starved while a cheap one is over-tuned (Zheng *et al.*, 2020).

Table 16.1: Ansor template-free auto-scheduling: reported maximum end-to-end DNN speedups over prior search/library baselines, by hardware class (Zheng *et al.*, 2020).

Hardware class	Representative target	Max speedup
Server / desktop CPU	Intel x86 (AVX-512)	$3.8\times$
Edge / mobile CPU	ARM (NEON)	$2.6\times$
Server GPU	NVIDIA (CUDA)	$1.7\times$

For a serving team the practical reading is that auto-tuning is not free wall-clock time; it is a measured search whose payoff depends on tuning the operators that actually appear on the batch-1 decode critical path.

The **pitfall** is the operating point. Auto-tuning maximizes whatever workload you **profile** against, and the default reflex is to tune on large, batched, prefill-shaped GEMMs because they are easy to benchmark and show big numbers (Kwon *et al.*, 2023). Schedules tuned at that shape over-tile for throughput and assume launch cost is amortized across many tokens—both assumptions break at batch-1 decode, where the GEMM is a thin matrix-vector product and the launch floor dominates (Vellaisamy *et al.*, 2026; Chen, 2026a). The fix is mechanical: tune on the decode operating point (BS=1, realistic context), and rank candidates by measured ms/token after decomposing bytes/token and launches/token, not by the tuner’s raw cost-model score on a synthetic shape (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). The same discipline differs by **hardware** class: GPU tuning chases coalescing and occupancy, CPU tuning chases cache locality and vector width, NPU “tuning” is mostly static tile assignment with little runtime freedom (AMD, 2024b; NVIDIA, 2022).

16.4.1 Search objective for decode

AutoTVM and Ansor measure candidate schedules on a reward that defaults to end-to-end latency for a given workload shape, and the entire search inherits whatever that shape implies (Chen *et al.*, 2018a; Zheng *et al.*, 2020). At the decode operating point, the reward should be composed from the same counters the rest of this book uses: bytes/token first, then launches or DMA edges per token, then measured ms/token at the served context bucket (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). A schedule that minimizes a synthetic cost-model score on a batched shape will rank differently when the real objective is batch-1 memory traffic (Kwon *et al.*, 2023; Dao *et al.*, 2022). TVM lets users provide the measurement callback for search, so serving teams should wire the decode-cell

benchmark into the tuner rather than accepting the default benchmark builder (Chen *et al.*, 2018a; Zheng *et al.*, 2020). Search is only as honest as the workload it measures.

16.4.2 MetaSchedule and the modern stack

MetaSchedule unifies the earlier search strategies over TensorIR, providing program sampling, evolutionary search, and a learned cost model inside the current TVM workflow (Chen *et al.*, 2018b; Zheng *et al.*, 2020). For decode teams the relevant property is that MetaSchedule still searches within what the target description and pass pipeline allow: it can find a better tiling for an attention fusion, but it cannot fuse an operator that graph passes left outside the region (Chen *et al.*, 2018b; Davies *et al.*, 2025). The division of labor is therefore graph passes decide residency, MetaSchedule decides the best legal schedule for the residency that exists (Lai *et al.*, 2023; Chen, 2026a). Upgrading TVM versions changes both layers, so a regression after an upgrade should be bisected between graph-pass output and schedule-search results before blaming either (Chen, 2020; Davies *et al.*, 2025).

16.4.3 Budget management and measurement

Autotuning budget is wall-clock time on real hardware, and the task scheduler exists precisely because operators are not equally important (Zheng *et al.*, 2020; Chen *et al.*, 2018a). The decode-critical operators are the ones on the per-token path: attention projections, KV-loop scores, softmax, output projection, and the MLP block (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Operators that run once per prefill or outside the steady-state loop deserve less budget (Davies *et al.*, 2025; Chen, 2026a). The budget ledger should record trials per operator, the measured reward trend, and the final schedule fingerprint so a later search can resume rather than restart (Chen *et al.*, 2018a; Chen, 2020). Teams that treat tuning as an overnight job with no objective record cannot answer whether a later regression came from the search, the compiler, or the workload change (Vellaisamy *et al.*, 2026; Chen, 2026a).

16.4.4 MoE-aware search constraints

Mixture-of-experts graphs distort tuning in two directions: the expert body multiplies the number of operators that could each be tuned, and data-dependent routing makes any single-shape measurement unrepresentative (Liu *et al.*, 2024; Kwon *et al.*, 2023). Search should constrain the expert region to a canonical schedule

tuned once and replicated, leaving budget for the router and the shared attention path (Davies *et al.*, 2025; Chen *et al.*, 2018b). Routing skew means the reward should be measured on request mixes, not on a single synthetic batch (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024). If the search cannot see routing distributions, its best schedule optimizes the average case the tuner invented rather than the fleet case (Chen, 2026a; Davies *et al.*, 2025). The practical fix is to feed the tuner the serving trace and rank candidates by p99 over that trace, not by mean latency over a uniform synthetic set (Kwon *et al.*, 2023; Davies *et al.*, 2025).

16.4.5 Re-tuning cadence and upgrade triggers

Search results decay on the same triggers as every other compiler decision: TVM upgrades, target-description changes, driver updates, model architecture changes, and new context buckets all invalidate previous schedule fingerprints (Chen *et al.*, 2018b; Chen, 2026a; Davies *et al.*, 2025). A lightweight nightly job should re-measure the pinned decode artifact and alert when bytes/token or launches/token drift beyond tolerance (Vellaisamy *et al.*, 2026; Chen, 2020). The alert triggers a bounded re-search over the affected operators with the previous schedule as a seed, not a full re-tune of the model (Zheng *et al.*, 2020; Chen *et al.*, 2018a). Full re-tunes belong on a release cadence with the artifact manifest updated atomically, so a bad search can be rolled back to the prior schedule in one step (Chen, 2026a; Chen and YiRage Contributors, 2026). This cadence converts autotuning from an event into a maintained contract, which is the only model that survives framework churn and team turnover (Davies *et al.*, 2025; Chen, 2026a).

16.5 TVM case study

Make the contradiction concrete with one decoder layer as the **case**. Tune it with Ansor on three targets and **benchmark** at the decode operating point, not the tuner’s default (Zheng *et al.*, 2020; Vellaisamy *et al.*, 2026). The cross-hardware reading is consistent: per-operator schedules that win an isolated GEMM benchmark can still lose end-to-end because each operator is a separate launch and each unfused boundary is an HBM round trip (Chen, 2026a; Dao *et al.*, 2022). The lever that moves p99 is fusing the layer so activations stay on-chip and capturing the launch sequence so the host stops paying per-kernel dispatch every token (NVIDIA, 2024; Vellaisamy *et al.*, 2026).

The arithmetic makes the lever concrete. Take a decoder layer at hidden size $d=4096$ in BF16, where each materialized activation vector costs $4096 \times 2 = 8$ KB.

An unfused schedule that writes and re-reads roughly six such intermediates—layer-norm output, Q/K/V projections, attention output, MLP input—per token spends about 48 KB/layer/token of avoidable HBM traffic before the KV cache is even touched (Chen, 2026a; Dao *et al.*, 2022). Fusing the layer so those vectors stay on-chip removes that traffic outright; at 32 layers it is on the order of 1.5 MB/token of HBM reads and writes that simply disappear, which is why bytes/token moves even when tensor-core utilization looks unchanged (Vellaisamy *et al.*, 2026). No tuner finds this if it is pointed at a single isolated GEMM.

This **cross-hardware** story is the book’s recurring thesis: the optimization rank differs by silicon, so no single tuned artifact is portable in the way the build matrix suggests (Davies *et al.*, 2025). On the GPU the win is fusion plus graph capture; on the CPU it is cache-resident fusion and SIMD; on the XDNA NPU it is a static tile/DMA schedule with the fusion decided entirely at compile time (AMD, 2024b,a). **YiRage** operationalizes the case: it carries the Chapter 2 residency rules as checkable IR metadata, so a TVM-style schedule that would spill between graph and TensorIR levels is rejected at compile time across CUDA, CPU, and NPU back-ends rather than discovered in production telemetry (Chen and YiRage Contributors, 2026).

16.5.1 Layer schedule experiment

Run the layer experiment the way the earlier arithmetic suggests. Export one decoder layer, fuse it at the Relax level, schedule the fused region with MetaSchedule or Ansor at the batch-1 decode shape, and then compare three artifacts against an eager PyTorch baseline: the fused operator count, the emitted kernel or DMA-edge count, and bytes/token at the served context buckets (Chen *et al.*, 2018b; Zheng *et al.*, 2020; Davies *et al.*, 2025). The experiment should be repeated after changing the TVM version, the target string, or the driver, because each change can silently alter which fusions survive lowering (Chen, 2026a, 2020). Record the winning artifact hash in the deployment manifest. The result of the experiment is not a speedup number but a reproducible claim that this layer, on this target, at this context, moves these counters (Vellaisamy *et al.*, 2026; Chen, 2026a).

16.5.2 Numerical discipline inside search

Schedule search should not be allowed to change numerics silently. AutoTVM and Ansor explore loop orders, vector widths, and parallel strategies that are mathematically equivalent in the abstract but can differ in floating-point rounding, especially inside softmax, reductions, and mixed-precision graphs (Zheng *et al.*,

2020; Chen *et al.*, 2018a; Dao *et al.*, 2022). The reward function should include a bounded-error check against a reference implementation at the searched shapes, and candidates that exceed tolerance should be rejected regardless of speed (Dao *et al.*, 2022; Davies *et al.*, 2025). Mixed-precision search adds a second axis: allowing FP16 or BF16 accumulations enlarges the space and multiplies the numerical risk (Lai *et al.*, 2023; Chen, 2026a). Compiler teams that skip the error term optimize schedules that may pass latency gates and fail quality gates in production (Chen, 2020; Vellaisamy *et al.*, 2026).

16.5.3 Operator ranking by counter impact

When the layer experiment shows a remaining gap, rank operators by which counter they bind. A layer-norm or residual chain that materializes each intermediate binds bytes/token and should be fused first; attention softmax state that crosses a reduction binds both bytes and launch count; the per-token GEMMs bind launch cost only when their host dispatch dominates (Dao *et al.*, 2022; Chen, 2026a; Vellaisamy *et al.*, 2026). The ranking should come from the decode-cell profile, not from FLOPs, because FLOPs rank the GEMMs highest while the profile usually points at orchestration and memory seams (Davies *et al.*, 2025; Kwon *et al.*, 2023). After changing one operator’s schedule, re-run the whole-layer benchmark rather than the operator microbenchmark, since the bottleneck may simply move to the next seam (Chen, 2026a; Davies *et al.*, 2025). This ranking loop is the practical use of TVM’s schedule power for decode: not tuning everything, but tuning the seams that move the counters (Chen *et al.*, 2018b; Chen and YiRage Contributors, 2026).

16.5.4 Practical decision rules

Four rules compress this chapter for a design review. First, compile the decoder layer as a named function, not the whole model, so fusion and capture apply to the unit the serving loop repeats (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Second, rank schedule investment by bytes/token seams before launches before FLOPs, and refuse cost-model scores that cannot be traced to measured counters (Chen, 2026a; Davies *et al.*, 2025). Third, choose the executor and capture strategy by measuring their per-token host cost at batch-1 rather than by convenience of the training stack (Kwon *et al.*, 2023; NVIDIA, 2024). Fourth, pin the TVM version, target description, driver, and search artifacts together so any regression can be bisected to one change (Chen, 2020; Chen and YiRage Contributors, 2026). Teams that follow these rules get reproducible decode wins; teams that skip them get a

compiler that was faster in the demo and slower in the fleet (Vellaisamy *et al.*, 2026; Chen, 2026a).

16.5.5 Failure autopsies for TVM decode

TVM decode failures cluster into recognizable signatures. A schedule that wins isolated GEMM benchmarks yet loses whole-layer latency usually means fusion stopped at an import seam or a layout rewrite materialized an intermediate (Lai *et al.*, 2023; Chen, 2026a). A win on one GPU that disappears on another usually traces to target strings and tile defaults that differ between SKUs, which artifact pinning exposes as a config diff (Chen *et al.*, 2018b; NVIDIA, 2022). A regression after a TVM upgrade should be bisected between graph passes and schedule search by dumping the module at both stages (Chen, 2020; Davies *et al.*, 2025). Each autopsy should start from the pinned artifact set and the emitted TIR diff, not from framework timers (Chen, 2026a; Chen *et al.*, 2018b). The discipline turns every recurrence into a reproducible check instead of a folklore workaround (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

16.5.6 From one layer to the decoder loop

The layer experiment generalizes to the full decoder loop only when the loop structure survives compilation. A serving loop that calls the compiled layer function once per token should present a stable shape signature to TVM so each call reuses the same fused artifact and captured launch sequence (Dao *et al.*, 2022; Kwon *et al.*, 2023). If the loop instead grows the KV dimension dynamically per call, the runtime either falls back to a generic path or recompiles, and the per-token cost returns even though the layer artifact is perfect (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Bucket the KV length and compile one artifact per bucket, then measure the loop end-to-end including the host dispatch between calls (Chen, 2026a; NVIDIA, 2024). The final claim for TVM in decode is therefore not a single-kernel speedup but an end-to-end artifact set: fused layer, captured or static launch plan, bucketed shapes, and a host loop that does not undo the compiler’s work (Chen *et al.*, 2018b; Chen, 2026a; Davies *et al.*, 2025).

16.5.7 Continuous batching and p99

Real decode serving rarely runs one request alone; continuous batching interleaves requests at different positions, which changes what the TVM artifact must optimize (Kwon *et al.*, 2023; Davies *et al.*, 2025). At batch one the compiler should minimize

per-token latency and host dispatch; under continuous batching it must also tolerate requests crossing context buckets and attention loops of different lengths inside one scheduler window (Vellaisamy *et al.*, 2026; Chen, 2026a). The compiled artifacts remain per-bucket, so the runtime layer, not the compiler, owns the bucket handoff and the eager hull for dynamic requests (NVIDIA, 2024; Chen *et al.*, 2018b). Benchmarking should report p99 over the real request mix rather than mean latency on uniform synthetic data, because a few uncaptured fallback requests dominate tail latency exactly when they are invisible in the mean (Davies *et al.*, 2025; Kwon *et al.*, 2023). TVM tuning that stops at the kernel leaves this tail to the runtime; decode teams should tune the artifact and the scheduler together, and re-qualify the artifact set whenever the scheduler policy changes, because the compiler and the runtime jointly determine the p99 the product promises (Chen, 2026a; Davies *et al.*, 2025). Document the request mix and the bucket-handoff policy beside the artifact manifest, so a tail regression is attributable to the scheduler, the compiler, or the workload rather than to a generic TVM slowdown (Kwon *et al.*, 2023; Davies *et al.*, 2025). Make the benchmark script part of the release so the tail number stays reproducible after every compiler and runtime change (Chen, 2026a; Vellaisamy *et al.*, 2026).

16.6 Key Takeaways

- **The build matrix is not the decode matrix.** TVM lowers one tensor-expression definition to CPU, GPU, and accelerator back-ends, but a target that compiles is not automatically decode-legal (Chen *et al.*, 2018b). Validate each matrix entry against bytes/token and launches/token at batch-1, not against a per-operator microbenchmark (Vellaisamy *et al.*, 2026; Chen, 2026a).
- **Co-design graph and TensorIR passes.** Relax graph fusion and TensorIR lowering live in one `IRModule` and must be optimized together; fusion that “succeeds” at the graph level still loses if lowering spills the fused producer to DRAM (Lai *et al.*, 2023; Chen and YiRage Contributors, 2026). Diff lowered IR for new allocations before merge (Chen *et al.*, 2018b).
- **Grade codegen by orchestration, not by one kernel.** At batch-1 the launch floor and surviving HBM round trips dominate, so a coarser fused-and-captured kernel beats a beautiful per-operator emit (Vellaisamy *et al.*, 2026; NVIDIA, 2024). Graph capture recovered roughly 1.26× on an H100 decode cell with no MMA change (Chen, 2026a).
- **Tune at the decode operating point.** AutoTVM and Ansor maximize

whatever shape you profile; tuning on prefill-shaped GEMMs over-tiles for throughput and breaks at batch-1 (Chen *et al.*, 2018a; Zheng *et al.*, 2020). Profile BS=1 ms/token and decompose it before trusting a cost-model score (Davies *et al.*, 2025).

- **Optimization rank is hardware-specific.** GPU wants coalescing, fusion, and capture; CPU wants cache locality and SIMD; XDNA/AIE wants static tile and DMA assignment (AMD, 2024b,a). The same Ansor-tuned definition yields different legal schedules—and different speedups—per class (Zheng *et al.*, 2020).

16.7 Conclusion

TVM’s compute/schedule split and its learning-based search remain the cleanest demonstration that performance portability is a compiler problem, not a library-porting problem (Chen *et al.*, 2018b; Zheng *et al.*, 2020). But the stack only pays off for interactive serving when its graph and TensorIR passes, its codegen back-end, and its auto-tuner are all pointed at the decode operating point and graded on bytes/token and launches/token (Vellaisamy *et al.*, 2026; Chen, 2026a). That is the same residency contract YiRage enforces as IR metadata, and it is the bridge into the next compiler surface: Chapter 17 turns from TVM’s whole-graph search to Triton’s Pythonic, block-level GPU kernel compiler, where the same fusion-and-launch discipline reappears at the level of a single hand-written kernel (Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

Chapter 17

OpenAI Triton: Pythonic GPU Kernel Compiler

Triton made GPU kernel authoring look like Python and still hit cuBLAS/cuDNN-class throughput, which is exactly why it is dangerous in a decode pipeline if you stop at “the kernel is fast” (Tillet *et al.*, 2019). The language is built around one abstraction—the *tile*, a statically shaped multi-dimensional sub-array—so a programmer writes a per-block program over tiles and the compiler handles coalescing, shared-memory staging, and instruction scheduling (Tillet *et al.*, 2019). That productivity is real: FlashAttention-style fused attention is practical to write and maintain in Triton in a way it never was in raw CUDA (Dao *et al.*, 2022). The decode trap is the same one Chapter 16 raised for TVM. A beautiful Triton `matmul` does nothing for batch-1 latency if the decoder layer still dispatches a dozen separate kernels per token. Dense Llama-3.2-1B already averages 847.5 GPU kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). Triton helps only when it is used to *fuse* the decode loop into fewer, residency-aware kernels, and when those kernels are captured to amortize host launch cost (Chen, 2026a; NVIDIA, 2024). This chapter treats Triton as a block-level kernel compiler graded on bytes/token and launches/token, not as a Python tutorial (Davies *et al.*, 2025).

17.1 Triton HW matrix

Triton’s support **matrix** is deliberately narrower than a general tensor compiler’s, and pretending otherwise is the first **hardware** mistake. Triton is GPU-first: the



Figure 17.1: Triton compiles a per-block Python tile program to a fused GPU kernel; the block model assumes a GPU memory hierarchy.

production back-ends emit NVIDIA PTX and AMD AMDGCN, with CPU and Intel GPU back-ends emerging (OpenAI / triton-lang contributors, 2024). Its programming model assumes a SIMT machine with warps, shared memory, and a coalesced global-memory **target**—so it maps cleanly onto GPUs and not onto static-dataflow fabrics (NVIDIA, 2022; Tillet *et al.*, 2019).

The honest multi-hardware reading is that Triton is the right abstraction on one class and the wrong one on another. On a **gpu** it lets you express residency—which tiles live in shared memory, how loads are pipelined—at exactly the granularity decode needs (Tillet *et al.*, 2019; Chen, 2026a). On a **cpu** the same block program can be lowered but loses its *raison d’être*, since cache blocking and SIMD want a different schedule (Goodfellow *et al.*, 2016). On an **npu** such as XDNA/AIE the block-over-warps model does not apply at all: those fabrics demand static tile-to-DMA assignment decided at compile time, not a SIMT block grid (AMD, 2024b,a).

Triton’s design point also explains where it fits between raw CUDA and a whole-graph compiler. It abstracts away intra-block concerns—thread indexing, explicit shared-memory management, and memory coalescing are the compiler’s job, not the author’s—while leaving inter-block tiling and the fusion boundary to the programmer (Tillet *et al.*, 2019). That is more control than a graph compiler that decides fusion for you and more productivity than CUDA where you manage every load by hand. For decode the consequence is specific: Triton gives you exactly the knob that matters—where the fusion boundary sits—but it does not decide that boundary for you, so a careless kernel can still be fast in isolation and wasteful in a layer (Vellaisamy *et al.*, 2026; Chen, 2026a).

Multi-hardware delta. A Hopper GPU lets a Triton kernel hide latency with asynchronous copy and pipelined loads; an XDNA NPU cannot run the block model and needs a different compiler path entirely (SemiAnalysis, 2024; AMD, 2024a). This is why a serving stack treats Triton as one back-end among several rather than a universal target, and why YiRage carries Triton as a GPU code path while routing NPU work elsewhere (Chen and YiRage Contributors, 2026).

17.1.1 Where Triton sits in the decode stack

Triton occupies a specific layer: below the graph compiler that decides fusion, and above the CUDA code the device finally runs (Tillet *et al.*, 2019; Chen, 2020). A graph compiler such as XLA or IREE can lower a fused region into Triton source, or a team can write the Triton kernel directly and call it from a serving runtime (OpenXLA Project, 2024; OpenAI / triton-lang contributors, 2024). The direct route gives the team complete control over the fusion boundary, while the compiler route gives them the boundary that the graph passes found legal (Davies *et al.*, 2025; Chen, 2026a). Decode teams need to know which route their stack uses because the debugging surface differs: direct kernels are debugged in isolation and integrated by hand, while compiler-emitted Triton is debugged by inspecting the graph passes that produced it (Chen, 2020; Davies *et al.*, 2025). Both routes still feed the same counters this book uses, and both must be measured at the batch-1 decode operating point rather than on the author’s favorite benchmark shape (Vellaisamy *et al.*, 2026; Chen, 2026a).

17.1.2 Triton, torch.compile, and Inductor

Most PyTorch users meet Triton through torch.compile, whose Inductor backend emits Triton kernels for fused regions instead of requiring hand-written CUDA (Davies *et al.*, 2025; Chen, 2020). That path inherits Triton’s strengths—block-level codegen, automatic coalescing, software pipelining—and its limits: Inductor decides the fusion boundaries from the graph it receives, so an eager model with attention broken into framework ops produces different Triton kernels than a model whose attention is already a fused custom region (Dao *et al.*, 2022; Davies *et al.*, 2025). For decode, torch.compile can collapse elementwise and projection chains, but it does not automatically invent online softmax or a persistent decoder kernel if the source graph does not contain that structure (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). The engineering takeaway is that Inductor is a productive starting point whose output must be audited with the same TTGIR and PTX diffs as hand-written Triton, and whose fusion boundaries are only as good as the export that feeds them (Chen, 2020, 2026a).

17.1.3 GPU generation differences

The Triton source may be hardware-agnostic, but the schedule that makes it fast is not (OpenAI / triton-lang contributors, 2024; NVIDIA, 2022). Hopper and Blackwell GPUs differ in shared-memory capacity per SM, tensor-core MMA



Figure 17.2: Triton’s MLIR pass pipeline: the Triton dialect (TTIR) is lowered to the TritonGPU dialect (TTGIR), then to LLVM and a GPU assembly target.

shapes, async-copy depth, and warp-specialization support, so a kernel with one `num_stages` and layout configuration can be optimal on one generation and spill or stall on the next (SemiAnalysis, 2024; NVIDIA, 2022). The same applies across AMD GPUs with different vector widths and cache hierarchies (OpenAI / triton-lang contributors, 2024). A fleet with mixed GPU generations should therefore compile and autotune per SKU, store the winner beside the SKU identifier, and never copy a cache entry from another generation (Chen, 2026a; Davies *et al.*, 2025). The build matrix for Triton is small in vendor count but large in schedule diversity, and the schedule diversity is where decode bytes/token is won (Vellaisamy *et al.*, 2026; Chen, 2026a).

17.1.4 Portability expectations beyond NVIDIA

Triton’s AMD backend demonstrates that the block abstraction is portable in syntax but not in performance outcomes (OpenAI / triton-lang contributors, 2024). The same kernel emits AMDGCN with different vector widths, cache behavior, and instruction selection, so a config tuned on NVIDIA can be far from optimal on AMD even when it compiles cleanly (OpenAI / triton-lang contributors, 2024; Goodfellow *et al.*, 2016). Emerging CPU and Intel GPU back-ends widen the matrix further but inherit the fundamental mismatch: Triton’s tile-over-warps model assumes a SIMT memory hierarchy, which is precisely what CPUs and static NPUs do not provide (AMD, 2024b; Goodfellow *et al.*, 2016). A multi-vendor fleet should therefore treat each Triton backend as a separate qualification cell with its own autotune cache and benchmark artifacts (Davies *et al.*, 2025; Chen, 2026a). Portability in Triton means the source expresses the algorithm once; the schedule, the cache, and the qualification still belong to each backend (Tillet *et al.*, 2019; Vellaisamy *et al.*, 2026).

17.2 Triton arch passes

Modern Triton’s backend was rewritten on MLIR, so the **pass** pipeline is an explicit **dialect** tower: the Triton **dialect** (TTIR) captures the tile program;

Table 17.1: Triton’s MLIR lowering stages and what each level controls; the TTGIR stage is where decode residency is decided (OpenAI / triton-lang contributors, 2024).

Stage	Dialect / form	What it controls
TTIR	Triton dialect	Tile program, target-agnostic
TTGIR	TritonGPU dialect	Layout, coalescing, pipelining
LLVM IR	LLVM	Scalar/vector lowering, registers
PTX / AMDGCN	GPU assembly	Final emit (<code>ptxas</code> / ROCm)

the TritonGPU dialect (TTGIR) attaches GPU-specific layout and scheduling; both then **lower** to LLVM IR and a GPU assembly target (OpenAI / triton-lang contributors, 2024). Unlike TVM’s fixed two-level IR, Triton commits to this progressive-lowering dialect stack and concentrates its **optimization** in the TTGIR stage (Chen, 2020; Tillet *et al.*, 2019).

The TTGIR passes are where decode residency is won or lost. Layout assignment decides whether a tile lives in registers, shared memory, or an MMA-friendly layout for tensor cores; coalescing passes decide whether global loads are contiguous; the software-pipelining pass overlaps loads with compute by issuing them `num_stages` iterations ahead (OpenAI / triton-lang contributors, 2024; SemiAnalysis, 2024). Get the layout pass wrong and the kernel silently spills a tile to global memory between blocks—which reads downstream as an unexplained bytes/token regression rather than a compile error (Chen, 2026a; Vellaisamy *et al.*, 2026).

Triton’s layout system is the part most CUDA programmers underestimate. A blocked layout distributes a tile across threads for coalesced loads; an MMA layout matches the register fragment shape a tensor-core instruction expects; a shared layout stages data for cross-thread reuse (OpenAI / triton-lang contributors, 2024). The compiler inserts conversions between layouts automatically, and an unnecessary conversion is a hidden shared-memory round trip—cheap in a throughput kernel, measurable in a batch-1 decode kernel that runs thousands of times per token window (Vellaisamy *et al.*, 2026; SemiAnalysis, 2024). This is the level at which fusion either keeps the decoder layer on-chip or quietly does not.

What to inspect before merge. Triton can dump every stage (`TRITON_KERNEL_DUMP`, `MLIR_ENABLE_DUMP`); diff the TTGIR and the emitted PTX for new shared-memory allocations or un-pipelined loads, and count launches/token at the call site (OpenAI / triton-lang contributors, 2024; NVIDIA, 2024). A pass reorder that breaks pipelining will not change correctness, only p99 (Davies *et al.*, 2025).

17.2.1 Layout assignment as the residency decision

The TTGIR layout assignment pass is the single most consequential step between a Triton program and the emitted GPU code because it decides where every tile physically lives (OpenAI / triton-lang contributors, 2024; SemiAnalysis, 2024). Register layouts suit values consumed immediately by arithmetic; shared layouts suit values reused across warps or staged by async copies; MMA layouts match the exact fragment shape a tensor-core instruction consumes (OpenAI / triton-lang contributors, 2024; NVIDIA, 2022). When the compiler cannot assign one layout to a tile across its whole lifetime it inserts a layout conversion, which for large tiles means a round trip through shared memory (OpenAI / triton-lang contributors, 2024; Chen, 2026a). A decode kernel that runs per token thousands of times cannot hide those conversions behind other work the way a prefill kernel can, so layout conversions are a first-order p99 cost (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Reviewers should diff the TTGIR layout annotations and treat newly inserted conversions as regressions until proven otherwise (Chen, 2020, 2026a).

17.2.2 Coalescing and memory access patterns

Triton promises the author automatic coalescing, but the promise is conditional on the block sizes and pointer patterns the program expresses (Tillet *et al.*, 2019; OpenAI / triton-lang contributors, 2024). A tile whose logical layout does not match the physical addresses being loaded will produce scattered transactions even when the compiler applies its best vectorization (SemiAnalysis, 2024; Chen, 2026a). At batch-1 decode, KV-cache reads are the canonical example: block tables and paged layouts make the address stream non-contiguous, and the Triton kernel must express the gather or use a page-consistent tile shape so the coalescing passes have something contiguous to work with (Kwon *et al.*, 2023; Davies *et al.*, 2025). The emitted PTX should be inspected for vector-width eight-byte or sixteen-byte loads where the source expected them; scalarized loads are the signature of a layout mismatch that no autotuner config will fix (OpenAI / triton-lang contributors, 2024; Chen, 2026a).

17.2.3 Software pipelining and the decode shape

The `num_stages` pass overlaps global loads with computation by prefetching future tiles, which is exactly what a memory-bound attention kernel needs at long context (OpenAI / triton-lang contributors, 2024; SemiAnalysis, 2024). Each added stage costs shared memory, so the maximum legal depth is set by the tile size and the

GPU’s shared-memory capacity, not by the programmer’s preference (NVIDIA, 2022; Chen, 2026a). At batch-1 decode, a deep pipeline is usually wrong: the KV loop is long enough to hide latency, but the shared-memory cost of four stages can cut occupancy on the thin attention GEMMs that follow (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). The right depth therefore depends on context length, tile size, and SKU, which makes `num_stages` an autotune axis rather than a constant in the source (OpenAI / triton-lang contributors, 2024; NVIDIA, 2022). When profiling shows shared-memory-related occupancy loss, halve the stages before shrinking the block, because fewer stages preserve the larger transaction width that decode needs (Chen, 2026a; SemiAnalysis, 2024).

17.2.4 TTGIR debugging workflow

The practical way to find a layout or pipelining regression is to make the IR visible at every stage and diff it across changes (OpenAI / triton-lang contributors, 2024; Chen, 2020). Enable the Triton and MLIR dump flags, capture the TTIR and TTGIR for the decode kernel, and archive them with the source revision (OpenAI / triton-lang contributors, 2024). When a Triton upgrade changes behavior, diff the TTGIR before and after: new layout conversions, changed pipelining order, or different shared-memory allocations identify the regression without guesswork (Chen, 2026a; Davies *et al.*, 2025). The same dump workflow catches authoring mistakes such as masks that defeat vectorization or reductions that force shared-memory round trips (OpenAI / triton-lang contributors, 2024; Chen, 2026a). A team that diffed the IR first will spend minutes on regressions that cost other teams days of profiler archaeology (Vellaisamy *et al.*, 2026; Chen, 2020).

17.2.5 Masks, padding, and vectorization

Triton programs handle variable lengths with masks and padding, and both interact with the compiler’s vectorization and layout decisions (OpenAI / triton-lang contributors, 2024; Kwon *et al.*, 2023). A mask that is uniform across the tile lets the compiler specialize and keep vector loads; a per-element mask on a paged KV walk can force scalarization or predication that defeats coalescing (OpenAI / triton-lang contributors, 2024; Chen, 2026a). Decode kernels should prefer structure that makes masks cheap: bucket context lengths, pad at tile boundaries, and arrange the page layout so masked lanes sit at tile edges (Kwon *et al.*, 2023; Davies *et al.*, 2025). The cost of padding is capacity waste, and the cost of masks is throughput loss, so the block size choice is a balance between the two at the served context distribution (Chen, 2026a; Vellaisamy *et al.*, 2026).

When profiling shows low vector width on a masked kernel, restructure the data layout before tuning block shapes, because no config fixes scalarized loads (OpenAI / triton-lang contributors, 2024; Chen, 2026a).

17.3 Triton codegen

Codegen in Triton is per-GPU and explicit. The NVIDIA **backend** lowers LLVM IR to PTX and runs `ptxas` to **emit** a CUBIN; the AMD backend emits AMDGCN for ROCm (OpenAI / triton-lang contributors, 2024). The **platform** you target therefore changes the final assembly, but the controlling knobs are the same two parameters that govern the emitted schedule: `num_warps` (warps per CTA, hence threads = `num_warps`×32) and `num_stages` (software-pipelining depth) (OpenAI / triton-lang contributors, 2024).

The trap is grading the emitted kernel by its standalone throughput. At batch-1 decode the dominant cost is host orchestration and surviving HBM round trips, not the quality of a single emitted MMA (Vellaisamy *et al.*, 2026; Chen, 2026a). Two codegen decisions actually move ms/token. First, fuse: emit one kernel for the decoder layer (e.g. a Triton FlashAttention plus the projection epilogue) so activations never leave the chip between operators (Dao *et al.*, 2022). Second, capture: wrap the fused launch sequence in a CUDA Graph so the host stops paying per-kernel dispatch every token—the Memory-Floor study recovered roughly $1.26\times$ on an H100 Qwen-2.5-7B decode cell from capture alone, with no change to the emitted math (Chen, 2026a; NVIDIA, 2024).

There is a second-order codegen cost that decode exposes: register pressure. The LLVM stage allocates registers per thread, and a kernel that fuses an entire decoder layer can exceed the per-thread budget, forcing spills to local memory or capping occupancy through `maxnreg` (OpenAI / triton-lang contributors, 2024; NVIDIA, 2022). On a throughput kernel low occupancy is often fine because each block does enough work; on a batch-1 decode kernel the launch is already thin, so a register spill is pure overhead with nothing to hide behind (Vellaisamy *et al.*, 2026; Chen, 2026a). The practical consequence is that “fuse everything” is not free—past a point, a slightly less fused kernel with healthy occupancy beats a maximally fused one that spills.

Multi-hardware delta. Raising `num_stages` prefetches more aggressively and hides latency on a Hopper GPU with deep async-copy queues, but it costs shared memory and can reduce occupancy on a smaller SKU (SemiAnalysis, 2024; NVIDIA, 2022). The same Triton source therefore wants a different `num_stages` per GPU

generation, and none of it transfers to a static NPU fabric (AMD, 2024b).

17.3.1 Compile caching and ptxas

Triton compiles each kernel configuration through LLVM and `ptxas` at first use and caches the result keyed by source hash, target, and config (OpenAI / triton-lang contributors, 2024). The cache is a deployment surface: a cold cache turns a serving pod’s first requests into compile stalls, while a warm cache hides compile time entirely (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Production stacks should precompile the exact kernel set for the shipped model and SKU at build time and ship the cache or compiled artifact with the release (Chen, 2020, 2026a). The cache key must include the Triton version, LLVM version, and target string, because any one of them can change the emitted PTX and invalidate a performance assumption (OpenAI / triton-lang contributors, 2024; Davies *et al.*, 2025). Teams that let the cache build itself on first request are choosing warmup latency as an implicit product cost (Kwon *et al.*, 2023; Chen, 2026a).

17.3.2 Register budget as a fusion constraint

Register pressure is the wall that limits how much of a decoder layer one Triton kernel can fuse (NVIDIA, 2022; OpenAI / triton-lang contributors, 2024). The LLVM stage allocates a physical register file per thread, and a fused kernel holding attention state, projection tiles, and MLP intermediates simultaneously will either spill to local memory or force the compiler to serialize register lifetimes (OpenAI / triton-lang contributors, 2024; Chen, 2026a). Spills to local memory look like DRAM traffic in profiles and are frequently misdiagnosed as bandwidth regressions (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). The tuning response is to inspect the PTX register count and the `ptxas` spill report before changing anything else: if the kernel spills, reduce fusion width or tile size before tuning block shapes (OpenAI / triton-lang contributors, 2024; NVIDIA, 2022). A kernel that fits registers at batch-1 attention can still spill when the same source is generalized to a longer context tile, which is why register review belongs in the per-bucket qualification record (Chen, 2026a; Chen and YiRage Contributors, 2026).

17.3.3 Inspecting emitted code

Triton exposes the full IR stack for inspection, and decode teams should use it on every kernel that will run per token (OpenAI / triton-lang contributors, 2024). The workflow is to dump TTIR, TTGIR, LLVM IR, and PTX; verify

that the fused region the author intended actually survived as one kernel; check the TTGIR for layout conversions and shared-memory allocations; and confirm the PTX contains the expected vector loads and MMA instructions (Chen, 2020, 2026a). Any difference between the intended fusion and the emitted kernel is a bug in the author’s assumptions, not a compiler quirk, because the compiler is allowed to do anything that preserves semantics (OpenAI / triton-lang contributors, 2024; Davies *et al.*, 2025). The dumps should be archived with the benchmark so a later Triton upgrade can be diffed before it is shipped (Chen, 2026a; Vellaisamy *et al.*, 2026).

17.3.4 Counting kernels in the serving loop

The codegen review should answer a serving question that no single kernel dump answers: how many Triton kernels, plus eager or vendor kernels, execute for one token step (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). That count is the launches/token counter at the Triton layer, and it is the number capture amortizes (NVIDIA, 2024; Chen, 2026a). A fused attention kernel that reduces attention to one launch still leaves projections, MLP, and sampling as separate launches unless the layer integration fuses or captures them (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). The serving integration should therefore log kernel counts per token per bucket, so a new Triton kernel that adds an unexpected second launch shows up in telemetry before customers feel it (Davies *et al.*, 2025; Chen, 2026a). Codegen quality at decode is measured by this count together with bytes/token, never by the PTX beauty of one kernel (Chen, 2020, 2026a).

17.3.5 Composing with vendor kernels

Triton kernels rarely run alone in a serving stack; they compose with cuBLAS, cuDNN, or framework kernels around them (Davies *et al.*, 2025; OpenAI / triton-lang contributors, 2024). Each composition boundary inherits the seam rules of this book: if a Triton attention kernel writes its output to HBM so a cuBLAS projection can read it, the fused benefit stops at that edge (Chen, 2026a; Dao *et al.*, 2022). The integration review should therefore count the kernels on both sides of the Triton region and verify that capture wraps the whole sequence, not just the Triton portion (NVIDIA, 2024; Vellaisamy *et al.*, 2026). Vendor kernels are opaque to Triton’s compiler, so layouts must be agreed by contract at the boundary rather than discovered by inspection (Chen, 2020; Davies *et al.*, 2025). Teams that write beautiful Triton attention and bolt it onto an unfused layer repeat at a smaller scale the graph-level mistake this Part keeps correcting: the kernel is not

the token (Chen, 2026a; Davies *et al.*, 2025). A mismatch discovered in telemetry is a boundary-contract bug, not a scheduling accident, and it gets fixed in the integration rather than inside the kernel (Chen, 2026a; Davies *et al.*, 2025).

17.4 Triton tuning

Triton’s auto-tuning is a decorator, not a search service: `@triton.autotune` takes a list of `triton.Config` entries—block sizes plus `num_warps` and `num_stages`—benchmarks them at runtime for a given key, and caches the winner (OpenAI / triton-lang contributors, 2024). Because the candidate set is author-supplied, the quality of **tuning** is bounded by which configurations you thought to list. This is leaner than TVM’s learned-cost-model search but puts the burden on the engineer to enumerate sensible blocks.

Autotuning is also not free at runtime. Each config in the list must be compiled and benchmarked the first time a new key is seen, so a large config set inflates warmup latency and the first request after a shape change pays the search (OpenAI / triton-lang contributors, 2024). In a serving context that means the config list should be curated for the shapes you actually decode at, not a kitchen-sink sweep—and the cache key must include the dimensions that change the optimal schedule, or the autotuner silently reuses a config tuned for the wrong shape.

The **pitfall** is the operating point you **profile**. The reflexive config list is copied from a GEMM example tuned for large, batched, prefill-shaped matmuls: big `BLOCK_M/BLOCK_N`, high `num_stages` (Kwon *et al.*, 2023). At batch-1 decode the attention and projection GEMMs are matrix-vector products, so a large block tile is mostly idle lanes and a deep pipeline over-allocates shared memory for loads that never amortize (Vellaisamy *et al.*, 2026; Chen, 2026a). The fix is mechanical: autotune at the decode shape (`BS=1`, realistic context), include small-block and low-`num_stages` configs, and rank candidates by measured end-to-end ms/token after decomposing bytes/token and launches/token—not by the kernel’s isolated GB/s (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). The right config is also **hardware**-specific: a Hopper SKU tolerates a deeper pipeline than a memory-smaller inference GPU, so the cached winner does not port across generations (NVIDIA, 2022).

17.4.1 Designing the config list

Because Triton only benchmarks the configurations the author lists, config-list design is the real tuning skill (OpenAI / triton-lang contributors, 2024; Davies

et al., 2025). A good list spans the plausible trade: small blocks that maximize occupancy, medium blocks that balance occupancy and reuse, and larger blocks that reduce loop iterations, each crossed with two or three pipeline depths and warp counts (Chen, 2026a; NVIDIA, 2022). Configs that are obviously wrong for the shape should be omitted rather than benchmarked, because each entry costs compile and run time on every new cache key (OpenAI / triton-lang contributors, 2024; Vellaisamy *et al.*, 2026). The list should also be shape-aware: the attention kernel needs configs tuned to the KV-block dimension, while projection kernels need configs tuned to the hidden dimension (Kwon *et al.*, 2023; Chen, 2026a). Serving teams should version the config list with the kernel source so a regression can be traced to a candidate-set change as easily as to a code change (Chen, 2020; Davies *et al.*, 2025).

17.4.2 Warmup, cache keys, and first-token cost

The autotuner’s runtime benchmark happens once per cache key, and the first token that observes a new key pays it (OpenAI / triton-lang contributors, 2024; Davies *et al.*, 2025). The cache key must include every dimension that changes the optimal schedule—block structure, context length, head dimension, batch—or the tuner will reuse a winner measured for a different shape (Vellaisamy *et al.*, 2026; Chen, 2026a). Serving stacks should warm the cache at startup for every shipped bucket and model configuration, and should treat an unexpected cache miss in production as a first-token SLO event worth alarming on (Kwon *et al.*, 2023; Davies *et al.*, 2025). Offline pre-tuning is preferable: benchmark the config list against recorded decode shapes in CI, write the winning configs into a deployment manifest, and launch serving with autotuning disabled or cache prefilled (Chen, 2020; Vellaisamy *et al.*, 2026). This converts the tuner from a runtime risk into a build-time artifact (Chen, 2026a; OpenAI / triton-lang contributors, 2024).

17.4.3 Ranking candidates by the decode contract

The tuner returns whichever candidate its benchmark ranks first, so the benchmark must implement the decode contract rather than kernel GB/s (Davies *et al.*, 2025; Chen, 2026a). Rank candidates by end-to-end ms/token at the served context bucket after decomposing bytes/token and launches/token (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022). A candidate that wins the isolated kernel benchmark by ten percent can still lose the layer benchmark if its tile shape forces a layout conversion at the fusion seam or its occupancy starves the following kernel (OpenAI / triton-lang contributors, 2024; Chen, 2026a). The benchmark harness should therefore

call the kernel exactly as serving does: same stream, same input pointers, same capture context, same surrounding kernels (NVIDIA, 2024; Davies *et al.*, 2025). Config ranking that stops at the kernel boundary repeats the exact mistake this chapter warns against (Vellaisamy *et al.*, 2026; Chen, 2026a).

17.4.4 Re-tuning cadence and cache hygiene

Triton winners decay on the same triggers as other compiler artifacts: Triton and LLVM upgrades, driver changes, model updates, and context-bucket changes (OpenAI / triton-lang contributors, 2024; Chen, 2026a; Davies *et al.*, 2025). A nightly job should re-run the decode-cell benchmark against the pinned kernel set and alert when bytes/token or launches/token drift (Vellaisamy *et al.*, 2026; Chen, 2020). When a drift appears, re-autotune the affected kernels over the existing config list and diff the TTGIR before promoting a new winner (OpenAI / triton-lang contributors, 2024; Chen, 2026a). Cache hygiene matters equally: stale cache entries for old models or buckets should be pruned, and the shipped cache should contain exactly the kernels the release will execute (Davies *et al.*, 2025; Kwon *et al.*, 2023). Teams without this cadence discover Triton drift through fleet p99, which is the most expensive possible detection point (Vellaisamy *et al.*, 2026; Chen, 2026a).

17.5 Triton case study

Make it concrete with a fused decode attention kernel as the **case**. Writing FlashAttention in Triton keeps the query–key–value tiles and the running softmax state on-chip, so the kernel computes attention without materializing the full score matrix to HBM (Dao *et al.*, 2022; Tillet *et al.*, 2019). **Benchmark** it at the decode operating point and the reading matches the book’s thesis: an isolated per-operator kernel can win a microbenchmark yet lose end-to-end because each unfused boundary is an HBM round trip and each kernel is a separate launch (Vellaisamy *et al.*, 2026; Chen, 2026a).

The arithmetic shows why fusion is the lever. A non-fused attention step writes the full score vector for the new query against the cached context to HBM and reads it back for the softmax; at context length $L=2048$ in FP32 that score vector is $2048 \times 4 = 8$ KB written and re-read per head per token (Dao *et al.*, 2022). A Triton FlashAttention kernel keeps the running max and sum in registers and never materializes that vector, so across many heads and 32 layers it removes on the order of hundreds of kilobytes of avoidable HBM traffic per token—bytes/token

drops even though the multiply-accumulate count is unchanged (Vellaisamy *et al.*, 2026; Chen, 2026a). An isolated GEMM benchmark cannot see this win because it never had the inter-operator round trip in the first place.

This case also points at where Triton meets the book’s megakernel thesis. Fusing one attention block is a start, but the larger decode win comes from collapsing a whole decoder layer, or even several, into one persistent kernel that keeps weights and running state resident and issues a single launch per token rather than dozens. Triton makes that practical to express, because the programmer controls exactly where the fusion boundary sits and can widen it until register pressure or shared memory pushes back. The discipline is to grow the fused region while watching occupancy and the launch count together, stopping at the point where one more fused operator would force a spill that costs more than the launch it saved. That trade is measured per token and per hardware generation, never assumed, which is precisely the residency contract the rest of this book builds on.

The **cross-hardware** conclusion is the honest limit of Triton. On the GPU, the fused-and-captured Triton kernel is close to the best available decode path; on a CPU the same tile program is a poor fit for cache-blocked SIMD; on an XDNA/AIE NPU the block-over-warps model does not lower at all and a static-dataflow compiler is required instead (AMD, 2024b,a; Goodfellow *et al.*, 2016). **YiRage** reflects this directly: it uses Triton as its GPU kernel back-end while carrying the Chapter 2 residency rules as checkable IR metadata, so a Triton schedule that would spill a tile between blocks is rejected at compile time, and non-GPU targets are routed to backends whose machine model actually fits (Chen and YiRage Contributors, 2026).

17.5.1 Counter worksheet for fused attention

The fused-attention case produces a worksheet with four rows before it is shippable: the eager baseline’s launches/token and bytes/token at the served context bucket, the Triton attention kernel’s numbers in isolation, the numbers with the kernel composed into the layer, and the numbers with the layer wrapped in a captured launch sequence (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a). Each row must come from the same model, context, and driver, or the comparison attributes the wrong lever (Chen, 2020; Davies *et al.*, 2025). The worksheet should show where the win comes from: if the isolated kernel is faster but the composed layer is not, the problem is a seam outside the kernel; if capture moves latency but not bytes, orchestration is the remaining lever (Chen, 2026a; NVIDIA, 2024). Publishing this worksheet with every Triton kernel PR is how the team keeps “faster kernel” honest about “faster token” (Vellaisamy *et al.*, 2026; Chen, 2026a).

17.5.2 Fusion depth and the persistent-kernel decision

The case study stops at a decision boundary: fuse one attention block, fuse a full layer, or go persistent across tokens (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Attention-only fusion wins on the KV-loop bytes but leaves projections and MLP as separate launches; full-layer fusion wins on round trips but raises register pressure; a persistent kernel removes even the per-token launch but concentrates failure modes and complicates debugging (Chen, 2026a; NVIDIA, 2024). Triton expresses each level with the same block abstraction, so the decision is a measured escalation rather than a rewrite (Tillet *et al.*, 2019; OpenAI / triton-lang contributors, 2024). The escalation should stop at the first level where both bytes/token and launches/token stop improving, because pushing one more operator past that point trades a small launch saving for a spill or occupancy loss (Chen, 2026a; NVIDIA, 2022). The decision belongs in the release notes with the worksheet, so the next engineer can see why the stack is not one giant kernel (Chen, 2020; Davies *et al.*, 2025).

17.5.3 Failure signatures for Triton decode

Three recurring failures should be recognized from their profiles. A bytes/token regression with correct-looking TTGIR usually means a layout conversion or register spill that the IR dump exposes only on close reading (OpenAI / triton-lang contributors, 2024; Chen, 2026a). A p99 regression that appears only after serving traffic starts usually means cache misses or autotune re-runs on new shape keys, visible as compile stalls in the runtime timeline (Davies *et al.*, 2025; Kwon *et al.*, 2023). A kernel that is fast on an isolated benchmark but slow in the layer usually means the fusion boundary or occupancy interaction sits outside the kernel (Vellaisamy *et al.*, 2026; Chen, 2026a). Each signature has a fix path—IR diff, cache warm-up, or layer-level fusion—and none of them is solved by rewriting the kernel arithmetic (Chen, 2020; Davies *et al.*, 2025). Archiving the dumps and worksheet with each release makes the next autopsy a diff instead of a re-derivation (Chen, 2026a; Vellaisamy *et al.*, 2026).

17.5.4 Reusing kernels across backends and models

The same Triton attention kernel often serves several models because the block structure depends on tile sizes and head dimensions, not on weights (Dao *et al.*, 2022; OpenAI / triton-lang contributors, 2024). Reuse is safe when the shape signature, dtype, and page layout match; it is unsafe when a model change alters the fusion boundary or the runtime context (Davies *et al.*, 2025; Chen, 2026a). Kernel

libraries should therefore expose the shape contract explicitly and benchmark each (model, bucket, SKU) combination against the shared kernel rather than assuming one autotune winner generalizes (Vellaisamy *et al.*, 2026; Chen, 2020). A reused kernel that was tuned for one head dimension can still run correctly but suboptimally on another, and correctness hides that inefficiency (OpenAI / triton-lang contributors, 2024; Davies *et al.*, 2025). The case study’s final artifact is a small kernel library with per-signature winners, an archived worksheet per signature, and a serving layer that selects the winner from telemetry keys (Chen, 2026a, 2020).

17.5.5 Worked counter expectations

A realistic Triton attention kernel should change counters in predictable directions. Compared with an unfused eager baseline, the kernel should reduce bytes/token because score and probability materialization disappear; it should reduce kernels per token because the KV loop, softmax, and accumulation collapse into one launch; and it should keep ms/token within the range the memory-bound floor predicts once launches are captured (Dao *et al.*, 2022; Chen, 2026a; Vellaisamy *et al.*, 2026). If bytes/token drops but ms/token does not, host dispatch or a neighboring seam is the remaining cost; if ms/token drops but bytes/token is flat, the win is orchestration rather than residency (Davies *et al.*, 2025; NVIDIA, 2024). These expectations make benchmark review mechanical: a PR that claims a Triton attention win must show which counter moved, and the answer determines whether the PR belongs in the residency chapter of this book or the orchestration chapter (Chen, 2026a, 2020). Publishing the expectations next to the worksheet keeps reviewers and authors speaking the same counter language (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

17.6 Key Takeaways

- **Triton is a GPU block compiler, not a universal target.** Its tile/SIMT model maps to NVIDIA PTX and AMD AMDGCN but not to static-dataflow NPUs, so treat it as one back-end in a multi-hardware stack (Tillet *et al.*, 2019; OpenAI / triton-lang contributors, 2024). The build matrix is GPU-shaped (AMD, 2024b).
- **Win residency in the TTGIR passes.** Layout assignment, coalescing, and software pipelining at the TritonGPU dialect decide whether tiles stay on-chip; a bad layout pass spills silently and shows up only as a bytes/token

regression (OpenAI / triton-lang contributors, 2024; Chen, 2026a). Dump and diff the IR before merge (Vellaisamy *et al.*, 2026).

- **Grade codegen by orchestration, not one kernel.** At batch-1 the launch floor and HBM round trips dominate, so fuse the decoder layer and capture the launch sequence; capture alone recovered roughly $1.26\times$ on an H100 decode cell (Chen, 2026a; NVIDIA, 2024). A fast standalone MMA is not a fast token (Dao *et al.*, 2022).
- **Autotune at the decode operating point.** `@triton.autotune` only explores the configs you list; default GEMM configs over-tile and over-pipeline for batch-1 (OpenAI / triton-lang contributors, 2024; Kwon *et al.*, 2023). Include small-block, low-`num_stages` configs and rank by ms/token (Davies *et al.*, 2025).
- **Tuned kernels do not port across GPUs.** Optimal `num_warps/num_stages` depend on shared-memory size and async-copy depth, so a cached winner must be re-tuned per generation (NVIDIA, 2022; SemiAnalysis, 2024). Portability is a re-tune, not a copy (Goodfellow *et al.*, 2016).

17.7 Conclusion

Triton’s tile abstraction and MLIR dialect pipeline are the cleanest way to write fused, residency-aware GPU kernels at Python productivity, and they are why decode-critical kernels like FlashAttention are now maintainable (Tillet *et al.*, 2019; Dao *et al.*, 2022). But the payoff is conditional: Triton helps interactive serving only when its TTGIR passes, its PTX/AMDGCN codegen, and its autotuner are aimed at the batch-1 decode shape and graded on bytes/token and launches/token (Vellaisamy *et al.*, 2026; Chen, 2026a). That is the residency contract YiRage enforces as IR metadata across its backends (Chen and YiRage Contributors, 2026). Triton also exposes the limit of a GPU-only kernel compiler, which motivates the next surface: Chapter 18 turns to IREE, an MLIR-native compiler and runtime that carries the same dialect-lowering discipline across a wider hardware matrix (Goodfellow *et al.*, 2016).

Chapter 18

IREE: MLIR-Native Runtime and Compilation

IREE attacks the decode problem from a different angle than a kernel compiler: it compiles the model’s *scheduling logic and its device execution together*, ahead of time, into one deployable artifact (IREE / iree-org contributors, 2024). Where a Python serving loop decides per-token what to launch at runtime, IREE bakes the host orchestration into virtual-machine bytecode at compile time—which is exactly the lever Chapter 1 identified, because at batch-1 the launch and dispatch path, not the matmul, sets ms/token (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Dense Llama-3.2-1B already averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). A compiler that schedules the whole program can collapse that orchestration the way captured CUDA Graphs do, but earlier and across more targets (Chen, 2026a; NVIDIA, 2024). The catch is the same as every other surface in this Part: IREE will happily produce a correct binary for a target whose schedule still re-reads activations from DRAM every token. This chapter treats IREE as an MLIR-native compiler+runtime graded on bytes/token and launches/token, not as a build-system walkthrough (Goodfellow *et al.*, 2016).

18.1 IREE HW matrix

IREE’s support **matrix** is the broadest in this Part: one MLIR input compiles to llvm-cpu, CUDA, ROCm/HIP, Vulkan/SPIR-V, and Metal, with AMD AIE still experimental (IREE / iree-org contributors, 2024). The Hardware Abstraction

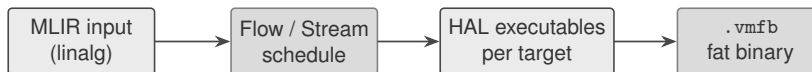


Figure 18.1: IREE lowers one MLIR input through whole-program scheduling into a single `.vmfb` that carries host instructions and per-target device kernels.

Layer (HAL) is what makes that breadth tractable—it is a Vulkan-inspired **target** abstraction, roughly 1:1 with IREE’s HAL C API, that exposes buffers, command buffers, and dispatches as a common surface across device types (IREE / iree-org contributors, 2024).

The honest reading is that a wide build matrix is not a wide *decode* matrix. A green HAL **target** means the model lowers and runs there; it says nothing about whether the schedule keeps a decoder layer on-chip (Vellaisamy *et al.*, 2026; Chen, 2026a). The hardware truth is unchanged across the matrix: a **gpu** is bound by shared-memory residency and tensor-core feeding, a **cpu** by cache blocking and NUMA, an **npu** like XDNA/AIE by static tile SRAM and DMA chains—and on the IREE matrix the NPU path is still experimental, so it is exactly where the build-vs-decode gap is widest (NVIDIA, 2022; AMD, 2024b,a).

What makes the breadth credible rather than aspirational is that IREE was designed to scale from the datacenter down to mobile and edge, where the same compiled artifact must run under tight memory and power budgets (IREE / iree-org contributors, 2024). The runtime is deliberately small and optional: a fully featured server can use the VM for dynamic deployment with multi-threaded dispatch, while an embedded target can bypass the runtime entirely and link the generated kernels directly (IREE / iree-org contributors, 2024). For a decode engineer that flexibility is a double-edged sword. The same model that serves on an H100 can be compiled for an edge accelerator, but the residency math is completely different at the two ends of the matrix: an edge SKU with megabytes of on-chip memory cannot hold the working set a datacenter GPU does, so a dispatch-region fusion that is a clear win on H100 can overflow on-chip storage and spill on the edge target.

Multi-hardware delta. The same `.vmfb` can carry kernels for several vendors, and IREE can probe the device at load time to pick one (IREE / iree-org contributors, 2024). But a Hopper GPU wants async-copy double buffering while an XDNA fabric wants a static DMA schedule, so the portable artifact still hides per-target residency decisions that decode telemetry will expose (SemiAnalysis, 2024; AMD, 2024a). YiRage treats this as a routing problem: it carries the Chapter 2 residency rules as checkable metadata and selects a backend whose machine model fits,

rather than trusting that a green matrix cell is decode-legal (Chen and YiRage Contributors, 2026).

18.1.1 Input paths into IREE

The input path determines how much scheduling structure the compiler can see. IREE consumes several front ends: TensorFlow and TFLite models convert into the stablehlo-based input dialects, PyTorch models arrive through a torch-MLIR path, and ONNX models convert through the ONNX-MLIR lowering (IREE / iree-org contributors, 2024; Chen, 2020). Each conversion has a granularity problem similar to the ones XLA and TVM face: operations that arrive as fused framework calls may stay opaque, while operations that arrive element-by-element give the flow stage the seams it needs to form dispatch regions (Davies *et al.*, 2025; Chen, 2026a). For decode, the recommended input is a graph where one decoder layer is a recognizable unit with attention and MLP structure preserved, because the dispatch-region formation can then fuse inside the layer instead of re-deriving structure from flat operator lists (Dao *et al.*, 2022; IREE / iree-org contributors, 2024). Teams should inspect the converted module before compilation and treat poor structure as an export bug rather than an IREE limitation (Chen, 2020; Davies *et al.*, 2025).

18.1.2 VM and embedded runtime modes

IREE’s runtime flexibility is a deployment decision with decode consequences. The full VM runtime provides dynamic features such as module loading, multi-threaded dispatch, and debugging hooks, while the embedded mode links compiled kernels directly for small footprints and lower per-call overhead (IREE / iree-org contributors, 2024). A batch-1 serving path that runs in the VM still benefits from compiled scheduling because the bytecode is generated ahead of time, but every dynamic runtime call has a cost that shows up when measured per token (Vellaisamy *et al.*, 2026; Chen, 2026a). Embedded mode removes most of that cost at the price of compile-time specialization: every shape and target variant is a separate linked artifact (IREE / iree-org contributors, 2024; Davies *et al.*, 2025). The serving team should measure both modes at the decode operating point and choose by p99, not by deployment convenience (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Neither mode repairs a poorly fused dispatch region; both only execute the schedule the compiler built (Dao *et al.*, 2022; Chen, 2026a).

18.1.3 Multi-target artifact strategy

Because one `.vmfb` can carry specialized kernels for multiple targets, the artifact strategy shapes the build matrix in production (IREE / iree-org contributors, 2024). Shipping one fat binary that supports every GPU vendor simplifies rollout but multiplies binary size and compile time; shipping per-SKU artifacts optimizes size and startup at the cost of more build jobs and a more complex release matrix (Davies *et al.*, 2025; Chen, 2026a). For decode fleets with mixed hardware, the per-SKU approach usually wins because each target needs its own tuned configuration and residency proof anyway (NVIDIA, 2022; AMD, 2024a). The artifact manifest should record the targets, compiler version, input model hash, and decode benchmark for each binary so a rollout is reproducible (Chen, 2020; Vellaisamy *et al.*, 2026). This is the IREE instance of the rule that the deployed compiler output is a first-class artifact, not a build byproduct (Davies *et al.*, 2025; Chen, 2026a).

18.1.4 Server, mobile, and edge decode contracts

IREE’s breadth across server GPUs and mobile accelerators means the same model source serves multiple decode contracts, and the contracts differ in more than speed (IREE / iree-org contributors, 2024; Chen, 2026a). A server contract can assume a large on-chip working set, deep async pipelines, and a fast host; an edge contract must fit the working set into smaller SRAM or cache, tolerate power limits, and pay closer attention to binary size and startup time (IREE / iree-org contributors, 2024; Goodfellow *et al.*, 2016). Dispatch-region fusion that is safe on a datacenter GPU can overflow the smaller on-chip storage of an edge target, so each contract needs its own fusion and codegen configuration (AMD, 2024b; Chen, 2026a). The decode benchmark for an edge contract should include memory and power envelopes, not just latency, because a schedule that wins on ms/token by using more memory may fail the device budget (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). IREE’s matrix is broad enough to support all three contracts, and exactly broad enough to make per-contract qualification non-optional (IREE / iree-org contributors, 2024; Chen, 2020).

18.2 IREE arch passes

IREE’s compilation is a sequence of MLIR **dialects**, and the **pass** pipeline is where decode goodput is decided long before any device code is generated (IREE / iree-org

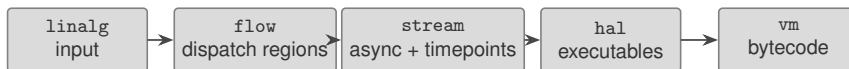


Figure 18.2: IREE’s dialect pipeline: **flow** forms dispatch regions, **stream** schedules them asynchronously with timepoints, **hal** translates per target, **vm** emits host bytecode.

Table 18.1: IREE’s ahead-of-time compilation phases; **flow** fusion sets bytes/token and **stream** scheduling sets launches/token (IREE / iree-org contributors, 2024).

Phase	What it does
Input	Lower to core dialects (linalg , etc.)
Flow	Form and fuse dispatch regions; partition
Stream	Async schedule with timepoints; encode resources
HAL	Per-target executable translation (codegen)
VM	Emit host scheduling bytecode

contributors, 2024). After the input **dialect** (**linalg** and friends), dispatch-creation fuses operations and forms *dispatch regions*—the unit of work that becomes one kernel; the **flow** dialect models data flow and partitioning; the **stream** dialect **lowers** tensors into opaque resources and schedules the dispatches asynchronously using *timepoints* so independent work overlaps; the **hal** dialect translates each executable per target; and the **vm** dialect emits the host bytecode (IREE / iree-org contributors, 2024).

Two of these passes are the ones a decode engineer must watch. Dispatch-region formation is the **optimization** that sets bytes/token: if it fuses a decoder layer’s element-wise and reduction work into one region, intermediates stay on-chip; if it leaves them separate, each boundary is a DRAM round trip (Vellaisamy *et al.*, 2026; Chen, 2026a). Stream scheduling is the one that sets launches/token: by sequencing dispatches with timepoints instead of a host-side wait per kernel, it removes exactly the per-launch orchestration that dominates batch-1 (IREE / iree-org contributors, 2024; NVIDIA, 2024).

What to inspect before merge. IREE can dump the executable for each dispatch region (`-iree-hal-dump-executable-*`), so diff which operators were fused into a region and which spilled out; an aggressive-fusion flag exists, but more fusion is only a win until register or shared-memory pressure forces a spill (IREE / iree-org contributors, 2024; Davies *et al.*, 2025). A regression here is silent: the binary is correct, only bytes/token moved (Goodfellow *et al.*, 2016).

18.2.1 Dispatch-region formation as fusion

The flow stage’s central decision is where to cut the graph into dispatch regions, and the cut determines every downstream counter (IREE / iree-org contributors, 2024; Chen, 2026a). A region that spans an elementwise chain, a reduction, and its consumer keeps the intermediate on-chip; a cut at any of those boundaries creates a kernel boundary and an HBM round trip (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). IREE uses an aggressive fusion policy by default, but legality still depends on shapes, layouts, and the target’s on-chip budget (IREE / iree-org contributors, 2024; Chen, 2020). For decode, the review question is how many dispatch regions one decoder layer produced and which operators each region contains (Davies *et al.*, 2025; Chen, 2026a). If the layer fragments into more regions than the intended fusion design, the fix is upstream in the input structure or fusion flags, not in the codegen (Chen, 2020; Davies *et al.*, 2025).

18.2.2 Stream scheduling and timepoints

The stream stage turns a fused dispatch list into an asynchronous schedule by attaching timepoints that express dependencies instead of inserting a host wait after each dispatch (IREE / iree-org contributors, 2024; NVIDIA, 2024). That design removes the per-kernel host synchronization that makes eager batch-1 decode expensive, because the device can run dependent work in order without the host round-tripping between dispatches (Vellaisamy *et al.*, 2026; Chen, 2026a). Decode graphs are mostly sequential, so the timepoint win is modest compared with a wide parallel graph, but the removal of per-dispatch host waits is exactly the launches/token lever this book tracks (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). The stream stage also decides how independent work overlaps, which matters for fused regions that split into compute and copy phases (IREE / iree-org contributors, 2024; SemiAnalysis, 2024). When profiling shows gaps between dispatches, inspect the stream schedule for unexpected serialization before blaming the kernels (Chen, 2026a; Davies *et al.*, 2025).

18.2.3 VM bytecode and host control flow

The VM stage is what makes IREE different from a kernel library: the host side of the model becomes bytecode that encodes the dispatch sequence, buffer handling, and control flow (IREE / iree-org contributors, 2024). Decode loops, bucket selection, and per-token calls can be represented in that bytecode, which is why a compiled IREE program can replace a Python serving loop for the steady-state

path (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Control flow that stays host-side and dynamic, such as sampling and MoE routing, should be kept outside the compiled region and called explicitly so the VM does not try to encode logic that changes per request (Kwon *et al.*, 2023; Liu *et al.*, 2024). The boundary between compiled bytecode and dynamic host logic is a design decision, and the telemetry should attribute latency to whichever side actually spent it (Chen, 2026a, 2020).

18.2.4 Resource encoding and buffer reuse

Between flow and stream, tensors become opaque resources with lifetimes the scheduler can manage (IREE / iree-org contributors, 2024; Chen, 2020). Resource encoding is where IREE plans buffer reuse across the whole module, and reuse is what lets a decoder layer stream a small working set instead of allocating fresh storage per operator (Chen, 2026a; IREE / iree-org contributors, 2024). When encoding chooses a new resource for every intermediate, the runtime footprint grows and cache behavior degrades even if fusion succeeded (Davies *et al.*, 2025; Chen, 2026a). The compile report should show resource counts and reuse decisions for the decode layer so reviewers can verify that intermediates are not accumulating per region (Chen, 2020; Vellaisamy *et al.*, 2026). Buffer reuse is the silent half of residency: fusion keeps values on-chip, and resource encoding keeps the off-chip footprint from growing (Chen, 2026a; IREE / iree-org contributors, 2024).

18.3 IREE codegen

IREE’s **codegen** runs per dispatch region during the HAL executable-translation stage, and each **backend emits** a different low-level form: PTX for CUDA, HSACO for ROCm/HIP, SPIR-V for Vulkan, and native code for the CPU (IREE / iree-org contributors, 2024). Because compilation is ahead-of-time, the **platform** mix is resolved on the build host and packaged into the FlatBuffers **.vmfb** fat binary, which carries host VM instructions alongside the device kernels so the runtime can dispatch without a JIT (IREE / iree-org contributors, 2024).

The decode-relevant point is that IREE’s runtime is not a Python loop; the VM replays a compiled schedule, so the host orchestration cost is paid at compile time rather than per token (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026). This is structurally the same win that capturing a launch graph buys on a GPU—the Memory-Floor study recovered roughly 1.26× on an H100 decode cell purely by removing host overhead—except IREE bakes it into the deployable artifact and extends it to CPU and Vulkan targets (Chen, 2026a; NVIDIA, 2024).

What IREE cannot do is rescue a poorly fused dispatch region: if codegen emits a layer as many small kernels because fusion failed upstream, no amount of clean scheduling removes the HBM traffic those kernels create (Dao *et al.*, 2022).

The fat-binary format is itself a decode-relevant design choice. Because the kernels are committed at build time, IREE can ship several specialized variants of the same dispatch region—aligned and unaligned shapes, large and small matmuls—and let the runtime probe the device and select one without a JIT step (IREE / iree-org contributors, 2024). For batch-1 decode that avoids the warmup and recompilation stalls a JIT stack pays on the first request after a shape change, which is precisely the moment an interactive session is most latency-sensitive. The cost is binary size and a build matrix that grows with every target and specialization, so a serving team trades disk and compile time for predictable per-token latency—usually the right trade for an interactive product, the wrong one for a one-off batch job.

Multi-hardware delta. The HAL maps the same dispatch onto workgroups that become GPU thread blocks or CPU threads, and subgroups that become GPU SIMT lanes or CPU SIMD (IREE / iree-org contributors, 2024). The abstraction is genuine, but the best tile shape and async depth still differ per target, so one dispatch region’s codegen is not optimal everywhere (SemiAnalysis, 2024; NVIDIA, 2022).

18.3.1 Translation strategies per target

The HAL executable stage chooses a translation strategy for each dispatch region based on the target and the operator mix in the region (IREE / iree-org contributors, 2024; Chen, 2020). A region that is mostly GEMMs may translate through a vendor library call or a specialized codegen path, while an elementwise chain translates through the generic vector pipeline (IREE / iree-org contributors, 2024; Davies *et al.*, 2025). The strategy choice is where per-target tuning lives: the CPU path wants vector widths and cache blocking, the CUDA path wants shared-memory and MMA-friendly codegen, and the Vulkan path wants SPIR-V shader workgroup sizes that fit the driver’s best configuration (IREE / iree-org contributors, 2024; SemiAnalysis, 2024). For decode, the report should show which strategy each dispatch region used, because a GEMM region that took the generic path instead of a vendor call is usually a missed configuration rather than a compiler deficiency (Chen, 2026a; Vellaisamy *et al.*, 2026). Teams should re-run strategy selection at the decode shape and compare the report against the prefill report, since strategy

choices that win on large shapes can be wrong for thin batch-1 work (Kwon *et al.*, 2023; Davies *et al.*, 2025).

18.3.2 Variant specialization and binary growth

Specialized variants trade binary size and compile time for predictable runtime behavior (IREE / iree-org contributors, 2024; Chen, 2026a). Each aligned and unaligned shape pair, each matmul size, and each target can produce its own executable variant inside the fat binary, and the runtime selects one at load or dispatch time (IREE / iree-org contributors, 2024; Davies *et al.*, 2025). For a decode server with a small bucket set, that specialization is exactly right: the compiler emits one good variant per bucket instead of forcing one general kernel to serve all shapes (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). The cost is a build and disk footprint that must be managed in the release process (Chen, 2020, 2026a). Teams should define the specialization set from the serving bucket list and prune variants that no request ever uses, because dead variants still cost compile time and confuse binary triage (Davies *et al.*, 2025; Chen, 2026a).

18.3.3 Device probing and dispatch

IREE’s runtime can probe device capabilities and select among variants without a JIT step, which is the advantage the fat binary buys over a runtime compiler (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026). Probe logic should be deterministic and logged: which device, which variant, and which benchmark produced the selection (Davies *et al.*, 2025; Chen, 2026a). When a fleet mixes GPUs, the probe result explains why the same binary performs differently and points to the per-SKU variant set as the tuning surface (NVIDIA, 2022; AMD, 2024a). A probe that silently falls back to a generic variant is a p99 risk, so the runtime should expose the selected variant in telemetry (Kwon *et al.*, 2023; Chen, 2026a). This turns device diversity from an incident source into a managed configuration dimension (Chen, 2020; Davies *et al.*, 2025).

18.3.4 Measuring dispatch overhead

Even with VM bytecode and stream timepoints, dispatch overhead does not vanish; it shrinks to the compiled cost of moving work through the HAL (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026). That cost should be measured by comparing the device kernel time against the wall time of a token step at batch-1 (Davies *et al.*, 2025; Chen, 2026a). If device time accounts for nearly all wall time,

the schedule is doing its job; if a gap remains, the gap is runtime dispatch, variant selection, or host-call overhead inside the VM (IREE / iree-org contributors, 2024; Davies *et al.*, 2025). Reducing the gap may mean moving to embedded mode, tightening the VM module, or moving per-token control flow out of the compiled region (Chen, 2020; Vellaisamy *et al.*, 2026). The measurement belongs in the compile-time benchmark harness so every configuration change reports the overhead fraction alongside ms/token (Chen, 2026a; IREE / iree-org contributors, 2024). This fraction is the IREE-specific view of launches/token: not how many launches, but how much each remaining launch costs through the runtime (NVIDIA, 2024; Chen, 2026a).

18.4 IREE tuning

IREE **tuning** happens at compile time through target configuration: the executable-configurations stage selects a translation strategy per dispatch region, and flags steer dispatch-region fusion and per-target codegen (IREE / iree-org contributors, 2024). Because the schedule is compiled, **profile**-first discipline shifts earlier than with a JIT stack—you compile a candidate, run it, and read where time went before changing flags, rather than letting a runtime autotuner converge.

There is a second discipline that whole-program compilation rewards: compile-time visibility. Because every dispatch region and its scheduling are decided before deployment, the compiler can report exactly which operators fused, how many regions a layer became, and how they are sequenced—information a runtime-dispatched stack only learns by profiling the live system. A decode team should treat that report as the first artifact to read after a build, comparing the dispatch-region count per layer against expectation before ever launching the model. When the region count is higher than the number of fused units you designed for, fusion failed somewhere upstream, and the fix belongs in the compilation flags, not in a runtime workaround.

The **pitfall** is benchmarking the wrong shape and the wrong layer. Prefill compiles into large, well-fused dispatch regions that look excellent in isolation; decode is dominated by the thin, latency-bound per-token path where dispatch granularity and host scheduling matter more than peak occupancy (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Compile and profile at the batch-1 decode operating point, decompose end-to-end ms/token into bytes/token and launches/token, and only then adjust fusion and codegen flags (Davies *et al.*, 2025; Chen, 2026a). The right configuration is **hardware**-specific: the CPU path wants cache-blocked dispatch regions, the CUDA path wants fused kernels plus async copy, and the

experimental AIE path wants static tiling—so the tuned configuration for one HAL target does not transfer to another (AMD, 2024b; Goodfellow *et al.*, 2016).

18.4.1 Compile-time benchmarking workflow

Because IREE schedules at compile time, the tuning loop is: compile a candidate configuration, benchmark the artifact at the decode operating point, inspect the dispatch and strategy reports, adjust flags, and recompile (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026). The workflow should be scripted end to end so every flag change produces an artifact with the same manifest fields and the same benchmark harness (Chen, 2020, 2026a). The benchmark must measure the full token step through the VM or embedded runtime, not individual dispatch regions, because the schedule between regions is part of what is being tuned (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Each candidate should report dispatch-region count per layer, strategy selections, and bytes/token alongside ms/token, so the winning configuration is chosen for a reason that survives review (Chen, 2026a; Davies *et al.*, 2025). This workflow replaces runtime autotuning with build-time search, which is more expensive per candidate but gives a deterministic deployment (IREE / iree-org contributors, 2024; Chen, 2020).

18.4.2 Fusion flags and their limits

Aggressive-fusion flags widen dispatch regions, but the limit is always the target’s on-chip budget (IREE / iree-org contributors, 2024; Chen, 2026a). A region that fuses an entire decoder layer may exceed registers, shared memory, or tile SRAM on the actual target and spill during codegen or execution (NVIDIA, 2022; AMD, 2024a; Dao *et al.*, 2022). The correct level is not the maximum fusion but the fusion that keeps occupancy healthy and intermediates resident (Vellaisamy *et al.*, 2026; Chen, 2026a). For decode, fusion decisions should be made per dispatch-region structure: attention carriers and their consumers fuse well, while routing and sampling logic should stay outside regions (Dao *et al.*, 2022; Liu *et al.*, 2024). The executable dumps show whether a requested fusion survived or was split by legality, and that report is the tuning signal (IREE / iree-org contributors, 2024; Davies *et al.*, 2025). Teams that assume a fusion flag is a promise rather than a request will be surprised by spills that the report would have predicted (Chen, 2026a, 2020).

18.4.3 Fingerprinting compiled artifacts

Whole-program compilation makes artifact fingerprinting both easier and more necessary: easier because the artifact captures the schedule, more necessary because a small flag or compiler change can alter every dispatch region (IREE / iree-org contributors, 2024; Chen, 2020). The fingerprint should include the IREE version, the input module hash, the target configuration, the fusion and codegen flags, and the benchmark results of the artifact (Chen, 2026a; Davies *et al.*, 2025). Release manifests should store the fingerprint beside the `.vmfb` so a rollout can be bisected to the exact configuration (Vellaisamy *et al.*, 2026; Chen, 2020). When a decode regression appears after an upgrade, the fix is to diff fingerprints and re-run the compile-time workflow, not to tune at runtime (Chen, 2026a; Davies *et al.*, 2025). This discipline is the deployment half of IREE’s design: compiled determinism only helps if the compilation itself is recorded (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026).

18.4.4 Quantization and mixed precision at compile time

Whole-program compilation makes precision decisions visible at the module level instead of scattered across kernel wrappers (IREE / iree-org contributors, 2024; Chen, 2020). Quantized weights, FP16 or BF16 activations, and per-tensor scales flow through the input dialects into dispatch regions, where the chosen precision determines bytes moved per token (Chen, 2026a; Davies *et al.*, 2025). Mixed-precision compile must respect numerical boundaries: softmax carriers and accumulators need higher precision even when the surrounding matmuls run in reduced precision (Dao *et al.*, 2022; Chen, 2026a). The compile report should state the precision of every dispatch region so a reviewer can verify that reduced precision did not silently creep into a numerically sensitive seam (Chen, 2020; Davies *et al.*, 2025). Quantization is therefore a compile-time residency decision in IREE: it changes bytes/token before fusion ever runs, and the two must be tuned together rather than separately (IREE / iree-org contributors, 2024; Chen, 2026a).

18.5 IREE case study

Take a single decoder layer compiled through IREE as the **case**, and **benchmark** it at the decode operating point on more than one HAL target (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026). The reading matches the book’s thesis: a layer that compiles into one fused dispatch region with stream-scheduled launches

keeps activations on-chip and pays host orchestration once; a layer that fragments into many dispatch regions re-reads activations from DRAM and pays a launch per region every token (Chen, 2026a; Dao *et al.*, 2022). The arithmetic is the same as for any fusion: at hidden size 4096 in BF16 each materialized activation vector is 8 KB, so a handful of avoided round trips per layer is tens of kilobytes of HBM traffic removed per token before the KV cache is even read (Vellaisamy *et al.*, 2026). Across the full depth of a model that avoided traffic accumulates into a meaningful fraction of the per-token memory budget, which is why the dispatch-region count, not the peak throughput of any single region, is the number that tracks decode latency. The same compiled layer that fragments into a dozen regions will read and rewrite those vectors a dozen times, and the runtime schedule cannot recover bytes the compiler chose to spill.

The **cross-hardware** conclusion is where IREE’s breadth helps and where it stops. Because IREE keeps the same dispatch-region formation across targets, a layer fused well on the CPU path is usually fused on the CUDA path too, which makes A/B comparison and bring-up faster (IREE / iree-org contributors, 2024). But the experimental AIE/NPU path is exactly where a green build hides an unfinished decode story, and a Vulkan target on a phone has different residency limits than an H100 (AMD, 2024a; NVIDIA, 2022). **YiRage** uses IREE-style MLIR lowering as one path while enforcing the Chapter 2 residency rules as checkable IR metadata, so a dispatch region that would spill is rejected at compile time and non-GPU work is routed to a backend whose machine model fits (Chen and YiRage Contributors, 2026).

18.5.1 Setting the region-count target

The case study becomes measurable when the team states a dispatch-region target per decoder layer before compiling (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026). A reasonable first target for a fused layer is a small number of regions: one for the attention core that respects online-softmax carriers, one or two for the projections and MLP if the on-chip budget forces a split, and none for trivial elementwise chains that should have fused away (Dao *et al.*, 2022; Chen, 2026a). The compiler report should be checked against that target after every build, and a higher count is a finding, not an accepted default (Davies *et al.*, 2025; Chen, 2020). Region count is the compile-time proxy for bytes/token because each extra region is a potential HBM boundary (Chen, 2026a; Vellaisamy *et al.*, 2026). When the count is right, the benchmark verifies the proxy; when the count drifts, the benchmark explains the regression (IREE / iree-org contributors, 2024; Davies *et al.*, 2025).

18.5.2 Reading the VM schedule

Beyond region count, the compiled VM schedule shows how regions are sequenced and where host-side waits remain (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026). The schedule should read like a dependency graph with timepoints, not a list of kernels with a wait after each one (IREE / iree-org contributors, 2024; NVIDIA, 2024). Gaps between dependent regions in a profile are expected only when a real dependency exists; gaps between independent regions indicate scheduling that serialized work unnecessarily (SemiAnalysis, 2024; Chen, 2026a). The decode case study should therefore inspect both the VM-level schedule and the device timeline, because the former explains the latter (Davies *et al.*, 2025; Chen, 2020). A compiled program whose schedule contains per-region host waits has lost the structural advantage IREE promises, and the fix is in the stream stage configuration (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026).

18.5.3 Failure autopsies for IREE decode

Three signatures recur. A correct binary with high bytes/token usually means dispatch-region fusion stopped early, visible as a high region count in the compile report (Chen, 2026a; IREE / iree-org contributors, 2024). A correct binary with high ms/token but healthy region count usually means stream scheduling serialized work or the runtime selected a generic variant instead of the specialized one (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). A regression that appears only after an IREE upgrade usually traces to changed fusion or translation defaults, recoverable by diffing the compile reports between versions (Chen, 2020, 2026a). Each autopsy starts from the artifact fingerprint and compile report, which makes the diagnosis a diff instead of a re-derivation (Davies *et al.*, 2025; IREE / iree-org contributors, 2024). Teams that archive reports with binaries turn every recurrence into a ten-minute check (Chen, 2026a; Vellaisamy *et al.*, 2026).

18.5.4 Integrating the compiled module into serving

The compiled `.vmfb` still needs a serving integration around it: tokenization, sampling, KV-cache management, and request scheduling live in the application, while the steady-state decoder path runs in the compiled module (Kwon *et al.*, 2023; IREE / iree-org contributors, 2024). The integration should call the module through the runtime API with preallocated buffers and avoid per-token allocation, because allocation on the hot path reintroduces the host cost the compiler removed (Davies *et al.*, 2025; Chen, 2026a). KV-cache state that changes shape per token

must be handled by the compiled bucket contract: either the module accepts the current KV pointer and length as arguments, or the application switches modules at bucket boundaries (Chen, 2020; IREE / iree-org contributors, 2024). The telemetry boundary should attribute time to the module call, the runtime, and the application logic separately so regressions are attributable (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). A clean integration is the difference between IREE’s compiled schedule being the serving path and being one kernel among many in an eager host (Chen, 2026a; Davies *et al.*, 2025).

18.5.5 Multi-model packaging and load

Production serving often compiles several models or several buckets of one model into the same deployment, which makes module load and memory sharing a design question (Kwon *et al.*, 2023; IREE / iree-org contributors, 2024). Loading every variant eagerly wastes memory; loading lazily pays first-token latency on cache misses (Davies *et al.*, 2025; Chen, 2026a). A serving team should define the resident module set from traffic data and preload the hot set at startup (Chen, 2020; Vellaisamy *et al.*, 2026). The runtime can share one HAL device context across modules, which reduces per-module overhead, but the sharing must be verified under concurrent dispatch (IREE / iree-org contributors, 2024; Davies *et al.*, 2025). Release testing should include the load path: startup time, memory before and after load, and first-token latency per module (Chen, 2026a; IREE / iree-org contributors, 2024). Multi-model packaging turns IREE’s compile-time determinism into a fleet-level property rather than a per-binary property (Kwon *et al.*, 2023; Chen, 2020).

18.5.6 Working example checks

Before shipping an IREE decode artifact, run three working checks against the compiled output: the dispatch report matches the region target per layer, the VM schedule contains no per-region host waits, and the benchmark shows device time close to wall time at batch-1 (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a). Each check is fast once the workflow is scripted, and each catches a different failure class: fusion loss, scheduling serialization, and runtime overhead (Davies *et al.*, 2025; Chen, 2020). The checks should run on every target in the artifact because a configuration that passes on CUDA can fail on Vulkan or CPU (IREE / iree-org contributors, 2024; AMD, 2024a). Teams that automate the checks get release candidates that are already explained; teams that skip them get binaries whose behavior is discovered by customers (Chen, 2026a; Davies *et al.*,

2025).

18.5.7 Reading target deltas

When the same decode module ships on multiple targets, the target delta report should compare region counts, strategy selections, and bytes/token so differences are understood rather than discovered (IREE / iree-org contributors, 2024; Chen, 2026a). A target that fuses less is not wrong if its on-chip budget is smaller; it needs its own region target and its own benchmark (AMD, 2024b; Chen, 2020). A target whose dispatch overhead is higher needs a different runtime mode or variant selection (IREE / iree-org contributors, 2024; Davies *et al.*, 2025). The delta report turns IREE’s breadth from a support promise into a measurable matrix of decode contracts (Vellaisamy *et al.*, 2026; Chen, 2026a). Without it, a multi-target rollout treats every difference as an incident instead of a configuration (Davies *et al.*, 2025; IREE / iree-org contributors, 2024). The report belongs in the release manifest so that a later IREE upgrade can be evaluated target by target instead of by a single headline latency number, and so that the residency contract for each HAL target stays visible to the whole serving team (IREE / iree-org contributors, 2024; Chen, 2026a). Each manifest cell should name the runtime mode, the variant set, and the measured dispatch-overhead fraction, making the delta auditable across releases and across the engineers who own those targets (IREE / iree-org contributors, 2024; Davies *et al.*, 2025).

18.6 Key Takeaways

- **IREE compiles scheduling and execution together.** Host orchestration is baked into VM bytecode ahead of time, so the per-token launch path is set at compile time, not by a runtime loop (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026). That is the structural fix for the launches/token problem decode hits (Davies *et al.*, 2025).
- **Dispatch-region fusion sets bytes/token.** Whether the `flow` stage fuses a decoder layer into one region decides how many DRAM round trips survive; dump and diff the executables before trusting a build (IREE / iree-org contributors, 2024; Chen, 2026a). A correct binary can still be a slow token (Goodfellow *et al.*, 2016).
- **Stream scheduling sets launches/token.** The `stream` dialect sequences dispatches with timepoints, removing per-kernel host waits the way a captured

launch graph does—worth roughly $1.26\times$ on an H100 decode cell from host-overhead removal alone (Chen, 2026a; NVIDIA, 2024).

- **A wide build matrix is not a wide decode matrix.** IREE targets CPU, CUDA, ROCm, Vulkan, and Metal, but the NPU/AIE path is experimental and a green target only means it lowers and runs (IREE / iree-org contributors, 2024; AMD, 2024b). Validate each target against decode metrics, not compilation success (AMD, 2024a).
- **Tuned configurations are per-HAL-target.** Cache-blocked CPU dispatch, fused CUDA kernels, and static AIE tiling are different strategies, so a configuration tuned for one HAL target does not transfer (SemiAnalysis, 2024; NVIDIA, 2022). Compile and profile at the batch-1 decode shape (Kwon *et al.*, 2023).

Practical decision rules. Four rules compress this chapter for review. First, set a dispatch-region target per decoder layer and treat the compile report as a gate, because region count predicts bytes/token before any benchmark runs (IREE / iree-org contributors, 2024; Chen, 2026a). Second, benchmark through the VM or embedded runtime at batch-1, never individual regions, because the schedule between regions is part of the compiled product (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Third, fingerprint every artifact with the compiler version, flags, input hash, and benchmark so upgrades are reviewable diffs (Chen, 2020, 2026a). Fourth, choose VM versus embedded mode and fat-binary versus per-SKU packaging from measured p99, not from build convenience (Kwon *et al.*, 2023; IREE / iree-org contributors, 2024). Teams that apply these rules treat IREE as a production compiler with a deployable schedule; teams that skip them treat it as a build system and discover the schedule only in fleet latency (Vellaisamy *et al.*, 2026; Chen, 2026a).

18.7 Conclusion

IREE’s contribution is to make whole-program scheduling a compiler artifact: by lowering through Flow, Stream, HAL, and VM it compiles host orchestration and device kernels together and packages them into one retargetable binary (IREE / iree-org contributors, 2024). For interactive serving that matters because it attacks launches/token at compile time and dispatch-region fusion attacks bytes/token—provided both are graded at the batch-1 decode operating point rather than on prefill-shaped regions (Vellaisamy *et al.*, 2026; Chen, 2026a). That is the same

residency contract YiRage enforces as IR metadata across backends (Chen and YiRage Contributors, 2026). IREE also shows that breadth and depth are different axes: a wide HAL matrix still needs per-target residency proof, which sets up the next surface—Chapter 19 turns to Glow, a lighter-weight AI compiler that makes the opposite trade between generality and a focused lowering path (Goodfellow *et al.*, 2016).

Chapter 19

Glow: Lightweight AI Compiler

Glow is the compiler in this Part that optimizes for the opposite end of the hardware spectrum from a datacenter GPU: it targets CPUs, accelerators, and memory-constrained edge devices, and it makes a deliberate bet that ahead-of-time compilation into a self-contained object file beats a runtime interpreter (Rotem *et al.*, 2018; PyTorch / Glow contributors, 2024). That bet matters for the same reason Chapter 1 gave: eliminating per-operator dispatch overhead is the lever that moves batch-1 latency, and Glow removes it at compile time by emitting one bundle with a single entry function rather than dispatching operators through a framework (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). The honest framing is that Glow is not where you optimize H100 LLM decode—it has no CUDA-graph-class GPU path—but it is exactly where you optimize decode on a Cortex-M or a small accelerator, and its design choices (graph lowering to a few primitives, operator stacking, profile-guided quantization) are the ones a residency-first engineer should recognize (Chen, 2026a; Goodfellow *et al.*, 2016). This chapter treats Glow as a lightweight AOT compiler graded on bytes/token and launches/token at the edge, not as a build-tool tutorial (NVIDIA, 2024).

19.1 Glow HW matrix

Glow’s support **matrix** is shaped by one design decision: rather than implement every operator on every **target**, the compiler lowers the high-level graph to a small set of linear-algebra primitives, so a new **hardware** backend only has to implement those primitives (Rotem *et al.*, 2018). That is why Glow spreads cheaply across accelerators and edge parts, and why its production sweet spot is the LLVM CPU



Figure 19.1: Glow lowers a model to a few linear-algebra primitives, stacks element-wise work into single-traversal kernels, and emits a self-contained AOT bundle.

backend cross-compiled to embedded cores like Arm Cortex-M, not a server GPU (PyTorch / Glow contributors, 2024).

The honest multi-hardware reading inverts the usual GPU-first story. On a **cpu** or microcontroller Glow is excellent: the AOT bundle removes interpreter and dispatch overhead and fits a fixed memory budget (Rotem *et al.*, 2018; Vellaisamy *et al.*, 2026). On a **gpu** Glow exists (an OpenCL backend) but is not the decode tool—there is no captured-launch-graph story to rival the CUDA path Chapters 16–18 described (Chen, 2026a; NVIDIA, 2024). On an **npu** or custom accelerator Glow’s small-primitive lowering is precisely the abstraction that makes a new backend tractable, which is its strategic value to silicon vendors (AMD, 2024b,a).

Multi-hardware delta. The same model compiles to an x86 bundle for a host and a Cortex-M bundle for an edge device, but the residency math differs by orders of magnitude: an MCU with kilobytes of SRAM cannot hold the working set an x86 cache can, so a fusion that helps on the host may not even fit on the edge target (Goodfellow *et al.*, 2016; AMD, 2024a). YiRage treats this as routing: it carries the Chapter 2 residency rules as checkable metadata and selects an edge-style AOT path or a GPU path per target rather than assuming one bundle is universally optimal (Chen and YiRage Contributors, 2026).

19.1.1 Model input and deployment forms

Glow imports models from the formats edge teams actually carry, chiefly ONNX and TFLite, and lowers them into its own graph IR before any optimization (Rotem *et al.*, 2018; PyTorch / Glow contributors, 2024). The import granularity determines what the domain-specific passes can see, exactly as it does for the datacenter compilers: an ONNX export that keeps batch normalization, bias, and activation as separate nodes leaves room for Glow’s merges, while a graph that arrives pre-baked into opaque custom operators hides that structure (Chen, 2020; Davies *et al.*, 2025). For a small decoder at the edge, the import should preserve attention and MLP seams so stacking and quantization can work on real layer structure rather than on a flat list of tensors (Dao *et al.*, 2022; Rotem *et al.*, 2018). Deployment form matters too: Glow can emit bundles linked into an

application, and the choice of bundle boundary—whole model, one layer, or one bucket—sets the granularity at which the host calls into compiled code (PyTorch / Glow contributors, 2024; Davies *et al.*, 2025).

19.1.2 The primitive set as a backend contract

The small linear-algebra primitive set is the architectural bet that makes Glow’s hardware matrix cheap to extend (Rotem *et al.*, 2018; AMD, 2024a). A silicon vendor adds a backend by implementing a bounded set of primitives plus the memory and thread abstractions, instead of porting every framework operator (Rotem *et al.*, 2018; Chen, 2020). For decode, the contract has two edges: high-level graphs must lower cleanly into those primitives, and the primitives must preserve enough structure for operator stacking and quantization to remain profitable (Rotem *et al.*, 2018; Vellaisamy *et al.*, 2026). When a new operator cannot lower without decomposing into many tiny primitives, the backend gets correctness but loses the single-traversal properties that edge decode depends on (Chen, 2026a; Davies *et al.*, 2025). Teams evaluating an accelerator backend should therefore count how many primitives a decoder layer expands into, because that count predicts both dispatch cost and buffer traffic (Rotem *et al.*, 2018; Chen, 2026a).

19.1.3 Edge decode budgets

Decode on an edge device obeys budgets that datacenter compilers ignore: SRAM and cache capacity, power envelope, binary size, and startup time are first-class constraints rather than afterthoughts (Goodfellow *et al.*, 2016; AMD, 2024a). The per-token working set of a small decoder must fit the on-chip budget or the bundle spills to slower memory on every token (Chen, 2026a; Vellaisamy *et al.*, 2026). Static memory planning is therefore not a convenience on the edge; it is the mechanism that proves the bundle fits before deployment (Rotem *et al.*, 2018; Goodfellow *et al.*, 2016). Power affects the optimization objective: a schedule that saves latency by using more memory may exceed the thermal budget, so edge tuning should report energy per token alongside latency (Davies *et al.*, 2025; Chen, 2026a). Teams that port a datacenter decoder to an edge device without re-deriving these budgets will discover them through field failures, not compile errors (Chen, 2020; AMD, 2024a).



Figure 19.2: Glow’s two-phase IR: a high-level node graph for domain-specific optimization, lowered via IRGen to a low-level address-only instruction IR before code generation.

19.1.4 Silicon-vendor adoption view

For a silicon vendor, Glow’s lowering-to-primitives design is attractive because a new backend implements a bounded kernel set and inherits the graph optimizations (Rotem *et al.*, 2018; AMD, 2024a). The adoption decision should include the decode lens from this chapter: the vendor must prove that the primitive implementations preserve the stacking, static allocation, and precision properties the compiler assumed (Chen, 2026a; Davies *et al.*, 2025). A primitive that materializes intermediates internally breaks the single-traversal contract even if it is numerically correct (Rotem *et al.*, 2018; Chen, 2026a). Backend validation should therefore include decoder-shaped graphs with byte counters, not only the operator conformance suite (Chen, 2020; Vellaisamy *et al.*, 2026). Vendors that validate with decoder graphs get a credible edge-decode story; vendors that validate only with image-classification graphs inherit surprises (Davies *et al.*, 2025; Chen, 2026a).

19.1.5 Bundle lifecycle and field updates

Ahead-of-time compilation moves risk from runtime to release: the bundle’s schedule, memory plan, and precision are fixed when it ships, so updating a model means shipping a new bundle (Rotem *et al.*, 2018; Davies *et al.*, 2025). Edge products need an update path that can stage and validate new bundles before they replace the running one (Chen, 2020, 2026a). Each bundle generation should carry a manifest with the model hash, compiler version, target, memory plan, and benchmark, so field teams can compare generations after an update (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). A field regression after an OTA update is then attributable to the bundle diff rather than to nondeterministic runtime behavior (Rotem *et al.*, 2018; Chen, 2026a). The ability to roll back to the previous bundle generation is the edge counterpart of runtime feature flags, and it belongs in the product requirements before the first fleet deployment (AMD, 2024a; Chen, 2020).

Table 19.1: Glow’s two-phase IR and the optimization class each level owns (Rotem *et al.*, 2018).

IR level	Form	Optimizations
High-level	Node dataflow graph	Domain-specific (BN+conv, layout, rescale)
Low-level	Address-only instructions	Scheduling, static alloc, copy elimination
Codegen	LLVM IR	Stacked kernels, target-specific emit

19.2 Glow arch passes

Glow’s **pass** pipeline is built on a two-phase strongly-typed IR, and each phase owns a different class of **optimization** (Rotem *et al.*, 2018). The high-level IR is a static-shaped dataflow node graph where domain-specific rewrites live: merge batch-norm into convolution, change a matrix layout, eliminate a numeric rescale. IRTGen then **lowers** each high-level node into one or more low-level instructions, producing an address-only IR whose operands are typed pointers to buffers; this is where memory optimizations run—instruction scheduling, static memory allocation, and copy elimination (Rotem *et al.*, 2018).

It is worth being precise about what Glow is and is not here. Glow predates the MLIR **dialect** approach that IREE and modern TVM use, and instead of an open, extensible dialect tower it hard-codes this fixed two-level design (Rotem *et al.*, 2018). The trade is less generality but a shorter, more predictable lowering path—appropriate for an edge compiler where build reproducibility and a tight memory plan matter more than supporting every research IR. For decode, the low-level IR’s static memory allocation is the feature that matters: a fixed buffer plan computed at compile time is what lets a bundle run in a known footprint without an allocator in the hot path (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

What to inspect before merge. Because both IR levels are dumpable, diff the low-level IR for buffer reuse and copy elimination, and confirm that element-wise chains were stacked rather than left as separate traversals (Rotem *et al.*, 2018; Chen, 2026a). A regression shows up as extra buffers or extra tensor passes, not as a wrong answer (Goodfellow *et al.*, 2016).

19.2.1 High-level rewrites that survive to decode

Glow’s high-level graph rewrites were developed around convolutional networks, but the class of transform matters for decoder graphs too: merge adjacent arithmetic,

eliminate redundant rescales, fold constants, and change layouts so later passes see friendly structure (Rotem *et al.*, 2018; Chen, 2020). In a small decoder, residual adds, layer-normalization scaling, and activation functions create chains that benefit from the same merge discipline (Dao *et al.*, 2022; Chen, 2026a). The high-level IR is statically shaped, which makes these rewrites safe and predictable: every tensor dimension is known at compile time, so the compiler can prove whether a transform preserves semantics (Rotem *et al.*, 2018; Davies *et al.*, 2025). Reviewers should check that the imported decoder graph actually exposes these chains, because an export that already split them into custom calls leaves the high-level passes nothing to rewrite (Chen, 2020; Davies *et al.*, 2025).

19.2.2 Static memory allocation and buffer reuse

The low-level IR’s address-only form exists so memory can be planned statically: every buffer has a fixed address or a slot in a reused pool before codegen (Rotem *et al.*, 2018; Chen, 2026a). The planner reuses scratch across instructions whose lifetimes do not overlap, which is how a deep model runs inside a small footprint without a hot-path allocator (Rotem *et al.*, 2018; Goodfellow *et al.*, 2016). For decode, buffer reuse is the edge form of residency: if the planner cannot overlap two intermediates, the bundle needs both simultaneously, and the sum must fit the device (Chen, 2026a; Vellaisamy *et al.*, 2026). The low-level IR dump shows the reuse decisions directly, and a footprint regression after a model change can be traced to a lifetime that stopped overlapping (Rotem *et al.*, 2018; Chen, 2020). Teams should treat the static memory report as a first-class artifact, because it is the compile-time proof of the memory budget the edge device requires (AMD, 2024a; Chen, 2026a).

19.2.3 Scheduling and copy elimination

Instruction scheduling in the low-level IR orders operations to reduce traffic and latency, and copy elimination removes moves whose source and destination can be unified (Rotem *et al.*, 2018; Chen, 2026a). On a small core these passes matter more than on a big GPU because the memory system is smaller and every extra traversal is proportionally more expensive (Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026). The scheduling pass should also respect operator-stacking boundaries: work that will later fuse into one traversal should not be scheduled in a way that forces an intermediate materialization (Rotem *et al.*, 2018; Davies *et al.*, 2025). Diffing the low-level IR before and after a compiler change shows whether scheduling and copy elimination still behave; a regression appears as extra copy instructions or split

traversals (Chen, 2026a, 2020). These low-level details are where the AOT promise of predictable latency is actually delivered (Rotem *et al.*, 2018; Vellaisamy *et al.*, 2026).

19.2.4 Counting traversals

The low-level IR makes traversal count an explicit property: each instruction that reads and writes a full tensor is one pass over that tensor, and chains that fuse share one traversal (Rotem *et al.*, 2018; Chen, 2026a). The compile report should count traversals per decoder layer and compare them against the minimum implied by the fused design (Chen, 2020; Vellaisamy *et al.*, 2026). A layer whose elementwise chains each traverse separately has left the stacking opportunity on the table even if every kernel is individually fast (Chen, 2026a; Rotem *et al.*, 2018). This count is the edge equivalent of bytes/token because on a small memory system traversal count and bytes moved are nearly proportional (Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026). Setting a traversal budget per layer at design time gives reviewers a number to enforce and gives the compiler a target to optimize toward (Davies *et al.*, 2025; Chen, 2026a).

19.2.5 MCU pipeline shape

Microcontroller decode graphs are small enough that whole-model scheduling is feasible, and Glow’s fixed IR is designed for that scale (Rotem *et al.*, 2018; AMD, 2024a). The low-level instruction stream for a small decoder should be short, with most work inside stacked kernels and few control-flow branches (Rotem *et al.*, 2018; Chen, 2026a). Branchy schedules that look reasonable on a host can bloat an MCU binary and introduce cache-unfriendly jumps (Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026). Static allocation on an MCU must also respect alignment and memory-mapped regions that the host planner cannot assume (AMD, 2024a; Davies *et al.*, 2025). Reviewing the emitted instruction count and branch structure for the decode loop catches these issues before device bring-up (Chen, 2020, 2026a). The MCU pipeline shape is the extreme case of the AOT principle: predictability comes from keeping the compiled schedule small and fixed (Rotem *et al.*, 2018; Vellaisamy *et al.*, 2026).

19.3 Glow codegen

Glow’s primary **codegen** path is its LLVM-based CPU **backend**, and its signature trick is operator stacking: instead of one kernel per element-wise operator, Glow generates LLVM IR for a single stacked kernel that performs all the chained operations during one traversal of the tensor (PyTorch / Glow contributors, 2024; Rotem *et al.*, 2018). That is the bytes/token lever in its purest form—an unstacked chain reads and writes the tensor once per operator, while the stacked kernel touches it once (Vellaisamy *et al.*, 2026; Chen, 2026a).

Because the backend is LLVM, Glow can **emit** a self-contained object file (a bundle) and cross-compile it for a different **platform** via standard LLVM target flags—for example an Arm Cortex-M7 core on an embedded board (PyTorch / Glow contributors, 2024). The decode-relevant consequence is that the bundle has a single inference entry point and no framework dispatch, so the per-token launch overhead that dominates batch-1 on a Python stack simply does not exist; the work is one compiled function call (Vellaisamy *et al.*, 2026; NVIDIA, 2024). This is the AOT counterpart to capturing a launch graph on a GPU, achieved by never having a launch loop in the first place (Chen, 2026a).

There is a subtler reason operator stacking matters more at the edge than the headline FLOP count suggests. On a memory-constrained device the limiter is rarely arithmetic; it is the bandwidth between the compute core and whatever memory tier holds the activations, and every extra tensor traversal pays that bandwidth tax again (Vellaisamy *et al.*, 2026; Chen, 2026a). By collapsing a chain of element-wise operators—bias add, activation function, rescale—into one traversal, Glow turns several bandwidth-bound passes into one, which on a small core is often a larger win than any single-operator speedup (Rotem *et al.*, 2018). The same logic is why the static memory plan matters: with buffers assigned at compile time, the bundle reuses a small pool of scratch memory instead of allocating per operator, so the resident footprint stays inside the device’s on-chip budget and the allocator never appears in the per-token path (Goodfellow *et al.*, 2016; Davies *et al.*, 2025).

Multi-hardware delta. On a CPU or MCU the stacked kernel plus a static buffer plan is close to the best available edge decode path; on a GPU the same approach does not exploit warps or tensor cores, so Glow’s CPU strength does not transfer upward (NVIDIA, 2022; SemiAnalysis, 2024). The lowering-to-primitives design is what lets a new accelerator backend reuse the high-level optimizations without reimplementing them (AMD, 2024a).

19.3.1 Stacked kernel structure

Operator stacking succeeds when the fused chain is profitable in both compute and memory terms. Elementwise work with the same iteration domain stacks cleanly; reductions and reshape-style operations break the single-traversal property unless the compiler can express them in the same loop structure (Rotem *et al.*, 2018; Chen, 2026a). The generated LLVM IR should show one loop nest reading the input tensor once and applying the whole chain, with temporaries in registers or a small scratch region (PyTorch / Glow contributors, 2024; Vellaisamy *et al.*, 2026). For decoder chains of bias, activation, scaling, and residual adds, that single traversal is the whole win, and splitting it re-introduces the bandwidth tax the edge device cannot afford (Dao *et al.*, 2022; Chen, 2026a). Reviewers should therefore inspect the emitted IR for loop count per tensor and treat a chain that materialized intermediate tensors as a fusion failure (Rotem *et al.*, 2018; Davies *et al.*, 2025).

19.3.2 Cross-compilation and bundle boundaries

Glow emits bundles for the host or for a cross-compiled target through LLVM target flags, and the bundle boundary is a deployment decision (PyTorch / Glow contributors, 2024; Rotem *et al.*, 2018). One bundle per model keeps the host call simple but forces the whole model’s schedule to be fixed together; per-layer or per-bucket bundles give the host more control at the cost of more entry points and buffer handoffs (Davies *et al.*, 2025; Chen, 2026a). For a decoder with multiple context buckets, separate bundles per bucket let the quantized ranges and memory plans differ per bucket while the host selects the right entry (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). The handoffs between bundles must reuse preallocated buffers, because a bundle boundary that allocates on the host reintroduces the allocator churn the AOT design removes (Chen, 2026a; Davies *et al.*, 2025). The artifact manifest should list every bundle, its target, its memory plan, and the bucket it serves (Chen, 2020; Rotem *et al.*, 2018).

19.3.3 Counting calls per token

The edge analog of launches/token is the number of compiled entry calls and buffer handoffs the host performs for one decode token (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). A single whole-model bundle reduces that count to one call; a per-bucket split adds a bucket-selection branch; a per-layer split multiplies calls by layer depth (Rotem *et al.*, 2018; Chen, 2026a). On a microcontroller, each

call has a measurable cost in stack setup, buffer pointer marshaling, and possible cache effects, so the call count belongs in the benchmark (Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026). The right granularity balances call count against memory: a whole-model bundle may need more SRAM than the device has, forcing a split even though the single call would be cheaper (AMD, 2024a; Chen, 2026a). The design review should therefore state the call-count target and the memory budget together, because optimizing one without the other fails on edge hardware (Davies *et al.*, 2025; Rotem *et al.*, 2018).

19.3.4 Vectorization on small cores

The LLVM backend’s vectorization matters on edge CPUs that support SIMD and on microcontrollers where only scalar code fits (PyTorch / Glow contributors, 2024; Goodfellow *et al.*, 2016). A stacked kernel’s loop should vectorize over the contiguous tensor dimension so each traversal also uses the widest available loads and arithmetic (Rotem *et al.*, 2018; Davies *et al.*, 2025). Cross-compiling the same bundle to a core without SIMD should degrade gracefully rather than fail, because the deployment target may be narrower than the build target (PyTorch / Glow contributors, 2024; Chen, 2020). The compile report should state the vector width actually selected for each stacked kernel so a perf regression can be traced to a target flag change (Chen, 2026a; Vellaisamy *et al.*, 2026). Operator stacking and vectorization are complementary: stacking removes passes, and vectorization makes each remaining pass cheaper (Rotem *et al.*, 2018; Chen, 2026a).

19.4 Glow tuning

Glow’s most distinctive tuning mechanism is **profile**-guided quantization, and it is profile-first by construction (Rotem *et al.*, 2018). The flow is two-phase: statically instrument the network with profiling nodes, run inference on representative data to record the numeric range of each activation, then recompile using that profile to convert the float network into int8 islands of computation, fusing away rescales between nodes (PyTorch / Glow contributors, 2024). On a memory-bound edge target this is a direct bytes/token win—int8 activations and weights move a quarter of the bytes of float32—and it reduces the model footprint enough to fit parts that float32 never would (Vellaisamy *et al.*, 2026; Chen, 2026a).

The **pitfall** is the profiling data and the operating point. The recorded ranges are only as representative as the inputs you profiled, so a profile captured on prefill-style batches can mis-estimate the activation ranges seen during long-context

batch-1 decode, producing saturation or precision loss exactly where it hurts output quality (Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Two disciplines follow: profile on decode-representative sequences at the context lengths you actually serve, and re-profile when the workload shifts, because the AOT bundle freezes the quantization decision at compile time. As always the right configuration is **hardware**-specific—an MCU may force int8 everywhere while a host CPU can keep sensitive layers in float—so the tuned bundle for one target does not transfer to another (AMD, 2024b; Davies *et al.*, 2025).

19.4.1 Quantization profile discipline

Profile-guided quantization turns a compiler feature into a data-quality problem: the recorded activation ranges determine both speed and output quality (Rotem *et al.*, 2018; Davies *et al.*, 2025). The profiling corpus must cover the context lengths, attention patterns, and temperature behavior the served workload actually exhibits, because saturation in a rare long-context request is invisible in a profile of short prompts (Kwon *et al.*, 2023; Dao *et al.*, 2022). Per-channel and per-tensor range choices should be explicit in the compile report so a precision regression can be traced to the layer whose range was mis-calibrated (Chen, 2020; Davies *et al.*, 2025). Re-profiling must be triggered by workload shifts, not just model changes, since the same weights can produce different activation ranges under different serving mixes (Chen, 2026a; Davies *et al.*, 2025). Teams that skip profile discipline get int8 bundles that pass unit tests and fail on real traffic (Vellaisamy *et al.*, 2026; Rotem *et al.*, 2018).

19.4.2 Islands, rescales, and fusion

Quantization in Glow converts parts of the graph into int8 islands separated by rescale operations, and the compiler’s goal is to eliminate as many rescales as possible so the islands fuse into single-traversal kernels (Rotem *et al.*, 2018; PyTorch / Glow contributors, 2024). Every surviving rescale is a potential full-tensor traversal, so the island boundary design determines how much of the int8 bytes win survives into actual traffic reduction (Chen, 2026a; Vellaisamy *et al.*, 2026). Decoder layers have natural island boundaries around matmuls, with elementwise chains inside islands and rescales at the numerical edges (Dao *et al.*, 2022; Davies *et al.*, 2025). The compile report should show island structure per layer, and a chain that splits into multiple int8 islands deserves the same scrutiny as an unfused chain in the datacenter compilers (Rotem *et al.*, 2018; Chen, 2020). Reviewers should ask where each rescale lives and whether it could have been

folded, because each removable rescale is a full bandwidth pass on the edge (Chen, 2026a; Davies *et al.*, 2025).

19.4.3 Target-specific precision policies

One float model should not quantize identically on every device. A microcontroller may need int8 everywhere to fit memory, while a host CPU can keep numerically sensitive layers in float and quantize only the bandwidth-bound matmuls (Rotem *et al.*, 2018; Goodfellow *et al.*, 2016; AMD, 2024a). Glow’s compile-time flow supports target-specific policies because the precision plan is part of the AOT bundle, not a runtime decision (PyTorch / Glow contributors, 2024; Rotem *et al.*, 2018). The policy should be derived from the residency budget and the quality requirement together: choose per layer the precision that fits the device and meets the error budget, then verify the bundle numerically (Dao *et al.*, 2022; Davies *et al.*, 2025). A policy table per target belongs in the release manifest so a precision change is a reviewable diff rather than an undocumented flag (Chen, 2020, 2026a). Porting a bundle between targets should force a policy review, never a silent copy (Rotem *et al.*, 2018; Vellaisamy *et al.*, 2026).

19.4.4 Benchmarking the bundle

The bundle benchmark should measure what the device will actually do per token: the compiled call, the buffer handoffs, and the memory traffic (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). It should run on the target or an accurate simulator because host benchmarks hide cache, power, and instruction-set differences (Goodfellow *et al.*, 2016; Chen, 2026a). The report should include latency, energy if measurable, peak memory from the static plan, and output quality error against the float reference (Rotem *et al.*, 2018; Davies *et al.*, 2025). Each candidate configuration (precision policy, island layout, bundle boundary) should produce one benchmark row so the trade becomes visible (Chen, 2020; Vellaisamy *et al.*, 2026). Edge decode teams should treat this benchmark as the acceptance gate, because no compiler flag matters more than the measured behavior on the actual device (Chen, 2026a; Rotem *et al.*, 2018).

19.4.5 Numerical error budgets across layers

Quantization error compounds through a decoder, so the per-layer range decisions should be governed by an end-to-end error budget rather than by local accuracy checks (Dao *et al.*, 2022; Davies *et al.*, 2025). Layers that amplify error, such

as attention accumulations and normalization, deserve wider ranges or higher precision even when their bytes could be reduced (Chen, 2026a; Dao *et al.*, 2022). The budget should be verified on decode-representative outputs, comparing the int8 bundle against the float reference with a bounded-error metric (Rotem *et al.*, 2018; Davies *et al.*, 2025). When the end-to-end error exceeds the budget, the trace locates the dominating layer and the fix targets that layer only (Chen, 2020; Rotem *et al.*, 2018). Teams that tune per-layer precision without an end-to-end budget will chase accuracy regressions one layer at a time after every workload shift (Vellaisamy *et al.*, 2026; Chen, 2026a). Run the bounded-error check at every shipped context bucket and request type, because rare quality failures hide in the tail of the workload distribution (Davies *et al.*, 2025; Dao *et al.*, 2022). A bundle that passes average-error gates can still violate the product quality SLO on the longest prompts or the least common routing pattern, because quantization error compounds with both attention depth and sequence length (Kwon *et al.*, 2023; Chen, 2026a).

19.5 Glow case study

Take a small decoder model compiled to an edge bundle as the **case**, and **benchmark** the int8 AOT bundle against a float interpreter at the decode operating point (Rotem *et al.*, 2018; Vellaisamy *et al.*, 2026). The reading matches the book’s thesis from the edge side: the win is not a faster matmul but the removal of dispatch overhead and HBM/DRAM round trips—the stacked kernel keeps element-wise work in one traversal and the static buffer plan avoids allocator churn, so bytes/token and launches/token both drop while the arithmetic is unchanged (Chen, 2026a; Dao *et al.*, 2022). The quantization adds a second bytes/token win by shrinking every activation to int8 (PyTorch / Glow contributors, 2024). The arithmetic is direct: a decoder layer that moves float32 activations spends four bytes per element, and converting to int8 cuts that to one, so on a bandwidth-bound edge core the memory traffic for those activations drops by roughly four times before any kernel fusion is counted (Vellaisamy *et al.*, 2026). That is the dominant reason int8 bundles are recommended for memory-constrained deployment: the speedup comes from moving fewer bytes, not from faster multiplies.

The comparison also exposes a limit worth stating plainly. An interpreter dispatches each operator through a generic runtime, paying overhead per node that, at batch-1, has no batch to amortize against—which is precisely the per-token tax Chapter 1 measured on the Python stack (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). The AOT bundle removes that tax structurally by compiling the whole

network into one function, so the speedup over an interpreter widens as the model gets deeper and the per-operator overhead would otherwise accumulate (Rotem *et al.*, 2018; Chen, 2026a).

A second lesson from the case is about debuggability under quantization. Because Glow emits debug information in terms of its own IR rather than raw machine code, a precision regression introduced by an int8 conversion can be traced back to the specific node whose recorded range was too tight, rather than guessing from a wrong output (Rotem *et al.*, 2018). For an edge decode deployment that traceability is the difference between a quick re-profile of one layer and a blind re-quantization of the whole model, and it is the kind of compile-time visibility the AOT path buys that a runtime interpreter cannot (PyTorch / Glow contributors, 2024; Davies *et al.*, 2025).

The **cross-hardware** conclusion is where Glow’s scope is honest. The same bundle source cross-compile from x86 to Cortex-M, but the two targets have wildly different residency budgets, so the fusion and quantization plan that fits the host may overflow the MCU’s SRAM (Rotem *et al.*, 2018; AMD, 2024a). And none of Glow’s CPU/edge strength competes with a CUDA-graph-captured Triton or IREE path on a datacenter GPU—which is the correct division of labor, not a deficiency (NVIDIA, 2024; SemiAnalysis, 2024). **YiRage** reflects exactly this: it uses Glow-style AOT lowering and operator stacking for edge and CPU targets while routing datacenter-GPU decode to its GPU backends, and carries the Chapter 2 residency rules as checkable IR metadata so an edge bundle that would overflow on-chip memory is rejected at compile time (Chen and YiRage Contributors, 2026).

19.5.1 Worked bundle worksheet

The edge case study produces a worksheet with the same rows as its datacenter counterparts, translated to edge units: the interpreter baseline’s per-token calls and memory passes, the AOT bundle’s compiled call count and traversal count, and the int8 bundle’s bytes-per-element reduction (Rotem *et al.*, 2018; Vellaisamy *et al.*, 2026; Chen, 2026a). Each row should list peak SRAM from the static plan so the memory claim is part of the benchmark (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). The win attribution should separate the three levers: operator stacking removes passes, static allocation removes allocator churn, and quantization shrinks bytes per element (Rotem *et al.*, 2018; Chen, 2026a). A bundle that only wins on one lever is still valuable but should be labeled accordingly, because the three levers fail differently on different workloads (Chen, 2020; Davies *et al.*, 2025). Publishing the worksheet with each bundle release keeps the edge team’s claims reproducible

(Vellaisamy *et al.*, 2026; Rotem *et al.*, 2018).

19.5.2 Precision regression tracing

When an int8 bundle changes output quality, the trace starts at the layer whose profile range was tightest relative to the observed traffic (Rotem *et al.*, 2018; Davies *et al.*, 2025). Glow’s IR-level debug information maps the wrong output back to the node and its recorded range, which turns a quality incident into a targeted re-profile (PyTorch / Glow contributors, 2024; Rotem *et al.*, 2018). The workflow is to compare the profile ranges against the ranges in the failing traffic, widen the mis-calibrated layer, recompile, and re-run the quality gate (Chen, 2020; Davies *et al.*, 2025). The fix should never be a global precision downgrade, because that surrenders the bytes win on layers that were calibrated correctly (Chen, 2026a; Vellaisamy *et al.*, 2026). This traceability is the practical payoff of compile-time visibility, and it is a capability the runtime interpreter cannot offer because it has no recorded compilation state (Rotem *et al.*, 2018; Chen, 2026a).

19.5.3 Edge host orchestration

The compiled bundle still needs a small host program to manage sensors, request scheduling, tokenization, and output handling around it (Kwon *et al.*, 2023; Davies *et al.*, 2025). That host code is part of the decode path, so it must be measured with the bundle: a host loop that copies buffers or allocates per token erases the AOT advantage (Chen, 2026a; Vellaisamy *et al.*, 2026). Buffer ownership should be fixed at startup, with the bundle reading and writing preallocated memory (Rotem *et al.*, 2018; Davies *et al.*, 2025). If the edge device runs several bundles or models, the host scheduler owns their interleaving and must respect the static memory plan of each (AMD, 2024a; Chen, 2026a). The boundary between compiled code and host logic is the edge analog of the eager-hull design from Chapter 12, and it deserves the same telemetry so per-token time is attributed correctly (NVIDIA, 2024; Vellaisamy *et al.*, 2026).

19.5.4 Failure autopsies for edge decode

Three signatures dominate Glow edge incidents. A bundle that fits in the compile report but faults on device usually means the static plan assumed more memory than the runtime left available, so the plan must be checked against the real allocator layout (Rotem *et al.*, 2018; AMD, 2024a). A quality regression after quantization traces to profile ranges, fixed by widening the mis-calibrated layers

(Rotem *et al.*, 2018; Davies *et al.*, 2025). A latency regression after a host change usually means the orchestration around the bundle started copying or allocating per token (Chen, 2026a; Vellaisamy *et al.*, 2026). Each autopsy starts from the artifact manifest and the compile reports, which makes it a diff against the last known-good bundle (Chen, 2020, 2026a). Teams that archive reports with bundles turn every incident into a bounded check (Rotem *et al.*, 2018; Vellaisamy *et al.*, 2026).

19.5.5 Quantization-first versus float-first bring-up

Bring-up ordering changes how quickly a team learns whether a model fits an edge device (Rotem *et al.*, 2018; Davies *et al.*, 2025). A float-first bring-up isolates precision and scheduling problems before quantization adds its own variables, but it may conclude the model does not fit when an int8 version would (Chen, 2026a; Vellaisamy *et al.*, 2026). A quantization-first bring-up measures the realistic footprint early, but it risks conflating quantization errors with schedule errors when something fails (Dao *et al.*, 2022; Davies *et al.*, 2025). The recommended path is a quick float feasibility compile to confirm the graph imports and the memory plan roughly fits, followed immediately by a profile-guided int8 compile at the served workload (Rotem *et al.*, 2018; Davies *et al.*, 2025). Each step publishes the same worksheet fields, so the team always knows whether the current blocker is structure, schedule, memory, or precision (Chen, 2020, 2026a). This ordering converts edge bring-up from a debugging spiral into a sequence of gates (Vellaisamy *et al.*, 2026; Rotem *et al.*, 2018).

19.6 Key Takeaways

- **Glow is the edge/CPU compiler, not the GPU-decode tool.** Its LLVM CPU backend and AOT bundles target memory-constrained devices, and lowering to a few linear-algebra primitives makes new accelerator backends cheap (Rotem *et al.*, 2018; PyTorch / Glow contributors, 2024). For datacenter GPU decode, prefer the CUDA-graph paths of Chapters 16–18 (NVIDIA, 2024).
- **Operator stacking is the bytes/token lever.** Glow’s stacked kernel performs chained element-wise work in a single tensor traversal instead of one kernel per operator, removing redundant DRAM round trips (PyTorch / Glow contributors, 2024; Chen, 2026a). Confirm chains were stacked by diffing the low-level IR (Vellaisamy *et al.*, 2026).

- **AOT bundles remove the launch loop entirely.** A self-contained object file with a single inference entry point has no framework dispatch, so the batch-1 launch overhead that dominates a Python stack does not exist (Rotem *et al.*, 2018; Vellaisamy *et al.*, 2026). It is the edge counterpart to capturing a launch graph (NVIDIA, 2024).
- **Profile-guided quantization needs decode-representative data.** Glow records activation ranges by running inference, then recompiles to int8; a profile captured on the wrong shape mis-estimates ranges and loses precision where decode needs it (Rotem *et al.*, 2018; Kwon *et al.*, 2023). Profile at served context lengths and re-profile on workload shift (Davies *et al.*, 2025).
- **The two-phase IR trades generality for a predictable footprint.** A fixed high-level graph plus low-level address-only IR (with static memory allocation) is less general than an MLIR dialect tower but yields a known memory plan—the right trade for the edge (Rotem *et al.*, 2018; Goodfellow *et al.*, 2016). Tuned bundles are per-target (AMD, 2024b).

Practical decision rules. Four rules summarize Glow for an edge decode review. First, set a traversal-count target per tensor chain and verify stacking in the low-level IR, because each traversal is a bandwidth pass on a memory-constrained core (Rotem *et al.*, 2018; Chen, 2026a). Second, treat the static memory plan as a hard contract: the plan plus the host’s reservations must fit the device before any benchmark matters (Goodfellow *et al.*, 2016; AMD, 2024a). Third, derive every quantization decision from a decode-representative profile and keep the per-layer ranges in the release manifest (Rotem *et al.*, 2018; Davies *et al.*, 2025). Fourth, measure the whole host-plus-bundle path on the target device, including calls, handoffs, and energy where available (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Teams that apply these rules get predictable edge decode; teams that skip them get a bundle that compiled cleanly and failed in the field (Chen, 2026a; Rotem *et al.*, 2018).

19.7 Conclusion

Glow shows that the residency-first thesis is not GPU-specific: at the edge, lowering to a few primitives, stacking operators into single-traversal kernels, statically planning memory, and quantizing under a recorded profile are the moves that drop bytes/token and erase launch overhead (Rotem *et al.*, 2018; PyTorch /

Glow contributors, 2024). Its deliberate scope—CPU, accelerators, and memory-constrained devices rather than datacenter GPUs—makes it the right backend for one slice of the hardware matrix and the wrong one for another, which is exactly the routing decision YiRage encodes as IR metadata (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). The broader point is that a lightweight compiler is not a lesser one: by giving up generality it buys a predictable memory plan and a dispatch-free runtime, and those are precisely the properties that make decode latency stable on a small device where a heavyweight stack would never fit (Rotem *et al.*, 2018; Davies *et al.*, 2025). Having now surveyed established compiler surfaces from TVM through Glow, Chapter 20 turns to Mirage and the emerging wave of AI compilers that push these ideas—whole-program scheduling, superoptimization, and residency-aware fusion—past what today’s production stacks ship (Chen, 2026a; Goodfellow *et al.*, 2016).

Chapter 20

Mirage and Emerging AI Compilers

The compilers of Chapters 16–19 all answer the same question—given this operator, schedule it well—but Mirage asks a different one: given this whole tensor program, *superoptimize* it, discovering optimizations that span algebraic rewrites, schedules, and entirely new custom kernels at once (Wu *et al.*, 2024). That shift matters for decode because the highest-leverage transformation is not a better matmul tile; it is collapsing a token’s worth of operators into one kernel so the host stops launching thousands of them. Dense Llama-3.2-1B averages 847.5 GPU launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). Mirage’s descendant, the Mirage Persistent Kernel (MPK), takes this to its conclusion by transforming LLM inference into a single megakernel and cutting end-to-end latency by 1.2–6.7 $\times$ (Mirage Project contributors, 2025). This chapter is also where the book’s own reference compiler comes from: **YiRage** is a Mirage-lineage, multi-backend descendant, so the ideas here—multi-level search, megakernel fusion, residency-as-metadata—are the spine of everything we have called YiRage in earlier chapters (Chen and YiRage Contributors, 2026). We treat Mirage and the emerging wave as production surfaces graded on bytes/token and launches/token, not as research curiosities (Chen, 2026a; Davies *et al.*, 2025).

20.1 Mirage HW matrix

Mirage’s “**matrix**” is not a list of vendor targets like IREE’s; it is the GPU compute hierarchy itself (Wu *et al.*, 2024). A μ Graph represents a tensor program



Figure 20.1: Mirage superoptimizes a tensor program over a multi-level μ Graph search space and emits a verified, fused kernel—one launch per token in the MPK limit.

uniformly at three levels—kernel, thread block, and thread—and that hierarchy *is* the **hardware** model the superoptimizer searches against (Wu *et al.*, 2024). The consequence is that Mirage is GPU-first by construction: the kernel/block/thread structure maps directly onto a CUDA **target**, and its strongest results are on NVIDIA GPUs (Mirage Project contributors, 2025).

The honest multi-hardware reading follows from that. On a **gpu** Mirage is exactly aligned with the residency hierarchy—block-level tensors live in shared memory, kernel-level tensors in device memory—so its search naturally optimizes bytes/token (Wu *et al.*, 2024; Chen, 2026a). On a **cpu** the kernel/block/thread abstraction does not map cleanly, so the superoptimization approach is emerging rather than production there (Goodfellow *et al.*, 2016). On an **npu** like XDNA/AIE the static-dataflow model is different again, and the value Mirage offers is conceptual—multi-level search and verified fusion—rather than a ready backend (AMD, 2024b,a).

Mirage is also a representative of a broader emerging wave, and the family resemblance is what matters more than any one project. The common thread across the new compilers is a move away from fixed, hand-authored pass pipelines toward *search with verification*: instead of trusting a human-proved rewrite, these systems explore a large space of equivalent programs and certify the winner, which lets them discover fusions and custom kernels that no pass library encodes (Wu *et al.*, 2024). The second shared theme is end-to-end fusion as a first-class goal rather than a peephole optimization—the megakernel is the clearest example, but the same instinct drives whole-program scheduling in IREE and aggressive operator fusion elsewhere (Mirage Project contributors, 2025; IREE / iree-org contributors, 2024). For a decode engineer the practical signal is that the frontier has shifted from “which library has the fastest GEMM” to “which compiler can collapse my whole token into the fewest, most resident launches and prove it correct” (Vellaisamy *et al.*, 2026; Chen, 2026a).

Multi-hardware delta. A Hopper GPU rewards the megakernel fusion Mirage discovers because it removes launches and keeps tensors in shared memory; a static NPU fabric needs the fusion decided at compile time with a fixed tile/DMA

plan instead (SemiAnalysis, 2024; AMD, 2024a). This is precisely why YiRage generalizes the Mirage idea across backends rather than shipping a single GPU artifact (Chen and YiRage Contributors, 2026).

20.1.1 Placement in the compiler family

Mirage belongs to the search-based branch of the compiler family, alongside TVM’s AutoTVM and Anso, but it differs in the object of search (Wu *et al.*, 2024; Chen *et al.*, 2018a; Zheng *et al.*, 2020). Where TVM searches schedules for operators whose math is fixed, Mirage searches the math itself, the schedule, and the kernel structure in one space (Wu *et al.*, 2024; Chen *et al.*, 2018b). That is why its discoveries can look like entirely new kernels rather than faster versions of known ones (Wu *et al.*, 2024; Davies *et al.*, 2025). For a decode engineer the practical consequence is that Mirage is not a replacement for the graph compilers of the previous chapters; it is a deeper optimizer that can be pointed at the fused regions those compilers produce (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026). The evaluation should therefore ask not whether Mirage compiles the whole model, but whether the regions it superoptimizes are the regions where the token spends its bytes and launches (Chen, 2026a; Davies *et al.*, 2025).

20.1.2 Where emerging compilers pay off

Superoptimization and verified megakernels pay off only on programs that are compiled once and executed many times, because the search cost is amortized over executions (Wu *et al.*, 2024; Kwon *et al.*, 2023). A served LLM’s decoder layers are the canonical example: the same layer graph runs on every token of every request, so a long search that finds a better fused kernel repays itself within hours of production traffic (Vellaisamy *et al.*, 2026; Chen, 2026a). One-off graphs, preprocessing pipelines, and rapidly changing research code do not justify the same budget (Wu *et al.*, 2024; Chen, 2020). The deployment rule is to compile hot regions with the deepest optimizer available and cold regions with the fastest conventional path (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). This hybrid is how a production stack can afford emerging-compiler techniques without making every build slow (Wu *et al.*, 2024; Chen, 2026a).

20.1.3 Reproducibility of search results

Search-based compilation introduces a reproducibility obligation: the same source can produce different kernels depending on search time, random seed, hardware

state, and library version (Wu *et al.*, 2024; Chen, 2020). A serving artifact must therefore record the search configuration, the verification report, and the measured reward of the accepted candidate (Chen, 2026a; Davies *et al.*, 2025). Reproducing a regression means re-running the same search with the same budget and seed, not merely recompiling (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026). Teams should archive the accepted μ Graph and the emitted kernel beside the benchmark so the compiled artifact is never a black box (Chen, 2020, 2026a). Search reproducibility is the price of admitting search into the production toolchain, and it must be paid in the release process rather than discovered during an incident (Wu *et al.*, 2024; Davies *et al.*, 2025).

20.1.4 Evaluation plan for an emerging compiler

Before adopting any emerging compiler in a decode stack, run a bounded evaluation with the same shape as this chapter’s gate: pick the served model and SKU, compile the hot decoder layers with the new tool and with the incumbent path, and compare launches/token, bytes/token, and ms/token at every shipped context bucket (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a). The evaluation must also record compile time, verification time, and artifact size, because those decide whether the approach is operationally viable (Wu *et al.*, 2024; Davies *et al.*, 2025). Include a rollback test that restores the incumbent path and confirms identical behavior (Chen, 2020, 2026a). An emerging compiler that wins the kernel benchmark but cannot produce a verifiable, rollback-capable artifact is not ready for serving regardless of its speedups (Davies *et al.*, 2025; Wu *et al.*, 2024).

20.1.5 Adoption maturity criteria

An emerging compiler deserves production adoption only when it meets the same maturity bar as the established surfaces: reproducible builds, verification artifacts, bounded compile time, rollback paths, and per-SKU regeneration (Wu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2026a). Research demos and published speedups are evidence of potential, not evidence of operational readiness (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026). The adoption checklist should mirror this chapter’s evaluation plan and add the maintenance dimension: who regenerates kernels when the model changes, and how long does that take (Chen, 2020; Wu *et al.*, 2024). A compiler that cannot be operated by the serving team is not an improvement regardless of its peak results (Davies *et al.*, 2025; Chen, 2026a). Maturity criteria keep the book’s yardstick honest as the field advances (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026).



Figure 20.2: Mirage’s μ Graph spans kernel (device memory), block (shared memory), and thread (register) levels; candidates are checked by a probabilistic equivalence verifier.

Table 20.1: Mirage’s μ Graph levels and the memory tier each one keeps tensors in; the level structure is the residency contract (Wu *et al.*, 2024).

Level	Memory tier	Role
Kernel graph	Device memory (HBM)	Whole-GPU kernels; tensors shared via DRAM
Block graph	Shared memory	Thread-block compute; intermediates on-chip
Thread graph	Registers	Per-thread scope; finest reuse

20.2 Mirage arch passes

Mirage’s “**pass**” pipeline is really a search, and that is the conceptual departure from every prior chapter (Wu *et al.*, 2024). Instead of a fixed sequence of rewrites, the superoptimizer explores a space of μ Graphs that jointly combine algebraic transformations, schedule transformations, and the generation of new custom kernels—an **optimization** no single hand-written pass would find (Wu *et al.*, 2024). The μ Graph levels carry the residency contract directly: a kernel graph keeps tensors in device memory, a block graph keeps intermediates in shared memory to cut device-memory access, and a thread graph **lowers** scope to registers (Wu *et al.*, 2024).

The level structure is not bookkeeping; it is the whole reason the search can reason about bytes/token. When the optimizer chooses to express a sub-computation as a block graph, it is committing those intermediates to shared memory and therefore removing the device-memory traffic a kernel-level expression would incur (Wu *et al.*, 2024; Chen, 2026a). When an intermediate is too large for shared memory, Mirage splits the work into multiple block graphs so that each one fits, which keeps the on-chip residency invariant intact instead of silently spilling (Wu *et al.*, 2024). A human writing CUDA makes these decisions implicitly and inconsistently; the μ Graph makes them explicit and searchable, which is what lets the superoptimizer trade device-memory traffic for shared-memory reuse deliberately.

Two ideas make this tractable and trustworthy. First, abstraction-based pruning shrinks the enormous search space while preserving a certain optimality guarantee, so the search is not brute force (Wu *et al.*, 2024). Second, a probabilistic equivalence

verifier checks each candidate μ Graph against the reference by random testing over finite fields—polynomial identity testing generalized to the LAX fragment (linear algebra plus division and exponentiation)—which avoids floating-point noise and drives the probability of accepting a wrong kernel arbitrarily low (Wu *et al.*, 2024). This is a different correctness model from a compiler that trusts each hand-proved rewrite, and it is what lets Mirage emit kernels a human never wrote. Notably, Mirage does not lower through an MLIR **dialect** tower; the multi-level μ Graph is a single uniform representation, which is why it can search across levels that a staged dialect pipeline keeps separate (Lai *et al.*, 2023; Wu *et al.*, 2024).

What the verifier buys. Because equivalence is checked rather than assumed, a discovered megakernel can be deployed with quantified confidence, not just empirical testing—the discipline a decode team needs before replacing a known-correct kernel-per-operator path with a fused one (Wu *et al.*, 2024; Davies *et al.*, 2025).

20.2.1 Search space and pruning guarantees

The μ Graph search space is too large for brute force, and Mirage’s abstraction-based pruning is what makes the search practical while preserving a formal optimality property (Wu *et al.*, 2024). The pruning works over the algebraic structure of the graph rather than by random sampling, so the search can discard whole equivalence classes without enumerating them (Wu *et al.*, 2024; Chen, 2020). For decode, the useful consequence is that the optimizer can reason about layers at the granularity of attention and MLP blocks instead of individual matmuls (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). The search should therefore be seeded with the layer structure the model actually has, so the optimizer explores megakernel arrangements rather than re-deriving the layer decomposition from scratch (Wu *et al.*, 2024; Davies *et al.*, 2025). Teams should record which pruning assumptions held in the compile report, because a future model whose structure violates them may silently narrow the effective search (Chen, 2026a; Wu *et al.*, 2024).

20.2.2 Verification coverage boundaries

The probabilistic verifier certifies candidates inside the LAX fragment—linear algebra plus division and exponentiation—and the guarantee is scoped to that fragment (Wu *et al.*, 2024). Operators outside the fragment need a different correctness argument, which means the deployable confidence of a fused kernel is only as broad as the fragment that generated it (Wu *et al.*, 2024; Davies *et al.*,

2025). A decode layer mixes in-fragment operations (matmul, softmax-related exponentials, elementwise arithmetic) with framework behavior that may not be covered, such as sampling or custom kernels (Dao *et al.*, 2022; Mirage Project contributors, 2025). The deployment checklist should therefore enumerate which parts of the fused kernel are verifier-certified and which rest on conventional testing (Chen, 2020, 2026a). This boundary is not a weakness; it is a precise statement of where the quantified confidence ends and where empirical validation must begin (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026).

20.2.3 Verification in the release pipeline

The equivalence check should run as a first-class step in CI for every accepted candidate, alongside the byte and launch benchmarks (Wu *et al.*, 2024; Davies *et al.*, 2025). The verification report should state the fragment coverage, the number of random tests, and the resulting bound on accepting a wrong kernel (Wu *et al.*, 2024; Chen, 2020). A model or compiler upgrade that changes the candidate should trigger a fresh verification before the artifact is released (Chen, 2026a; Vellaisamy *et al.*, 2026). Because the verifier is probabilistic, the CI configuration should choose the test budget to meet the serving team’s correctness bar, and that bar should be documented in the runbook (Wu *et al.*, 2024; Davies *et al.*, 2025). Teams that treat verification as an optional research step will eventually ship a search artifact whose correctness they merely hope for (Chen, 2026a; Wu *et al.*, 2024).

20.2.4 Choosing megakernel boundaries

The megakernel boundary is a search decision with operational consequences: fusing too little leaves launch overhead, while fusing too much risks spills and long device-side dependencies (Wu *et al.*, 2024; Mirage Project contributors, 2025; Chen, 2026a). The search should be allowed to propose boundaries that match layer structure—attention core, attention plus projections, or the full decode layer—and the accepted boundary should be the one that minimizes measured ms/token at the served bucket without violating on-chip budgets (Dao *et al.*, 2022; Wu *et al.*, 2024). A boundary that looks elegant in the IR but produces a long serial dependency chain may be slower than a two-kernel split that overlaps better (Mirage Project contributors, 2025; Davies *et al.*, 2025). The compile report should justify the chosen boundary with the task graph, not with a naming convention (Wu *et al.*, 2024; Chen, 2026a). Decode teams should treat boundary selection as a benchmarked parameter that changes per model and SKU (Vellaisamy *et al.*, 2026; NVIDIA, 2022).

20.3 Mirage codegen

Mirage’s **codegen** turns a verified μ Graph into CUDA, and MPK extends this to **emit** an entire LLM forward pass as one megakernel (Wu *et al.*, 2024; Mirage Project contributors, 2025). The mechanism is an SM-level task graph (tGraph) that captures data dependencies at the granularity of individual streaming multiprocessors, so the **backend** can schedule cross-operator software pipelining and communication–computation overlap *inside* a single kernel launch, with decentralized scheduling across SMs (Mirage Project contributors, 2025). The **platform** is the GPU, and the generated CUDA includes the intra-SM optimizations—software pipelining, register reuse, bank-conflict-aware layouts—that a per-operator emit cannot coordinate across operators (Mirage Project contributors, 2025).

For decode this is the most direct attack on launches/token in the book. A kernel-per-operator stack pays host dispatch for every one of the thousands of launches a token requires; a megakernel pays it once (Vellaisamy *et al.*, 2026; Chen, 2026a). MPK reports $1.2\text{--}6.7\times$ end-to-end latency reductions from exactly this collapse, pushing inference toward hardware limits, and it does so from a few dozen lines of Python that name the fused layers—for example one call fusing RMSNorm and a linear projection (Mirage Project contributors, 2025). What codegen still cannot fix is an algorithm that is genuinely device-memory bound: if the working set does not fit on chip, even a perfect megakernel must stream it (Goodfellow *et al.*, 2016).

The deeper reason the megakernel helps is that it changes what can overlap. In a kernel-per-operator schedule, the boundary between two operators is also a synchronization point: the host waits for one kernel to finish before launching the next, so the GPU idles through dispatch latency and any tail effects (Vellaisamy *et al.*, 2026). Inside a single megakernel those boundaries become intra-kernel dependencies that the SM-level task graph can pipeline—one SM can start the next task’s loads while another finishes the current task’s compute, and collective communication can overlap with computation instead of stalling behind it (Mirage Project contributors, 2025). At batch-1 decode, where there is no batch dimension to hide latency behind, this fine-grained overlap is often the only source of parallelism left, which is why the megakernel wins are largest exactly in the regime that hurts a conventional stack most (Chen, 2026a; Kwon *et al.*, 2023).

Multi-hardware delta. The megakernel’s single-launch win is a GPU phenomenon—it removes CUDA launch and host-dispatch overhead; on an NPU the equivalent is a static single-dispatch schedule, and on a CPU it is the AOT

bundle of Chapter 19 (AMD, 2024a; Rotem *et al.*, 2018). Same principle, different mechanism per class (SemiAnalysis, 2024).

20.3.1 SM-level task graph scheduling

The tGraph moves scheduling from the host into the device by expressing dependencies at SM granularity, which is what makes intra-kernel pipelining and overlap possible (Mirage Project contributors, 2025). A task graph node corresponds to work an SM can start once its inputs are ready, and the scheduler keeps SMs busy by advancing independent tasks while a dependency resolves (Mirage Project contributors, 2025; Vellaisamy *et al.*, 2026). For decode, this replaces the kernel-boundary stalls of a per-operator stack with fine-grained producer-consumer overlap (Mirage Project contributors, 2025; Chen, 2026a). The codegen review should inspect the task graph for serialization: a long chain of dependent tasks with no overlap opportunity indicates that fusion created a schedule but not a pipelined one (Davies *et al.*, 2025; Mirage Project contributors, 2025). The tGraph is also where communication tasks attach, so collective and copy operations appear as first-class graph nodes whose latency the scheduler can hide (Mirage Project contributors, 2025; SemiAnalysis, 2024).

20.3.2 Register and shared-memory budgets during codegen

Emitting a megakernel concentrates every intermediate into the same kernel, so the codegen stage must respect the GPU’s register file and shared-memory capacity while scheduling the task graph (Mirage Project contributors, 2025; NVIDIA, 2022). When the fused working set exceeds those budgets, the emitter either spills or splits the megakernel into sub-kernels, and the split point becomes a new launch boundary (Chen, 2026a; Vellaisamy *et al.*, 2026). The compile report should state the peak shared-memory and register usage of the accepted kernel and compare them to the device limits (Wu *et al.*, 2024; Davies *et al.*, 2025). A megakernel that only fits by spilling may lose the launch-collapse benefit, because the spill traffic can exceed the dispatch overhead it removed (Chen, 2026a; Mirage Project contributors, 2025). The right fusion level is therefore not the maximum the search can express but the maximum that fits the target without spilling, measured at the decode shape (NVIDIA, 2022; Chen, 2026a).

20.3.3 Portability across GPU generations

Mirage’s accepted kernels are generated against a concrete hardware model—SM count, shared-memory capacity, register budget, and async-copy behavior (Wu *et al.*, 2024; NVIDIA, 2022). Moving the same model to a different GPU generation requires regenerating and re-verifying the kernel rather than reusing the artifact (Wu *et al.*, 2024; Davies *et al.*, 2025). A fleet with mixed GPUs therefore needs per-SKU search artifacts and a dispatch layer that selects the matching kernel (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Driver changes can also invalidate an artifact if they alter scheduling behavior or instruction selection, so the artifact manifest must pin the driver and CUDA version used during search and codegen (Chen, 2020, 2026a). Portability in the Mirage world means the reference program is portable and the search re-runs per target; it does not mean the emitted kernel is universal (Wu *et al.*, 2024; NVIDIA, 2022).

20.4 Mirage tuning

In Mirage, **tuning** and compilation are the same act: the superoptimizer searches the μ Graph space and the probabilistic verifier accepts only equivalent candidates, so “tuning” means letting the search run rather than hand-listing configurations (Wu *et al.*, 2024). You still **profile** candidates—the search is guided by measured performance and bounded by the abstraction-based pruning—but the engineer’s job is to set the search budget and the operating point, not to write schedules (Wu *et al.*, 2024).

The **pitfall** is twofold and specific. First, superoptimization is expensive: searching and verifying a large μ Graph space costs real compile time, so it pays for hot, reused programs (a served LLM’s decode layer) and not for one-off graphs (Wu *et al.*, 2024; Kwon *et al.*, 2023). Second, the correctness guarantee is probabilistic and scoped to the LAX fragment; operators outside that fragment do not get the same verification strength, so a team must know which parts of its model the guarantee actually covers (Wu *et al.*, 2024). As elsewhere, the right search target is **hardware**-specific—a μ Graph tuned for one GPU’s shared-memory size and SM count is not automatically optimal on another, so re-search per SKU rather than assuming transfer (NVIDIA, 2022; Davies *et al.*, 2025). A practical way to budget the search is to spend it where the token spends its time: the decode-critical attention and projection layers are reused on every step and every request, so a long superoptimization run there amortizes over billions of tokens, whereas one-off preprocessing graphs rarely justify the cost (Wu *et al.*, 2024; Kwon *et al.*, 2023).

20.4.1 Budgeting search across a model

The search budget should follow where the token spends its time, not where the graph is largest (Wu *et al.*, 2024; Kwon *et al.*, 2023). Attention and projection layers execute once per layer per token, so their superoptimization amortizes immediately; sampling, tokenization, and rare routing paths do not justify long searches (Vellaisamy *et al.*, 2026; Chen, 2026a). The budget ledger should record search seconds per region, the reward improvement found, and the measured wall-clock cost of the search (Wu *et al.*, 2024; Davies *et al.*, 2025). When a model changes, only the affected regions need re-search if the accepted artifacts for unchanged regions are cached and still valid for the target (Chen, 2020; Wu *et al.*, 2024). Teams should also bound search by an acceptance threshold: stop when additional search time stops improving the measured decode counters, because the marginal benefit of the last few hours is often negligible (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026). Budgeting turns an open-ended superoptimizer into a production tool with predictable compile cost (Chen, 2026a; Davies *et al.*, 2025).

20.4.2 Measured reward and candidate selection

Search candidates must be ranked by the same counters the rest of the book uses: launches per token, bytes per token, and ms/token at the served shape (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). A candidate that wins an isolated kernel benchmark but raises bytes/token when composed into the layer should be rejected even if its local numbers look better (Wu *et al.*, 2024; Chen, 2026a). The reward measurement should run the candidate exactly as serving will, inside the same launch context and with the same input pointers (Davies *et al.*, 2025; Mirage Project contributors, 2025). When two candidates are close in reward, prefer the one with the smaller shared-memory footprint and simpler task graph, because those properties predict robustness across context buckets (Wu *et al.*, 2024; Chen, 2026a). The accepted candidate’s reward row belongs in the release manifest next to the verification report (Chen, 2020; Vellaisamy *et al.*, 2026).

20.4.3 Caching results across model families

Different models share structural subgraphs—attention blocks, normalization-plus-linear chains—and cached search results for a shared structure can save hours of recompilation (Wu *et al.*, 2024; Dao *et al.*, 2022). The cache key must include the subgraph structure, data types, tensor shapes, and target description, because

any of those changes can invalidate the accepted kernel (Wu *et al.*, 2024; Davies *et al.*, 2025). Reusing a cached kernel for a new model still requires verification against that model’s reference, since equivalence is a property of the candidate and the reference program together (Wu *et al.*, 2024; Chen, 2020). A cache that skips verification is not a cache; it is a correctness risk (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026). Well-designed caches make superoptimization economical across a model zoo, which is the deployment condition that makes the approach sustainable (Davies *et al.*, 2025; Chen, 2026a).

20.4.4 Nightly revalidation cadence

Discovered artifacts decay on the same triggers as every other compiled output: model updates, GPU driver changes, CUDA upgrades, and new context buckets all require regeneration and re-verification (Wu *et al.*, 2024; Chen, 2026a; Davies *et al.*, 2025). A nightly job should re-run the accepted kernels against the current reference at the served buckets and alert on counter drift or numerical divergence (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026). The alert triggers a bounded re-search seeded with the previous accepted candidate, which converges faster than a cold search (Wu *et al.*, 2024; Chen, 2020). Full re-verification belongs in the release pipeline whenever the artifact manifest changes (Wu *et al.*, 2024; Chen, 2026a). This cadence converts superoptimization from a one-time research deliverable into a maintained production contract (Davies *et al.*, 2025; Wu *et al.*, 2024).

20.4.5 Building the hot-region list

The search budget should be driven by an explicit hot-region list derived from a decode profile: rank every subgraph by its contribution to per-token time and bytes, then assign superoptimization only to the top regions that dominate the total (Wu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2026a). The list should be regenerated when the model or the serving shape changes, because a region that was hot at one context length may not be at another (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Each region on the list gets a search budget proportional to its measured share, which prevents the optimizer from spending hours on a chain that contributes a fraction of a percent (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026). The hot-region list and the budget ledger belong in the release manifest, making the search strategy itself a reviewable artifact (Chen, 2020, 2026a). This is how a serving team keeps an expensive superoptimizer pointed at the bytes and launches that actually move the token (Wu *et al.*, 2024; Davies *et al.*, 2025). Update the list when the traffic mix changes, because hot regions track the workload and the context distribution

as much as they track the model (Davies *et al.*, 2025; Chen, 2026a).

20.5 Mirage case study

The decisive **case** is MPK compiling a real LLM—an open Hugging Face model such as Qwen3-8B—into a single megakernel and **benchmarking** it against a kernel-per-operator serving stack (Mirage Project contributors, 2025). The result restates the book’s thesis from the superoptimizer’s side: fusing the whole decode step into one launch removes the per-operator host dispatch that dominates batch-1, and the SM-level task graph overlaps the communication and computation that a kernel-per-operator schedule serializes, yielding the reported $1.2\text{--}6.7\times$ (Mirage Project contributors, 2025; Vellaisamy *et al.*, 2026). The arithmetic is familiar—each avoided inter-operator round trip saves the activation bytes that would otherwise stream through device memory per token—only now the compiler discovers the fusion rather than a human writing it (Chen, 2026a; Dao *et al.*, 2022).

It is worth being concrete about why the search, not just the fusion, is the contribution. A hand-written megakernel for one model is a heroic engineering effort that does not transfer to the next architecture; Mirage’s superoptimizer regenerates the fused kernel from a reference implementation whenever the model or the GPU changes, and the probabilistic verifier certifies each result (Wu *et al.*, 2024; Mirage Project contributors, 2025). For a serving team that means the megakernel is not a brittle artifact to maintain by hand but a compiler output to regenerate, which is the only way end-to-end fusion is sustainable across a model zoo that changes monthly. The cost, as the tuning section noted, is search time—paid once per model-target pair and amortized over every token served.

The **cross-hardware** conclusion is where the emerging wave and this book converge. Mirage proves the GPU case; the open problem is carrying multi-level search and verified megakernel fusion to CPUs, NPUs, and heterogeneous deployments where the launch-collapse mechanism differs (AMD, 2024b,a). **YiRage** is the book’s answer: a Mirage-lineage compiler that generalizes superoptimization and megakernel fusion across multiple backends and carries the Chapter 2 residency rules as checkable IR metadata, so a discovered fusion that would spill on a given target is rejected at compile time rather than shipped (Chen and YiRage Contributors, 2026). That is why every prior chapter measured its compiler against the same YiRage yardstick—this is where the yardstick comes from (Davies *et al.*, 2025).

20.5.1 Worked RMSNorm-plus-linear fusion

The smallest MPK example is representative: a call that fuses RMSNorm and the following linear projection into one kernel (Mirage Project contributors, 2025). A kernel-per-operator version reads the hidden state, computes normalization statistics, writes the normalized tensor, and reads it back for the projection; the fused version carries the normalized values on-chip between the two operations (Mirage Project contributors, 2025; Chen, 2026a). At hidden size $d=4096$ in BF16, that removes one full-width write and one read per layer per token (Chen, 2026a; Vellaisamy *et al.*, 2026). The win is small per layer and large across depth, which is why it matters only when measured over the whole model at the decode shape (Dao *et al.*, 2022; Davies *et al.*, 2025). The worked example should be reproduced on the serving model and SKU before trusting the published numbers, because the search and codegen are hardware-specific (Wu *et al.*, 2024; NVIDIA, 2022).

20.5.2 Verification workflow in serving

Promoting a discovered megakernel into serving should follow a fixed workflow: run the probabilistic verifier on the candidate, record the fragment coverage and confidence, benchmark the candidate at the served buckets, and only then replace the reference path behind a runtime switch (Wu *et al.*, 2024; Davies *et al.*, 2025). The switch should keep the reference path available so a field regression can fall back without redeployment (Mirage Project contributors, 2025; Chen, 2026a). Telemetry should tag which kernel variant ran for each request so the megakernel’s actual behavior is observable in production (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Because the verifier is probabilistic, the serving team should also keep a bounded-error numerical check against the reference on real traffic as a second line of defense (Wu *et al.*, 2024; Chen, 2026a). This workflow converts a research-strength verifier into a deployable confidence gate (Wu *et al.*, 2024; Chen, 2020).

20.5.3 Autopsies for megakernel regressions

Megakernel failures have distinctive signatures. A latency regression with correct output usually means the fused kernel spills at the served context bucket even though it fit the search shape; the fix is to re-search with the real bucket’s footprint (Mirage Project contributors, 2025; Chen, 2026a). A numerical drift usually means part of the fused graph fell outside the verified fragment or the verifier budget was too small; the fix is to expand coverage or testing (Wu *et al.*, 2024; Davies *et al.*, 2025). A regression after a driver or GPU change usually means the artifact

was not regenerated for the new environment, which the manifest should have prevented (Chen, 2020; Vellaisamy *et al.*, 2026). Each autopsy starts from the verification report and the artifact manifest, making the diagnosis a bounded check rather than a rewrite (Wu *et al.*, 2024; Chen, 2026a). Teams that keep the reference path and telemetry in place can treat megakernel incidents as configurable events instead of emergencies (Davies *et al.*, 2025; Mirage Project contributors, 2025).

20.5.4 The memory-bound limit

Megakernel fusion removes launches and inter-operator round trips, but it cannot create on-chip capacity that does not exist (Wu *et al.*, 2024; Chen, 2026a). When a model’s working set—weights, KV, and activations—exceeds the GPU’s on-chip storage at the served context bucket, the fused kernel must stream from HBM and the fusion win shrinks toward the memory floor (Chen, 2026a; Vellaisamy *et al.*, 2026). The case study should therefore report the ratio of working set to on-chip capacity at each bucket, because that ratio predicts how much of the megakernel’s theoretical win is reachable (Wu *et al.*, 2024; Davies *et al.*, 2025). At long context, KV streaming dominates and the fusion of the attention loop remains valuable even when whole-layer fusion is no longer profitable (Dao *et al.*, 2022; Mirage Project contributors, 2025). Knowing the memory-bound limit keeps expectations honest: the megakernel is the best possible schedule of the bytes that must move, not a substitute for moving fewer bytes (Chen, 2026a; Wu *et al.*, 2024).

20.5.5 Transitioning from incumbent kernels

Replacing a known-good kernel-per-operator path with a discovered megakernel should be staged like any risky deployment (Mirage Project contributors, 2025; Davies *et al.*, 2025). Run shadow traffic through both paths, compare outputs under a bounded-error metric, and verify that the latency win survives the real request mix before switching production traffic (Wu *et al.*, 2024; Chen, 2026a). The transition window should keep telemetry on both paths so any difference is attributable to the artifact change (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Rollback must be a runtime switch rather than a redeploy, because a megakernel incident discovered in the field needs a fast escape hatch (Mirage Project contributors, 2025; Chen, 2020). The staged transition is also the best evidence the search reward function matches the production objective, since a kernel that wins the search benchmark but loses shadow traffic reveals a reward mismatch (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026).

20.5.6 Cross-backend generalization

The megakernel’s mechanism is GPU-specific, but the ideas generalize to other hardware classes with different mechanisms (Wu *et al.*, 2024; AMD, 2024b; Rotem *et al.*, 2018). An NPU needs the fused schedule fixed at compile time as a static dataflow graph; a CPU needs the AOT bundle with stacked kernels; both are forms of end-to-end fusion with verification, expressed in the machine model of each class (AMD, 2024b; Rotem *et al.*, 2018; Goodfellow *et al.*, 2016). Emerging compilers are beginning to explore these translations, and the evaluation criterion is always the same: launches or dispatch count falls, bytes moved falls, and the artifact is verified against the reference (Wu *et al.*, 2024; Chen, 2026a). YiRage embodies this direction by generalizing search and megakernel ideas across backends while keeping residency rules as checkable metadata (Chen and YiRage Contributors, 2026). Teams should watch emerging compilers for cross-backend support but should not assume the GPU result transfers until it is verified on their target (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

20.5.7 Caveats about probabilistic verification

Probabilistic verification is powerful but has limits that deployment must respect (Wu *et al.*, 2024; Davies *et al.*, 2025). The confidence bound assumes the random testing configuration is correct, so the CI configuration must be part of the artifact record (Wu *et al.*, 2024; Chen, 2020). Verification covers the fragment the verifier understands; operators, dtypes, or control flow outside it need conventional testing with real traffic (Wu *et al.*, 2024; Chen, 2026a). Floating-point behavior adds another layer: equivalence over finite fields proves algebraic identity, not bitwise identity, so a bounded numerical tolerance check remains necessary in production (Dao *et al.*, 2022; Wu *et al.*, 2024). Teams should document the combination of probabilistic verification and numerical testing that a release actually received, because the two together form the correctness argument (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026).

20.6 Key Takeaways

- **Superoptimization replaces hand-written passes with search.** Mirage jointly searches algebraic rewrites, schedules, and new custom kernels over a multi-level μ Graph, finding fusions no fixed pass would (Wu *et al.*, 2024). For decode the prize is collapsing per-token launches, not a faster single matmul

(Vellaisamy *et al.*, 2026).

- **The μ Graph encodes residency by level.** Kernel graphs use device memory, block graphs use shared memory, thread graphs use registers, so the search optimizes bytes/token by construction (Wu *et al.*, 2024; Chen, 2026a). The representation is the hardware model.
- **Verified megakernels attack launches/token directly.** MPK fuses an LLM forward pass into one launch via an SM-level task graph, reporting $1.2\text{--}6.7\times$ latency reduction by removing per-operator host dispatch (Mirage Project contributors, 2025; NVIDIA, 2024). One launch per token is the limit case of this book’s thesis.
- **Probabilistic equivalence makes discovered kernels deployable.** Checking candidates by polynomial identity testing over finite fields (LAX fragment) gives quantified correctness, so a fused kernel can replace a known-good path with confidence—but only within the verified fragment (Wu *et al.*, 2024). Know what the guarantee covers (Davies *et al.*, 2025).
- **Search cost and SKU-specificity are the trade-offs.** Superoptimization pays for hot, reused decode layers, not one-off graphs, and a μ Graph tuned for one GPU is not optimal on another (Wu *et al.*, 2024; NVIDIA, 2022). Budget the search and re-tune per target (Kwon *et al.*, 2023).

Practical decision rules. Four rules summarize this chapter for a serving team. First, point superoptimization at hot, reused decode regions and keep conventional compilation for cold graphs, because search cost only amortizes over repeated execution (Wu *et al.*, 2024; Kwon *et al.*, 2023). Second, treat the verification report as a release artifact: record the fragment coverage, the confidence bound, and the reference it was checked against (Wu *et al.*, 2024; Davies *et al.*, 2025). Third, keep the reference path and telemetry in production behind a runtime switch so a discovered kernel can be rolled back without redeployment (Mirage Project contributors, 2025; Chen, 2026a). Fourth, regenerate and re-verify artifacts whenever the model, GPU, driver, or served bucket changes, because nothing about a search result transfers without re-validation (Wu *et al.*, 2024; NVIDIA, 2022). Teams that apply these rules get the megakernel’s latency win without giving up correctness; teams that skip them get a fast kernel they cannot debug (Vellaisamy *et al.*, 2026; Chen, 2026a).

20.7 Conclusion

Mirage reframes compilation as multi-level superoptimization with verified correctness, and MPK shows its decode payoff: a single megakernel that collapses a token’s launches and overlaps communication with computation inside one kernel, for $1.2\text{--}6.7\times$ (Wu *et al.*, 2024; Mirage Project contributors, 2025). This is the intellectual source of the YiRage compiler used as the yardstick throughout the book—multi-level search, megakernel fusion, and residency encoded as checkable metadata—generalized beyond a single GPU artifact (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). The broader lesson is that the field is converging on search plus verification as the way to reach fewer, more resident launches, and that decode is the workload where this convergence pays off first because it has the least batch parallelism to fall back on (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026). Having now surveyed each compiler surface individually, Chapter 21 steps back to a unified analysis and a cross-hardware benchmark, comparing these stacks on the same decode metrics rather than on each project’s favored microbenchmark (Chen, 2026a; Goodfellow *et al.*, 2016).

Chapter 21

Unified AI Compiler Analysis and Cross-Hardware Benchmark

Each compiler in this Part advertises its own headline number on its own favored workload, which makes them almost impossible to compare honestly. This chapter does the comparison the projects do not: it rates TVM, Triton, IREE, Glow, and Mirage on one axis that actually predicts interactive serving cost—how well each collapses a token’s work into few, resident kernel launches—rather than on peak GEMM throughput (Chen *et al.*, 2018b; Tillet *et al.*, 2019; IREE / iree-org contributors, 2024; Rotem *et al.*, 2018; Wu *et al.*, 2024). The yardstick is the one Chapter 1 set: dense Llama-3.2-1B averages 847.5 GPU launches per output token on H100 and OLMoE-1B/7B averages 9,305, so the metric that matters is launches/token and bytes/token at batch-1, not datasheet FLOPs (Vellaisamy *et al.*, 2026; Chen, 2026a). The point is not to crown a winner but to show that these stacks occupy different niches, and that the right choice is a routing decision—which is exactly what the book’s YiRage compiler encodes (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

21.1 Compiler ratings

A useful **compiler rating** for decode is not a single score; it is a profile across five axes: decode-fusion strength, launch collapse, multi-hardware breadth, search/compile cost, and correctness guarantee (Davies *et al.*, 2025). Rated this way the stacks separate cleanly. TVM is broad and search-driven but graph-and-TensorIR-staged; Triton is a GPU block compiler with hand-placed

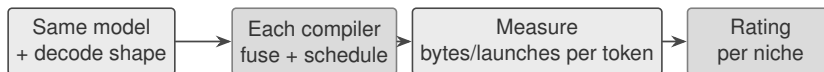


Figure 21.1: A fair comparison rates each **compiler** on the same decode metrics—bytes/token and launches/token—instead of each project’s favored microbenchmark.

Table 21.1: Decode-oriented **rating** of the surveyed compilers; “launch collapse” is the batch-1 lever and the column that most predicts ms/token (Vellaisamy *et al.*, 2026; Chen, 2026a).

Compiler	Primary niche	Launch-collapse mechanism
TVM	Broad, search-driven	Fusion + graph executor / AOT
Triton	GPU block kernels	Hand-placed fusion + CUDA Graphs
IREE	Widest HW matrix	Whole-program stream scheduling
Glow	Edge / CPU	AOT bundle (no launch loop)
Mirage	GPU superoptimizer	Searched megakernel

fusion boundaries; IREE is the widest hardware matrix with whole-program scheduling; Glow is the edge/CPU specialist; Mirage is the superoptimizer that discovers fusion and verifies it (Chen *et al.*, 2018b; Tillet *et al.*, 2019; IREE / iree-org contributors, 2024; Rotem *et al.*, 2018; Wu *et al.*, 2024).

It is worth being explicit about why the niches are real and not marketing. The differences trace to architectural commitments made early in each project. TVM committed to a learned-cost-model search over a graph-plus-TensorIR split, which buys breadth and automation at the cost of a staged lowering that can leave fusion on the table (Chen *et al.*, 2018b; Zheng *et al.*, 2020). Triton committed to a Python tile abstraction over a GPU SIMT machine, which buys productivity and precise GPU control but leaves the fusion boundary to the author and excludes static NPUs (Tillet *et al.*, 2019; OpenAI / triton-lang contributors, 2024). IREE committed to whole-program compilation through an MLIR dialect stack and a hardware abstraction layer, which buys the widest matrix and compiled host scheduling at the cost of hiding per-target residency (IREE / iree-org contributors, 2024). Glow committed to a two-level IR and AOT bundles, which buys a predictable edge footprint by giving up datacenter-GPU depth (Rotem *et al.*, 2018). Mirage committed to multi-level superoptimization with verification, which buys reachable megakernels at the cost of search time (Wu *et al.*, 2024).

The rating exposes a recurring truth: a stack can score high on isolated kernel quality and low on launch collapse, and at batch-1 the second column is the one that moves p99 (Vellaisamy *et al.*, 2026; NVIDIA, 2024). On a GPU the

launch-collapse mechanisms differ—Triton relies on the author to widen the fused region then captures it, IREE schedules the whole program, Mirage searches for the megakernel—but all three are attacking the same orchestration tax (Chen, 2026a; Mirage Project contributors, 2025). The mistake a team makes is to read one project’s headline speedup as a global ranking; the speedups are measured against different baselines on different workloads, and only a common decode harness makes them comparable (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

21.1.1 Rating methodology

The rating methodology must fix the variables that published benchmarks leave free: the model, the checkpoint, the tokenizer, the context buckets, the batch size, the driver and runtime versions, and the metric definitions (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a). Every compiler under test should receive the same reference program and be asked to produce its best artifact for the same decode shape (Chen, 2020; Davies *et al.*, 2025). The harness then measures each artifact on the same three counters and publishes the configuration with the numbers, because a benchmark without a pinned configuration is a claim without an experiment (Chen, 2026a; Vellaisamy *et al.*, 2026). The methodology should also state what each compiler was allowed to tune: a search-based stack gets its search budget, an AOT stack gets its compile time, and the budget should be proportional to the deployment model the comparison is meant to inform (Wu *et al.*, 2024; Zheng *et al.*, 2020). Without this discipline, the rating measures the benchmark authors’ generosity rather than the compilers.

21.1.2 What the columns mean operationally

The five rating axes translate directly into deployment questions. Decode-fusion strength asks whether the stack can keep a layer’s intermediates on-chip at batch-1; launch collapse asks how many host dispatches survive per token; multi-hardware breadth asks whether one program covers the fleet; search and compile cost asks how long a new model takes to deploy; and correctness guarantee asks what confidence the artifact carries (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026). A team choosing a compiler for one product should weight these columns by their actual constraints rather than by an average score (Chen, 2020; Davies *et al.*, 2025). A single-SKU GPU serving team weights launch collapse and correctness highest; a fleet with many device types weights breadth; a product that recompiles nightly weights compile cost (Kwon *et al.*, 2023; Davies *et al.*, 2025). The rating table is therefore a decision aid with weights supplied by the reader, not a leaderboard

with a universal winner (Vellaisamy *et al.*, 2026; Chen, 2026a).

21.1.3 Reading a rating table

Rating tables mislead when read as ranks instead of profiles (Davies *et al.*, 2025; Chen, 2026a). A cell that says a compiler has weak multi-hardware breadth is not a bug in that compiler; it may be the deliberate price of deep GPU residency reasoning (OpenAI / triton-lang contributors, 2024; Wu *et al.*, 2024). The reader should locate the workload’s constraints in the table first and then identify the stacks whose strengths match those constraints (Chen, 2020; Vellaisamy *et al.*, 2026). The table should also carry the evidence for each cell—which artifact, which bucket, which benchmark produced the score—so a disputed cell can be re-run rather than argued (Davies *et al.*, 2025; Chen, 2026a). When two stacks score equally on the columns that matter, choose by operational factors outside the table: team skill, ecosystem integration, and maintenance burden (Chen, 2020; Davies *et al.*, 2025). A rating is a starting point for a routing decision, never the decision itself (Vellaisamy *et al.*, 2026; Chen, 2026a).

21.1.4 Baseline discipline

Every compiler rating needs an explicit baseline measured in the same harness, because a speedup without a baseline definition is uninterpretable (Davies *et al.*, 2025; Chen, 2026a). The baseline should be the stack a team would actually use without the compiler under test—usually eager PyTorch or a vendor library path—measured with the same warm-up, buckets, and counters (Vellaisamy *et al.*, 2026; Chen, 2020). Published speedups from each project should be re-derived against this common baseline rather than copied, because their original baselines differ (Wu *et al.*, 2024; Zheng *et al.*, 2020; Mirage Project contributors, 2025). The rating table should show absolute counters alongside speedup ratios so a 2x win on a bad baseline is visible for what it is (Chen, 2026a; Davies *et al.*, 2025). Baseline discipline is the unglamorous work that makes every other column of the comparison trustworthy (Chen, 2020; Vellaisamy *et al.*, 2026).

21.2 Compile overhead

The second axis is **compile overhead**, and it is the cost teams under-budget. The stacks pay in different places. TVM’s Ansor and AutoTVM pay a measured search—thousands of on-device trials per operator—reporting up to $3.8\times$ (Ansor,

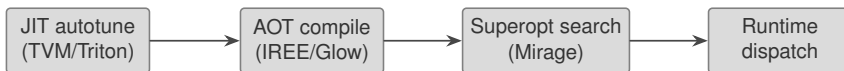


Figure 21.2: Where each stack pays its **compile** cost: JIT autotuning, ahead-of-time compilation, or superoptimization search.

Intel CPU) and $1.2\text{--}3.8\times$ (AutoTVM) in exchange (Zheng *et al.*, 2020; Chen *et al.*, 2018a). Triton pays a per-shape autotune warmup the first time a kernel key is seen, so the first request after a shape change is slow (OpenAI / triton-lang contributors, 2024). IREE and Glow pay their cost ahead of time, producing a deployable artifact with no runtime compile (IREE / iree-org contributors, 2024; Rotem *et al.*, 2018). Mirage pays the largest one-time cost—superoptimization search plus probabilistic verification—and reports $1.1\text{--}2.9\times$, with its MPK megakernel reaching $1.2\text{--}6.7\times$ (Wu *et al.*, 2024; Mirage Project contributors, 2025).

The shape of the overhead also differs in ways that matter operationally, not just in total cost. A JIT autotuner’s cost is incremental and amortizing: it is paid lazily, per new shape, and warm shapes are free—but it makes the first request after any shape change slow, which is visible to a user at exactly the wrong moment (OpenAI / triton-lang contributors, 2024; Kwon *et al.*, 2023). An AOT compile front-loads everything into the build, so runtime latency is predictable and there is no first-request penalty, at the cost of a longer build and a larger artifact that must be regenerated when the model or target changes (IREE / iree-org contributors, 2024; Rotem *et al.*, 2018). A superoptimization search is the most expensive but also the most one-time: once a megakernel is found and verified for a model-target pair, it serves every subsequent token with no further search (Wu *et al.*, 2024; Mirage Project contributors, 2025). A team that values predictable tail latency leans AOT or pre-searched; a team iterating rapidly on shapes tolerates JIT.

The engineering rule that falls out is to match the **overhead** model to the deployment. A served LLM whose decode layers are reused across billions of tokens justifies an expensive one-time search or AOT compile, because the cost amortizes; a rapidly changing or one-off graph does not, and there a JIT autotuner’s incremental cost or a lighter pass pipeline wins (Kwon *et al.*, 2023; Davies *et al.*, 2025). The **compile**-time/runtime-latency trade is itself a design decision, not an afterthought.

21.2.1 Budgeting compile cost in the release pipeline

Compile overhead must be budgeted in the release pipeline, not discovered at deployment time (Chen, 2020, 2026a). The budget has three components that fail

differently: wall-clock compile time per artifact, on-device trial time for autotuners, and verification or numerical-test time for searched kernels (Zheng *et al.*, 2020; Wu *et al.*, 2024; Davies *et al.*, 2025). Each component belongs in the release manifest so a model change that inflates compile time is visible before it delays a rollout (Chen, 2020; Vellaisamy *et al.*, 2026). The pipeline should also separate build-time compilation from serving-time fallback: any compilation that can happen in CI should happen there, leaving the runtime with a warm cache and no first-request penalty (OpenAI / triton-lang contributors, 2024; Kwon *et al.*, 2023). Teams that skip this budgeting treat compile time as an externality and discover it during the first production rollout (Chen, 2026a; Davies *et al.*, 2025).

21.2.2 First-token latency and the compile contract

The user-visible cost of compilation is the first-token latency it produces (Kwon *et al.*, 2023; Davies *et al.*, 2025). A JIT stack that autotunes on the first request converts a cold start into a user-visible stall; an AOT or pre-searched stack pays the same cost at build time and serves every token predictably (OpenAI / triton-lang contributors, 2024; IREE / iree-org contributors, 2024). The product contract should state where compile cost is allowed to appear: never in steady-state serving, and only in bounded, predictable places during warmup or model load (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). When a stack cannot meet that contract, the deployment must precompile or pre-warm the exact shapes the product will serve (Kwon *et al.*, 2023; Chen, 2020). Measuring first-token latency under a cold cache, warm cache, and pre-compiled artifact is therefore part of the compiler benchmark, not an optional operational nicety (Chen, 2026a; Davies *et al.*, 2025).

21.2.3 When expensive compilation pays

The amortization question determines whether a search-based or AOT compiler is worth its cost (Wu *et al.*, 2024; Kwon *et al.*, 2023). A decoder layer that runs once per token across many concurrent users accrues billions of executions, so even a multi-hour search costs a negligible fraction of a nanosecond per execution (Vellaisamy *et al.*, 2026; Wu *et al.*, 2024). A preprocessing graph that runs once per model version does not justify the same expense, and a lighter compiler or a direct kernel library is the rational choice (Chen, 2020, 2026a). The break-even calculation should include artifact storage, verification, and regeneration frequency, because a model that changes weekly forces the search cost weekly (Wu *et al.*, 2024; Davies *et al.*, 2025). Publishing the break-even arithmetic with the benchmark makes the compile-cost axis a decision rather than a preference (Vellaisamy *et al.*,

2026; Chen, 2026a).

21.2.4 Measuring compile overhead in CI

Compile overhead should be measured in the same CI that builds the artifacts, with a separate report that tracks wall-clock time, trial time, artifact size, and verification time per configuration (Chen, 2020, 2026a). The report makes regressions visible: a compiler upgrade that doubles search time or an input change that inflates artifact size shows up before it delays a release (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). The measurement should run on representative hardware because on-device autotuning time does not correlate with build-host CPU time (Zheng *et al.*, 2020; OpenAI / triton-lang contributors, 2024). Budgets attached to the report can fail the build when compile cost exceeds the product’s deployment tolerance (Chen, 2026a; Davies *et al.*, 2025). This turns the compile-overhead axis from an anecdote into a gated metric that the release process enforces (Chen, 2020; Vellaisamy *et al.*, 2026).

21.2.5 Warm caches and build reuse

Compile overhead accounting must distinguish cold builds from warm builds, because caches change the real cost dramatically (Chen, 2020; OpenAI / triton-lang contributors, 2024). A CI job with a warm autotune cache and reusable search results measures the steady-state cost of a model change, while a cold cache measures the worst-case onboarding cost (Zheng *et al.*, 2020; Wu *et al.*, 2024). Both numbers belong in the report, since a team iterating on a model sees the warm number and a new team onboarding sees the cold one (Davies *et al.*, 2025; Chen, 2026a). Build reuse policies—which artifacts can be cached across model versions and which must regenerate—should be explicit and versioned (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026). The distinction keeps compile-cost budgets honest across the life of a project (Chen, 2026a, 2020).

21.3 Hardware binding

The third axis is how each stack performs **hardware binding**—how late and how flexibly it commits to a target. TVM binds through target strings and forks schedules per backend; Triton binds to a GPU through its TritonGPU dialect and PTX/AMDGCN codegen and does not target static NPU; IREE binds latest, through a Vulkan-inspired HAL that abstracts CPU, GPU, and more behind one

interface; Glow binds through LLVM and cross-compiles to edge cores; Mirage binds to the GPU compute hierarchy by construction (Chen *et al.*, 2018b; OpenAI / triton-lang contributors, 2024; IREE / iree-org contributors, 2024; Rotem *et al.*, 2018; Wu *et al.*, 2024).

Binding time is really a question of where the abstraction boundary sits between the program and the silicon. TVM’s target strings bind at schedule-selection time, so the same compute is re-scheduled per backend with a fork in the pass pipeline (Chen *et al.*, 2018b). IREE’s HAL pushes the boundary as late as possible—the dispatch regions are formed once and only the executable translation is per-target—so one program description fans out to many devices (IREE / iree-org contributors, 2024). Glow binds through LLVM, inheriting LLVM’s target support for free and gaining cross-compilation to embedded cores as a side effect (Rotem *et al.*, 2018). Triton and Mirage bind earliest and narrowest, to a GPU machine model, which is why their reasoning about shared memory and launches is the most precise and least portable (Tillet *et al.*, 2019; Wu *et al.*, 2024).

The decode consequence of late versus early **binding** is concrete. A late-binding HAL like IREE’s makes a model portable across a wide matrix but hides per-target residency decisions that only batch-1 telemetry exposes, so a green build is not a green decode (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026). An early, narrow binding like Triton’s or Mirage’s gives up breadth but lets the compiler reason precisely about one machine’s shared memory and launch model, which is why the deepest decode wins are GPU-specific (Chen, 2026a; SemiAnalysis, 2024). Across all of them the hardware truth is unchanged: GPUs are bound by shared-memory residency, CPUs by cache, NPUs by static tile/DMA plans, so no single binding is optimal everywhere (NVIDIA, 2022; AMD, 2024b,a).

21.3.1 Binding scenarios

Hardware binding becomes concrete through three scenarios. A datacenter GPU workload binds early and deeply when the team wants shared-memory-aware scheduling, choosing Triton or Mirage and accepting that the artifact is GPU-specific (OpenAI / triton-lang contributors, 2024; Wu *et al.*, 2024). A heterogeneous fleet binds late through an abstraction like IREE’s HAL so one program covers GPU, CPU, and accelerator targets, accepting that per-target residency must be validated separately (IREE / iree-org contributors, 2024; AMD, 2024a). An edge product binds through an AOT path like Glow’s LLVM backend so the same source cross-compiler to the exact embedded core, accepting a fixed schedule per target (Rotem *et al.*, 2018; Goodfellow *et al.*, 2016). Each scenario is a different answer to the same question: how much portability does the product need, and how much

depth is it willing to trade for it (Davies *et al.*, 2025; Chen, 2026a). The scenario frame prevents binding discussions from becoming abstract ideology (Vellaisamy *et al.*, 2026; Chen, 2020).

21.3.2 Validation burden of binding

The later a stack binds, the more per-target validation the deployment owes (IREE / iree-org contributors, 2024; AMD, 2024a). A single portable artifact that lowers to many devices still needs residency and byte validation on each device, because the compiler cannot prove decode legality across targets from one build (Chen, 2026a; Davies *et al.*, 2025). An early-bound compiler narrows the validation surface but concentrates it: one GPU artifact must be validated for every SKU generation the fleet runs (NVIDIA, 2022; Wu *et al.*, 2024). The validation matrix should therefore be derived from the binding choice: late binding produces a wide but shallow matrix that grows with targets, while early binding produces a deep but narrow matrix that grows with SKUs (Chen, 2020; Vellaisamy *et al.*, 2026). Compile reports, byte ledgers, and numerical checks are the artifacts that populate either matrix (Chen, 2026a; Davies *et al.*, 2025). Teams that bind late without building the per-target validation matrix have only delayed the work, not avoided it (IREE / iree-org contributors, 2024; Chen, 2026a).

21.3.3 Per-target residency proof

Whatever the binding time, every deployment cell needs a per-target residency proof: the artifact must show that its fused regions fit the target’s on-chip budgets and that the measured bytes/token matches the schedule’s intent (Chen, 2026a; Davies *et al.*, 2025; AMD, 2024a). The proof has three parts: a compile-time report of buffers and capacities, a runtime measurement of bytes and launches, and a numerical check against the reference (Chen, 2020, 2026a). Early-bound compilers can generate most of the proof from their machine model; late-bound compilers must generate it per target at validation time (Wu *et al.*, 2024; IREE / iree-org contributors, 2024). A cell without all three parts is not a supported deployment, it is an experiment (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). The binding debate therefore resolves into a simple accounting question: who owns the per-target proof and when is it generated (Chen, 2026a, 2020).

21.3.4 Device-class cells

The hardware binding comparison is cleanest when expressed in device-class cells rather than vague categories (Davies *et al.*, 2025; Chen, 2026a). A datacenter GPU cell specifies the GPU generation, the context buckets, and the batch-1 shape; an edge CPU cell specifies the core, the SRAM budget, and the power envelope (NVIDIA, 2022; Goodfellow *et al.*, 2016; AMD, 2024a). Each stack earns a rating per cell, and the rating may change between cells of the same class because shared-memory capacity and cache size differ (Chen, 2020, 2026a). The cell definition forces the comparison to state the machine model each compiler reasoned against, which is the real subject of the binding axis (Wu *et al.*, 2024; IREE / iree-org contributors, 2024). Device-class cells also make the validation matrix concrete: each cell lists the compilers that can serve it and the artifacts that prove it (Vellaisamy *et al.*, 2026; Chen, 2026a).

21.4 Search space

The fourth axis is the **search** space each stack explores and how it is navigated, because that determines what optimizations are even reachable. AutoTVM searches hand-written schedule templates with a learned cost model; Ansor removed the templates and samples a hierarchical space, **autotuning** with evolutionary search and a task scheduler that spends budget where it moves end-to-end latency (Chen *et al.*, 2018a; Zheng *et al.*, 2020). Triton’s autotuner explores only the configuration list the author supplies—leaner, but bounded by human imagination (OpenAI / triton-lang contributors, 2024). Mirage searches the largest space of all: a multi-level μ Graph that can invent new custom kernels, pruned by abstraction and certified by probabilistic equivalence (Wu *et al.*, 2024).

There is a spectrum here from human-specified to machine-discovered optimization, and it correlates with the verification burden. At the human-specified end, Triton’s config list and AutoTVM’s templates are trustworthy by construction—the schedules are variations a human vetted—but they cannot exceed human imagination (OpenAI / triton-lang contributors, 2024; Chen *et al.*, 2018a). At the machine-discovered end, Mirage’s superoptimizer can synthesize a kernel no one wrote, which is powerful precisely because it is not bounded by templates, but that power forces the question of correctness: a discovered kernel must be proven equivalent, which is why probabilistic equivalence verification is not optional for it (Wu *et al.*, 2024). Ansor sits in between, sampling a structured space without explicit templates but within a fixed schedule grammar (Zheng *et al.*, 2020).

The reachability difference is the point. A template-bounded **search** can only find schedules a human anticipated, so it cannot discover a megakernel; a superoptimizer’s search can, which is why Mirage and MPK reach fusions the others structurally cannot (Wu *et al.*, 2024; Mirage Project contributors, 2025). The cost, as the overhead axis noted, is search time—so the right **autotuning** strategy depends on how reused the program is and how much of the optimization you are willing to let the compiler discover rather than specify (Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

21.4.1 Choosing a search strategy

The search axis is a strategy selection problem: how much of the optimization space to explore, who defines it, and how to trust the result (Wu *et al.*, 2024; Zheng *et al.*, 2020; Chen *et al.*, 2018a). When the team knows the schedule grammar well and wants fast, predictable compilation, a template or config-list search is right (Chen *et al.*, 2018a; OpenAI / triton-lang contributors, 2024). When the team needs reach beyond human templates and can afford compile time, a hierarchical sampler such as Ansor expands the space without leaving the schedule grammar (Zheng *et al.*, 2020). When the goal is discovering kernels no grammar encodes, a superoptimizer with verification is the only option, and its cost and confidence requirements must be accepted together (Wu *et al.*, 2024; Mirage Project contributors, 2025). The choice should be recorded with the deployment because it determines both the reachable performance and the maintenance burden of every future model (Chen, 2020; Davies *et al.*, 2025).

21.4.2 Reachability and verification correlation

The spectrum from human-specified to machine-discovered optimization carries a verification correlation that teams underweight (Wu *et al.*, 2024; Davies *et al.*, 2025). Schedules a human wrote can be trusted by review plus testing; schedules a search discovered need an independent correctness argument, which is why Mirage’s probabilistic verifier is part of its design rather than an add-on (Wu *et al.*, 2024). Ansor’s sampled schedules sit in a middle zone: they stay inside a grammar whose semantics are understood, so the verification burden is lighter than for a fully invented kernel (Zheng *et al.*, 2020; Chen, 2020). The deployment should match verification strength to discovery distance: the further the optimizer roams from human-vetted structure, the stronger the proof required before production (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026). Teams that adopt a discover-it-all optimizer while applying verify-by-testing discipline are taking the risk without the safety

mechanism (Wu *et al.*, 2024; Chen, 2026a).

21.4.3 Search budget under model churn

Model churn is the adversary of expensive search, so the budget must be planned against the model-release cadence (Wu *et al.*, 2024; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). A team that ships a new model weekly cannot spend a week of superoptimization per release unless the hot regions are stable enough to cache across models (Wu *et al.*, 2024; Davies *et al.*, 2025). Search caches keyed on verified subgraph structure can amortize across releases, but cache hits must still pass verification against the new reference (Wu *et al.*, 2024; Chen, 2026a). The practical policy is a graduated budget: cheap search for every model, moderate search for hot regions, and deep search only for the flagship model whose gains will be served longest (Chen, 2020; Davies *et al.*, 2025). Publishing the budget policy with the benchmark makes compile cost a predictable release metric rather than a surprise (Vellaisamy *et al.*, 2026; Chen, 2026a). Model churn and search depth are therefore managed together, not separately (Wu *et al.*, 2024; Kwon *et al.*, 2023).

21.4.4 Cost metric conversions

Comparing search strategies requires converting their costs into the same units the deployment uses (Wu *et al.*, 2024; Chen *et al.*, 2018a; Davies *et al.*, 2025). Wall-clock compile time, on-device trial minutes, verification time, and artifact bytes all convert into release latency and infrastructure cost (Vellaisamy *et al.*, 2026; Chen, 2026a). The conversion should be part of the search axis so a stack whose search is cheap in trials but expensive in human tuning effort is scored fairly (Chen *et al.*, 2018a; Wu *et al.*, 2024). A template search may require the team to write the templates, which is human cost invisible to a wall-clock timer; a superoptimizer converts that effort into machine time, which is visible but automatable (Chen *et al.*, 2018a; Wu *et al.*, 2024). Publishing the cost conversions with the benchmark lets a team pick the axis that matches its resources: engineers who can write templates, machines that can search, or products that can amortize either (Davies *et al.*, 2025; Chen, 2026a).

21.5 YiRage benchmark

This is where the book’s yardstick is defined. **YiRage** is the Mirage-lineage compiler used throughout the book to **benchmark** the others, and its discipline

is to measure every stack on the same two decode quantities—bytes/token and launches/token at batch-1—rather than each project’s preferred microbenchmark (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). It generalizes Mirage’s searched-and-verified megakernel across multiple backends (CUDA, CPU, Triton, and more) and carries the Chapter 2 residency rules as checkable IR metadata, so a candidate schedule that would spill is rejected before it ships (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024).

The reason a common yardstick is necessary rather than pedantic is that the alternative—trusting each project’s published number—systematically misleads. Ansor’s $3.8\times$, AutoTVM’s $1.2\text{--}3.8\times$, Mirage’s $1.1\text{--}2.9\times$, and MPK’s $1.2\text{--}6.7\times$ are all real, but each is measured against a different baseline, on a different workload, often at a throughput-shaped operating point rather than batch-1 decode (Zheng *et al.*, 2020; Chen *et al.*, 2018a; Wu *et al.*, 2024; Mirage Project contributors, 2025). Reduced to one harness that fixes the model, the decode shape, and the metric, the numbers become comparable and the niches become visible: the megakernel path wins where launches dominate, the AOT bundle wins where the device is small, and the broad scheduler wins where portability is the constraint (Vellaisamy *et al.*, 2026; Chen, 2026a).

Run as a **benchmark** harness, YiRage reframes the comparison as a routing problem: on a datacenter GPU it favors a searched megakernel path of the kind MPK demonstrates ($1.2\text{--}6.7\times$); on an edge CPU it favors a Glow-style AOT bundle; on a broad portability target it favors IREE-style whole-program scheduling (Mirage Project contributors, 2025; Rotem *et al.*, 2018; IREE / iree-org contributors, 2024). The **cross-hardware** reading is that no single compiler dominates—each is the right answer for a region of the hardware matrix—and the value of a unified yardstick is that it makes the routing decision measurable instead of tribal (Davies *et al.*, 2025; Chen, 2026a). Concretely, the routing inputs are the target class and the reuse profile of the workload: a high-reuse decode layer on a datacenter GPU routes to a searched megakernel because the one-time search amortizes and the launch collapse is largest there, while the same layer on a memory-constrained edge part routes to an AOT bundle because the binding fits the device and there is no launch loop to begin with (Mirage Project contributors, 2025; Rotem *et al.*, 2018). Encoding that decision as metadata rather than tribal preference is what lets the choice survive a model or hardware change without re-litigating it (Chen and YiRage Contributors, 2026).

21.5.1 Harness definition and shared artifacts

The unified harness needs a precise definition before any compiler runs: the reference model and checkpoint, the exact tokenizer, the batch-1 decode shape, the context buckets, the warm-up protocol, and the metric formulas for bytes/token and launches/token (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a). Every stack receives the same reference program and is asked to produce its best artifact under a documented budget, then all artifacts run through the same measurement code (Chen, 2020; Davies *et al.*, 2025). The artifacts themselves should be archived: compiled binaries, configuration files, compile reports, and verification outputs (Chen, 2026a; Vellaisamy *et al.*, 2026). A benchmark whose artifacts cannot be replayed is a narrative, not a measurement (Davies *et al.*, 2025; Chen, 2026a). The harness definition belongs in version control beside the compilers under test, so an upgrade to any component triggers a clean re-run rather than a silent comparison against stale numbers (Chen, 2020; Vellaisamy *et al.*, 2026).

21.5.2 Benchmark lifecycle and routing updates

The unified benchmark is not a one-time study; it is a living artifact that must be re-run when any input changes (Chen, 2026a; Davies *et al.*, 2025). Model releases, compiler upgrades, driver updates, and new context buckets can each change the routing decision that the last benchmark produced (Vellaisamy *et al.*, 2026; Chen, 2020). The routing table should carry a timestamp and the benchmark commit that produced it, and a re-run should be scheduled on every relevant dependency change (Davies *et al.*, 2025; Chen, 2026a). When a re-run changes a routing cell, the deployment should treat it as a configuration change with the same review as a flag change (Chen, 2020; Vellaisamy *et al.*, 2026). This lifecycle turns compiler selection from a one-time architecture decision into a maintained routing contract (Davies *et al.*, 2025; Chen, 2026a).

21.5.3 Limits of the unified comparison

A common harness makes compilers comparable, but it does not make them equivalent (Davies *et al.*, 2025; Chen, 2026a). Each stack still carries ecosystem constraints—framework integration depth, team skill, library maturity, and long-term maintenance—that no bytes/token measurement captures (Chen, 2020; Vellaisamy *et al.*, 2026). The unified numbers answer the narrow question of decode efficiency; the routing decision answers the broader question of production fit (Davies *et al.*, 2025; Chen, 2026a). The benchmark should therefore publish

both the measured rows and the caveats, so a reader can apply the table without over-reading it (Chen, 2020; Vellaisamy *et al.*, 2026). Recognizing these limits is what keeps the yardstick honest rather than turning it into a religion (Davies *et al.*, 2025; Chen, 2026a).

21.5.4 Report schema

Every benchmark row should carry the same schema so rows are comparable across time: model, checkpoint, compiler and version, target and SKU, context bucket, the three counters, compile time, verification status, and the artifact hash (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a). The schema makes diffs between runs mechanical: a regression is a row whose counters changed under an unchanged schema (Chen, 2020; Davies *et al.*, 2025). It also makes the routing table auditable, because every routing decision can point at the row that justified it (Chen, 2026a; Vellaisamy *et al.*, 2026). Publishing the schema as part of the harness forces each compiler project’s measurement conventions into one comparable form instead of relying on their favored units (Davies *et al.*, 2025; Chen, 2026a). A stable report schema is the smallest unit of benchmark governance that actually works (Chen, 2020; Vellaisamy *et al.*, 2026).

21.5.5 Routing table maintenance

The routing table is a map from workload and target cells to compiler choices, and maintaining it is an engineering task, not a one-time survey (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a). Each cell should name the chosen stack, the benchmark row that justified it, and the rollback cell if the choice fails in production (Chen, 2020; Vellaisamy *et al.*, 2026). When a workload moves between cells—say a model graduates from edge to datacenter serving—the routing change must be reviewed as a configuration change rather than applied silently (Davies *et al.*, 2025; Chen, 2026a). The table should be generated from the benchmark repository so it cannot drift from the evidence (Chen, 2020, 2026a). A living routing table is the operational form of the book’s claim that compiler choice is a routing decision (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

21.5.6 Worked routing scenario

Consider one Llama-class model served on three cell types. On an H100 cell with steady traffic, the benchmark favors a searched megakernel because launch collapse

dominates and the search amortizes over high reuse (Wu *et al.*, 2024; Mirage Project contributors, 2025; Vellaisamy *et al.*, 2026). On a mid-tier GPU with mixed batch sizes, an IREE whole-program artifact may win because portability across the cell’s shapes matters and dispatch collapse suffices (IREE / iree-org contributors, 2024; Chen, 2026a). On a power-constrained edge box, a Glow-style AOT bundle wins because the device has no launch loop and memory is the binding constraint (Rotem *et al.*, 2018; AMD, 2024a). Each cell uses the same reference model and the same counters, so the three choices are comparable even though the mechanisms differ (Davies *et al.*, 2025; Chen, 2026a). The scenario demonstrates why no single compiler dominates: the routing table is the actual deliverable, and the benchmark is its engine (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

21.5.7 Re-run policy for the yardstick

The unified benchmark needs an explicit re-run policy so its conclusions do not silently age (Davies *et al.*, 2025; Chen, 2026a). Re-runs should be scheduled when the model changes, when a compiler under test releases a new version, when a driver or runtime is upgraded, and when the fleet adds a new device class (Vellaisamy *et al.*, 2026; Chen, 2020). Each re-run publishes the full report schema, so a routing change is always traceable to a row diff (Davies *et al.*, 2025; Chen, 2026a). The policy should also define who owns the benchmark, because an unowned benchmark decays into folklore within two quarters (Chen, 2020; Vellaisamy *et al.*, 2026). A yardstick with a re-run policy is the difference between compiler routing as a maintained system and compiler routing as a stale slide (Davies *et al.*, 2025; Chen, 2026a).

21.5.8 Documenting the machine model

The binding axis is only comparable when each compiler’s machine model is documented alongside its rating (Wu *et al.*, 2024; OpenAI / triton-lang contributors, 2024; IREE / iree-org contributors, 2024). The machine model states which hardware properties the compiler reasons about: shared memory, SM count, cache hierarchy, DMA edges, or tile SRAM (SemiAnalysis, 2024; AMD, 2024a). A compiler that omits a property from its model cannot optimize for it, and the rating should note that omission rather than penalize it blindly (Wu *et al.*, 2024; Chen, 2026a). Documenting the machine model turns binding from an abstraction debate into a concrete capability list: this stack can reason about these resources and therefore can optimize this class of targets (OpenAI / triton-lang contributors,

2024; AMD, 2024b). The documentation belongs in the benchmark repository with the device-class cells, so the capability claims are checkable (Davies *et al.*, 2025; Chen, 2026a).

21.6 Key Takeaways

- **Rate compilers on launch collapse, not peak FLOPs.** At batch-1, launches/token and bytes/token predict ms/token; a stack can win an isolated kernel benchmark and lose end-to-end (Vellaisamy *et al.*, 2026; Chen, 2026a). The launch-collapse column is the one that matters (NVIDIA, 2024).
- **Match compile-overhead model to deployment.** TVM/Triton pay JIT search; IREE/Glow pay AOT; Mirage pays superoptimization search (Zheng *et al.*, 2020; IREE / iree-org contributors, 2024; Wu *et al.*, 2024). Expensive one-time compilation amortizes over a reused decode layer but not a one-off graph (Kwon *et al.*, 2023).
- **Binding time trades breadth for depth.** Late-binding HALs (IREE) maximize portability but hide per-target residency; early, narrow bindings (Triton, Mirage) give up breadth for precise GPU reasoning (IREE / iree-org contributors, 2024; OpenAI / triton-lang contributors, 2024). A green build is not a green decode (Vellaisamy *et al.*, 2026).
- **Search reachability sets the ceiling.** Template search finds only anticipated schedules; a μ Graph superoptimizer can invent megakernels (Chen *et al.*, 2018a; Wu *et al.*, 2024). What the search space contains bounds what the compiler can ever find (Zheng *et al.*, 2020).
- **No compiler dominates; routing does.** Each stack owns a region of the hardware matrix, so the win is a measured routing decision on common decode metrics—the YiRage discipline (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Megakernel on GPU, AOT bundle on edge, whole-program scheduling for breadth, and the routing itself recorded as metadata so it survives the next model or accelerator (Mirage Project contributors, 2025; Rotem *et al.*, 2018).

Practical decision rules. Five rules compress the comparison chapter for a design review. First, define the benchmark configuration before choosing the compiler, because a comparison without pinned variables measures nothing (Davies *et al.*, 2025; Chen, 2026a). Second, weight the rating columns by your deployment

constraints rather than by an average, because a leaderboard hides the columns that matter (Vellaisamy *et al.*, 2026; Chen, 2020). Third, account for compile cost in the release pipeline and match it to the model’s reuse profile (Wu *et al.*, 2024; Kwon *et al.*, 2023). Fourth, build the per-target residency proof for every cell, regardless of the binding time of the chosen stack (AMD, 2024a; Chen, 2026a). Fifth, maintain the routing table as a living artifact that changes with the benchmark, not as a one-time architecture choice (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Teams that follow these rules make compiler selection measurable; teams that skip them repeat the religious debates this chapter exists to end (Chen, 2026a; Vellaisamy *et al.*, 2026).

21.7 Conclusion

Compared on one honest axis—how few, how resident the kernel launches per token—the surveyed compilers stop looking like competitors and start looking like specialists: TVM and IREE for breadth, Triton for GPU kernel authoring, Glow for the edge, and Mirage for searched-and-verified megakernels (Chen *et al.*, 2018b; IREE / iree-org contributors, 2024; Tillet *et al.*, 2019; Rotem *et al.*, 2018; Wu *et al.*, 2024). The unifying contribution of a benchmark like YiRage’s is to make that specialization measurable, so a serving team routes a workload to the stack whose niche it falls in rather than arguing microbenchmarks (Chen and YiRage Contributors, 2026; Chen, 2026a). The practical payoff is that compiler selection stops being a one-time religious choice and becomes a per-workload, per-target decision backed by the same two numbers the whole book has tracked, which is the only form of comparison that survives the next model release or the next accelerator (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). With the compilers now rated on common ground, Chapter 22 turns from comparing stacks to driving one end to end: a compiler-driven performance-tuning workflow that takes a model from profile to shipped decode kernel (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Chapter 22

Compiler-Driven End-to-End Performance Tuning

A decode compiler team can win every microbenchmark and still lose the fleet when nobody owns the *end-to-end* tuning loop—profile, locate, change, gate, repeat—under the bytes/token and launches/token counters Chapter 1 made non-negotiable (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a). Part VI gave you backends; Chapter 21 gave you benchmark harnesses; this chapter wires them into a **compiler-driven E2E workflow** that treats each compiler knob as a hypothesis until decode telemetry confirms it (Chen, 2020; Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). Memory-Floor CUDA Graphs on Qwen-2.5-7B (BS=1, ctx=2048) yields $1.259\times$ on H100 by removing host overhead—yet cannot fix illegal HBM traffic when fusion boundaries still export activations (Chen, 2026a; NVIDIA, 2024). E2E tuning exists to catch that gap: the moment a compiler change improves a GEMM kernel but raises bytes/token, the workflow rejects it before merge (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

The loop has four stations—hardware-aware profiling, bottleneck locating, structured tuning workflow, regression gates—each bound to multi-hardware residency rules from Chapter 2 (Chen, 2020; AMD, 2024b,a; SemiAnalysis, 2024). Fregly’s profile-first culture applies throughout: Nsight and roofline charts justify knob changes more than intuition about tile sizes (Davies *et al.*, 2025; SemiAnalysis, 2024; Chen, 2026a). Chapter 25 extends this loop with automated search; Chapter 26 packages the artifacts for production (Chen and YiRage Contributors,

2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024).

Industrial teams document the loop as a runbook, not tribal knowledge. New hires should reproduce a baseline profile on day one, classify a bottleneck on day two, and propose a single-knob experiment by day three—all against the same decode cell (Davies *et al.*, 2025; Chen, 2020; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). When the runbook is missing, compiler teams revert to GEMM microbench heroics (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; AMD, 2024a; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Ownership matters: profiling belongs to whoever changes IR; gates belong to release engineering; classification belongs to a cross-functional review with serving SREs present (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Compiler-only tuning loops ship illegal schedules that frameworks mask until load rises (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Anti-patterns to post on the wall: (1) tune tiles while launches/token exceed kernel time; (2) merge compiler wins without canary; (3) profile prefill, ship decode; (4) skip IR diff review; (5) treat framework batching wins as compiler fusion wins (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). The sections below replace each anti-pattern with a concrete practice (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

E2E tuning also bounds *when to stop*: if bytes/token and launches/token meet SLO after graph replay, MegaKernel investment waits—mode selection from Chapter 12 applies before endless fusion (NVIDIA, 2024; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016;



Figure 22.1: Hardware-aware profiling pipeline: frozen decode cell → residency counters → roofline/timeline → bytes/token and launches/token report.

Counter	GPU tool surface	CPU / NPU delta
bytes/token	Nsight memory chart, CUPTI	perf/LBR, DMA byte counts
launches/token	CUDA API trace, CUPTI	thread launches, sync points
ms/token	end-to-end decode cell	same cell, power cap noted
legality	graph capture, shared mem	static schedule, tile slots

Table 22.1: Hardware-aware profiling maps the same decode questions to different toolchains.

SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Good enough under measured decode goodput beats perfect under imaginary FLOPs (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

22.1 HW aware profiling

Hardware-aware profiling fails when teams profile prefill shapes, batch-32 training graphs, or synthetic GEMMs while production runs BS=1 decode with paged KV (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). The fix is a *frozen decode cell*: pinned model revision, context bucket, sampling flags, warmup protocol, and SKU—logged beside every trace so comparisons are apples-to-apples (Chen, 2026a, 2020; Chen and YiRage Contributors, 2026).

GPU profiling binds to shared-memory occupancy, TMA transactions, and global bytes moved per decode step—not SM utilization alone (NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025). CPU profiling binds to LLC misses, NUMA cross-socket traffic, and OpenMP gang efficiency (Goodfellow *et al.*, 2016; Chen, 2020). NPU/XDNA profiling counts DMA edges and static tile residency; many GPU counters simply do not exist—substitute compile-time buffer accounting plus on-device cycle estimates (AMD, 2024b,a; Chen and YiRage Contributors, 2026).

Translate profiler output into Chapter 1 vocabulary before opening a compiler ticket: if bytes/token is within 10% of analytic HBM roofline (R_{floor}), orchestration tweaks (graphs, fusion) may still help; if bytes dominate, tile changes without fusion are theater (Chen, 2026a; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026). YiRage and similar stacks export IR-level residency metadata so profiles can be tied to specific fusion boundaries rather than mystery kernels (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Kwon *et al.*, 2023).

Warmup discipline separates steady-state decode from cold-start stories: graph capture, allocator priming, and CUDA context creation belong in the profile protocol document—not in ad-hoc Slack threads (NVIDIA, 2024; Chen, 2026a; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2022). Teams that compare profiles without matched warmup compare different phases and waste compiler cycles (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; AMD, 2024a; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Tooling hygiene: store raw ‘.nsys-rep’, CUPTI CSV, or NPU trace bundles beside summarized bytes/token tables in artifact storage keyed by git SHA (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Summaries alone rot; raw traces enable re-analysis when classification rules improve (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Prefill vs decode profiling split: compiler teams often inherit prefill-heavy traces from training infra (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022). Mandate decode-cell scripts in CI and reject profile uploads that omit BS=1 metadata (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Hardware-aware profiling is a contract, not a one-off Nsight session (Davies *et al.*, 2025; Chen and

YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Multi-hardware delta. Hopper profiles expose TMA and warp-specialization; XDNA profiles expose tile SRAM fill; CPU profiles expose cache hierarchy—identical decode step, incomparable dashboards unless normalized to bytes/token (SemiAnalysis, 2024; AMD, 2024b; Goodfellow *et al.*, 2016).

Example 22.1.1. Decode cell (BS=1, ctx=2048): a profile showing low SM occupancy but high HBM throughput is bandwidth-bound—compiler tuning should target fusion and KV residency, not bigger GEMM tiles (Chen, 2026a; Davies *et al.*, 2025; Dao *et al.*, 2022).

22.1.1 Defining the decode cell protocol

The decode cell protocol is the first artifact of the profiling station, and it must be precise enough that two engineers on two continents reproduce the same trace (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026). The protocol records the model revision, tokenizer, weight format, context buckets, sampling flags, page-table layout, warm-up steps, and the exact measurement window (Kwon *et al.*, 2023; Chen, 2020). It also records what is excluded: cold-start costs, allocator priming, driver compilation, and graph capture belong in separate phases with separate labels (NVIDIA, 2024; Chen, 2026a). A cell that omits any of these fields produces numbers that look comparable and are not (Davies *et al.*, 2025; Chen, 2020). The protocol should be checked into the repository with a schema validator so every uploaded profile either conforms or is rejected (Chen, 2026a; Vellaisamy *et al.*, 2026).

22.1.2 Profiler overhead discipline

Profiling changes what it measures: capture buffers, symbolization, and per-kernel callbacks add overhead that shifts timings and can alter scheduling (Davies *et al.*, 2025; SemiAnalysis, 2024; Chen, 2026a). The protocol should therefore measure each workload twice—once with lightweight counters and once with full tracing—and report both so the overhead is visible (Chen, 2020; Vellaisamy *et al.*, 2026). Lightweight counters establish the steady-state baseline; full traces explain the baseline’s composition (Chen, 2026a; Davies *et al.*, 2025). When profiling overhead exceeds an agreed fraction of total runtime, the profile is a different

workload and should be labeled as such (Vellaisamy *et al.*, 2026; Chen, 2020). Teams that skip this discipline optimize artifacts they never actually run (Chen, 2026a; Davies *et al.*, 2025).

22.1.3 Raw trace storage and re-analysis

Summarized tables answer today’s question, but raw traces answer next quarter’s question, which is why both must be stored (Davies *et al.*, 2025; Chen, 2020, 2026a). Each raw trace bundle should carry the git SHA, the decode-cell protocol version, the tool version, and the machine fingerprint (Vellaisamy *et al.*, 2026; Chen, 2026a). Storage cost is real, so the policy should keep full traces for the shipped cell matrix and retain summaries plus derived counters for exploratory runs (Chen, 2020; Davies *et al.*, 2025). When classification rules improve, re-analysis of old traces can explain historical regressions that summaries cannot (Chen, 2026a; Davies *et al.*, 2025). Raw-trace retention is the difference between a profiling culture and a profiling archive (Vellaisamy *et al.*, 2026; Chen, 2026a).

22.1.4 Normalizing counters across toolchains

GPU profilers, CPU perf counters, and NPU traces speak different languages, so the report layer must normalize them to the same three counters before comparison (Davies *et al.*, 2025; AMD, 2024b; Goodfellow *et al.*, 2016). Bytes moved per token and launches or dispatches per token are the portable vocabulary; raw tool metrics such as SM occupancy or DMA edge count supplement rather than replace them (Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016). The normalization rules belong in the protocol, including how DMA edge counts map to the launch concept and how cache hits count as bytes (Chen, 2020; Vellaisamy *et al.*, 2026). A normalized report lets a multi-hardware team compare a GPU regression to an NPU regression without translating profiler dialects by hand (Davies *et al.*, 2025; Chen, 2026a). Without normalization, every cross-target discussion starts from incompatible dashboards (AMD, 2024b; Goodfellow *et al.*, 2016).

22.1.5 Runbook onboarding exercises

The tuning runbook becomes real when new engineers complete three exercises against it (Chen, 2020, 2026a; Vellaisamy *et al.*, 2026). Day one is reproducing the baseline profile from the frozen decode cell and confirming the counters match the stored report (Davies *et al.*, 2025; Chen, 2026a). Day two is classifying a synthetic bottleneck injected into the cell—for example an extra unfused activation—and

writing the evidence-based label (Chen, 2020; Davies *et al.*, 2025). Day three is filling a hypothesis card, running the experiment, and publishing the verdict (Chen, 2026a; Vellaisamy *et al.*, 2026). Each exercise checks that the workflow artifacts exist and that the process is teachable rather than tribal (Chen, 2020; Davies *et al.*, 2025). A runbook that cannot onboard an engineer in three days is documentation, not a workflow (Chen, 2026a; Vellaisamy *et al.*, 2026).

22.1.6 Environmental drift capture

Profiles change when the environment changes, even when the model does not (Chen, 2020, 2026a; Davies *et al.*, 2025). Driver versions, CUDA libraries, allocator behavior, machine co-tenancy, and thermals all shift counters, so every report should record the environment fingerprint and flag out-of-band runs (Vellaisamy *et al.*, 2026; Chen, 2026a). The fingerprint includes the driver, CUDA, library versions, host model, and the machine’s measured baseline on a calibration kernel (Chen, 2020; Davies *et al.*, 2025). When a report’s environment differs from the reference cell, the comparison is invalid and should be labeled as such rather than silently combined (Chen, 2026a; Vellaisamy *et al.*, 2026). Environmental drift capture is what keeps a three-month-old baseline meaningful (Davies *et al.*, 2025; Chen, 2026a).

22.1.7 Cold-start and steady-state labeling

Decode products experience both cold-start and steady-state phases, and the two must never share a counter row (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Cold start includes model load, graph capture, allocator priming, and first-token compile costs; steady state measures the repeated per-token path after warm-up (NVIDIA, 2024; Chen, 2026a). The protocol should label every row by phase so an optimization that improves steady state but worsens cold start is visible as a trade rather than a regression or a win (Chen, 2020; Davies *et al.*, 2025). Products whose SLO binds cold start need a different tuning target than products that bind steady state (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Phase labeling is the smallest change that prevents most profile-comparison mistakes (Chen, 2026a; Davies *et al.*, 2025).

22.1.8 Profiling skills for compiler engineers

Compiler engineers need profiling literacy because their changes are validated in profiles, not in unit tests (Chen, 2020; Davies *et al.*, 2025; Chen, 2026a). The



Figure 22.2: Bottleneck locating: profile facts → launch vs memory vs residency class → compiler knob → prioritized experiment.

team should run a shared training curriculum covering the decode cell, the profiler toolchain per target, the counter normalization rules, and the classification tree (Vellaisamy *et al.*, 2026; Chen, 2026a). Every engineer should be able to produce a labeled profile and defend the label with evidence before changing a compiler pass (Chen, 2020; Davies *et al.*, 2025). Cross-training between compiler and serving teams matters because the boundary tickets require both vocabularies (Kwon *et al.*, 2023; Chen, 2026a). Profiling literacy is what prevents the workflow from becoming a ceremony performed only by a few specialists (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

22.1.9 Normalization quality checks

Counter normalization can itself introduce errors, so the workflow should include quality checks that validate the normalized numbers against a hand-verified reference trace (Chen, 2020, 2026a; Davies *et al.*, 2025). The checks compare the normalized bytes/token and launches/token against the raw profiler output for a small reference workload and alert on unexplained divergence (Vellaisamy *et al.*, 2026; Chen, 2026a). They also verify that toolchain version changes do not alter the mapping rules silently (Chen, 2020; Davies *et al.*, 2025). A normalization bug can mislabel every workload in the fleet, so the check runs before the nightly matrix rather than after a suspicious report appears (Chen, 2026a; Vellaisamy *et al.*, 2026). Normalization quality checks are the metadata hygiene that keeps cross-target comparisons trustworthy (Davies *et al.*, 2025; Chen, 2026a). They should also run on every profiling-toolchain upgrade before new traces are trusted, because a changed counter definition is as dangerous as a changed workload (Davies *et al.*, 2025; Chen, 2026a).

22.2 bottleneck locating

Bottleneck locating is where E2E tuning separates senior compiler engineers from kernel tourists (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2020). The production surprise is not peak FLOPs—it is misclassification: teams fuse

operators while launches/token remain launch-bound, or capture CUDA Graphs while bytes/token stay at eager levels (Chen, 2026a; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Use a decision tree grounded in measured counters:

1. **Launch-bound:** high launches/token, low R_{floor} proximity $\rightarrow$ graph mode, op fusion, MegaKernel seams (Chapter 12) (NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).
2. **Memory-bound:** bytes/token near roofline $\rightarrow$ fusion depth, KV layout, attention kernel choice (Dao *et al.*, 2022, 2023; SemiAnalysis, 2024; Chen, 2026a).
3. **Residency-illegal:** correct counters but p99 spikes $\rightarrow$ spill paths, graph illegality, dynamic alloc in capture region (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).
4. **Framework-bound:** compiler clean but Python/scheduler overhead $\rightarrow$ hand off to serving stack (Chapter 28), do not keep retuning tiles (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Diff lowered IR or CUDA when classification is unclear: new global allocations between attention and MLP usually explain bytes/token regressions better than guesswork (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022). On NPU, diff static DMA schedules; on CPU, diff cache-blocking parameters in oneDNN/TorchInductor output (AMD, 2024b,a; Goodfellow *et al.*, 2016; Chen, 2020).

MoE and speculative decode add **routing-bound** buckets: expert imbalance shows up as latency tails even when per-expert kernels look healthy (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). Locate whether the bottleneck is compile-time expert fusion, runtime routing, or both before changing schedules (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Attention-backend swaps (FlashAttention, SDPA, vendor libs) can move bytes/token without changing launches/token—classify whether the bottleneck lives in attention or MLP/residual before celebrating attention wins (Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Liu *et al.*, 2024; Dao

et al., 2023; NVIDIA, 2024, 2022). TaxBreak-style launch counts decompose where framework fragmentation still dominates (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022).

Cross-SKU classification differs: a launch-bound Hopper cell may be memory-bound on a bandwidth-starved SKU at the same model (Chen, 2026a; SemiAnalysis, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Bottleneck labels are properties of (model, bucket, SKU), not universal truths (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

When profiles disagree with production dashboards, suspect bucket mismatch (context length), batching policy, or speculative path divergence—reconcile framework counters with compiler-cell counters before filing compiler bugs (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Bottleneck locating is forensic work (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Review gate. Every bottleneck ticket attaches profile artifacts, classification label, and the single compiler knob under test—no unfocused “make it faster” requests (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026a, 2020).

22.2.1 Evidence rules for classification

Bottleneck labels must be earned with evidence, not asserted (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026). A launch-bound label requires the launch count and kernel-time comparison; a memory-bound label requires bytes/token against the analytic roofline; a residency label requires the buffer accounting and spill evidence; a framework label requires the host-side timeline (NVIDIA, 2024; Dao *et al.*, 2022; Chen, 2020). Each label should cite the counter row that

supports it and the counter row that rules out the alternatives (Chen, 2026a; Davies *et al.*, 2025). When two labels are plausible, the ticket should say so and list the discriminating experiment (Chen, 2020; Vellaisamy *et al.*, 2026). Evidence rules are what keep bottleneck meetings from becoming opinion contests (Davies *et al.*, 2025; Chen, 2026a).

22.2.2 Compiler-boundary and framework-boundary tickets

The most expensive classification error is opening a compiler ticket for a framework problem or vice versa (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). If the compiled kernels are clean but the timeline shows Python dispatch, allocator churn, or scheduler gaps between kernels, the bottleneck is upstream of the compiler and belongs to the serving stack (Kwon *et al.*, 2023; Chen, 2026a). If the host timeline is clean and the device timeline shows spills or extra launches, the bottleneck is inside the compiled artifact (Chen, 2020, 2026a). The boundary decision should be recorded on the ticket with the evidence for both sides, so the right owner picks it up immediately (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Boundary tickets also need an escalation path when the two teams disagree about who owns the gap (Chen, 2020; Davies *et al.*, 2025).

22.2.3 Multi-SKU classification matrix

Bottleneck labels are not portable across SKUs, so the team should maintain a classification matrix with one row per device class and context bucket (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026). The same model can be launch-bound on a high-bandwidth GPU, memory-bound on a smaller SKU, and framework-bound on a CPU deployment (NVIDIA, 2022; Goodfellow *et al.*, 2016; Chen, 2026a). Each matrix cell records the classification evidence and the last date it was measured (Chen, 2020; Davies *et al.*, 2025). When a fleet adds a new SKU, the first task is to fill its matrix cells before tuning anything (Chen, 2026a; Vellaisamy *et al.*, 2026). The matrix prevents the classic mistake of promoting an optimization that was correct for the measured SKU to an unmeasured one (Davies *et al.*, 2025; Chen, 2026a).

22.2.4 Worked classification walkthrough

Walk one dense decoder cell through the tree to make the method concrete. The profile shows hundreds of launches per token with low bytes/token relative to the roofline, so the first label is launch-bound (Vellaisamy *et al.*, 2026; Chen,

2026a). The discriminating experiment captures the launch sequence and re-measures: if ms/token drops toward the memory floor, the label is confirmed and the fix is graph capture plus fusion of the remaining seams (NVIDIA, 2024; Chen, 2026a). If bytes/token rises during capture, the second label appears—residency-illegal—and the fix moves upstream to fusion legality before launch collapse (Dao *et al.*, 2022; Chen, 2026a). The walkthrough is the template every ticket should follow: one profile, one primary label, one discriminating experiment, one knob (Davies *et al.*, 2025; Chen, 2020). Repeating the walkthrough weekly builds the pattern recognition that makes classification fast (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

22.2.5 Stop conditions and SLO boundaries

End-to-end tuning needs explicit stop conditions so effort does not continue past the point of diminishing returns (Chen, 2020, 2026a; Vellaisamy *et al.*, 2026). When the measured bytes/token and launches/token meet the product SLO at the shipped buckets, the loop pauses and the remaining budget moves to the next bottleneck or the next tier (Davies *et al.*, 2025; Chen, 2026a). Stop conditions must be written in terms of counters and SLOs, not in terms of compiler aesthetics, because the temptation is always to fuse one more operator (Chen, 2020; Vellaisamy *et al.*, 2026). The boundary should also define what is out of scope: if the residual gap is framework scheduling, the ticket leaves the tuning loop and enters the serving stack (Kwon *et al.*, 2023; Davies *et al.*, 2025). A workflow without stop conditions is a treadmill, not a system (Chen, 2026a; Vellaisamy *et al.*, 2026).

22.2.6 Profiling budget ownership

Profiling consumes machine time and engineer attention, so it needs a budget and an owner just like the experiment queue (Chen, 2020, 2026a; Davies *et al.*, 2025). The budget covers the nightly matrix, the canary tiers, and the exploratory runs attached to hypothesis cards (Vellaisamy *et al.*, 2026; Chen, 2026a). An owner reviews which profiles are being collected, which are being re-analyzed, and which have become redundant (Chen, 2020; Davies *et al.*, 2025). Profile collection without analysis is waste that makes the archive harder to search (Chen, 2026a; Vellaisamy *et al.*, 2026). Budget ownership keeps the profiling station sustainable rather than letting it grow without bound (Davies *et al.*, 2025; Chen, 2026a).

22.3 tuning workflow

Tuning workflow turns classifications into a repeatable experiment queue (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Industrial teams use a kanban-style pipeline: *hypothesis* (knob + expected counter movement) $\rightarrow$ *branch build* $\rightarrow$ *decode-cell A/B* $\rightarrow$ *review* $\rightarrow$ *merge or discard* (Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).

One change per experiment: fusion depth *or* tile size *or* backend switch—never three at once, or postmortems cannot attribute wins (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Keep a *human prior* schedule checked in as baseline; candidates must beat prior on bytes/token, launches/token, and ms/token simultaneously on the target SKU (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022; Chen and YiRage Contributors, 2026; Chen, 2020).

Workflow stages map to Part VI backends: XLA flags, TVM schedules, Triton configs, MLIR pass pipelines—each logged with version hashes (Chen, 2020; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024; AMD, 2024b). Cross-backend experiments require separate hypothesis cards; promoting a Triton win to an XLA fleet without re-profile repeats the GPU-only culture Chapter 1 rejected (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b).

Canary deployment belongs in the workflow: 5–10% of decode replicas run candidate schedules with automatic rollback when p99 or error rate breaches SLO (Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Compiler tuning without canary is how clusters learn about illegal graph capture at 2am (NVIDIA, 2024; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2022).

Experiment queues should cap WIP: three concurrent knob experiments per layer block maximum, or attribution collapses and profiles diverge (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Document expected counter deltas on each hypothesis card—“ $\downarrow$ launches/token” vs “ $\downarrow$ bytes/token”—so reviewers reject mismatched outcomes quickly (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Rollback paths must be rehearsed: fingerprinted human-prior schedules restore in one deploy step when canary breaches SLO (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Tuning workflow is incomplete without a tested undo button (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Multi-hardware tuning runs parallel queues per fleet tier—never merge GPU and NPU experiment conclusions into one narrative (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Shared methodology, separate queues, separate promotion gates (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024a; Goodfellow *et al.*, 2016; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Example 22.3.1. Hypothesis card: “Increase MLP fusion depth by one operator; expect $\downarrow$ launches/token, $\downarrow$ bytes/token if residency legal; measure $\text{ctx} \in \{512, 2048, 8192\}$.” Reject if any bucket regresses (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; NVIDIA, 2024; Kwon *et al.*, 2023).

22.3.1 Hypothesis card schema

Every experiment starts as a hypothesis card with a fixed schema: the knob, the expected counter movement, the mechanism, the target cells, and the acceptance rule (Chen, 2020; Davies *et al.*, 2025; Chen, 2026a). The knob must be one variable, named with its exact flag or schedule; the expected movement must name one or more counters and predict the direction (Vellaisamy *et al.*, 2026; Chen, 2026a). The mechanism sentence explains why the change should move that counter, which is what lets reviewers reject cards whose mechanism contradicts the classification

evidence (Davies *et al.*, 2025; Chen, 2020). The acceptance rule states the bucket set and the tolerance, so the experiment has a predefined verdict rather than a negotiated one (Chen, 2026a; Vellaisamy *et al.*, 2026). Cards with vague knobs or vague expectations should be returned before any build starts (Chen, 2020; Davies *et al.*, 2025).

22.3.2 Experiment queue operations

The tuning queue needs the same operational discipline as a code queue: owners, WIP limits, priority by expected impact, and a review gate for every merge or discard (Chen, 2020, 2026a; Davies *et al.*, 2025). Priority should come from the classification matrix—fix the bottleneck that costs the most ms/token first—rather than from whoever shouts loudest (Vellaisamy *et al.*, 2026; Chen, 2026a). Each completed experiment should publish its verdict even when the change is discarded, because negative results prevent other engineers from repeating the same dead end (Chen, 2020; Davies *et al.*, 2025). Queue metrics—cycle time, merge rate, discard reasons—should be reviewed monthly to find where the workflow itself is slow (Chen, 2026a; Vellaisamy *et al.*, 2026). A tuning queue without operational metrics is a list, not a system (Davies *et al.*, 2025; Chen, 2026a).

22.3.3 Canary design specifics

Canary design must specify the traffic split, the decision window, and the rollback condition before the candidate ships (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). The canary should compare p99 and error rate against the baseline tier under the same request mix, with a predefined breach threshold (Chen, 2026a; Davies *et al.*, 2025). Compiler candidates also need artifact-level canaries: confirm the shipped binary is the benchmarked binary by matching fingerprints at load time (Chen, 2020, 2026a). A canary that runs for days without a decision is a failed process, so the window should be bounded by traffic volume rather than calendar time (Kwon *et al.*, 2023; Davies *et al.*, 2025). The canary report belongs with the experiment verdict so promotion is traceable to evidence (Chen, 2026a; Vellaisamy *et al.*, 2026).

22.3.4 Rollback drills

A rollback path that has never been exercised is a hope, not a mechanism (Chen, 2026a, 2020; Davies *et al.*, 2025). The tuning workflow should include a scheduled rollback drill that restores the previous artifact under simulated load and measures

the recovery time (Vellaisamy *et al.*, 2026; Chen, 2026a). The drill verifies that the fingerprinted human-prior schedule is available, that the deploy step is one command, and that telemetry confirms the switch (Chen, 2020; Davies *et al.*, 2025). Rollback drills also expose dependencies that were forgotten: a candidate that changed a shared runtime component cannot be rolled back by swapping only its kernels (Chen, 2026a; Vellaisamy *et al.*, 2026). Teams that drill rollback treat incidents as rehearsed operations; teams that skip drills improvise under load (Davies *et al.*, 2025; Chen, 2026a).

22.3.5 Feedback cadence and re-tuning triggers

The loop does not end at merge; it returns to profiling on a cadence and on explicit triggers (Chen, 2020, 2026a; Vellaisamy *et al.*, 2026). Nightly dashboards compare the shipped counters against the hypothesis-card expectations and alert on drift (Davies *et al.*, 2025; Chen, 2026a). Explicit triggers—driver upgrades, compiler upgrades, model releases, new context buckets, new SKUs—reopen the classification matrix before any tuning starts (Vellaisamy *et al.*, 2026; Chen, 2020). A monthly review should walk the highest-impact drift and decide whether it is environment, workload, or artifact movement (Davies *et al.*, 2025; Chen, 2026a). This cadence is what keeps the workflow alive between projects and prevents it from fossilizing into a document nobody reads (Chen, 2020; Vellaisamy *et al.*, 2026).

22.3.6 Escape hatches and incident response

The tuning workflow should define what happens when a candidate that passed every gate still fails in production, because that moment is when trust in the system is made or broken (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026). The incident response starts by switching traffic to the fingerprinted prior artifact, then preserves the failing artifact and its reports for diagnosis (Chen, 2020, 2026a). The postmortem compares the failing cell against the canary cell to find the difference—traffic mix, bucket, driver, or environment (Davies *et al.*, 2025; Kwon *et al.*, 2023). The outcome should update the workflow: a new gate, a wider canary window, or a new classification cell (Chen, 2020; Vellaisamy *et al.*, 2026). Teams that treat escape-hatch incidents as workflow failures improve the system; teams that blame the engineer repeat the incident (Chen, 2026a; Davies *et al.*, 2025).

22.3.7 Cross-backend promotion cards

Promoting a change from one backend to another requires a new hypothesis card, not a copy of the old one, because the counters and knobs differ (Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). A Triton fusion win on GPU cannot be assumed to survive an XLA or IREE compile on the same model; the mechanism must be re-derived for the new backend’s passes (IREE / iree-org contributors, 2024; Chen *et al.*, 2018b; Chen, 2026a). The promotion card should state which artifact moved, which backend it came from, and which new artifact and bucket set will validate it (Chen, 2020; Davies *et al.*, 2025). Cross-backend promotion without a new card is how teams inherit someone else’s illegal schedule (Chen, 2026a; Vellaisamy *et al.*, 2026). Each promotion that passes becomes a row in the classification matrix, extending the workflow’s evidence base (Davies *et al.*, 2025; Chen, 2026a).

22.3.8 Artifact generation inside the workflow

Every experiment should produce the artifact set its gate will consume: the compiled binary, its fingerprint, the compile report, the IR diff, and the benchmark rows (Chen, 2020, 2026a; Vellaisamy *et al.*, 2026). Artifact generation should be scripted in the same build system that release uses, so the experiment artifact is a subset of the release artifact rather than a one-off build (Davies *et al.*, 2025; Chen, 2026a). A candidate whose artifact cannot be regenerated is not a candidate; it is an anecdote (Chen, 2020; Vellaisamy *et al.*, 2026). The workflow should therefore fail fast when artifact generation diverges from the release path, because that divergence is where benchmarked binaries and shipped binaries stop being the same thing (Davies *et al.*, 2025; Chen, 2026a).

22.4 regression gates

Regression gates encode the workflow into CI so human discipline survives headcount churn (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Minimum bar for merge:

- Decode-cell A/B on agreed buckets passes on bytes/token, launches/token, ms/token (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).
- No new global allocations in captured graph regions (GPU) or new DDR

hops (NPU) (NVIDIA, 2024; AMD, 2024a; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023).

- IR diff attached; roofline or buffer accounting for target SKU (SemiAnalysis, 2024; Chen, 2020; Davies *et al.*, 2025; AMD, 2024b; Goodfellow *et al.*, 2016; Dao *et al.*, 2022; Liu *et al.*, 2024; Dao *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; NVIDIA, 2022; Chen and YiRage Contributors, 2026; AMD, 2024a; Kwon *et al.*, 2023).
- Fingerprint or schedule artifact versioned with compiler git SHA (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Nightly jobs replay the full cell matrix across representative SKUs; drift alerts fire when counters move more than agreed ϵ without an approved compiler bump (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Gates fail closed: ambiguous profiles block merge until reproduced (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026a, 2020; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Chapter 21 benchmarks supply the harness; Chapter 25 automates search inside the same gates (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Production runbooks (Chapter 26) assume these gates ran before containers ship (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024a; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Flaky gates erode trust: quarantine tests with high variance until environment drift is fixed—do not weaken thresholds to greenwash CI (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Pin driver, CUDA, and compiler versions in gate metadata; unexplained drift triggers infra

tickets before compiler blame (NVIDIA, 2022, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; AMD, 2024b).

Security and determinism gates belong beside performance: numerics diffs beyond agreed tolerance block merge even when ms/token improves (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Regression gates protect users, not just leaderboard scores (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Publish gate failures publicly inside the compiler team: failed experiments teach more than silent reverts (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). A culture that hides negative results repeats the same illegal fusion twice (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

22.4.1 Gate hierarchy

Regression gates should be layered so each layer is cheap and catches the failures the previous layer cannot (Chen, 2020, 2026a; Vellaisamy *et al.*, 2026). The compile layer rejects artifacts that fail to build, fail legality checks, or introduce new allocations in captured regions (NVIDIA, 2024; AMD, 2024a). The cell layer replays the decode-cell benchmark on the agreed buckets and rejects counter regressions (Davies *et al.*, 2025; Chen, 2026a). The integration layer runs the artifact through the serving harness and checks numerical and behavioral parity (Chen, 2020; Davies *et al.*, 2025). The canary layer is the final gate that confirms the artifact under real traffic (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Each layer publishes its report so a failure at any level points to the exact artifact and measurement (Chen, 2026a; Davies *et al.*, 2025). Layering keeps CI fast because most candidates die at the cheap compile layer (Chen, 2020; Vellaisamy *et al.*, 2026).

22.4.2 Tolerance derivation

Gate tolerances should be derived from measurement noise, not chosen by negotiation (Davies *et al.*, 2025; Chen, 2026a, 2020). Run the same artifact repeatedly on the target hardware, measure the counter distributions, and set epsilon from the observed variance plus a safety margin (Vellaisamy *et al.*, 2026; Chen, 2026a). A tolerance tighter than the hardware noise will produce flaky gates; a tolerance looser than the product SLO will let regressions ship (Chen, 2020; Davies *et al.*, 2025). The derivation should be re-run when hardware, driver, or workload changes, because noise characteristics change with all three (Chen, 2026a; Vellaisamy *et al.*, 2026). Publishing the derivation with the gate makes tolerance decisions reviewable instead of mysterious (Davies *et al.*, 2025; Chen, 2026a).

22.4.3 Quarantining flaky gates

A flaky gate is a process failure that should be quarantined, diagnosed, and re-enabled, never silently weakened (Chen, 2026a, 2020; Davies *et al.*, 2025). Quarantine means the gate stops blocking merges while an owner investigates the variance source—usually environment drift, shared machine contention, or an inadequate tolerance (Vellaisamy *et al.*, 2026; Chen, 2026a). The quarantine should be visible in the dashboard with a ticket and an owner, so it cannot become a permanent bypass (Chen, 2020; Davies *et al.*, 2025). Re-enabling requires the diagnosis and the tolerance derivation update (Chen, 2026a; Vellaisamy *et al.*, 2026). Teams that weaken thresholds to greenwash CI trade short-term green for long-term blindness (Davies *et al.*, 2025; Chen, 2026a).

22.4.4 Numerics and determinism gates

Performance gates must be paired with numerical gates because a faster artifact that changes outputs is a defect (Chen, 2020; Davies *et al.*, 2025; Chen, 2026a). Each candidate should be compared against the reference under a bounded-error metric at the served buckets, with the tolerance derived from the model’s quality requirements (Dao *et al.*, 2022; Chen, 2026a). Deterministic mode should lock reduction order where the product needs bitwise reproducibility, while interactive serving may allow bounded drift if it is measured and logged (Chen, 2020; Davies *et al.*, 2025). Numerical gates belong beside performance gates in the same CI run, because separating them lets a fast-but-wrong candidate pass review twice (Chen, 2026a; Davies *et al.*, 2025).

22.4.5 Cross-hardware gate matrix

Multi-target fleets need a gate matrix so a change that helps GPU but hurts CPU or NPU cannot merge silently (Chen, 2020; AMD, 2024a; Chen, 2026a). The matrix rows are the device classes and context buckets the change affects; the columns are the compile, cell, integration, and numerical gates (Davies *et al.*, 2025; AMD, 2024b; Chen, 2026a). A candidate that only runs on GPU should be labeled GPU-only and blocked from heterogeneous promotion, not given a pass because its GPU numbers improved (Chen, 2020; Vellaisamy *et al.*, 2026). The matrix cells should be filled by the per-target classification evidence from the profiling station (Chen, 2026a; Davies *et al.*, 2025). When a fleet adds a target, the matrix must be extended and the prior artifacts re-gated before the rollout (AMD, 2024b; Vellaisamy *et al.*, 2026). A gate matrix is how the workflow enforces the book’s multi-hardware thesis at CI time (Davies *et al.*, 2025; Chen, 2026a).

22.4.6 Operating documentation outputs

The workflow should produce living operating documents rather than a single tuning guide (Chen, 2020, 2026a; Vellaisamy *et al.*, 2026). The decode-cell protocol, the classification matrix, the current hypothesis queue, and the gate dashboard are each maintained artifacts with owners and update cadences (Davies *et al.*, 2025; Chen, 2026a). Release notes should link the artifact fingerprint, the gate reports, and the hypothesis cards that the release contains (Chen, 2020; Vellaisamy *et al.*, 2026). On-call runbooks should include the rollback drill and the incident template from this workflow (Chen, 2026a; Davies *et al.*, 2025). Operating documentation turns the four stations into an institutional capability that survives individual engineers (Chen, 2020, 2026a).

22.4.7 Gate failure learning review

Gate failures are learning data, and the workflow should hold a lightweight review of them on a cadence (Chen, 2020, 2026a; Davies *et al.*, 2025). The review clusters failures by cause—misclassification, tolerance error, environment drift, or a genuine regression—and decides what the workflow should change (Vellaisamy *et al.*, 2026; Chen, 2026a). A cluster of misclassifications means the classification training or the decision tree needs improvement; a cluster of environment failures means the protocol needs stronger fingerprints (Chen, 2020; Davies *et al.*, 2025). The review should publish its conclusions next to the gate dashboards so the loop visibly improves (Chen, 2026a; Vellaisamy *et al.*, 2026). Without a learning review, the

gate system punishes failures without capturing their lessons (Davies *et al.*, 2025; Chen, 2026a).

22.4.8 Gate report cadence

The gate system should publish reports on a cadence so the whole team can watch the workflow’s health (Chen, 2020, 2026a; Vellaisamy *et al.*, 2026). The report includes pass rates per gate layer, quarantine list, tolerance revisions, and the counter drift dashboard (Davies *et al.*, 2025; Chen, 2026a). A weekly digest that summarizes these numbers keeps the workflow visible without requiring every engineer to query the CI system (Chen, 2020; Davies *et al.*, 2025). When pass rates drop, the digest should point to the dominant failure cluster and the open diagnosis ticket (Chen, 2026a; Vellaisamy *et al.*, 2026). A visible gate report is the difference between a process the team trusts and a process the team discovers after an incident (Davies *et al.*, 2025; Chen, 2026a).

22.4.9 SLO-to-counter mapping

Product SLOs must be translated into counter targets before the workflow can gate on them (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026). An ms/token SLO decomposes into a launch budget and a bytes budget at the served buckets, and those budgets become the hypothesis-card acceptance rules (Chen, 2020; Davies *et al.*, 2025). The mapping should be explicit and reviewed when the SLO changes, because a looser SLO may end tuning early while a tighter one reopens the classification matrix (Chen, 2026a; Vellaisamy *et al.*, 2026). Teams that gate only on ms/token cannot tell which lever moved; teams that gate only on counters lose the product meaning (Davies *et al.*, 2025; Chen, 2026a). The mapping document connects the workflow’s measurements to the product’s promises (Chen, 2020; Vellaisamy *et al.*, 2026).

22.5 Key Takeaways

- **Profile the production shape, not a proxy.** Hardware-aware profiling fails the moment a team measures prefill shapes, batch-32 training graphs, or synthetic GEMMs while production runs batch-1 decode with paged KV (Kwon *et al.*, 2023; Davies *et al.*, 2025). Freeze decode cells and normalize GPU, CPU, and NPU tools to the same bytes/token and launches/token so the numbers are comparable across targets (Chen, 2026a; Vellaisamy *et al.*,

2026).

- **Classify the bottleneck before touching a knob.** Bottleneck locating—deciding whether a workload is launch-, memory-, residency-, or framework-bound—is what separates senior compiler engineers from kernel tourists, because changing a compiler flag before classifying wastes effort on the wrong term (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). The production surprise is usually orchestration or residency, not the matmul the tourist reaches for first (NVIDIA, 2024; Vellaisamy *et al.*, 2026).
- **Change one knob per experiment against a human baseline.** A repeatable tuning workflow turns classifications into an experiment queue: establish a human-prior baseline, vary one knob at a time, and canary on the fleet before promotion (Chen, 2020; Chen and YiRage Contributors, 2026). Bundling changes makes a regression impossible to attribute, which is why the discipline is one variable per measurement (Davies *et al.*, 2025).
- **Encode the workflow as regression gates.** Human discipline does not survive headcount churn, so the workflow must live in CI: enforce counter non-regression, residency legality, and versioned, fingerprinted artifacts on every merge (Chen, 2020; Chen and YiRage Contributors, 2026). A gate that checks bytes/token and launches/token at the decode cell is what stops a microbench-only win from shipping (Chen, 2026a; Vellaisamy *et al.*, 2026).
- **Profile-first beats tile-size folklore.** The recurring lesson of end-to-end tuning is that measured decode counters, not inherited tile-size heuristics, decide what to change (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Folklore tuned on someone else’s prefill workload is exactly the trap a frozen decode cell and a profile-first loop are designed to avoid (SemiAnalysis, 2024).

Practical decision rules. Six rules compress this chapter for a working session. First, never tune before the frozen decode cell and classification label exist, because unclassified tuning is gambling (Davies *et al.*, 2025; Chen, 2026a). Second, change one knob per experiment and write the expected counter movement on the card before building (Chen, 2020; Vellaisamy *et al.*, 2026). Third, keep the human-prior schedule fingerprinted and reachable so every candidate competes against a known baseline (Chen, 2026a; Davies *et al.*, 2025). Fourth, gate every merge on the layered CI plus canary, with tolerances derived from measured noise (Chen, 2020, 2026a). Fifth, quarantine flaky gates with owners instead of weakening them, and publish negative results alongside wins (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Sixth,

re-open the classification matrix on every driver, compiler, model, bucket, and SKU change rather than assuming the last labels still hold (Chen, 2026a; Davies *et al.*, 2025). Teams that internalize these rules turn tuning into a system; teams that skip them repeat the same misclassification every quarter (Vellaisamy *et al.*, 2026; Chen, 2026a).

22.6 Conclusion

Compiler-driven end-to-end performance tuning is the operational glue between backends (Part VI), benchmarks (Chapter 21), automated search (Chapter 25), and production packaging (Chapter 26) (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Without hardware-aware profiling and regression gates, compiler teams optimize kernels while decode goodput flatlines (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Gate merges on buffer accounting and reproducible decode cells—never on a single Nsight screenshot (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026). This profile-first, regression-gated workflow is general, but it pays off most on the hardest decode targets, which is where the next chapter applies it: Chapter 23 specializes the workflow for LLM and MoE compilation, where data-dependent expert routing and growing KV caches stress exactly the launches/token and bytes/token counters this chapter taught teams to measure (Liu *et al.*, 2024; Kwon *et al.*, 2023).

Chapter 23

LLM and MoE Specialized Compilation

Dense decoder LLMs and mixture-of-experts (MoE) models break the assumptions generic compiler stacks were built for: batch-1 decode, growing KV residency, routed expert subgraphs, and framework-owned paging policies (Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). Part VI taught backend lowering; Chapter 22 taught E2E tuning loops; this chapter specializes those tools for **LLM decode compilation** and **MoE-aware schedules** across GPU, CPU, and NPU targets (Chen, 2020; Chen and YiRage Contributors, 2026; AMD, 2024b,a; Goodfellow *et al.*, 2016).

TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). Memory-Floor CUDA Graphs on Qwen-2.5-7B (BS=1, ctx=2048) yields 1.259 $\times$ on H100 by removing host overhead—yet cannot fix illegal HBM traffic when MoE routing and KV export remain unfused (Chen, 2026a; NVIDIA, 2024; Liu *et al.*, 2024). Specialized compilation means every pass knows whether it is optimizing prefill, decode, or expert subgraphs—and refuses to merge them into one misleading benchmark (Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020).

Four topics anchor the chapter: **LLM decode compile** (single-request residency), **MoE scheduling** (routing + expert fusion), **KV hardware adaptation** (layout and paging boundaries), and **heterogeneous MoE load balancing** (multi-SKU expert placement) (Liu *et al.*, 2024; AMD, 2024b; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). DeepSeek-class deployments disaggregate prefill and decode; compiler specialization must follow that split instead of fighting it (Davies

et al., 2025; Liu *et al.*, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Anti-patterns: compiling whole-model graphs without decode role tags; fusing through PagedAttention handles; ignoring routing entropy in MoE profiles; promoting GPU expert schedules to CPU fallback without re-profile (Kwon *et al.*, 2023; Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Each section below replaces one anti-pattern with a specialized compile practice (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Speculative decoding and draft models add another decode subgraph: compiler specialization must treat draft and target as separate capture regions with shared KV invariants (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Merging draft control flow into main decode capture breaks graph legality (NVIDIA, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Quantized decode (INT4/FP8 weights, FP16 activations) changes fusion legality: specialized passes must preserve dequant residency on-chip across attention-MLP seams (SemiAnalysis, 2024; NVIDIA, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024; AMD, 2024b). Profile bytes/token with quantization enabled—kernel names change but counters do not (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Industrial runbooks tie LLM/MoE compile artifacts to model card revisions: tokenizer, rope, expert count, and paging policy must hash into schedule fingerprints (Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a;



Figure 23.1: LLM decode compile path: framework export $\rightarrow$ fusion $\rightarrow$ target codegen $\rightarrow$ decode-cell validation (not prefill GEMM).

Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Silent drift between training and serving configs is a top source of “compiler regression” tickets that are actually model mismatches (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

23.1 LLM decode compile

LLM decode compile starts when the compiler owns a *decoder slice*—attention, MLP, norms, rotary, residual—exported at BS=1 with explicit KV shapes (Kwon *et al.*, 2023; Davies *et al.*, 2025; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020). Training-shaped graphs (large batch, no paging) mislead tile choices: shared-memory budgets that fit prefill often spill KV on decode (Chen, 2026a; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Compiler passes should tag operators with *decode role*: KV producer, KV consumer, stateless compute (Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024b,a; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; SemiAnalysis, 2024; NVIDIA, 2024, 2022). Fusion without role tags reunifies nodes that frameworks intentionally split for paging—breaking vLLM/SGLang invariants (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

FlashAttention and Flash-Decoding shrink attention bytes but MLP/residual boundaries often remain separate kernels unless MegaKernel plans reunify them (Dao *et al.*, 2022, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; NVIDIA, 2024, 2022). Decode compile therefore tracks *whole-step* bytes/token, not

Decode compile invariant	Violation symptom
KV stays paged / blocked	capacity collapse, OOM tails
Graph capture legality	silent eager fallback
Same IR, per-SKU codegen	“works on H100” only
Bytes/token gate on merge	fast GEMM, slow decode

Table 23.1: LLM decode compile invariants for specialized compiler stacks.

attention microbenches alone (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Export boundaries matter: FX/ONNX dumps often freeze prefill shapes (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Specialized LLM pipelines inject dynamic seq-len and paging metadata before lowering (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Without metadata, “successful” compile produces undeployable schedules (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Rotary, norm, and sampling tails are easy to ignore in decode compile yet dominate launches/token on small models (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Include them in fusion plans and profile whole-step counters (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Multi-hardware delta. Hopper decode favors TMA-friendly KV tiles; XDNA requires static KV slot assignment; CPU decode fights LLC capacity—identical transformer block, different fusion limits (SemiAnalysis, 2024; AMD, 2024b,a;

Goodfellow *et al.*, 2016; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Example 23.1.1. Decode cell (BS=1, ctx=2048): compiler fusion that materializes full KV tensors between layers doubles bytes/token even if attention kernel is optimal (Chen, 2026a; Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

23.1.1 Role-tagged IR and paging contracts

Specialized decode compilation depends on role tags that generic IR does not carry (Davies *et al.*, 2025; Chen, 2020; Chen and YiRage Contributors, 2026). Every operator in the decoder slice should be tagged as a KV producer, a KV consumer, or stateless compute, and the tags must survive lowering so backend passes can check paging legality at every seam (Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen, 2026a). A fusion pass that merges across a KV boundary without understanding the framework’s page contract can silently densify paged tensors, breaking capacity guarantees the serving stack relies on (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a). The paging contract should be explicit IR metadata: which tensors are paged, which block-table layout they use, and which operators may legally read through the block table (Chen, 2020; Chen and YiRage Contributors, 2026). When a pass cannot prove that a transform preserves the contract, it must refuse the transform rather than produce a correct-looking binary with broken serving semantics (Chen, 2026a; Davies *et al.*, 2025).

23.1.2 Export metadata injection

Framework exports often freeze prefill-shaped graphs, so the specialized pipeline must inject dynamic-sequence and paging metadata before lowering (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2020). The injection step records context buckets, head counts, GQA and MQA structure, page sizes, and the serving framework’s KV contract (Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). This metadata drives tile selection, layout assignment, and fusion legality checks downstream (Chen, 2020, 2026a; Vellaisamy *et al.*, 2026). A compiler that lowers an export without this step inherits training assumptions that are false at decode time (Davies *et al.*, 2025; Kwon *et al.*, 2023). The injection should be

deterministic and versioned so the same model card always produces the same decode artifact (Chen, 2020; Davies *et al.*, 2025).

23.1.3 Attention-backend selection

Attention kernels are a compiler selection problem: the same HLO or IR region can lower to a fused FlashAttention-style path, a Triton kernel, a vendor custom call, or an eager fallback (Dao *et al.*, 2022, 2023; Vellaisamy *et al.*, 2026). Each choice has different fusion boundaries, and the selection must be made with knowledge of the surrounding MLP and norm seams (Chen, 2026a; Davies *et al.*, 2025). A compiler that always selects the fastest isolated attention kernel can leave the output-projection and MLP boundaries unfused, so the decode-step counters decide rather than the attention microbenchmark (Vellaisamy *et al.*, 2026; Chen, 2026a). The selection should be recorded as IR metadata so a later pass can fuse around the chosen boundary or justify why it cannot (Chen, 2020; Chen and YiRage Contributors, 2026). Attention-backend selection is where decode specialization shows up as a concrete compile-time choice (Dao *et al.*, 2022; Davies *et al.*, 2025).

23.1.4 Slice-level CI artifacts

Decode compile should be validated on slice-level CI artifacts rather than whole-model builds alone (Chen, 2020, 2026a; Vellaisamy *et al.*, 2026). The artifact set includes the exported decoder slice, the compiled per-bucket binaries, the fusion report, the byte sheet, and the decode-cell benchmark rows (Davies *et al.*, 2025; Chen, 2026a). Each artifact is keyed by the model card revision so a mismatch between training and serving configurations is caught at build time (Kwon *et al.*, 2023; Chen, 2020). The slice-level gate runs on every model change, while full-model gates run on release cadence (Chen, 2026a; Vellaisamy *et al.*, 2026). Slice artifacts also make cross-backend comparison tractable because the same slice can be compiled through several stacks with the same harness (Davies *et al.*, 2025; Chen, 2020).

23.1.5 Decode IR versus training IR

The same model often flows through a training-shaped export and a decode-shaped export, and the compiler should treat them as different programs (Davies *et al.*, 2025; Chen, 2020, 2026a). Training IR assumes large batches, no paging, and throughput objectives; decode IR assumes batch-1, paged KV, and per-token latency (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022). Sharing one

artifact between the two hides the wrong trade in both directions (Chen, 2020; Davies *et al.*, 2025). The decode pipeline should therefore start from a decode-specific export or a transform that converts the training graph with explicit role and paging metadata (Chen, 2026a; Vellaisamy *et al.*, 2026). CI should gate decode artifacts on decode cells and training artifacts on training cells, never interchanging the two (Davies *et al.*, 2025; Chen, 2026a).

23.1.6 Speculative and sampling tails

Speculative decoding and sampling control flow are part of the decode program even when they live outside the compiled kernel body (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). The compiler should compile the draft model and the target model as separate capture regions with shared KV invariants, and the sampling logic should stay in an eager hull (NVIDIA, 2024; Chen, 2020). Merging draft branches into the target capture breaks graph legality and reintroduces host dispatch at the wrong point (Davies *et al.*, 2025; Chen, 2026a). The acceptance path and the rejection path both need profiling because the target model recomputes over different KV state after rejection (Kwon *et al.*, 2023; Dao *et al.*, 2023). Compile specialization for LLMs therefore includes the scheduling shell around the kernels, not only the kernels (Chen, 2020; Vellaisamy *et al.*, 2026).

23.1.7 Numerics contracts under quantization

Quantized decode changes the numerical contract that fusion passes must preserve (SemiAnalysis, 2024; Davies *et al.*, 2025; Chen, 2026a). Weight dequantization must happen on-chip and stay inside the fused region where possible; separate dequant kernels add both launches and bytes (Chen, 2020; Vellaisamy *et al.*, 2026). Softmax carriers and layer-norm accumulators need higher precision than the quantized matmuls, and the compiler should enforce that split by dtype metadata (Dao *et al.*, 2022; Chen, 2026a). Numerical tests for quantized decode artifacts should run at the served context buckets because error grows with KV length and depth (Davies *et al.*, 2025; Dao *et al.*, 2022). The numeric contract belongs in the model card and the artifact fingerprint so a precision change is traceable (Chen, 2020, 2026a).

23.1.8 Model-card artifact binding

Every decode artifact should be bound to the model card that produced it, because LLM configs drift silently between training and serving (Davies *et al.*, 2025; Chen, 2020, 2026a). The binding records tokenizer version, RoPE configuration, head

counts, expert topology, KV policy, and quantization format (Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). A compiler regression ticket that turns out to be a model mismatch wastes both teams’ time; binding prevents it by failing at build time when the model card changed (Chen, 2020; Davies *et al.*, 2025). The artifact manifest should hash the model card fields that affect the schedule (Chen, 2026a; Vellaisamy *et al.*, 2026). Model-card binding is the specialized form of the general rule that the artifact and its configuration must be reproducible together (Davies *et al.*, 2025; Chen, 2026a).

23.1.9 Decode artifact safety tests

Decode artifacts need safety tests that catch serving-contract violations before rollout (Chen, 2020, 2026a; Davies *et al.*, 2025). The tests should verify paged KV stays paged through every fusion seam, graph capture remains legal at each context bucket, and no hidden allocation appears on the hot path (Kwon *et al.*, 2023; NVIDIA, 2024; Vellaisamy *et al.*, 2026). They should also check that dynamic sequence lengths select the correct bucket artifact and that fallback behavior is logged (Davies *et al.*, 2025; Chen, 2026a). Safety tests run on every model-card change and every compiler change that touches KV or routing (Chen, 2020, 2026a). A decode artifact that passes performance gates but fails a safety test is not deployable (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

23.1.10 Artifact validation and signing

Decode artifacts should be validated and signed at build time so serving can verify that the loaded binary is the benchmarked artifact (Chen, 2020, 2026a; Davies *et al.*, 2025). The validation step re-runs the artifact through the compile report checks and compares the fingerprint against the manifest (Chen, 2026a; Vellaisamy *et al.*, 2026). Serving should refuse to load an artifact whose signature or fingerprint does not match its model card (Chen, 2020; Davies *et al.*, 2025). Signing also makes fleet forensics tractable: when a behavior changes, the loaded artifact hash identifies the exact binary and configuration (Chen, 2026a; Davies *et al.*, 2025). Artifact validation and signing are the operational floor beneath all the specialization this chapter describes (Chen, 2020; Vellaisamy *et al.*, 2026).



Figure 23.2: MoE scheduling: router $\rightarrow$ active experts $\rightarrow$ fused compute $\rightarrow$ combine (compile-time vs runtime split).

23.2 moe scheduling

MoE scheduling is where decode compilation meets *dynamic routing* (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020). The production surprise is not expert GEMM peak FLOPs—it is launch storms when every token activates a different expert subset without batched grouping (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Compiler specialization options:

1. **Compile expert templates once** per shape bucket; runtime selects indices (Liu *et al.*, 2024; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).
2. **Fuse router + top- k + expert prologue** when graph capture legal; isolate when control flow diverges (NVIDIA, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).
3. **Expert parallelism passes** that align with DeepEP-style dispatch when disaggregated (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

OLMoE-scale launch counts (TaxBreak) warn that MoE decode without fusion remains orchestration-bound even when experts are “fast” (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage

Contributors, 2026; Chen, 2020, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Profile launches/token *per layer* with routing entropy logged—mean ms/token hides expert imbalance tails (Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Review gate. MoE compile merges require expert activation histograms from production-like traffic, not uniform random routing (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Shared-expert and routed-expert splits (DeepSeek-class) need distinct compile templates: shared experts fuse like dense MLP; routed experts batch by token groups (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Collapsing both into one schedule loses routing visibility (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

When expert counts exceed on-chip weight residency, compilation must coordinate with runtime prefetch—bytes/token includes HBM traffic for cold experts (Liu *et al.*, 2024; SemiAnalysis, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b,b). MoE scheduling is a memory problem disguised as a routing problem (Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

23.2.1 Routing entropy measurement

MoE compilation cannot be tuned on uniform routing, because real router distributions are skewed and bursty (Liu *et al.*, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*,

2026). The profiling station should record per-expert activation histograms from production-like traffic and attach them to every schedule decision (Chen, 2026a; Kwon *et al.*, 2023; Chen, 2020). Entropy and skew statistics belong in the MoE profile report, because they predict whether batching, prefetching, or duplication will pay (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). A schedule that assumes balanced routing will thrash when real traffic concentrates on a few experts (Liu *et al.*, 2024; Chen, 2026a). The histogram should be versioned with the model card and re-measured when the traffic mix changes (Chen, 2020; Davies *et al.*, 2025).

23.2.2 Expert templates and grouping

Compiling every expert as an independent kernel multiplies compile time and launch count; the specialized approach is to compile expert templates once per shape bucket and reuse them across experts of the same structure (Liu *et al.*, 2024; Chen, 2020; Chen and YiRage Contributors, 2026). Tokens that route to the same expert can group into one dispatch when the runtime batches them, turning many small expert calls into fewer grouped kernels (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). The grouping policy is a runtime decision, but the compiler must provide the templates, the buffer contracts, and the launch metadata that make grouping cheap (Chen, 2020, 2026a). When expert weights differ only numerically, one compiled template serves all of them; when hidden structures differ, templates must split by structure (Liu *et al.*, 2024; Davies *et al.*, 2025). Template reuse is the compile-time half of MoE launch reduction (Chen, 2020; Vellaisamy *et al.*, 2026).

23.2.3 Router fusion legality

Fusing the router, top-k selection, and expert prologue into one captured region removes launches, but only when the control flow remains capture-safe (NVIDIA, 2024; Liu *et al.*, 2024; Davies *et al.*, 2025). Data-dependent expert counts, dynamic gather indices, and host-visible routing decisions break static capture unless the graph is bucketed (Kwon *et al.*, 2023; Chen, 2026a, 2020). The legality pass should prove that the fused router has bounded top-k, static dispatch structure within a bucket, and no allocation in the captured region (NVIDIA, 2024; Chen and YiRage Contributors, 2026). When proof fails, the compiler keeps the router in an eager hull and fuses only the expert body (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Router fusion is therefore a legality decision per MoE layer, not a global feature flag (Chen, 2020, 2026a).

23.2.4 Expert capacity and prefetch

When all expert weights cannot fit on-chip, the schedule must prefetch cold experts from HBM while hot experts compute (Liu *et al.*, 2024; SemiAnalysis, 2024; Chen, 2026a). The prefetch distance is a compiler decision expressed as bytes moved and overlap achieved, not a runtime nicety (Chen, 2020; Davies *et al.*, 2025). The byte sheet must count cold-expert HBM traffic per token, because that traffic is part of bytes/token even when the expert kernel itself looks efficient (Chen, 2026a; Vellaisamy *et al.*, 2026). Prefetch policies should follow the activation histogram: frequently hot experts stay resident, cold experts stream (Liu *et al.*, 2024; Davies *et al.*, 2025). Compiler and runtime must agree on the resident set through metadata so a policy change does not invalidate the compiled schedule (Chen, 2020; Chen and YiRage Contributors, 2026).

23.2.5 Batched decode grouping

MoE decode at batch-1 still benefits from grouping across concurrent requests when the scheduler batches tokens through the expert layer (Kwon *et al.*, 2023; Liu *et al.*, 2024; Davies *et al.*, 2025). The compiler should provide grouped dispatch templates that accept a token list per expert and a combine kernel that routes outputs back to their requests (Chen, 2020; Vellaisamy *et al.*, 2026). Grouping reduces the per-token expert launch count without changing any expert’s arithmetic (Liu *et al.*, 2024; Chen, 2026a). The group size and dispatch threshold are runtime parameters that should be tuned against the routing histogram, not fixed in the kernel (Davies *et al.*, 2025; Chen, 2026a). Compiler and scheduler must share the same token-to-expert mapping format so grouping does not require a copy (Chen, 2020; Chen and YiRage Contributors, 2026).

23.2.6 Routing-bound classification

MoE profiles need a routing-bound classification distinct from launch, memory, and residency bounds (Liu *et al.*, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). A layer is routing-bound when its latency is set by expert imbalance or dispatch serialization rather than by any expert kernel (Chen, 2026a, 2020). The evidence is the per-expert wait histogram: when a few experts queue while others idle, the fix is scheduling, duplication, or routing policy, not kernel tuning (Davies *et al.*, 2025; Liu *et al.*, 2024). Routing-bound labels should be recorded per layer and per bucket, because imbalance at short context may vanish at long context or vice versa (Chen, 2026a; Vellaisamy *et al.*, 2026). Classifying routing separately

prevents the recurring mistake of tuning expert GEMMs while the router owns the tail (Davies *et al.*, 2025; Chen, 2026a).

23.2.7 Combine kernel compilation

The combine step that merges expert outputs is a separate compile surface from the expert GEMMs, and it is where routing round trips reappear if compiled carelessly (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2026a). The combine kernel should read each selected expert’s output in the layout the expert produced and accumulate into the correct token row without an intermediate gather to dense form (Chen, 2020; Vellaisamy *et al.*, 2026). Compiler passes should fuse the combine with the following residual and norm where legal, because those seams are frequent launch sources in MoE layers (Chen, 2026a; Davies *et al.*, 2025). The combine cost belongs in the MoE byte sheet, and routing-bound classifications should include it (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). Combine compilation is the difference between an MoE layer that feels fused and one that merely has fast experts (Chen, 2020, 2026a).

23.2.8 MoE memory planning

MoE scheduling is also a memory-planning problem because expert weights, activation buffers, and the combine workspace compete for the same on-chip and HBM budgets (Liu *et al.*, 2024; Chen, 2026a; Davies *et al.*, 2025). The compiler should plan expert staging alongside activation residency so the resident-expert set and the fused region do not fight (Chen, 2020; SemiAnalysis, 2024). When weights exceed the budget, the plan must state which experts stream and what that costs per token (Chen, 2026a; Vellaisamy *et al.*, 2026). Memory planning should be per-tier because on-chip capacity differs between GPU, CPU, and NPU expert hosts (AMD, 2024b,a; Goodfellow *et al.*, 2016). A schedule that ignores memory planning will produce launch-efficient MoE kernels that thrash on capacity (Liu *et al.*, 2024; Chen, 2026a).

23.2.9 Per-bucket routing histograms

Routing skew is not uniform across context lengths, so histograms should be collected per bucket rather than aggregated globally (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2026a). Long-context requests may route differently from short prompts because the attention state changes token distributions (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Compile decisions that use a global histogram will

be wrong for the bucket where the fleet actually spends its p99 (Chen, 2026a, 2020). The artifact manifest should reference per-bucket histograms so a schedule can be justified against the bucket it serves (Liu *et al.*, 2024; Davies *et al.*, 2025). Per-bucket routing data turns MoE compilation from a load-balancing guess into an evidence-based placement (Chen, 2026a; Vellaisamy *et al.*, 2026).

23.3 kv hardware

KV hardware adaptation splits responsibility: frameworks page and schedule KV blocks; compilers choose layout, fusion around KV edges, and on-chip residency for attention/MLP seams (Kwon *et al.*, 2023; Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022). Illegal fusion that assumes dense KV tensors breaks PagedAttention invariants (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Hardware-specific KV tactics:

- **GPU:** block-sparse attention tiles, TMA-friendly KV loads, graph-safe KV pointer stability (NVIDIA, 2022; SemiAnalysis, 2024; Dao *et al.*, 2022; NVIDIA, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; AMD, 2024b).
- **CPU:** cache-line blocked KV, NUMA-aware pinning for long context (Goodfellow *et al.*, 2016; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).
- **NPU/XDNA:** static KV slot tables, bounded context buckets compiled upfront (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024).

Context-length buckets matter: a schedule legal at $\text{ctx}=2048$ may spill at $\text{ctx}=8192$ when KV exceeds shared memory or LLC (Chen, 2026a; Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b,b). Compile separate artifacts per bucket rather than one dynamic kernel that silently allocates (NVIDIA, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Multi-query and grouped-query attention change KV head layout; KV hardware passes must read head counts from model metadata, not hard-coded MHA (Kwon *et al.*, 2023; Dao *et al.*, 2022; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Wrong layout passes compile yet corrupt generation (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Prefix caching and long system prompts shift KV working set: compilers should not assume empty cache at step zero (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Profile cells include warm prefix lengths representative of production (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Example 23.3.1. Paged KV + fused MLP: compiler must treat KV pointers as opaque handles through attention–MLP seam—materializing paged KV to dense breaks the framework contract (Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

23.3.1 KV ownership boundary

The cleanest specialization rule is that the framework owns KV block allocation and paging policy while the compiler owns layout and residency around the KV edges (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020). The IR should represent paged KV as opaque handles with a block-table contract, not as dense tensors the compiler may reshape freely (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2026a). Any pass that needs the dense view must insert an explicit gather whose byte cost is counted, and must prove the gather is legal under the framework’s page state (Chen, 2020; Davies *et al.*, 2025). Ownership boundaries prevent the classic failure where the compiler improves the kernel by violating the serving contract (Kwon *et al.*, 2023; Chen, 2026a). The boundary also tells on-call engineers which team owns a KV regression: page capacity belongs to serving, layout and fusion belong to the compiler (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

23.3.2 Bucket strategy for growing KV

KV grows with context, so a single compiled artifact cannot be optimal or even legal at every length (Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026). The compiler should emit per-bucket artifacts whose on-chip staging, page walks, and fusion boundaries were chosen for that bucket’s KV footprint (Dao *et al.*, 2022; Chen, 2020). Bucket boundaries should be derived from on-chip capacity thresholds, not round numbers alone, because the legal schedule changes where shared memory or LLC overflows (SemiAnalysis, 2024; Chen, 2026a). The runtime selects the artifact for the current context and logs the selection so telemetry can show bucket transitions (Kwon *et al.*, 2023; Davies *et al.*, 2025). Artifacts at the longest buckets should receive extra profiling because spills concentrate there (Chen, 2026a; Vellaisamy *et al.*, 2026). A bucket strategy turns KV growth from a runtime hazard into a compile-time plan (Chen, 2020; Davies *et al.*, 2025).

23.3.3 GQA, MQA, and MLA layout passes

Grouped-query and multi-head latent attention change the KV layout problem in ways a compiler must derive from model metadata rather than hard-coded assumptions (Kwon *et al.*, 2023; Dao *et al.*, 2022; Liu *et al.*, 2024). Head counts, group sizes, latent projections, and block scales determine which layout minimizes bytes and whether head-interleaving helps (Davies *et al.*, 2025; Chen, 2026a, 2020). Layout passes that assume dense MHA will produce correct-looking kernels that

corrupt generation for GQA or MLA models (Kwon *et al.*, 2023; Davies *et al.*, 2025). The layout derivation should be a named pass with unit tests per attention variant and per target (Chen, 2020, 2026a). When a model card adds a new variant, the layout tests gate the artifact before any performance claim (Davies *et al.*, 2025; Chen, 2026a).

23.3.4 KV compression and scale layout

Quantized and compressed KV caches add scale and metadata tensors whose layout must participate in the same passes as the cache itself (Liu *et al.*, 2024; Davies *et al.*, 2025; SemiAnalysis, 2024). Per-block scales, group offsets, and latent projections change the byte sheet and the legal fusion boundaries around attention (Chen, 2026a; Dao *et al.*, 2022). The compiler must keep dequant and scale logic inside the fused attention region where possible, and must count any separate scale pass as bytes moved (Chen, 2020; Davies *et al.*, 2025). Precision rules apply: softmax and accumulator paths need higher precision even when KV bytes are reduced (Dao *et al.*, 2022; Chen, 2026a). KV compression is a residency and bytes optimization that only pays when its layout and numerics are compiled together with the attention kernel (Liu *et al.*, 2024; Chen, 2026a).

23.3.5 Write and read asymmetry

KV cache hardware adaptation must respect the asymmetry between writes and reads (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a). Each new token writes one KV pair, then every subsequent token reads the full prefix, so layouts that optimize read bandwidth may pay for writes once and layouts that optimize writes may tax every read (Dao *et al.*, 2023; Chen, 2026a). The compiler should make the trade explicit by counting both directions in the byte sheet (Chen, 2020; Vellaisamy *et al.*, 2026). At long context, read bandwidth dominates and attention tile shapes should follow the read pattern; at short context, write cost is proportionally larger (Davies *et al.*, 2025; Chen, 2026a). A per-bucket layout decision that tracks this asymmetry is the KV analog of the prefill-decode split (Kwon *et al.*, 2023; Chen, 2026a).

23.3.6 Prefix-aware compile cells

Production KV caches are rarely empty at step zero, so compile and profile cells must include warm prefixes (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). The warm-prefix length should match the product’s typical prompt

distribution, because attention over a long system prompt has different residency and latency behavior than a cold start (Chen, 2026a; Davies *et al.*, 2025). Cells that only measure empty caches optimize the wrong operating point and miss prefix-cache interactions (Chen, 2020, 2026a). The compile cell matrix should therefore list cold, short-prefix, and long-prefix rows with the same counters (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Prefix-aware cells are how decode compilation survives real serving workloads (Kwon *et al.*, 2023; Chen, 2026a).

23.3.7 Attention input tiling

KV layout and attention tiling must be chosen together because the tile shape determines how block-table pages map onto on-chip memory (Dao *et al.*, 2022; SemiAnalysis, 2024; Chen, 2026a). A tile that spans several physical pages changes the page-walk cost; a tile aligned to page boundaries keeps the walk regular (Kwon *et al.*, 2023; Chen, 2026a). The compiler should derive tile dimensions from the page size, head dimension, and on-chip budget, and should expose the derivation in the compile report (Chen, 2020; Davies *et al.*, 2025). When the framework changes page size, the tiles must be re-derived and the artifact re-gated (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Attention input tiling is where KV hardware adaptation becomes a concrete compiler decision with measurable byte impact (Chen, 2026a; Dao *et al.*, 2022).

23.3.8 Graph-safe KV pointer handling

Captured decode graphs need KV pointers that are stable across replay or a graph-update mechanism that the compiler and runtime both understand (NVIDIA, 2024; Kwon *et al.*, 2023; Chen, 2026a). The compiler should distinguish buffers that are fixed for the capture lifetime from buffers that advance per token, and generate the update path accordingly (Davies *et al.*, 2025; Chen, 2020). Paged KV adds a second indirection: the page table itself may move as blocks allocate, so the capture contract must pin or update both the data pointers and the table (Kwon *et al.*, 2023; Chen, 2026a). A graph that silently falls back to eager when KV state changes is a p99 regression hiding in plain sight (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Graph-safe KV pointer handling is a compile-time and runtime contract that must be tested together (NVIDIA, 2024; Chen, 2026a).

23.3.9 Page-size metadata contract

The framework’s page size is an input to compiler layout, so the artifact should record it as part of the model card and reject mismatches at load (Kwon *et al.*, 2023; Chen, 2020, 2026a). Changing page size changes tile alignment, block-walk costs, and bucket legality, so serving and compiler must agree on the value through the artifact manifest (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). A mismatch between the compiled page assumption and the runtime policy is a correctness and performance hazard that only manifests at long context (Chen, 2026a; Davies *et al.*, 2025). The page-size contract is the smallest concrete example of the KV ownership boundary in action (Kwon *et al.*, 2023; Chen, 2020). It is enforced by the same manifest hash that guards every other decode artifact, which keeps the framework and compiler from drifting apart without a failing build, and it makes page-size drift a build failure rather than a long-context incident (Chen, 2020, 2026a).

23.4 hetero moe

Heterogeneous MoE load balancing spans *compile-time* expert placement and *runtime* routing policy (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2020; Chen and YiRage Contributors, 2026; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Expert parallelism across GPU tiers (prefill pool vs decode pool) fails when compiled expert weights assume uniform NVLink topology (Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Compiler stacks should emit *expert placement metadata*: which experts are colocated, which tiers use INT4/FP8 kernels, and which backends are legal per chip (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Runtime load balancers (eplb-class policies in open DeepSeek infra) consume that metadata; compilers should not hard-code routing (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023;

NVIDIA, 2024, 2022).

On CPU/NPU fallback paths, subset experts may run on host while GPU serves hot experts—compilation must quarantine backends per expert, not share one schedule (AMD, 2024b,a; Goodfellow *et al.*, 2016; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Hetero MoE gates in Chapter 22 apply per tier: bytes/token and launches/token on each participating SKU (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Fleet skew—one expert dominating activations—is a *routing* problem first, but compilation can mitigate via duplicated hot-expert kernels on multiple chips when policy allows (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Measure load imbalance with production histograms; synthetic uniform routing hides tail latency (Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Chapter 24 extends heterogeneity to cluster deployment; Chapter 28 clarifies framework vs compiler KV ownership (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Specialized LLM/MoE compilation succeeds when it respects those boundaries (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

23.4.1 Expert placement metadata schema

Heterogeneous MoE compilation should emit placement metadata that the runtime consumes rather than embedding routing policy in compiled kernels (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2020). The schema records expert identity, target chip, precision, backend legality, co-residency constraints, and the byte cost of moving that expert to another chip (AMD, 2024b; Chen and YiRage Contributors,

2026; Chen, 2026a). Load balancers use this metadata to route tokens while the compiler uses it to pick schedules (Liu *et al.*, 2024; Davies *et al.*, 2025). The schema must be versioned with the model card so a placement change is traceable (Chen, 2020; Vellaisamy *et al.*, 2026). When compiler and runtime disagree about where an expert runs, the metadata is the single source of truth for the postmortem (Davies *et al.*, 2025; Chen, 2026a).

23.4.2 Per-tier fallback quarantine

Mixed hardware deployments need per-tier quarantine so a backend that cannot run an expert never receives it silently (AMD, 2024b,a; Chen, 2020). The compiler should emit a legality list per expert and per backend, and the runtime should consult it before dispatch (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026a). A fallback to CPU or NPU is acceptable only when it is intentional, logged, and measured with that tier’s counters (Goodfellow *et al.*, 2016; AMD, 2024a; Vellaisamy *et al.*, 2026). Silent fallback is how heterogeneous fleets discover that a GPU-only schedule was shipped to an NPU cell (AMD, 2024b; Davies *et al.*, 2025). Quarantine lists turn backend mismatch from a runtime surprise into a compile-time and dispatch-time check (Chen, 2020, 2026a).

23.4.3 Skew mitigation through duplication

When production histograms show a few experts dominating activation, duplication can reduce the routing tail (Liu *et al.*, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). The compiler’s role is to compile duplicated copies of hot experts for the chips that will host them and to record the duplication in placement metadata (Chen, 2020, 2026a). Duplication is only worthwhile when the load imbalance exceeds the memory cost of the extra copies (Davies *et al.*, 2025; Chen, 2026a). The decision should be revisited when histograms change, because a traffic shift can make yesterday’s duplication wasteful (Liu *et al.*, 2024; Chen, 2020). Skew mitigation is a joint routing and compilation decision, and the compiler half is providing cheap, correct duplicate artifacts (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

23.4.4 Fleet-level gates

Heterogeneous MoE changes must pass gates on every participating tier, not only on the tier where the change was developed (Davies *et al.*, 2025; AMD, 2024b; Chen, 2026a). The gate matrix from Chapter 22 extends with columns for expert

dispatch cost, load imbalance, and fallback rate per tier (Chen, 2020; Vellaisamy *et al.*, 2026). A placement change that improves the GPU tier but raises CPU fallback should fail unless the fallback is intentional and within budget (AMD, 2024a; Davies *et al.*, 2025). Fleet gates also verify that the placement metadata the runtime consumes matches the artifacts the compiler emitted (Chen, 2020, 2026a). Cross-tier gate reports should be published with the same schema as single-tier reports so comparisons stay readable (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

23.4.5 Placement simulation

Expert placement across heterogeneous tiers should be simulated before it is deployed, because the routing distribution and the network topology interact in ways intuition misses (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2026a). The simulation replays production routing histograms over candidate placements and reports per-tier expert latency, cross-tier transfer bytes, and imbalance (Chen, 2020; Vellaisamy *et al.*, 2026). Compilation supplies the byte and kernel costs per candidate; the simulator combines them with routing (Liu *et al.*, 2024; Davies *et al.*, 2025). A placement that looks balanced on average can still produce tails when a hot expert sits on a slow tier (Chen, 2026a; Davies *et al.*, 2025). Placement simulation is the cheapest place to test duplication, fallback, and load-balancer policies before fleet rollout (Chen, 2020; Vellaisamy *et al.*, 2026).

23.4.6 Topology-aware metadata

Placement metadata must include topology facts: which chips share NVLink or coherent memory, which pairs pay PCIe or network transfer costs, and which tiers are in different failure domains (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2020). Expert movement costs depend on these facts, and the compiler cannot choose sane schedules without them (Chen, 2026a; Vellaisamy *et al.*, 2026). Topology metadata belongs in the artifact manifest and should be regenerated when the fleet layout changes (Davies *et al.*, 2025; Chen, 2026a). A placement computed for one cluster shape silently degrades when the cluster is rebuilt without the metadata update (Chen, 2020; Davies *et al.*, 2025). Topology-aware metadata is what makes heterogeneous MoE compilation reproducible across fleet changes (Liu *et al.*, 2024; Chen, 2026a).

23.4.7 Placement rollback

Heterogeneous placement changes need the same rollback discipline as kernel changes (Davies *et al.*, 2025; Chen, 2026a, 2020). The previous placement metadata and the artifacts it references must be retained so a load-imbalance regression can be reverted in one dispatch-policy update (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). Rollback should be testable at the dispatch layer without redeploying expert kernels, because the placement metadata and the kernels are separate artifacts (Chen, 2020; Davies *et al.*, 2025). When a rollback occurs, the postmortem compares the placement metadata diff to explain the imbalance shift (Liu *et al.*, 2024; Chen, 2026a). Placement rollback turns heterogeneous MoE tuning into an iterative operation instead of a risky one-way change (Davies *et al.*, 2025; Chen, 2026a).

23.4.8 Telemetry mapping for placement

Heterogeneous MoE telemetry should map every request to the placement metadata version and the dispatch decision that served it (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2026a). That mapping lets a load-imbalance regression be traced to a routing-policy change, a placement change, or a traffic shift (Chen, 2020; Vellaisamy *et al.*, 2026). Per-expert latency and transfer bytes should be tagged with the expert’s target tier and fallback status (Davies *et al.*, 2025; Chen, 2026a). When the fleet scales, the telemetry reveals whether the placement still matches the topology (Liu *et al.*, 2024; Chen, 2026a). Placement telemetry turns heterogeneous MoE from a compile-time bet into a measured, maintainable system (Chen, 2020; Davies *et al.*, 2025).

23.5 Key Takeaways

- **Own the decoder slice at batch-1.** LLM decode compilation begins when the compiler owns a whole decoder slice—attention, MLP, norms, rotary, residual—exported at batch-1 with explicit KV shapes, and gates it on whole-step bytes/token rather than per-operator FLOPs (Kwon *et al.*, 2023; Davies *et al.*, 2025). Optimizing the slice as a unit while respecting the framework’s paging is what keeps the attention-MLP seam resident (Dao *et al.*, 2022; Chen, 2026a).
- **MoE compilation meets dynamic routing.** The hard part of MoE scheduling is data-dependent routing: template the experts so they share a kernel, fuse the router prologue only when legal, and profile launches/token

under realistic routing entropy rather than a uniform-load proxy (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). A schedule that looks fine on balanced routing can thrash when real traffic skews load across experts (Davies *et al.*, 2025; Kwon *et al.*, 2023).

- **KV is a split responsibility.** Frameworks page and schedule KV blocks; the compiler chooses layout, fuses around the KV edges, and secures on-chip residency for the attention/MLP seam—and that split is realized differently per GPU, CPU, and NPU (Kwon *et al.*, 2023; AMD, 2024b). Treating KV layout as a compiler transform rather than a runtime afterthought is what makes the paged attention fast instead of merely correct (Dao *et al.*, 2022; SemiAnalysis, 2024).
- **Heterogeneous MoE spans compile-time and runtime.** Load balancing splits across compile-time expert placement and runtime routing policy, so the compiler emits placement metadata and per-tier gates while the runtime routes, and backends are quarantined across fallback paths (Liu *et al.*, 2024; Davies *et al.*, 2025). Without per-tier gates a placement tuned for one accelerator silently regresses another (Chen and YiRage Contributors, 2026; AMD, 2024a).
- **MoE and KV specialization still defer to the decode counters.** However specialized the compilation, the acceptance metric is unchanged: bytes/token and launches/token at the batch-1 decode operating point, not peak FLOPs (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). MoE’s launch density and KV’s bandwidth pressure are precisely the terms those counters expose (Chen, 2026a; Goodfellow *et al.*, 2016).

Practical decision rules. Six rules summarize LLM and MoE specialization for a working compiler team. First, tag every decode operator with a KV or routing role and keep those tags through lowering, because untagged fusion breaks serving contracts (Kwon *et al.*, 2023; Chen, 2020). Second, inject dynamic-sequence, paging, and head metadata before lowering and bind the artifact to the model card (Davies *et al.*, 2025; Chen, 2026a). Third, compile expert templates once per shape bucket and group tokens at runtime, because launch reduction in MoE comes from grouping, not from faster single experts (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). Fourth, treat KV layout, bucket strategy, and attention tiling as compile-time decisions derived from the framework’s page contract (Kwon *et al.*, 2023; Chen, 2026a). Fifth, emit placement metadata and topology facts for heterogeneous fleets and let the runtime route against them (Liu *et al.*, 2024; Davies *et al.*, 2025). Sixth,

gate every artifact on whole-step bytes/token and launches/token at the served buckets with realistic routing and warm prefixes (Chen, 2026a; Davies *et al.*, 2025). Teams that apply these rules get specialized compilation that survives serving reality; teams that skip them get kernels that are fast in the lab and broken in the fleet (Vellaisamy *et al.*, 2026; Chen, 2026a).

23.6 Conclusion

LLM and MoE specialized compilation narrows generic compiler stacks to decode-shaped graphs, routed experts, and KV contracts frameworks own (Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Gate merges on bytes/token, launches/token, and paging legality—not expert GEMM peaks alone (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Chapter 24 carries heterogeneity to cluster scale; Chapter 25 automates search inside the same invariants (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Chapter 24

Heterogeneous Compilation and Multi-Hardware Deployment

Production LLM serving is no longer a single-GPU problem; it is a fleet problem, where prefill and decode run on different pools, edge and datacenter run different models, and the same logical workload must compile for GPUs, CPUs, and NPUs that have nothing in common but the math (Davies *et al.*, 2025; IREE / iree-org contributors, 2024). This chapter is where every per-target decision from the MegaKernel and compiler chapters becomes a deployment-level routing decision across a heterogeneous **cluster**. The decode metrics still govern: dense Llama-3.2-1B issues 847.5 launches per token on H100 and OLMoE-1B/7B issues 9,305, and removing host overhead bought 1.259 $\times$ on a Qwen-2.5-7B cell—gains that must now be preserved across a mix of silicon, not a single SKU (Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024). The mistake to avoid is deploying one hand-tuned artifact everywhere. A kernel optimal on an H100 is illegal on an XDNA tile and wasteful on a CPU, so heterogeneous deployment demands per-target compilation with a unified front end and a runtime that adapts to the hardware it finds (AMD, 2024b; Rotem *et al.*, 2018).

24.1 Heterogeneous cluster

A **heterogeneous cluster** exists because prefill and decode have opposite hardware appetites. Prefill is compute-bound and batches well, so it maps to dense GPU pools; decode is memory- and orchestration-bound at batch-1, so it is provisioned differently and far more heavily (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).

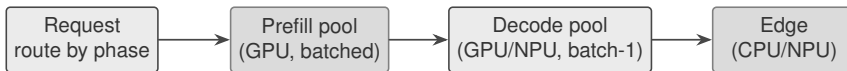


Figure 24.1: A heterogeneous serving cluster routes work by phase and target: batched prefill, batch-1 decode, and edge each map to different silicon.

Table 24.1: Roles in a heterogeneous serving cluster; each target owns a slice of traffic with a preferred compiled artifact (Davies *et al.*, 2025; Mirage Project contributors, 2025).

Hardware	Typical role	Preferred artifact
Datacenter GPU	High-throughput decode	Searched megakernel
CPU	Cost-sensitive / low-QPS	AOT bundle, cache-blocked
NPU / edge	Power-constrained	AOT bundle, static tiling

DeepSeek-V3’s production layout is the canonical evidence: its minimum prefill unit spans 4 nodes (32 GPUs) while its minimum decode unit spans 40 nodes (320 GPUs), a roughly 10× capacity skew that reflects how differently the two phases use silicon (Davies *et al.*, 2025). Disaggregating them into separate pools lets each be optimized and scaled independently, which is the cluster-level form of the prefill/decode split this book has drawn since Chapter 1.

The **heterogeneous** dimension deepens when the **cluster** mixes vendor silicon. A fleet may run datacenter GPUs for high-throughput decode, CPUs for low-QPS or cost-sensitive endpoints, and NPUs for edge or power-constrained deployment, each with the distinct residency and synchronization models of Chapters 6–9 (AMD, 2024b,a; Rotem *et al.*, 2018). The deployment decision is which target each slice of traffic routes to, and the answer is workload-specific: a high-reuse decode layer justifies a searched GPU megakernel, while an edge endpoint wants a Glow-style AOT bundle (Mirage Project contributors, 2025; Rotem *et al.*, 2018). Routing across this **cluster** is the deployment-level analogue of the per-region compiler choice of Chapter 21.

The reason routing beats standardizing is the same one this book has argued since Chapter 1: the optimal schedule is hardware-specific, so a single artifact is either illegal or leaves performance on the table somewhere in the **cluster** (AMD, 2024b; SemiAnalysis, 2024). A **heterogeneous** deployment embraces this by compiling per target and routing each request to the silicon whose strengths match the work, rather than forcing a lowest-common-denominator kernel across the fleet. The routing policy itself becomes an artifact to maintain—which traffic class goes to which pool—and it shifts as models and hardware evolve, which is why encoding the rationale rather than hand-wiring the routes matters at fleet

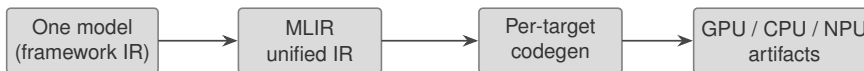


Figure 24.2: A unified MLIR-based stack lowers one model description to per-target artifacts, sharing front-end optimizations across hardware.

scale (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

24.2 Unified stack

Heterogeneous deployment is only tractable with a **unified** compiler front end, and **mlir** has become the de facto substrate for one. By providing a common, extensible IR that many backends share, **mlir** lets a single model description carry through shared high-level optimizations before forking into per-target codegen, rather than maintaining a separate compiler per vendor (IREE / iree-org contributors, 2024; Lai *et al.*, 2023). IREE is the concrete instance: it is an **mlir**-native compiler and runtime that lowers one input through its dialect stack to llvm-cpu, CUDA, ROCm, Vulkan, Metal, and experimental AMD AIE, giving a genuinely **unified** path across the matrix (IREE / iree-org contributors, 2024). TVM’s Relax and Triton’s TritonGPU dialect are further evidence that the field is converging on **mlir**-style staged lowering as the way to span hardware (Lai *et al.*, 2023; Tillet *et al.*, 2019).

The value of a **unified** stack for decode specifically is that the residency, role, pipeline, and synchronization passes of Chapters 6–9 can be written once against the shared IR and specialized per target, rather than reimplemented per backend (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). This is what makes the per-target guarantees of those chapters scalable across a fleet: the same memory pass checks a 228 KB H100 budget and a 64 KB XDNA tile, the same sync pass proves deadlock-freedom on both, because they operate on one **unified** representation (NVIDIA, 2022; AMD, 2024a). Without a **unified** stack, heterogeneous deployment degenerates into a matrix of separately maintained, separately verified kernels.

It is worth being clear about what **mlir** does and does not solve. It provides the shared infrastructure—a common IR, a pass framework, reusable dialects—so that backends do not each reinvent the lowering machinery, and it lets cross-level optimizations be expressed once (Lai *et al.*, 2023; IREE / iree-org contributors, 2024). What it does not do is make the per-target decisions for free: a residency budget, a role split, and a pipeline depth still differ per device, and **mlir** only

ensures those decisions are made on a common substrate rather than in incompatible silos. The **unified** stack is therefore a force multiplier for the per-target passes, not a replacement for them—it is what lets one team maintain those passes across a growing matrix instead of one team per vendor (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024).

The convergence on **mlir** across TVM (Relax), Triton (TritonGPU), and IREE is itself a signal that the industry has accepted heterogeneity as permanent (Lai *et al.*, 2023; Tillet *et al.*, 2019; IREE / iree-org contributors, 2024). Rather than betting on one winning architecture, these stacks bet on a **unified** compiler layer that can absorb whatever silicon arrives, which is the only durable response to a hardware landscape that adds a new accelerator class every year (AMD, 2024a; Chen *et al.*, 2018b).

24.3 Deployment contract

The useful unit of deployment is not a model file and not a container image. It is a **deployment contract**: a small, reviewable record that says which graph regions may run on which hardware, which artifact realizes each region, what runtime feature selects it, and which measurement can reject it. Treating that record as first-class is how a team prevents the familiar accident in which a GPU-only optimization is silently inherited by a CPU fallback, or an edge build is accepted even though it materializes an intermediate that the datacenter path keeps resident. The compiler has already made most of these choices by the time a request arrives; the contract makes those choices visible to the serving and release systems (IREE / iree-org contributors, 2024; Chen and YiRage Contributors, 2026; Rotem *et al.*, 2018).

For a GPU target, the contract should name the relevant compute capability, memory budget, supported asynchronous-copy or matrix instructions, allowed precision modes, and the kernel variant selected for each important shape. For a CPU target, it should instead name the ISA and vector-width assumptions, thread and NUMA policy, cache blocking assumptions, and the reference fallback. An NPU or XDNA target needs its tile-memory budget, static schedule or DMA constraints, supported operator set, and quantized layout. These are not cosmetic metadata. They are the conditions under which a compiled schedule is legal: a tile that fits in an H100 shared-memory budget need not fit on an AIE tile, and a loop ordering that is sensible for a GPU warp does not automatically expose useful SIMD reuse on a CPU (NVIDIA, 2022; AMD, 2024a,b).

The front end keeps the contract coherent by describing an operation in terms of

layouts, shapes, precision, and dependencies before it commits to a device-specific implementation. A graph split then draws the boundary between regions with different natural homes. The usual split is not “GPU versus CPU” in the abstract. It is a data-movement decision: keep a dense, high-reuse prefill region near the GPU tensor cores; retain a cache-friendly control or low-QPS path on the CPU when the transfer would dominate; emit a static NPU region only when its layout and shape constraints are satisfied. An MLIR-based stack can preserve those requirements through lowering and expose a target-specific legality failure before code generation, instead of discovering it as an opaque runtime failure (IREE / iree-org contributors, 2024; Lai *et al.*, 2023; Chen and YiRage Contributors, 2026).

This changes the release question. “Does the model run?” is a smoke test. “Does every admitted traffic class have an artifact whose assumptions match the detected hardware and whose fallback preserves correctness?” is a deployment test. The latter is what lets the same model description serve a datacenter GPU pool, a CPU pool, and an edge accelerator without reducing all three to their least capable common denominator (IREE / iree-org contributors, 2024; Rotem *et al.*, 2018).

24.3.1 A graph-splitting decision

Graph splitting deserves an explicit rule because it is where a unified compiler can become an unhelpful abstraction. Start from the request path, not from the set of available devices. Mark the tensors that cross a proposed device boundary, their byte size, their lifetime, and whether the next consumer can use the same layout. A boundary is attractive only when the work it moves gains more from the target’s compute, memory, or power characteristics than it loses in transfer, synchronization, and an additional materialized representation. For batch-1 decode, that is a demanding test: a tiny control operation may look cheap to offload, yet its synchronization can sit on the critical token path (Chen, 2026a; Vellaisamy *et al.*, 2026).

Consider a layer that has a GPU-resident KV cache and a CPU-side policy decision. Moving the policy computation is reasonable only if it consumes a compact summary and returns a compact decision while leaving the KV cache in place. Moving the KV transform itself merely to “use all available hardware” is usually a warning sign, because the boundary exports a large, latency-sensitive state only to import it again. The same logic applies on an NPU. An NPU can be excellent for a static, power-constrained subgraph, but it is a poor fit for a region that depends on a dynamic layout, a device-resident cache, or a fallback that would reconstruct the state elsewhere (AMD, 2024a; Rotem *et al.*, 2018; Chen, 2026a).

The compiler should therefore annotate a split with its reason: capacity, power, supported instruction, residency preservation, or operational isolation. Those reasons are more valuable than a bare placement label. When a future hardware generation changes the balance, an engineer can re-evaluate the relevant assumption instead of reverse-engineering why a graph happened to be cut at a particular node. YiRage’s per-target metadata is useful in this role because it associates a backend choice with the layout, residency, and legality constraints from which the choice followed (Chen and YiRage Contributors, 2026).

24.3.2 Artifact manifests, not opaque binaries

Every deployable artifact should have a manifest alongside it. At minimum, the manifest identifies the model and compiler revision, the target triple or device family, required runtime features, input-shape and precision envelope, kernel or executable entry points, and the benchmark record that admitted it. A multi-target IREE `.vmfb` can package host instructions and device kernels in one portable container, but portability does not remove the need to record which contained variant is expected on which device (IREE / iree-org contributors, 2024). The manifest is the bridge between the compiler’s internal decision and an operator’s rollout decision.

The shape envelope is particularly important. A specialization may be correct for all inputs but profitable only for a narrow set of aligned dimensions or context lengths. The runtime needs permission to dispatch it only inside that envelope, and it needs a known-safe alternative outside it. Without that distinction, “fat binary” turns into an unreviewable collection of variants that appears portable until a rare request lands on the slow or unsupported path. Recording the envelope also makes performance incidents diagnosable: the operator can determine whether a tail-latency regression came from the selected hardware, an unexpected shape, or a fallback rather than guessing from aggregate throughput (IREE / iree-org contributors, 2024; Chen, 2026a).

A manifest should also distinguish correctness fallback from service fallback. A correctness fallback is an implementation that returns the right result on an admitted device when the fast kernel is unavailable. A service fallback is a routing action—for example, sending a request from an unavailable edge target to a compatible remote pool—that may change latency or cost while preserving the user’s request. The compiler owns the first contract; the runtime and scheduler own the second. Keeping them separate avoids falsely claiming that a CPU kernel offers the same service-level behavior as a nearby GPU simply because both execute the graph (Davies *et al.*, 2025; IREE / iree-org contributors, 2024).

24.4 Runtime adaptation

A **unified** build still has to meet the specific device it runs on, which is the job of **runtime** adaptation. The cleanest model is the ahead-of-time fat binary: IREE compiles a FlatBuffers `.vmfb` that carries host instructions and device kernels for multiple targets, and at load time the **runtime** probes the device and dispatches the matching kernel, so one artifact serves a heterogeneous fleet without a JIT (IREE / iree-org contributors, 2024). The **runtime**’s ability to **detect** the hardware—its vendor, its memory sizes, its feature set—is what lets it pick the right kernel variant, and the fat binary can carry several specializations (aligned vs unaligned shapes, large vs small matmuls) for the **runtime** to choose among (IREE / iree-org contributors, 2024).

The **detect**-and-adapt pattern also covers graceful fallback. When a preferred path is unavailable—an NPU backend that is experimental, or a GPU feature the device lacks—the **runtime** must fall back to a kernel that runs, even if slower, rather than fail (IREE / iree-org contributors, 2024; Rotem *et al.*, 2018). For decode this matters because the residency-optimal path differs per device, so the **runtime** adaptation is not cosmetic: detecting an H100 versus a smaller GPU should select a different pipeline depth and tile size, the very parameters Chapters 6–8 tuned per target (NVIDIA, 2022; Chen, 2026a). Runtime adaptation is thus the deployment-time completion of compile-time per-target specialization. The division of labor is clean: the compiler decides everything it can ahead of time, per target, and the **runtime** decides only what genuinely depends on the device it lands on. For a heterogeneous fleet this keeps the per-request path thin while still letting one binary serve many targets, which is the property that makes large-scale deployment manageable rather than a per-device build explosion (IREE / iree-org contributors, 2024; Davies *et al.*, 2025).

There is a tension between adaptation and predictability that a serving team must manage. The more the **runtime** adapts—probing the device, selecting among kernel variants, falling back when needed—the more paths exist that must be tested, and the harder it is to guarantee a specific latency on a specific device (IREE / iree-org contributors, 2024; Davies *et al.*, 2025). The AOT fat binary mitigates this by resolving most choices at build time and leaving the **runtime** only a fast **detect**-and-dispatch, so the per-request path is short and predictable, in contrast to a JIT that recompiles on first use (IREE / iree-org contributors, 2024; Rotem *et al.*, 2018). For interactive decode, where tail latency is the product metric, this argues for front-loading adaptation into the build and keeping the **runtime** decision to a quick device **detect**, not a recompilation.

24.5 Ownership and observability

Heterogeneous deployment fails when a decision has no owner. The compiler team can produce a legal GPU, CPU, and NPU artifact; the runtime team can detect a device; the serving team can route traffic; and yet the combined system can still copy a KV cache across a boundary, select a generic fallback, or admit an unsupported shape. The remedy is to assign each decision to the layer that has the information to make it, then export enough evidence that the next layer can verify it. A unified stack reduces duplicated code, but it does not eliminate this division of responsibility (IREE / iree-org contributors, 2024; Chen and YiRage Contributors, 2026).

The compiler owns graph semantics and artifact legality. It decides which operations can fuse, which layouts and precisions an executable expects, which device features it requires, and which specializations are worth emitting. It should reject a target combination whose memory, tile, ISA, or synchronization requirements cannot be met, rather than leave the runtime to discover the mismatch under live traffic. This is the compiler’s version of mechanical sympathy: encode the hardware constraint close to the lowering that relies on it (NVIDIA, 2022; AMD, 2024a; Chen and YiRage Contributors, 2026).

The runtime owns selection among admitted artifacts. Given the detected device and the request shape, it chooses an executable whose manifest covers both. Its job is deliberately narrower than re-optimizing the model: it validates feature and shape predicates, binds buffers, performs a bounded dispatch, and reports the chosen path. If it falls back, it must say so in a form the serving layer can act on. A runtime that silently picks “something compatible” can preserve correctness while concealing the loss of a searched kernel, a residency property, or a latency target (IREE / iree-org contributors, 2024; Rotem *et al.*, 2018).

The serving scheduler owns traffic-class placement. It knows the SLO, queue state, tenancy policy, locality requirements, and cost envelope that the compiler cannot see in an individual graph. It may choose a GPU pool for interactive decode and a cheaper CPU or edge path for background work, but it should only select from the runtime’s admitted choices. Asking the scheduler to infer kernel legality from a device name is as fragile as asking a compiler to infer a tenant’s latency promise from an IR operation (Davies *et al.*, 2025; Kwon *et al.*, 2023). The separation is healthy precisely because each layer has a different source of truth.

24.5.1 The observability record

Every request need not emit a verbose compiler trace, but every rollout needs enough observability to reconstruct the route of a representative request. The minimum useful record joins five identities: the request traffic class, the detected hardware class, the artifact-manifest revision, the selected executable or fallback, and the measurement window used to judge it. With those identities, an engineer can distinguish three very different incidents that otherwise all appear as “latency got worse”: the scheduler sent a request to a different pool, the runtime selected a different variant, or the same variant regressed after a compiler or driver change (IREE / iree-org contributors, 2024; Davies *et al.*, 2025).

The record should also retain the shape and precision category rather than every raw tensor dimension. A searched megakernel may be correct across a broad shape range but only intended for a subset; an NPU bundle may be intentionally static; a CPU variant may depend on a vector-friendly alignment. Grouping requests by the manifest’s declared envelope makes these changes visible without exposing request content. It lets the fleet answer a practical question during an incident: did the distribution of work leave the benchmarked envelope, or did a path inside the envelope regress? (Chen, 2026a; IREE / iree-org contributors, 2024).

This is where end-to-end goodput metrics complement conventional runtime counters. Queue depth, error rate, and device utilization can show that a pool is under stress, but they cannot by themselves show whether a new artifact has increased materialized movement or launch work per generated token. Record bytes/token and launches/token for the selected path alongside latency measurements, then compare them within the same traffic and shape category. The result is an observability model that can connect a fleet symptom back to a compiler-level cause rather than merely reporting that a device is busy (Chen, 2026a; Vellaisamy *et al.*, 2026).

24.5.2 An escalation path

When a target regresses, start with the manifest rather than the source code. Confirm that the expected hardware, driver, model revision, compiler revision, and shape envelope were actually selected. If any identity differs, the problem is a routing or admission issue; return it to the runtime or scheduler owner with the recorded route. If the identities match but bytes/token or launches/token changed, inspect the lowering, fusion, residency, and dispatch changes in the compiler artifact. If those metrics match but tail latency changed, the serving owner should inspect contention, queueing, and fleet placement. This sequence

avoids assigning a queueing problem to a kernel team or a backend-legality problem to an SRE (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026).

The same escalation path makes a heterogeneous fleet easier to evolve. Adding a new accelerator becomes an explicit admission exercise: define its hardware contract, compile and validate its artifacts, register its runtime detection and fallback behavior, add its rows to the regression matrix, and authorize traffic classes only after the measurements exist. Nothing in that process requires a new ad-hoc deployment model. The shared IR and manifests provide continuity, while the target-specific evidence prevents a portability claim from becoming a performance promise (IREE / iree-org contributors, 2024; Lai *et al.*, 2023; AMD, 2024a).

This is the practical meaning of multi-hardware ownership: share the model and the front-end reasoning, but localize every hardware-specific assumption, decision, and measurement. It keeps the GPU path free to use a residency-aware megakernel, gives the CPU path a deliberate cache and SIMD strategy, and lets the NPU path remain static where static scheduling is its advantage. The cluster is unified by the contract and the evidence, not by pretending its devices are interchangeable (Chen and YiRage Contributors, 2026; Rotem *et al.*, 2018; AMD, 2024b).

24.6 Version regression

Deploying across many targets multiplies the surface for **regression**, so a heterogeneous fleet needs disciplined **version** management and automated **regression** testing. A kernel **version** that improves decode on one GPU generation can regress on another, and a model or compiler **version** bump can silently change which fusion is chosen, so every target-**version** pair must be benchmarked on the same decode metrics rather than assumed equivalent (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). This is the deployment-scale form of the book’s measurement discipline: track bytes/token and launches/token per target-**version**, and gate a rollout on no **regression** across the matrix.

The autobooks-style loop that produced this book is itself a model for **regression** control: a fixed harness scores every change, improvements are kept and **regressions** reverted via version control, and a results log records each experiment (Chen and YiRage Contributors, 2026). Applied to deployment, the same pattern is a **regression** matrix—each target a row, each kernel or model **version** a column, each cell a decode benchmark—that must stay green before a **version** ships. Reproducibility is the prerequisite: pinned configurations, recorded inputs,

and logged measurements, the transparency standard the fact-verification gate of this book applies to its own numbers (Chen, 2026a; Goodfellow *et al.*, 2016).

The combinatorics are what make automation essential here. With several hardware targets, multiple model **versions**, and a range of decode shapes (context lengths, batch sizes), the **regression** matrix has too many cells to check by hand, so the testing must be scripted and run on every change (Davies *et al.*, 2025). The autobooks loop’s keep/revert discipline scales to this directly: a change is kept only if it improves or holds the metric across the matrix, and reverted otherwise, with the results log providing an audit trail of which **version** was good on which target (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). Crucially, the metric must be the decode goodput metric—bytes/token and launches/token—not a microbenchmark, because a **version** that wins an isolated GEMM can still regress end-to-end latency, the recurring trap of this book (Chen, 2026a).

24.7 Rollout worksheet

A regression matrix tells a team whether an artifact is safe to promote; a rollout worksheet explains how to promote it without turning a good benchmark into a fleet incident. The worksheet begins with a traffic class, because the same artifact can be a good choice for a throughput-oriented prefill service and a bad choice for interactive decode. Record the SLO in terms the request path experiences, then record the corresponding compiler metrics: bytes moved per output token, launches or dispatches per token, and the relevant residency or synchronization condition. This keeps the release review grounded in end-to-end goodput instead of a single operator score (Chen, 2026a; Vellaisamy *et al.*, 2026).

Next, fill in one row for each admitted target. The row names the device family and runtime driver, the artifact-manifest revision, the shapes exercised, the selected kernel path, and the fallback path. It also names the owner of the hardware assumption. This sounds bureaucratic until a driver update, a compiler pass change, or a model revision alters one of those assumptions. At that point the row tells the team exactly which cells require a rerun. A target matrix is not a claim that all targets perform identically; it is an explicit inventory of the conditions under which each target is allowed to serve traffic (IREE / iree-org contributors, 2024; Davies *et al.*, 2025).

The worksheet should ask four questions in this order:

1. **Is the path legal on the detected hardware?** Verify feature requirements, memory or tile budget, precision support, and graph-partition constraints

before measuring speed. A faster schedule that spills, relies on an absent instruction, or crosses an unsupported runtime boundary is not a candidate (NVIDIA, 2022; AMD, 2024a; Chen and YiRage Contributors, 2026).

2. Does the selected artifact preserve the intended data residency?

Trace the inputs, KV state, temporary activations, and outputs through the split. If an intermediate is newly materialized or copied between devices on each token, treat it as a design change and re-measure the full request path (Chen, 2026a; Kwon *et al.*, 2023).

3. Does the measured path meet the traffic class’s objective?

Compare the same context lengths and batch regime used in production, rather than borrowing a prefill-throughput result for batch-1 decode. This is where bytes/token and launches/token make a useful common language across CPU, GPU, and NPU rows (Vellaisamy *et al.*, 2026; Chen, 2026a).

4. What happens when the fast path is absent?

Exercise device detection, unsupported shapes, missing features, and the service-routing alternative. A fallback that is never tested is a second implementation with no release evidence (IREE / iree-org contributors, 2024; Rotem *et al.*, 2018).

This ordering is deliberate. It prevents a team from spending days optimizing an artifact that cannot be safely admitted, and it stops a good GPU result from masking a broken CPU or NPU path. It also offers a practical way to decide when to stop specializing. If a target row cannot show a residency-preserving path and a traffic class that benefits from it, the honest answer may be to route that class elsewhere. Broad portability is valuable, but it is not an obligation to force every request through every accelerator (Davies *et al.*, 2025; Rotem *et al.*, 2018).

24.7.1 Canary and rollback semantics

Promotion should be monotonic in evidence. First compile and validate the artifact in isolation. Then run the full target matrix against the pinned model, runtime, and driver configuration. Then admit a narrow traffic class whose request shapes are represented in that matrix. Only after that can the scheduler widen the route. At every stage, retain the previous manifest and route as a reversible choice. The important property is not that every rollout is slow; it is that the rollback target is known, compatible with the current state, and comparable in the same metrics (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

For a GPU canary, observe whether the searched megakernel remains on its intended shape and residency path. For a CPU canary, observe whether the intended vector and cache-blocked path is selected rather than a generic scalar fallback. For an NPU canary, observe that the static bundle accepts the layout and does not divert a dynamic case into an unmeasured bridge. These are different implementation details, but their release semantics are identical: identify the selected artifact, compare it with the recorded matrix cell, and stop expansion when the selected path or the request envelope differs (IREE / iree-org contributors, 2024; AMD, 2024a; Rotem *et al.*, 2018).

Rollback should restore both the artifact and the routing decision. Reverting only the binary while leaving a scheduler route pointed at a device whose assumptions no longer hold can be as harmful as leaving the faulty binary in place. Conversely, an application-level routing rollback can protect users while compiler engineers investigate a target-specific regression. This separation lets a fleet remain available without pretending that a performance regression has been fixed merely because traffic was hidden from it (Davies *et al.*, 2025; Chen, 2026a).

24.7.2 Failure modes worth rehearsing

The first failure mode is a *false-unified* build: the front end is shared, but each backend has quietly acquired divergent graph semantics, precision behavior, or fallback rules. Its symptom is that a model passes a generic test yet produces a different execution path on a particular target. Counter it with target-specific correctness tests and manifests that tie an artifact to the lowering and runtime features it requires (IREE / iree-org contributors, 2024; Lai *et al.*, 2023).

The second is a *fallback cliff*. The runtime correctly detects that a fast path is unavailable, but the fallback changes the data path so much that latency, power, or cost violates the traffic class’s objective. A CPU fallback for a low-QPS endpoint can be acceptable; the same fallback on an interactive decode pool may be an incident. The contract must therefore make the fallback observable and give the scheduler a separate route when the service objective cannot be preserved (Davies *et al.*, 2025; Rotem *et al.*, 2018).

The third is a *benchmark mirage*: a new kernel improves a local operation while adding copies, launch overhead, or synchronization to the request path. The cure is not another isolated benchmark. It is a regression cell that measures the complete decode path in the representative batch regime and records the hardware and software configuration that produced the result (Vellaisamy *et al.*, 2026; Chen, 2026a). This is why the matrix and the manifest belong together: the former

provides the measured outcome, while the latter explains which executable and assumptions produced it.

Finally, rehearse an artifact mismatch. Deliberately attempt to load a GPU artifact on an unsupported feature set, send an out-of-envelope shape, and remove the preferred NPU path. The expected result is not necessarily identical performance; it is a bounded, observable transition to the known correctness or service fallback. A deployment team that has practised those transitions can make hardware heterogeneity a controlled routing decision rather than a source of surprise during a model or driver release (IREE / iree-org contributors, 2024; AMD, 2024a).

24.8 Industrial cases

The clearest industrial **case** is again disaggregated **production** serving. DeepSeek-V3’s separate prefill and decode pools, with the $10\times$ capacity skew, show a **production** system mapping each phase to the hardware footprint it needs rather than forcing both onto one uniform pool (Davies *et al.*, 2025). The lesson generalizes: a **production** deployment that measures its phases separately will provision them separately, because the decode pool’s bottleneck (memory and orchestration at batch-1) is different from the prefill pool’s (compute throughput) (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).

A second **case** class is the megakernel in **production**. Mirage Persistent Kernel compiles an LLM forward pass into a single megakernel and reports $1.2\text{--}6.7\times$ end-to-end latency reduction over kernel-per-operator serving, a **production**-relevant result precisely because it attacks the launch storm that dominates batch-1 decode (Mirage Project contributors, 2025; Chen, 2026a). A third is edge **production** on NPUs and CPUs, where Glow-style AOT bundles deploy quantized models within fixed memory budgets (Rotem *et al.*, 2018; AMD, 2024a). Each **case** is the right answer for its slice of the fleet, which is exactly why a heterogeneous deployment routes rather than standardizes.

A useful **production** pattern that ties the cases together is the cost-versus-latency frontier. A datacenter GPU pool delivers the lowest decode latency but at the highest cost per token; a CPU pool is cheaper per token at low utilization but cannot meet aggressive tail-latency targets; an edge NPU trades absolute throughput for power and locality (Davies *et al.*, 2025; Rotem *et al.*, 2018). A **production** deployment places each traffic class on the point of that frontier its service-level objective demands—interactive chat to the low-latency GPU pool, batch or background generation to cheaper hardware—and the heterogeneous

cluster is what makes that placement possible. The compiler’s role is to ensure that whichever point is chosen, the artifact for it is residency-correct and decode-optimal, so the frontier is defined by genuine hardware trade-offs rather than by which target happened to get a hand-tuned kernel (Chen and YiRage Contributors, 2026; Chen, 2026a).

24.9 YiRage cluster

The book’s reference compiler is the cross-**cluster** router that ties these threads together. **YiRage** compiles one model description to multiple backends—CUDA, CPU, Triton, and more—and carries the Chapter 2 residency rules as checkable IR metadata, so each target gets a verified, per-hardware artifact from a single source (Chen and YiRage Contributors, 2026). Across a heterogeneous **cluster** it makes the routing decision measurable: a high-reuse decode layer on a datacenter GPU routes to a searched megakernel, an edge endpoint to an AOT bundle, a broad-portability target to whole-program scheduling, each choice backed by the same bytes/token and launches/token metrics (Mirage Project contributors, 2025; IREE / iree-org contributors, 2024; Rotem *et al.*, 2018).

What makes **YiRage** a **cluster**-level tool rather than a single-kernel one is that the residency, role, pipeline, and sync passes of Chapters 6–9 all run per target from the **unified** representation, so deploying to a new device in the **cluster** is a recompile-and-verify, not a hand-port (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). The metadata that records why a target was routed a certain way also survives model and hardware changes, so the routing decision does not have to be re-litigated each release—the deployment-scale payoff of encoding intent rather than hand-tuning (Davies *et al.*, 2025; Chen, 2026a).

This closes the arc the book has traced. The residency budgets of Chapter 6, the static roles of Chapter 7, the pipelines of Chapter 8, and the synchronization of Chapter 9 were each presented as a per-target compiler pass; the compiler survey of Chapters 16–21 showed which existing stacks implement pieces of that vision; and this chapter assembles them into a fleet-level deployment where one model description compiles, via a **unified mlir** stack, to verified per-hardware artifacts that a **runtime** dispatches and a **regression** matrix guards (Chen and YiRage Contributors, 2026; IREE / iree-org contributors, 2024; Wu *et al.*, 2024). **YiRage** is the name this book gives to the compiler that does all of it together across a heterogeneous **cluster**, and the reason every earlier chapter measured against it is that it is where the book’s recurring thesis—decode is bound by movement and orchestration, fixable only by residency-aware, schedule-free, per-

target compilation—is operationalized at scale (Vellaisamy *et al.*, 2026; Mirage Project contributors, 2025).

24.10 Key Takeaways

- **Disaggregate by phase across the cluster.** Prefill is compute-bound and batches; decode is memory- and orchestration-bound at batch-1, so they provision differently—DeepSeek-V3 runs a $10\times$ larger decode pool than prefill pool (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Measure the two phases separately, and you will end up provisioning them separately (Kwon *et al.*, 2023).
- **Unify the front end with MLIR.** A shared MLIR-based IR (IREE, Relax, TritonGPU) lets one model carry shared optimizations before per-target codegen, so the residency/role/pipeline/sync passes are written once and specialized per target (IREE / iree-org contributors, 2024; Lai *et al.*, 2023). Without it, deployment becomes an unwieldy matrix of hand-maintained kernels (Chen and YiRage Contributors, 2026).
- **Adapt at runtime by detecting the device.** A fat binary carrying multiple target kernels lets the runtime probe the hardware and dispatch the right variant, with graceful fallback when a path is unavailable (IREE / iree-org contributors, 2024; Rotem *et al.*, 2018). Detected device sizes should select pipeline depth and tile size (NVIDIA, 2022).
- **Gate rollouts on a per-target regression matrix.** A kernel or model version that helps one target can regress another, so benchmark every target-version pair on bytes/token and launches/token and require no regression before shipping (Vellaisamy *et al.*, 2026; Chen, 2026a). Reproducibility (pinned configs, logged measurements) is the prerequisite (Goodfellow *et al.*, 2016).
- **Route, don’t standardize.** Each production case—searched megakernel on GPU ($1.2\text{--}6.7\times$), AOT bundle on edge, whole-program scheduling for breadth—owns a slice of the fleet, and YiRage routes per slice with verified per-target artifacts (Mirage Project contributors, 2025; Rotem *et al.*, 2018; Chen and YiRage Contributors, 2026). One artifact everywhere is the anti-pattern, because a kernel optimal on one target is illegal or wasteful on another (AMD, 2024b).

24.11 Conclusion

Heterogeneous deployment is where the book’s per-target discipline becomes a fleet-level routing problem: disaggregate prefill and decode, unify the front end with MLIR so the residency, role, pipeline, and synchronization passes run once and specialize per target, adapt at runtime by detecting the device, and gate every rollout on a per-target regression matrix (Davies *et al.*, 2025; IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026). The YiRage compiler is the cross-cluster router that makes this measurable, compiling one model to verified per-hardware artifacts and recording the routing intent as metadata so it survives change (Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025). Deployment, however, is increasingly something compilers do for themselves: Chapter 25 turns to auto-optimization and AI-assisted compilation, where the search and routing decisions of this book are themselves driven by learned and automated systems (Wu *et al.*, 2024; Chen, 2026a).

Chapter 25

Auto-Optimization and AI-Assisted Compilation

When a decode compiler exposes thousands of tile sizes, fusion boundaries, and backend flags, hand-tuning one schedule per SKU stops scaling—yet blind autotune that optimizes GEMM FLOPs alone still ships schedules that lose on bytes/token (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). Part VI established that each backend (XLA, TVM, Triton, MLIR) owns a different search grammar; this chapter asks how *auto-optimization* should search when the objective is decode goodput across GPU, CPU, and NPU fleets, not a single microbench winner (Chen, 2020; Chen and YiRage Contributors, 2026; AMD, 2024b; Goodfellow *et al.*, 2016).

TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). Memory-Floor CUDA Graphs on Qwen-2.5-7B (BS=1, ctx=2048) yields $1.259\times$ on H100 by removing host overhead—yet cannot fix illegal HBM traffic (Chen, 2026a; NVIDIA, 2024). An autotuner that rewards median kernel time without residency legality will pick launches that spill KV every token—exactly the failure mode Chapter 1 framed as “fast kernels, slow decode” (Davies *et al.*, 2025; Dao *et al.*, 2022; Kwon *et al.*, 2023).

Three mechanisms stack: **RL-guided search** over compiler knobs when the space is too large for exhaustive grid search; **hardware-aware reward functions** that score bytes/token, launches/token, and legality on the target tier; and **YiRage-style integrated search** that binds IR metadata, backend registry, and fingerprint caches so auto-optimization does not fork a second toolchain (Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024). The sections below treat each layer as a production contract—what to measure, what to forbid in the reward,

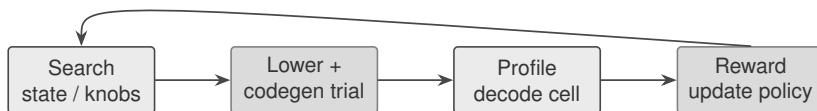


Figure 25.1: RL autotune loop: compiler knobs $\rightarrow$ trial schedule $\rightarrow$ decode-profiled reward $\rightarrow$ policy update (not peak GEMM alone).

and when AI-assisted compilation helps versus hides regressions (SemiAnalysis, 2024; AMD, 2024a; Goodfellow *et al.*, 2016).

This chapter sits at the end of Part VII for a reason: you cannot auto-tune what you cannot measure, and you cannot trust measurement that ignores residency (Chen, 2026a; Dao *et al.*, 2022; Davies *et al.*, 2025). Earlier chapters gave you backends and benchmarks; here the compiler team operationalizes *search* under decode SLOs (Chen, 2020; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023). Treat autotune artifacts—policies, fingerprints, profile scripts—with the same rigor as source code: versioned, reviewed, rollback-ready (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026a, 2020).

Anti-pattern catalog: (i) optimizing prefill-only benchmarks for decode fleets; (ii) single-number rewards that hide bytes/token debt; (iii) external autotuners that bypass compiler legality checks; (iv) promoting GPU winners to NPU fleets without re-profile (Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). The rest of the chapter turns each anti-pattern into a concrete gate (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; AMD, 2024a; Goodfellow *et al.*, 2016).

25.1 RL autotune

RL autotune enters when the Cartesian product of tile sizes, pipeline stages, and fusion predicates exceeds what grid search or genetic algorithms can explore before your release train departs (Chen, 2020; Goodfellow *et al.*, 2016). Classical TVM/Ansor-style autotuning treats the schedule as a structured program with measurable features; RL layers add a policy that proposes knob vectors conditioned on IR shape, target SKU, and prior trial outcomes (Chen, 2020; Chen and YiRage Contributors, 2026). The production mistake is training that policy on prefill

shapes or batch-32 training graphs while deployment runs BS=1 decode—the search converges to schedules that never touch launches/token (Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023).

Search spaces diverge by hardware class even when the math is identical. GPU trials must respect shared-memory caps, TMA legality, and CUDA Graph capture constraints (NVIDIA, 2022; SemiAnalysis, 2024; NVIDIA, 2024). CPU trials swap tile axes for cache lines and NUMA pinning; reward spikes from L3 residency can invert when KV grows past LLC (Goodfellow *et al.*, 2016; Chen, 2020). NPU/XDNA trials are compile-time static: illegal dynamic buffers fail before profiling, so RL exploration needs a *legality oracle* ahead of expensive simulation (AMD, 2024b,a). One global search space imported from GPU codegen is how teams burn weeks on trials that never legalize on the fleet they ship (AMD, 2024b; Chen and YiRage Contributors, 2026).

Ansor and auto-scheduler literature established *program sampling*—mutate loop orders, split factors, and cache levels under measured cost models (Chen, 2020; Goodfellow *et al.*, 2016). RL replaces random mutation with a learned proposal distribution when cost models lie on decode shapes (thin MLPs, small batch GEMMs, attention with paged KV) (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen, 2026a; Davies *et al.*, 2025). The handoff from classical autotune to RL is not “smarter magic”—it is better exploration when analytic models miss orchestration (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022). Keep classical grid search as sanity check on small knob subsets (Chen, 2020, 2026a; Vellaisamy *et al.*, 2026).

Decode-aware RL should treat each trial as a *cell*: fixed context bucket, BS=1, steady-state after warmup, metrics logged as ms/token *and* bytes/token *and* launches/token (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Prefill-only rewards overweight arithmetic intensity; decode rewards expose orchestration cliffs when graph replay is absent (NVIDIA, 2024; Chen, 2026a). MoE and speculative paths add non-stationary branches—policies must either exclude those subgraphs from the MDP or include explicit state for routing entropy (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).

Operational guardrails: cap trial wall-clock per layer block; freeze RNG seeds for reproducible CI; store *fingerprinted* schedules keyed by IR hash + SKU + context bucket (Chen and YiRage Contributors, 2026; Chen, 2020). When RL proposes a schedule that improves GEMM but regresses bytes/token, the reward function failed—not the hardware (Chen, 2026a; Dao *et al.*, 2022; SemiAnalysis, 2024). YiRage encodes iron rules from Chapter 2 as checkable IR metadata so

Knob family	GPU decode focus	CPU / NPU delta
Tile / block	Shared mem, TMA stages	Cache line, static tile slots
Fusion depth	Graph capture legality	DMA chain length
Backend choice	Triton vs CUDA vs CUTLASS	oneDNN / AIE kernel gen
Objective	min launches/token	min bytes + DMA edges

Table 25.1: RL autotune search axes differ by target; shared math does not imply shared knob grammar.

illegal trials fail fast before GPU hours burn (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024).

Multi-hardware delta. Hopper RL may explore TMA double-buffer stages; XDNA policies must emit static tile assignments with bounded DMA depth; CPU policies optimize cache blocking with OpenMP gang sizes—identical transformer math, incomparable action spaces (SemiAnalysis, 2024; AMD, 2024b; Goodfellow *et al.*, 2016).

Example 25.1.1. Decode cell (BS=1, ctx=2048): an RL policy that reduces MLP tile width may cut shared memory but force an extra KV spill—reward must penalize bytes/token, not only kernel duration (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026).

Industrial teams keep a *human prior* schedule as RL baseline: never accept a policy update that fails A/B against the prior on all three counters (Davies *et al.*, 2025; Chen, 2020, 2026a). AI-assisted compilation is not autonomy—it is search acceleration with decode metrics as the constitution (Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016; AMD, 2024a).

Feature engineering for RL policies should mirror what Ansor-style cost models already encode—loop extent, reuse distance, parallel axis—but add decode-specific state: context length bucket, graph-capture eligibility, and whether KV is paged or dense (Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024). Without those features the policy collapses to “pick bigger tiles” because that wins microbench GEMMs on prefill-shaped tensors (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). Export the feature vector alongside every trial so postmortems can explain *why* a schedule won, not only that loss decreased (Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Exploration versus exploitation trade-offs show up in fleet rollout: ϵ -greedy trial injection can destabilize p99 when a bad schedule briefly serves traffic (Davies

et al., 2025; Kwon *et al.*, 2023). Canary pools per SKU—10% of decode replicas run candidate schedules with automatic rollback on SLO breach—keep RL useful without gambling the whole cluster (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2026a). Offline RL from logged profiles is safer when experimentation budget is thin; online RL requires hard legality masks (Chen, 2020; AMD, 2024a; Goodfellow *et al.*, 2016).

Part VI compiler chapters (XLA, TVM, Triton) each expose different knob surfaces; RL autotune should treat backend choice as part of the action space, not a pre-search constant (Chen, 2020; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024). A policy that only tunes Triton tile sizes while XLA would legalize a cheaper fusion is incomplete search (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; NVIDIA, 2022, 2024; Dao *et al.*, 2023). Document backend-specific action spaces in runbooks so ML and compiler engineers share vocabulary (Chen and YiRage Contributors, 2026; Chen, 2020).

Chapter 21 compiler benchmarks supply the harness RL loops need: frozen decode cells, pinned model weights, and agreed context buckets (Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a). Without that harness, RL trials become incomparable noise—one engineer profiles with speculative decoding disabled, another leaves it on (Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026). Wire benchmark IDs into trial logs so CI can replay the exact cell that produced each fingerprint (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024).

Sample-efficiency tricks from production: warm-start policies from grid-search winners; reuse trials across adjacent context buckets when IR shape is unchanged; early-terminate trials that violate legality or blow memory budgets before full profile (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; AMD, 2024b,a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022). RL autotune is as much about *discarding bad trials quickly* as about finding good ones (Chen, 2020; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022). Budget caps per layer prevent a single attention block from consuming an entire weekly GPU pool (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*,



Figure 25.2: Hardware-aware reward: profile trace $\rightarrow$ buffer residency $\rightarrow$ scalar score gated by target legality.

2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024).

When RL policies overfit to a single SKU stepping, transfer learning across related chips (same architecture generation, different memory width) can seed exploration—but always re-profile bytes/token on the destination SKU (SemiAnalysis, 2024; AMD, 2024b; NVIDIA, 2022; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; AMD, 2024a; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024). Never promote transferred schedules without decode-cell A/B (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024). RL autotune without transfer discipline recreates the “works on my H100” culture Part I warned against (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024).

25.2 HW reward

The production surprise at **hardware-aware reward design** is not peak FLOPs; it is optimizing the wrong counter (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). A reward of “minimize kernel latency” on Hopper selects schedules that win Nsight Compute yet lose decode when activations round-trip through HBM every token (Chen, 2026a; Dao *et al.*, 2022; SemiAnalysis, 2024). A reward of “maximize occupancy” encourages register-heavy tiles that shrink fusion windows and raise launches/token (Vellaisamy *et al.*, 2026; NVIDIA, 2022). Hardware-aware rewards start from tier accounting: what bytes moved between register file, shared memory, L2, and HBM/DDR for one decode step (SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Chen, 2020).

Composite rewards work best as constrained scalars, not single numbers. A

practical template:

$$R = -\alpha \cdot \text{ms/token} - \beta \cdot \text{bytes/token} - \gamma \cdot \text{launches/token} - \lambda \cdot \mathbb{I}[\text{illegal}] \quad (25.1)$$

with SKU-specific weights: on bandwidth-saturated cards raise β ; on launch-storm eager stacks raise γ (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Graph-capture-eligible regions add a legality bit—schedules that allocate inside captured DAGs receive $-\infty$ reward regardless of microbench speed (NVIDIA, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

Trace residency before trusting R . Reward producers (attention, MLP, norm) must feed consumers without a DDR round trip (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). Silent spill reads as “mysterious” p99 regression when mean ms/token looks flat (Davies *et al.*, 2025; SemiAnalysis, 2024). Diff lowered IR or CUDA for new global allocations; count DMA edges on NPU trials like launches on GPU (AMD, 2024a,b; Chen, 2020).

Nsight Compute and roofline charts remain the auditor’s tools—not the reward itself (SemiAnalysis, 2024; Davies *et al.*, 2025; Chen, 2026a). Translate profiler counters into bytes moved per decode step before they enter R ; otherwise teams overfit to SM occupancy (NVIDIA, 2022; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024b,a; Dao *et al.*, 2022, 2023; NVIDIA, 2024; Chen, 2026a). Chapter 1’s bytes/token counter is the bridge between profiler vocabulary and autotune objectives (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Graph-mode decode adds reward terms for capture legality: schedules that introduce allocations inside captured regions should fail regardless of ms/token (NVIDIA, 2024; Chen, 2026a; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). MegaKernel trials add fusion-depth terms—reward schedules that reunify MLP and residual even when graph replay already fixed launches (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; AMD, 2024a; Goodfellow *et al.*, 2016). Mode-aware rewards prevent autotune from fighting Chapter 12 execution-mode decisions (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2022; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023).

Review gate. Attach roofline or buffer accounting for the target SKU before merging any autotune winner (Chen, 2026a; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024). Rewards must be reproducible from checked-in profile scripts—not from a engineer’s laptop Nsight session (Davies *et al.*, 2025; Chen, 2020).

CPU rewards emphasize LLC residency and cross-socket traffic; mis-pinning a decode thread can dominate ms/token even when GEMM kernels are “optimal” (Goodfellow *et al.*, 2016; Chen, 2020). NPU rewards emphasize compile-time static legality and DMA depth; a schedule with perfect FLOPs but one extra DDR hop fails fleet power caps (AMD, 2024b,a). Treat *same reward formula* across targets as an anti-pattern—share methodology, not weights (Chen and YiRage Contributors, 2026; AMD, 2024a; Goodfellow *et al.*, 2016).

Example 25.2.1. Two trials tie on ms/token at BS=1; trial A lowers bytes/token by 12% with one fewer launch—hardware-aware ranking picks A even if trial B wins a GEMM roofline chart (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Penalty terms matter as much as primary metrics. Add soft penalties for register spill events, uncoalesced global loads on KV tensors, and host-side synchronizations inserted by overly conservative codegen (SemiAnalysis, 2024; NVIDIA, 2022; Davies *et al.*, 2025). On CPU, penalize cross-NUMA migrations; on NPU, penalize DDR round trips that static analysis could have folded (Goodfellow *et al.*, 2016; AMD, 2024b,a; Chen, 2020). These penalties keep RL from “winning” on ms/token while hiding residency debt that explodes at ctx=8192 (Chen, 2026a; Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

Reward hacking appears when engineers reuse the same profile script but change warmup: steady-state ms/token improves while cold-start SLO regresses (Chen, 2026a; NVIDIA, 2024; Davies *et al.*, 2025). Lock warmup protocol in CI—fixed bucket sequence, primed graph caches, identical sampling flags—before comparing rewards (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; AMD, 2024a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022). Fregly-style goodput analysis always states which phase is measured (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2026a).

Multi-objective Pareto fronts beat single scalars when leadership must choose between 5% bytes/token and 2% launches/token (Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024). Publish Pareto sets per SKU; let product owners pick operating points—compiler teams should not hide trade-offs inside opaque

R (Davies *et al.*, 2025; Chen, 2020; Chen and YiRage Contributors, 2026; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Hardware-aware reward design is governance, not just math (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; AMD, 2024a; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Observability closes the loop: every trial should emit structured logs—knob vector, legality outcome, the three decode counters, profiler deep link, and git SHA of compiler stack (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Dashboards that only show “best ms/token so far” invite reward hacking; dashboards that plot bytes/token and launches/token alongside expose orchestration regressions early (Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). On-call engineers promoting autotune winners need the same artifacts SREs use for binary rollbacks (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026a, 2020; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024).

Stability testing belongs in the reward pipeline: run promoted schedules across context buckets {512, 2048, 8192}, cold and warm starts, and at least two driver versions (Chen, 2026a; NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2022; Liu *et al.*, 2024). A schedule that wins at ctx=2048 but fragments at ctx=8192 should fail promotion even if mean reward improved (Chen, 2026a; Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2023; NVIDIA, 2024, 2022; Liu *et al.*, 2024). Hardware-aware rewards without stability gates recreate the prefill/decode split Chapter 1 documented (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024).

25.3 YiRage auto

YiRage auto-optimization is where search meets the runtime stack from Chapters 2 and 29: ‘superoptimize’ over multi-backend IR, ‘HardwareRegistry’ for SKU detection, and fingerprint caches that reuse proven schedules without rerunning full RL episodes (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023). The failure mode is bolting an external autotuner onto PyTorch exports—two sources of truth for fusion legality, diverging cache keys, and CI that cannot reproduce which schedule shipped (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

YiRage binds decode iron rules into IR metadata before search starts: residency predicates, same-backend constraints, and fallback paths when a trial illegalizes on the detected chip (Chen and YiRage Contributors, 2026; Liu *et al.*, 2024; AMD, 2024b). Search proposes knob vectors; lowering validates legality; profiling writes rewards back to the fingerprint store keyed by IR hash, backend, and context bucket (Chen, 2020, 2026a). That closed loop is how AI-assisted compilation stays *auditable*—every winning schedule traces to a profile artifact, not a black-box policy (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

Cross-hardware practice: maintain one decode cell suite per fleet tier (Hopper H100, MI300, XDNA NPU, CPU reference) and require non-regression on bytes/token before promoting a fingerprint (AMD, 2024b; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Chen, 2020). GPU-first winners that spill on NPU should remain quarantined in backend-specific namespaces—shared YAML knobs are convenient and dangerous (AMD, 2024a; Chen and YiRage Contributors, 2026; AMD, 2024b). When ‘superoptimize’ exhausts its budget, fall back to human-prior schedules from Part VI compiler chapters rather than shipping under-measured RL noise (Chen, 2020, 2026a; Vellaisamy *et al.*, 2026).

Integration with serving stacks: vLLM/SGLang own batching and KV paging; YiRage auto targets intra-layer fusion and PersistentKernel handoff—reward functions must not double-count framework-level wins (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). DeepSeek-class MoE adds routing entropy; auto-optimization should segment experts into separate search cells or rewards absorb load imbalance (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). FlashAttention-class kernels shrink attention bytes; auto must still score MLP/residual bytes/token or reports lie (Dao *et al.*, 2022, 2023; Chen, 2026a).

‘HardwareRegistry’ and same-backend rules from YiRage docs prevent silent backend mismatch: a fingerprint compiled for CUDA must not replay on a CPU fallback without re-search (Chen and YiRage Contributors, 2026; Chen, 2020;

AMD, 2024b,a; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Search budgets (‘superoptimize’ iteration caps) should scale with layer criticality—attention—MLP seams first, sampling tails last (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022). PersistentKernel runtime from Chapter 29 consumes fingerprints; auto-optimization is wasted if runtime cannot load them idempotently (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; AMD, 2024b; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Multi-hardware delta. CUDA backend search explores Triton/CUTLASS tiles; CPU backend search targets oneDNN primitives; MLIR/NKI paths enforce static XDNA schedules—‘HardwareRegistry’ selects grammar, not just clock speed (Chen and YiRage Contributors, 2026; AMD, 2024b,a; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Example 25.3.1. Fingerprint hit: IR hash + H100 + ctx2048 reuses a prior ‘superoptimize’ result; miss triggers bounded RL search with decode reward—cold start latency includes search unless warm caches are primed in CI (Chen and YiRage Contributors, 2026; Chen, 2026a; Kwon *et al.*, 2023).

Production checklist: (1) legality oracle before profile; (2) composite decode reward with SKU weights; (3) fingerprint store with IR/backend/bucket keys; (4) A/B against human prior on bytes/token and launches/token; (5) quarantine per-backend winners (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a). That is auto-optimization as engineering discipline—not “turn on AI and hope” (Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Liu *et al.*, 2024).

CI integration: store ‘superoptimize’ fingerprints as build artifacts keyed by git SHA and compiler version (Chen and YiRage Contributors, 2026; Chen, 2020). When LLVM or Triton bumps change legality, invalidate caches selectively rather than rerunning full RL from scratch (Chen, 2020; SemiAnalysis, 2024; AMD, 2024b). Nightly jobs replay decode cells on representative SKUs; drift alerts fire when bytes/token moves more than agreed epsilon (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; AMD, 2024a; Goodfellow *et al.*, 2016; NVIDIA, 2024, 2022; Dao *et al.*, 2023).

Human-in-the-loop remains mandatory for safety-critical deployments: RL proposes, engineers approve promotion, on-call runbooks list rollback fingerprints (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a). LLM-assisted *synthesis* of schedule hints—suggesting fusion candidates from IR text—can seed RL policies but must never bypass legality oracles (Chen and YiRage Contributors, 2026; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Chen, 2026a; Davies *et al.*, 2025; Chen, 2020). AI assists search; decode metrics retain veto power (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

Chapter 24 heterogeneous deployment assumes auto winners are tagged per backend; Chapter 26 packaging assumes fingerprint stores are reproducible in containers (Chen, 2020; Chen and YiRage Contributors, 2026; AMD, 2024a; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024; AMD, 2024b; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). YiRage auto closes the loop between Part VI compilers and Part VIII runtime co-design (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Operator playbook summary: detect chip via registry; load fingerprint or start bounded search; validate legality; profile decode cell; compare against human prior; canary; promote or rollback (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022; Liu *et al.*, 2024). Each step has an owner—compiler, runtime, SRE—so auto-optimization does not become a headless cron job (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024). Chapter 27 trends toward more autonomous search; this chapter insists autonomy remains subordinate to measured decode goodput (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024).

When external LLM tools suggest fusion patterns, treat output as *hints* fed

into the same legality and reward pipeline—never as merged schedules without profile (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022; Liu *et al.*, 2024). AI-assisted compilation accelerates hypothesis generation; hardware-aware rewards and YiRage integration retain veto (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024). That division of labor is how industrial teams adopt ML without surrendering residency discipline (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024).

25.4 Key Takeaways

- **RL autotune.** Use policies over compiler knobs when search spaces explode; train on decode cells (BS=1, bucketed ctx), not prefill FLOPs alone (Vellaisamy *et al.*, 2026; Chen, 2026a, 2020).
- **Hardware-aware reward.** Composite R over ms/token, bytes/token, launches/token, and legality—SKU-specific weights, shared methodology (Davies *et al.*, 2025; SemiAnalysis, 2024; AMD, 2024b).
- **YiRage auto.** Closed-loop ‘superoptimize’ + ‘HardwareRegistry’ + fingerprint caches; quarantine cross-backend winners (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024).
- **Measure useful decode work.** Prefer kernels/token and bytes/token over peak FLOPs when ranking autotune trials (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2026a).
- **Mechanical sympathy bounds the search.** Run legality oracles before spending GPU-hours so the autotuner never explores schedules that cannot fit the target’s residency budget (Dao *et al.*, 2022; Chen, 2020). Folding residency accounting into every reward keeps the search anchored to on-chip behavior rather than to a microbenchmark the policy can game (AMD, 2024a; Chen, 2026a).

25.5 Conclusion

Auto-optimization and AI-assisted compilation extend Part VI backends with search—but only decode metrics from Chapter 1 confer legitimacy (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). RL explores; hardware-aware rewards judge; YiRage integrates both without forking the toolchain (Chen and YiRage Contributors, 2026; Chen, 2020). Chapter 26 assumes these baselines when packaging production deployments; Chapter 28–30 assume fused schedules won autotune gates before framework/runtime co-design (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; AMD, 2024a). Gate merges on buffer accounting and reproducible profile artifacts—never on a single GEMM roofline screenshot (SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Liu *et al.*, 2024).

Chapter 26

Production Deployment and Engineering Best Practices

Shipping a decode compiler is not shipping a Git tag—it is shipping *reproducible artifacts* (fingerprints, containers, driver matrices) that replay the same bytes/token and launches/token counters your tuning loop proved in CI (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Chen and YiRage Contributors, 2026). Chapters 22–25 built profiling, gates, LLM/MoE specialization, and automated search; this chapter operationalizes **production deployment** across GPU, CPU, and NPU fleets without letting environment drift erase compiler wins (AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024).

TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). Memory-Floor CUDA Graphs on Qwen-2.5-7B (BS=1, ctx=2048) yields $1.259\times$ on H100 by removing host overhead—yet production still fails when packaging omits graph warmup or pins the wrong driver (Chen, 2026a; NVIDIA, 2024). Production engineering translates compile-time legality into *fleet-time* legality (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Four practices anchor the chapter: **environment packaging**, **multi-hardware operations**, **production pitfalls**, and **heterogeneous scheduling** (Davies *et al.*, 2025; Chen, 2020; Chen and YiRage Contributors, 2026; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Part VIII (Chapters 28–30) assumes these baselines when separating

framework, runtime, and compiler responsibilities (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Production maturity means the same decode cell run in CI is one command away in staging and prod (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). When that link breaks, “compiler regressions” are often packaging or scheduling mistakes (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Ownership RACI: compiler engineers sign fingerprint manifests; release engineering signs container promotion; SRE signs fleet config; framework owners sign batching invariants (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Blurred ownership produces dashboards nobody trusts (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Anti-patterns for production: promoting schedules without staging replay; mixing prefill and decode fingerprints; autoscaling on CPU utilization; silent eager fallback after driver upgrade (NVIDIA, 2024; Chen, 2026a; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Each section names the fix (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Compliance and audit trails: retain profile artifacts and packaging manifests long enough to explain quarter-over-quarter decode SLO shifts (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).



Figure 26.1: Environment packaging: CI compiler artifacts → immutable container → pinned driver matrix → staged fleet rollout.

Package layer	Must pin
Compiler stack	git SHA, pass flags, backend registry
Runtime deps	CUDA/cuDNN, ROCm, oneDNN, XDNA toolkit
Model assets	weights hash, tokenizer, paging policy
Profile harness	decode cell script + bucket list

Table 26.1: Production packaging layers for multi-hardware decode fleets.

Regulators and customers increasingly ask *why* latency changed—fingerprints answer that question (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

26.1 env packaging

Environment packaging binds compiler git SHA, LLVM/Triton/CUDA toolkit versions, and schedule fingerprints into deployable units (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). The anti-pattern is “works in CI” containers that omit warmup scripts, graph capture caches, or model-weight layout flags present in the tuning cell (Chen, 2026a; NVIDIA, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Multi-hardware fleets package *per-tier* images: Hopper decode, MI300 decode, XDNA edge, CPU fallback—shared YAML env vars are convenient and dangerous (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024,

2022). Rollout attaches SBOM + fingerprint manifest so on-call can diff a regressed pod in minutes (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Deploy and packaging pipelines should fail closed when fingerprint files are missing, checksums mismatch, or decode harness scripts differ from CI (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). “Best effort” deploy is how illegal graph capture reaches production (NVIDIA, 2024; Chen, 2026a; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Secrets and model weights belong in the same manifest chain as compiler artifacts—rotating weights without updating fingerprints is a common silent regression (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Package deploy hooks that re-run a smoke decode cell before accepting traffic (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Multi-hardware delta. GPU images ship CUDA Graph warmup jobs; NPU images ship static compile bundles; CPU images ship ORT/TorchInductor session plans—same model card, different immutable artifacts (NVIDIA, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Example 26.1.1. Deploy promotes fingerprint `fp-8f3a` only when CI decode cell at `ctx` ∈ {512, 2048, 8192} matches golden bytes/token within ϵ (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022).



Figure 26.2: Multi-hardware operations: SKU registry → runbooks → decode counter dashboards → fingerprint rollback.

26.2 multi HW ops

Multi-hardware operations keep production honest after deploy (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Dashboards must show bytes/token, launches/token, and ms/token *per tier*—aggregating GPU and NPU into one chart hides fallback paths (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Runbooks document: legal context buckets, graph capture warmup, paging limits, MoE routing flags, and rollback fingerprints (Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). On-call steps start with “diff packaging manifest” not “retune tiles” (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

SRE collaboration: compiler teams own fingerprint promotion; SRE owns fleet capacity and incident response; framework teams own batching—escalation paths must be explicit (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Review gate. Production promotion requires tier-specific decode-cell replay in staging that mirrors driver and power caps (Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

26.3 pitfalls

Production pitfalls recur across fleets (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022):

1. **Driver/toolkit skew** — CI green, prod slow; graph capture silently falls back to eager (NVIDIA, 2024; Chen, 2026a; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).
2. **Wrong context bucket** — schedule legal at ctx=2048 spills at ctx=8192 (Chen, 2026a; Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).
3. **Benchmark theater** — prefill FLOPs celebrated while decode bytes/token flatlines (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).
4. **Cross-tier promotion** — GPU fingerprint replayed on NPU without re-profile (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Postmortems should log counter deltas, packaging diffs, and routing histograms—not only “kernel X slower” (Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Lessons feed back into Chapter 22 gates and Chapter 25 search budgets (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Pitfall drills: game-day exercises that upgrade drivers, shrink memory, or skew routing—teams rehearse fingerprint rollback before incidents (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Lessons learned belong in runbooks, not slide decks (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Operation teams should track *packag* drift— undeclared env vars, sidecar injections, and mutating init scripts—as first-class incident causes (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Production operation metrics include deploy rollback rate, not only availability (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

26.4 scheduling

Heterogeneous scheduling places prefill pools, decode pools, MoE expert shards, and CPU/NPU fallbacks on distinct fleet slices (Davies *et al.*, 2025; Liu *et al.*, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016; Chen, 2026a; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Compiler packaging must tag which fingerprints are valid on which pool—schedulers should not load Hopper graphs onto bandwidth-starved SKUs (Chen, 2026a; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Kubernetes-style orchestration needs decode-aware autoscaling: scale on ms/token and queue depth, not CPU percent (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Prefill bursts

and decode steady-state belong on different node pools with different compiler artifacts (Davies *et al.*, 2025; Liu *et al.*, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

ClusterTopology-style metadata (Chapter 24) informs which compilers may fuse across NUMA/NVLink boundaries; scheduling respects those edges (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Canary pools per schedule fingerprint remain mandatory—fleet schedulers amplify mistakes faster than laptops (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Fleet schedul policies should expose which fingerprint each replica runs—debugging p99 without that label wastes hours (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Schedul changes (pool resize, spot preemption, NUMA rebinding) trigger mandatory decode-cell smoke tests when compiler residency is NUMA-sensitive (Goodfellow *et al.*, 2016; Chen, 2020; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Cost-aware fleet scheduling trades spot GPU pools against SLO headroom—compiler fingerprints validated on stable pools first, spot second (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Goodput per dollar still defers to bytes/token per dollar under decode load (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

26.5 Key Takeaways

- **Package the whole compile, not just the model.** A deployable unit must bind the compiler git SHA, the LLVM/Triton/CUDA toolkit versions, and the schedule fingerprints into one immutable image, so a rebuild reproduces the exact kernels (Chen, 2020; Chen and YiRage Contributors, 2026). Without that pinning, a driver or toolkit drift silently changes the emitted schedule and the decode numbers move for reasons no one can trace (Davies *et al.*, 2025; Chen, 2026a).
- **Operate multi-hardware with runbooks, dashboards, and rollback.** Production stays honest after deploy only with per-tier runbooks, counter dashboards tracking bytes/token and launches/token, and fingerprint-keyed rollback to a known-good schedule (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). The same model on GPU, CPU, and NPU has different failure modes, so the operational tooling must be tier-aware rather than one-size-fits-all (AMD, 2024b,a).
- **Know the recurring pitfalls.** Driver skew, shape-bucket mismatch, benchmark theater (microbench wins that never move ms/token), and cross-tier promotion of a schedule tuned for the wrong SKU recur across fleets (NVIDIA, 2024; Vellaisamy *et al.*, 2026). Each is invisible to a FLOPs dashboard and visible only to a decode-cell regression on the actual target (Chen, 2026a; Davies *et al.*, 2025).
- **Schedule prefill and decode as distinct fleet slices.** Heterogeneous scheduling places prefill pools, decode pools, MoE expert shards, and CPU/NPU fallbacks on separate slices, each with its own fingerprints and decode-aware autoscaling (Davies *et al.*, 2025; Kwon *et al.*, 2023). Forcing both phases onto one uniform pool wastes the 10 $\times$ capacity skew that disaggregation exists to exploit (Liu *et al.*, 2024; Chen, 2026a).
- **Hold SLOs to decode goodput, not peak FLOPs.** Production service-level objectives must track bytes/token and launches/token at the real operating point, because those are what predict ms/token and p99 (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). A green FLOPs or utilization dashboard that ignores them is the most common way a regression ships unnoticed (Chen, 2026a; Goodfellow *et al.*, 2016).

26.6 Conclusion

Production deployment and engineering best practices turn compiler wins into fleet contracts (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Package reproducibly, operate per tier, document pitfalls, and schedule heterogeneously—then Chapter 27 explores where silicon trends stress those contracts (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Part VIII implements framework–runtime–compiler co-design assuming these production gates held (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Gate merges on buffer accounting and reproducible decode cells—never on laptop demos alone (SemiAnalysis, 2024; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Chapter 27

Future Trends in AI Compilers

Trend chapters fail when they read like keynote slides—buzzwords without counters (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Part VII established production packaging and gates; this closing chapter projects **where AI compilers go next** while keeping Chapter 1’s decode metrics as the constitution: bytes/token, launches/token, and residency legality across GPU, CPU, and NPU fleets (Chen, 2020; Chen and YiRage Contributors, 2026; AMD, 2024b,a; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). Memory-Floor CUDA Graphs on Qwen-2.5-7B (BS=1, ctx=2048) yields 1.259× on H100 by removing host overhead—yet tomorrow’s silicon adds new memory tiers and routing paths that obsolete today’s fusion maps (Chen, 2026a; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024b,a). Trends matter only when they name *which counter moves* and *which gate breaks* (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Four trajectories close Part VII: **hardware–software co-design**, **new architecture adaptation**, **edge–cloud unified compilation**, and **autonomous kernel generation with legality gates** (Chen, 2020; Chen and YiRage Contributors, 2026; AMD, 2024b,a; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Goodfellow *et al.*, 2016). Part VIII (Chapters 28–30) implements the three-party contract among inference frameworks, YiRage runtime,



Figure 27.1: Co-design trend: silicon capabilities encoded as IR legality $\rightarrow$ compiler passes $\rightarrow$ decode-cell proof (not FLOPs alone).

and compiler co-design that these trends converge toward (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Trend chapters fail operationally when teams adopt silicon before IR contracts exist—the same failure mode as shipping eager fallbacks after graph capture (NVIDIA, 2024; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Each section below names a trend, the counter it should move, and the gate that must not break (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Anti-patterns for trend adoption: chasing peak TFLOPs without decode cells; porting GEMM first; treating autonomous kernels as exempt from legality; merging edge and cloud fingerprints (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). The fix is always the same methodology from Chapter 22—profile, gate, ship (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

27.1 codesign

Hardware–software co-design stops being a slogan when ISA and memory hierarchy changes ship with *compiler metadata contracts*—what can be fused, what must stay paged, which tiers exist (Chen, 2020; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024; AMD, 2024b,a; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Dao *et al.*,

Trend	Counter moved	Gate
Co-design	bytes/token residency	IR legality oracle
New arch	launches/token on port	decode-cell regression
Edge-cloud	ms/token incl. RTT	tier fingerprint isolation
Autonomous kernels	search throughput	legality before profile

Table 27.1: Future trends mapped to decode counters and non-negotiable gates.

2022, 2023; NVIDIA, 2024, 2022). Hopper-class TMA and warp specialization pushed fusion boundaries; the next co-design wave encodes those boundaries in IR so backends do not rediscover them per vendor (NVIDIA, 2022; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Co-design for *decode* differs from training: batch-1 residency, paging, and graph capture dominate; co-design documents must lead with bytes/token impact, not peak TFLOPs (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2023; NVIDIA, 2024, 2022). YiRage-style iron rules (Chapter 2) foreshadow vendor-neutral co-design: checkable predicates shipped alongside silicon (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Vendor roadmaps will keep adding memory tiers, expert routing, and capture modes—co-design is the process of turning those into compiler-visible facts before backends guess (SemiAnalysis, 2024; Liu *et al.*, 2024; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Multi-hardware delta. GPU co-design chases bandwidth and launch orchestration; NPU co-design chases static DMA depth; CPU co-design chases cache residency—shared methodology, incomparable constraints (AMD, 2024b,a; Goodfellow *et al.*, 2016; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).



Figure 27.2: New architecture trend: adapter passes bridge legacy IR to new codegen with arch-specific decode regression suites.

Example 27.1.1. New tier exposes faster on-chip buffer: co-design wins only if compiler IR marks which tensors may legally reside there at BS=1 decode (SemiAnalysis, 2024; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

27.2 new architectures

New architecture adaptation is the recurring tax every AI compiler team pays—MLIR dialects, backend forks, and HardwareRegistry entries—when silicon generations turn over (Chen, 2020; Chen and YiRage Contributors, 2026; AMD, 2024b,a; SemiAnalysis, 2024; NVIDIA, 2022; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024; Goodfellow *et al.*, 2016). The anti-pattern is “port GEMM first, decode later”; decode-first ports expose paging and graph legality early (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Trend: modular adapter passes per architecture generation, each gated by the same decode cells from Chapter 22 (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Architecture trend lines without regression suites become marketing, not engineering (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

MoE-heavy architectures (DeepSeek-class) accelerate the trend: expert routing and disaggregated pools become first-class IR concepts, not runtime hacks (Liu *et al.*,

2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Compiler teams should budget one adapter generation per silicon refresh—trying to stretch a Hopper-only fusion map across Blackwell and XDNA without adapter passes guarantees launch explosions (Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Review gate. New architecture support merges only when bytes/token and launches/token match or beat prior generation on agreed buckets (Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

27.3 edge cloud

Edge–cloud unified compilation targets one model artifact with tier-specific fingerprints—edge NPU static schedules, cloud GPU graph+capture, CPU fallback—without pretending one schedule fits all (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). The edge trend is longer context on device; the cloud trend is fleet goodput—unified IR must express both without lowest-common-denominator fusion (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Adaptive compilation at runtime (detect chip, select backend) is the operational face of this trend—HardwareRegistry patterns from YiRage generalize to edge fleets (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Kwon *et al.*, 2023; Davies *et al.*, 2025; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Cloud offload decisions (which layers run edge vs cloud) belong in policy metadata compiled into the same IR family (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis,

2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Privacy and bandwidth drive edge trends, but **decode counters** still arbitrate: edge wins only if ms/token and bytes/token beat cloud RTT-inclusive baselines for the target workload (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Unified edge–cloud compile pipelines version fingerprints per tier and forbid silent cross-tier promotion (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Fleet operators increasingly run hybrid graphs: prefill on cloud GPU, decode on edge NPU, or the reverse for privacy-sensitive prompts (AMD, 2024b,a; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Unified compilation must emit *split fingerprints*—not one schedule pretending both tiers share memory (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Example 27.3.1. Same LLM runs on XDNA edge and Hopper cloud: two fingerprint namespaces, one IR export, shared decode-cell methodology (AMD, 2024b; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

27.4 autonomous kernels

Autonomous kernel generation—LLM-assisted or search-driven—is the most hyped trend and the highest risk (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Chapter 25 placed RL and auto-optimization under decode rewards; the trend extends toward models proposing fusion patterns or kernel text (Chen and YiRage Contributors,

2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Non-negotiable guardrails for autonomous kernels:

1. **Legality oracle before profile** — same as human-authored schedules (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).
2. **Decode-cell reward** — bytes/token, launches/token, ms/token; no GEMM-only scores (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).
3. **Human prior + fingerprint rollback** — autonomy subordinate to production gates (Chapter 26) (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Autonomous *kernel* text without IR integration repeats CUDA string injection failures—trend winners compile through the same multi-backend stacks Part VI built (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). The autonomous trend succeeds when it accelerates search under existing gates, not when it bypasses them (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Search budgets for autonomous kernels should cap candidate evaluations per merge request—unbounded search is how illegal schedules reach main (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Chapter 25’s RL autotune patterns extend here: reward

decode goodput, penalize legality violations, require human-readable diffs (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

27.5 Key Takeaways

- **Co-design ships as compiler metadata contracts.** Hardware–software co-design stops being a slogan when an ISA or memory-hierarchy change arrives with a contract the compiler can read—what may be fused, what must stay paged, which tiers exist (Chen and YiRage Contributors, 2026; SemiAnalysis, 2024). Encoding silicon capabilities as IR legality and proving each on decode cells is what turns a new feature into a usable, checkable optimization rather than a press release (Chen, 2020; Davies *et al.*, 2025).
- **New architectures are a recurring, decode-first tax.** Every silicon generation forces adapter passes, backend forks, and hardware-registry entries, and the discipline that keeps the cost bounded is decode-first porting with architecture-specific regression (Chen, 2020; Chen and YiRage Contributors, 2026). Bringing a new target up on prefill alone hides exactly the residency and launch gaps that batch-1 decode will later expose (Chen, 2026a; Vellaisamy *et al.*, 2026).
- **Edge–cloud means one artifact, tier-specific fingerprints.** Unified compilation targets a single model with per-tier fingerprints—edge NPU static schedules, cloud GPU graph capture, CPU fallback—without pretending one schedule fits all (AMD, 2024b,a). Adaptive backend selection at runtime with no cross-tier promotion is what prevents a schedule tuned for one tier from silently regressing another (Davies *et al.*, 2025; Kwon *et al.*, 2023).
- **Autonomous kernels need legality, reward, and rollback.** LLM-assisted and search-driven kernel generation is the most hyped and highest-risk trend, because a generated kernel can be fast and wrong (Chen, 2020; Chen and YiRage Contributors, 2026). It is safe only inside a frame that verifies legality, rewards decode goodput rather than microbench latency, and can roll back—treating the generator as a hint source, not a bypass of the residency contract (Chen, 2026a; Davies *et al.*, 2025).
- **Part VIII makes these trends deployable.** The co-design, portability,

and automation trends above are not abstractions; framework, runtime, and compiler co-design is where they become a shipped serving stack (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023). The next part descends from trend to implementation, carrying the same decode-goodput yardstick (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

27.6 Conclusion

Future trends in AI compilers converge on one discipline: faster silicon and smarter automation must still answer to decode goodput (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Co-design, new architectures, edge-cloud unity, and autonomous kernels are directions—not excuses to drop bytes/token gates (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Part VIII opens with inference frameworks (Chapter 28), YiRage PersistentKernel runtime (Chapter 29), and three-party co-design (Chapter 30) (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Gate merges on buffer accounting and reproducible decode cells—the trend line starts where Chapter 1 started (SemiAnalysis, 2024; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Chapter 28

LLM Inference Frameworks and Serving Runtimes

Who owns decode goodput—the inference *framework* that batches requests and manages KV, or the *compiler/runtime* that fuses kernels and cuts bytes/token? The industry default splits responsibility badly: frameworks optimize scheduling and capacity; compilers optimize isolated GEMMs; neither measures the same counters on batch-1 decode (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026). Part VIII opens by naming what frameworks *must* keep (request lifecycle, paging, sampling policy) versus what YiRage and MegaKernel stacks can absorb (intra-step residency, launches/token) (Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). The chapter goal is operational clarity: every engineer can point to the decode step box on a diagram and attach counters (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Chapter 27 flagged codesign and autonomous-kernel trends; Part VIII opens by naming who owns each decode counter when those trends land in production (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025). TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100—often before framework-level batching helps at BS=1 (Vellaisamy *et al.*, 2026). vLLM-style serving reduces KV fragmentation and raises fleet utilization, but leaves operator boundaries inside the decoder step unless a compiler reunifies them (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026). Memory-Floor reports $1.259\times$ median speedup on H100 from CUDA Graph replay alone at BS=1, ctx=2048—orchestration relief that paging cannot replace

when bytes/token stay high inside the step (Chen, 2026a; NVIDIA, 2024). Those three citations anchor the split: launches, paging, and graphs solve different layers of the same decode story (Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; NVIDIA, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). This chapter maps framework layers to Chapter 1 counters so Part VI–VII compiler theory lands in a deployable stack, not a slide (Chen, 2020; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; AMD, 2024b,a).

Readers should leave with a deployment taxonomy: classify any production stack into framework-native, partial-export, or compiler-owned decode core (Table 28.1), then profile *both* scheduler queue time and in-step ms/token/bytes/token (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026). Chapter 29 descends into YiRage Layer 5 Persistent Kernel runtime; Chapter 30 stitches the three-party contract (Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Until then, treat every framework feature as a hypothesis that must survive BS=1 counter worksheets (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a). If you remember nothing else, remember the decode step box: everything inside is compiler and runtime turf; everything outside is framework turf (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Draw that box before every integration meeting and attach Chapter One counters to each side per decode token (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Fregly-style mechanical sympathy applies at the *boundary* between framework and compiler: the framework must not hide allocator behavior that forces HBM spill; the compiler must not assume contiguous KV or static batch shapes that break paging (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; NVIDIA, 2024; Goodfellow *et al.*, 2016). Part VI–VII taught pass ordering and dialect lowering; Part VIII asks where those passes *attach* in a live server (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). The answer is never “replace the framework” wholesale—it is to shrink the decode step until launches/token and bytes/token move on the same SKU you ship (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Dao *et al.*, 2022, 2023; NVIDIA, 2024; Goodfellow *et al.*, 2016).

This chapter also connects backward to Chapter 12 execution modes and Chap-



Figure 28.1: Framework landscape: requests enter scheduling and KV policy; bytes/token and launches/token are decided inside the decode step boundary.

ter 23 LLM compile specialization (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Eager versus graph versus MegaKernel is not only a training-time PyTorch concern—it is reproduced inside serving binaries through framework dispatch, CUDA graphs, and custom fused ops (NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). DeepSeek-scale stacks add expert parallel communication below the scheduler; open components in `deps/DeepEP` and `deps/FlashMLA` show kernels that must compose with paging and batching rather than bypass them (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Read framework documentation as architecture diagrams with counters attached, not as marketing feature lists (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

28.1 framework landscape

Framework landscape spans HuggingFace *eager* paths, vLLM/SGLang *continuous-batching* servers, TensorRT-LLM/ONNX Runtime *compiled* deployments, and custom C++ runtimes (Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2020; Chen and YiRage Contributors, 2026). Each makes different fusion depth assumptions: TRT-LLM may fuse within exported regions; HF PyTorch often does not; vLLM optimizes KV and batching while reusing framework attention backends (Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026). SGLang extends the same serving layer with structured generation and radix-cache reuse—still framework-side orchestration unless a compiler owns the decoder cell (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

Production stacks rarely pick a single label from the list above. A common pattern is HuggingFace weights loaded into a vLLM server with FlashAttention

backends and optional TensorRT plugins for narrow subgraphs (Kwon *et al.*, 2023; Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). The compiler engineer job is to diagram where the decode step begins and ends in that stack: from slot assignment and block table lookup through final hidden state before sampling (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Anything inside that window is fair game for fusion and residency optimization; anything outside is framework policy (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Ambiguity at the boundary is the main reason teams debate whether a regression is serving or compiler (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Compiler engineers should classify any deployment into three buckets (Table 28.1): (1) *framework-native* decode (Python loop + vendor libs); (2) *partial export* (compiled attention or MLP only); (3) *compiler-owned decode core* (YiRage PersistentKernel or MegaKernel inside a thin framework shell) (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Liu *et al.*, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Bucket choice determines whether your lever is batching policy or bytes/token (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026). Teams that pick bucket (1) for time-to-market but benchmark only fleet tokens/s will misattribute compiler regressions to “scheduling” when launches/token never moved (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025).

HuggingFace vs serving servers. HF `transformers` optimizes researcher ergonomics: dynamic shapes, easy checkpoint swap, and Python-visible tensors (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Production servers trade that flexibility for slot-based scheduling, pinned memory pools, and optional CUDA graphs (Kwon *et al.*, 2023; NVIDIA, 2024; Chen, 2026a). Compiler integration rarely starts in HF notebooks—it starts when a serving team exports a decode cell and asks which IR metadata (softmax carriers, page tables) survives the cut (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024).

Research notebooks prove model correctness; serving binaries prove goodput under load (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024b,a;

Table 28.1: Framework deployment buckets and where compiler/runtime leverage applies.

Bucket	Typical stack	Primary goodput lever
Framework-native	HF eager, vanilla PyTorch loop	Continuous batching, KV paging; launches/token often high (Vellaisamy <i>et al.</i> , 2026; Kwon <i>et al.</i> , 2023)
Partial export	vLLM + compiled attention; TRT-LLM sub-graph	Export boundaries; graph capture buckets (NVIDIA, 2024; Davies <i>et al.</i> , 2025)
Compiler-owned core	Thin shell + YiRage PK / MegaKernel	Fusion, residency, launch collapse inside step (Chen and YiRage Contributors, 2026; Chen, 2020)

Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Teams should freeze a serving configuration—block size, max batch, dtype—before comparing compiler exports (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Otherwise HF eager baselines become moving targets that invalidate A/B conclusions (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

TensorRT-LLM and compiled paths. TRT-LLM and similar engines push fusion into builder-time regions but still sit behind a framework or C++ server for batching and KV (Davies *et al.*, 2025; NVIDIA, 2024; Goodfellow *et al.*, 2016). Partial export wins when attention or MLP dominates bytes/token; it fails when export boundaries fall *inside* online-softmax state or paged KV gather/scatter (Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Chen, 2026a; Chen and YiRage Contributors, 2026). Profile export candidates on BS=1 decode, not only engine build logs (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2026a).

Operational ownership. In production, three teams touch the same decode path: serving (framework), compiler (graph lowering), and hardware (SKU drivers) (Davies *et al.*, 2025; Chen, 2020; Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b,a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024). Without Table 28.1 vocabulary, incidents become blame loops while bytes/token flatlines

(Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Runbooks should record bucket, shape bucket, paging block size, and export boundary hash alongside fleet dashboards (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

SGLang and structured serving. Structured generation adds mask and grammar state to the framework layer—still outside compiler fusion unless exported (Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). RadixAttention-style prefix reuse compounds paging benefits; compiler cores must not invalidate shared prefixes with per-request recompilation (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Multi-hardware delta. Framework support matrices differ: CUDA-first servers dominate cloud; CPU/OpenVINO paths serve edge; Ascend/MACA stacks require vendor runtimes (AMD, 2024a,b; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). YiRage targets the same decode core across backends—framework integration must not reintroduce GPU-only assumptions at the API boundary (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Liu *et al.*, 2024; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024). An integration that hard-codes CUDA graph capture without NPU static-shape buckets will pass GPU CI and fail edge deployment (AMD, 2024a; Chen, 2020; Chen and YiRage Contributors, 2026).

Example 28.1.1. A team runs vLLM for KV paging but keeps PyTorch-eager MLP boundaries: paging wins capacity while launches/token stay TaxBreak-high—framework choice fixed scheduling, not fusion (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026a). Moving to bucket (3) requires exporting the full decode cell with page-table layout IR, not swapping attention alone (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Dao *et al.*, 2022).

Migration path from eager to compiler-owned core. Most teams land in bucket (1), prove fleet demand, then partial-export attention or MLP while paging stabilizes (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026;



Figure 28.2: Continuous batching architecture: the scheduler interleaves prefill and decode into shared GPU slots backed by paged KV.

Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). The risky middle phase is partial export with hidden boundaries inside RMSNorm or rotary embedding: microbenchmarks show wins while bytes/token flatlines because carriers spill between exported regions (Dao *et al.*, 2022, 2023; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). A disciplined migration keeps the framework scheduler and block allocator fixed while shrinking the decode cell boundary each release, measuring launches/token and bytes/token on the same context buckets after every export change (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). Skip the worksheet step and you will ship bucket (2) forever, wondering why graph capture helped but cost per million tokens did not (Chen, 2026a; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

28.2 continuous batching

Continuous batching interleaves prefill and decode requests to raise GPU occupancy (Kwon *et al.*, 2023; Davies *et al.*, 2025; Liu *et al.*, 2024; Goodfellow *et al.*, 2016). Classic static batching waits until N requests align; continuous batching admits new sequences when others finish prefill or free decode slots—improving *fleet* goodput (tokens/s per GPU) (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a). It does not automatically cut *per-request* bytes/token at BS=1 inside a single decode step (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020). A scheduler that doubles fleet utilization while leaving hundreds of launches per token untouched has solved capacity, not mechanical sympathy inside the decoder (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Serving engineers often first encounter continuous batching as a throughput

win in load tests (Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). Compiler engineers should insist on paired microbench: same server binary, same paging policy, but forced BS=1 decode cell measurement between scheduler iterations (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). If only fleet metrics move, the next investment belongs in export boundaries and PersistentKernel integration, not longer batching windows (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Prefill vs decode in the scheduler. Prefill is compute-heavy and benefits from large token-parallel batches; decode at BS=1 per sequence is memory- and orchestration-bound (Dao *et al.*, 2022; Davies *et al.*, 2025; Chen, 2026a). Frameworks therefore use iteration-level scheduling: each engine step may mix prefill chunks and decode tokens from different clients (Kwon *et al.*, 2023; Davies *et al.*, 2025; Liu *et al.*, 2024). Compiler integration must preserve batching invariants—dynamic sequence lengths, slot assignment, cancellation—while still allowing graph capture or MegaKernel regions for steady decode cells (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016; AMD, 2024b,a; NVIDIA, 2022). Exporting a fused kernel that assumes fixed batch shapes breaks continuous batching unless the runtime provides shape buckets (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a).

Fleet goodput vs per-request counters. Fleet metrics—tokens/s, cost per million tokens, GPU utilization—reward batching and paging (Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Per-request counters—ms/token, bytes/token, launches/token at BS=1—reward fusion and residency inside the step (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020). Production teams need both: fleet metrics for capacity planning; per-request counters for compiler A/B and p99 latency under load (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026). Framework schedulers expose *when* a decode step runs; compilers decide *what* executes inside the step (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Profile both: scheduler queue time vs in-step ms/token (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen,

2020; Goodfellow *et al.*, 2016).

Document both metric families on the same release dashboard (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Fleet tokens/s may rise while BS=1 bytes/token stalls—label that as a scheduling win, not a fusion win (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Fusion wins that break batching legality should fail integration tests even when microbenchmarks improve (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). The contract runs both directions: compilers preserve paging and mask invariants; frameworks expose stable plugin ABIs (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Chunked prefill and decode interleaving. Long prefills can starve decode slots; frameworks chunk prefills and interleave decode steps to bound latency (Kwon *et al.*, 2023; Davies *et al.*, 2025; Liu *et al.*, 2024). Compiler-owned cores must support partial prefill shapes without recompilation storms—IR bucket tags for context length and batch composition are framework-facing contracts (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Chen, 2026a). DeepSeek-scale MoE adds expert-parallel scheduling on top of token batching (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026).

Scheduling policies and SLOs. Frameworks expose FCFS, priority queues, and preemption for premium tenants (Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). p99 latency is often scheduler-bound under burst load even when BS=1 kernels are optimal (Davies *et al.*, 2025; Chen, 2026a; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Capacity planning should separate queueing SLOs from in-step SLOs (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; SemiAnalysis, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016).

Watermarking and backpressure. When KV pools exhaust, frameworks reject or delay requests (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024b,a; NVIDIA, 2022; Goodfellow *et al.*, 2016). Compiler compression of KV shifts watermarks but requires framework allocator awareness (Liu *et al.*, 2024; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Multi-hardware delta. CPU-served frameworks batch differently—OpenVINO and llama.cpp paths use thread pools and blocked KV rather than GPU slot tables (Goodfellow *et al.*, 2016; AMD, 2024a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020). NPU serving often static-buckets requests by shape; continuous batching semantics must be emulated in software or sacrificed for compile-time legality (AMD, 2024b,a; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a).

Example 28.2.1. Under load, fleet tokens/s rises 40% after enabling continuous batching, but BS=1 microbench launches/token unchanged—expected if batching fixed occupancy, not fusion (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026). Next lever: compiler-owned decode core inside the same scheduler shell (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024).

Iteration-level scheduling mechanics. Continuous batching implementations maintain a running batch tensor whose rows map to live sequence slots (Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). Each engine iteration may append new prefill tokens for one client while advancing decode by one token for others in the same launch (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; SemiAnalysis, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016). Compiler-owned cores must accept dynamic active-row masks without recompilation per arrival: shape buckets cover max batch width while runtime masks disable finished sequences (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow

et al., 2016). Framework engineers own the slot table; compiler engineers own legality of fused kernels across masked rows (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

28.3 kv paging serv

KV paging serv (PagedAttention and descendants) treats KV cache as allocatable blocks—framework memory management, not compiler math (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022; NVIDIA, 2022, 2024). Without paging, contiguous KV reservations fragment GPU memory as sequences grow and shrink; paging maps logical token positions to physical blocks, improving fleet capacity (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a). Compilers must consume page tables as layout IR (Chapter 13), not fight them with contiguous-KV-only fusion plans (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Paging is the reason large-context serving became economically viable on single-GPU deployments: operators trade small internal fragmentation for predictable allocation (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; NVIDIA, 2022, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). It is not a substitute for lowering bytes moved during attention itself (Dao *et al.*, 2022, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Teams celebrating paging-only wins should still run Memory-Floor style byte ledgers on decode steps (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Block size and attention tiles. Block size trades fragmentation vs attention tile efficiency: smaller blocks reduce waste; larger blocks improve vectorized loads (Kwon *et al.*, 2023; Dao *et al.*, 2022, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; AMD, 2024b,a; NVIDIA, 2022, 2024). FlashAttention-style kernels assume block-aligned KV access; paging block size should co-tune with compiler tile search on the same context buckets

(Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). A paging policy chosen by SREs without compiler input can force gather/scatter patterns that erase fusion wins (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026).

Run a joint review when either team changes block size or attention tile width (Kwon *et al.*, 2023; Dao *et al.*, 2022, 2023; Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; NVIDIA, 2022, 2024; Goodfellow *et al.*, 2016). The review output is a signed counter sheet, not a slide deck (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Prefix caching and radix reuse. SGLang-style radix caches reuse KV blocks across prompts with shared prefixes—framework-side deduplication (Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Compiler cores must treat cached blocks as valid layout inputs: same page table semantics, different allocation provenance (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025). Exporting attention without page-table metadata breaks both vanilla paging and prefix reuse (Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026; Chen, 2026a).

Prefix reuse changes the economics of long system prompts: the framework avoids recomputing shared KV while the compiler still must legalize attention reads against block tables for the suffix tokens (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; SemiAnalysis, 2024; NVIDIA, 2024; AMD, 2024b,a; NVIDIA, 2022; Goodfellow *et al.*, 2016). Compiler teams should test prefix-hit and prefix-miss paths separately because gather patterns and occupancy differ (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Dao *et al.*, 2022; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Treat prefix cache as another paging policy knob co-tuned with tile search (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

MLA and compressed KV. DeepSeek-V3 MLA reduces KV footprint; paging and compiler layout must agree on compressed head dimensions (Liu *et al.*, 2024; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a). Open components under `deps/FlashMLA` illustrate MLA kernel expectations that framework paging must

feed (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025).

Memory accounting worksheet. Fleet operators should track bytes per KV block, active blocks per sequence, peak concurrent sequences, and fragmentation ratio (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024b,a; NVIDIA, 2022; Goodfellow *et al.*, 2016). Compiler teams add bytes moved per decode step at BS=1 for the same paging policy (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Disagreement between allocator metrics and profiler HBM counters signals layout IR mismatch (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Dao *et al.*, 2023; Goodfellow *et al.*, 2016).

Copy and migration costs. Disaggregated prefill/decode copies KV blocks—framework orchestration with compiler-visible layout (Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022, 2023; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024b,a; NVIDIA, 2022; Goodfellow *et al.*, 2016). Fusion plans that assume KV stays local break under migration unless gathers are legal across NUMA or NVLink paths (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Goodfellow *et al.*, 2016).

Multi-hardware delta. GPU paging uses CUDA virtual memory and block pools; CPU paths use pinned host buffers; NPUs use statically reserved SRAM windows (Goodfellow *et al.*, 2016; AMD, 2024a,b; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2023). A compiler that lowers paged KV only for contiguous GPU tensors will fail on XDNA static layouts (AMD, 2024b,a; Chen, 2020; Chen and YiRage Contributors, 2026).

Example 28.3.1. Block size 16 cuts fragmentation but increases attention loop trips; co-tuning with FlashDecoding tile width recovered bytes/token while keeping paging benefits (Kwon *et al.*, 2023; Dao *et al.*, 2023; Chen, 2026a; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

PagedAttention internals for compiler authors. Logical token position

maps through a block table to physical GPU memory blocks; attention kernels index KV via block id and offset rather than a flat stride (Kwon *et al.*, 2023; Dao *et al.*, 2022, 2023; Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; SemiAnalysis, 2024; NVIDIA, 2022, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). Compiler layout IR must represent gathers from block tables as first-class ops with known alignment and dtype, not as opaque custom calls that break fusion legality analysis (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; NVIDIA, 2022, 2024; Goodfellow *et al.*, 2016). When MegaKernel fusion plans assume contiguous KV tensors, paging forces either illegal fusion or a gather prologue that dominates bytes/token (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Liu *et al.*, 2024; AMD, 2024b,a; NVIDIA, 2022, 2024; Goodfellow *et al.*, 2016). Treat block tables like weight layouts in Chapter 13: co-design block policy with tile search (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

28.4 framework bottlenecks

Framework bottlenecks mirror Chapter 1: launches/token, bytes/token, and orchestration tax—often introduced *above* the decoder kernel by Python loops, dispatcher churn, and unfused vendor op sequences (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; NVIDIA, 2022). Framework A/B that reports only tokens/s without per-step byte ledgers misattributes compiler regressions to scheduling (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

When triaging a slow server, split the timeline before optimizing kernels (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Queue wait implicates admission control and batching policy (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*,

2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016). High in-step milliseconds with moderate launches implicates bytes/token and fusion gaps (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). High launches with moderate bytes implicates orchestration and graph capture opportunities (Chen, 2026a; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Skipping this split sends compiler teams to rewrite attention while the framework still dispatches hundreds of tiny kernels (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Python and host overhead. Eager HF loops pay GIL, tensor metadata, and per-op Python dispatch (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a). Serving frameworks move hot paths to C++/CUDA but still coordinate sampling, stop criteria, and RPC on host (Kwon *et al.*, 2023; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2026a). CUDA Graph capture removes launch storms for steady decode shapes; it does not fuse illegal HBM traffic (Chen, 2026a; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022). Teams that graph-capture unfused graphs lock in high bytes/token at lower launch count (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; NVIDIA, 2024; Chen and YiRage Contributors, 2026).

Operator chains inside the step. Even “optimized” servers often call separate attention, norm, and MLP kernels each token (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024). TaxBreak counts the launches; Memory-Floor counts the bytes (Vellaisamy *et al.*, 2026; Chen, 2026a). Framework bottlenecks end where compiler fusion begins—but only if integration exports the whole decode cell, not a single op (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Walk one token through the server call stack in a profiler and count CUDA API entries between scheduler entry and sampling exit (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Each entry is a fusion boundary candidate (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Kwon *et al.*,

2023; Goodfellow *et al.*, 2016). Prioritize boundaries that move activations to HBM between norm and matmul or between attention and MLP (Dao *et al.*, 2022, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; SemiAnalysis, 2024; NVIDIA, 2024; AMD, 2024a,b; NVIDIA, 2022; Goodfellow *et al.*, 2016). That ordered list becomes the export roadmap for bucket (3) migration (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Speculative decoding overhead. Draft models add scheduler complexity: verify steps, acceptance sampling, and dual KV streams (Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016). Frameworks must slot draft and target sequences; compilers should fuse verify cells where numerics allow (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; NVIDIA, 2024). Profile speculative paths separately—fleet speedup can hide verify-step launch storms (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016).

Observability split. Instrument four spans per request: queue wait, prefill steps, decode steps, and sampling/host (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). nsys rows alone are insufficient if framework queue time dominates p99 (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Export traces with bucket id, block size, and export hash for compiler A/B (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Goodfellow *et al.*, 2016).

RPC and tokenizer tax. Front-end frameworks pay tokenization, detokenization, and gRPC serialization—outside GPU counters but inside user-perceived latency (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Compiler wins on bytes/token do not remove host bottlenecks; they prevent misallocated engineering weeks (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Multi-hardware delta. CPU-served frameworks hit GIL and memcopy costs; GPU frameworks hit launch storms; NPU frameworks hit static-shape bucket misses (Goodfellow *et al.*, 2016; AMD, 2024a,b; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2023). Bottleneck triage starts with counter worksheets per target, not framework brand (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Example 28.4.1. Replacing Python loop with CUDA graphs improved ms/token 12% at BS=1 but bytes/token flat—graphs fixed orchestration, not fusion (Chen, 2026a; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020; Dao *et al.*, 2022; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Fair framework comparison methodology. Benchmark posts that compare vLLM, SGLang, and TRT-LLM using only aggregate tokens/s at fixed concurrency often reflect scheduler defaults and paging block sizes rather than intrinsic decode efficiency (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; SemiAnalysis, 2024; AMD, 2024b,a; NVIDIA, 2022; Goodfellow *et al.*, 2016). A fair comparison fixes model checkpoint, dtype, context distribution, block size, and GPU SKU, then reports BS=1 launches/token and bytes/token alongside fleet curves (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Without the fixed-cell worksheet, framework marketing numbers cannot inform compiler roadmap prioritization (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Chapter 1 counters exist precisely to prevent apples-to-oranges serving benchmarks from driving fusion decisions (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

28.5 compiler integration hooks

Compiler integration hooks are the seams where YiRage attaches: exported FX/ONNX regions, custom op *plugin* registration, CUDA graph capture buckets,

and PersistentKernel call sites (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; NVIDIA, 2022). Hooks should preserve numerics carriers from Chapter 10 and residency metadata from Chapter 6—exporting a subgraph without those invariants recreates silent HBM spill (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2020; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022).

Integration design meetings often fail because participants use the word kernel loosely (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). A framework custom op may wrap one CUDA kernel or an entire PersistentKernel program with internal loops (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). The hook contract must specify which side owns slot tables, which side owns softmax state, and which side owns graph capture replay (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Ambiguous ownership produces double-free style bugs in performance: two layers both “optimize” the same boundary and break numerics (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

FX and ONNX export regions. PyTorch FX traces decode cells for compiler lowering; ONNX export targets TRT and ORT runtimes (Davies *et al.*, 2025; Chen, 2020; Goodfellow *et al.*, 2016; NVIDIA, 2024; Chen and YiRage Contributors, 2026). Export boundaries must include page-table gathers, rotary embeddings, and online-softmax state—not only matmuls (Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Davies *et al.*, 2025; Liu *et al.*, 2024; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016). Broken exports pass unit tests on square tensors and fail on paged decode (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022).

Custom op plugins. Frameworks register `torch.library` or vLLM custom ops that delegate to compiled `.so` modules (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; NVIDIA, 2022; Dao *et al.*, 2023). Plugin APIs are the stable contract: framework passes slot ids, page tables, and sampling flags; compiler returns fused launch descriptors (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Version skew between framework paging and plugin layout IR is a common production outage mode (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a).

Treat each plugin like a narrow ABI: document struct layouts, alignment, dtype, and error codes; reject silent fallbacks to eager ops (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Fallbacks hide fusion failures behind correct-but-slow paths that pass functional tests while burning fleet budget (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Integration CI should fail when fallback triggers except in explicitly whitelisted debug modes (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

CUDA graph buckets and PersistentKernel. Graph capture requires static launch templates per shape bucket (NVIDIA, 2024; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022). YiRage PersistentKernel mode collapses launches inside the compiled core while the framework keeps scheduling—Chapter 29 details runtime handoff (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022). Integration tests should cover: bucket switch, cancel mid-sequence, and prefix-cache hit (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Shape buckets should align with framework batch width limits and max context policies so capture replay does not miss rare production shapes (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). PersistentKernel programs extend buckets with internal iteration over layers while presenting one host launch per decode step—the pattern Chapter 29 documents in `deps/YiRage` (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Frameworks must not assume one graph per model; they should register one graph per bucket per compiled fingerprint (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Design rule: frameworks own *request and KV lifecycle*; compilers own *decode-step residency*; runtime owns *launch and synchronization* inside the compiled core (Chapter 29) (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022). Chapter 30 formalizes the split for production stacks (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). DeepSeek open components (`deps/DeepEP`, `deps/FlashMLA`) show what must plug below framework schedulers for MoE at scale (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Integration test matrix. Minimum CI scenarios: BS=1 ctx 512/2k/8k; continuous batch mix; prefix-cache hit; cancel mid-stream; bucket switch after compile (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Each scenario logs launches/token and bytes/token (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Rollback and fingerprint caches. YiRage superoptimize caches fused artifacts by IR fingerprint; framework upgrades that change page-table ABI must bump fingerprint salts (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*,

2026; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Silent cache hits across incompatible paging policies are production footguns (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Multi-hardware delta. Hooks differ by backend: CUDA plugins vs XDNA AIE binaries vs CPU AVX512 kernels—the API shape should stay constant while layout IR lowers per target (AMD, 2024b,a; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2022, 2023; Liu *et al.*, 2024).

Example 28.5.1. vLLM custom op wrapping YiRage superoptimized decode cell: framework passes block tables and slot map; plugin returns single PK launch—launches/token dropped from hundreds to single digits while paging unchanged (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2026a; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

ABI stability across releases. Plugin boundaries serialize slot indices, block table pointers, sampling parameters, and optional speculative-draft handles (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Framework minor version bumps that reorder block table fields without coordinated compiler releases produce silent wrong-token bugs worse than a hard crash (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Pin paired framework and compiler versions in production manifests; run golden-token tests on every bump (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen, 2026a; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Chapter 30 extends this into a three-party release train (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Metric	Framework layer meaning
Queue wait	Scheduler capacity and admission
Fleet tokens/s	Batching and slot utilization
Block utilization	Paging fragmentation and capacity
Launches/token (in-step)	Orchestration inside decode box
Bytes/token (in-step)	Export boundaries before compiler fusion
Graph bucket hit rate	Capture stability under ctx growth
Plugin fallback rate	Silent eager regressions

Table 28.2: Framework decode counter worksheet (Part VIII).

28.6 Framework decode benchmarking methodology

Benchmarking inference frameworks extends Chapter 1 counters with scheduler-owned and in-step ledgers: queue wait, fleet tokens per second, paging block utilization, launches per token inside the decode box, and bytes per token after drawing the boundary (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; SemiAnalysis, 2024; AMD, 2024b,a; NVIDIA, 2022; Goodfellow *et al.*, 2016). Build a decode cell with BS=1, ctx buckets {512, 2048, 8192}, fixed model checkpoint, dtype, paging block size, and GPU SKU—then repeat at representative fleet concurrency for queue wait (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024).

Warm-up must include scheduler steady state, paging allocator warm pools, CUDA graph capture if enabled, and at least one thousand decode steps—single-request microbenches that never cross the decode boundary misattribute framework wins (Kwon *et al.*, 2023; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Report queue ms, in-step ms, launches per token, and bytes per token on the same worksheet row (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Example 28.6.1. Framework gate: at ctx=2048 and BS=1, queue wait must stay within baseline tolerance while in-step launches per token and bytes per token move only with documented plugin or paging changes—attach block size, bucket

id, and decode-box diagram (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024).

Boundary-first worksheets. Draw the decode step box before any framework A/B: slot mask ingress through hidden state before sampling (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Counters outside the box are framework-owned; counters inside belong to compiler integration until Chapter 29 runtime handoff (Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Continuous batching benchmarks. Measure BS=1 in-step counters separately from fleet goodput curves (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Batching wins that never appear at batch-1 hide operator-first decode tax (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Paging policy sweeps. Sweep block size and max-num-seqs against bytes per token and block utilization (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Paging fixes capacity, not activation residency between ops inside a layer (Kwon *et al.*, 2023; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Graph capture versus fusion. Benchmark launches per token alongside bytes per token when enabling CUDA Graphs (NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Graph wins

without byte wins lock in unfused HBM ping-pong (Chen, 2026a; NVIDIA, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Plugin integration replay. After framework-only baseline, replay identical traffic through YiRage custom op plugins from Section 28.5 (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Deltas isolate compiler fusion; unchanged queue wait confirms scheduler isolation (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Fair cross-framework comparison. Fix checkpoint, dtype, ctx distribution, block size, and SKU before comparing vLLM, SGLang, or TRT-LLM headline tokens per second (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Publish BS=1 launches per token and bytes per token beside fleet curves (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Speculative decode worksheets. Profile draft and verify paths separately with dual KV stream ledgers (Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Acceptance-rate marketing hides verify-step launch storms (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Observability spans. Instrument queue wait, prefill steps, decode steps, and sampling or host egress per request id (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Export traces with bucket id, block size, and plugin fingerprint for compiler A/B (Davies *et al.*, 2025;

Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Long-context stress. Extend framework benchmarks to ctx=16384 when paging allows; graph buckets tuned at 2K often miss at 32K (Kwon *et al.*, 2023; Dao *et al.*, 2022; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Fail CI when bucket hit rate drops without documented policy change (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Cancel and prefix-cache stress. Benchmark randomized cancel at 1% rate and prefix-cache hits—framework paths that microbenches skip (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Pool and slot leaks show as rising queue wait with flat in-step counters (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Multi-hardware matrix. Run identical worksheets on GPU, CPU, and NPU serving targets where fleets are heterogeneous (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Framework brand comparisons without fixed-cell worksheets mislead compiler roadmaps (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Regression bisect. When p99 stalls, sequence: (1) split queue wait versus in-step ms; (2) diff launches per token; (3) diff bytes per token; (4) check plugin fallback rate; (5) verify paging ABI semver (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Skipping boundary split sends compiler teams to rewrite attention while scheduler

queue dominates (Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

CI golden framework pair. Store framework-native and plugin-integrated decode scripts side by side; PRs that raise in-step launches per token or bytes per token without documented paging changes fail merge (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Golden pairs prevent silent eager relapses at plugin boundaries (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Documentation contract. Publish framework benchmark artifacts with every scheduler or paging promotion: decode-box diagram, block size, ctx buckets, queue wait, in-step counters, and plugin fingerprint (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Without published ledgers, serving teams cannot replay regressions (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Handoff to Chapter 29. Framework worksheets complete when decode-box boundaries and in-step counters are pinned for PersistentKernel integration (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Missing artifacts force runtime teams to guess slot and paging contracts (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Disaggregated prefill-decode benchmarks. When prefill and decode pools split, tag queue wait and in-step counters by pool id (Liu *et al.*, 2024; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Network KV transfer bytes are framework-owned; in-step fusion bytes remain compiler-owned—conflating them hides regressions (Liu *et al.*, 2024; Davies

et al., 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

MoE scheduling benchmarks. Expert-parallel decode adds framework routing tax; count launches per token per layer separately from dense baselines (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Framework wins on fleet tokens per second that ignore routing launches mislead compiler fusion priorities (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Host and RPC tax isolation. Tokenization, detokenization, and gRPC serialization sit outside GPU counters but inside user latency (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Benchmark host spans separately so compiler byte wins are not credited for framework queue fixes (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Bucket migration cadence. When moving workloads from framework-native to partial-export to compiler-owned decode core per Table 28.1, benchmark after each bucket shrink (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Partial exports that fuse MLP but leave attention eager often show launch wins without byte wins—document both (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Paging ABI semver gates. Framework minor bumps that reorder block table fields require coordinated plugin releases and golden-token tests (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis,

2024). Benchmark CI must fail when paging ABI semver drifts without fingerprint salt bump (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Efficiency metric. Track *framework-attributed decode efficiency*: fleet tokens per second divided by roofline tokens per second after subtracting in-step bytes and launches explained by compiler artifacts (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Framework teams own queue wait numerator; compiler teams own in-step byte denominator—publish both weekly (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Nightly CI cadence. Run golden framework pairs nightly on pinned GPU runners; plugin replay weekly; multi-hardware legs monthly when pools allow (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Flaky tests that skip paging assertions teach teams to ignore critical framework release gates in every weekly train (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Worksheet archive policy. Version framework worksheets beside paging block size and plugin fingerprints so bisect compares scheduler and in-step counters across releases (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Archive JSON schema fields match on-call postmortem templates for fleet operators (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Executive readout template. One-page promotion summaries show queue wait versus in-step counter deltas side by side—executives approve scheduler changes, not kernel names alone (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao

et al., 2023; NVIDIA, 2022; SemiAnalysis, 2024). Hide in-step regressions behind fleet p50 charts and teams rediscover operator-first serving habits (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Admission control benchmarks. Sweep max concurrent sequences and token budgets; queue wait should move predictably while in-step counters stay flat unless paging policy changes (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Over-admission inflates p99 without touching decode fusion (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Prefix-cache interaction. Prefix hits change prefill mix but must not alter in-step decode counters at BS=1 unless plugin boundaries move (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Benchmark prefix scenarios separately from steady decode cells (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Quantization and paging. INT8 and FP8 serving paths need explicit scale tensors in framework export regions so plugins do not silently dequantize per op (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Framework benchmarks must pin quant policy beside block size (Liu *et al.*, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Autoscaling warm pools. Cold-start replicas skip warmed graph capture and paging pools—benchmark scale-out events separately from steady decode (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024;

Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Autoscaling policies must budget warmup time when promotion gates require green worksheets (Chen, 2026a; NVIDIA, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Postmortem schema. Store JSON per incident with request id, queue ms, in-step ms, launches per token, bytes per token, block size, bucket id, and plugin fingerprint (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Incidents without schema fields should not close until worksheets are archived (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Final gate sentence. Do not merge framework paging or scheduler changes without green golden pair, paging ABI match, and in-step counter movement on three ctx buckets at BS=1 (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Anything less is costly framework drift wearing compiler fusion labels in production fleets (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Closing synthesis. Framework decode benchmarking is complete when queue wait, in-step launches per token, and in-step bytes per token align on three ctx buckets with paging and plugin pins—otherwise keep changes in staging (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). That discipline prepares Part VIII runtime and co-design chapters with measurable evidence, not marketing slogans or isolated kernel anecdotes (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Rotary and export boundaries. Framework export regions often stop before rotary embedding and attention gathers—benchmark norm plus QKV plus rotary

as one in-step region (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Skipping rotary in decode-box audits understates framework export tax (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

SwiGLU export chains. Gate, up, and down MLP projections triple operator boundaries in framework-native stacks (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Framework benchmarks should count in-step launches per token across full MLP chains, not only attention (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Schedule sympathy at the boundary. Profile queue wait first, paging second, in-step launches third, in-step bytes fourth, then tune framework batch windows (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Post the boundary sympathy mantra beside every framework worksheet template in the team wiki (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Acceptance criteria recap. Green framework promotion on three ctx buckets requires queue wait flat or improved, in-step launches per token down or flat, in-step bytes per token down, and plugin fallback rate zero except whitelisted debug modes (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Pair with golden-token tests on every paging ABI bump (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Benchmark artifact schema. Store JSON lines per decode step with request id, queue ms, in-step ms, block size, bucket id, launches count, bytes estimate, and ctx length (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Per-step JSON enables fast and reproducible postmortems; aggregates belong in fleet dashboards only after per-step validation passes cleanly (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Onboarding benchmarks. New serving engineers should run decode-box worksheets on day one before proposing batch window or block size changes (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Onboarding docs must include paired framework-native and plugin-integrated worksheets—culture follows measurable counters at the scheduler and decode boundary every weekly production release train (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

28.7 Key Takeaways

- **Frameworks schedule; compilers fuse.** Continuous batching interleaves prefill and decode requests to raise GPU occupancy and fleet utilization, but it does nothing about the batch-1 bytes/token inside a single decode step (Kwon *et al.*, 2023; Davies *et al.*, 2025). The framework owns request scheduling and the compiler owns intra-step fusion, so a serving win at one layer must not be mistaken for a win at the other (Vellaisamy *et al.*, 2026; Chen, 2026a).
- **Paged KV is framework layout the compiler must lower.** PagedAttention treats the KV cache as allocatable blocks—framework memory management, not compiler math—but the compiler must still lower those page tables as first-class IR so fusion can see page boundaries (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026). Hiding the paging from the compiler forces gather/scatter that breaks on-chip residency at the attention seam (Chen, 2020; Davies *et al.*, 2025).

- **Measure both layers, not one.** Framework bottlenecks mirror Chapter 1—launches/token, bytes/token, orchestration tax—but are often introduced *above* the kernel by Python loops and dispatcher churn, so the diagnosis must report queue/scheduling time alongside in-step ms/token and bytes/token (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). A fast kernel behind a slow scheduler still misses the SLO (Chen, 2026a; Goodfellow *et al.*, 2016).
- **Integration hooks are contracts, and parity is explicit.** The seams where a compiler attaches—exported FX/ONNX regions, custom-op plugins, CUDA-graph capture buckets, PersistentKernel call sites—must preserve softmax carriers and capture legality across the boundary, or fusion silently regresses (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; NVIDIA, 2024). Framework defaults are CUDA-centric, so multi-hardware parity requires explicit triplet tests on the YiRage backends rather than assumed equivalence (AMD, 2024b,a).
- **Migrate by shrinking the decode export, and version the ABI.** Move a workload bucket by bucket, shrinking the decode export boundary, and never confuse a paging win with a fusion win; before any PersistentKernel work, log the bucket, decode-box boundaries, queue wait, and in-step counters at context 512/2k/8k (Kwon *et al.*, 2023; Davies *et al.*, 2025). Framework paging and compiler plugins must release together with golden-token tests, since an ABI skew between them corrupts output in ways a counter dashboard will not catch (Chen and YiRage Contributors, 2026; Chen, 2026a).

28.8 Conclusion

Inference frameworks solve fleet scheduling and KV capacity; they do not replace compiler-side fusion and runtime-side launch collapse (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022). Classify your stack with Table 28.1, profile scheduler and in-step counters separately, and treat every plugin boundary as a numerics and layout contract (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Dao *et al.*, 2022; NVIDIA, 2024; Goodfellow *et al.*, 2016). The practical outcome of this chapter is a shared vocabulary between serving and compiler teams: when paging wins but bytes/token stalls, the next owner is the decode cell export, not the scheduler (Kwon *et al.*, 2023; Vellaisamy

et al., 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; NVIDIA, 2024; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). Before opening Chapter 29, run one end-to-end worksheet on your production stack: name the bucket, draw the decode step box, log queue wait and in-step counters for ctx 512/2k/8k, and list every plugin boundary with numerics carriers attached (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). That worksheet becomes the acceptance criteria for PersistentKernel integration in the next chapter (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Chapter 29 descends into YiRage Layer 5 Persistent Kernel runtime (**deps/YiRage** submodule); Chapter 30 stitches framework, compiler, and runtime into one decode goodput workflow (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Chapter 29

YiRage Runtime Layer and Persistent Kernel Execution

Chapter 28 placed inference *frameworks* above the decode step; this chapter covers YiRage *below* exported graphs—the C++ runtime, Persistent Kernel launcher, and backend registry that turn `superoptimize()` results into repeatable decode cells (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Source tree: git submodule `deps/YiRage` (upstream (Chen and YiRage Contributors, 2026)); init via `git submodule update -init -recursive deps/YiRage` (see `deps/README.md`).

TaxBreak counts launches/token inside the decode box; Memory-Floor counts bytes/token and graph replay relief (Vellaisamy *et al.*, 2026; Chen, 2026a). Compiler passes from Part VI–VII emit fused plans; **Layer 5 runtime** owns device memory discipline, launch templates, and backend selection so those plans survive fleet load (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). Without runtime contracts, `superoptimize()` wins evaporate at the framework plugin boundary (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Readers should map each Layer 5 API to Chapter 1 counters: `PersistentKernel` collapses launches/token; runtime residency metadata cuts bytes/token; `HardwareRegistry` constrains search to the SKU you ship (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*,

2026; Chen, 2026a; AMD, 2024b,a; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Chapter 30 stitches framework schedulers to this runtime; here we stay inside `deps/YiRage` (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Fregly mechanical sympathy at runtime means treating device memory like registers with lifetimes: allocations inside the decode step are bugs unless proven on-chip (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Layer 5 code paths should log residency class transitions the way Chapter 6 planners annotate IR (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). When runtime logs show unexpected HBM allocs per token, return to Layer 3 fusion legality before tuning scheduler batch windows (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Kwon *et al.*, 2023; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

This chapter is the mirror of Chapter 28: frameworks own everything outside the decode box; runtime owns everything inside once the framework plugin invokes PK (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Integration meetings should draw that box first, then assign Layer 1–5 owners (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Runtime engineering begins where compiler IR ends. The fused graph is no longer a data structure in Python; it is a device resident program with allocation pools, synchronization points, and launch templates that must survive millions of decode steps (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Teams new to YiRage often stop measuring after superoptimize reports a winning tile config. Production goodput is decided in Layer 5 when paging pointers arrive from vLLM, when CUDA graphs replay, and when softmax carriers stay in registers instead of spilling to HBM (Kwon *et al.*, 2023; Chen, 2026a; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Goodfellow *et al.*, 2016). Treat runtime as a first class performance surface with its own counters, owners, and release train (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025;

Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

The submodule layout under `deps` mirrors this responsibility. Python examples in the book intentionally point at pinned upstream APIs, but Layer 5 behavior lives in C++ under `libyirage` runtime (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). When documentation disagrees with profiler traces, trust traces and file issues against runtime launch paths rather than re tuning scheduler batch sizes alone (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Chapter 30 will show how framework schedulers call into this stack; here we keep the camera inside Layer 5 (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Three questions gate every runtime review. First, does warmed PersistentKernel beat eager launches/token on the fleet SKU at batch one (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Second, does bytes/token beat the pre fusion baseline at context 512, 2048, and 8192 (Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Third, does integration preserve paging ABI and softmax carriers through ten thousand cancel and resume cycles (Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). If any answer is no, stop feature work and return to superoptimize or descriptor design (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Runtime tuning knobs cannot fix illegal fusion or missing manifest fields (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Runtime ownership should appear on the org chart, not only in architecture slides. Layer five engineers maintain launch descriptor schemas, pool allocator invariants, and graph capture bucket tables that framework teams consume as stable ABI. When paging block size changes in vLLM, runtime must publish a semver bump before framework plugins ship; when superoptimize emits a new softmax carrier layout, runtime validates manifests at load time rather than discovering divergence during golden token tests. On-call runbooks should list the three counter worksheets from Chapter one beside PK warmup steps so incidents do not devolve into guessing whether regression lives in search, runtime, or scheduler batching.

Production fleets rarely fail because a single kernel is slow; they fail because two replicas run different artifact digests, because cancel paths leak pool slots

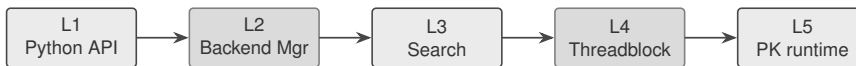


Figure 29.1: YiRage five-layer stack: Layers 1–4 search and lower; Layer 5 Persistent Kernel runtime executes decode cells on device.

under continuous batching, or because a canary node reports the wrong chip id to the registry. Layer five observability therefore emphasizes per-step JSON logs with fingerprint, registry id, descriptor version, launches count, and byte estimate aggregated into dashboards that framework SREs can read without reading C++. Postmortems should replay ten thousand decode steps from a single request id and verify pool high-water marks return to baseline after cancel. Treat runtime regressions as release train defects: roll back the artifact bundle, not only the framework container image.

29.1 yirage stack layers

YiRage stack layers follow the five-layer model in Table 29.1: Python API → Backend Manager → Search/Strategy → Threadblock ops → **Persistent Kernel runtime** (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Parts IV–VI focused on compiler passes and IR legality; Part VIII completes the stack with execution and device memory management (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024; Chen, 2020).

Layer boundaries and debugging. Production incidents often misattribute Layer 3 search regressions to Layer 5 runtime or vice versa (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). When bytes/token regress, diff the fused μ Graph fingerprint first (Layer 3 output); when launches/token regress, inspect PK launch descriptors and graph capture buckets (Layer 5) (Chen and YiRage Contributors, 2026; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen, 2020; Kwon *et al.*, 2023; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Layer 2 must reject cross-backend artifacts silently loaded from cache—a Hopper-tuned plan must not execute on XDNA without re-search (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Industrial teams should staff Layer 5 with engineers who read nsys and who un-

Table 29.1: YiRage layer responsibilities and decode goodput levers.

Layer	Component	Primary lever on decode
L1	Python <code>graph</code> , <code>PersistentKernel</code>	User-facing API; mode and backend selection (Chen and YiRage Contributors, 2026; Chen, 2020)
L2	Backend Manager	Routes CUDA/CPU/NPU backends; same-backend enforcement (Chen and YiRage Contributors, 2026; AMD, 2024b,a)
L3	Search / <code>superoptimize</code>	Fusion plans, tile configs, fingerprint cache (Chen and YiRage Contributors, 2026; Vellaisamy <i>et al.</i> , 2026; Chen, 2026a)
L4	Threadblock ops	Lowered primitives before PK packaging (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024)
L5	PK runtime	Launch collapse, device memory pools, graph replay hooks (Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2026a; Davies <i>et al.</i> , 2025)

derstand paging ABIs from Chapter 28 (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Compiler teams own Layers 1–4 artifacts; runtime teams own launch stability, pool lifetimes, and graph replay (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). When both groups share one on-call rotation, counter worksheets prevent thrash: if launches/token improved but bytes/token worsened, rollback Layer 3; if both stalled, inspect descriptor versions at the framework boundary (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Layer 3 logs should include enough context to reproduce PK configs without re-running full search (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Threadblock lowering is the last human-readable checkpoint before opaque device code (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). When numerics diverge, compare threadblock dumps against eager reference at the same paging layout (Kwon *et al.*,

2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Do not tune softmax epsilons in runtime unless threadblock IR confirms carrier layout (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). This discipline keeps runtime hotfixes from masking compiler legality bugs (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Python API surface. Layer 1 exposes `yirage` as `yr`, `yr.new_graph()`, tensor helpers, and `graph.superoptimize(backend=...)` entry points documented in upstream `docs/API_REFERENCE.md` (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Application code should treat graphs as immutable after `superoptimize`: runtime consumes frozen configs (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Mixing eager edits post-search breaks fingerprint caches and framework plugin contracts from Chapter 28 (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Backend Manager routing. Layer 2 selects installed backends (CUDA, CPU, NPU) and validates build flags from native extensions (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Import failures here surface as missing `yirage.core` or `libyirage_runtime`—not as silent CPU fallbacks in production (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). CI should compile each target backend separately; triplet tests compare counters, not import success alone (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024b,a; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Search versus runtime split. Layer 3 explores fusion and tiling; Layer 5 assumes a fixed plan (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Dao *et al.*, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Autotune during serving is an anti-pattern—it destroys graph capture stability and p99 latency (Chen, 2026a; NVIDIA, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Offline `superoptimize` plus pinned artifacts is the production pattern (Chen

and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Layer 4 threadblock ops are the last IR-like surface before opaque device binaries—debug dumps here when PK numerics diverge from eager reference (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Observability per layer. Layer 1 logs Python call sites; Layer 5 emits device timestamps and byte counters (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Correlate layer logs with framework traces from Chapter 28 using shared request ids (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Without correlation, runtime teams optimize kernels while queue time dominates p99 (Davies *et al.*, 2025; Chen, 2026a; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Export a single JSON line per decode step containing layer id, registry id, artifact fingerprint, launches count, and byte estimate (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Aggregates belong in fleet dashboards; per step JSON enables postmortems (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Teach on-call engineers to filter on fingerprint first, registry id second, and framework version third (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Most regressions are mismatched artifacts, not mysterious hardware noise (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Layer handoffs deserve explicit interface tests in CI, not tribal knowledge. After superoptimize completes, assert that Layer four threadblock dumps include every carrier field the Layer five loader expects. After PersistentKernel construction, assert that warmup completes within a bounded step count and that steady decode reuses the same pool handles across one thousand tokens. When Layer two rejects a backend, the error message should name the missing native extension so developers do not confuse import success with PK readiness. Stack diagrams in documentation are useful, but automated boundary tests prevent silent drift when upstream YiRage refactors internal module names while preserving Python API

spelling.

Multi-hardware delta. Layer 4–5 differ sharply: CUDA PK uses graph capture and TMA-aware pools; XDNA uses static AIE binaries; CPU uses vectorized host loops (AMD, 2024b,a; NVIDIA, 2022; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Same Python API must not imply identical launch semantics (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a,b; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Example 29.1.1. Search on laptop CPU then deploy CUDA PK without re-superoptimize: numerics may match while launches/token and bytes/token diverge—violates same-backend rule (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b,a; Goodfellow *et al.*, 2016).

Worked layer walkthrough. Start from a minimal decode graph in Layer 1 and execute superoptimize on the fleet GPU (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Inspect Layer 3 logs for chosen fusion boundaries and tile sizes, then dump Layer 4 threadblock lowering for the attention seam (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Instantiate PersistentKernel in decode mode and run ten tokens at context 512, 2048, and 8192 (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Record launches per token, HBM bytes per token, and wall time per token separately for each bucket (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Only after Layer 5 counters flatten should you integrate the framework plugin from Chapter 28 (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Skipping the isolated PK microbench hides descriptor bugs that become production incidents under continuous batching (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

29.2 persistent kernel decode

Persistent kernel decode uses `yr.PersistentKernel(mode="decode", backend=..., fallback_backends=[...])` to select backend execution with



Figure 29.2: Persistent kernel decode path: framework passes slot and page metadata; PK runtime launches fused decode cells with collapsed host overhead.

graceful CPU/NPU fallbacks (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022). The `persistentkernel` object wraps a long-lived device program: internal loops over decoder layers replace per-op host launches (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). `mode="decode"` selects shape buckets and numerics carriers tuned for BS=1 token steps rather than prefill-only tiles (Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Same-backend rule. Search and execute on the installed target—do not profile on CPU and ship CUDA kernels without re-search (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; AMD, 2024b,a; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Fleet labels (GPU SKU, driver, CUDA version) must match `HardwareRegistry` entries used during `superoptimize` (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Warmup passes prime PK device memory pools and graph capture—cold first token latency is a runtime concern distinct from compiler fusion quality (Chen, 2026a; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Decode PK differs from training kernels in occupancy goals (Davies *et al.*, 2025; Dao *et al.*, 2022; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Training chases large batch matmul utilization; decode chases minimal bytes per token at batch one (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Persistent loops amortize layer weights across tokens inside one launch, but only if softmax and norm carriers remain on chip between layers (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Profile carrier lifetimes explicitly when PK shows

unexpected HBM traffic (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Cancellation and mid-sequence abort must release PK pool slots without leaking device memory (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Frameworks free paging blocks on cancel; runtime must mirror that lifecycle in pool accounting (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Long-running servers accumulate leaks if cancel paths skip pool teardown (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Stress tests should randomize cancel timing under continuous batching before fleet promotion (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Fallback backends. `fallback_backends` enable dev laptops and CI smoke tests without forking model code (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024a,b; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Production must pin primary backend to fleet SKU; fallbacks are for correctness checks, not performance baselines (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Accidentally serving through CPU fallback while GPU PK is available reads as mysterious fleet cost overrun (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Device memory pools. Layer 5 allocates persistent buffers for weights, KV views, and softmax carriers across decode steps (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Framework paging passes block table pointers into PK launch descriptors—runtime must not copy KV to contiguous staging each token (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Silent staging copies show up as bytes/token regressions with unchanged fusion plan (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Launch descriptor contract. Each decode step receives slot mask, page table

handle, step id, and sampling flags from the framework plugin (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). PK runtime validates descriptor version before launch—version skew must hard-fail, not produce wrong tokens (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Golden-token tests compare eager reference against PK for each descriptor version bump (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Goodfellow *et al.*, 2016).

Prefill versus decode PK modes. `mode="decode"` selects persistent loops tuned for single-token advance; prefill may use separate superoptimize profiles (Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Mixing prefill tiles in decode PK wastes bandwidth—tag modes explicitly in artifact manifests (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Framework schedulers call the correct PK mode per engine iteration (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Goodfellow *et al.*, 2016).

Shape buckets for decode should align with framework max batch width and max context policies (Kwon *et al.*, 2023; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Each bucket needs a warmed PK instance and a graph capture template (Chen, 2026a; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Missing buckets fall back to slow paths that show up as tail latency spikes rather than mean regression (Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Document bucket coverage in runbooks beside paging block size (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Persistent kernel tuning is a memory lifetime problem disguised as a launch problem. Each decoder layer expects weight tensors, norm statistics, and attention softmax carriers to arrive at fixed addresses across steps so the device loop can branch minimally. When runtime allocates fresh staging buffers per token, bytes per token rises even though TaxBreak launch counts look excellent. Profile with

memory transaction counters and verify that page table pointers from the framework pass straight into gather kernels without intermediate copies. When carriers spill to high bandwidth memory between layers, return to Layer three fusion legality before widening graph capture buckets.

Stress decode under adversarial scheduler patterns: alternating prefill and decode on the same slot, random cancels at ten percent of steps, and context growth from five hundred twelve to eight thousand one hundred ninety two within one session. PK pools must survive these patterns without monotonic growth in reserved bytes. Framework plugins should not assume PK warmup happens once per process lifetime if workers restart frequently in Kubernetes; document whether warmup is idempotent and how long it blocks readiness probes. Serving teams often tune liveness thresholds without accounting for first capture latency, then blame runtime for pod churn that is actually missing warmup in the probe design.

Multi-hardware delta. CUDA PK pairs with CUDA Graph buckets; XDNA PK loads AIE binaries with static DMA chains; CPU PK uses OpenMP-blocked loops (NVIDIA, 2024; AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Identical Python `persistentkernel` constructor arguments may map to different launch paths (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a,b; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Example 29.2.1. Decode cell (BS=1, ctx=2048): PK launch drops launches/token from TaxBreak hundreds to single digits while framework paging unchanged—runtime owns launch collapse, not batching (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; NVIDIA, 2024; Chen, 2020; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

PK versus CUDA graphs alone. Memory-Floor style graph replay removes host launch storms but cannot fix illegal fusion (Chen, 2026a; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). PersistentKernel combines graph stability with compiler chosen fusion inside the replayed region (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Teams seeing graph wins without byte wins should escalate to superoptimize rather than adding more graph buckets (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*,

2023; Goodfellow *et al.*, 2016). PK is not a replacement for paging or batching; it replaces per op launches inside the decode cell (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

MoE and expert parallel hooks. Large MoE models add expert parallel communication below the PK entry point (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Open DeepEP components illustrate all to all patterns that must compose with PK memory pools rather than allocate per token (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Profile EP separately from PK fusion when bytes/token regress on MoE checkpoints (Liu *et al.*, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

29.3 superoptimize handoff

Superoptimize handoff bridges `graph.superoptimize(backend=...)` outputs to runtime launch: fused μ Graphs, fingerprint verification, and cached kernel configs (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; NVIDIA, 2022, 2024; Dao *et al.*, 2023; Goodfellow *et al.*, 2016). Layer 3 search emits a serialized plan; Layer 5 loads it into PK launch templates (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Framework hooks (Chapter 28) must not strip metadata required for capture or MegaKernel legality (Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Handoff is where many projects stall: search finishes, a demo looks fast, but no artifact pipeline reaches serving (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Treat superoptimize output like a container image: digest, manifest, signature, and rollback tag (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Framework plugins should load by digest at process start, not by mutable local paths (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Canary deploys swap digest on

a fraction of replicas while comparing counter dashboards (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Without digest discipline, two replicas silently run different PK programs under the same endpoint (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Fingerprints and cache keys. Superoptimize caches results keyed by IR fingerprint plus backend id and hardware registry hash (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Framework upgrades that change page-table ABI must bump cache salts or risk loading incompatible PK binaries (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). CI should verify fingerprint mismatch fails closed—never silently fall back to unfused eager (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Graph to kernel packaging. Threadblock lowering produces device-specific binaries or CUDA C++ sources consumed by PK runtime (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; NVIDIA, 2022; AMD, 2024b,a; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016). Handoff artifacts include tile sizes, double-buffer depth, and softmax carrier layout (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Missing carrier metadata passes unit tests on square tensors and fails on paged decode (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016).

Warmup and steady state. First PK launch may compile, allocate pools, and capture graphs; steady decode reuses them (Chen, 2026a; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Fleet benchmarks must separate warmup from sustained tokens/s (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Production SLOs often ignore

first-token compile; decode p99 must use warmed PK state (Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Goodfellow *et al.*, 2016).

Artifact storage layout. Store superoptimize outputs as versioned bundles: fingerprint, registry id, backend, blob, and manifest JSON (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Framework plugins fetch bundles by hash at startup—avoid embedding blobs inside Python wheels without hash checks (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Rollback is redeploy prior bundle, not revert framework scheduler alone (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Goodfellow *et al.*, 2016).

Legality metadata handoff. Export softmax carriers, page-table layouts, and graph-capture flags as manifest fields (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Runtime refuses launch if manifest missing required fields—fail closed (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). This prevents partial exports from Chapter 28 plugins (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Manifest evolution requires semver: breaking carrier layout bumps major version; additive fields bump minor (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Framework plugins declare supported manifest major before loading artifacts (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Mixed major versions across replicas is a deployment error, not a runtime tuning opportunity (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Chapter 30 runbooks formalize this three party release train (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Artifact promotion should resemble a binary signing pipeline. Search cluster produces a bundle; staging loads it read only; golden token harness compares hashes against eager reference on three context buckets; production receives a

digest pin in the framework config. Never promote because a demo video looked fast on one laptop. Store the promotion checklist JSON beside the bundle so auditors can reconstruct who signed off on launches per token and bytes per token deltas. When a framework upgrade changes only scheduler policy but not paging ABI, artifacts may carry forward; when block table layout changes, invalidate caches even if IR fingerprint unchanged. Handoff meetings should read manifest fields aloud until boring repetition makes missing fields impossible.

Offline search and online runtime must share a single hardware registry revision checked into version control. Drift between search cluster registry and serving cluster labels produces plans that are legal yet slow, which is harder to detect than hard failures because tokens still look plausible. Automate a nightly job that compares detect current chip output on every node class against expected registry ids and pages when mismatches exceed zero. Superoptimize caches keyed only on IR miss these failures until bytes per token creep upward over weeks.

Multi-hardware delta. Cache entries are not portable across backends—re-superoptimize per target even when IR matches (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Example 29.3.1. Fingerprint hit loads cached PK plan; framework block size change without re-search yields wrong KV gathers—cache must invalidate on paging ABI change (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Handoff checklist before fleet deploy. Verify manifest lists backend id, registry id, fingerprint, paging ABI version, and softmax carrier schema (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Run golden tokens on three context buckets with warmed PK (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Compare launches/token and bytes/token against prior artifact in the same dashboard row (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Sign off only when both counters move or regressions are explained with profiler evidence (Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen,

2020; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Store the checklist JSON beside the artifact bundle for audit (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

29.4 hw registry runtime

Hw registry runtime (`HardwareRegistry`, `detect_current_chip()`) supplies chip models for search configs—Hopper vs XDNA vs M3 Max tile candidates (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Davies *et al.*, 2025; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024). The `hardwareregistry` maps detected devices to search spaces: SM count, TMA availability, L2 size, NPU tile SRAM, CPU vector width (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016). `detect_current_chip()` runs at import or first superoptimize—runtime detection must align with fleet labels used in framework schedulers (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a,b; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Search space pruning. Registry entries prune illegal tile shapes before auto-tune explosion (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; AMD, 2024b,a; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016). Hopper profiles enable TMA double-buffer candidates; XDNA profiles restrict to static DMA chains (NVIDIA, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Wrong chip detection yields legal-but-slow plans—profile bytes/token on representative ctx buckets (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Registry entries should be versioned documents checked into the same repo as artifact manifests (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). When hardware vendors ship new SKUs, add entries before marketing promises fleet migration (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Each

entry lists peak bandwidth, on-chip capacities, and forbidden fusion patterns for that target (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Search reads these as hard constraints, not hints (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Detect current chip at process start and log the resolved registry id in every performance trace (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Post-incident review should filter traces by registry id to spot partial rollouts (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Mixed ids behind one load balancer explain bimodal latency histograms (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Registry maintenance is a hardware catalog discipline, not a one time bootstrap script. Each entry documents peak bandwidth, on chip SRAM, vector width, tensor engine availability, and forbidden fusion patterns for that SKU. When vendors ship new accelerators, add entries before marketing announces fleet migration dates. Search reads entries as hard constraints; runtime logs the resolved id on every decode step so capacity planners can correlate artifact lines with instance types. Retire entries only after all pinned artifacts migrate and dashboards show zero traffic on the old id for two release cycles.

Virtualized and partitioned GPUs distort detection unless runbooks override generic ids with fleet labels from node metadata. Multi instance GPU slices may report parent SKU names that mislead tile search toward shapes that fit the parent but not the slice memory budget. Time shared virtual machines may hide NPU presence entirely until first superoptimize call. Document override procedures and require platform teams to validate overrides quarterly against vendor tools output.

Fleet inventory sync. Kubernetes node labels, cloud instance types, and driver versions should map to registry ids (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Mismatch between scheduler placement and PK chip model causes silent suboptimal tiles (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Pin superoptimize artifacts per registry id in artifact stores (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Multi-SKU rollouts. Rolling PK updates across heterogeneous fleets requires parallel artifact lines per chip entry (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024b,a; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Never reuse Hopper PK binaries on Blackwell without re-search even when CUDA version matches (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; Goodfellow *et al.*, 2016).

Registry-driven autotune budgets. Cap search trials per chip model to keep CI bounded (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b,a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Registry metadata feeds roofline estimates that prune hopeless tile shapes before device trials (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Log pruned candidates for audit when bytes/token underperform expectations (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Virtualization pitfalls. Containers sharing GPUs must expose consistent chip ids to `detect_current_chip` (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). MIG slices and time-sliced VMs may report generic ids—override with fleet labels when needed (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Multi-hardware delta. `detect_current_chip` on VM passthrough GPU may report wrong SKU—validate against `nvidia-smi` product name in runbooks (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; NVIDIA, 2022; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022;

AMD, 2024a,b; Dao *et al.*, 2023; Goodfellow *et al.*, 2016).

Example 29.4.1. Fleet moves from H100 to H200: rerun superoptimize with updated registry entry; reuse old fingerprint cache caused 8% bytes/token regression until invalidated (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; SemiAnalysis, 2024; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Registry operations runbook. Onboard a new GPU SKU by adding a registry entry, running a bounded superoptimize search, and publishing artifact bundles tagged with the new id (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Update Kubernetes node labels to match registry ids before shifting traffic (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). During incidents, compare `detect_current_chip` output with node labels before blaming framework schedulers (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Retire old entries only after all pinned artifacts migrate (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

29.5 deps build native

Deps build native: `import yirage` requires `yirage.core + libyirage_runtime`—no Python-only native build path (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Submodule init: `git submodule update -init -recursive deps/YiRage`; build via `YIRAGE_BACKEND=cuda pip install -e deps/YiRage` (see upstream docs/INSTALLATION.md) (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Native extension layout. Python package wraps C++ runtime under `deps/YiRage/yirage/` with compiled `yirage.core` module linking `libyirage_runtime` (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024; Goodfellow *et al.*, 2016). Missing `.so` files import as pure Python stubs that fail at first PK call—smoke tests must invoke `PersistentKernel`, not only `import`

`yirage` (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Building from source ties book examples to reproducible hashes (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Pin submodule commit in release notes whenever prose references API symbols (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Nested submodules may require recursive init; document network and disk expectations in deps README (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Agents editing prose should not hand modify generated native sources; changes belong upstream in YiRage (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Wheel-based installs are convenient for demos but hide compiler flags (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Production should prefer editable installs from pinned submodule with recorded `nvcc` and driver versions (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). When wheels and submodule builds diverge, counter worksheets become incomparable across environments (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Backend build flags. `YIRAGE_BACKEND=cuda|cpu|npu` selects compile targets; nested submodules may pull vendor SDKs (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Docker images for serving should bake the same backend flags as search CI (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Drift between build and runtime containers is a common PK load failure (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Version pin with book repo. Pin submodule SHA in `.gitmodules` or recorded commit; Chapter 30 runbooks require matching `deps/YiRage` revision with framework plugin ABI (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Book examples reference APIs at pinned SHA—update prose when bumping submodule

(Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Developer workflow. Typical loop: edit IR or search config → **superoptimize** on target SKU → run PK decode microbench → export artifact hash for framework plugin (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Skipping native rebuild after C++ runtime change produces stale .so mismatches (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

CI smoke template. Minimal pipeline: submodule init, backend-specific compile, import yirage, superoptimize tiny graph, PersistentKernel decode ten tokens, assert golden hash (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Extend with BS=1 counter worksheet from Chapter 1 before promoting artifacts to staging (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Docker and reproducibility. Lock CUDA driver, toolkit, and YiRage SHA in serving images (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Non-reproducible PK builds frustrate bisect when fleet regressions appear (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Record compiler flags in artifact manifest (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Serving images should compile YiRage once in the build stage and copy artifacts into the runtime stage (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Runtime stage installs matching driver user space libraries only; never compile on production pods (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Init containers can verify artifact digest against manifest before accepting traffic (Chen

and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). This pattern prevents silent drift between search cluster and serving cluster (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Native build failures often masquerade as Python packaging issues. Missing `nvcc`, wrong driver user space libraries, or stale build directories produce import success followed by opaque failures at first `PersistentKernel` call. CI must execute the full smoke template on the same backend flag as production containers, including ten token decode and golden hash comparison. Developers on laptops without GPUs may compile CPU backend for unit tests but must never promote CPU searched artifacts to CUDA serving. Record compiler flags, submodule SHA, and backend enum in artifact manifests so `bisect` can distinguish runtime code changes from toolchain drift.

Book readers reproducing examples should init the submodule recursively, install editable with explicit backend flag, and run PK decode microbench before reporting API mismatches. Agents editing prose should not hand modify generated native sources; changes belong upstream in YiRage with submodule bump and manifest semver bump in this repository. When documentation and profiler traces disagree, trust traces and file issues against runtime launch paths.

Multi-hardware delta. CUDA builds need compatible `nvcc` and driver; XDNA builds need Ryzen AI SDK paths documented upstream (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). CI matrices should include at least one compile plus PK smoke per supported backend (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024a,b; Goodfellow *et al.*, 2016).

Example 29.5.1. `pip install -e deps/YiRage` on dev laptop without CUDA toolkit: import succeeds, PK decode fails at runtime—gate merges on PK smoke, not import (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Local dev versus fleet parity. Developers may compile CPU backend locally for unit tests while fleet runs CUDA PK (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024a,b; Goodfellow *et al.*, 2016). Never promote CPU searched artifacts to GPU serving (Chen and YiRage Contribu-

tors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Provide a small GPU CI runner that executes the same smoke template as production (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Document driver and toolkit versions in deps README when bumping submodule SHA (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Book readers should clone submodule recursively before reporting API mismatches (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

29.6 YiRage runtime decode benchmarking methodology

Benchmarking Layer 5 decode extends Chapter 1 counters with runtime-specific ledgers: PK warmup steps, pool high-water marks, graph capture bucket hits, and artifact fingerprint stability across ctx buckets (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). Build a decode cell with BS=1, ctx {512, 2048, 8192}, pinned **deps/YiRage** SHA, matching registry id, and framework paging ABI from Chapter 28 (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Warm-up must include PersistentKernel construction, first graph capture, pool allocator steady state, and at least one thousand steady decode steps—cold import success alone is not a valid production benchmark (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Report first-token latency and token-*k* steady ms/token separately; product SLOs often bind to the latter while UX marketing highlights the former (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Triplet runs should compare eager framework decode, PK-only microbench, and full framework plugin path on the same SKU (Chen and YiRage Contributors,

Metric	Layer 5 decode meaning
Launches/token	PK + graph replay vs eager baseline
Bytes/token	Pool residency + paging traffic
Pool high-water	Leaks under cancel/resume
Fingerprint match	Artifact drift across replicas
Registry id	SKU-aligned search vs runtime
Graph bucket hit rate	Capture stability under ctx growth

Table 29.2: Runtime decode counter worksheet (extends Chapter 1 TaxBreak / Memory-Floor).

2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). A PK win that disappears after plugin integration signals descriptor ABI mismatch, not “framework overhead” (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Example 29.6.1. Runtime decode gate: warmed PK at ctx=2048 must show launches/token within 5% of search logs *and* bytes/token within 10% of Memory-Floor worksheet; attach fingerprint, registry id, and ten-step pool ledger (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Cancel and resume stress. Continuous batching injects cancel paths that microbenches skip (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Benchmark ten thousand decode steps with randomized cancel/resume at 1% rate; pool high-water must return to baseline within bounded slots (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Fail CI when cancel paths allocate fresh HBM per step (Chen, 2026a; Chen and YiRage Contributors,

2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Artifact pinning in CI. Nightly jobs must load the same artifact bundle as staging: fingerprint, manifest semver, compiler flags, and submodule SHA (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Drift between search cluster artifacts and serving replicas explains bimodal latency histograms better than kernel tuning (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Publish golden hashes and pool ledgers beside every Layer 5 release note in CI (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Framework boundary replay. After PK microbench passes, replay the same counters through the Chapter 28 plugin with identical paging block size and request ids (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Deltas here isolate scheduler tax from runtime tax (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Never promote PK artifacts to staging or production without this second worksheet (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Multi-SKU fleet sweeps. Heterogeneous fleets need parallel benchmark matrices keyed by registry id—reuse Hopper numbers on Blackwell only after re-search and re-warmup (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; AMD, 2024b,a; Vellaisamy *et al.*,

2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016). Dashboards should slice launches/token and bytes/token by registry id, not aggregate fleet averages (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Operator documentation. Ship runtime benchmark artifacts with every promotion: ctx buckets, warmup protocol, pool ledger, graph bucket table, and triplet comparison summary (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Without published ledgers, on-call engineers cannot distinguish artifact drift from framework queueing (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Release checklist. Before tagging a Layer 5 release, require green PK smoke, cancel stress, plugin replay, and fingerprint match across three ctx lengths (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Partial green is how production inherits week-long triage across runtime and framework teams (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Observability hooks. Emit structured logs at PK load, warmup complete, steady decode entry, and graph capture miss events (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Production builds strip verbose fields but retain stable event names so SRE dashboards correlate runtime regressions with artifact promotions (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen,

2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Long-context stress exports. Extend runtime benchmarks to ctx=16384 when product SLAs allow paging beyond 8K (Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Graph capture buckets tuned at 2K often miss at 32K—fail CI when bucket hit rate drops below threshold at long ctx (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Speculative decode at runtime. Draft-verify stacks double PK instances; benchmark acceptance-rate sweeps with separate pool ledgers per model (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Alias bugs between draft and verify KV pools show up only under load—include them in cancel stress matrices (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Closing note. Treat Layer 5 decode benchmarking as a shipping contract alongside native builds—without nightly PK replay and published pool ledgers, every latency spike becomes a costly and painfully slow multi-team blame game (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Cross-stack triage playbook. When decode latency spikes after a runtime merge, run this sequence before blaming silicon: (1) compare artifact fingerprints across replicas; (2) verify registry id matches node labels; (3) replay pool ledger on

golden PK export; (4) diff launches/token and bytes/token against last green worksheet; (5) only then sweep driver and power caps (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Skipping fingerprint checks wastes days chasing framework queue noise (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Efficiency metric. Track *runtime-attributed decode efficiency*: measured tokens/s divided by roofline tokens/s after subtracting framework scheduler overhead measured at the plugin boundary (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Runtime teams own the PK numerator; framework teams own scheduler tax in the denominator—publish both weekly (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Native build parity in benchmarks. Benchmark jobs must compile deps/YiRage with the same YIRAGE_BACKEND flag as serving containers (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). CPU-backend smoke tests are valid for API drift but must never gate GPU artifact promotion (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024a,b; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Record nvcc, driver, and submodule SHA in every benchmark manifest (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Handoff to Chapter 30. The runtime worksheet completes when warmed PK counters, plugin replay deltas, and manifest hashes are pinned for the production workflow in the next chapter (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Missing any field forces Chapter 30 integration meetings to guess Layer 5 state (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Regression harness cadence. Check golden PK decode scripts into the book repo beside static graph exports: same tokenizer, ctx buckets, pool counters, and fingerprint assertions every nightly CI run (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Submodule bumps that change descriptor layout must fail CI before prose updates merge (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Documentation contract. Every Layer 5 promotion note must list ctx buckets, warmup steps, steady ms/token, launches/token, bytes/token, and golden fingerprint at ctx=2048 (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Partners cannot replay regressions without these fields—support cycles lengthen when benchmarks stay in private spreadsheets instead of versioned artifact stores (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Summary. Layer 5 benchmarks are complete when PK, plugin, and cancel-stress worksheets agree within published tolerances on all three ctx buckets—anything

else is an integration draft, not a shippable production fleet release candidate (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

29.7 Key Takeaways

- **The runtime is Layer 5, distinct from the compiler.** Compiler passes from Parts VI–VII emit fused plans, but the runtime layer owns device-memory discipline, launch templates, and backend selection so the plan actually executes within budget (Chen and YiRage Contributors, 2026; Chen, 2020). Treating runtime as an afterthought is how a correctly-compiled megakernel still misses p99 when the host reconstructs state every token (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).
- **Same-backend search.** Profile and ship on the target SKU; `hardwareregistry` must match fleet labels (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b,a).
- **Handoff is contractual.** Fingerprints, page-table ABI, and softmax carriers must survive `superoptimize` to PK launch (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026).
- **The submodule is the source of truth.** The runtime depends on the native `deps/YiRage` build, so CI must compile that submodule and smoke-test a Persistent Kernel decode rather than testing only the Python import (Chen and YiRage Contributors, 2026; Chen, 2020). An import-only test passes while the native runtime is broken, which is precisely the failure that reaches production undetected (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).
- **Counters at runtime.** Measure launches/token and bytes/token after PK warmup, not only import or search logs (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024; Goodfellow *et al.*, 2016).

29.8 Conclusion

YiRage runtime closes the gap between fused IR and fleet decode: Layer 5 `persistentkernel` programs collapse launches/token; device memory pools and graph hooks address bytes/token and orchestration tax (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Chen, 2020; Liu *et al.*, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022). Treat `superoptimize` artifacts, registry ids, and native builds as release-train inputs alongside framework paging policy (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Before Chapter 30, complete one Layer 5 worksheet: warmed PK, three context buckets, launches and bytes per token logged, manifest hash pinned (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024; Goodfellow *et al.*, 2016). Chapter 30 integrates this layer with vLLM-class frameworks and Chapter 12 execution modes into one production workflow (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Chapter 30

Framework, Compiler, and Runtime Co-Design for Decode Goodput

Production decode is a *three-way* contract: the inference **framework** schedules requests and owns KV paging; the **compiler** (YiRage middle-end + **superoptimize**) owns intra-step fusion and layout; the **runtime** (Persistent Kernel Layer 5) owns launches, device memory, and backend fallbacks (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022). When p99 latency stalls, teams that optimize only one leg recreate Chapter 1’s framework fragmentation at warehouse scale (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Part VIII closes by making that contract explicit: every integration meeting draws the decode step **boundary** first, then assigns owners and counters (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Chapter 28 mapped framework buckets and scheduler policy; Chapter 29 descended into YiRage Layer 5 Persistent Kernel runtime (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Goodfellow *et al.*, 2016). This chapter stitches the three parties into one production workflow: who owns prefill versus decode handoff, how vLLM paging composes with YiRage fused cores, which execution mode applies per subgraph on a heterogeneous fleet, and how regression

gates prevent silent drift across submodule bumps (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). The book’s industrial arc ends here—not with another isolated kernel trick, but with deployable co-design anchored on decode goodput counters (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Readers should leave with a release train mental model. Framework teams ship scheduler and paging policy; compiler teams ship fused plans and manifest schemas; runtime teams ship PK launch descriptors and pool invariants (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). No party optimizes in isolation: a framework upgrade that changes block table layout without bumping artifact cache salts breaks compiler handoff; a compiler win that ignores paging ABI wastes bytes per token; a runtime regression that leaks pool slots masquerades as scheduler overload (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Co-design means shared worksheets, shared request ids in traces, and shared rollback pins—not shared blame in postmortems (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Fregly mechanical sympathy at the three-way boundary treats each handoff as an API with semver, golden tests, and profiler evidence (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022; NVIDIA, 2024; Goodfellow *et al.*, 2016). Chapter 12 execution modes reappear here as fleet policy: eager hulls for sampling branches, CUDA Graph buckets for launch-bound cells, MegaKernel and Persistent Kernel paths for byte-bound decoder cores (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). DeepSeek-class disaggregation adds a fourth axis—prefill and decode pool routing—but the same boundary discipline applies inside each pool’s decode step (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Three questions gate every co-design review before fleet promotion. First, does the responsible party for the moving counter actually own the lever being tuned (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Second, do handoff

manifests survive framework, compiler, and runtime version bumps without silent fallback (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Third, does triplet regression on GPU, CPU, and NPU backends show bytes per token and launches per token moving together or explained separately (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016). If any answer is no, pause feature work and fix the contract before tuning batch windows (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

This closing chapter is deliberately operational. Earlier parts taught rooflines, fusion legality, dialect lowering, and autotune search (Chen, 2020; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Part VIII asked where those ideas attach in servers humans operate (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). The answer is never a wholesale framework replacement—it is a negotiated boundary with versioned handoffs and regression gates (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Teams that skip co-design and chase isolated kernel records on leaderboard shapes reproduce the fragmentation Chapter 1 diagnosed (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Co-design is the antidote measured in the same counters from prompt to token (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Warehouse-scale serving adds constraints single-GPU labs hide (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Continuous batching interleaves unrelated clients; cancel paths free slots unpredictably; disaggregated pools move KV across networks (Kwon *et al.*, 2023; Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Compiler and runtime optimizations must tolerate that chaos without leaking memory or violating paging ABI (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Framework optimizations must expose enough telemetry for compiler teams to see in-step counters, not only fleet aggregates (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Co-design is the discipline of making those telemetry paths first-class product requirements (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).



Figure 30.1: Responsibility split: framework, compiler, and runtime meet at the decode boundary; counters attach per party.

Leadership reviews should ask which party moved which counter last quarter, not which kernel microbenchmark improved (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Roadmaps that fund framework features without compiler handoff metadata or runtime descriptor tests recreate integration debt (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Conversely, compiler teams that ship artifacts without framework plugin support plan fail at the boundary (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Co-design funding follows counter movement with named owners (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). This chapter gives vocabulary for those staffing and budget conversations (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Part VIII therefore completes the narrative arc from mindset and hardware bounds through compiler stacks to deployable serving (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Readers who implement only one chapter should still implement this one if they operate fleets (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Theory without co-design contracts becomes shelfware; serving without counters becomes guesswork (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). The combined worksheet—boundary diagram, integration digest, mode map, triplet rows—is the minimum viable artifact set for production decode (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Goodfellow *et al.*, 2016).

30.1 responsibility split

Responsibility split is the operating table every production runbook must publish before on-call rotation (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026;

Table 30.1: Framework, compiler, and runtime responsibility split at the decode boundary.

Party	Owens	Primary counter
Framework	Batching, paging, routing, sampling policy, request lifecycle (Kwon <i>et al.</i> , 2023; Davies <i>et al.</i> , 2025)	Queue wait, fleet tokens/s, KV utilization
Compiler	Tiling, fusion, layout, legality, superoptimize artifacts (Chen and YiRage Contributors, 2026; Chen, 2020)	Bytes/token inside decode step
Runtime	PK launch, sync, pools, graph replay, backend registry (Chen and YiRage Contributors, 2026; NVIDIA, 2024)	Launches/token, warmup latency, pool high-water

Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022). Table 30.1 maps the three parties to concrete artifacts and counters. Ambiguity at the **boundary** between framework scheduling and compiler-owned decode cells is the dominant cause of misattributed regressions in fleet postmortems (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

The **framework** owns everything outside the decode step box: continuous batching, paged KV block tables, speculative decoding policy, disaggregated routing between prefill and decode pools, and client-facing SLO dashboards (Kwon *et al.*, 2023; Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Framework engineers optimize queue depth, slot assignment, and memory capacity—not softmax carrier layout inside a fused attention kernel (Davies *et al.*, 2025; Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). When fleet tokens per second rise but bytes per token inside the step is flat, the framework did its job; the compiler and runtime did not (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

The **compiler** owns intra-step fusion and residency: which operators merge, which tensors stay on chip between layers, which layouts match paging gathers, and which plans are legal for graph capture (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Compiler output is not PyTorch source—it is versioned artifacts with fingerprints, manifest fields, and hardware registry ids from Chapter 29 (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Framework plugins consume those artifacts; they must not silently strip softmax carrier metadata or paging ABI version fields (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

The **runtime** executes compiler plans on device: Persistent Kernel loops, memory pool lifetimes, CUDA Graph capture buckets, backend fallbacks, and descriptor validation at each decode step (Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Runtime engineers read `nsys` and pool accounting; they do not rewrite scheduler batching policy to mask launch storms (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). When launches per token regress after a runtime bump, bisect artifact digest and descriptor version before touching framework code (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Draw the decode **boundary** on every architecture diagram: from slot mask and block table handle ingress through final hidden state before sampling (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Inside the box, compiler and runtime own bytes per token and launches per token; outside, framework owns queue time and paging efficiency (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Integration meetings that skip the boundary redraw recreate Chapter 1 fragmentation: compiler teams tune GEMMs while scheduler queue time dominates p99 (Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

On-call triage by party. Queue wait up, in-step ms flat: framework scheduling or capacity (Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). In-step bytes up, launches flat: compiler fusion or layout regression (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow

et al., 2016). Launches up, bytes flat: runtime graph capture or PK descriptor regression (Vellaisamy *et al.*, 2026; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). All three up: likely boundary mismatch—paging ABI changed without artifact invalidation (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Org design implications. Staff framework, compiler, and runtime teams with separate counter dashboards but shared release train (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Shared on-call without shared worksheets devolves into thrash: each party tunes its leg while p99 stalls (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Co-design standups read one row per request id: queue ms, in-step ms, launches, bytes, artifact digest (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Example 30.1.1. Regression after framework block size change: compiler cache hit loads stale PK plan—boundary contract requires cache salt bump on paging ABI change, not runtime knob tuning (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Industrial teams document the split in runbooks linked from fleet config repos (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). New hires should trace one decode token through framework slot assignment, compiler artifact load, runtime PK launch, and sampling egress before proposing optimizations (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). That trace is the co-design literacy test for Part VIII (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Release trains should version the responsibility table alongside framework, compiler, and runtime artifacts (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). When a new feature spans two parties—for example speculative decoding in the framework plus extra PK launches in runtime—file a joint design doc with counter acceptance criteria before implementation (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Escalation paths should name DRI per party, not a

generic platform team that owns none of the levers (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Quarterly audits replay archived traces from peak traffic week and verify each party still matches the published table (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Contract tests between parties catch boundary drift early (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Framework publishes paging ABI fixtures; compiler asserts superoptimize output loads against fixtures; runtime asserts PK descriptors accept fixture block tables (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Breaking any contract test blocks merge until semver bump and coordinated release notes ship (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). This is how co-design scales beyond a single heroic integration engineer (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Worked boundary exercise. Pick one production model and one fleet SKU (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). List ten operations in decode order from scheduler slot assignment through sampling (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Mark each operation framework, compiler, or runtime owned (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Attach one counter per party (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Run one request through traces and verify marks match reality (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Mismatches indicate undocumented boundary creep—fix the runbook before optimizing (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Repeat after every major framework or submodule bump (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Vendor and open-source upgrades flow through the same table (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). CUDA driver updates may change graph capture behavior; vLLM releases may change block tables; YiRage bumps may change manifest majors (NVIDIA, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026;



Figure 30.2: Decode pipeline handoff: prefill may stay framework-orchestrated; decode enters compiler-owned cores; sampling remains an eager hull outside capture.

Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Assign each upgrade type a default owner and regression template so on-call does not debate responsibility during incidents (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). The responsibility split is living documentation, not a one-time slide (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

30.2 decode pipeline handoff

Decode pipeline handoff sequences three phases with different owners: **prefill** ingestion, per-token **decode** compute, and **sampling** egress (Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; NVIDIA, 2022). Prefill often stays framework-orchestrated because shapes vary widely and batching policy dominates (Kwon *et al.*, 2023; Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Decode steps enter compiler-owned cores when bytes per token and launches per token dominate at batch one (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Sampling and speculative branches remain *eager hulls* outside CUDA Graph capture per Chapter 12 legality rules (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Prefill handoff. Framework schedulers batch prompt tokens, allocate paging blocks, and may route long prompts to disaggregated prefill pools (Kwon *et al.*, 2023; Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Compiler involvement during prefill varies: some stacks superoptimize prefill profiles separately from decode; others reuse decode tiles inefficiently (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016). Document which PK

mode the framework invokes at prefill-to-decode transition—mixing modes without manifest tags causes silent bandwidth waste (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Prefill completion must commit KV layout metadata the decode artifact expects: block table dimensions, head layout, and rope cache handles (Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Decode handoff. Each decode iteration passes slot mask, page table pointer, step id, and hidden state buffers into the compiler-owned cell (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Runtime validates descriptor version, launches PK or graph replay, and returns logits or hidden states to the framework (Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Handoff latency includes framework Python overhead, plugin marshalling, and device sync—profile each separately before blaming fusion quality (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Continuous batching may swap slots mid-flight; decode artifacts must tolerate dynamic slot masks without re-capture on every step when graph mode is enabled (Kwon *et al.*, 2023; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Goodfellow *et al.*, 2016).

Sampling handoff. Top-k, top-p, temperature, and stop criteria execute outside captured regions (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Speculative decoding adds draft and verify loops that re-enter decode cores multiple times per user token (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Compiler plans must declare which tensors sampling reads and whether logits stay on device (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Framework policy changes to sampling should not require re-superoptimize unless carrier layout changes (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Handoff contracts deserve semver and golden-token tests at each phase boundary (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). CI should run prefill-only, decode-only, and full generate paths with shared artifact digests (Chen and YiRage Contributors,

2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Fail closed when manifest major version mismatches—never silently fall back to eager per-op decode (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

DeepSeek-class **disaggregated** stacks extend handoff across network boundaries: prefill pools write KV to remote decode pools (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Bytes per token then includes transfer cost; compiler and runtime optimizations inside decode pools still matter but must compose with transfer scheduling (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Profile end-to-end goodput, not in-pool ms alone (Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Example 30.2.1. Prefill completes at `ctx=8192`; decode PK warmed only to `ctx=2048` bucket—tail latency spikes on long sessions until runtime bucket table matches framework max context policy (Kwon *et al.*, 2023; Chen, 2026a; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Multi-hardware delta. Prefill may use wide batch tiles on GPU while decode PK targets batch one; CPU and NPU backends need separate handoff manifests per Chapter 29 same-backend rule (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; NVIDIA, 2022; Dao *et al.*, 2022, 2023; NVIDIA, 2024; Goodfellow *et al.*, 2016).

Handoff latency budgets belong in SLO documents with separate allocations for framework marshalling, plugin sync, and device execution (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Teams that collapse these into one in-step millisecond metric misallocate optimization effort (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Log each segment with shared request id so postmortems identify whether regression entered at prefill commit, decode launch, or sampling return (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Long context sessions stress handoff most at prefill-to-decode transition when bucket tables must expand—warm expanded buckets during prefill completion, not on first decode token (Kwon *et al.*, 2023; Chen, 2026a; NVIDIA, 2024; Chen

and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Timing diagram discipline. Draw prefill, decode, and sampling phases on one timeline per request class (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Mark where KV becomes visible to decode workers in disaggregated setups (Liu *et al.*, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Mark where logits cross back to framework for sampling (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Ambiguous timing diagrams produce integrations that double-copy KV or synchronize too aggressively (Kwon *et al.*, 2023; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Review diagrams in design docs with the same rigor as API schemas (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Streaming responses add another handoff: partial tokens must egress while decode continues (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Framework streaming APIs should not force synchronous device sync each token unless product requires it (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Compiler and runtime plans should declare which steps tolerate async logits delivery (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Profile streaming workloads separately from batch benchmark harnesses (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Tail latency under streaming often traces to handoff sync policy, not fusion quality (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

30.3 vllm yirage stack

Vllm yirage stack is the reference **integration** pattern for Part VIII: vLLM continuous batching and paged KV plus YiRage fused decode core invoked each step (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022). Measure queue wait and in-step counters separately—fleet tokens per second can rise while launches per token inside the step stays at

TaxBreak hundreds (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Goodfellow *et al.*, 2016). The stack does not replace vLLM; it shrinks the decode cell vLLM calls (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Integration architecture. Typical flow: vLLM scheduler assigns slots and block tables; a custom attention or decoder backend loads YiRage artifact by digest at worker start; each decode step marshals descriptors into PersistentKernel (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Plugin code lives at the framework boundary documented in Chapter 28 partial-export bucket (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024; Goodfellow *et al.*, 2016). Keep plugin surface minimal: slot mask, page table handle, step id, artifact digest—not raw tensor graphs per request (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

vLLM responsibilities retained. Paging, continuous batching, speculative decoding orchestration, and disaggregated routing stay in vLLM (Kwon *et al.*, 2023; Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). YiRage does not reimplement block allocators or queue policies (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). When KV utilization drops, tune framework batch windows before re-running superoptimize (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). When bytes per token inside the step stalls, escalate to compiler and runtime (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

YiRage responsibilities absorbed. Fusion across attention, norm, and MLP seams; softmax carrier residency; PK launch collapse; graph capture buckets aligned to vLLM shape policies (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Chen, 2020; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Artifacts pin at worker boot from `deps/YiRage` submodule SHA recorded in fleet config (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Submodule bumps trigger regression triplet from Section 30.5 before traffic shift (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Integration testing template. Stand up vLLM worker with pinned artifact;

run BS=1 decode at ctx 512, 2048, 8192; log queue ms, in-step ms, launches per token, bytes per token (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Compare against vLLM plus FlashAttention baseline on same SKU (Dao *et al.*, 2022; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Promote only when both in-step counters improve or regressions are explained with profiler evidence (Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Run cancel and resume stress under continuous batching before fleet promotion (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Open components under `deps/DeepEP` and `deps/FlashMLA` illustrate MoE and MLA patterns that must compose with vLLM paging rather than bypass it (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Integration docs should show where expert parallel all-to-all sits relative to PK entry—below scheduler, adjacent to decode cell (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Example 30.3.1. vLLM upgrade changes block table layout: YiRage cache hit serves stale gathers—integration contract requires artifact invalidation on paging ABI bump, not plugin hotfix alone (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Operational ownership. vLLM SREs own scheduler SLOs and paging capacity; YiRage owners own artifact pipeline and PK stability (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Shared dashboards correlate framework version, artifact digest, and runtime descriptor version per request id (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Integration failures at this boundary are the most common Part VIII incident class (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Rollout playbooks for vLLM plus YiRage integration should mirror compiler artifact promotion: dev cluster golden tokens, staging canary at five percent traffic, production shift with rollback digest pinned (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Document which vLLM worker flags enable the

custom backend and which remain framework defaults (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Training new operators on integration dashboards prevents conflating queue depth spikes with PK failures (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). When disaggregated prefill is enabled, verify YiRage artifacts on decode pool workers separately from prefill pool configs (Liu *et al.*, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Plugin minimization principle. The vLLM integration surface should stay thin enough to audit in one code review (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Heavy logic in plugins duplicates compiler and runtime responsibilities and forks across teams (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Push fusion and residency decisions into superoptimize artifacts; push launch stability into Persistent Kernel runtime (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Plugins marshal descriptors and handle error propagation only (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). When plugins grow past that scope, refactor boundary before next fleet promotion (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Capacity planning for integrated stacks counts both framework KV footprint and runtime pool high-water marks (Kwon *et al.*, 2023; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Scaling replicas without warming PK buckets on new nodes produces temporary tail spikes (Chen, 2026a; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Autoscalers should call runtime warmup hooks before marking pods ready (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Integration runbooks document expected warmup duration per artifact digest and context bucket (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Ignoring warmup in capacity math causes false headroom during traffic ramps (Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

30.4 mode selection fleet

Mode selection fleet applies Chapter 12 execution modes at production scale: **eager** for debug and sampling hulls; **graph** for launch-bound decode cells; **megakernel** and Persistent Kernel paths for byte-bound fused cores—chosen per subgraph, not per monolithic binary (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022). Fleet configs bucket context lengths, batch widths, and backend SKUs consistently so capture templates and PK warmup tables match scheduler policies (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Eager mode. Use for development, numerics bisect, and any branch outside graph legality—sampling, dynamic control flow, debugging hooks (NVIDIA, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Eager baselines establish golden tokens for PK and graph paths (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Never ship eager per-op decode at scale when TaxBreak launch counts exceed fleet budget (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Graph mode. CUDA Graph replay removes host launch storms when fusion depth is insufficient to reach MegaKernel but launches per token dominate (Chen, 2026a; NVIDIA, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Bucket graphs by shape signatures aligned to framework max batch and context policies (Kwon *et al.*, 2023; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Goodfellow *et al.*, 2016). Graph mode without byte wins indicates need for compiler fusion, not more buckets (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

MegaKernel and PK mode. Compiler-owned decode cores collapse layers inside Persistent Kernel loops, targeting bytes per token and residual launches (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Combine with graph capture where legality allows—PK provides fusion; graphs stabilize host overhead (Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Profile carrier lifetimes when bytes per token stall despite PK adoption (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Fleet bucketing policy. Document which mode applies to each subgraph in fleet config: prefill profile, decode profile, sampling hull, MoE communication (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Goodfellow *et al.*, 2016). Heterogeneous fleets maintain parallel artifact lines per hardware registry id from Chapter 29 (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024b,a; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Never reuse Hopper PK binaries on Blackwell without re-search (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; Goodfellow *et al.*, 2016).

Mode selection meetings should attach counter worksheets: if launches per token move but bytes do not, stop adding graph buckets and fix fusion (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). If bytes move but queue wait dominates p99, return to framework scheduling (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Fleet-wide mode policy prevents per-team improvisation that breaks artifact compatibility (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Multi-hardware delta. Graph mode is CUDA-centric; XDNA fleets use static AIE binaries; CPU fleets use OpenMP-blocked eager or PK host loops (AMD, 2024b,a; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Same fleet config schema should encode backend-specific mode enums (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024a,b; Goodfellow *et al.*, 2016).

Example 30.4.1. Team enables graph capture on entire decoder including sampling—illegal capture causes silent fallback to eager launches; mode map must exclude sampling hull (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Fleet configuration management should treat mode maps as first-class config objects with review gates (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024; Goodfellow *et al.*, 2016). Changing a subgraph from graph to PK mode without re-warming pools causes cold-start p99 spikes (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Autoscaling policies must account for warmup time when mode maps include capture-heavy paths (Chen, 2026a; NVIDIA, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Document mode map version in artifact manifest so bisect correlates performance changes with policy edits (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Multi-region fleets replicate mode maps only after triplet passes in each region’s SKU mix (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Debug versus production mode policy. Development clusters may run eager decode for fast iteration (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Production clusters must enforce mode maps checked into config management (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Goodfellow *et al.*, 2016). Accidental eager fallback in production reads as cost overrun and tail latency inflation (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Goodfellow *et al.*, 2016). Alert when launch counts exceed eager thresholds on production endpoints (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Separate debug mode toggles from feature flags so marketing demos do not become silent production configs (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Hybrid mode maps are normal: prefill eager or wide-tile, decode PK plus graph, sampling eager hull, MoE communication custom (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Document each subgraph mode in fleet config with owner and counter expectation (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Changing one subgraph mode without re-running triplet on affected rows invites silent regressions (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon

et al., 2023; Goodfellow *et al.*, 2016). Mode maps should be diff-reviewed like code (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Goodfellow *et al.*, 2016).

SRE teams should store mode map version in incident tickets alongside framework and artifact versions (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Replaying incidents without mode context wastes hours tuning the wrong subgraph (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Quarterly drills rotate mode map edits through staging triplet before any production diff (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Fleet mode policy is co-design infrastructure as critical as paging block size (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Goodfellow *et al.*, 2016).

30.5 production regression

Production regression gates fleet promotion on triplet counter suites: bytes per token, launches per token, and framework-level tokens per second—measured on GPU, CPU, and NPU backends where applicable (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024; Goodfellow *et al.*, 2016). Version pins for **deps/YiRage** submodule, framework release, and artifact digest must move together through canary deploys (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Submodule bumps require re-run of search caches and PK warmup profiles before traffic shift (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Triplet worksheet. Row one: launches per token on warmed PK at BS=1 for ctx 512, 2048, 8192 (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Row two: bytes per token on same buckets (Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Row three: end-to-end tokens per second including queue wait at representative fleet load (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Sign off only when each row improves or regressions carry profiler evidence and owner (Davies *et al.*, 2025; Chen, 2026a; Chen and

YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Regression triggers. Framework paging ABI change; compiler manifest major bump; runtime descriptor version change; hardware registry id addition; CUDA driver or toolkit upgrade (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; NVIDIA, 2024; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b,a; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Each trigger reruns triplet on primary SKU plus one secondary backend in CI matrix (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024a,b; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Canary and rollback. Shift artifact digest on fraction of replicas; compare triplet dashboards against control cohort (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Rollback redeploys prior artifact bundle—not framework scheduler alone (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Store rollback pins beside fleet config for one-click revert (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Bytes-focused regression. Memory-Floor style suites catch HBM traffic regressions graph replay hides (Chen, 2026a; NVIDIA, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Pair byte counters with carrier lifetime profiles when fusion changes (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Silent staging copies from paging integration show as byte regressions with unchanged launch counts (Chen, 2026a; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

CI pipelines pin submodule SHA, compile native extensions, run superoptimize smoke, PK decode ten tokens, and compare golden hash (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Extend smoke with vLLM integration template from Section 30.3 before production promotion (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Record worksheet

JSON in artifact store for audit (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Example 30.5.1. Submodule bump passes import test but fails PK golden hash—regression gate blocks merge; fleet stays on prior digest until triplet reruns green (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Long-horizon **regression** debt accumulates when teams skip triplet on minor framework patches (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Schedule weekly automated reruns on pinned production configs even without code changes—driver drift and thermal throttling move counters (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Treat regression suites as production infrastructure, not optional research scripts (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Cross-team regression review meetings compare triplet trends across framework, compiler, and runtime release notes (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). A flat launches per token row with rising bytes per token row signals compiler or paging integration regression (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Rising queue wait with flat in-step rows signals framework capacity or scheduling regression (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Archive worksheet JSON for each production promotion to enable quarter-over-quarter audits (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Regressions without archived worksheets are not reproducible and should not close incident tickets (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Closing the book loop. Return to Chapter 1 counters with co-design eyes (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). TaxBreak launches per token, Memory-Floor bytes per token, and framework tokens per second now have owners and handoffs (Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow

et al., 2016). MegaKernel craft from Part IV–V lives inside compiler artifacts (Chen, 2020; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Compiler theory from Part VI–VII lives in manifest schemas and legality metadata (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Runtime execution from Chapter 29 lives in PK pools and graph buckets (Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Framework policy from Chapter 28 lives outside the decode boundary (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). The book ends where production begins: pinned digests, triplet gates, and shared traces (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Regression culture. Treat triplet failures as cross-team signals, not blame events (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Framework teams publish paging ABI changelogs; compiler teams publish manifest semver notes; runtime teams publish descriptor diffs (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). On-call runbooks link each changelog entry to required regression reruns (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). When all three parties follow the rhythm, co-design becomes routine operations instead of heroic firefighting (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). That operational maturity is the practical definition of decode goodput at warehouse scale (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

30.6 Co-design decode benchmarking methodology

Benchmarking three-way co-design extends Chapter 1 counters with party-owned ledgers: framework queue wait and fleet tokens per second, compiler bytes per token inside the decode boundary, runtime launches per token and pool high-water marks (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022). Build a decode cell with BS=1, ctx buckets

Metric	Co-design party owner
Queue wait / fleet t/s	Framework scheduler
Bytes/token (in-step)	Compiler fusion plan
Launches/token	Runtime PK + graph replay
Artifact digest match	Compiler + runtime handoff
Mode map version	Fleet policy (Section 30.4)
Paging ABI semver	Framework + compiler contract

Table 30.2: Co-design decode counter worksheet (closes Part VIII).

{512, 2048, 8192}, pinned framework release, `deps/YiRage` SHA, artifact digest, mode map version, and paging ABI from Chapter 28 (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Warm-up must include framework slot assignment, compiler artifact load, runtime PK construction, graph capture steady state, and at least one thousand decode steps—microbenches that skip the boundary misattribute wins to the wrong party (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Report queue ms, in-step ms, launches per token, and bytes per token on the same dashboard row (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024).

Example 30.6.1. Co-design gate: warmed decode at `ctx=2048` must show in-step bytes per token within 10% of Memory-Floor worksheet, launches per token within 5% of TaxBreak logs, and queue wait flat versus baseline at representative fleet load—attach artifact digest, mode map hash, and request-id trace slice (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Boundary replay discipline. Run three worksheets on identical traffic: framework-only eager hull, compiler artifact through PK microbench, and full vLLM–YiRage plugin path from Section 30.3 (Kwon *et al.*, 2023; Chen and YiRage

Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Deltas between worksheet two and three isolate framework scheduler tax; deltas between one and two isolate compiler residency (Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Never promote co-design releases without all three rows green or explained (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Handoff manifest pinning. Nightly CI must load the same manifest bundle as staging: paging ABI version, artifact fingerprint, descriptor semver, submodule SHA, and mode map digest (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Drift between compiler search clusters and serving replicas produces bimodal latency histograms that no single-party tuning fixes (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024).

Triplet backend matrix. Extend co-design benchmarks to GPU, CPU, and NPU registry ids where fleets are heterogeneous (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024b,a; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Framework paging policy may differ per backend, but in-step byte and launch counters must use identical ctx buckets for cross-SKU comparison (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024).

Mode map interaction benchmarks. Hybrid fleet configs mix eager sampling hulls, CUDA Graph decode buckets, and PK MegaKernel cores (NVIDIA, 2024;

Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Benchmark each subgraph mode independently before composing the fleet map—graph capture on sampling branches fails silently and inflates launches per token (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Regression bisect by party. When p99 stalls, sequence: (1) diff artifact digest across replicas; (2) compare mode map version; (3) replay boundary trace on golden request id; (4) diff bytes per token worksheet; (5) diff launches per token; (6) compare queue wait at fleet load (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Skipping boundary replay keeps teams in multi-week blame loops (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Organizational metrics. Align OKRs with party-owned counters—framework teams own queue wait and fleet tokens per second, compiler teams own in-step bytes per token, runtime teams own launches per token and pool high-water (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). PRs that move one counter without worksheet evidence fail co-design review (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Canary and rollback worksheets. Shift artifact digest on a fraction of replicas; compare all three party rows against control cohort (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Rollback redeploys prior artifact bundle and mode map—not framework scheduler knobs alone (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025;

Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Documentation contract. Publish co-design benchmark artifacts with every fleet promotion: boundary diagram, three-row worksheet, manifest pins, mode map diff, and triplet backend summary (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Without published ledgers, partners cannot replay co-design regressions (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Release checklist. Before tagging a co-design release, require green boundary replay, manifest match, mode map triplet pass, and counter movement on all ctx buckets (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Partial green is how production inherits mixed schedules behind one load balancer (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Long-context co-design stress. Extend benchmarks to ctx=16384 when paging allows; framework block growth and compiler gather layouts must move bytes per token together (Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Fail CI when queue wait rises without in-step counter movement—likely boundary ABI drift (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Speculative decode co-design. Draft-verify stacks span framework policy and dual runtime PK instances (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Benchmark acceptance-rate sweeps with separate worksheets per model and shared request-id traces (Chen

and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024).

Efficiency metric. Track *co-design-attributed decode efficiency*: fleet tokens per second divided by roofline tokens per second after subtracting unexplained queue wait and attributing in-step bytes and launches to named parties (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Publish party numerators weekly in the same dashboard (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

CI golden triple policy. Store framework baseline, PK microbench, and full plugin scripts side by side; any PR that moves bytes per token or launches per token in the wrong direction fails before merge (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Golden triples prevent silent operator-first relapses at the boundary (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Disaggregated pool benchmarks. Prefill and decode pools route KV across networks; co-design worksheets must tag which pool owns each counter row (Liu *et al.*, 2024; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Network transfer bytes are framework-owned; in-step fusion bytes remain compiler-owned—conflating them hides regressions (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Postmortem artifact schema. Store JSON per incident with request id, queue ms, in-step ms, launches per token, bytes per token, artifact digest, mode map version, and paging ABI semver (Chen and YiRage Contributors, 2026; Chen, 2020;

Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Incidents without schema fields should not close until worksheets are archived in the fleet artifact store (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Executive readout template. One-page promotion summaries show three party counter deltas side by side—executives approve co-design releases, not isolated kernel names (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Hide counter regressions behind fleet p50 charts and teams rediscover Chapter 1 fragmentation (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Final gate sentence. Do not merge co-design changes without green golden triple, manifest match, and mode map diff reviewed by framework, compiler, and runtime DRIs on the same pinned artifact hash (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Anything less is costly three-way drift wearing co-design labels in production fleets (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Closing synthesis. Co-design benchmarking is complete when framework, compiler, and runtime worksheets agree within published tolerances on three ctx buckets with manifest and mode map pinned—otherwise keep the release in staging (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). That discipline closes the book loop back to Chapter 1 counters with named owners and shared rollback pins (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Nightly cadence. Run golden triple scripts nightly on pinned GPU CI runners with fixed seeds; replay through framework plugin weekly and triplet backends monthly when hardware pools allow (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Flaky tests that skip manifest assertions teach teams to ignore critical co-design release gates in every weekly production release train cadence (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Worksheet archive policy. Version co-design worksheets beside compiler fingerprints and mode maps so bisect compares party-owned counters across releases without re-running ad hoc scripts (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024). Archive JSON schema fields match postmortem templates so on-call and promotion reviews share one authoritative source of truth for fleet operators and on-call engineers (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

30.7 Key Takeaways

- **Three-way co-design.** Framework, compiler, and runtime each own distinct counters; optimize the leg that moves your worksheet row (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a, 2020; Goodfellow *et al.*, 2016).
- **Boundary is API.** Decode step boundary, paging ABI, capture buckets, and PK entry points documented with semver (Kwon *et al.*, 2023; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025).
- **vLLM plus YiRage integration.** Scheduler and paging stay framework-side; fused decode core shrinks in-step bytes and launches (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026).
- **Map execution mode per subgraph, not per binary.** Eager, graph-captured, and megakernel/Persistent-Kernel modes should be chosen per

decode cell, because a single binary-wide mode is wrong for some subgraphs (NVIDIA, 2024; Chen and YiRage Contributors, 2026). A dynamic-shape preprocessing step may stay eager while the fused decoder layer runs as a captured PK, and forcing one mode everywhere leaves goodput on the table (Vellaisamy *et al.*, 2026; Chen, 2026a).

- **Triplet regression gates.** Bytes per token, launches per token, tokens per second on pinned submodule and artifact digest (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).
- **Pin the submodule and smoke-test in CI.** Pin `deps/YiRage` to a digest and run a Persistent-Kernel decode smoke test, since an import-only check passes while the native co-designed stack is silently broken (Chen and YiRage Contributors, 2026; Chen, 2020). The three-way co-design only holds if the framework, compiler, and runtime versions are tested together against the same pinned artifact (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

30.8 Conclusion

This chapter closes Part VIII and the book’s industrial arc: hardware constraints (Part II–III), MegaKernel craft (Part IV–V), compiler theory and stacks (Part VI), production tuning (Part VII), and now *deployable* framework–compiler–runtime cooperation anchored on YiRage and decode goodput counters (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Chapter 26 production runbooks and Chapter 27 trend baselines supply fleet pins and paging ABI expectations; this chapter adds triplet regression gates and mode maps on top (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Draw the decode boundary, assign framework/compiler/runtime owners, integrate vLLM paging with YiRage artifacts, map execution modes per subgraph on your fleet, and gate every promotion on triplet regression (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Goodfellow *et al.*, 2016). Future work tracks autonomous search from Chapter 25 and new hardware entries in `HardwareRegistry`—always profiled on bytes per token and launches per token at batch one, never on FLOPs alone (Chen and YiRage Contributors, 2026; Davies

et al., 2025; Vellaisamy *et al.*, 2026; Chen, 2026a, 2020; Goodfellow *et al.*, 2016). The loop closes where it opened: mechanical sympathy measured in counters your fleet can audit (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Before you close the repository, run one end-to-end worksheet on your target SKU: warmed Persistent Kernel, three context buckets, vLLM or equivalent scheduler at representative load, triplet rows logged, artifact digest pinned in config (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). If that worksheet passes review, Part VIII is not theory—it is your production baseline (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). If it fails, return to the section whose counter moved and fix the contract before shipping features (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Co-design is maintenance, not a one-time integration project (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Share the worksheet JSON with adjacent teams so framework, compiler, and runtime on-call engineers interpret the same numbers during incidents (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). That shared artifact closes the book loop in operational practice, not only in prose (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Thirty chapters of mechanical sympathy converge on this worksheet—run it once on hardware you ship (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Archive the passing run in your fleet config repo as proof Part VIII is live (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Appendix A

Reproducible Benchmark Harness

Chapter 1 and Chapter 2 reference this appendix for the triplet decode-cell harness, TaxBreak/Memory-Floor worksheets, and regression matrices. Full harness scripts and spreadsheet templates are maintained with the book build artifacts; pin model, ctx bucket, and driver versions per cell before comparing chapters.

Appendix B

Hopper Decode Verification Checklist

This appendix holds execution checklists referenced from Chapter 4. The chapter body states principles; this section is the checkbox layer for CI and release gates.

B.1 Environment pins

- Model weights, tokenizer, and hidden size pinned per benchmark cell.
- Driver, CUDA toolkit, and YiRage commit recorded in the artifact manifest.
- Context buckets {512, 2048, 8192}, batch size = 1, decode step only.
- Warmup includes graph capture and ≥ 1000 steady decode steps before medians.

B.2 Core goodput gates

- Launches/token, bytes/token, and ms/token reported together—not TC utilization alone.
- Ablation matrix: graph-only, fusion-only, graph+fusion (Table 4.8 template).
- No regression on any ctx bucket beyond published tolerance.

- Triplet legality summary: CUDA, CPU, and XDNA exports compile or document deferral.

B.3 Hopper-specific counters

- Shared-memory high-water versus 227 KB usable block cap.
- TMA transaction count and TMA/MMA busy ratio on KV loop bodies.
- Graph bucket hit rate across ctx buckets.
- Spill check: `local memory loads` must not rise after fusion merges.

B.4 Paged KV and long-context stress

- vLLM block-table path exercised when product ships paging.
- 32K ctx smoke when fleet SLO requires it (may differ from 2K TMA legality).
- Numerical diffs at long ctx after any softmax PTX change.

B.5 Release train extras

- Driver upgrade re-run of launches/token and bytes/token cells.
- Speculative-decode draft graph capture (when enabled in product).
- Tensor-parallel TP= 4 sweep for fleet configs (not single-GPU chapter default).
- Artifact store: residency diagram, byte sheet, Nsight screenshot set, ablation rows.

Bibliography

AMD (2024a). Ai engine technology. <https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html>. 6, 10, 11, 13, 14, 21, 27, 30, 31, 42, 45, 47, 48, 50, 56, 57, 58, 61, 64, 65, 72, 78, 83, 93, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 129, 132, 133, 134, 135, 138, 139, 140, 142, 143, 144, 145, 147, 148, 150, 152, 155, 159, 161, 165, 168, 169, 173, 174, 177, 183, 184, 186, 194, 200, 201, 202, 205, 208, 211, 213, 215, 216, 217, 220, 226, 227, 230, 231, 233, 239, 241, 243, 244, 245, 247, 254, 256, 257, 261, 262, 263, 265, 266, 267, 268, 269, 271, 273, 274, 275, 276, 278, 279, 280, 281, 282, 283, 285, 286, 288, 289, 290, 291, 292, 293, 294, 295, 296, 298, 299, 300, 301, 302, 303, 304, 305, 306, 308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 318, 320, 321, 322, 323, 324, 325, 326, 327, 328, 329, 330, 331, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 347, 349, 350, 351, 352, 353, 355, 356, 357, 358, 359, 360, 362, 363, 364, 365, 366, 367, 368, 371, 389, 391, 392, 393, 394, 400, 404, 406, 418, 423, 425, 430, 432, 434, 436, 438, 441, 442, 443, 445, 446, 447, 449, 451, 453, 454, 456, 459, 460, 466, 470, 483, 484, 485, 491, 493, 494, 495, 496, 497, 498, 499, 502, 503, 506, 507, 511, 512, 514, 517, 518, 519, 520, 521, 522, 526, 527, 530, 531, 532, 536, 537, 538, 539, 541, 542, 544, 545, 546, 547, 550, 552, 554, 555, 556, 561, 562, 563, 564, 565, 566, 567, 568, 569, 570, 571, 572, 573, 574, 575, 576, 577, 578, 579, 580, 581, 582, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 594, 596, 597, 598, 599, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 627, 628, 630, 631, 632, 633, 634, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 661, 663, 667, 669, 670, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688

AMD (2024b). AMD XDNA architecture. <https://www.AMD.com/en/technologies/XDNA.html>. 3, 9, 10, 11, 14, 16, 17, 19, 21, 25, 27, 29, 30, 31, 32, 34, 35, 37, 40, 41, 42, 45, 46, 47, 48, 49, 50, 51, 53, 56, 57, 58, 60, 63, 64, 65, 66, 72, 74, 75, 78, 82, 83, 84, 87, 88, 89, 92, 93, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 129, 134, 135, 138, 139, 140, 141, 143, 144, 145, 147, 148, 149, 150, 151, 152, 153, 155, 156, 159, 168, 169, 173, 174, 176, 177, 178, 179, 180, 182, 183, 184, 186, 188, 200, 201, 202, 204, 205, 208, 211, 213, 214, 215, 216, 217, 220, 226, 227, 230, 231, 233, 234, 236, 239, 241, 242,

243, 244, 245, 247, 249, 256, 257, 262, 269, 272, 273, 277, 278, 279, 280, 281, 283, 284, 288, 289, 290, 291, 292, 293, 294, 295, 296, 298, 299, 300, 301, 302, 303, 304, 305, 306, 308, 309, 310, 311, 312, 313, 314, 316, 317, 318, 320, 321, 322, 323, 324, 325, 326, 327, 328, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360, 362, 363, 364, 365, 366, 367, 368, 371, 382, 389, 390, 391, 392, 393, 394, 395, 397, 400, 404, 406, 408, 413, 418, 420, 423, 425, 432, 437, 438, 441, 450, 456, 459, 470, 473, 483, 492, 494, 495, 496, 497, 498, 499, 502, 503, 506, 507, 511, 512, 514, 517, 518, 519, 520, 521, 522, 526, 527, 530, 531, 532, 536, 537, 538, 541, 542, 543, 544, 546, 552, 558, 560, 561, 562, 563, 564, 565, 566, 567, 568, 569, 570, 571, 572, 574, 575, 576, 577, 578, 579, 580, 581, 582, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 594, 596, 597, 598, 599, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 627, 628, 630, 631, 632, 633, 634, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 661, 663, 667, 669, 670, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688

Arya, X. (2024). A preliminary performance analysis of llm inference on edge accelerators. 15, 17, 19

Chen, J. (2026a). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 2, 3, 4, 6, 7, 9, 11, 12, 13, 15, 16, 18, 19, 20, 21, 22, 23, 24, 25, 26, 28, 29, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 44, 45, 46, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 106, 107, 108, 109, 110, 111, 112, 113, 114, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 192, 193, 194, 195, 197, 198, 199, 200, 201, 202, 203, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 227, 228, 229, 230, 231, 232, 234, 235, 238, 239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250, 253, 254, 255, 256, 257, 258, 259, 260, 261, 262, 263, 264, 265, 266, 267, 268, 269, 270, 271, 272, 273, 274, 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 288, 289, 290, 291, 292, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 318, 319, 320, 321, 322, 323, 324, 325, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 362, 363, 364, 365, 366, 367, 368, 369, 370, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 386, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 415, 416, 417, 418, 419, 420, 421, 422, 423, 424, 425, 426, 427, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437, 438, 440, 441, 442, 443, 444, 445, 446, 447, 448, 449, 450, 451, 452, 453, 454, 455, 456, 457,

458, 459, 460, 461, 462, 463, 464, 465, 466, 467, 468, 469, 470, 471, 472, 473, 474, 475, 476, 477, 478, 479, 480, 481, 482, 483, 484, 485, 487, 488, 489, 490, 491, 492, 493, 494, 495, 496, 497, 498, 499, 500, 501, 502, 503, 504, 505, 506, 507, 508, 509, 510, 511, 512, 513, 514, 515, 516, 517, 518, 519, 520, 521, 522, 523, 524, 525, 526, 527, 528, 529, 530, 531, 532, 533, 534, 535, 536, 537, 538, 539, 540, 541, 542, 543, 547, 548, 549, 551, 552, 553, 554, 555, 556, 557, 558, 559, 560, 561, 562, 563, 564, 565, 566, 567, 568, 569, 570, 571, 572, 573, 574, 575, 576, 577, 578, 579, 580, 581, 582, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 593, 594, 595, 596, 597, 598, 599, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 627, 628, 629, 630, 631, 632, 633, 634, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 660, 661, 662, 663, 664, 665, 666, 667, 668, 669, 670, 671, 672, 673, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688, 689

Chen, T., Zheng, L., Yan, E., Jiang, Z., Moreau, T., Ceze, L., Guestrin, C., and Krishnamurthy, A. (2018a). Learning to optimize tensor programs. In *Advances in Neural Information Processing Systems (NeurIPS)*. AutoTVM; template-based statistical cost model (GBT/TreeGRU), transfer learning; 1.2–3.8x end-to-end. 396, 397, 398, 399, 401, 404, 460, 480, 485, 486, 487, 488, 492

Chen, T., Moreau, T., Jiang, Z., Zheng, L., Yan, E., Cowan, M., Shen, H., Wang, L., Hu, Y., Ceze, L., Guestrin, C., and Krishnamurthy, A. (2018b). TVM: An automated end-to-end optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 578–594. Graph- and operator-level optimization, operator fusion, memory latency hiding, ML-based cost model; CPU/GPU/FPGA. 47, 49, 58, 59, 379, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 398, 399, 400, 401, 402, 403, 404, 460, 476, 477, 483, 493, 510, 546

Chen, X. (2020). Compiler-based hardware profiling for software-defined architecture. 3, 9, 17, 19, 20, 21, 22, 23, 24, 25, 26, 28, 29, 30, 31, 32, 33, 34, 35, 36, 38, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 53, 54, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 70, 72, 73, 74, 76, 77, 78, 79, 81, 82, 83, 84, 86, 87, 88, 89, 90, 93, 94, 95, 96, 97, 98, 99, 100, 101, 103, 104, 105, 106, 110, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 132, 133, 134, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250, 253, 254, 255, 256, 258, 259, 260, 261, 262, 263, 264, 265, 266, 267, 268, 269, 270, 271, 272, 273, 274, 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 288, 289, 290, 291, 292, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 318, 319, 320, 321, 322, 323, 324, 325, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349,

350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 372, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 388, 389, 390, 391, 392, 393, 395, 396, 398, 399, 400, 401, 402, 407, 409, 410, 411, 413, 414, 416, 417, 418, 419, 420, 424, 425, 427, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437, 438, 441, 442, 443, 445, 446, 448, 449, 450, 451, 452, 453, 454, 455, 460, 461, 463, 464, 467, 468, 469, 471, 472, 473, 478, 479, 480, 481, 482, 484, 485, 486, 487, 489, 490, 491, 493, 494, 495, 496, 497, 498, 499, 500, 501, 502, 503, 504, 505, 506, 507, 508, 509, 510, 511, 512, 513, 514, 515, 516, 517, 518, 519, 520, 521, 522, 523, 524, 525, 526, 527, 528, 529, 530, 531, 532, 533, 534, 535, 536, 537, 538, 539, 540, 541, 542, 560, 561, 562, 563, 564, 565, 566, 567, 568, 569, 570, 571, 572, 573, 574, 575, 576, 577, 578, 579, 580, 581, 582, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 593, 594, 595, 596, 597, 598, 599, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 627, 628, 629, 630, 631, 632, 633, 634, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 660, 661, 662, 663, 664, 665, 666, 667, 668, 669, 670, 671, 672, 673, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688, 689

Chen, X. (2026b). Towards high-goodput llm serving with prefill-decode multiplexing. 5, 9, 12, 15, 19

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. <https://github.com/chenxingqiang/YiRage>. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 2, 3, 4, 8, 9, 10, 11, 13, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250, 253, 254, 255, 256, 257, 258, 259, 260, 261, 262, 263, 264, 265, 266, 267, 268, 269, 270, 271, 272, 273, 274, 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 288, 289, 290, 291, 292, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 318, 319, 320, 321, 322, 324, 325, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 371, 372, 374, 375, 376, 378, 379, 380, 381, 382, 383, 384, 385, 386, 389, 390, 391, 392, 395, 399, 400, 401, 402, 403, 404, 406, 413, 418, 421, 424, 434, 439, 441, 453, 457, 458, 460, 470, 473, 475, 476, 488, 490, 491, 492, 493, 494, 495, 496, 497, 502, 503, 506, 507, 510, 511, 512, 516, 517, 518, 519, 520, 521,

522, 523, 526, 527, 528, 529, 531, 532, 533, 536, 537, 538, 541, 542, 545, 546, 547, 548, 550, 552, 553, 554, 557, 558, 559, 560, 561, 562, 563, 564, 565, 566, 567, 568, 569, 570, 571, 572, 573, 574, 575, 576, 577, 578, 579, 580, 581, 582, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 593, 594, 595, 596, 597, 598, 599, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 627, 628, 629, 630, 631, 632, 633, 634, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 660, 661, 662, 663, 664, 665, 666, 667, 668, 669, 670, 671, 672, 673, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688, 689

Dao, T. *et al.* (2023). Flash-decoding for long-context inference. PyTorch blog and Stanford CRFM technical report. Parallelizes attention along KV sequence; up to $8\times$ decode speedup at long context. 6, 9, 10, 11, 14, 15, 26, 33, 34, 69, 70, 74, 75, 76, 77, 78, 79, 80, 81, 82, 84, 86, 88, 90, 93, 100, 103, 104, 106, 109, 110, 113, 114, 116, 117, 118, 119, 120, 122, 123, 130, 132, 138, 140, 144, 145, 147, 148, 152, 158, 162, 183, 184, 188, 191, 196, 197, 198, 201, 206, 207, 209, 212, 213, 215, 216, 217, 225, 228, 229, 232, 239, 249, 253, 254, 255, 256, 257, 258, 259, 260, 261, 262, 265, 266, 267, 268, 269, 270, 272, 273, 275, 276, 279, 280, 282, 283, 284, 285, 286, 287, 289, 290, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 312, 313, 314, 315, 316, 317, 318, 321, 322, 323, 324, 325, 326, 328, 329, 330, 331, 333, 334, 335, 336, 337, 338, 339, 340, 341, 344, 345, 349, 350, 351, 357, 362, 363, 364, 365, 366, 367, 368, 495, 496, 497, 498, 502, 503, 506, 507, 511, 512, 517, 518, 519, 520, 521, 522, 523, 524, 526, 527, 531, 532, 534, 536, 537, 542, 561, 562, 564, 565, 566, 567, 568, 569, 570, 571, 572, 574, 575, 576, 577, 578, 579, 580, 581, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 594, 595, 596, 597, 598, 599, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 628, 630, 632, 633, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 663, 667, 669, 670, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 2, 3, 9, 10, 11, 19, 25, 27, 31, 33, 34, 40, 41, 42, 54, 57, 62, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 85, 86, 87, 88, 89, 90, 96, 99, 100, 101, 102, 103, 104, 106, 107, 108, 109, 110, 111, 113, 114, 116, 117, 118, 119, 120, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 136, 137, 138, 139, 140, 144, 145, 146, 147, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 160, 161, 162, 163, 164, 165, 166, 167, 168, 172, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 228, 229, 232, 233, 234, 235, 236, 237, 238, 239, 240, 241, 242, 243, 244, 245, 247, 248, 249, 250, 253, 254, 255, 256, 257, 258, 259, 260, 261, 262, 263, 264, 265, 266, 267, 268, 269, 270, 271, 272, 273, 274, 275, 276, 277, 278, 279, 280, 281, 282, 284, 286, 287, 288, 289, 290, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 318, 321, 322, 323, 324,

325, 326, 327, 328, 329, 330, 331, 332, 333, 334, 336, 337, 338, 339, 340, 341, 343, 344, 345, 346, 347, 348, 350, 351, 355, 357, 358, 362, 363, 364, 365, 366, 367, 368, 376, 378, 380, 382, 383, 390, 392, 397, 398, 399, 400, 401, 402, 405, 407, 412, 414, 416, 417, 419, 420, 421, 424, 427, 429, 432, 433, 434, 441, 445, 448, 450, 451, 452, 455, 463, 464, 468, 470, 471, 472, 473, 494, 495, 496, 497, 498, 502, 503, 505, 506, 507, 511, 512, 513, 517, 518, 519, 520, 521, 522, 523, 524, 526, 527, 531, 532, 533, 534, 535, 536, 537, 540, 541, 542, 560, 561, 562, 564, 565, 566, 567, 568, 569, 570, 571, 572, 574, 575, 576, 577, 578, 579, 580, 581, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 593, 594, 595, 596, 597, 598, 599, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 627, 628, 629, 630, 632, 633, 634, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 660, 663, 664, 667, 668, 669, 670, 671, 672, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688, 689

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes. 1, 2, 4, 5, 6, 7, 9, 11, 18, 19, 21, 24, 25, 27, 28, 29, 30, 31, 32, 34, 36, 37, 38, 39, 40, 41, 42, 43, 45, 46, 47, 48, 49, 50, 51, 53, 54, 55, 56, 58, 59, 60, 61, 62, 63, 64, 65, 66, 68, 69, 70, 71, 72, 73, 74, 75, 76, 78, 79, 80, 81, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250, 253, 254, 255, 256, 257, 258, 259, 260, 261, 262, 263, 264, 265, 266, 267, 268, 269, 270, 271, 272, 273, 274, 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 288, 289, 290, 291, 292, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 318, 319, 320, 321, 322, 323, 324, 325, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 352, 353, 354, 355, 356, 357, 358, 359, 362, 363, 364, 365, 366, 367, 368, 369, 370, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 386, 388, 390, 391, 392, 393, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 407, 408, 409, 410, 411, 413, 414, 415, 416, 417, 418, 419, 420, 421, 422, 424, 425, 426, 427, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437, 438, 440, 441, 442, 443, 444, 445, 446, 447, 448, 449, 450, 451, 452, 453, 454, 455, 456, 457, 458, 460, 461, 463, 464, 466, 467, 468, 469, 470, 471, 472, 473, 474, 476, 478, 479, 480, 481, 482, 484, 485, 486, 487, 488, 489, 490, 491, 492, 493, 494, 495, 496, 497, 498, 499, 500, 501, 502, 503, 504, 505, 506, 507, 508, 509, 510, 511, 512, 513, 514, 515, 516, 517, 518, 519, 520, 521, 522, 523, 524, 525, 526, 527, 528, 529, 530, 531, 532, 533, 534, 535, 536, 537, 538, 539, 540, 541, 542, 543, 544, 545, 548, 549, 550, 551, 552, 553, 554, 555, 556, 557, 558, 559, 560, 561, 562, 563, 564, 565, 566, 567, 568, 569, 570, 571, 572, 573, 574, 575, 576, 577, 578, 579, 580, 581,

582, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 593, 594, 595, 596, 597, 598, 599, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 627, 628, 629, 630, 631, 632, 633, 634, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 660, 661, 662, 663, 664, 665, 666, 667, 668, 669, 670, 671, 672, 673, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688, 689

Eto, X. (2025). Llm performance bottlenecks on cgla. 5

Fan, X. (2025). Ellie: Energy-efficient llm inference at the edge via prefill-decode splitting. 2, 4, 6, 10, 11, 12, 19, 20, 21, 39

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 20, 21, 24, 29, 34, 37, 38, 39, 42, 43, 44, 45, 46, 47, 48, 49, 51, 54, 55, 56, 60, 61, 65, 66, 69, 70, 71, 72, 74, 75, 76, 78, 79, 82, 85, 86, 87, 88, 91, 95, 99, 100, 101, 102, 103, 114, 123, 124, 125, 126, 127, 129, 130, 131, 132, 133, 134, 135, 136, 137, 140, 143, 145, 146, 147, 148, 149, 150, 151, 152, 156, 165, 167, 168, 169, 173, 174, 182, 183, 184, 188, 189, 192, 193, 198, 199, 200, 204, 205, 208, 210, 213, 215, 216, 217, 222, 226, 227, 229, 230, 235, 236, 239, 242, 243, 244, 245, 246, 247, 249, 254, 255, 256, 257, 259, 260, 261, 262, 264, 265, 266, 268, 269, 271, 272, 273, 274, 275, 276, 278, 279, 280, 281, 283, 284, 285, 286, 287, 288, 289, 290, 291, 295, 296, 298, 299, 300, 301, 302, 303, 304, 305, 306, 308, 309, 310, 311, 312, 313, 314, 316, 317, 318, 321, 322, 323, 324, 325, 326, 327, 328, 330, 331, 333, 334, 335, 336, 337, 338, 339, 340, 341, 344, 345, 347, 349, 352, 353, 354, 355, 356, 357, 359, 362, 363, 364, 365, 366, 367, 368, 378, 389, 393, 394, 397, 404, 406, 408, 415, 418, 421, 422, 425, 426, 432, 437, 439, 440, 441, 442, 444, 445, 446, 447, 449, 450, 451, 453, 456, 457, 459, 465, 473, 475, 478, 483, 485, 486, 494, 495, 496, 497, 498, 499, 501, 502, 503, 504, 506, 507, 510, 511, 512, 516, 517, 518, 519, 520, 521, 522, 526, 527, 530, 531, 532, 536, 537, 538, 541, 542, 553, 558, 560, 561, 562, 563, 564, 565, 566, 567, 568, 569, 570, 571, 572, 573, 574, 575, 576, 577, 578, 579, 580, 581, 582, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 593, 594, 595, 596, 597, 598, 599, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 627, 628, 629, 630, 631, 632, 633, 634, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 660, 661, 662, 663, 664, 665, 666, 667, 668, 669, 670, 671, 672, 673, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688, 689

IREE / iree-org contributors (2024). IREE: Intermediate representation execution environment — an MLIR-based end-to-end compiler and runtime. <https://iree.dev>. AOT MLIR compiler+runtime; Flow/Stream/HAL/VM dialects; dispatch-region fusion; .vmfb fat binary; targets llvm-cpu, CUDA, ROCm/HIP, Vulkan/SPIR-V, Metal, AMD AIE (experimental). 60, 63, 64, 65, 83, 84, 115, 422, 423, 424, 425, 426, 427, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437, 438, 459, 460, 476, 477, 480, 481, 483, 484, 485, 488, 491, 492, 493, 510, 543, 545, 546, 547, 548, 549, 550, 551, 552, 553, 554, 555, 556, 557, 558, 559

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang,

- H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 2, 3, 9, 10, 11, 12, 14, 15, 18, 19, 20, 21, 27, 28, 30, 34, 37, 38, 39, 40, 41, 42, 47, 48, 49, 50, 54, 55, 56, 61, 62, 65, 71, 72, 73, 74, 75, 76, 84, 86, 87, 88, 90, 92, 93, 94, 95, 100, 101, 104, 105, 106, 108, 111, 113, 114, 117, 118, 119, 120, 121, 122, 126, 129, 130, 131, 132, 133, 135, 137, 139, 140, 144, 145, 147, 148, 152, 155, 158, 159, 160, 162, 164, 165, 166, 167, 168, 169, 170, 174, 176, 180, 182, 183, 184, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 203, 206, 207, 208, 209, 210, 211, 212, 213, 215, 216, 217, 220, 223, 225, 228, 229, 230, 232, 233, 235, 239, 241, 244, 246, 247, 249, 253, 254, 255, 257, 258, 259, 260, 261, 262, 263, 264, 266, 268, 269, 270, 272, 273, 274, 275, 276, 277, 279, 280, 282, 283, 284, 285, 286, 287, 288, 289, 290, 291, 292, 293, 294, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 316, 317, 318, 319, 320, 321, 322, 323, 324, 325, 326, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 343, 344, 345, 347, 349, 350, 351, 354, 355, 356, 358, 359, 362, 363, 364, 365, 366, 367, 368, 370, 371, 374, 377, 379, 380, 381, 382, 383, 384, 385, 386, 388, 390, 394, 395, 396, 397, 398, 399, 401, 402, 403, 410, 411, 413, 415, 416, 417, 419, 421, 424, 428, 430, 431, 435, 436, 438, 448, 450, 452, 454, 456, 460, 465, 467, 468, 469, 474, 478, 480, 481, 486, 487, 492, 493, 495, 496, 497, 498, 500, 501, 502, 503, 504, 505, 506, 507, 508, 509, 511, 512, 515, 517, 518, 519, 520, 521, 522, 523, 524, 525, 526, 527, 528, 529, 531, 532, 533, 534, 535, 536, 537, 540, 541, 542, 543, 550, 554, 556, 558, 560, 561, 562, 563, 564, 565, 566, 567, 568, 569, 570, 571, 572, 573, 574, 575, 576, 577, 578, 579, 580, 581, 582, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 593, 594, 595, 596, 597, 598, 599, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 627, 628, 629, 630, 631, 632, 633, 634, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 660, 661, 662, 663, 664, 665, 666, 667, 668, 669, 670, 671, 672, 673, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688, 689
- Lai, R., Shao, J., Feng, S., Jin, Y., *et al.* (2023). Relax: Composable abstractions for end-to-end dynamic machine learning. *arXiv preprint arXiv:2311.02103*. TVM Unity; cross-level IRModule (Relax graph + TensorIR), first-class symbolic shapes, call_tir DPS; replaces Relay. 50, 57, 58, 59, 60, 369, 370, 371, 372, 373, 386, 389, 390, 391, 392, 393, 394, 395, 398, 401, 402, 403, 463, 545, 546, 547, 552, 555, 558
- Lazuka, X. (2024). Llm-pilot: Characterize and optimize performance of your llm inference services. 7
- Li, X. (2024). Llm inference serving: Survey of recent advances and opportunities. 2, 5, 10, 11, 14, 15
- Li, X. (2025). Llm-powered automated detection and optimization of inference bottlenecks in distributed systems: A static analysis approach. 14
- Liu, A. *et al.* (2024). Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*. 26, 30, 33, 34, 36, 39, 40, 58, 59, 62, 69, 72, 80, 81, 82, 83, 84, 86, 87, 88, 93, 94, 100, 101,

108, 111, 113, 120, 122, 126, 152, 160, 172, 184, 191, 215, 235, 253, 258, 259, 260, 261, 263, 264, 265, 266, 267, 268, 269, 272, 274, 275, 276, 278, 279, 280, 282, 283, 285, 286, 287, 288, 289, 290, 292, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 308, 309, 310, 312, 313, 314, 316, 317, 318, 319, 320, 321, 322, 323, 324, 325, 326, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 344, 345, 348, 350, 351, 352, 353, 354, 356, 357, 358, 359, 362, 363, 364, 365, 366, 367, 368, 379, 380, 382, 398, 399, 428, 432, 495, 496, 497, 498, 502, 503, 506, 507, 511, 512, 517, 518, 519, 520, 521, 522, 525, 526, 527, 528, 529, 530, 531, 532, 533, 534, 536, 537, 538, 539, 540, 541, 542, 560, 561, 562, 563, 564, 565, 566, 567, 568, 569, 570, 571, 572, 573, 574, 575, 576, 577, 578, 579, 580, 581, 582, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 593, 594, 595, 596, 597, 598, 599, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 627, 628, 630, 632, 633, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 660, 661, 663, 667, 669, 670, 671, 672, 673, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688

Mirage Project contributors (2025). Mirage persistent kernel (MPK): A compiler and runtime for mega-kernelizing tensor programs. <https://github.com/mirage-project/mirage>. Transforms LLM inference into a single megakernel; SM-level task graph (tGraph); cross-operator pipelining; $1.2\text{--}6.7\times$ latency reduction. 47, 65, 128, 132, 134, 142, 143, 144, 147, 150, 151, 152, 154, 155, 156, 157, 158, 160, 161, 164, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 187, 188, 189, 191, 192, 194, 195, 199, 202, 203, 204, 205, 206, 208, 209, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 224, 225, 226, 227, 231, 232, 233, 234, 235, 236, 237, 238, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250, 458, 459, 464, 465, 466, 468, 470, 471, 472, 474, 475, 478, 479, 480, 486, 488, 491, 492, 544, 556, 557, 558, 559

Nakai, X. (2025). Efficient batch processing for private cloud llm inference: Modeling and performance comparison. 20

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 26, 27, 29, 31, 32, 36, 40, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 58, 59, 60, 61, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 83, 84, 85, 86, 87, 89, 91, 92, 95, 96, 98, 101, 102, 107, 108, 113, 118, 119, 122, 123, 124, 125, 126, 127, 129, 132, 133, 134, 135, 136, 140, 141, 142, 144, 145, 148, 149, 150, 151, 152, 153, 155, 156, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 173, 174, 175, 176, 177, 179, 180, 182, 183, 184, 186, 187, 188, 194, 195, 196, 197, 198, 202, 204, 205, 206, 208, 210, 211, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 224, 225, 226, 227, 228, 229, 231, 232, 233, 234, 236, 237, 238, 239, 240, 241, 242, 243, 245, 246, 247, 249, 254, 256, 257, 259, 260, 262, 263, 265, 266, 270, 274, 278, 280, 281, 282, 283, 284, 285, 286, 287, 288, 289, 290, 293, 299, 301, 302, 303, 304, 305, 306, 308, 309, 310, 312, 313, 314, 315, 318, 321, 322, 323, 325, 326, 327, 330, 331, 332, 333, 334, 335, 336, 337, 339, 340, 341, 344, 345, 347, 348, 349, 350, 351, 352, 353, 358, 362, 363, 364, 365, 366, 367, 368, 378, 380, 385, 386, 389, 393, 397, 402, 406, 407, 408, 410, 411, 412, 413, 415, 416, 419, 421, 423, 425, 429, 430, 432, 434, 438, 447, 464, 466, 467, 471, 474, 483, 484, 485, 495, 496, 497, 498,

502, 503, 504, 506, 507, 511, 512, 517, 519, 520, 521, 522, 526, 527, 531, 532, 536, 537, 542, 545, 546, 549, 550, 554, 558, 561, 562, 564, 565, 566, 567, 568, 570, 571, 572, 574, 575, 576, 577, 578, 579, 580, 581, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 597, 598, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 628, 630, 632, 633, 634, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 663, 667, 669, 670, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 13, 16, 17, 19, 20, 24, 27, 34, 36, 39, 48, 52, 54, 68, 70, 76, 77, 78, 79, 84, 85, 87, 88, 89, 90, 91, 92, 93, 94, 95, 99, 103, 106, 108, 109, 110, 111, 116, 129, 154, 155, 156, 157, 158, 160, 167, 169, 170, 171, 172, 175, 176, 177, 178, 180, 182, 183, 184, 189, 192, 195, 204, 206, 208, 209, 211, 212, 215, 216, 217, 225, 230, 232, 235, 239, 240, 242, 244, 246, 249, 253, 258, 263, 264, 265, 266, 267, 268, 269, 271, 272, 273, 274, 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 286, 287, 288, 289, 290, 291, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 316, 317, 318, 319, 320, 321, 322, 323, 324, 329, 330, 331, 333, 334, 336, 337, 338, 339, 340, 341, 344, 345, 346, 347, 348, 349, 352, 353, 354, 355, 356, 358, 359, 362, 363, 364, 365, 366, 367, 368, 369, 370, 376, 377, 383, 384, 385, 386, 388, 392, 393, 394, 395, 396, 399, 401, 402, 403, 405, 409, 412, 414, 417, 418, 419, 420, 421, 422, 426, 427, 428, 431, 435, 438, 440, 441, 447, 453, 454, 455, 456, 474, 477, 492, 494, 495, 496, 497, 498, 500, 502, 503, 505, 506, 507, 511, 512, 516, 517, 518, 519, 520, 521, 522, 524, 525, 526, 527, 528, 531, 532, 535, 536, 537, 542, 543, 560, 561, 562, 563, 564, 565, 566, 567, 568, 570, 571, 572, 574, 575, 576, 577, 578, 579, 580, 581, 582, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 593, 594, 595, 596, 597, 598, 599, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 627, 628, 630, 631, 632, 633, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 660, 662, 663, 664, 665, 666, 667, 668, 669, 670, 671, 673, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688, 689

OpenAI / triton-lang contributors (2024). Triton: A language and compiler for custom deep-learning primitives. <https://github.com/triton-lang/triton>. MLIR-based backend: Triton IR (TTIR) → TritonGPU IR (TTGIR) → LLVM → PTX/AMDGCN; @triton.autotune with num_warps/num_stages. 406, 407, 408, 409, 410, 411, 412, 413, 414, 415, 416, 417, 419, 420, 421, 477, 479, 480, 481, 482, 483, 485, 486, 491, 492

OpenXLA Project (2024). XLA: Accelerated linear algebra — OpenXLA architecture. <https://openxla.org/xla/architecture>. StableHLO → HLO → LLVM/PTX/TPU; operator fusion as primary optimization (no HBM intermediates); PriorityFusion / Multi-Output passes; CommandBufferThunk launch capture; Triton emitters. 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 386, 407

Pichlmeier, X. (2024). Performance characterization of expert router for scalable llm inference. 5

- PyTorch / Glow contributors (2024). Glow: Compiler for neural network hardware accelerators. <https://github.com/pytorch/glow>. LLVM-based CPU backend with operator stacking; `model-compiler` AOT bundles; profile-guided quantization; cross-compile to edge/MCU. 440, 441, 442, 447, 448, 449, 450, 451, 452, 453, 454, 455, 456
- Qiao, X. (2026). Tellme: An efficient end-to-end ternary llm prefill and decode accelerator with table-lookup matmul on edge fpgas. 5
- Recasens, X. (2025). Mind the memory gap: Unveiling gpu bottlenecks in large-batch llm inference. 2, 3, 5, 6
- Rotem, N., Fix, J., Abdurasool, S., Catron, G., Deng, S., Dzhabarov, R., Gibson, N., Hegeman, J., Lele, M., Levenstein, R., Montgomery, J., Maher, B., Nadathur, S., Olesen, J., Park, J., Rakhov, A., and Smelyanskiy, M. (2018). Glow: Graph lowering compiler techniques for neural networks. *arXiv preprint arXiv:1805.00907*. Two-phase strongly-typed IR (high-level graph + low-level address-only); lowering to linear-algebra primitives; profile-guided quantization; AOT bundles. 45, 47, 48, 54, 55, 56, 57, 60, 65, 74, 128, 129, 130, 131, 132, 133, 134, 135, 137, 138, 141, 142, 145, 146, 149, 150, 151, 152, 183, 184, 215, 440, 441, 442, 443, 444, 445, 446, 447, 448, 449, 450, 451, 452, 453, 454, 455, 456, 457, 466, 473, 476, 477, 480, 483, 488, 491, 492, 493, 543, 544, 546, 547, 549, 550, 552, 554, 555, 556, 557, 558
- SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 5, 13, 14, 17, 21, 31, 46, 47, 48, 52, 53, 54, 65, 66, 67, 68, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 85, 86, 87, 89, 90, 91, 92, 95, 96, 98, 101, 107, 108, 109, 110, 111, 113, 114, 119, 123, 124, 125, 126, 127, 130, 131, 132, 133, 134, 135, 136, 137, 140, 141, 144, 145, 146, 147, 148, 150, 151, 152, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 175, 176, 178, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 192, 193, 194, 195, 196, 197, 198, 200, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 232, 233, 234, 235, 236, 237, 238, 239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250, 254, 256, 257, 260, 262, 263, 265, 266, 267, 268, 269, 270, 272, 273, 274, 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 288, 289, 290, 291, 292, 293, 295, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 318, 321, 322, 323, 324, 325, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 347, 349, 351, 353, 354, 356, 362, 363, 364, 365, 366, 367, 368, 369, 371, 374, 381, 389, 394, 406, 408, 409, 410, 411, 412, 421, 423, 427, 429, 435, 438, 447, 453, 460, 466, 483, 491, 494, 495, 496, 497, 498, 502, 503, 506, 507, 511, 512, 516, 517, 518, 519, 520, 521, 522, 524, 526, 527, 529, 530, 531, 532, 533, 534, 535, 536, 537, 541, 542, 544, 561, 562, 563, 564, 565, 566, 567, 568, 569, 570, 571, 572, 573, 574, 575, 576, 577, 578, 579, 580, 581, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 594, 596, 597, 598, 599, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 627, 628, 630, 631, 632, 633, 634, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648,

649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 660, 661, 663, 664, 667, 669, 670, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688

Shah, J., Bikshandi, G., Zhang, Y., Thakkar, V., Ramani, P., and Dao, T. (2024). FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. In *Advances in Neural Information Processing Systems (NeurIPS)*. Hopper warp specialization (producer TMA / consumer WGMMA), persistent kernels with tile scheduler, pingpong scheduling. 47, 51, 52, 53, 54, 58, 59, 63, 73, 76, 81, 85, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250

Sun, X. (2019). Power-driven dnn dataflow optimization on fpga. 26

Tang, X. (2026). Tpla: Tensor parallel latent attention for efficient disaggregated prefill & decode inference. 6

Tillet, P., Kung, H. T., and Cox, D. (2019). Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL@PLDI)*, pages 10–19. Tile = statically shaped sub-array; Triton-C + LLVM-based Triton-IR; matmul/conv on par with cuBLAS/cuDNN. 376, 377, 386, 405, 406, 407, 408, 409, 410, 417, 419, 420, 421, 476, 477, 483, 493, 545, 546

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 2, 4, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 18, 19, 20, 21, 22, 23, 24, 25, 26, 28, 29, 30, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 57, 58, 59, 60, 61, 62, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 108, 109, 110, 111, 112, 113, 114, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 228, 229, 230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250, 253, 254, 255, 256, 257, 258, 259, 260, 261, 262, 263, 264, 265, 266, 267, 268, 269, 270, 271, 272, 273, 274, 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 288, 289, 290, 291, 292, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 318, 319, 320, 321, 322, 323, 324, 325, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338,

339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 362, 363, 364, 365, 366, 367, 368, 369, 370, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 386, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 415, 416, 417, 418, 419, 420, 421, 422, 423, 424, 425, 426, 427, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437, 438, 440, 441, 442, 443, 444, 445, 446, 447, 448, 449, 450, 451, 452, 453, 454, 455, 456, 457, 458, 459, 460, 461, 463, 464, 465, 466, 467, 468, 469, 470, 471, 472, 473, 474, 475, 476, 477, 478, 479, 481, 482, 483, 484, 485, 486, 487, 488, 489, 490, 491, 492, 493, 494, 495, 496, 497, 498, 499, 500, 501, 502, 503, 504, 505, 506, 507, 508, 509, 510, 511, 512, 513, 514, 515, 516, 517, 518, 519, 520, 521, 522, 523, 524, 525, 526, 527, 528, 529, 530, 531, 532, 533, 534, 535, 536, 537, 538, 539, 540, 541, 542, 543, 547, 551, 552, 553, 554, 555, 556, 558, 559, 560, 561, 562, 563, 564, 565, 566, 567, 568, 569, 570, 571, 572, 573, 574, 575, 576, 577, 578, 579, 580, 581, 582, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 593, 594, 595, 596, 597, 598, 599, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 627, 628, 629, 630, 631, 632, 633, 634, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 660, 661, 662, 663, 664, 665, 666, 667, 668, 669, 670, 671, 672, 673, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688, 689

Wang, X. (2025a). Decoupled analysis of dvfs effects in prefill and decode stages of large language model inference. 8, 11, 13, 15

Wang, X. (2025b). Pdacc: A 541 tokens/s@llama-1.2b effective throughput prefill-decode disaggregated mcm-based accelerator for multi-task large language model applications. 1, 2, 9, 11

Wu, M., Cheng, X., Liu, S., Shi, C., Ji, J., Ao, M. K., Velliengiri, P., Miao, X., Padon, O., and Jia, Z. (2024). Mirage: A multi-level superoptimizer for tensor programs. *arXiv preprint arXiv:2405.05751*. μ Graph kernel/block/thread levels; joint algebraic+schedule+custom-kernel superoptimization; abstraction-based pruning; probabilistic equivalence via PIT over finite fields (LAX); $1.1\text{--}2.9\times$. 48, 53, 57, 58, 63, 64, 65, 128, 130, 131, 132, 133, 134, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 154, 155, 156, 157, 158, 159, 160, 161, 163, 164, 166, 170, 171, 172, 173, 174, 175, 177, 178, 179, 180, 181, 182, 183, 184, 187, 188, 189, 191, 192, 194, 197, 201, 202, 203, 204, 205, 206, 208, 209, 211, 213, 215, 216, 217, 218, 219, 220, 221, 224, 227, 228, 231, 232, 233, 234, 235, 236, 237, 238, 239, 240, 241, 242, 244, 245, 246, 247, 248, 249, 250, 382, 458, 459, 460, 461, 462, 463, 464, 465, 466, 467, 468, 469, 470, 471, 472, 473, 474, 475, 476, 477, 478, 479, 480, 481, 482, 483, 484, 485, 486, 487, 488, 491, 492, 493, 545, 546, 557, 559

Yu, X. (2026). Precision bottlenecks and sensitivity analysis in llm quantizationllm quantization bottlenecks. 5, 6, 19

Zheng, L., Jia, C., Sun, M., Wu, Z., Yu, C. H., Haj-Ali, A., Wang, Y., Yang, J., Zhuo, D., Sen, K., Gonzalez, J. E., and Stoica, I. (2020). Ansor: Generating high-performance

tensor programs for deep learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 863–879. Template-free auto-scheduler; hierarchical sampling, evolutionary search, task scheduler; up to 3.8x/2.6x/1.7x on Intel CPU/ARM CPU/NVIDIA GPU. 387, 396, 397, 398, 399, 400, 404, 460, 477, 478, 479, 480, 481, 482, 485, 486, 488, 492

Index

- AIE, 90
- Apache TVM, 321
- Arithmetic intensity, iv
- Asynchronous copy, 161

- Barrier, 187

- Compiler comparison, 400
- Compiler constraints, 42
- Cross-hardware benchmark, 400

- Data residency, 39, 114
- dataflow, 90
- Dataflow-driven design, 39
- Deadlock, 187
- Decode phase, 3
- Double buffering, 161

- Export edge, 39

- Glow, 368
- Goodput, 3

- Hardware abstraction layer, 352
- Hardware architecture, 42
- Heterogeneous deployment, 452
- HLO, 305
- Hopper, 64

- Intermediate representation, iv
- IREE, 352
- Iron rules, 39

- Latency, iv
- Lifetime planning, 114

- LLM inference, 3

- mbarrier, 187
- MegaKernel, 39
- Megakernel, 384
- Memory hierarchy, 42
- Mirage, 384
- MLIR, 352
- Multi-hardware, 452

- On-chip memory, 114
- Operator stacking, 368
- Operator-driven design, 39

- Persistent kernel, 136
- Prefill phase, 3
- Profile-guided quantization, 368

- Schedule-free kernel, 136
- Software pipeline, 161
- StableHLO, 305
- Static role assignment, 136
- Superoptimization, 384
- Synchronization, 187

- Tensor core, 64
- Tensor Memory Accelerator, *see* TMA
- TensorIR, 321
- Throughput, iv
- Tile programming model, 336
- TMA, 64, 161
- Triton, 336
- TritonGPU IR, 336
- TVM, 321

Warp specialization, 136

XDNA, 90

XLA, 305